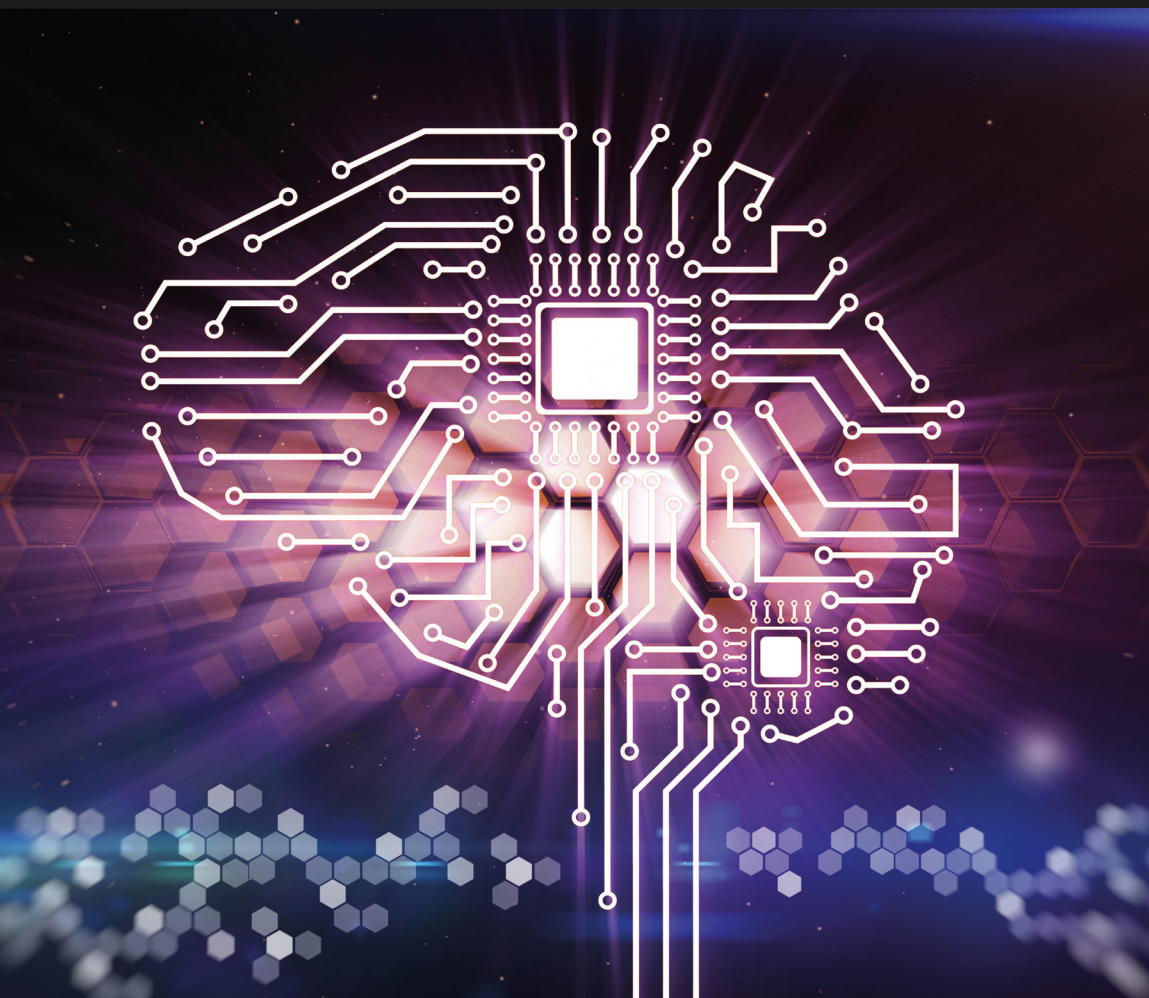


Fourth Edition

Computer

A History of the Information Machine

Martin Campbell-Kelly, William F. Aspray, Jeffrey R. Yost,
Honghong Tinn, and Gerardo Con Díaz



COMPUTER

Computer: A History of the Information Machine traces the history of the computer and its unlimited information-processing potential.

Comprehensive and accessibly written, this fully updated fourth edition adds new chapters on the globalization of information technology, the rise of social media, fake news, and the gig economy, and the regulatory frameworks being put in place to tame the ubiquitous computer. *Computer* is an insightful look at the pace of technological advancement and the seamless way computers are integrated into the modern world. The authors examine the history of the computer, including the first steps taken by Charles Babbage in the nineteenth century, and how wartime needs and the development of electronics led to the giant ENIAC, the first electronic computer. For a generation IBM dominated the computer industry. In the 1980s, the desktop PC liberated people from room-sized mainframe computers. Next, laptops and smartphones made computers available to half of the world's population, leading to the rise of Google and Facebook, and powerful apps that changed the way we work, consume, learn, and socialize.

The volume is an essential resource for scholars and those studying computer history, technology history, and information and society, as well as a range of courses in the fields of computer science, communications, sociology, and management.

Martin Campbell-Kelly is Professor Emeritus in the Department of Computer Science at Warwick University. His most recent book is *Cellular: An Economic and Business History of the International Mobile-Phone Industry* (with Daniel Garcia-Swartz, 2022).

William F. Aspray is Senior Research Fellow at the Charles Babbage Institute, University of Minnesota. His recent books include *Fake News Nation: The Long History of Lies and Misinterpretations in America* (with James Cortada, 2019).

Jeffrey R. Yost is Director of the Charles Babbage Institute and Research Professor in the Program in the History of Science, Technology, and Medicine at the University of Minnesota. His books include *Making IT Work: A History of the Computer Services Industry* (2017).

Honghong Tinn is Assistant Professor in the Program in the History of Science, Technology, and Medicine and the Department of Electrical and Computer Engineering at the University of Minnesota. Her works on the history and globalization of information technology have been published in *Technology and Culture*, *IEEE Annals of the History of Computing*, and *Osiris*.

Gerardo Con Díaz is Associate Professor in the Department of Science and Technology Studies at the University of California, Davis. He is the author of *Software Rights: How Patent Law Transformed Software Development in America* (2019) and has held fellowships at Yale Law School and the Hoover Institution, Stanford University.



Taylor & Francis

Taylor & Francis Group

<http://taylorandfrancis.com>

Computer

A History of the Information Machine

Fourth Edition

**Martin Campbell-Kelly,
William F. Aspray, Jeffrey R. Yost,
Honghong Tinn, and Gerardo Con Díaz**



Routledge
Taylor & Francis Group

NEW YORK AND LONDON

Designed cover image: Computer circuit board in the form of the human brain. Alexey Kotelnikov/Alamy Stock Photo

Fourth edition published 2023

by Routledge

605 Third Avenue, New York, NY 10158

and by Routledge

4 Park Square, Milton Park, Abingdon, Oxon, OX14 4RN

Routledge is an imprint of the Taylor & Francis Group, an informa business

© 2023 Martin Campbell-Kelly, William F. Aspray, Jeffrey R. Yost, Gerardo Con Díaz and Honghong Tinn

The right of Martin Campbell-Kelly, William F. Aspray, Jeffrey R. Yost, Honghong Tinn, and Gerardo Con Díaz to be identified as authors of this work has been asserted in accordance with sections 77 and 78 of the Copyright, Designs and Patents Act 1988.

All rights reserved. No part of this book may be reprinted or reproduced or utilised in any form or by any electronic, mechanical, or other means, now known or hereafter invented, including photocopying and recording, or in any information storage or retrieval system, without permission in writing from the publishers.

Trademark notice: Product or corporate names may be trademarks or registered trademarks and are used only for identification and explanation without intent to infringe.

First edition published by Westview Press 2014

Third edition published by Routledge 2018

Library of Congress Cataloging-in-Publication Data

Names: Campbell-Kelly, Martin, author. | Aspray, William, author. | Yost, Jeffrey R, author.

Title: Computer : a history of the information machine / Martin Campbell-Kelly, William Aspray, Jeffrey R. Yost, Gerardo Con Díaz and Honghong Tinn.

Description: Fourth edition. | New York, NY : Routledge, 2023. |

Includes bibliographical references and index.

Identifiers: LCCN 2022056504 (print) | LCCN 2022056505 (ebook) | ISBN 9781032203430 (paperback) | ISBN 9781032203478 (hardback) | ISBN 9781003263272 (ebook)

Subjects: LCSH: Computers—History. | Electronic data processing—History.

Classification: LCC QA76.17 .C36 2023 (print) | LCC QA76.17 (ebook) | DDC 004.09—dc23/eng/20221202

LC record available at <https://lcn.loc.gov/2022056504>

LC ebook record available at <https://lcn.loc.gov/2022056505>

ISBN: 978-1-032-20347-8 (hbk)

ISBN: 978-1-032-20343-0 (pbk)

ISBN: 978-1-003-26327-2 (ebk)

DOI: 10.4324/9781003263272

Typeset in Times New Roman

by Apex CoVantage, LLP

Contents

<i>Acknowledgments</i>	vii
<i>Preface to the Fourth Edition</i>	viii
Introduction	1
PART ONE	
Before the Computer	7
1 When Computers Were People	9
2 The Mechanical Office	27
3 Babbage's Dream Comes True	48
PART TWO	
Creating the Computer	71
4 Inventing the Computer	73
PHOTOS: <i>From Babbage's Difference Engine to System/360</i>	95
5 The Computer Becomes a Business Machine	108
6 The Maturing of the Mainframe: The Rise of IBM	130
PART THREE	
Innovation and Expansion	145
7 Real Time: Reaping the Whirlwind	147

8 Software	168
PHOTOS: <i>From SAGE to the Internet</i>	191
9 New Ways of Computing	209
 PART FOUR	
Getting Personal	233
10 The Shaping of the Personal Computer	235
11 Broadening the Appeal	256
12 The Internet	279
 PART FIVE	
The Ubiquitous Computer	299
13 Globalization	301
14 The Interactive Web: Clouds, Devices, and Culture	319
15 Computing and Governance	336
 <i>Bibliography</i>	352
<i>Index</i>	369

Acknowledgments

THIS BOOK has its origins in the vision of the Alfred P. Sloan Foundation that it is important for the public to understand the technology that has so profoundly reshaped Western society during the past century. In the fall of 1991 Arthur Singer of the Sloan Foundation invited us (Aspray and Campbell-Kelly) to write a popular history of the computer. It was a daunting, yet irresistible, opportunity. Without the invitation, encouragement, generous financial support, and respectful treatment we received from the Sloan Foundation, this book would never have been written.

It is a pleasure to thank many colleagues who, over four editions of *Computer*, have given us help and advice or checked sections of our manuscript; among them are Jon Agar, Kenneth Beauchamp, Jonathan Bowen, I. Bernard Cohen, John Fauvel, Ernest Fortnum, Jack Howlett, Thomas Misa, Arthur Norberg, Judy O'Neill, Emerson Pugh, and Steve Russ. We particularly thank Nathan Ensmenger, who made important contributions to the third edition. Our thanks go to numerous archivists who helped us to locate suitable illustrations and other historical materials. For the first edition of this book Susan Rabiner of Basic Books took us by the hand and coached us on how to write for a general readership. Our successive editors at Westview Press and Routledge have been a source of wisdom and encouragement. Notwithstanding these many contributions, we take full responsibility for the contents.

Preface to the Fourth Edition

SINCE THE appearance of the third edition in 2014, computing has continued to evolve rapidly. Most obviously, the internet has grown to maturity such that it is now an integral part of most people's lives in the developed world. Although computing was widely diffused by the end of the twentieth century, it has become truly ubiquitous only in the present century – a transition brought about by internet commerce, consumer computing in the form of smartphones and tablet computers, and social networking.

The study of the history of computing has also matured as an academic enterprise. When the first edition of this book appeared in 1996, the history of computing had only recently begun to attract the attention of the academy, and research on this topic tended to be quite technically oriented. Since that time many new scholars with different perspectives have joined the field, and it is rare to find a science, technology, or business history conference that does not discuss developments in, and impacts of, computing technology. In short, the user experience and business applications of computing have become central to much of the historical discourse. To harness these new perspectives in our narrative we have been joined by two additional authors, Gerardo Con Díaz and Honghong Tinn – both scholars from the rising generation.

As always in a new edition, we have revised the text to reflect changing perspectives and updated the bibliography to incorporate the growing literature of the history of computing. Most importantly, we have added an entirely new fifth part to the book “The Ubiquitous Computer,” consisting of three chapters. These chapters reflect the business, technical, social, and regulatory concerns of the all-pervasive computer. Chapter 13 discusses the globalization of the production and consumption of computing. Chapter 14 discusses the many economic and societal consequences of the internet and the World Wide Web. Lastly, Chapter 15 discusses the regulatory responses to the environmental, legal, and social issues raised by our current computer age.

With these changes we hope that, for the next several years, the fourth edition of *Computer* will continue to serve as an authoritative, semi-popular history of computing.

Martin Campbell-Kelly
William F. Aspray
Jeffrey R. Yost
Honghong Tinn
Gerardo Con Díaz



Taylor & Francis

Taylor & Francis Group

<http://taylorandfrancis.com>

Introduction

IN JANUARY 1983, *Time* magazine selected the personal computer as its “Man of the Year,” and public fascination with the computer has continued to grow ever since. That year was not, however, the beginning of the computer age. Nor was it even the first time that *Time* had featured a computer on its cover. Thirty-three years earlier, in January 1950, the cover had sported an anthropomorphized image of a computer wearing a navy captain’s hat to draw readers’ attention to the feature story, about a calculator built at Harvard University for the US Navy. Sixty years before that, in August 1890, another popular American magazine, *Scientific American*, devoted its cover to a montage of the equipment constituting the new punched-card tabulating system for processing the US Census. As these magazine covers indicate, the computer has a long and rich history, and we aim to tell it in this book.

In the 1970s, when scholars began to investigate the history of computing, they were attracted to the large one-of-a-kind computers built a quarter century earlier, sometimes now referred to as the “dinosaurs.” These were the first machines to resemble in any way what we now recognize as computers: they were the first calculating systems to be readily programmed and the first to work with the lightning speed of electronics. Most of them were devoted to scientific and military applications, which meant that they were bred for their sheer number-crunching power. Searching for the prehistory of these machines, historians mapped out a line of desktop calculating machines originating in models built by the philosophers Blaise Pascal and Gottfried Leibniz in the seventeenth century and culminating in the formation of a desk calculator industry in the late nineteenth century. According to these histories, the desk calculators were followed in the period between the world wars by analog computers and electromechanical calculators for special scientific and engineering applications; the drive to improve the speed of calculating machines during World War II led directly to the modern computer.

Although correct in the main, this account is not complete. Today, research scientists and atomic weapons designers still use computers extensively, but the vast majority of computers in organizations are employed for other purposes,

2 *Introduction*

such as word processing and keeping business records. How did this come to pass? To answer that question, we must take a broader view of the history of the computer as the history of the information machine.

This history begins in the early nineteenth century. Because of the increasing population and urbanization in the West resulting from the Industrial Revolution, the scale of business and government expanded, and with it grew the scale of information collection, processing, and communication needs. Governments began to have trouble enumerating their populations, telegraph companies could not keep pace with their message traffic, and insurance agencies had trouble processing policies for the masses of workers.

Novel and effective systems were developed for handling this increase in information. For example, the Prudential Assurance Company of England developed a highly effective system for processing insurance policies on an industrial scale using special-purpose buildings, rationalization of process, and division of labor. But by the last quarter of the century, large organizations had turned increasingly to technology as the solution to their information-processing needs. On the heels of the first large American corporations came a business-machine industry to supply them with typewriters, filing systems, and duplication and accounting equipment.

The desk calculator industry was part of this business-machine movement. For the previous two hundred years, desk calculators had merely been hand-made curiosities for the wealthy. But by the end of the nineteenth century, these machines were being mass-produced and installed as standard office equipment, first in large corporations and later in progressively smaller offices and retail establishments. Similarly, the punched-card tabulating system developed to enable the US government to cope with its 1890 census data gained wide commercial use in the first half of the twentieth century, and was in fact the origin of IBM.

Also beginning in the nineteenth century and reaching maturity in the 1920s and 1930s was a separate tradition of analog computing. Engineers built simplified physical models of their problems and measured the values they needed to calculate. Analog computers were used extensively and effectively in the design of electric power networks, dams, and aircraft.

Although the calculating technologies available through the 1930s served business and scientific users well, during World War II they were not up to the demands of the military, which wanted to break codes, prepare firing tables for new guns, and design atomic weapons. The old technologies had three shortcomings: they were too slow in doing their calculations, they required human intervention in the course of a computation, and many of the most advanced calculating systems were special-purpose rather than general-purpose devices.

Because of the exigencies of the war, the military was willing to pay whatever it would take to develop the kinds of calculating machines it needed. Millions of dollars were spent, resulting in the production of the first electronic,

stored-program computers – although, ironically, none of them was completed in time for war work. The military and scientific research value of these computers was nevertheless appreciated, and by the time of the Korean War a small number had been built and placed in operation in military facilities, atomic energy laboratories, aerospace manufacturers, and research universities.

Although the computer had been developed for number crunching, several groups recognized its potential as a data-processing and accounting machine. The developers of the most important wartime computer, the ENIAC, left their university posts to start a business building computers for the scientific and business markets. Other electrical manufacturers and business-machine companies, including IBM, also turned to this enterprise. The computer makers found a ready market in government agencies, insurance companies, and large manufacturers.

The basic functional specifications of the computer were set out in a report written by John von Neumann in 1945, and these specifications are still largely followed today. However, decades of continuous innovation have followed the original conception. These innovations are of two types. One is the improvement in components, leading to faster processing speed, greater information-storage capacity, improved price/performance, better reliability, less required maintenance, and the like: today's computers are literally millions of times better than the first computers on almost all measures of this kind. These innovations were made predominantly by the firms that manufactured computers.

The second type of innovation was in the mode of operation, but here the agent for change was most often the academic sector, backed by government financing. In most cases, these innovations became a standard part of computing only through their refinement and incorporation into standard products by the computer manufacturers. There are five notable examples of this kind of innovation: high-level programming languages, real-time computing, time-sharing, networking, and graphically oriented human-computer interfaces.

While the basic structure of the computer remained unchanged, these new components and modes of operation revolutionized our human experiences with computers. Elements that we take for granted today – such as having a computer on our own desk, equipped with a mouse, monitor, and disk drive – were not even conceivable until the 1970s. At that time, most computers cost hundreds of thousands, or even millions, of dollars and filled a large room. Users would seldom touch or even see the computer itself. Instead, they would bring a stack of punched cards representing their program to an authorized computer operator and return hours or days later to pick up a printout of their results. As the mainframe became more refined, the punched cards were replaced by remote terminals, and response time from the computer became almost immediate – but still only the privileged few had access to the computer. All of this changed with the development of the personal computer and the growth of the internet. The mainframe has not died out, as many have predicted, but computing is now available to the masses.

4 *Introduction*

As computer technology became increasingly less expensive and more portable, new and previously unanticipated uses for computers were discovered – or invented. Today, for example, the digital devices that many of us carry in our briefcases, backpacks, purses, or pockets serve simultaneously as portable computers, communications tools, entertainment platforms, digital cameras, monitoring devices, and conduits to increasingly omnipresent social networks. The history of the computer has become inextricably intertwined with the history of communications and mass media, as our discussion of the personal computer and the internet clearly illustrates. But it is important to keep in mind that even in cutting-edge companies like Facebook and Google, multiple forms and meanings of the computer continue to coexist, from the massive mainframes and server farms that store and analyze data to the personal computers used by programmers to develop software to the mobile devices and applications with which users create and consume content. As the computer itself continues to evolve and acquire new meanings, so does our understanding of its relevant history. But it is important to remember that these new understandings do not refute or supersede these earlier histories but rather extend, deepen, and make them even more relevant.

WE HAVE ORGANIZED the book in five parts. The first covers the way information processing was handled before the arrival of electronic computers. The next two parts describe the mainframe computer era, roughly from 1945 to 1980, with one part devoted to the computer's creation and the other to its evolution. The fourth part discusses the origins of personal computing and the internet. The fifth part examines the computer as it has become more ubiquitous and more global, as it becomes increasingly intertwined with issues of telecommunication, and as new legal problems arise as the computer raises new issues about the distribution of content and the control of people. In this fourth edition of the book, we have streamlined but not substantially changed the material that appears in the first four parts. The fifth part presents entirely new material, providing extended coverage of some of the most important developments related to computers of the twenty-first century.

Part One, on the early history of computing, includes three chapters. Chapter 1 discusses manual information processing and early technologies. People often suppose that information processing is a twentieth-century phenomenon; this is not so, and the first chapter shows that sophisticated information processing could be done with or without machines – slower in the latter case, but equally well. Chapter 2 describes the origins of office machinery and the business-machine industry. To understand the post-World War II computer industry, we need to realize that its leading firms – including IBM – were established as business-machine manufacturers in the last decades of the nineteenth century and were major innovators between the two world wars. Chapter 3 describes Charles Babbage's failed attempt to build a calculating engine in the 1830s and

its realization by Harvard University and IBM a century later. We also briefly discuss the theoretical developments associated with Alan Turing.

Part Two of the book describes the development of the electronic computer, from its invention during World War II up to the establishment of IBM as the dominant mainframe computer manufacturer in the mid-1960s. Chapter 4 covers the development of the ENIAC at the University of Pennsylvania during the war and its successor, the EDVAC, which was the blueprint for almost all subsequent computers up to the present day. Chapter 5 describes the early development of the computer industry, which transformed the computer from a scientific instrument for mathematical computation into a machine for business data processing. In Chapter 6 we examine the development of the mainframe computer industry, focusing on the IBM System/360 range of computers, which created the first stable industry standard and established IBM's dominance.

Part Three presents a selective history of some key computer innovations in the quarter century between the invention of the computer at the end of the war and the development of the first personal computers. Chapter 7 is a study of one of the key technologies of computing, real time. We examine this subject in the context of commonly experienced applications, such as airline reservations, banking and ATMs, and supermarket barcodes. Chapter 8 describes the development of software technology, the professionalization of programming, and the emergence of a software industry. Chapter 9 covers the development of some of the key features of the computing environment at the end of the 1960s: time-sharing, minicomputers, and microelectronics. The purpose of the chapter is, in part, to redress the commonly held notion that the computer transformed from the mainframe to the personal computer in one giant leap.

Part Four gives a history of the developments of the last quarter of the twentieth century when computing became "personal." Chapter 10 describes the development of the microcomputer from the first hobby computers in the mid-1970s up to its transformation into the familiar personal computer by the end of the decade. Chapter 11's focus is on the personal-computer environment of the 1980s, when the key innovations were user-friendliness and the delivery of "content," by means of CD-ROM storage and consumer networks. This decade was characterized by the extraordinary rise of Microsoft and the other personal-computer software companies. Chapter 12 begins a discussion of the internet and its consequences. The chapter describes the creation of the internet and the World Wide Web, their precedents in the information sciences, and the ever-evolving commercial and social applications.

Part Five discusses the growing ubiquity of the computer, in terms of both parts of the world more actively participating in computing and also the ongoing spread of computing into various aspects of people's personal and work lives. It is indicative of these recent changes that *Time* magazine selected Apple's iPhone as "Person of the Year" for 2007. Chapter 13 discusses globalization. Topics include the off-shoring of software and services, the outsourcing of manufacture

6 *Introduction*

to other countries, and innovation in both Asia and beyond. Companies that receive their first attention in the book include Tata, Foxconn, Lenovo, Huawei, and the Taiwan Semiconductor Manufacturing Corporation (TSMC). When we wrote the first edition, we had no idea how the internet would unfold. It has exceeded every expectation – both good and bad. In Chapter 14 we discuss this unfolding of the internet through issues about the platformization of commerce and culture. We discuss such topics as the ascendancy of mobile devices and “cloud computing,” surveillance capitalism, fake news and the online threat to democracy, threats to brick-and-mortar businesses from online enterprises, the creation of new forms of labor in the gig economy, the disruptions to media delivery created by streaming, and the rise of cryptocurrencies. Chapter 15 discusses recent computer developments in the context of the environment, law, and politics. Topics include the environmental impacts of manufacturing and online services, intellectual property (IP) concerns, unfair competition, online political organization and activism, and privacy and surveillance issues for individuals and nations in a networked society.

We have included notes at the end of each chapter and a bibliography toward the end of the book.

Note on sources

Between the extremes of giving a popular treatment with no citations at all and full scholarly annotation, we have decided on a middle course in which we have attempted to draw a line between pedantry and adequacy – as Kenneth Galbraith eloquently put it. In the notes to every chapter, we have given a short literature review in which we have identified a dozen or so reliable secondary sources. Where possible, we have used the monographic literature, falling back on the less accessible periodical literature only where the former falls short. The reader can assume, unless explicitly indicated, that every assertion made in the text can be readily found in this secondary literature. This has enabled us to restrict citations to just two types: direct quotations and information that does not appear in the monographs already cited.

Full publication data is given in the bibliography. The reader should note that its purpose is largely administrative – to put all our sources in one place. It is not intended as a reading list. If a reader should want to study any topic more deeply, we recommend beginning with one of the books cited in the literature review in the notes to each chapter.

Part One

Before the Computer



Taylor & Francis

Taylor & Francis Group

<http://taylorandfrancis.com>

1 When Computers Were People

THE WORD *computer* may seem a surprising name for the information machine that is today central to our economy and society. The computer exists in many forms – as behemoths in corporations, desktop PCs and laptops, in our smartphones, and embedded in automobiles and appliances. If we go back to the Victorian period, or even the World War II era, the word *computer* meant an occupation, defined in the *Oxford English Dictionary* as “one who computes; a calculator, reckoner; specifically a person employed to make calculations in an observatory, in surveying, etc.”

The reason the computer is so-called is that these machines have their origins in the large-scale mathematical calculations once performed by human computers.

Logarithms and Mathematical Tables

The first attempt to organize calculations on a large scale using human computers was for the production of mathematical tables, such as logarithmic and trigonometric tables. Logarithmic tables revolutionized mathematical computation in the sixteenth and seventeenth centuries by enabling time-consuming arithmetic operations, such as multiplication and division and the extraction of roots, to be performed using only the simple operations of addition and subtraction. Trigonometric tables enabled a similar speeding up of calculations of angles and areas in connection with surveying and astronomy. However, logarithmic and trigonometric tables were merely the best-known general-purpose tables. By the late eighteenth century, specialized tables were being produced for several different occupations: navigational tables for mariners, star tables for astronomers, life insurance tables for actuaries, civil engineering tables for architects, and so on. All these tables were produced by human computers, without any mechanical aid.

For a maritime nation such as Great Britain, and later the United States, the timely production of reliable navigation tables free of error was of major economic importance. In 1766 the British government sanctioned the astronomer royal, Nevil Maskelyne, to produce each year a set of navigational tables to

be known as the *Nautical Almanac*. This was the first permanent table-making project to be established in the world. Often known as the Seaman's Bible, the *Nautical Almanac* dramatically improved navigational accuracy. It has been published without a break every year since 1766.

The *Nautical Almanac* was not computed directly by the Royal Observatory, but by a number of freelance human computers dotted around Great Britain. The calculations were performed twice, independently, by two computers and checked by a third "comparator." Many of these human computers were retired clerks or clergymen with a facility for figures and a reputation for reliability who worked from home. We know almost nothing of these anonymous drudges. Probably the only one to escape oblivion was the Reverend Malachy Hitchins, an eighteenth-century Cornish clergyman who was a computer and comparator for the *Nautical Almanac* for a period of forty years. A lifetime of computational dedication earned him a place in the *Dictionary of National Biography*. When Maskelyne died in 1811 – Hitchins had died two years previously – the *Nautical Almanac* "fell on evil days for about 20 years, and even became notorious for its errors."¹

Charles Babbage and Table Making

During this period Charles Babbage became interested in the problem of table making and the elimination of errors in tables. Born in 1791, the son of a wealthy London banker, Babbage spent his childhood in Totnes, Devon, a country town in the west of England. He experienced indifferent schooling but succeeded in teaching himself mathematics to a considerable level. He went to Trinity College, Cambridge University, in 1810, where he studied mathematics. Cambridge was the leading English university for mathematics, and Babbage was dismayed to discover that he already knew more than his tutors. Realizing that Cambridge (and England) had become a mathematical backwater compared to continental Europe, Babbage and two fellow students organized the Analytical Society, which succeeded in making major reforms of mathematics in Cambridge and eventually the whole of England. Even as a young man, Babbage was a talented propagandist.

Babbage left Cambridge in 1814, married, and settled in Regency London to lead the life of a gentleman philosopher. His researches were mainly mathematical, and in 1816 his achievements were recognized by his election to the Royal Society, the leading scientific organization in Britain. He was then twenty-five – an enfant terrible with a growing scientific reputation.

In 1819 Babbage made the first of several visits to Paris, where he met a number of the leading members of the French Scientific Academy, such as the mathematicians Pierre-Simon Laplace and Joseph Fourier, with whom he formed lasting friendships. It was probably during this visit that Babbage learned of the great French table-making project organized by Baron Gaspard de Prony. This project would show Babbage a vision that would determine the future course of his life.

De Prony began the project in 1790, shortly after the French Revolution. The new government planned to reform many of France's ancient institutions and, in particular, to establish a fair system of property taxation. To achieve this, up-to-date maps of France were needed. De Prony was charged with this task and was appointed head of the Bureau du Cadastre, the French ordnance survey office. His task was made more complex by the fact that the government had simultaneously decided to reform the old imperial system of weights and measures by introducing the new rational metric system. This created within the bureau the job of making a complete new set of decimal tables, to be known as the *tables du cadastre*. It was by far the largest table-making project the world had ever known, and de Prony decided to organize it much as one would organize a factory.

De Prony took as his starting point the most famous economics text of his day, Adam Smith's *Wealth of Nations*, published in 1776. It was Smith who first advocated the principle of division of labor, which he illustrated by means of a pin-making factory. In this famous example, Smith explained how the making of a pin could be divided into several distinct operations: cutting the short lengths of wire to make the pins, forming the pin head, sharpening the points, polishing the pins, packing them, and so on. If a worker specialized in a single operation, the output would be vastly greater than if a single worker performed all the operations that went into making a pin. De Prony "conceived all of a sudden the idea of applying the same method to the immense work with which I had been burdened, and to manufacture logarithms as one manufactures pins."²

De Prony organized his table-making "factory" into three sections. The first section consisted of half a dozen eminent mathematicians, including Adrien-Marie Legendre and Lazare Carnot, who decided on the mathematical formulas to be used in the calculations. Beneath them was another small section – a kind of middle management – that, given the mathematical formulas to be used, organized the computations and compiled the results ready for printing. Finally, the third and largest section, which consisted of sixty to eighty human computers, did the actual computation. The computers used the "method of differences," which required only the two basic operations of addition and subtraction, and not the more demanding operations of multiplication and division. Hence the computers were not, and did not need to be, educated beyond basic numeracy and literacy. In fact, most of them were hairdressers who had lost their jobs because "one of the most hated symbols of the *ancien régime* was the hairstyles of the aristocracy."³

Although the Bureau was producing mathematical tables, the operation was not itself mathematical. It was fundamentally the application of an organizational technology, probably for the first time outside a manufacturing or military context, to the production of information. Its like would not be seen again for another forty years.

The whole project lasted about a decade, and by 1801 the tables existed in manuscript form all ready for printing. Unfortunately, for the next several decades, France was wracked by one financial and political crisis after another, so that the large sum of money needed to print the tables was never found. Hence, when Babbage learned of the project in 1819, all there was to show of it was the manuscript tables in the library of the French Scientific Academy.

In 1820, back in England, Babbage gained some firsthand experience of table making while preparing a set of star tables for the Astronomical Society, a scientific society that he and a group of like-minded amateur scientists had established the same year. Babbage and his friend John Herschel were supervising the construction of the star tables, which were being computed in the manner of the *Nautical Almanac* by freelance computers. Babbage's and Herschel's roles were to check the accuracy of the calculations and to supervise the compilation and printing of the results. Babbage complained about the difficulty of table making, finding it error-prone and tedious; and if he found it tedious just supervising the table making, so much the worse for those who did the actual computing.

Babbage's unique role in nineteenth-century information processing was due to the fact that he was in equal measure a mathematician and an economist. The mathematician in him recognized the need for reliable tables and knew how to make them, but it was the economist in him that saw the significance of de Prony's organizational technology and had the ability to carry the idea further.

De Prony had devised his table-making operation using the principles of mass production at a time when factory organization involved manual labor using very simple tools. But in the thirty years since de Prony's project, best practice in factories had itself moved on, and a new age of mass-production machinery was beginning to dawn. The laborers in Adam Smith's pin-making factory would soon be replaced by a pin-making machine. Babbage decided that rather than emulate de Prony's labor-intensive and expensive manual table-making organization, he would ride the wave of the emerging mass-production technology and invent a machine for making tables.

Babbage called his machine a Difference Engine because it would use the same method of differences that de Prony and others used in table making. Babbage knew, however, that most errors in tables came not from calculating them but from printing them, so he designed his engine to set the type ready for printing as well. Conceptually, the Difference Engine was very simple: it consisted of a set of adding mechanisms to do the calculations and a printing part.

Babbage applied his considerable skills as a publicist to promote the idea of the Difference Engine. He began his campaign by writing an open letter to the president of the Royal Society, Sir Humphrey Davy, in 1822, proposing that the government finance him to build the engine.⁴ Babbage argued that high-quality tables were essential for a maritime and industrial nation, and that his Difference Engine would be far cheaper than the nearly one hundred overseers and human computers in de Prony's table-making project. He had the letter printed

at his own expense and ensured that it got into the hands of people of influence. As a result, in 1823 he obtained government funding of £1,500 to build the Difference Engine, with the understanding that more money would be provided if necessary.

Babbage managed to rally much of the scientific community to support his project. His boosters invariably argued that the merit of his Difference Engine was that it would eliminate the possibility of errors in tables through the “unerring certainty of mechanism.”⁵ It was also darkly hinted that the errors in the *Nautical Almanac* and other tables might “render the navigator liable to be led into difficulties, if not danger.”⁶ Babbage’s friend Herschel went a step further, writing: “An undetected error in a logarithmic table is like a sunken rock at sea yet undiscovered, upon which it is impossible to say what wrecks may have taken place.”⁷ Gradually the danger of errors in tables grew into lurid tales that “navigational tables were full of errors which continually led to ships being wrecked.”⁸ Historians have found no evidence for this claim, although reliable tables certainly helped Britain’s maritime activity run smoothly.

Unfortunately, the engineering was more complicated than the conceptualization. Babbage completely underestimated the financial and technical resources he would need to build his engine. He was at the cutting edge of production technology, for although relatively crude machines such as steam engines and power looms were in widespread use, sophisticated devices such as pin-making machines were still a novelty. By the 1850s such machinery would be commonplace, and there would exist a mechanical-engineering infrastructure that made building them relatively easy. While building the Difference Engine in the 1820s was not in any sense impossible, Babbage was paying the price of being a first mover; it was rather like building the first computers in the mid-1940s: difficult and extremely expensive.

Babbage was now battling on two fronts: first, designing the Difference Engine and, second, developing the technology to build it. Although the Difference Engine was conceptually simple, its design was mechanically complex. In the London Science Museum today, one can see evidence of this complexity in hundreds of Babbage’s machine drawings for the engines and in thousands of pages of his notebooks. During the 1820s, Babbage scoured the factories of Europe seeking gadgets and technology that he could use in the Difference Engine. Not many of his discoveries found their way into the Difference Engine, but he succeeded in turning himself into the most knowledgeable economist of manufacturing of his day. In 1832 he published his most important book, an economics classic titled *Economy of Machinery and Manufactures*, which ran to four editions and was translated into five languages. In the history of economics, Babbage is a seminal figure who connects Adam Smith’s *Wealth of Nations* to the Scientific Management movement, founded in America by Frederick Winslow Taylor in the 1880s.

The government continued to advance Babbage money during the 1820s and early 1830s, eventually totaling £17,000; and Babbage claimed to have spent much the same again from his own pocket. These would be very large sums in today's money. By 1833, Babbage had produced a beautifully engineered prototype Difference Engine that was too small for real table making and lacked a printing unit, but showed beyond any question the feasibility of his concept. (It is still on permanent exhibit in the London Science Museum, and it works as perfectly today as it did then.)

To develop a full-scale machine Babbage needed even more money, which he requested in a letter in 1834 to the prime minister, the Duke of Wellington.⁹ Unfortunately, at that time, Babbage had an idea of such stunning originality that he just could not keep quiet about it: a new kind of engine that would do all the Difference Engine could do but much more – it would be capable of performing *any* calculation that a human could specify for it. This machine he called the Analytical Engine. In almost all important respects, it had the same logical organization as the modern electronic computer. In his letter to the Duke of Wellington, Babbage hinted that instead of completing the Difference Engine he should be allowed to build the Analytical Engine. Raising the specter of the Analytical Engine was the most spectacular political misjudgment of Babbage's career; it fatally undermined the government's confidence in his project, and he never obtained another penny. In fact, by this time, Babbage was so thoroughly immersed in his calculating-engine project that he had completely lost sight of the original objective: to make tables. The engines had become an end in themselves, as we shall see in Chapter 3.

Clearing Houses and Telegraphs

While Babbage was struggling with his Difference Engine, the idea of large-scale information processing was highly unusual – whether it was organized manually or used machinery. The volume of activity in ordinary offices of the 1820s simply did not call for large clerical staffs. Nor was there any office machinery to be had; even adding machines were little more than a scientific novelty at this date, and the typewriter had yet to be invented. For example, the Equitable Society of London – then the largest life insurance office in the world – was entirely managed by an office staff of eight clerks, equipped with nothing more than quill pens and writing paper.

In the whole of England there was just one large-scale data-processing organization that had an organizational technology comparable with de Prony's table-making project. This was the Bankers' Clearing House in the City of London, and Babbage wrote the only contemporary published account of it.¹⁰

The Bankers' Clearing House was an organization that processed the rapidly increasing number of checks being used in commerce. When the use of checks became popular in the eighteenth century, a bank clerk physically had

to take a check deposited by a customer to the bank that issued it to have it exchanged for cash. As the use of checks gained in popularity in the middle of the eighteenth century, each of the London banks employed a “walk clerk,” whose function was to make a tour of all the other banks in the City, the financial district of London, exchanging checks for cash. In the 1770s, this arrangement was simplified by having all the clerks meet at the same time in the Five Bells Public House on Lombard Street. There they performed all the exchanging of checks and cash in one “clearing room.” This obviously saved a lot of walking time and avoided the danger of robbery. It also brought to light that if two banks had checks drawn on each other, the amount of cash needed for settlement was simply the difference between the two amounts owed, which was usually far less than the total amount of all the checks. As the volume of business expanded, the clearing room outgrew its premises and moved several times. Eventually, in the early 1830s, the London banks jointly built a Bankers’ Clearing House at 10 Lombard Street, in the heart of London’s financial center.

The Bankers’ Clearing House was a secretive organization that shunned visitors and publicity. This was because the established banks wanted to exclude the many banks newly formed in the 1820s (which the Clearing House succeeded in doing until the 1850s). Babbage, however, was fascinated by the clearing house concept and pulled strings to gain entry. The secretary of the Bankers’ Clearing House was a remarkable man by the name of John Lubbock, who, besides being a leading figure in the City, was also an influential amateur scientist and vice president of the Royal Society. Babbage wrote to Lubbock in October 1832 asking if it were “possible that a stranger be permitted as a spectator.”¹¹ Lubbock replied, “*You* can be taken to the clearing house . . . but we wish it not to be mentioned, so that the public may fancy they can have access to the sanctum sanctorum of banking, and we wish of course not to be named.” Babbage was captivated by Lubbock’s scientifically organized system, which, despite Lubbock’s proscription, he described in glowing terms in the *Economy of Manufactures*:

In a large room in Lombard Street, about thirty clerks from the several London bankers take their stations, in alphabetical order, at desks placed round the room; each having a small open box by his side, and the name of the firm to which he belongs in large characters on the wall above his head. From time to time other clerks from every house enter the room, and, passing along, drop into the box the checks due by that firm to the house from which this distributor is sent.¹²

Most of the day was spent by clerks exchanging checks with one another and entering the details in ledgers. At four o’clock in the afternoon, the settlements between the banks would begin. The clerk for each bank would total all the

checks received from other banks for payment and all the checks presented for payment to other banks. The difference between these two numbers would then be either paid out or collected.

At five o'clock, the inspector of the clearing house took his seat on a rostrum in the center of the room. Then, the clerks from all the banks who owed money on that day paid the amount due in cash to the inspector. Next, the clerks of all the banks that were owed money collected their cash from the inspector. Assuming that no mistakes had been made (and an elaborate accounting system using preprinted forms ensured this did not happen very often), the inspector was left with a cash balance of exactly zero.

The amount of money flowing through the Bankers' Clearing House was staggering. In the year 1839, £954 million was cleared – the equivalent of several hundred billion dollars in today's money. On the busiest single day more than £6 million was cleared, and about half a million pounds in bank notes were used for the settlement. Eventually, the need for cash was eliminated altogether through an arrangement by which each bank and the Clearing House had an account with the Bank of England. Settlements were then made simply by transferring an amount from a bank's account to that of the Clearing House, or vice versa.

Babbage clearly recognized the significance of the Bankers' Clearing House as an example of the "division of mental labor," comparable with de Prony's table-making project and his own Difference Engine. He remained deeply interested in large-scale information processing all his life. For example, in 1837 he applied unsuccessfully to become registrar general and director of the English population census. But by the time really big information-processing organizations came along in the 1850s and 1860s – such as the great savings banks and industrial insurance companies – Babbage was an aging man who had ceased to have any influence.

The Bankers' Clearing House was an early example of what would today be called "financial infrastructure." The Victorian period was the great age of physical and financial infrastructure investment. Between 1840 and 1870, Britain's investment in rail track grew from 1,500 to over 13,000 miles. Alongside this physical and highly visible transport infrastructure grew a parallel, unseen information infrastructure known as the Railway Clearing House, which was modeled very closely on the Bankers' Clearing House. Established in 1842, the Railway Clearing House rapidly became one of the largest data-processing bureaucracies in the world. By 1870 there were over 1,300 clerks processing nearly 5 million transactions a year.

Another vital part of the Victorian information infrastructure was the telegraph, which began to compete with the ordinary postal system in the 1860s. Telegrams were expensive – one shilling for a twenty-word message compared with as much as you could want to write in a letter for one penny – but very fast.

A telegram would speed across the country in an instant and arrive at its final destination in as little as an hour, hand-delivered by a telegraph boy.

Rather like the Internet in our own time, the telegraph did not originate as a planned communications system. Instead it began as a solution to a communications problem in the early rail system. There was a widely held fear among the public of a passenger train entering a section of track while another was heading in the opposite direction (in practice there were very few such accidents, but this did not diminish public concern). To solve the problem, inventive engineers strung up an electrical communication system alongside the track so that the signalmen at each end could communicate. Now a train could not enter a single-track section until two human operators agreed that it was safe to do so. Of course, it was not long before a commercial use was found for the new electrical signaling method. Newspapers and commercial organizations were willing to pay for news and market information ahead of their competitors. Suddenly, telegraph poles sprang up alongside railway tracks; some systems were owned by the railway companies, some by newly formed telegraph companies. Although messages were sent by electricity, the telegraph still needed a large clerical labor force to operate the machines that transmitted the messages. Much of the labor was female – the first time that female clerical labor had been used on any scale in Britain. The reason for this was that telegraph instruments were rather delicate and it was believed that women – especially seamstresses familiar with sewing machines – would have more dexterity than men.

By the mid-1860s there were over 75,000 miles of telegraph lines in Britain, operated by six main companies. However, each system operated independently and it was difficult for a telegram originating in one network to make use of another. In 1870, the British government stepped in to integrate the systems into a national telegraph network. Once this was done, telegraph usage simply exploded. More telegraph lines were erected and old ones were renewed. Telegraph offices were established in every significant town. Telegraph schools were set up in London and the provinces to train young men and women in Morse telegraphy. The cost of a telegram fell to sixpence for a dozen words.

The transmission of telegrams presented some interesting technical problems. Chief among these was the need to send telegrams between locations that were not directly connected by a telegraph line. Consider the problem of a cigar manufacturer in Edinburgh, Scotland, negotiating with a tobacco importer in Bristol, England, some 350 miles south. There was no direct connection between these two great cities. Instead the telegram had to be passed, rather like a baton in a relay race, through the telegraph offices of intermediate cities: Edinburgh to Newcastle, Newcastle to York, York to Manchester, Manchester to Birmingham, and finally Birmingham to Bristol. At each intervening telegraph office the message was received by a telegraphist on a Morse sounder and written out in longhand. The message would then be resent by another telegraphist to

the next telegraph office in the chain. Although labor-intensive, the system was very resilient. If the telegraph lines between York and Manchester, say, were storm-damaged or simply very busy, the operator might send the message via Sheffield, which was not too much of a diversion. Sheffield would then send the message on its southerly route. Telegraphists needed to have a good knowledge of national geography.

After the government took over the telegraph system, and because London was the political and commercial center of Britain, it made sense for all major cities to have direct lines into the capital. In 1874 a central hub, the Central Telegraph Office, was established with a direct connection to “every town of importance in the United Kingdom.”¹³ The Central Telegraph Office occupied a purpose-built structure in St. Martin’s Le Grand, sited between Parliament, on the one hand, and the financial district and newspaper offices in Fleet Street, on the other. The office was the epitome of scientific modernity and featured in illustrated books and magazines. From the day it began operations, the great majority of national telegraph traffic passed through it: now our cigar manufacturer in Edinburgh could communicate with his tobacco importer in Bristol with just a single hop. This was faster, cheaper, and less likely to introduce transcription errors in the message.

In 1874, the *Illustrated London News* produced a full-page engraving of a gallery in the Central Telegraph Office. The image shows an information factory frozen in time: row upon row of young men and women are operating telegraph instruments, supervisors (generally just a little older than their charges) organize the work from a massive sorting table at the front of the room, while messenger boys (mostly fresh out of elementary school) run up and down the rows of telegraph stations collecting messages as they are transcribed and distributing them for onward transmission. The writer of the article explained – in words that spoke not only of the telegraph but of the times in which he lived:

It is a cheerful scene of orderly industry, and it is, of course, not the less pleasing because the majority of the persons here are young women, looking brisk and happy, not to say pretty, and certainly quite at home. Each has her own instrument on the desk before her. She is either just now actually busied in working off or in reading some message, or else, for the moment she awaits the signal, from a distant station, to announce a message for her reception. Boys move here and there about the galleries, with the forms of telegrams, which have been received in one part of the instrument-room, and which have to be signaled from another, but which have first to be conveyed, for record, to the nearest check-tables and sorting-tables in the centre.¹⁴

The journalist – evidently a man with a taste for statistics – noted that there were 1,200 telegraphists, of whom 740 were female, and 270 boy messengers.

Each day between 17,000 and 18,000 messages were transmitted between provincial telegraph offices, while nearly as many again were transmitted within London. But this was only the beginning. By the turn of the century, the Central Telegraph Office employed no fewer than 4,500 clerks and transmitted between 120,000 and 165,000 telegrams a day. It was the largest office in the world.

Herman Hollerith and the 1890 Census

Compared to Europe, the United States was a latecomer to large-scale data processing. That's because it lagged twenty or thirty years behind Europe in its economic development. When Britain, Germany, and France were industrializing in the 1830s, the United States was still primarily an agricultural country. It was not until after the Civil War that US businesses began to develop big offices, but this delay enabled them to take full advantage of the newly emerging office technologies.

Before the Civil War the only American data-processing bureaucracy of major importance was the Bureau of the Census in Washington, DC.¹⁵ The population census was established by an act of Congress in 1790 to determine the "apportionment" of members of the House of Representatives. The first census in 1790 estimated the population of the United States at 3.9 million and consequently established that there should be an apportionment of one representative for each 33,000 people, or 105 representatives. The early census data processing was very small scale, and no records exist as to how it was organized. Even by 1840, when the population was 17.1 million, there were still only 28 clerks in the Bureau of the Census. Twenty years later, however, by the 1860 census, a major bureaucracy was in place that employed 184 clerks to count a population of 31.4 million. For the 1870 census, there were 438 clerks, and the census reports amounted to 3,473 pages.

After that, the growth of the census was simply explosive. By the late nineteenth century not only was the US population burgeoning from waves of immigrants from Europe, China, and Latin America, but there was also a need for richer data on occupations, ethnicity, literacy, and health. The census of 1880 probably represented a high point in manual data processing in the United States, when no fewer than 1,495 clerks were employed to process the census data. The data-processing method in use then was known as the "tally system."¹⁶ This can best be understood by an example. One of the reports produced by the census was a table of the age structure of the population for each of the states, and the major cities and towns (that is, the number of males and females, in their ethnic categories, of each age). A tally clerk was provided with a large tally sheet, ruled into a grid, with a dozen columns and many rows. Each pair of columns corresponded to males and females of a particular ethnic group. The rows corresponded to the age of an individual – under one year of age, one year of age,

two years of age, and so on, up to a hundred years of age and over. The tally clerk would take a stack of census forms from an “enumeration district” (representing about a hundred families) and would proceed to examine the age, sex, and ethnic origin of each person on each form, putting a check mark in the appropriate cell of the grid. When the pile of forms had been processed in this way, the clerk would count the number of check marks in each of the cells and write the result at the side in red ink. This would be repeated for every enumeration district in the city. Finally, another clerk would total all the red-inked numbers on all the tally sheets for a city, entering the results onto a consolidation sheet, which would eventually become one of the tables in the census report.

The work of the census clerks was tedious beyond belief, prompting a journalist of the period to write: “The only wonder . . . is, that many of the clerks who toiled at the irritating slips of tally paper in the census of 1880 did not go blind and crazy.”¹⁷ More than twenty-one thousand pages of census reports were produced for the 1880 census, which took some seven years to process. This unreasonably long time provided a strong motive to speed up the census by mechanization or any other means that could be devised.

One person who was acutely aware of the census problem was a remarkable young engineer by the name of Herman Hollerith (1859–1929). He later developed a mechanical system for census data processing, commercialized his invention by establishing the Tabulating Machine Company in 1896, and laid the foundations of IBM. Along with Babbage, Hollerith is regarded as one of the seminal nineteenth-century figures in the development of information processing. Though Hollerith was not a deep thinker like the polymath Babbage, he was practical where Babbage was not. Hollerith also had a strong entrepreneurial flair, so that he was able to exploit his inventions and establish a major industry.

Hollerith grew up in New York and attended Columbia University, where one of his professors was an adviser to the Bureau of the Census in Washington. He invited the newly graduated Hollerith to become his assistant. While with the Census Bureau, Hollerith saw for himself the extraordinary scale of clerical activity – there was nothing else in the country to compare with it at the time. This familiarity with census operations enabled him to devise an electrical tabulating system that would mechanize much of the clerical drudgery.

Hollerith’s key idea was to record the census return for each individual as a pattern of holes on punched paper tape or a set of punched cards, similar to the way music was recorded on a string of punched cards on fairground organettes of the period. It would then be possible to use a machine to automatically count the holes and produce the tabulations. In later years Hollerith would reminisce on the origins of the idea: “I was traveling in the West and I had a ticket with what I think was called a punch photograph. [The conductor] punched out a description of the individual, as light hair, dark eyes, large nose, etc. So you see, I only made a punch photograph of each person.”¹⁸

In 1888 the preparations for the 1890 census had begun under the direction of a newly appointed superintendent of the census, Robert P. Porter. Porter, an English-born naturalized American, was a charismatic individual. He was a diplomat, an economist, and a journalist; founder and editor of the *New York Press*; an industrial pundit; and a well-known exponent on statistics. As soon as he was appointed superintendent, he set up a commission to select by competition an alternative to the system of tally sheets used in the 1880 and previous censuses. He was already an enthusiast for Hollerith's system; as a journalist he had written an article about it and would later become chairman of the British Tabulating Machine Company (the ancestor of International Computers, which became Europe's largest computer company).¹⁹ But to ensure a fair contest, he disallowed himself from being a judge of the competition.

Three inventors entered the competition, including Hollerith, and all of them proposed using cards or slips of paper in place of the old tally sheets. One of Hollerith's rivals proposed transcribing the census return for each individual onto a slip of paper, using inks of different colors for the different questions, so that the data could be easily identified and rapidly counted and sorted by hand. The second competitor had much the same idea but used ordinary ink and cards of different colors, which would then be easy to identify and arrange by hand. These two systems were entirely manual and were similar to the card-based record systems that were beginning to emerge ad hoc in big commercial offices. The great advantage of the Hollerith system over these others was that once the cards had been punched, all the sorting and counting would be handled mechanically.

In the fall of 1889, the three competitors were required to demonstrate their systems by processing the 10,491 returns of the 1880 population census for the district of St. Louis. The trial involved both recording the returns on the cards or paper slips and tabulating them to produce the required statistical tables. So far as recording the data on cards was concerned, the Hollerith system proved little faster than the competing manual systems. But in the tabulation phase it came into its own, proving up to ten times as fast as the rival systems. Moreover, once the cards had been punched, the more tabulations that were required, the more cost-effective the Hollerith system would prove. The commission was unanimous in recommending that the Hollerith Electric Tabulating System be adopted for the 1890 census.

As preparations for the eleventh United States population census got into high gear, Superintendent Porter "rounded up what appeared to be every square foot of empty office and loft space in downtown Washington."²⁰ At the same time, Hollerith made the transition from inventor to supervising manufacturer and subcontracted Western Electric to assemble his census machines. He also negotiated with paper manufacturers to supply the 60 million-plus manila cards that would be needed for the census.

On 1 June 1890, an army of forty-five thousand census enumerators swept the nation, preparing the schedules for the 13 million households and dispatching them to Washington. At the Census Bureau two thousand clerks were assembled in readiness, and on 1 July they began to process the greatest and most complete census the world had ever seen. It was a moment for the American people to “feel their nation’s muscle.”²¹

On 16 August 1890, six weeks into the processing of the census, the grand total of 62,622,250 was announced. But this was not what the allegedly fastest-growing nation in the world wanted to hear:

The statement by Mr. Porter that the population of this great republic was only 62,622,250 sent into spasms of indignation a great many people who had made up their minds that the dignity of the republic could only be supported on a total of 75,000,000. Hence there was a howl, not of “deep-mouthed welcome,” but of frantic disappointment. And then the publication of the figures of New York! Rachel weeping for her lost children and refusing to be comforted was a mere puppet-show compared with some of our New York politicians over the strayed and stolen of Manhattan Island citizens.²²

The press loved the story. In an article headlined “Useless Machines,” the *Boston Herald* roasted Porter and Hollerith; “Slip Shod Work Has Spoiled the Census,” exclaimed the *New York Herald*; and the other papers soon took up the story.²³ But Hollerith and Porter never had any real doubts about the system.

After the rough count was over and the initial flurry of public interest had subsided, the Census Bureau settled into a routine. Recording all the data onto the cards was a task that kept the seven hundred card punches in almost constant operation. A punching clerk – doing what was optimistically described as “vastly interesting” work – could punch an average of seven hundred cards in a six-and-a-half-hour day. Female labor was heavily used for the first time in the census, which a male journalist noted “augurs well for its conscientious performance” because “women show a moral sense of responsibility that is still beyond the average.”²⁴ More than 62 million cards were punched, one for each citizen.

The cards then were processed by the census machines, each of which could do the work of twenty of the former tally clerks. Even so, the original fifty-six census machines eventually had to be increased to a hundred (and the extra rentals significantly added to Hollerith’s income). Each census machine consisted of two parts: a tabulating machine, which could count the holes in a batch of cards, and the sorting box, into which cards were placed by the operator ready for the next tabulating operation. A census machine operator processed the cards one by one, by reading the information using a “press” that consisted of a plate of 288 retractable spring-loaded pins. When the press was forced down on the card, a

pin meeting the solid material was pushed back into the press and had no effect. But a pin encountering a hole passed straight through, dipped into a mercury cup, and completed an electrical circuit. This circuit would then be used to add unity to one of forty counters on the front of the census machine. The circuit could also cause the lid of one of the twenty-four compartments of the sorting box to fly open – into which the operator would place the card so that it would be ready for the next phase of the tabulation.

Thus, if one counter of the tabulating machine and one sorting box compartment were wired up for a male subject and another counter and compartment were wired up for a female subject, the census machine – by reading a batch of cards – could determine the number of males and females represented and then separate the cards accordingly. In practice, counts were much more complicated than this, in order to extract as much information as possible from the cards. Counts were designed to use all forty counters on the census machine and as many as possible of the twenty-four compartments in the sorting box.

At any one time there would be upward of eighty clerks operating the machines. Each operator processed at least a thousand cards an hour. In a typical day the combined operation would eat through nearly half a million cards: “In other words, the force was piercing its way through the mass at the rate of 500 feet daily, and handling a stack of cards nearly as high as the Washington Monument.” As each card was read, the census machine gave a ring of a bell to indicate that it had been correctly sensed. The floor full of machines ringing away made a sound “for all the world like that of sleighing,” commented a journalist. It was an awe-inspiring sight and sound; and, as the same journalist noted, “the apparatus works as unerringly as the mills of the Gods, but beats them hollow as to speed.”²⁵ Hollerith personally supervised the whole operation, combating both natural and unnatural mechanical breakdowns. One veteran of the census recounted:

Herman Hollerith used to visit the premises frequently. I remember him as very tall and dark. Mechanics were there frequently too, to get ailing machines back in operation and loafing employees back at work. The trouble was usually that somebody had extracted the mercury from one of the little cups with an eye-dropper and squirted it into a spittoon, just to get some unneeded rest.²⁶

The 1890 census was processed in two and a half years, compared with the seven years of the previous census. Altogether, the census reports totaled 26,408 pages.

The total cost was \$11.5 million, and estimates indicated that without the Hollerith system it would have cost \$5 million more. For those with eyes to see, the Hollerith machines had opened up a whole new vista of mechanized information processing.

America's Love Affair With Office Machinery

While the Hollerith system was the most visible use of mechanical information processing in the United States, it was really only one of many examples of what we now call "information technology" that developed in the decades following the Civil War. The Hollerith system was not in any sense typical of this information technology. Most office machines were much more commonplace: typewriters, record-keeping systems, and adding machines. In addition there were many varieties of office supplies available to American business: a hundred different types and grades of pencil; dozens of makes of fountain pens; paper clips, fasteners, and staplers of every conceivable variety; patented check-cutting machines to prevent fraud; cashier's coin trays and gadgets for sorting cash; carbon paper and typewriter sundries; loose-leaf ledgers and filing cabinets; wooden desks equipped with pigeonholes. The list goes on and on.

As offices grew in size, in line with the increasing volume of business in American companies, clerical operations had to become less labor-intensive and record keeping more systematic. It was to satisfy these requirements that the business-machine companies emerged. In the closing decades of the nineteenth century, office equipment, in both its most advanced and its least sophisticated forms, was almost entirely an American phenomenon. Its like would not be seen in Europe until the new century and, in many businesses, not until after World War I.

There were two main reasons for America's affinity for office machinery. First, because of the American office's late start compared with Europe, it did not carry the albatross of old-fashioned offices and entrenched archaic working methods. For example, back in England, the British Prudential Assurance Company had been established in the 1850s with Victorian data-processing methods that it could not shake off because redesigning the office system to make use of typewriters, adding machines, or modern card index systems was never cost-effective. Indeed, there was not a single typewriter in the Prudential before the turn of the century, and it was not until 1915 that any advanced office machinery was introduced. By contrast, the American Prudential Company in New York,²⁷ which was established twenty years after the British company, immediately made use of any office appliances on the market and became a recognized leader in utilizing office technology – a reputation it sustained right into the 1950s, when it was one of the first American offices to computerize. Similarly, while the British clearing houses were unmechanized until well into the new century, their American counterparts became massive users of Comptometers and Burroughs adding machines during the 1890s.

But no simple economic explanation can fully account for America's love affair with office machinery. The fact is that America was gadget-happy and was caught up by the glamour of the mechanical office. American firms often bought office appliances simply because they were perceived as modern – much

as American firms bought the first computers in the 1950s. This attitude was reinforced by the rhetoric of the office-systems movement.

Just as Frederick W. Taylor was pioneering scientific management in American industry in the 1880s, focusing on the shop floor, a new breed of scientific manager – or “systematizer” – was beginning to revolutionize the American office. As an early systematizer puffed to his audience in 1886:

Now, administration without records is like music without notes – by ear. Good as far as it goes which is but a little way – it bequeathes nothing to the future. . . . Under rational management the accumulation of experience, and its systematic use and application, form the first fighting line.²⁸

Systematizers set about restructuring the office, introducing typewriters and adding machines, designing multipart business forms and loose-leaf filing systems, replacing old-fashioned accounting ledgers with machine billing systems, and so on. The office systematizer was the ancestor of today’s information-technology consultant.

Powered by the fad for office rationalization, the United States was the first country in the world to adopt office machinery on a large scale. This early start enabled the United States to become the world’s leading producer of information-technology goods. In turn, the United States dominated the typewriter, record-keeping, and adding-machine industries for most of their histories; it dominated the accounting-machine industry between the two world wars; and it established the computer industry after World War II. There is thus an unbroken line of descent from the giant office-machine firms of the 1890s to the computer makers of the twentieth century.

Notes to Chapter 1

A modern account of table making is *The History of Mathematical Tables* (2003), edited by Martin Campbell-Kelly et al. This contributed volume includes chapters on Napier’s logarithms, de Prony’s *tables du cadastre*, Babbage’s Difference Engines, and other episodes in the history of tables. David Grier’s *When Computers Were Human* (2005) includes excellent accounts of the computation of the *Nautical Almanac* and the *tables du cadastre*. The “tables crisis” and Babbage’s calculating engines are described in Doron Swade’s very readable *The Difference Engine: Charles Babbage and the Quest to Build the First Computer* (2001) and his earlier, beautifully illustrated *Charles Babbage and His Calculating Engines* (1991). Swade writes with great authority and passion, as he was the curator responsible for building Babbage’s Difference Engine No. 2 to celebrate the bicentennial of Babbage’s birth in 1991. Detailed accounts of Victorian data processing are given in Martin Campbell-Kelly’s articles on the Prudential Assurance Company, the Railway Clearing

House, the Post Office Savings Bank, and the British Census (1992, 1994, 1996, and 1998) and in Jon Agar's *The Government Machine* (2003). The early American scene is described in Patricia Cline Cohen's *A Calculating People* (1982).

The political, administrative, and data-processing background to the US Census is given in Margo Anderson's *The American Census* (1988). The most evocative description of the 1890 US Census is given in a contemporary account by Thomas C. Martin, "Counting a Nation by Electricity" (1891); a more prosaic account is given in Leon Truesdell's *The Development of Punch Card Tabulation in the Bureau of the Census, 1890–1940* (1965). The standard biography of Hollerith is Geoffrey D. Austrian's *Herman Hollerith: Forgotten Giant of Information Processing* (1982). The early office-systems movement is described in JoAnne Yates's *Control Through Communication* (1989).

Notes

1. Comrie 1933, p. 2.
2. Quoted in Grattan-Guinness 1990, p. 179.
3. Ibid.
4. Babbage 1822a.
5. Quoted in Swade 1991, p. 2.
6. Lardner 1834.
7. Quoted in Swade 1991, p. 2.
8. Hyman 1982, p. 49.
9. Babbage 1834.
10. For a description of the British and US clearing houses, see Cannon 1910.
11. Quoted in Alborn 1994, p. 10.
12. Babbage 1835, p. 89.
13. Anon. 1920. "The Central Telegraph Office, London." BT Archives, London, POST 82/66.
14. Anon. 1874, p. 506.
15. The administrative organization of the Bureau of the Census is described in Anderson 1988; head count and page count statistics appear on p. 242.
16. The tally system is described in Truesdell 1965, chap. 1.
17. Martin 1891, p. 521.
18. Austrian 1982, p. 15.
19. Campbell-Kelly 1989, pp. 16–17.
20. Austrian 1982, p. 59.
21. Ibid., p. 58.
22. Martin 1891, p. 522.
23. The quotations appear in Austrian 1982, pp. 76, 86.
24. Martin 1891, p. 528.
25. Ibid., pp. 522–525.
26. Quoted in Campbell-Kelly 1989, p. 13.
27. On the American Prudential, see May and Oursler 1950; Yates 1993.
28. Quoted in Yates 1989, p. 12.

2 The Mechanical Office

IN 1928 THE WORLD'S top four office-machine suppliers were Remington Rand, with annual sales of \$60 million; National Cash Register (NCR), with sales of \$50 million; the Burroughs Adding Machine Company, with \$32 million worth of business; and – trailing the three leaders by a considerable margin – IBM, with an income of \$20 million. Forty years later those same firms were among the top ten computer manufacturers, and of the four, IBM, whose sales exceeded those of the other three combined, was the third-largest corporation in the world, with annual revenues of \$21 billion and a workforce of a third of a million people.

To understand the development of the computer industry, and how this apparently new industry was shaped by the past, one must understand the rise of the office-machine giants in the years around the turn of the twentieth century. This understanding is necessary, above all, to appreciate how IBM's managerial style, sales ethos, and technologies combined to make it perfectly adapted to shape and then dominate the computer industry for a period of forty years.

The principal activities of offices were communications with customers and suppliers, the systematic storage of records, and bookkeeping and accounting. The business-machine companies of the late nineteenth century were established to serve these needs. Remington Rand was the leading supplier of typewriters for document preparation and filing systems for information storage. Burroughs dominated the market for adding machines used for simple calculations. IBM dominated the market for punched-card accounting machines. And NCR, having started out making cash registers in the 1880s, also became a major supplier of accounting machines.

The Typewriter

The present is shaped by the past. Nowhere is this more evident than in the history of the typewriter. Little more than a century ago the female office worker was a rarity; office jobs were held almost exclusively by men. Today, so much office work is done by women that this fact often goes unremarked. It was above

all the opportunities created by the typewriter that ushered female workers into the office.

A much smaller example of the past shaping the present can be seen on the top row of letters on a present-day computer keyboard: QWERTYUIOP. This wonderfully inconvenient arrangement was used in the first commercially successful typewriter produced by Remington in 1874. Because of the investment in the training of typists, and the natural reluctance of QWERTY-familiar people to change, there has never been a right moment to switch to a more convenient keyboard layout. And maybe there never will be.

Prior to the development of the inexpensive, reliable typewriter, one of the most common office occupations was that of a “writer” or “copyist.” These were clerks who wrote out documents in longhand. There were many attempts at inventing typewriters in the middle decades of the nineteenth century, but none of them was commercially successful because none could overcome both of the critical problems of document preparation: the difficulty of reading handwritten documents and the time it took a clerk to write them. In the nineteenth century virtually all business documents were handwritten, and the busy executive spent countless hours deciphering them. Hence the major attraction of the typewriter was that typewritten documents could be read effortlessly at several times the speed of handwritten ones. Unfortunately, using these early typewriters was a very slow business. Until the 1870s, no machine came even close to the twenty-five-words-per-minute handwriting speed of an ordinary copyist. Not until typewriter speeds could rival those of handwriting did the typewriter catch on.

Newfangled typewriting machines were scientific novelties, and articles describing them occasionally appeared in technical magazines. As a result of reading such an article in *Scientific American* in 1867, Christopher Latham Sholes, a retired newspaper editor, was inspired to invent and take out a series of patents on a new style of typewriter. This was the machine that became the first Remington typewriter. What differentiated Sholes’s invention from its predecessors was its speed of operation, which was far greater than that of any other machine invented up to then. This speed was due to a “keyboard and type-basket” arrangement, variations of which became almost universal in manual typewriters.

Sholes needed money to develop his invention, though, so he wrote – or rather typed – letters to all the financiers he knew of, offering to “let them in on the ground floor in return for ready funds.”¹ One of the letters was received by a Pennsylvania business promoter named James Densmore. A former newspaperman and printer, Densmore immediately recognized the significance of Sholes’s invention and offered him the necessary backing. Over the next three or four years, Densmore’s money enabled Sholes to perfect his machine and ready it for the marketplace.

One of the problems of the early models was that when operated at high speed, the type-bars would clash and jam the machine. On the very first machines, the letters of the keyboard had been arranged in alphabetical order, and the major cause of the jamming was the proximity of commonly occurring letter pairs (such as D and E, or S and T). The easiest way around this jamming problem was to arrange the letters in the type-basket so that they were less likely to collide. The result was the QWERTY keyboard layout that is still with us.² (Incidentally, a vestige of the original alphabetical ordering can be seen on the middle row of the keyboard, where the sequence FGHIJKL appears.)

Densmore made two attempts to get the Sholes typewriter manufactured by small engineering workshops, but they both lacked the necessary capital and skill to manufacture successfully and cheaply. As one historian of manufacturing has noted, the “typewriter was the most complex mechanism mass produced by American industry, public or private, in the nineteenth century.”³ Not only did the typewriter contain hundreds of moving parts, it also used unfamiliar new materials such as rubber, sheet iron, glass, and steel.

In 1873 Densmore managed to interest Philo Remington in manufacturing the typewriter. Remington was the proprietor of a small-arms manufacturer, E. Remington and Sons, of Ilion, New York. He had no great interest in office machinery, but in the years after the Civil War the firm, needing to turn swords into plow-shares, started to manufacture items such as sewing machines, agricultural equipment, and fire engines. Remington agreed to make a thousand typewriters.

The first ones were sold in 1874. Sales were slow at first because there was no ready market for the typewriter. Business offices had barely begun to use machines of any kind, so the first customers were individuals such as “reporters, lawyers, editors, authors, and clergymen.”⁴ One early enthusiast, for example, was Samuel Langhorne Clemens – better known as Mark Twain – who wrote for Remington one of the most famous testimonials ever:

Gentlemen:

Please do not use my name in any way. Please do not even divulge the fact that I own a machine. I have entirely stopped using the Type-Writer, for the reason that I never could write a letter with it to anybody without receiving a request by return mail that I would not only describe the machine but state what progress I had made in the use of it, etc., etc. I don't like to write letters, and so I don't want people to know that I own this curiosity breeding little joker.

Yours truly, Saml. L. Clemens.⁵

It took five years for Remington to sell its first thousand machines. During this period the typewriter was further perfected. In 1878 the company introduced the

Remington Number 2 typewriter, which had a shift key that raised and lowered the carriage so that both upper and lowercase letters could be printed. (The first Remington typewriter printed only capital letters.)

By 1880 Remington was making more than a thousand machines a year and had a virtual monopoly on the typewriter business. One of the crucial business problems that Remington addressed at an early date was sales and after-sales service. Sales offices enabled potential customers to see and try out different models of typewriter before making a purchase. Typewriters were delicate instruments; they were prone to break down and required trained repair personnel. In this respect, the typewriter was a little like the sewing machine, which had become a mass-produced item about a decade earlier. Following Singer's lead, the Remington Typewriter Company began to establish local branches in the major cities from which typewriters were sold and to which they could be returned for repair. Remington tried to get Singer to take over its overseas business through its branch offices abroad, but Singer declined, and so Remington was forced to open branch offices in most of the major European cities in the early years of the twentieth century.

By 1890 the Remington Typewriter Company had become a very large concern indeed, making twenty thousand machines a year. During the last decade of the century it was joined by several competitors: Smith Premier in 1889, Oliver in 1894, Underwood in 1895, and Royal a few years later. During this period the typewriter as we know it became routinely available to large and small businesses. By 1900 there were at least a dozen major manufacturers making a hundred thousand typewriters a year. Sales continued to soar right up to World War I, by which time there were several dozen American firms in existence and more in Europe. Selling for an average price of \$75, typewriters became the most widely used business machines, accounting for half of all office-appliance sales.

The manufacture and distribution of typewriters made up, however, only half the story. Just as important was the training of workers to use them. Without training, a typewriter operator was not much more effective than an experienced writing clerk. Beginning in the 1880s, organizations came into being to train young workers (mostly women) for the new occupation of "type-writer," later known as a "typist." Typing had a good deal in common with shorthand stenography and telegraphy, in that it took several months of intense practice to acquire the skill. Hence many typewriting schools grew out of the private stenography and telegraph schools that had existed since the 1860s. The typewriter schools helped satisfy a massive growth in the demand for trained office workers in the 1890s. Soon the public schools joined in the task of training this new workforce, equipping hundreds of thousands of young women and men with typewriting skills. The shortage of male clerks to fill the burgeoning offices at the turn of the century created the opportunity for women to enter the workplace in

ever-increasing numbers. By 1900 the US Census recorded 112,000 typists and stenographers in the nation, of whom 86,000 were female.⁶ Twenty years previously, there had been just 5,000, only 2,000 of them women.

By the 1920s office work was seen as predominantly female, and typing as universally female. As a writer on women's work characterized the situation with intentional irony, "a woman's place is at the typewriter."⁷ Certainly the typewriter was the machine that above all brought women into the office, but there was nothing inherently gendered about the technology and today "keyboarding" is the province equally of men and women.

For many years, historians of information technology neglected the typewriter as a progenitor of the computer industry. But now we can see that it is important to the history of computing in that it pioneered three key features of the office-machine industry and the computer industry that succeeded it: the perfection of the product and low-cost manufacture, a sales organization to sell the product, and a training organization to enable workers to use the technology.

The Rands

Left to itself, it is unlikely that the Remington Typewriter Company would have succeeded in the computer age – certainly none of the other typewriter companies did. However, in 1927 Remington became part of a conglomerate, Remington Rand, organized by James Rand Sr.⁸ and his son James Jr. The Rands were the inventor-entrepreneur proprietors of the Rand Kardex Company, the world's leading supplier of record-keeping systems.

Filing systems for record keeping were one of the breakthrough business technologies, occurring roughly in parallel with the development of the typewriter. The fact that loose-leaf filing systems are so obviously low-tech does not diminish their importance. Without the technology to organize loose sheets of paper, the typewriter revolution itself could never have occurred.

Before the advent of the typewriter and carbon copies, business correspondence was kept in a "letter book." There was usually both an ingoing and an outgoing letter book, and it was the duty of a copying clerk to make a permanent record of each letter as it was sent out or received – by copying it out in longhand into the letter book. It was inevitable that this cumbersome business practice would not survive the transition from small businesses to major enterprises in the decades after the Civil War. That process was greatly accelerated by the introduction of the typewriter and carbon copies, which made document preparation convenient.

The first modern "vertical" filing system was introduced in the early 1890s and won a Gold Medal at the 1893 World's Fair. The vertical filing system was a major innovation, but it has become so commonplace that its significance is rarely noted.

It is simply the system of filing records in wood or steel cabinets, each containing three or four deep drawers; in the drawers, papers are stored on their edges. Papers on a particular subject or for a particular correspondent are grouped together between cardboard dividers with index tabs. Vertical filing systems used only a tenth of the space of the box-and-drawer filing systems they replaced and made it much faster to access records.

At first, vertical filing systems coped very well, but as business volumes began to grow in the 1890s, they became increasingly ineffective. One problem was that organizing correspondence or an inventory worked quite well for a hundred or even a thousand records but became very cumbersome with tens or hundreds of thousands of records.

Around this time, James Rand Sr. (1859–1944) was a young bank clerk in his hometown of Tonawanda, New York. Fired up by the difficulties of retrieving documents in existing filing systems, in his spare time he invented and patented a “visible” index system of dividers, colored signal strips, and tabs, enabling documents to be held in vertical files and located through an index system three or four times faster than previously. The system could be expanded indefinitely, and it enabled literally millions of documents to be organized with absolute accuracy and retrieved in seconds.

In 1898 Rand organized the Rand Ledger Company, which quickly came to dominate the market for record-keeping systems in large firms. By 1908 the firm had branch offices in nearly forty US cities and agencies throughout the world. Rand took his son, James Rand Jr., into the business in 1908. Following his father’s example, Rand junior invented an ingenious card-based record-keeping system that the company began to market as the Kardex System. It combined the advantages of an ordinary card index with the potential for unlimited expansion and instant retrieval. In 1915 Rand junior broke away from his father and, in a friendly family rivalry, set up as the Kardex Company. Rand junior was even more successful than his father. By 1920 he had over a hundred thousand customers, ninety branch offices in the United States, a plant in Germany, and sixty overseas offices.

In 1925 Rand senior was in his mid-sixties and approaching retirement. At the urging of his mother, Rand junior decided that he and his father should resume their business association, and the two companies were merged as the Rand Kardex Company, with the father as chairman and the son as president and general manager. The combined firm was by far the largest supplier of business record-keeping systems in the world, with 4,500 field representatives in 219 branch offices in the United States and 115 agencies in foreign countries.

Rand junior set out on a vigorous series of mergers and business takeovers, aimed at making the company the world’s largest supplier of all kinds of office equipment. Several other manufacturers of loose-leaf card index systems were acquired early on, followed by the Dalton Adding Machine Company and, later,

the Powers Accounting Machine Company – the makers of a rival punched-card system to Hollerith's. In 1927, the company merged with the Remington Typewriter Company and became known as Remington Rand. With assets of \$73 million, it was far and away the largest business-machine company in the world.

The First Adding Machines

Typewriters aided the documentation of information, while filing systems facilitated its storage. Adding machines were primarily concerned with bookkeeping and accounting. The development of mass-produced adding machines followed the development of the typewriter by roughly a decade, and there are many parallels in the development of the two industries.

The first commercially produced adding machine was the Arithmometer, developed by Thomas de Colmar of Alsace at the early date of 1820. However, the Arithmometer was never produced in large quantities; it was hand-built probably at the rate of no more than one or two a month. The Arithmometer was also not a very reliable machine; a generation went by before it was produced in significant quantities (there is virtually no mention of it until the Great Exhibition of 1851 in London). Sturdy, reliable Arithmometers became available in the 1880s, but the annual production never exceeded a few dozen.

Although the Arithmometer was relatively inexpensive, costing about \$150, demand remained low because its speed of operation was too slow for routine adding in offices. To operate it, one had to register the number digit by digit on a set of sliders and then perform the addition by turning a handle. Arithmometers were useful in insurance companies and in engineering concerns, where calculations of seven-figure accuracy were necessary. But they were irrelevant to the needs of ordinary bookkeepers. The bookkeeper of the 1880s was trained to add up a column of four-figure amounts mentally almost without error and could do so far more quickly than by using an Arithmometer.

At the beginning of the 1880s, the critical challenge in designing an adding machine for office use was to speed up the rate at which numbers could be entered. Of course, once this problem had been solved a second problem came into view: financial organizations, particularly banks, needed a written record of numbers as they were entered into the adding machine so that they would have a permanent record of their financial transactions.

The solvers of these two critical problems, Dorr E. Felt and William S. Burroughs, were classic inventor-entrepreneurs. The machines they created, the Comptometer and the Burroughs Adding Machine, dominated the worldwide market well into the 1950s. Only the company founded by Burroughs, however, successfully made the transition into the computer age.

Dorr E. Felt was an obscure machinist living in Chicago when he began to work on his adding machine. The technological breakthrough that he achieved in

his machine was that it was “key-driven.” Instead of the sliders of the Arithmometer, or the levers used on other adding machines that had to be painstakingly set, Felt’s adding machines had a set of keys, a little like those of a typewriter. The keys were arranged in columns, labeled 1 through 9 in each column.⁹ If an operator pressed one figure in each column, say, the 7, 9, 2, 6, and 9 keys, the amount \$792.69 would be added into the total displayed by the machine. A dexterous operator could enter a number with as many as ten digits (that is, one digit per finger) in a single operation. In effect, Felt had done for the adding machine what Sholes had done for the typewriter.

In 1887, at the age of twenty-four, Felt went into partnership with a local manufacturer, Robert Tarrant, to manufacture the machine as the Felt & Tarrant Manufacturing Company. As with the typewriter, sales were slow at first, but by 1900 the firm was selling over a thousand machines a year.

There was, however, more to the Comptometer business than making and selling machines. As with the typewriter, the skill to use the Comptometer was not easily acquired. Hence the company established Comptometer training schools. By World War I schools had been established in most major cities of the United States, as well as in several European centers. At the Comptometer schools young women and men, fresh from high school, would in a period of a few months of intense training become adept at using the machines. Operating a Comptometer at speed was an impressive sight: in film archives one can find silent motion pictures showing Comptometer operators entering data far faster than it could be written down, far faster than even the movies could capture – their fingers appear as a blur. As with learning to ride a bicycle, once the skill was acquired it would last a working life – until Comptometers became obsolete in the 1950s.

Between the two world wars, Comptometers were phenomenally successful, and millions of them were produced. But the company never made it into the computer age. This was essentially because Felt & Tarrant failed to develop advanced electromechanical products in the 1930s. When the vacuum tube computer arrived after World War II, the company was unprepared to make the technological leap into computers. Eventually it was acquired in the 1960s in a complex merger and, like much of the old office-appliance industry, vanished into oblivion.

William S. Burroughs made the first successful attack on the second critical problem of the office adding machine: the need to print its results. Burroughs was the son of a St. Louis mechanic and was employed as a clerk at a bank, where the drudgery of long hours spent adding up columns of figures is said to have caused a breakdown in his health. He therefore decided at the age of twenty-four to quit banking and follow his father by becoming a mechanic. Putting the two together, Burroughs began to develop an adding machine for banks, which would not only add numbers rapidly like a Comptometer but also print the numbers as they were entered.

In 1885 Burroughs filed his first patent, founded the American Arithmometer Company, and began to hand-build his “adder-lister.” The first businesses targeted by Burroughs were the banks and clearing houses for which the machine had been especially designed. After a decade’s mechanical perfection and gradual buildup of production, by 1895 the firm was selling several hundred machines a year, at an average price of \$220 (equivalent to \$7000 today). When William Burroughs died in his mid-forties in 1898, the firm was still a relatively small organization, selling fewer than a thousand machines in that year.

In the early years of the new century, however, sales began to grow very rapidly. By 1904 the company was making 4,500 machines a year, had outgrown its St. Louis plant, and had moved to Detroit, where it was renamed the Burroughs Adding Machine Company. Within a year annual production had risen to nearly 8,000 machines, the company had 148 representatives selling the machines, and several training schools had been established. Three years later Burroughs was selling over 13,000 machines a year, and its advertisements boasted that there were 58 different models of machine: “One Built for Every Line of Business.”¹⁰

In the first decade of the new century, Felt & Tarrant and the Burroughs Adding Machine Company were joined by dozens of other calculating-machine manufacturers such as Dalton (1902), Wales (1903), National (1904), and Madas (1908). A major impetus to development of the US adding-machine industry occurred in 1913 with the introduction of a new tax law that adopted progressive tax rates and the withholding of tax from pay. A further increase in paperwork occurred during World War I, when corporate taxes were expanded.

Of all the many American adding-machine firms that came into existence before World War I, only Burroughs made a successful transition into the computer age. One reason was that Burroughs supplied not only adding machines but also the intangible know-how of incorporating them into an existing business organization. The company became very adept at this and effectively sold business systems alongside their machines. Another factor was that Burroughs did not cling to a single product but, instead, gradually expanded its range in response to user demand. Between the two world wars Burroughs moved beyond adding machines and introduced full-scale accounting machines. The knowledge of business accounting systems became deeply embedded in the sales culture of Burroughs, and this helped ease its transition to computers in the 1950s and 1960s.

The National Cash Register Company

In this brief history of the office-machine industry, we have repeatedly alluded to the importance to firms of their sales operations, in which hard selling was reinforced by the systematic analysis of customer requirements, the provision of after-sales support, and user training. More than any other business sector,

the office-machine industry relied on these innovative forms of selling, which were pioneered largely by the National Cash Register Company in the 1890s. When an office-machine firm needed to build up its sales operation, the easiest way was to hire someone who had learned the trade with NCR. In this way the techniques that NCR developed diffused throughout the office-machine industry and, after World War II, became endemic to the computer industry.

Several histories have been written of NCR, and they all portray John H. Patterson, the company's founder, as an aggressive, egotistical crank and identify him as the role model for Thomas J. Watson Sr., the future leader of IBM. At one point in his life, Patterson took up Fletcherism, a bizarre eating fad in which each mouthful of food was chewed sixty times before swallowing. Another time he became obsessed with calisthenics and physical fitness and took to horse riding at six o'clock each morning – and obliged his senior executives to accompany him. He was capricious, as likely to reward a worker with whom he was pleased with a large sum of money as to sack another on the spot for a minor misdemeanor. He was a leading figure in the Garden City movement, and the NCR factory in Dayton, Ohio, was one of its showpieces, its graceful buildings nestling in a glade planted with ever-green trees. However, for all his idiosyncrasies, there is no question that Patterson possessed an instinct for organizing the cash register business, which was so successful that NCR's "business practices and marketing methods became standard at IBM and in other office appliance firms by the 1920s."¹¹

As with the typewriter and the adding machine, there were a number of attempts to develop a cash register during the nineteenth century. The first practical cash register was invented by a Dayton restaurateur named James Ritty. Ritty believed he was being defrauded by his staff, so he invented "Ritty's Incorruptible Cashier" in 1879. When a sale was made, the amount was displayed prominently for all to see, as a check on the clerk's honesty. Inside the machine, the transaction was recorded on a roll of paper. At the end of the day, the store owner would add up the day's transactions and check the total against the cash received. Ritty tried to go into business manufacturing and selling his machine, but he was unsuccessful and sold out to a local entrepreneur. Apparently, before giving up, Ritty sold exactly one machine – to John H. Patterson, who at the time was a partner in a coal retailing business.

In 1884, at the age of forty, Patterson, having been outmaneuvered by some local mining interests, decided to quit the coal business, even though "the only thing he knew anything about was coal."¹² This was not quite true: he also knew about cash registers, since he was one of the very few users of the machine. He decided to use some of the money from the sale of the coal business to buy the business originally formed by Ritty, which he renamed the National Cash Register Company. Two years later the National Cash Register Company was selling more than a thousand machines a year.

Patterson understood that the cash register needed constant improvement to keep it technically ahead of the competition. In 1888 he established a small “inventions department” for this purpose. Although formal research and development departments had been established since the 1870s in the chemical and electrical industries, NCR’s was probably the first in the office-machine industry, and it was copied by IBM and others. Over the next forty years, the NCR inventions department was to accrue more than two thousand patents. Its head, Charles Kettering, went on to found the first research laboratory for General Motors.

With Patterson’s leadership, NCR came to dominate the world market for cash registers and grew phenomenally. By 1900 NCR was selling nearly 25,000 cash registers a year and had 2,500 employees. By 1910 it was selling 100,000 registers a year and had more than 5,000 employees. In 1922, the year Patterson died, NCR sold its two-millionth cash register.

Under Patterson, NCR had been a one-product company, but however successful that product had been, NCR would never have become a major player in the computer industry. But after Patterson’s death, the executives who remained in control of NCR decided to diversify into accounting machines. As Stanley Allyn (later a president of the company) put it, NCR decided to “turn off the worn, unexciting highway into the new land of mechanized accounting.”¹³

By the early 1920s, NCR had very sophisticated market research and technical development departments. Between them they initiated a major redirection of the company’s business, based on the Class 2000 accounting machine launched in 1926. A fully fledged accounting machine, the Class 2000 could handle invoicing, payroll, and many other business accounting functions. From this time on, the company was no longer called the National Cash Register Company but was known simply as NCR. The NCR Class 2000 accounting machine was as sophisticated as any accounting machine of its day and was fully equal to the machines produced by Burroughs.

NCR’s greatest legacy to the computer age, however, was the way it shaped the marketing of business machines and established virtually all of the key sales practices of the industry. It would be wrong to credit Patterson with inventing all of these sales techniques. Certainly he invented some basic ideas, but mostly he embellished and formalized existing practices. For example, he adopted the Singer Sewing Machine idea of establishing a series of branch retail outlets. Cash registers could be bought directly from the NCR outlets, and if they broke down they could be returned for repair by factory-trained engineers, with a loaner machine provided while the machine was being mended.

Patterson learned early on that store owners rarely walked off the street into an NCR store to buy a cash register; as late as the turn of the century, few retailers even knew what a cash register was. As Patterson put it, cash registers were sold, not bought. Thus he developed the most effective direct sales force in the country.

Patterson knew from personal experience that selling could be a lonely and soul-destroying job. Salesmen needed to be motivated both financially and spiritually. Financially, Patterson rewarded his salesmen with an adequate basic salary and a commission that could make them positively rich. In 1900 he introduced the concept of the sales quota, which was a stick-and-carrot sales incentive. The carrot was membership in the Hundred Point Club, for those achieving 100 percent of their quota. The reward for membership in the club was the prestige of attending an annual jamboree at the head office in Dayton, being recognized by the company brass, and hearing an inspirational address from Patterson himself. He would illustrate his talks using an oversized notepad resting on an easel, on which he would crayon the main points of his argument. As NCR grew in size, the Hundred Point Club grew to several hundred members, far exceeding the capacity of Dayton's hotels to accommodate them, so the annual get-together was moved to a tent city on a grassy field owned by NCR.

NCR salesmen were supported by a flood of literature produced by the NCR head office. From the late 1880s, NCR began to mail product literature to tens of thousands of sales "prospects" (an NCR term). At first these pamphlets were foldouts, but later a magazine-style booklet called *The Hustler* was produced, which targeted specific merchants such as dry-goods stores or saloons. These booklets showed how to organize retail accounting systems and gave inspirational encouragement: "You insure your life. Why not insure your money too! A National cash register will do it."¹⁴

Patterson also established sales training. He transformed a career in selling from being a somewhat disreputable occupation to something close to a profession. In 1887 he got his top salesman to set down his sales patter in a small booklet, which he called the *NCR Primer*. Effectively a sales script, it was "the first canned sales approach in America."¹⁵ In 1894 Patterson founded a sales training school through which all salesmen eventually passed: "The school taught the primer. It taught the demonstration of all the machines the company then made. It taught price lists, store systems, the manual, the getting of prospects, and the mechanism of the machine."¹⁶ An early graduate of the NCR sales school was Thomas J. Watson. Attending the school changed his life.

Thomas Watson and the Founding of IBM

Thomas J. Watson was born in Campbell, in rural New York state, in 1874, the son of a strict Methodist farmer. At the age of eighteen, Watson went to commercial college, then became a bookkeeper, earning \$6 a week. He disliked sedentary office life, however, and decided to take a riskier but better-paid (\$10-a-week) job selling pianos and organs. On the road, with a horse-drawn wagon, carrying a piano, he learned to sell. These were basic if naive lessons in developing a sales patter and a glad-handing approach.

Always looking to better himself, in 1895 Watson managed to get a post as a cash register salesman with NCR. He was then one among hundreds of NCR salesmen. Armed with the *NCR Primer* – and assiduously learning from established salesmen – Watson became an outstanding salesman. He attended the sales training school at NCR's headquarters and claimed that it enabled him to double his sales thereafter. After four years as one of NCR's top salesmen, he was promoted to overall sales agent for Rochester, New York. At the age of twenty-nine, he had a record that marked him out for higher things.

In 1903 Patterson summoned Watson to his Dayton headquarters and offered him the opportunity to run an "entirely secret" operation. At this time NCR was facing competition from sellers of secondhand cash registers. The proposal that Patterson put to Watson was that he should buy up secondhand cash registers, sell them cheaply, and hence drive the secondhand dealers out of business. As Patterson put it, "the best way to kill a dog is to cut off its head."¹⁷ Even by the buccaneering standards of the early twentieth century this was an unethical, not to say illegal, operation. Watson later came to regret his moral lapse, but it served to push him upward in NCR. This chicken would come home to roost a few years later.

At age thirty-three, Watson in 1908 was promoted to sales manager, and in that position he gained an intimate knowledge of what was acknowledged to be the finest sales operation in America. He found himself in charge of the company's sales school and, like Patterson, he had to give inspirational talks. Watson exceeded even Patterson as the master of the slogan: "ever onward . . . aim high and think in big figures; serve and sell; he who stops being better stops being good."¹⁸ And so on. Watson coined an even simpler slogan, the single word T-H-I-N-K. Soon notices bearing the word were pinned up in NCR branch offices.

In early 1912 NCR's monopolistic practices finally caught up with it. As the *Dayton Daily News* reported on its front page "Thirty officials and employees of the National Cash Register company of Dayton were indicted on charges of criminal violation of the Sherman anti-trust law."¹⁹ The Sherman Antitrust Act was enacted in 1890 to constrain the monopolistic behavior of major companies. The first prosecutions were of giant firms such as Standard Oil, American Tobacco, and US Steel, controlled by the infamous "robber barons" of the era. Myriad cases followed in the ensuing decades. In November 1912, the NCR officials and employees were arraigned in a Cincinnati court in a trial lasting several months. Twenty-nine of the thirty were found guilty, fined, and sentenced to prison, pending an appeal. Not long after the trial Watson and NCR parted company.

By this time Watson had become heir apparent as general manager of NCR – when the capricious Patterson fired him. At thirty-nine years of age, the super-salesman Thomas Watson found himself unemployed. He would shortly surface

as the president of C-T-R, the company that had acquired the rights to the Hollerith punched-card system.

BY ABOUT 1905, although the typewriter and adding-machine industries had become significant sectors of the economy, the punched-card machine industry was still in its infancy, for there were no more than a handful of installations in the world, compared with over a million typewriters and hundreds of thousands of adding machines. Twenty years later this picture would be dramatically altered.

When Hollerith first went into business with his electric tabulating system in 1886, it was not really a part of the office-machine industry in the same sense as Remington Typewriter, Burroughs Adding Machine, or NCR. These firms were characterized by the high-volume manufacture of relatively low-priced machines. By contrast, Hollerith's electric tabulating system was a very expensive, highly specialized system limited in its applications to organizations such as the Census Bureau. Hollerith lacked the vision and background to turn his company into a mainstream office-machine supplier, and it would never have become a major force in the industry until he passed control to a manager with a business-machine background.

The 1890 US population census was a major triumph for Hollerith. But as the census started to wind down in 1893, he was left with a warehouse full of census machines – and there most of them would remain until the 1900 census. In order to maintain the income of his business he needed to find new customers, and thus he adapted his machines so that they could be used by ordinary American businesses.

On 3 December 1896, Hollerith incorporated his business as the Tabulating Machine Company (TMC). He managed to place one set of machines with a railway company, but he was quickly diverted by the much easier profits of the upcoming 1900 US population census. The census sustained TMC for another three years, but as the census operation wound down, Hollerith was again faced with a declining income and turned his attention once more to private businesses. This time he did so with much more application, however, and managed to place several sets of machines with commercial organizations.

It was just as well that Hollerith had turned his attention to the commercial applications of his machines, at last, because his easy dealings with the Census Bureau were about to come to an end. The superintendent of the census was a political appointment, and Superintendent Robert Porter – who had supported Hollerith in both the 1890 and 1900 censuses – was the appointee of President McKinley (whose biography Porter wrote). In 1901, McKinley was assassinated and Porter's career in the public service came to an abrupt end. Porter returned to his home country of England and set up the British Tabulating Machine Company in London. Meanwhile a new superintendent had been appointed, one who did not care for the Census Bureau's cozy relationship with Hollerith's company and who considered the charges for the 1900 census to have been unacceptably high.

In 1905, unable to agree on terms, Hollerith broke off business relations with the Census Bureau and put all his efforts into the commercial development of his machines. Two years later the Bureau engaged a mechanical engineer named James Powers to improve and develop tabulating machines for the next census. Powers was an inventor-entrepreneur in the same mold as Hollerith, and he was soon to emerge as a serious competitor – producing a machine that improved on Hollerith's by printing its results.

Meanwhile, Hollerith continued to perfect the commercial machines, producing a range of “automatics” that was to stay in production for the next twenty years. There were three punched-card machines in the commercial setup: a key-punch to perforate the cards, a tabulator to add up the numbers punched on the cards, and a sorter to arrange the cards in sequence. All punched-card offices had at least one each of these three machines. Larger offices might have several of each, especially the keypunches. It was common to have a dozen or more female keypunch operators in a large office. The automatics made use of a new “45-column” card, $7\frac{1}{2} \times 3\frac{1}{4}$ inches in size, on which it was possible to store up to 45 digits of numerical information. This card became the industry standard for the next twenty years.

With the introduction of the automatic machines, TMC raced ahead. By 1908 Hollerith had thirty customers. During the next few years the company grew at a rate of over 20 percent every half-year. By 1911 it had approximately a hundred customers and had completely made the transition to becoming an office-machine company. A contemporary American journalist captured exactly the extent of the booming punched-card business:

The system is used in factories of all sorts, in steel mills, by insurance companies, by electric light and traction and telephone companies, by wholesale merchandise establishments and department stores, by textile mills, automobile companies, numerous railroads, municipalities and state governments. It is used for compiling labor costs, efficiency records, distribution of sales, internal requisitions for supplies and materials, production statistics, day and piece work. It is used for analyzing risks in life, fire and casualty insurance, for plant expenditures and sales of service, by public service corporations, for distributing sales and cost figures as to salesmen, department, customer, location, commodity, method of sale, and in numerous other ways. The cards, besides furnishing the basis for regular current reports, provide also for all special reports and make it possible to obtain them in a mere fraction of the time otherwise required.²⁰

All of these punched-card installations lay at the heart of sophisticated information systems. Hollerith abhorred what he perceived as the sharp selling practices of the typewriter and adding-machine companies, and he personally (or, to an increasing degree, one of his assistants) worked closely with customers to integrate the punched-card machines into the overall information system.

By 1911 Herman Hollerith was the well-to-do owner of a successful business, but he was fifty-one and his health was failing. He had always refused to sell the business in the past, or even to share the running of it; but his doctor's advice and generous financial terms finally overcame his resistance.

An offer to take over TMC came from one of the leading business promoters of the day, Charles R. Flint, the so-called Father of Trusts. Flint was probably the most celebrated exponent of the business merger. He proposed merging TMC with two other firms – the Computing Scale Company, a manufacturer of shopkeeper's scales, and the International Time Recording Company, which produced automatic recording equipment for clocking employees in and out of the workplace. Each of these companies was to contribute one word to the name of a new holding company, the Computing-Tabulating-Recording Company, or C-T-R.

In July 1911, TMC was sold for a sum of \$2.3 million, of which Hollerith personally received \$1.2 million. Now a very wealthy man, Hollerith went into semiretirement, although, never quite able to let go, he remained a technical consultant for another decade but made no further important contributions to punched-card machinery. Hollerith lived to see C-T-R become IBM in 1924, but he died on the eve of the stock market crash of 1929 – and so never lived to see the Great Depression, nor how IBM survived it and became a business-machine legend.

There were three reasons for the ultimate success of IBM: the development of a sales organization based on that of NCR, the "rent-and-refill" nature of the punched-card machine business, and technical innovation. All of these were intimately bound up with the appointment of Thomas J. Watson Sr. as C-T-R's general manager in 1914.

After being fired from NCR by Patterson, Watson was not short of offers. However, he was keen to be more than a paid manager and wanted to secure a lucrative, profit-sharing deal. He therefore decided to accept an offer from Charles Flint to become general manager of C-T-R. Although C-T-R was a minute organization compared with NCR, Watson understood – far more than Flint or Hollerith – the potential of the Hollerith patents. He therefore negotiated a small base salary plus a commission of 5 percent of the profits of the company. If C-T-R grew in the way that Watson intended, it would eventually make him the highest-paid businessman in America (which in time he became). Meanwhile the NCR antitrust case in which Watson was embroiled was at last dismissed on appeal. With the disappearance of the cloud that had been overhanging him, Watson was elevated from general manager to president of C-T-R.

Making a break with Hollerith's low-key selling methods, Watson immediately introduced wholesale the sales practices he had seen so successfully pioneered in NCR. He established sales territories, commissions, and quotas. Where NCR had its Hundred Point Club for salesmen who achieved their sales quotas, Watson introduced the One Hundred Per Cent Club in C-T-R. In a period

of about five years, Watson completely transformed the culture and prospects of C-T-R, turning it into the brightest star of the office-machine industry. By 1920 C-T-R's income had more than tripled, to \$15.9 million. Subsidiaries were opened in Canada, South America, Europe, and the Far East. In 1924 Watson renamed the company International Business Machines and announced, "Everywhere . . . there will be IBM machines in use. The sun never sets on IBM."²¹ Watson never forgot the lessons he had learned from Patterson. As a writer in *Fortune* magazine observed in 1932:

[L]ike a gifted plagiarist, a born imitator of effective practices of others, [Watson] fathomed the system and excelled over all. . . . It was Patterson's invariable habit to wear a white stiff collar and vest; the habit is rigorously observed at I.B.M. It was Patterson's custom to mount on easels gigantic scratch-pads the size of newspapers and on them to record, with a fine dramatic flourish, his nuggets of wisdom; nearly every executive office at I.B.M. has one. It was Patterson's custom to expect all employees to think, act, dress, and even eat the way he did; all ambitious I.B.M. folk respect many of Mr. Watson's predilections. It was Patterson's custom to pay big commissions to big men, to meet every depression head on by increasing his selling pressure; these are part of the I.B.M. technique.²²

The T-H-I-N-K motto was soon to be seen hanging on the wall in every office in the IBM empire.

If IBM was famous for its dynamic salesmen, it became even more celebrated for its relative immunity to business depressions. The "rent-and-refill" nature of the punched-card business made IBM less dependent on acquiring new orders than other companies. Because the punched-card machines were rented and not sold, even if IBM acquired no new customers in a bad year, its existing customers would continue to rent the machines they already had, ensuring a steady income year after year. The rental of an IBM machine repaid the manufacturing costs in about two to three years, so that after that period virtually all the income from a machine was profit. Most machines remained in service for at least seven years, often ten, and occasionally fifteen or twenty years. Watson fully understood the importance of IBM's leasing policy to its long-term development and – right into the 1950s – resisted pressure from government and business to sell IBM machines outright.

The second source of IBM's financial stability was its punched-card sales. The cards cost only a fraction of the dollar per thousand for which they were sold. A 1930s journalist explained that IBM belonged to a special class of business:

[A] type that might be called "refill" businesses. The use of its machines is followed more or less automatically and continually by the sale of cards. This type of business also has good precedents: in the Eastman Kodak Co., which

sells films to camera owners; in General Motors, whose AC spark plugs are sold to motor-car owners; in Radio Corp., whose tubes are bought by radio owners; in Gillette Razor Co., whose blades are bought by razor users.²³

In the case of razor blades and spark plugs (somewhat less in the case of photographic film and radio tubes), manufacturers had to compete with one another for the refill market. IBM, however, found itself secure in the market of tabulator cards for its machines, because the cards had to be made with great accuracy on a special paper stock that was not easily imitated. By the 1930s, IBM was selling 3 billion cards a year, which accounted for 10 percent of its income but 30–40 percent of its profits.

Technical innovation was the third factor that kept IBM at the forefront of the office-machine industry between the two world wars. In part, this was a response to competition from its major rival – the Powers Accounting Machine Company, which had been formed in 1911 by James Powers, Hollerith's successor at the Census Bureau. Almost as soon as he arrived at C-T-R, Watson saw the need for a response to the Powers printing machine and set up an experimental department similar to the inventions department that Patterson had set up at NCR, to systematically improve the Hollerith machines. The printing tabulator development was interrupted by World War I, but at the first postwar sales conference in 1919

Watson had one of the supreme moments of his career. The curtains behind the central podium were drawn back, Watson threw a switch, and the new machine standing at the centre of the stage began printing results as cards flowed through it. Salesmen stood up in the chairs and cheered, exuberant that they were going to be able to match the competition at last.²⁴

In 1927 Powers was acquired by Remington Rand, which had strong organizational capabilities, and the competitive threat to IBM increased significantly. The following year IBM brought out an 80-column card that had nearly twice the capacity of the 45-column card that both companies were then selling. Remington Rand countered two years later with a 90-column card. And so it went. This pattern of development, each manufacturer leapfrogging the other, characterized the industry both in the United States and in the rest of the developed world – where IBM and Remington Rand both had subsidiaries, and where there were other European punched-card machine manufacturers.

In the early 1930s IBM leapfrogged the competition by announcing the 400 Series accounting machines, which marked a high point in interwar punched-card machine development. The most important and profitable machine of the range was the model 405 Electric Accounting Machine. IBM was soon turning out 1,500 copies of this machine a year, and it was “the most lucrative of all IBM's mechanical glories.”²⁵ The 400 Series was to remain in manufacture

until the late 1960s, when punched-card machines finally gave way completely to computers. Containing 55,000 parts (2,400 different) and 75 miles of wire, the model 405 was the result of half a century of evolution of the punched-card machine art. The machine was incomparably more complex than an ordinary accounting machine, such as the NCR Class 2000, which contained a “mere” 2,000 parts. Thus by the 1930s IBM had a significant technical lead over its competitors, which eased its transition to computers in the 1950s.

Powers always came a poor second to IBM. This remained true even after it was acquired in the Remington Rand merger in 1927. While IBM’s machines were said to be more reliable and better maintained than those of Powers, IBM’s salesmanship was overwhelmingly better. IBM’s salesmen – or, rather, systems investigators – were ambassadors who had been trained to perfection in a special school in Endicott, New York state. It was estimated in 1935 that IBM had more than 4,000 accounting machines in customer installations. This gave it a market share of 85 percent, compared with Remington Rand’s 15 percent.

Watson became famous in the annals of American business history for his nerve during the Great Depression. During this period, business-equipment manufacturers generally suffered a 50 percent decline in sales. IBM suffered a similar decline in new orders, but its income was buttressed by card sales and the rental income from machines in the field. Against the advice of his board, Watson held on to his expensively trained workforce and maintained IBM’s factories in full production, building up frighteningly high stocks of machines in readiness for the upturn. Watson’s confidence was a beacon of hope in the Depression years, and it turned him into one of the most influential businessmen in America. He became president of the American Chamber of Commerce and, later, of the International Chamber of Commerce, and he was an adviser and friend of President Franklin D. Roosevelt.

The recovery for IBM, and the nation, came in 1935 with Roosevelt’s New Deal, of which Watson was a prominent supporter. Under the Social Security Act of 1935, it became necessary for the federal government to maintain the employment records of the entire working population. IBM, with its overloaded inventories and with factories in full production, was superbly placed to benefit.

In October 1936, the federal government took delivery of a large slice of IBM’s inventory – 415 punches and accounting machines – and located the equipment in a brick-built Baltimore loft building of 120,000 square feet. In what was effectively an information production line, half a million cards were punched, sorted, and listed every day. Nothing like it had been seen since the days of the 1890 census. IBM’s income for “the world’s biggest bookkeeping job”²⁶ amounted to \$438,000 a year – which was only about 2 percent of IBM’s total income, but soon government orders would account for 10 percent. Moreover, New Deal legislation had created obligations for employers in the private sector to comply with federal demands for information on which welfare, National Recovery Act codes, and public works projects were dependent. This fueled demand further.

Between 1936 and 1940, IBM's sales nearly doubled, from \$26.2 million to \$46.2 million, and its workforce rose to 12,656. By 1940 IBM was doing more business than any other office-machine firm in the world. With sales of less than \$50 million, it was still a relatively small firm, but the future seemed limitless.

Notes to Chapter 2

The best modern account of the office-machine industry is James W. Cortada's *Before the Computer* (1993).

Good histories of the typewriter are Wilfred Beeching's *Century of the Typewriter* and Bruce Bliven's *Wonderful Writing Machine* (1954). George Engler's *The Typewriter Industry* (1969) contains a fine analysis of the industry not available elsewhere. Two excellent accounts of filing systems are JoAnne Yates's article "From Press Book and Pigeon Hole to Vertical Filing" (1982) and Craig Robertson's *The Filing Cabinet* (2021). There are no reliable in-depth histories of Remington Rand, Felt & Tarrant, or Burroughs, but Cortada's *Before the Computer* covers most of the ground.

Useful histories of National Cash Register are Isaac F. Marcossion's *Wherever Men Trade* (1945) and Stanley C. Allyn's *My Half Century with NCR* (1967). In 1984 NCR produced an excellent, though difficult to find, company history in four parts (NCR 1984). The best biography of John Patterson is Samuel Crowther's *John H. Patterson, Pioneer in Industrial Welfare* (1923). NCR's R&D operations are described in Stuart W. Leslie's *Boss Kettering* (1983). Andrea Tone's *The Business of Benevolence* (2005) gives a detailed account of NCR in the context of progressive era America.

Histories of IBM abound. The best accounts of the early period are Saul Engelbourg's *International Business Machines: A Business History* (1954 and 1976), Emerson W. Pugh's *Building IBM* (1995), and Robert Sobel's *Colossus in Transition* (1981). The official biography of Thomas J. Watson is Thomas and Marva Beldens's *The Lengthening Shadow* (1962); it is not particularly hagiographic, but William Rodgers's *Think: A Biography of the Watsons and IBM* (1969) is a useful counterbalance. The most recent biography of Watson is Kevin Maney's *The Maverick and His Machine* (2003). JoAnne Yates' *Structuring the Information Age* (2005) is an important account of how interactions between IBM and its life-insurance customers helped shape IBM's products.

Notes

1. Quoted in Bliven 1954, p. 48.
2. On the origins of QWERTY see David 1986.
3. Hoke 1990, p. 133.
4. Cortada 1993a, p. 16.

5. Quoted in Bliven 1954, p. 62.
6. Davies 1982, pp. 178–9.
7. Ibid.
8. Hessen 1973.
9. Because zeros did not need to be added into the result, no zero keys were provided.
10. From an advertisement reproduced in Turck 1921, p. 142.
11. Cortada 1993a, p. 66.
12. Quoted in Crowther 1923, p. 72.
13. Allyn 1967, p. 54.
14. Quoted in Cortada 1993a, p. 69.
15. Belden and Belden 1962, p. 18.
16. Crowther 1923, p. 156.
17. Quoted in Rodgers 1969, p. 34.
18. Anonymous 1932, p. 37.
19. Quoted in Maney 2003, p. 21.
20. Quoted in Campbell-Kelly 1989, p. 31.
21. Anon. 1932, p. 38.
22. Ibid.
23. Ibid., p. 39.
24. Belden and Belden 1962, p. 114.
25. Anonymous 1940, p. 126.
26. Quoted in Eames and Eames 1973, p. 109.

3 Babbage's Dream Comes True

IN OCTOBER 1946 the computing pioneer Leslie John Comrie reminded British readers in the science news journal *Nature* of their government's failure to support the construction of Babbage's calculating engine a century earlier:

The black mark earned by the government of the day more than a hundred years ago for its failure to see Charles Babbage's difference engine brought to a successful conclusion has still to be wiped out. It is not too much to say that it cost Britain the leading place in the art of mechanical computing.¹

The reason for Comrie's remark was the completion of the first fully automatic computing machine at Harvard University, which he hailed as "Babbage's Dream Come True." Comrie, a New Zealander who graduated from the University of Auckland in 1916, was one of many people who, over the past century, had accepted at face value Babbage's claim that the demise of his calculating engines was the fault of the British government. But the story is much more complicated than this.

Babbage's Analytical Engine

Babbage invented two calculating machines, the Difference Engine and the Analytical Engine. Of the two, the Difference Engine was, historically and technically, the less interesting – although it was the machine that came closest to actually being built. Babbage worked on his Difference Engine for approximately a decade, starting in the early 1820s. This slow progress culminated in the model Difference Engine of 1833, described in Chapter 1. Babbage displayed this model in the drawing room of his London home, where he liked to use it as a conversation piece for his fashionable soirées. The great majority of visitors were incapable of understanding what the machine could do; but one exception was Ada Byron, daughter of the poet. A young gentlewoman aged about eighteen, Ada had developed an interest in mathematics when such a calling was very unusual in a woman. She managed to charm some of England's

leading mathematicians – including Babbage and Augustus de Morgan – into guiding her studies. She later repaid Babbage by writing the best nineteenth-century account of his Analytical Engine.

Babbage never built a full-scale Difference Engine because in 1833 he abandoned it for a new invention, the Analytical Engine, the chief work on which his fame in the history of computing rests. He was at that time at the very height of his powers: Lucasian Professor of Mathematics at Cambridge University, Fellow of the Royal Society, one of the leading lights in scientific London, and author of the most influential economics book of the 1830s, the *Economy of Manufactures*.

As important as the Difference Engine had been, it was fundamentally a limited conception in that all it could do was produce mathematical tables. By contrast, the Analytical Engine would be capable of any mathematical computation. The idea of the Analytical Engine came to Babbage when he was considering how to eliminate human intervention in the Difference Engine by feeding back the results of a computation, which he referred to as the engine “eating its own tail.”² Babbage took this simple refinement of the Difference Engine, and from it evolved the design of the Analytical Engine, which embodies almost all the important functions of the modern digital computer.

The most significant concept in the Analytical Engine was the separation of arithmetic computation from the storage of numbers. In the original Difference Engine, these two functions had been intimately connected: in effect, numbers were stored in the adding mechanisms, as in an ordinary calculating machine. Originally, Babbage had been concerned by the slow speed of his adding mechanisms, which he attempted to speed up by the invention of the “anticipatory carriage” – a method of rapid adding that has its electronic equivalent in the modern computer in the so-called carry-look-ahead adder. The extreme complexity of the new mechanism was such that it would have been far too expensive to have built any more adders than necessary, so he separated the arithmetic and number-storage functions. Babbage named these two functional parts of the engine the *mill* and the *store*, respectively. This terminology was a metaphor drawn from the textile industry. Yarns were brought from the store to the mill, where they were woven into fabric, which was then sent back to the store. In the Analytical Engine, numbers would be brought from the store to the arithmetic mill for processing, and the results of the computation would be returned to the store. Thus, even as he wrestled with the complexities of the Analytical Engine, Babbage the economist was never far below the surface.

For about two years Babbage struggled with the problem of organizing the calculation – the process we now call *programming*, but for which Babbage had no word. After toying with various mechanisms, such as the pegged cylinders of barrel organs, he hit upon the Jacquard loom. Invented in 1802, the Jacquard loom had started to come into use in the English weaving and ribbon-making industries in the 1820s. It was a general-purpose device that, when instructed

by specially punched cards, could weave infinite varieties of pattern. Babbage envisaged just such an arrangement for his Analytical Engine.

Meanwhile, the funding difficulties of the original Difference Engine were still not resolved, and Babbage was asked by the Duke of Wellington – then prime minister – to prepare a written statement. It was in this statement, dated 23 December 1834, that Babbage first alluded to the Analytical Engine – “a totally new engine possessing much more extensive powers.”³ Babbage was possibly aware that raising the subject of the Analytical Engine would weaken his case, but he claimed in his statement that he was merely doing the honorable thing by mentioning the new engine, so that “you may have fairly before you all the circumstances of the case.” Nonetheless, the clear messages the government read into the statement were, first, that it should abandon the Difference Engine – and the £17,470 it had already spent on it – in favor of Babbage’s new vision; and, second, that Babbage was more interested in building the engine than in making the tables that were the government’s primary interest and for which a working Difference Engine was all that was needed. As a result, the government’s confidence in the project was fatally undermined, and no decision on the engines was to be forthcoming for several years.

Between 1834 and 1846, without any outside financial support, Babbage elaborated the design of the Analytical Engine. There was no question at this time of his attempting construction of the machine without government finance, so it was largely a paper exercise – eventually amounting to several thousand manuscript pages. During this period of intense creative activity, Babbage essentially lost sight of the objective of his engines and they became an end in themselves. Thus, whereas one can see the original table-making Difference Engine as completely at one with the economic need for nautical tables, in the case of the Analytical Engine these issues were entirely forgotten. The Analytical Engine served no real purpose that the Difference Engine could not have satisfied – and it was an open question as to whether the Difference Engine itself was more effective than the more mundane team of human computers used by de Prony in the 1790s. The fact is that, apart from Babbage himself, almost no one wanted or needed the Analytical Engine.

By 1840 Babbage was understandably in low spirits from the rebuffs he had received from the British government. He was therefore delighted when he was invited to give a lecture on his engines to the Italian Scientific Academy in Turin. The invitation reinforced his delusion of being a prophet without honor in his own country. The visit to Turin was a high point in Babbage’s life, the crowning moment of which was an audience with King Victor Emmanuel, who took a very active interest in scientific matters. The contrast with his reception in England could not have been more marked.

In Turin, Babbage encouraged a young Italian mathematician, Lieutenant Luigi Menabrea, to write an account of the Analytical Engine, which was subsequently published in French in 1842. (Babbage chose well in Menabrea: he later

rose to become prime minister of Italy.) Back in England, Babbage encouraged Ada – who was now in her late twenties and married to the Earl of Lovelace – to translate the Menabrea article into English. However, she went far beyond a mere translation of the article, adding lengthy notes of her own that almost quadrupled the length of the original. Her *Sketch of the Analytical Engine* was the only detailed account published in Babbage's lifetime – indeed, until the 1980s. Lovelace had a poetic turn of phrase, which Babbage never had, and this enabled her to evoke some of the mystery of the Analytical Engine that must have appeared truly remarkable to the Victorian mind:

The distinctive characteristic of the Analytical Engine, and that which has rendered it possible to endow mechanism with such extensive faculties as bid fair to make this engine the executive right hand of abstract algebra, is the introduction into it of the principle which Jacquard devised for regulating, by means of punched cards, the most complicated patterns in the fabrication of brocaded stuffs. . . . We may say most aptly that the Analytical Engine *weaves algebraical patterns* just as the Jacquard loom weaves flowers and leaves.⁴

This image of weaving mathematics was an appealing metaphor among the first computer programmers in the 1940s and 1950s.

One should note, however, that the extent of Lovelace's intellectual contribution to the *Sketch* has been overstated. She has been pronounced the world's first programmer and even had a programming language (Ada) named in her honor. Later scholarship has shown that most of the technical content and all of the programs in the *Sketch* were Babbage's work. But even if the *Sketch* were based almost entirely on Babbage's ideas, there is no question that Ada Lovelace provided its voice. Her role as the prime expositor of the Analytical Engine was of enormous importance to Babbage, and he described her, without any trace of condescension, as his "dear and much admired Interpreter."⁵

Shortly after his return from Italy, Babbage once again began to negotiate with the British government for funding of the Analytical Engine. By this time there had been a change of government, and a new prime minister, Robert Peel, was in office. Peel sought the best advice he could get, including that of the astronomer royal, who declared Babbage's machine to be "worthless."⁶ It was never put this baldly to Babbage, but the uncompromising message he received from the government was that he had failed to deliver on his earlier promise and that the government did not intend to throw good money after bad. Today, one can well understand the government's point of view, but one must always be careful in judging Babbage by later standards. Today, a researcher would expect to demonstrate a measure of success and achievement before asking for further funding. Babbage's view, however, was that once he had hit upon the Analytical Engine, it was a pointless distraction to press ahead with the inferior Difference Engine merely to demonstrate concrete results.

By 1846 Babbage had gone as far as he was able with his work on the Analytical Engine. For the next two years he returned to the original Difference Engine project. He could now take advantage of a quarter of a century of improvements in engineering practice and the many design simplifications he had developed for the Analytical Engine. He produced complete plans for what he called Difference Engine Number 2.

The government's complete lack of interest in these plans finally brought home to Babbage that all prospects of funding his engines were gone forever. The demise of his cherished project embittered him, although he had too much English reserve to express his disappointment directly. Instead, he turned to sweeping and sarcastic criticism of the English hostility toward innovation:

Propose to an Englishman any principle, or any instrument, however admirable, and you will observe that the whole effort of the English mind is directed to find a difficulty, a defect, or an impossibility in it. If you speak to him of a machine for peeling a potato, he will pronounce it impossible: if you peel a potato with it before his eyes, he will declare it useless, because it will not slice a pineapple.⁷

Probably no other statement from Babbage so completely expresses his contempt for the bureaucratic mind.

As Babbage neared his mid-fifties he was an increasingly spent and irrelevant force in the scientific world. He published the odd pamphlet and amused himself with semi-recreational excursions into cryptography and miscellaneous inventions, some of which bordered on the eccentric. For example, he flirted with the idea of building a tic-tac-toe playing machine in order to raise funds for building the Analytical Engine.

One of the few bright spots that cheered Babbage's existence in the last twenty years of his life was the successful development of the Scheutz Difference Engine.⁸ In 1834 Dionysius Lardner, a well-known science lecturer and popularizer, had written an article in the *Edinburgh Review* describing Babbage's Difference Engine. This article was read by a Swedish printer, Georg Scheutz, and his son Edvard, who began to build a difference engine. After a heroic engineering effort of nearly twenty years that rivalled Babbage's own, they completed a full-scale engine, of which Babbage learned in 1852. The engine was exhibited at the Paris Exposition in 1855. Babbage visited the Exposition and was gratified to see the engine win the Gold Medal in its class. The following year it was purchased by the Dudley Observatory in Albany, New York state, for \$5,000. In Great Britain, the government provided £1,200 to have a copy made in order to compute a new set of life-insurance tables.

Perhaps the most significant fact about the Scheutz Difference Engine was that it cost just £1,200 – a fraction of what Babbage had spent. But inexpensiveness came at the cost of a lack of mechanical perfection; the Scheutz engine was

not a particularly reliable machine and had to be adjusted quite frequently. Some other difference engines were built during the next half-century, but all of them were one-of-a-kind machines that did not change the course of computing. As a result, difference engines were largely abandoned and table-makers went back to using teams of human computers and conventional desk calculating machines.

Charles Babbage was never quite able to give up on his calculating engines, though. In 1856, after a gap of approximately ten years, he started work on the Analytical Engine again. Then in his mid-sixties, he realized that there was no serious prospect that it would ever be built. Right up to the end of his life he continued to work on the engine, almost entirely in isolation, filling notebook after notebook with impenetrable scribbles. He died on 18 October 1871, in his eightieth year, with no machine completed.

Why did Babbage fail to build any of his engines? Clearly his abrasive personality contributed to the failure, and for Babbage, the best was the enemy of the good. He also failed to understand that the British government cared not an iota about his calculating engine: the government was interested only in reducing the cost of table making, and it was immaterial whether the work was done by a calculating engine or by a team of human computers. But the primary cause of Babbage's failure was that he was pioneering in the 1820s and 1830s a digital approach to computing some fifty years before mechanical technologies had advanced sufficiently to make this relatively straightforward. If he had set out to do the same thing in the 1890s, the outcome might have been entirely different. As it was, Babbage's failure undoubtedly contributed to what L. J. Comrie called the "dark age" of digital computing. Rather than follow where Babbage had failed, scientists and engineers preferred to take a different, nondigital path — a path that involved the building of models, but which we now call analog computing.⁹

The Tide Predictor and Other Analog Computing Machines

The adjective *analog* comes from the word *analogy*. The concept behind this mode of computing is that instead of computing with numbers, one builds a physical model, or analog, of the system to be investigated. A nice example of this approach was the orrery. This was a tabletop model of the planetary system named after the British Earl of Orrery, an aristocratic devotee of these gadgets in the early eighteenth century. An orrery consisted of a series of spheres. The largest sphere at the center of the orrery represented the sun; around the sun revolved a set of spheres representing the planets. The spheres were interconnected by a system of gear wheels, so that when a handle was turned, the planets would traverse their orbits according to the laws of planetary motion. In about ten minutes of handle turning, the Earth would complete a year-long orbit of the sun.

Orreries were much used for popular scientific lectures in Babbage's time, and many beautiful instruments were constructed for discerning gentlemen of

learning and taste. Today they are highly valued museum pieces. Orreries were not much used for practical computing, however, because they were too inaccurate. Much later – in the 1920s – the principles underlying these devices were used in the planetariums seen in many science museums. In the planetarium an optical system projected the positions of the stars onto the hemispherical ceiling of a darkened lecture theater with high precision. For museum-goers of the 1920s and 1930s, the sight of the sky at night moving through the millennia was a remarkable and stirring new experience.

The most important analog computing technology of the nineteenth century, from an economic perspective, was the mechanical tide predictor. Just as the navigation tables published in the *Nautical Almanac* helped minimize dangers at sea, so tide tables were needed to minimize the dangers of a ship being grounded when entering a harbor. Tides are caused primarily by the gravitational pull of the sun and the moon upon the Earth, but they vary periodically in a very complex way as the sun and moon pursue their trajectories. Up to the 1870s the computations for tide tables were too time-consuming to be done for more than a few of the major ports of the world. In 1876 the British scientist Lord Kelvin invented a useful tide-predicting machine.

Lord Kelvin (who was William Thomson until his ennoblement) was arguably the greatest British scientist of his generation. His achievements occupy an astonishing nine pages in the *British Dictionary of National Biography*. He was famous to the general public as superintendent of the first transatlantic telegraph cable in 1866, but he also made many fundamental scientific contributions to physics, particularly radio telegraphy and wave mechanics. The tide predictor was just one of many inventions in his long life. It was an extraordinary-looking machine, consisting of a series of wires, pulleys, shafts, and gear wheels that simulated the gravitational forces on the sea and charted the level of water in a harbor as a continuous line on a paper roll. Tide tables could then be prepared by reading off the high and low points of the curve as it was traced.

Many copies of Kelvin's tide predictor were built to prepare tide tables for the thousands of ports around the world. Improved and much more accurate models continued to be made well into the 1950s, when they were finally replaced by digital computers. However, the tide predictor was a special-purpose instrument, not suitable for solving general scientific problems. This was the overriding problem with analog computing technologies – a specific problem required a specific machine.

The period between the two world wars was the heyday of analog computing. When a system could not be readily investigated mathematically, the best way forward was to build a model. And many wonderful models were built. For example, when a power dam needed to be constructed, the many equations that determined its behavior were far beyond the capability of the most powerful computing facilities then available. The solution was to build a scale model of the dam. It was then possible, for example, to measure the forces at the base of

the dam and determine the height of waves caused by the wind blowing over the water's surface. The results could then be scaled up to guide the construction of the real dam. Similar techniques were used in land-reclamation planning in the Netherlands, where a fifty-foot scale model was constructed for tidal calculations of the Zuider Zee.¹⁰

A whole new generation of analog computing machines was developed in the 1920s to help design the rapidly expanding US electrical power system. During this decade, electricity supply expanded out of the metropolitan regions into the rural areas – especially in the South for textile manufacturing, and in California to power the land irrigation needed for the growth of farming. To meet this increased demand, the giant electrical machinery firms, General Electric and Westinghouse, began supplying alternating-current generating equipment that was installed in huge power plants. These plants were then interconnected to form regional power networks.

The new power networks were not well understood mathematically – and, in any case, the equations would have been too difficult to solve. Hence power-network designers turned to the construction of analog models. These laboratory models consisted of circuits of resistors, capacitors, and inductors that emulated the electrical characteristics of the giant networks in the real world. Some of these “network analyzers” became fantastically complex. One of the most elaborate was the AC Network Analyzer, built at MIT in 1930, which was twenty feet long and occupied an entire room. But like the tide predictor, the network analyzer was a special-purpose machine. Insofar as it became possible to expand the frontiers of analog computing, this was largely due to Vannevar Bush.

Vannevar Bush (1890–1974) is one of the key figures in twentieth-century American science, and although he was not a “computer scientist,” his name will appear several times in this book. He first became interested in analog computing in 1912 when, as a student at Tufts College, he invented a machine he called a “profile tracer,” which was used to plot ground contours. Bush described this machine in his autobiography *Pieces of the Action* in his characteristically folksy way:

It consisted of an instrument box slung between two small bicycle wheels. The surveyor pushed it over a road, or across a field, and it automatically drew the profile as it went. It was sensitive and fairly accurate. If, going down a road, it ran over a manhole cover, it would duly plot the little bump. If one ran it around a field and came back to the starting point, it would show within a few inches the same elevation as that at which it had started.¹¹

Bush's homey style belied a formidable intellect and administrative ability. In some ways, his style was reminiscent of the fireside talks of President Roosevelt, of whom he was a confidant. In World War II, Bush became chief scientific adviser to Roosevelt and director of the Office of Scientific Research and Development, which coordinated research for the war effort.

Tucked away in Bush's master's thesis of 1913 is a slightly blurred photograph taken a year earlier, showing a youthful and gangly Bush pushing his profile tracer across undulating turf.¹² With its pair of small bicycle wheels and a black box filled with mechanical gadgetry, it captures perfectly the Rube Goldberg world of analog computing with its mixture of technology, gadgetry, eccentricity, inventiveness, and sheer panache.

In 1919 Bush became an instructor in electrical engineering at MIT, and the profile tracer concept lay at the core of the computing machines he went on to develop. In 1924 he had to solve a problem in electrical transmission, which took him and an assistant several months of painstaking arithmetic and graph plotting. "So some young chaps and I made a machine to do the work," Bush recalled.¹³ This machine was known as the "product integrator."

Like its predecessor, the product integrator was a one-problem machine. However, Bush's next machine, the "differential analyzer," was a powerful generalization of his earlier computing inventions that addressed not just a specific engineering problem but a whole class of engineering problems that could be specified in terms of ordinary differential equations. (These were an important class of equations that described many aspects of the physical environment involving rates of change, such as accelerating projectiles or oscillating electric currents.)

Bush's differential analyzer, built between 1928 and 1931, was not a general-purpose computer in the modern sense, but it addressed such a wide range of problems in science and engineering that it was far and away the single most important computing machine developed between the wars. Several copies of the MIT differential analyzer were built in the 1930s, at the University of Pennsylvania, at General Electric's main plant in Schenectady, New York, at the Aberdeen Proving Ground in Maryland, and abroad, at Manchester University in England and Oslo University in Norway. The possession of a differential analyzer made several of these places major centers of computing expertise during the early years of World War II – most notably the University of Pennsylvania, where the modern electronic computer was invented.

A Weather-Forecast Factory

Differential analyzers and network analyzers worked well for a limited class of engineering problems: those for which it was possible to construct a mechanical or electrical model of the physical system. But for other types of problems – table making, astronomical calculations, or weather forecasting, for example – it was necessary to work with the numbers themselves. This was the digital approach.

Given the failure of Babbage's calculating engines, the only way of doing large-scale digital computations remained using teams of human computers. While the human computers often worked with logarithmic tables or calculating machines, the organizing principle was to use human beings to control the progress of the calculation.

One of the first advocates of the team computing approach was the Englishman Lewis Fry Richardson (1881–1953), who pioneered numerical meteorology – the application of numerical techniques to weather forecasting.

At that time, weather forecasting was more an art than a science. Making a weather forecast involved plotting temperature and pressure contours on a weather chart and predicting – on the basis of experience and intuition – how the weather conditions would evolve. Richardson became convinced that forecasting tomorrow's weather on the basis of past experience – in the belief that history would repeat itself – was fundamentally unscientific. He believed that the only reliable predictor of the weather would be the mathematical analysis of the existing weather system.

In 1913 he was appointed director of the Eskdalemuir Observatory of the British Meteorological Office, in a remote part of Scotland – the perfect place to develop his theories of weather forecasting. For the next three years he wrote the first draft of his classic book, *Weather Forecasting by Numerical Process*. But Europe was at war and Richardson, although a pacifist, felt obliged to leave the safety of his distant Scottish outpost and join in the war effort by entering the Friends Ambulance Unit. Somehow, amid the carnage of war, he found the time in the intervals between transporting wounded soldiers from the front to test out his theories by calculating an actual weather forecast.

Weather prediction by Richardson's method involved an enormous amount of computation. As a test case, he took a set of known weather conditions for a day in 1910 and proceeded to make a weather forecast for six hours later, which he was then able to verify against the actual conditions recorded. The calculation took him six weeks, working in an office that was nothing more than a "heap of hay in a cold rest billet."¹⁴ He estimated that if he had thirty-two human computers, instead of just himself, he would have been able to keep pace with the weather.

At the end of the war in 1918, Richardson returned to his work at the Meteorological Office and completed his book, which was published in 1922. In it he described one of the most extraordinary visions in the history of computing: a global weather-forecast factory. He envisaged having weather balloons spaced two hundred kilometers apart around the globe that would take readings of wind speed, pressure, temperature, and other meteorological variables. This data would then be processed to make reliable weather forecasts. Richardson estimated that he would need sixty-four thousand human computers to "race the weather for the whole globe." As he described his "fantasy" weather-forecast factory:

Imagine a large hall like a theatre, except that the circles and galleries go right round through the space usually occupied by the stage. The walls of this chamber are painted to form a map of the globe. The ceiling represents the north polar regions, England is in the gallery, the tropics in the upper circle, Australia on the dress circle and the Antarctic in the pit. A myriad computers are at work upon the weather of the part of the map where each sits, but

each computer attends only to one equation or part of an equation. The work of each region is coordinated by an official of higher rank. Numerous little “night signs” display the instantaneous values so that neighbouring computers can read them. Each number is thus displayed in three adjacent zones so as to maintain communication to the North and South on the map. From the floor of the pit a tall pillar rises to half the height of the hall. It carries a large pulpit on its top. In this sits the man in charge of the whole theatre; he is surrounded by several assistants and messengers. One of his duties is to maintain a uniform speed of progress in all parts of the globe. In this respect he is like the conductor of an orchestra in which the instruments are slide-rules and calculating machines. But instead of waving a baton he turns a beam of rosy light upon any region that is running ahead of the rest, and a beam of blue light upon those who are behind.

Richardson went on to explain how, as soon as forecasts had been made, they would be telegraphed by clerks to the four corners of the Earth. In the basement of the forecasting factory there would be a research department constantly at work trying to improve the numerical techniques. Outside, to complete his Elysian fantasy, Richardson imagined that the factory would be set amid “playing fields, houses, mountains and lakes, for it was thought that those who compute the weather should breathe of it freely.”

Richardson’s ideas about weather prediction were well received, but his weather-forecasting factory fantasy, published in 1922, was never taken seriously—and possibly he never intended that it should be. Nonetheless, he estimated that reliable weather forecasts would save the British economy £100 million a year, and even a weather-forecasting factory on the scale he proposed would have cost very much less than this to run.¹⁵

But whether or not Richardson was serious, he never had the opportunity to pursue the idea. Even before the book was published, he had decided to withdraw from the world of weather forecasting because the British Meteorological Office had been taken over by the Air Ministry. As a Quaker, Richardson was unwilling to work directly for the armed services, so he resigned. He never worked on numerical meteorology again, but instead devoted himself to the mathematics of conflict and became a pioneer in peace studies. Because of the computational demands of numerical meteorology, it remained a backwater until the arrival of electronic computers after World War II, when both numerical meteorology and Richardson’s reputation were resurrected. When Richardson died in 1953, it was clear that the science of numerical weather forecasting was going to succeed.

The Scientific Computing Service

Richardson had designed his weather-forecasting factory around a team of human computers because it was the only practical digital computing technique

he knew about, based on what he had learned of the working methods of the Nautical Almanac Office. In fact, computation for the *Nautical Almanac* had changed remarkably little since Babbage's time. The *Almanac* was still computed using logarithms by freelance computers, most of whom were retired members of staff. All this was soon to change, however, with the appointment of Leslie John Comrie (1893–1950), who would transform the approach to computing both in Britain and the United States.

Comrie did graduate work in astronomy at Cambridge University and then obtained a position at the Royal Observatory in Greenwich, where he developed a lifelong passion for computing. For centuries, astronomers had made the heaviest use of scientific calculation of any of the sciences, so Comrie was in exactly the right environment for a person with his peculiar passion.

He was soon elected director of the computing section of the British Astronomical Association, where he organized a team of twenty-four human computers to produce astronomical tables. After completing his PhD in 1923, he spent a couple of years in the United States, first at Swarthmore College and then at Northwestern University, where he taught some of the first courses in practical computing. In 1925 he returned to England to take up a permanent position with the Nautical Almanac Office.

Comrie was determined to revolutionize computing methods at the Nautical Almanac Office. Instead of using freelance computers, with their high degree of scientific training, he decided to systematize the work and make use of ordinary clerical labor and standard calculating machines. Almost all of Comrie's human computers were young, unmarried women with just a basic knowledge of commercial arithmetic.

Comrie's great insight was to realize that one did not need special-purpose machines such as differential analyzers; he thought computing was primarily a question of organization. For most calculations, he found that his "calculating girls" equipped with ordinary commercial calculating machines did the job perfectly well. Soon the Nautical Almanac Office was fitted out with Comptometers, Burroughs adding machines, and NCR accounting machines. Inside the Nautical Almanac Office, at first glance, the scene could easily have been mistaken for that in any ordinary commercial office. This was appropriate, for the Nautical Almanac Office was simply processing data that happened to be scientific rather than commercial.

Perhaps Comrie's major achievement at the Nautical Almanac Office was to bring punched-card machines – and therefore, indirectly, IBM – into the world of scientific computing. One of the most important and arduous computing jobs in the office was the annual production of a table of the positions of the moon, which was extensively used for navigation until recent times. Calculating this table kept two human computers occupied year-round. Comrie showed considerable entrepreneurial flair in convincing his stuffy, government-funded employers to let him rent the punched-card machines, which were very costly compared

with ordinary calculating machines. Before he could produce a single number, it was first necessary to transfer a large volume of tables, *Brown's Tables of the Moon*, onto half a million punched cards. Once this had been done, however, Comrie's punched-card machines churned out – in just seven months – enough tables to satisfy the *Nautical Almanac* for the next twenty years, and at a small fraction of the cost of computing them by hand. While Comrie was producing these tables he was visited by Professor E. W. Brown of Harvard University, the famous astronomer and the compiler of *Brown's Tables of the Moon*. Brown took away with him to the United States – and to IBM – the idea of using punched-card machines in scientific computation.

In 1930 Comrie's achievements were recognized by his promotion to director of the Nautical Almanac Office. From this position he became famous as the world's leading expert on computing, and he was a consultant to many companies and scientific projects. However, after a few years he became increasingly frustrated with the bureaucracy of the Nautical Almanac Office, which resisted change and was unwilling to invest money to advance the state of the art of computing. In 1937 he took the unprecedented step of setting up in business as a commercial operation, the Scientific Computing Service Limited. It was the world's first for-profit calculating agency.

Comrie had perfect timing in launching his enterprise on the eve of World War II. He liked to boast that within three hours of Britain declaring war on Germany, he had secured a War Office contract to produce gunnery tables. He began with a staff of sixteen human computers, most of them young women. Comrie had a considerable flair for publicity and, as one headline in the popular magazine *Illustrated* put it, his “girls do the world's hardest sums.”¹⁶

Comrie's work was just the tip of the iceberg of human computing during the war. In the United States, particularly, there were several computing facilities that each made use of as many as a hundred human computers. The computers were mostly young female office workers called into the war effort, who had discovered in themselves a talent for computing. They were paid little more than the secretaries and typists from whose ranks they came; their work was “vital to the entire research effort, but remained largely invisible. Their names never appeared on the reports.”¹⁷ When the war was over, these sisters of Rosie the Riveter returned to civilian life. Thus American society never had to confront the problem of what to do with the hundreds of human computers who would otherwise have soon been rendered obsolete by electronic computers.

The Harvard Mark I

The fact that there were so many human-computing organizations in existence is indicative of the demand for computing in the years up to World War II and during the war itself. In the period 1935–1945, just as there was a boom in human-computing organizations, there was at the same time a boom in the invention of

one-of-a-kind digital computing machines. There were at least ten such machines constructed during this period, not only by government organizations but also by industrial research laboratories such as those of AT&T and RCA, as well as the technical departments of office-machine companies such as Remington Rand, NCR, and IBM. Machines were built in Europe, too – in England and Germany.

These machines were typically used for table making, ballistics calculations, and code-breaking. By far the best known, and the earliest, of these machines was the IBM Automatic Sequence Controlled Calculator. Known more commonly as the Harvard Mark I, this machine was constructed by IBM for Harvard University during 1937–1943. The Harvard Mark I is important not least because it reveals how, as early as the 1930s, IBM was becoming aware of the convergence of calculating and office-machine technologies.

IBM's initial interest in digital computing dates from Professor Brown's visit to Comrie in the late 1920s. In 1929 Thomas Watson Sr., who was a benefactor of Columbia University, had endowed the university with a statistical laboratory and equipped it with standard IBM machines. One of Brown's most gifted students, Wallace J. Eckert (1902–1971), joined the laboratory almost as soon as it was opened. Learning of Comrie's work from his former supervisor, Eckert persuaded IBM to donate more punched-card machines in order to establish a real computing bureau. Watson agreed to supply the machines for reasons that were partly altruistic, but also because he saw in Eckert and the Columbia University laboratory a valuable listening post. After the war Eckert and Columbia University were to play a key role in IBM's transition to computing.¹⁸

Before the war, however, the chief beneficiary of IBM's computing largess was Howard Hathaway Aiken (1900–1973), a graduate student at Harvard University. Aiken was thirty-three years of age when he embarked on research toward a PhD at Harvard, but he had already enjoyed a successful career as an electrical engineer and was drawn to a life of research. Aiken cut an impressive figure: "tall, intelligent, somewhat arrogant, assertive," he was "always neat and well dressed, he did not look like a student at all."¹⁹ Aiken was working on a problem concerning the design of vacuum tubes, and in order to make progress with his dissertation he needed to solve a set of nonlinear differential equations. Although analog machines such as Bush's differential analyzer at nearby MIT were suitable for ordinary differential equations, they could not generally solve nonlinear equations. It was this obstacle in the path of his research that inspired Aiken in 1936 to put forward a proposal to the faculty of Harvard's physics department to construct a large-scale digital calculator. Not surprisingly, Aiken later recalled, his idea was received with "rather limited enthusiasm . . . if not a downright antagonism."

Undaunted, Aiken took his proposal to George Chase, the chief engineer of the Monroe Calculating Machine Company, one of the leading manufacturers of adding and calculating machines.²⁰ Chase was a very well-known and respected figure in the calculating-machine industry, and he urged his company to sponsor

Aiken's machine "in spite of the large anticipated costs because he believed the experience of developing such a sophisticated calculator would be invaluable to the company's future."²¹ Unfortunately for Chase and Aiken (and in the long run, for Monroe), the company turned the proposal down. Chase, however, remained convinced of the importance of the concept and urged Aiken to approach IBM.

In the meantime, word of Aiken's calculating-machine proposal had begun to spread in the Harvard physics laboratory, and one of the technicians approached him. Aiken recalled being told by the technician that he "couldn't see why in the world I wanted to do anything like this in the Physics Laboratory, because we already had such a machine and nobody ever used it."²² Puzzled, Aiken allowed the technician to take him up to the attic, where he was shown a fragment of Babbage's calculating engine set on a wooden stand about 18 inches square.

The engine fragment had been donated to Harvard in 1886 by Babbage's son, Henry. Henry had scrapped the remains of his father's engines, but not quite everything had been consigned to the melting pot. He had held on to a number of sets of calculating wheels and had them mounted on wooden display stands. Just one set was sent abroad – to the president of Harvard College. In 1886 the college was celebrating its 250th anniversary, and Henry likely thought that it was the most fertile place in which he could plant the seed of Babbage in the New World.

This was the first time that Aiken had ever heard of Babbage, and it was probably the first time the calculating wheels had seen the light of day in decades. Aiken was captivated by the artifact. Of course, the Babbage computing wheels were no more than a fragment. They were not remotely capable of computation, and they were built in a technology that had long been superseded by the modern calculating machine. But for Aiken they were a relay baton carried from the past, and he consciously saw himself as Babbage's twentieth-century successor. To find out more about Babbage, Aiken immediately went to the Harvard University library, where he obtained a copy of Babbage's autobiography, *Passages from the Life of a Philosopher*, published in 1864. In the pages of that book, Aiken read:

If, unwarned by my example, any man shall undertake and shall succeed in really constructing an engine embodying in itself the whole of the executive department of mathematical analysis upon different principles or by simpler mechanical means, I have no fear of leaving my reputation in his charge, for he alone will be fully able to appreciate the nature of my efforts and the value of their results.²³

When Aiken came across this passage, he "felt that Babbage was addressing him personally from the past."²⁴

In order to gain sponsorship for his calculator from IBM, Aiken secured an introduction to James Bryce, IBM's chief engineer. Bryce immediately

understood the importance of Aiken's proposal. Fortunately for Aiken, Bryce enjoyed much greater confidence from his employers than did Chase at Monroe. Bryce had no trouble convincing Watson to back the project, initially to the tune of \$15,000. The following month, December 1937, Aiken polished up his proposal, in which he referred to some of Babbage's concepts – in particular, the idea of using Jacquard cards to program the machine – and sent it to IBM. This proposal is the earliest surviving document about the Mark I,²⁵ and how much Aiken derived from Charles Babbage must remain something of an open question. In terms of design and technology, the answer is clearly very little; but in terms of a sense of destiny, Aiken probably derived a great deal more.

At this point Aiken's proposal barely constituted a computer design. Indeed, it really amounted to a general set of functional requirements. He had specified that the machine should be "fully automatic in its operation once a process is established" and that it should have a "control to route the flow of numbers through the machine."²⁶

At Bryce's suggestion, Aiken attended an IBM training school, where he learned how to use IBM machines and obtained a thorough understanding of the capabilities and limitations of the technology. Then, early in 1938, he visited IBM's Endicott Laboratory in New York state, where he began to work with a team of Bryce's most trusted and senior engineers. Bryce placed Claire D. Lake in charge of the project. Lake was one of IBM's old-time engineer inventors and the designer of IBM's first printing tabulator in 1919. He in turn was supported by two of IBM's very best engineers.

The initial estimate of \$15,000 to build the machine was quickly raised to \$100,000, and early in 1939 the IBM board gave formal approval for the construction of an "Automatic Computing Plant."²⁷ Aiken's role was now essentially that of a consultant and, other than spending parts of the summers of 1939 and 1940 with IBM, he took little part in the detailed design and engineering of the machine. By 1941 Aiken had been enlisted by the navy, and the calculating machine had become just one of many special development projects for the armed forces within IBM. There were many delays, and it was not until January 1943 that the machine ran its first test problem.

The five-ton calculator was a monumental piece of engineering. Because all the basic calculating units had to run in mechanical synchronism, they were all lined up and driven by a fifty-foot shaft, powered by a five-horsepower electric motor, like "a nineteenth-century New England textile mill."²⁸ The machine was fifty-one feet wide but only two feet deep. Altogether it contained three-quarters of a million parts and had hundreds of miles of wiring. In operation, the machine was described by one commentator as making the sound of "a roomful of ladies knitting."²⁹ The machine could store just seventy-two numbers and was capable of three additions or subtractions a second. Multiplication took six seconds, and calculating a logarithm or a trigonometric function took over a minute.

Even by the standards of the day, these were pedestrian speeds. The significance of the Mark I lay not in its speed but in the fact that it was the first fully automatic machine to be completed. Once set in operation, the machine would run for hours or even days doing what Aiken liked to call “makin’ numbers.”³⁰ The machine was programmed by a length of paper tape some 3 inches wide on which “operation codes” were punched. Most programs were repetitive in nature—for example, computing the successive values of a mathematical table—so that the two ends of the tape could be glued together to form a loop. Short programs might contain a hundred or so operations, and the whole loop would rotate about once a minute. Thus the machine would take perhaps half a day to calculate one or two pages of a volume of mathematical tables.

Unfortunately, the calculator was incapable of making what we now call a “conditional branch”—that is, changing the progress of a program according to the results of an earlier computation. This made complex programs physically very long. So that the program tapes did not get soiled on the floor, special racks had to be built onto which a program tape could be stretched over pulleys to take up the slack. When such a program was run, the rack had to be trundled from a storeroom and docked onto the machine. If Aiken had studied Babbage’s—and especially Lovelace’s—writings more closely, he would have discovered that Babbage had already arrived at the concept of a conditional branch. In this respect, the Harvard Mark I was less impressive than Babbage’s Analytical Engine, designed a century earlier.

The Mark I began to run test problems in secret in early 1943 but was not used on production work until it was moved to the Harvard campus a year later. Even then, the machine was not used heavily until Aiken, still an officer in the Naval Warfare School, was recalled to Harvard to take charge of the machine. Aiken liked to say, “I guess I’m the only man in the world who was ever commanding officer of a computer.”³¹ Serious computing began for the Bureau of Ships in May 1944, mainly producing mathematical tables.

Back at Harvard, Aiken planned a ceremony to inaugurate the calculator in collaboration with IBM. At this stage the calculator looked very much like what it was: a long row of naked electromechanical punched-card equipment. Aiken preferred the machinery this way so that it was easy to maintain and modify the calculator. Thomas Watson, however, was anxious to secure the maximum publicity for the machine and insisted that it be fitted out in suitable IBM livery. Overruling Aiken, he commissioned the industrial designer Norman Bel Geddes to give it a more attractive face. Geddes outdid himself, producing a gleaming creation in stainless steel and polished glass.³² It was the very personification of a “giant brain.”

The IBM Automatic Sequence Controlled Calculator was officially dedicated at Harvard on 7 August 1944, some seven years after Aiken’s first contact with IBM. At the ceremony, Aiken’s arrogance overcame his judgment to such a degree that his reputation never fully recovered. First, he took credit as the sole

inventor of the calculator and refused to recognize the contribution made by IBM's engineers, who had spent years making his original proposal concrete. And even worse, from Watson's point of view, he failed to acknowledge the fact that IBM had underwritten the entire project. According to his biographers, Watson was pale and shaken with anger: "Few events in Watson's life infuriated him as much as the shadow cast on his company's achievement by that young mathematician. In time his fury cooled to resentment and a desire for revenge, a desire that did IBM good because it gave him an incentive to build something better in order to recapture the spotlight."³³ (That "something better" turned out to be a machine known as the Selective Sequence Electronic Calculator, discussed in Chapter 5.)

The dedication of the Harvard Mark I captured the imagination of the public to an extraordinary extent and gave headline writers a field day. *American Weekly* called it "Harvard's Robot Super-Brain," while *Popular Science Monthly* declared "Robot Mathematician Knows All the Answers."³⁴ After the dedication and the press coverage, there was intense interest in the Harvard Mark I from scientific workers and engineers wanting to use it. This prompted Aiken and his staff to produce a five-hundred-page *Manual of Operation of the Automatic Sequence Controlled Calculator*, which was effectively the first book on automatic digital computing ever published. Back in England, Comrie wrote a review of it in *Nature*, in which he made his celebrated remark about the black mark earned by the British government for failing to construct Babbage's original calculating engine. If Comrie remained annoyed with the British government of the previous century, he was at least as annoyed at Aiken's lack of grace in not acknowledging IBM's contribution: "One notes with astonishment, however, the significant omission of 'IBM' in the title and in Prof. Aiken's preface."³⁵ Comrie well understood the role played by IBM and its engineers in making Aiken's original proposal a reality.

Although the Harvard Mark I was a milestone in the history of computing, it had a short heyday before being eclipsed by electronic machines, which operated much faster because they had no moving parts. This was true of virtually all the electromechanical computers built in the 1930s and 1940s.

The Harvard Mark I, however, was a fertile training ground for early computer pioneers, such as Grace Murray Hopper. Hopper later became a pioneer in computer programming. She was a gifted speaker and tale spinner. One of her anecdotes concerned a day in 1945 when a moth was trapped in the Mark II (Harvard's next automatic calculator) and the machine "conked out"³⁶ – she liked to call this the first computer bug.

The Harvard Mark I was an icon of the computer age – the first fully automatic computer to come into operation. The significance of this event was widely appreciated by scientific commentators, and the machine also had an emotional appeal as a final vindication of Babbage's life. In 1864 Babbage had written: "Half a century may probably elapse before anyone without those aids

which I leave behind me, will attempt so unpromising a task.”³⁷ Even Babbage had underestimated just how long it would take.

Theoretical Developments and Alan Turing

Independent of the computing activities described in this chapter, there were theoretical developments in mathematical logic that would later have a major impact on the progress of computer science as an academic discipline. The immediate effect of these mathematical developments on practical computer construction was slight, however – not least because very few people involved in building computers in the 1930s and 1940s were aware of developments in mathematical logic.

The individuals most closely associated with these theoretical developments were the young British mathematician Alan Mathison Turing and the American mathematician Alonzo Church. Both were trying to answer a question that had been posed by the German mathematician David Hilbert in 1928 concerning the foundations of mathematics. This question, known as the *Entscheidungsproblem*, asked: “Did there exist a definite method or process by which all mathematical questions could be decided?”³⁸ Turing and Church came up with equivalent results in the mid-1930s. Turing’s approach, however, was one of startling originality.

Whereas Church relied on conventional mathematics to make his arguments, Turing used a conceptual computer, later known as the Turing Machine. The Turing Machine was a thought experiment rather than a realistic computer: the “machine” consisted of a scanning head that could read and write symbols on a tape of potentially infinite length, controlled by an “instruction table” (which we would now call a program). It was computing reduced to its simplest form. Turing also showed that his machine was “universal” in the sense that it could compute any function that it was possible to compute. Generally, people described a computer as “general purpose” if it could solve a broad range of mathematical problems. Turing made a much stronger claim: that a universal computer could tackle problems not just in mathematics but in any tractable area of human knowledge. In short, the Turing Machine embodied all the logical capabilities of a modern computer.

Alan Turing was born in 1912 and at school he was drawn to science and practical experimenting. He won a scholarship to King’s College, Cambridge University, and graduated in mathematics with the highest honors in 1934. He became a Fellow of King’s College and, in 1936, published his classic paper “On Computable Numbers with an Application to the *Entscheidungsproblem*” in which he described the Turing Machine. Turing showed that not all mathematical questions were decidable, and that one could not always determine whether or not a mathematical function was computable. For non-mathematicians this was an obscure concept to grasp, although later – after World War II – Turing

explained the idea in an article in *Science News*. Turing had a gift for easy explanation to general audiences and illustrated his argument with puzzles rather than mathematics. He explained:

If one is given a puzzle to solve one will usually, if it proves to be difficult, ask the owner whether it can be done. Such a question should have a quite definite answer, yes or no, at any rate providing the rules explaining what you are allowed to do are perfectly clear. Of course the owner of the puzzle may not know the answer. One might equally ask, "How can one tell whether a puzzle is solvable?" but this cannot be answered so straightforwardly. The fact of the matter is that there is no systematic method of testing puzzles to see whether they are solvable or not.³⁹

Turing went on to illustrate with examples of sliding block puzzles, metal disentanglement puzzles, and knot tying. It was typical of Turing to be able to express a complex mathematical argument in terms that a nonmathematician could understand. The computability of mathematical functions would later become a cornerstone of computer science theory.

Turing's growing reputation earned him a research studentship at Princeton University to study under Alonzo Church in 1937. There he encountered John von Neumann, a founding professor of the Institute for Advanced Study at Princeton, who a few years later would play a pivotal role in the invention of the modern computer. Von Neumann was deeply interested in Turing's work and invited him to stay on at the Institute. Turing, however, decided to return to Britain. World War II was looming, and he was enlisted in the code-breaking operation at Bletchley Park, a former mansion some 80 miles north of London. There he played a vital part in using computing machinery to break Nazi codes. Bletchley Park was led by a cadre of outstanding administrators, "men of the professor type,"⁴⁰ and staffed by several thousand machine operators, clerks, and messengers. Even among men of the professor type, Turing stood out for the originality of his mind but also for his social clumsiness: "famous as 'Prof'" he was "shabby, nail-bitten, tieless, sometimes halting in speech and awkward of manner." After the war he was a pioneer in early British electronic computer development and the application of computers to mathematical biology. Turing was a practicing homosexual at a time when this was illegal in Britain; in 1952 he was convicted of a criminal offense and sentenced to hormone therapy to "cure" his sexual deviance. Two years later he was found dead of cyanide poisoning and a verdict of suicide was recorded. Turing never lived to see the unfolding of the computer age, but by the late 1950s his role as a founder of computer science theory had become widely appreciated in the world of computing. In 1965 the United States' professional society of computing, the Association for Computing Machinery, established the A. M. Turing Award in his honor; it is regarded as the Nobel Prize of computing. In 2010, when Turing's heroic role in wartime

code-breaking had also become publicly acclaimed (it is believed that his work shortened the war in Europe by many months), British Prime Minister Gordon Brown made an official apology for his treatment by the state. In March 2021 the Governor of the Bank of England unveiled the design of a new £50 banknote featuring an image of Turing.

It is likely that Turing's work significantly influenced von Neumann and, hence, the invention of the modern electronic computer.

Notes to Chapter 3

Michael R. Williams, *A History of Computing Technology* (1997), and William F. Aspray, ed., *Computing Before Computers* (1990), both give a good general overview of the developments described in this chapter. Alan Bromley's article "Charles Babbage's Analytical Engine, 1838" (1982) is the most complete (and most technically demanding) account of Babbage's great computing machine, whereas Bruce Collier and James MacLachlan's *Charles Babbage and the Engines of Perfection* (1998) is a very accessible account aimed at youngsters but perfect for mature readers, too. Sidney Padua's award-winning *The Thrilling Adventures of Lovelace and Babbage* is in a category of its own and cannot be too highly recommended. Bromley's contributed chapter "Analog Computing" in Aspray's *Computing Before Computers* is a concise historical overview of analog computing. More recent, in-depth treatments are James Small's *The Analogue Alternative: The Electronic Analogue Computer in Britain and the USA, 1930–1975* (2001) and Charles Care's *Technology for Modelling* (2010). Frederik Nebeker's *Calculating the Weather* (1995) is an excellent account of numerical meteorology. Comrie's Scientific Computing Service is described in Mary Croarken's *Early Scientific Computing in Britain* (1990). David Griers's *When Computers Were Human* (2005) is the definitive account of human computers in the United States.

This chapter tracks the development of automatic computing through the contributions of a number of pivotal figures: Charles Babbage, L. J. Comrie, Lewis Fry Richardson, Lord Kelvin, Vannevar Bush, Howard Aiken, and Alan Turing. Of these, all but Comrie have standard biographies. They are Anthony Hyman's *Charles Babbage: Pioneer of the Computer* (1984), Oliver Ashford's *Prophet – or Professor? The Life and Work of Lewis Fry Richardson* (1985), Crosbie Smith and Norton Wise's *Energy and Empire: A Biographical Study of Lord Kelvin* (1989), G. Pascal Zachary's *Endless Frontier: Vannevar Bush, Engineer of the American Century* (1997), I. Bernard Cohen's *Howard Aiken: Portrait of a Computer Pioneer* (1999), and Andrew Hodges's *Alan Turing: The Enigma* (1988). A vignette of Comrie is given by Mary Croarken in her article "L. J. Comrie: A Forgotten Figure in the History of Numerical Computation" (2000), while his milieu is well

described in her *Early Scientific Computing in Britain* (1990). Kurt Beyer's *Grace Hopper and the Invention of the Information Age* (2009) has the most complete account of her "bug."

Notes

1. Comrie 1946, p. 567.
2. Quoted in Bromley 1990a, p. 75.
3. Babbage 1834, p. 6.
4. Lovelace 1843, p. 121.
5. Toole 1992, pp. 236–9.
6. Quoted in Campbell-Kelly's introduction to Babbage 1994, p. 28.
7. Babbage 1852, p. 41.
8. For a history of the Scheutz engine, see Lindgren 1990.
9. For an excellent short history of analog computing, including information on orreries and Kelvin's tide predictor, see Bromley 1990b.
10. Van den Ende 1992, 1994.
11. Bush 1970, pp. 55–6.
12. The photograph is reproduced in Zachary 1997, facing p. 248, and in Wildes and Lindgren 1986, p. 86.
13. Bush 1970, p. 161.
14. Richardson 1922, pp. 219–20.
15. Ashford 1985, p. 89.
16. *Illustrated* headline ("Girls Do World's Hardest Sums"), 10 January 1942.
17. Ceruzzi 1991, p. 239.
18. See Brennan 1971.
19. Cohen 1998, pp. 22, 30, 66.
20. See I. B. Cohen's introduction to Chase 1980.
21. Pugh 1995, p. 73.
22. Cohen 1988, p. 179.
23. Babbage 1864, p. 450, 1994, p. 338.
24. Bernstein 1981, p. 62.
25. Aiken 1937.
26. *Ibid.*, p. 201.
27. Bashe et al. 1986, p. 26.
28. Ceruzzi's introduction to 1985 reprint of Aiken et al. 1946, p. xxv.
29. Bernstein 1981, p. 64.
30. Hopper 1981, p. 286.
31. Cohen's foreword to the 1985 reprint of Aiken et al. 1946, p. xiii.
32. *Ibid.*
33. Belden and Belden 1962, pp. 260–1.
34. Quoted in Eames and Eames 1973, p. 123, and Ceruzzi's introduction to the 1985 reprint of Aiken et al. 1946, p. xxxi.
35. Comrie 1946, p. 517.
36. Quoted in Beyer 2009, p. 64.
37. Babbage 1864, p. 450, 1994, p. 338.
38. Hodges 2004.
39. Turing 1954, p. 7.
40. Hodges 2004.



Taylor & Francis

Taylor & Francis Group

<http://taylorandfrancis.com>

Part Two

Creating the Computer



Taylor & Francis

Taylor & Francis Group

<http://taylorandfrancis.com>

4 **Inventing the Computer**

WORLD WAR II was a scientific war: its outcome was determined largely by the effective deployment of scientific research and technical developments. The best-known wartime scientific program was the Manhattan Project at Los Alamos to develop the atomic bomb. Another major program of the same scale and importance as atomic weapons was radar, in which the Radiation Laboratory at MIT played a major role. It has been said that although the bomb ended the war, radar won it.

Emphasis on these major programs can overshadow the rich tapestry of the scientific war effort. One of the threads running through this tapestry was the need for mathematical computation. For the atomic bomb, for example, massive computations had to be performed to perfect the explosive lens that assembled a critical mass of plutonium. At the outbreak of war, the only computing technologies available were analog machines such as differential analyzers, primitive digital technologies such as punched-card installations, and teams of human computers equipped with desktop calculating machines. Even relatively slow one-of-a-kind mechanical computers such as the Harvard Mark I lay some years in the future.

The scientific war effort in the United States was administered by the Office of Scientific Research and Development (OSRD). This organization was headed by Vannevar Bush, the former professor of electrical engineering at MIT and inventor of the differential analyzer. Although Bush had developed analog computing machines in the 1930s, by the outbreak of war he had ceased to have any active interest in computing, although he understood its importance in scientific research.

As it happened, the invention of the modern electronic computer occurred in an obscure laboratory, and it was not until very late in the war that it even came to Bush's notice. Thus, Bush's participation was entirely administrative, and one of his key contributions was to propose an Act of Congress by which the research laboratories of the Navy and War Departments were united with the hundreds of civil research establishments under the National Defense Research Committee (NDRC), which would "coordinate, supervise, and conduct scientific

research on the problems underlying the development, production, and use of mechanisms and devices of warfare, except scientific research on the problems of flight.”¹

The NDRC consisted of twelve leading members of the scientific community who had the authority and confidence to get business done. Bush allowed research projects to percolate up from the bottom as well as being directed from the top. He later wrote that “most of the worthwhile programs of NDRC originated at the grass roots, in the sections where civilians who had specialized intensely met with military officers who knew the problem in the field.”² Once a project had been selected, the NDRC strove to bring it to fruition. As Bush recalled:

When once a project got batted into [a] form which the section would approve, with the object clearly defined, the research men selected, a location found where they could best work, and so on, prompt action followed. Within a week NDRC could review the project. The next day the director could authorize, the business office could send out a letter of intent, and the actual work could start. In fact it often got going, especially when a group program was supplemented, before there was any formal authorization.

This description conveys well the way the first successful electronic computer in the United States grew out of an obscure project in the Moore School of Electrical Engineering at the University of Pennsylvania.

The Moore School

Computing was not performed for its own sake, but always as a means to an end. In the case of the Moore School, the end in view was ballistics computations for the Aberdeen Proving Ground in Maryland, which was responsible for commissioning armaments for the US Army. The proving ground was a large tract of land on the Chesapeake Bay, where newly developed artillery and munitions could be test-fired and calibrated. The Aberdeen Proving Ground lay about 60 miles to the southwest of Philadelphia, and the journey between it and the Moore School was easily made by railroad in a little over an hour.

In 1935, long before the outbreak of war, a research division was established at the proving ground, which acquired one of the first copies of the Bush Differential Analyzer. The laboratory had been founded during World War I and was formally named the Ballistics Research Laboratory (BRL) in 1938. Its main activity was mathematical ballistics – calculating the trajectories of weapons fired through the air or water – and it initially had a staff of about thirty people. When hostilities began in Europe in the 1930s, the scale of operations at the BRL escalated. When the United States entered the war, top-flight scientists in mathematics, physics, astrophysics, astronomy, and physical chemistry were

brought in, as well as a much larger number of enlisted junior scientists who assisted them. Among the latter was a young mathematician named Herman H. Goldstine, who was later to play an important role in the development of the modern computer.

What brought the BRL and the Moore School of Electrical Engineering together was the fact that the latter also had a Bush Differential Analyzer. The BRL used the Moore School's differential analyzer for its growing backlog of ballistics calculations and, as a result, established a close working relationship with the staff. Although the Moore School was one of the better-established electrical engineering schools in the United States, it had nowhere near the prestige or resources of MIT, which received the bulk of the wartime research projects in electronics.

In the months preceding World War II, the Moore School was placed on a war footing: the undergraduate programs were accelerated by eliminating vacations, and the school took on war training and electronics-research programs. The main training activity was the Engineering, Science, Management, War Training (ESMWT) program, which was an intensive ten-week course designed to train physicists and mathematicians for technical posts – particularly in electronics, in which there was a great labor shortage. Two of the outstanding graduates of the ESMWT program in the summer of 1941 were John W. Mauchly, a physics instructor from Ursinus College (a small liberal arts college not far from Philadelphia), who had an interest in numerical weather prediction, and Arthur W. Burks, a mathematically inclined philosopher from the University of Michigan. Instead of moving on to technical posts, they accepted invitations to stay on as instructors at the Moore School, where they would become two of the key players in the invention of the modern electronic computer. Another training program at the Moore School was organized for the BRL to train female “computers” to operate desk calculating machines. This was the first time women had been admitted to the Moore School. An undergraduate at the time recalled:

Mrs. John W. Mauchly, a professor's wife, had the job of instructing the women. Two classrooms were taken over for their training, and each girl was given a desk calculator. The girls sat at their desks for hours, pounding away at the calculators until the clickety clack merged into a symphony of metallic digits. After the girls were trained, they were put to work calculating ballistic missile firing tables. Two classrooms full of the girls doing just that, day after day, was impressive. The groundwork was being established for the invention of the computer.³

Eventually the hand-computing staff consisted of about two hundred women.

By early 1942, the Moore School was humming with computing activity. Down in the basement of the building, the differential analyzer – affectionately known as “Annie” – was in constant use. The same undergraduate recalled that

“on the very few occasions we were allowed to see Annie, we were amazed by the tremendous maze of shafts, gears, motors, servos, etc.”²⁴ In the same building, a hundred women computers were engaged in the same work, using desk calculating machines.

All this computing activity was needed to produce “firing tables” for newly developed artillery, and for older equipment being used in new theaters of war. As any novice rifleman soon discovers, one does not hit a distant target by firing directly at it. Rather, one fires slightly above it, so that the bullet flies in a parabolic trajectory – first rising into the air, reaching its maximum height midway, then falling toward the target. In the case of a World War II weapon, with a range of a mile or so, one could not possibly use guesswork and rule of thumb to aim the gun; it would have taken dozens of rounds to find the range. There were simply too many variables to take into account. The flight of the shell would be affected by the head- and cross-winds, the type of shell, the air temperature, and even local gravitational conditions. For most weapons, the gunner was provided with a firing table in the form of a pocket-sized booklet. Given the range of a target, the firing table would enable the gunner to determine the angle at which to fire the gun into the air (the elevation) and the angle to the target (the azimuth). More sophisticated weapons used a “director,” which automatically aimed the gun using data entered by the gunner. Whether the gun was aimed manually or using a director, the same kind of firing table had to be computed.

A typical firing table contained data for around 3,000 trajectories. To calculate a single one of these required the mathematical integration of an ordinary differential equation in seven variables, which took the differential analyzer ten to twenty minutes. Working flat out, a complete table would take about thirty days. Similarly, a “calculating girl” at a desk machine would take one or two days to calculate a trajectory, so that a 3,000-entry firing table would tie up a hundred-strong calculating team for perhaps a month. The lack of an effective calculating technology was thus a major bottleneck to the effective deployment of the multitude of newly developed weapons.

The Atanasoff-Berry Computer

In the summer of 1942 John Mauchly proposed the construction of an electronic computer to eliminate this bottleneck. Although he was an assistant professor with teaching responsibilities and did not have any formal involvement with the computing activity, he was married to a woman, Mary, who was an instructor of the women computers, and so was well aware of the sea of numbers in which the organization was beginning to drown. Moreover, he had a knowledge of computing because of his interest in numerical weather forecasting, a research interest that long predated his arrival at the Moore School.

Mauchly’s was by no means the only proposal for an electronic computer to appear during the war. For example, Konrad Zuse in Germany was already

secretly in the process of constructing a machine, and the code-breaking establishments of both the United States and Britain embarked on electronic computer projects at about the same time as the Moore School. But it would not be until the 1970s that the general public learned anything about these other projects; they had little impact on the development of the modern computer, and none at all on the Moore School's work. There was, however, another computer project at Iowa State University that was to play an indirect part in the invention of the computer at the Moore School. Although the Iowa State project was not secret, it was obscure, and little was known of it until the late 1960s.

The project was begun in 1937 by John Vincent Atanasoff, a professor of mathematics and physics. He enlisted the help of a graduate student, Clifford Berry, and by 1939 they had built a rudimentary electronic computing device. During the next two years they went on to build a complete machine – later known as the Atanasoff-Berry Computer (ABC). They developed several ideas that were later rediscovered in connection with electronic computers, such as binary arithmetic and electronic switching elements. In December 1940, while Mauchly was still an instructor at Ursinus College, Atanasoff attended a lecture Mauchly presented to the American Association for the Advancement of Science in which he described a primitive analog computing machine he had devised for weather prediction. Atanasoff, discovering a like-minded scientist, introduced himself and invited Mauchly to visit so as to see the ABC. Mauchly crossed the country to visit Atanasoff the following June, shortly before enrolling at the Moore School. He stayed for five days in Atanasoff's home and learned everything he could about the machine. They parted on very amicable terms.

Atanasoff, described by his first biographer as the “forgotten father of the computer,”⁵ occupies a curious place in the history of computing. According to some writers, and some legal opinions, he is the true begetter of the electronic computer; and the extent to which Mauchly drew on Atanasoff's work would be a key issue in determining the validity of the early computer patents fifteen years later. There is certainly no question that Mauchly visited Atanasoff in June 1941 and received a detailed understanding of the ABC, although Mauchly subsequently stated that he took “no ideas whatsoever”⁶ from Atanasoff's work. As for the ABC itself, in 1942, before the computer could be brought into reliable operation, Atanasoff was recruited for war research in the Naval Ordnance Laboratory. A brief effort to build computers for the navy failed, and he never resumed active research in computing after the war. It was only in the late 1960s that his early contributions became widely known.

The extent to which Mauchly drew on Atanasoff's ideas remains unknown, and the evidence is massive and conflicting. The ABC was quite modest technology, and it was not fully implemented. At the very least, we can infer that

Mauchly saw the potential significance of the ABC and that this may have led him to propose a similar, electronic solution to the BRL's computing problems.

Eckert and Mauchly

Mauchly, with a kind of missionary zeal, discussed his ideas for an electronic computing machine with any of his colleagues at the Moore School who would listen. The person most taken with these ideas was a young electronic engineer named John Presper Eckert. A twenty-two-year-old research associate who had only recently completed his master's degree, Eckert was "undoubtedly the best electronic engineer in the Moore School."⁷ He and Mauchly had already established a strong rapport by working on the differential analyzer, in which they had replaced some of the mechanical integrators with electronic amplifiers, getting it "to work about ten times faster and about ten times more accurately than it had worked before."⁸ In this work they frequently exchanged ideas about electronic computing – Mauchly invariably in the role of a conceptualizer and Eckert in that of a feet-on-the-ground engineer. Eckert immersed himself in the literature of pulse electronics and counting circuits and rapidly became an expert.

The Moore School, besides its computing work for BRL, ran a number of research and development projects for the government. One of the projects, with which Eckert was associated, was for a device called a delay-line storage system. This was completely unconnected with computing but would later prove to be a key enabling technology for the modern computer. The project had been subcontracted by MIT's Radiation Laboratory, which required some experimental work on delay-line storage for a Moving Target Indicator (MTI) device. The MTI device was a critical problem in the newly developed radar systems. Although radar enabled an operator to "see in the dark" on a cathode-ray tube (CRT) display, it suffered from the problem that it "saw" everything in its range – not just military objects but also land, water, buildings, and so on. The result was that the radar screen was cluttered with background information that made it difficult to discern military objectives. The aim of the MTI device was to discriminate between moving and stationary objects by canceling out the radar trace of the latter; this would remove much of the background clutter, allowing moving targets to stand out clearly.

The MTI device worked as follows.⁹ As a radar echo was received, it was stored and subtracted from the next incoming signal; this canceled out the radar signal reflected by objects whose position had not changed between consecutive bursts of radar energy. Moving objects, however, would not be canceled out since their position would have changed between one radar burst and the next; they would thus stand out on the display. To accomplish this, the device needed to store an electrical signal for a thousandth of a second or so. This was where the delay line came in. It made use of the fact that the velocity of sound is only

a small fraction of the velocity of light. To achieve storage, the MTI device converted the electrical signal to sound, passed it through an acoustic medium, and converted it back to an electrical signal at the other end – typically about a millisecond later. Various fluids had been tried, and at this time Eckert was experimenting with a mercury-filled steel tube.

By August 1942 Mauchly's ideas on electronic computing had sufficiently crystallized that he wrote a memorandum on "The Use of High Speed Vacuum Tube Devices for Calculating." In it he proposed an "electronic computer" that would be able to perform calculations in one hundred seconds that would take a mechanical differential analyzer fifteen to thirty minutes, and that would have taken a human computer "*at least* several hours."¹⁰ This memorandum was the real starting point for the electronic computer project. Mauchly submitted it to the director of research at the Moore School and to the Army Ordnance department, but it was ignored by both.

The person responsible for getting Mauchly's proposal taken seriously was Lieutenant Herman H. Goldstine. A mathematics PhD from the University of Chicago, Goldstine had joined the BRL in July 1942 and was initially assigned as the liaison officer with the Moore School on the project to train female computers. He had not received Mauchly's original memo, but during the fall of 1942 he and Mauchly had fairly frequent conversations on the subject of electronic computing. During this time the computation problem at BRL had begun to reach a crisis, and Goldstine reported in a memo to his commanding officer:

In addition to a staff of 176 computers in the Computing Branch, the Laboratory has a 10-integrator differential analyzer at Aberdeen and a 14-integrator one at Philadelphia, as well as a large group of IBM machines. Even with the personnel and equipment now available, it takes about three months of work on a two-shift basis to turn out the data needed to construct a director, gun sight, or firing table.¹¹

ENIAC and EDVAC: The Stored-Program Concept

By the spring of 1943 Goldstine had become convinced that Mauchly's proposal should be resubmitted, and he told Mauchly that he would use what influence he could to promote it. By this time the original copies of Mauchly's report had been lost, so a fresh version was retyped from his notes. This document, dated 2 April 1943, was to serve as the basis of a contract between the Moore School and BRL. Things now moved ahead very quickly. Goldstine had prepared the ground carefully, organizing a number of meetings in the BRL during which the requirements of an electronic calculator were hammered out. This resulted in the escalation of Eckert and Mauchly's original plan, for a machine containing 5,000 tubes and costing \$150,000, to one containing 18,000 tubes and costing \$400,000. On 9 April a formal meeting was arranged with Eckert and Mauchly,

the research director of the Moore School, Goldstine, and the director of the BRL. At this meeting the terms of a contract between the two organizations were finalized.

That same day witnessed the beginning of “Project PX” to construct this machine, known as the ENIAC (Electronic Numerical Integrator and Computer). It was Presper Eckert’s twenty-fourth birthday, and the successful engineering of the ENIAC was to be very much his achievement.

The ENIAC was not without its critics. Indeed, within the NDRC the attitude was “at best unenthusiastic and at worst hostile.”¹² The most widely voiced criticism against the project concerned the large number of vacuum tubes the machine would contain – estimated at around 18,000. It was well known that the life expectancy of a tube was around 3,000 hours, which meant, by a naive calculation, that there would be a tube failure every ten minutes. This estimate did not take account of the thousands of resistors, capacitors, and wiring terminals – all of which were potential sources of breakdown. Eckert’s critical engineering insight in building the ENIAC was to realize that, although the mean life of electronic tubes was 3,000 hours, this was only when a tube was run at its full operating voltage. If the voltage was reduced to, say, two-thirds of the nominal value, then its life was extended to tens of thousands of hours. Another insight was his realization that most tube failures occurred when tubes were switched on and warmed up. For this reason, radio transmitters of the period commonly never switched off the “heaters” of their electronic tubes. Eckert proposed to do the same for the ENIAC.

The ENIAC was many times more complex than any previous electronic system, in which a thousand tubes would have been considered unusual. Altogether, in addition to its 18,000 tubes, the ENIAC would include 70,000 resistors, 10,000 capacitors, 6,000 switches, and 1,500 relays. Eckert applied rigorous testing procedures to all the components. Whole boxes of resistors were brought in, and their tolerances were checked one by one on a specially designed test rig. The best resistors were kept to one side for the most critical parts of the machine while others were used in less sensitive parts. On other occasions whole boxes of components would be rejected and shipped back to the manufacturer.

The ENIAC was Eckert and Mauchly’s project. Eckert, with his brittle and occasionally irascible personality, was “the consummate engineer,” while Mauchly, always quiet, academic, and laid back, was “the visionary.”¹³ The physical construction of the ENIAC was undoubtedly Eckert’s achievement. He showed an extraordinary confidence for a young engineer in his mid-twenties:

Eckert’s standards were the highest, his energies almost limitless, his ingenuity remarkable, and his intelligence extraordinary. From start to finish it was he who gave the project its integrity and ensured its success. This is of course

not to say that the ENIAC development was a one-man show. It was most clearly not. But it was Eckert's omnipresence that drove everything forward at whatever cost to humans including himself.¹⁴

Eckert recruited to the project the best graduating Moore School students, and by the autumn of 1943 he had built up a team of a dozen young engineers. In a large backroom of the Moore School, the individual units of the ENIAC began to take shape. Engineers were each assigned a space at a worktable that ran around the room, while assemblers and wiremen occupied the central floor space.

After the construction of the ENIAC had been on-going for several months, it became clear that there were some serious defects in the design. Perhaps the most serious problem was the time it would take to reprogram the machine when one problem was completed and another was to be run. Machines such as the Harvard Mark I were programmed using punched cards or paper tape, so that the program could be changed as easily as fitting a new roll onto a player piano. This was possible only because the Harvard machine was electromechanical and its speed (three operations per second) was well matched to the speed at which instructions could be read from a paper tape. But the ENIAC would perform five thousand operations per second, and it was infeasible to use a card or tape reader at that kind of speed. Instead, Eckert and Mauchly decided that the machine would have to be specially wired-up for a specific problem. At first glance, when programmed, the ENIAC would look rather like a telephone exchange, with hundreds of patch cords connecting up the different units of the machine to route electrical signals from one point to another. It would take hours, perhaps days, to change the program – but Eckert and Mauchly could see no alternative.

During the early summer of 1944, when the ENIAC had been under construction for about eighteen months, Goldstine had a chance encounter with John von Neumann. Von Neumann was the youngest member of the Institute for Advanced Study in Princeton, where he was a colleague of Einstein and other eminent mathematicians and physicists. Unlike most of the other people involved in the early development of computers, von Neumann had already established a world-class scientific reputation (for providing the mathematical foundations of quantum mechanics and other mathematical studies). Despite being from a wealthy banking family, von Neumann had suffered under the Hungarian government's persecution of Jews in the early 1930s. He had a hatred for totalitarian governments and readily volunteered to serve as a civilian scientist in the war effort. His formidable analytical mind and administrative talents were quickly recognized, and he was appointed as a consultant to several wartime projects. These included the Manhattan Project to build the atomic bomb for which von Neumann was a consultant on both its construction and its devastating final deployment. He had been making a routine consulting trip to BRL when he met Goldstine. Goldstine, who had attained only the rank of

assistant professor in civilian life, was somewhat awestruck in the presence of the famous mathematician. He recalled:

I was waiting for a train to Philadelphia on the railroad platform in Aberdeen when along came von Neumann. Prior to that time I had never met this great mathematician, but I knew much about him of course and had heard him lecture on several occasions. It was therefore with considerable temerity that I approached this world-famous figure, introduced myself, and started talking. Fortunately for me von Neumann was a warm, friendly person who did his best to make people feel relaxed in his presence. The conversation soon turned to my work. When it became clear to von Neumann that I was concerned with the development of an electronic computer capable of 333 multiplications per second, the whole atmosphere of our conversation changed from one of relaxed good humor to one more like the oral examination for the doctor's degree in mathematics.¹⁵

At this time, von Neumann was heavily involved with the Manhattan Project to build the atomic bomb. The project was, of course, highly secret and was unknown to anyone at Goldstine's level in the scientific administration. Von Neumann had become a consultant to Los Alamos in late 1943 and was working on the mathematics of implosions. In the atomic bomb it was necessary to force two hemispheres of plutonium together, uniformly from all sides, to form the critical mass that gave rise to the nuclear explosion. To prevent premature detonation, the two hemispheres had to be forced together in a few thousandths of a second using an explosive charge surrounding them. The mathematics of this implosion problem, which involved the solution of a large system of partial differential equations, was very much on von Neumann's mind when he met Goldstine that day.

How was it that von Neumann was unaware of the ENIAC, which promised to be the most important computing development of the war? The simple answer is that no one at the top – that is, in the NDRC – had sufficient faith in the project to bother bringing it to von Neumann's attention. Indeed, while he had been wrestling with the mathematics of implosion, von Neumann had written in January 1944 to Warren Weaver, head of the OSRD applied mathematics panel and a member of the NDRC, inquiring about existing computing facilities. He had been told about the Harvard Mark I, about work going on at the Bell Telephone Laboratories, and about IBM's war-related computing projects. But the ENIAC was never mentioned: Weaver apparently did not want to waste von Neumann's time on a project he judged would never amount to anything.

However, now that von Neumann had discovered the ENIAC he wanted to learn more, and Goldstine arranged for him to visit the Moore School in early August. The whole group was in awe of von Neumann's reputation, and there was great expectation of what he could bring to the project. Goldstine recalls that

Eckert had decided he would be able to tell whether von Neumann was really a genius by his first question: if it turned out to be about the logical structure of the machine, then he would be convinced. Goldstine recalled, "Of course, this *was* von Neumann's first query."¹⁶

When von Neumann arrived, the construction of the ENIAC was in high gear. The Moore School's technicians were working two shifts, from 8:30 in the morning until 12:30 at night. The design of the ENIAC had been frozen two months previously, and, as had its designers, von Neumann soon became aware of its shortcomings. He could see that while the machine would be useful for solving the ordinary differential equations used in ballistics computations, it would be far less suitable for solving his partial differential equations – largely because the total storage capacity of twenty numbers was far too little. He would need storage for hundreds if not thousands of numbers. Yet the tiny amount of storage that was provided in the ENIAC accounted for over half of its 18,000 tubes. Another major problem with the ENIAC, which was already obvious, was the highly inconvenient and laborious method of programming, which involved replugging the entire machine. In short, there were three major shortcomings of the ENIAC: too little storage, too many tubes, and too lengthy reprogramming.

Von Neumann was captivated by the logical and mathematical issues in computer design and became a consultant to the ENIAC group to try to help them resolve the machine's deficiencies and develop a new design. This new design would become known as the "stored-program computer," on which virtually all computers up to the present day have been based. All this happened in a very short space of time.

Von Neumann's arrival gave the group the confidence to submit a proposal to the BRL to develop a new, post-ENIAC machine. It helped that von Neumann was able to attend the BRL board meetings where the proposal was discussed. Approval for "Project PY" was soon forthcoming, and funding of \$105,000 was provided. From this point on, while construction of the ENIAC continued, all the important intellectual activity of the computer group revolved around the design of ENIAC's successor: the EDVAC, for Electronic Discrete Variable Automatic Computer.

The shortcomings of the ENIAC were very closely related to its limited storage. Once that had been resolved, most of the other problems would fall into place. At this point, Eckert proposed the use of a delay-line storage unit as an alternative to electronic-tube storage. He had calculated that a mercury delay line about five feet in length would produce a one-millisecond delay; assuming numbers were represented by electronic pulses of a microsecond duration, a delay line would be able to store 1,000 such pulses. By connecting the output of the delay line (using appropriate electronics) to the input, the 1,000 bits of information would be trapped indefinitely inside the delay line, providing permanent read/write storage. By comparison, the ENIAC used an electronic "flip-flop" consisting of two electronic tubes to store each pulse. The delay line would

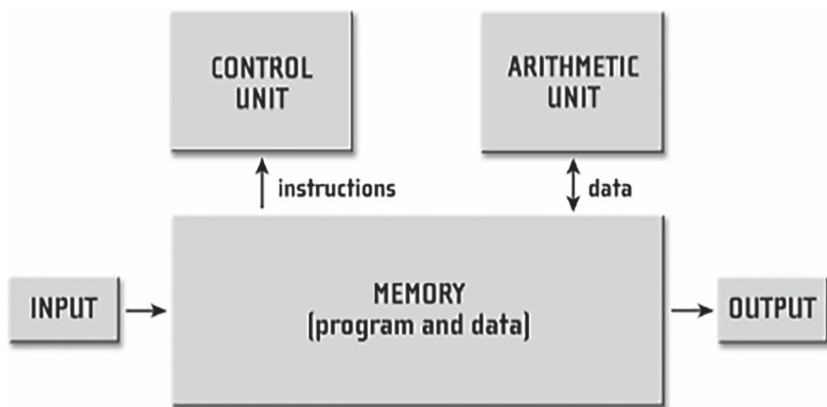


Figure 4.1 The stored-program computer of June 1945.

produce a 100-to-1 improvement in the amount of electronics used and make large amounts of storage feasible.

Eckert's "great new technological invention"¹⁷ provided the agenda for the first meetings with von Neumann. How could they use the mercury delay line to overcome the shortcomings of the ENIAC? Very early on, probably shortly after von Neumann's arrival, the crucial moment occurred when the stored-program concept was born: that *the computer's storage device would be used to hold both the instructions of a program and the numbers on which it operated*.

Goldstine likened the stored-program concept to the invention of the wheel: it was simple – once one had thought of it. This simple idea would allow for rapid program setup by enabling a program to be read into the electronic memory from punched cards or paper tape in a few seconds; it would be possible to deliver instructions to the control circuits at electronic speeds; it would provide two orders of magnitude more number storage; and it would reduce the tube count by 80 percent. But most significantly, it would enable a program to treat its own instructions as data. Initially this was done to solve a technical problem associated with handling arrays of numbers, but later it would be used to enable programs to create other programs – planting the seeds for programming languages and artificial intelligence (AI). But in August 1944, all this lay far in the future.

Several further meetings with von Neumann took place over the next few months. The other regular members of the meetings were Eckert, Mauchly, Goldstine, and Burks – the last named then working on the ENIAC and writing the technical documentation. During these meetings, Eckert and Mauchly's contributions focused largely on the delay-line research, while von Neumann, Goldstine, and Burks concentrated on the mathematical-logical structure of the machine. Thus there opened a schism in the group between the technologists

(Eckert and Mauchly) on the one side and the logicians (von Neumann, Goldstine, and Burks) on the other, which would lead to serious disagreements later on.

Of course, many details needed to be resolved before a complete blueprint of the EDVAC would be available. But at the top level, the architecture – then known as the logical design – was quickly established. Von Neumann played a key role in this aspect of the work, drawing on his training in mathematical logic and his sideline interest in the organization of the brain. During this period, the functional structure of the computer consisting of five functional parts emerged (see Figure 4.1). Von Neumann designated these five units as the central control, the central arithmetic part, the memory, and the input and output organs. The biological metaphor was very strong and, incidentally, is the origin of the term *computer memory*; this term soon displaced *storage*, which had been used since Babbage's time.

Another key decision was to use binary to represent numbers. The ENIAC had used decimal numbers, using ten flip-flops to store a single decimal digit. By using binary, the same number of flip-flops could store 10 binary digits – the equivalent of 3 decimal digits. Thus a delay line of 1,024 bits would be able to store 32 “words,” each of which could represent an instruction or a number of about 10 decimal digits. Von Neumann estimated that the EDVAC would need between 2,000 and 8,000 words of storage, which would require between 64 and 256 delay lines to implement. This would be a massive amount of equipment, but still far less than the ENIAC.

By the spring of 1945, the plans for EDVAC had evolved sufficiently that von Neumann decided to write them up. His report, entitled *A First Draft of a Report on the EDVAC*, dated 30 June 1945, was the seminal document describing the stored-program computer. It offered the complete logical formulation of the new machine and ultimately was the technological basis for the worldwide computer industry. Although the 101-page report was in draft form, with many references left incomplete, twenty-four copies were immediately distributed to people closely associated with Project PY. Von Neumann's sole authorship of the report seemed unimportant at the time, but it later led to his being given sole credit for the invention of the modern computer. Today, computer scientists routinely speak of “the von Neumann architecture” in preference to the more prosaic “stored-program concept”; this has done an injustice to von Neumann's co-inventors.

Although von Neumann's *EDVAC Report* was a masterly synthesis, it had the effect of driving the engineers and logicians further apart. For example, in the report von Neumann had pursued the biological metaphor by eliminating all the electronic circuits in favor of logical elements using the “neurons” of brain science.

This abstraction made reasoning about a computer far easier (and of course computer engineers now always work at the “gate level”). But Eckert, in particular, believed that the *EDVAC Report* skated over the really difficult engineering

problems of actually constructing the machine, especially of getting the memory to work. Somehow, Eckert felt, it was all too easy for von Neumann from his lofty height to wish away the engineering problems that would consume him in the years ahead.

Although originally intended for internal circulation to the Project PY group, the *EDVAC Report* rapidly grew famous, and copies found their way into the hands of computer builders around the world. This was to constitute publication in a legal sense, and it eliminated any possibility of getting a patent. For von Neumann and Goldstine, who wished to see the idea move into the public domain as rapidly as possible, this was a good thing; but for Eckert and Mauchly, who saw the computer as an entrepreneurial business opportunity, it was a blow that would eventually cause the group to break up.

The Engineers versus the Logicians

On 8 May 1945 Germany was defeated and the war in Europe was at an end. On 6 August the first atomic bomb was exploded over Hiroshima. A second bomb was exploded over Nagasaki three days later, and on 14 August Japan surrendered. Ironically, the ENIAC would not be completed for another six weeks, too late to help in the war effort.

Nonetheless, the fall of 1945 was an intensely exciting period, as the ENIAC neared completion. Even the once-indifferent NDRC had begun to take an interest in the machine and requested John von Neumann to prepare reports on both the ENIAC and the EDVAC. Suddenly, computers were in the air. In October the NDRC organized a secret conference at MIT “entirely without publicity and attendance by invitation only”¹⁸ at which von Neumann and his Moore School colleagues disclosed the details of ENIAC and EDVAC to the nascent American computing community. Subsequently von Neumann gave other high-level briefings and seminars on his own. Eckert was particularly irked by von Neumann’s unilateral presentations; not only did they have the effect of heaping yet more credit on von Neumann, but they also allowed him to promote his abstract-logical view of computers, which was completely at odds with Eckert’s down-to-earth engineering viewpoint.

Eckert clearly felt that von Neumann had taken away some of his own achievement. But if von Neumann’s presence overshadowed Eckert’s contribution to the invention of the stored-program computer, nothing could take away his unique achievement as chief engineer of the ENIAC.

Von Neumann, Goldstine, Burks, and some others had a different view. It appears that von Neumann had been interested not in taking personal credit for the stored-program concept, which he regarded as a team effort, but rather in seeing it disseminated as widely and quickly as possible so that it would come into use for scientific and military applications. In short, his academic interests conflicted with Eckert’s commercial interests. He noted that the government had

paid for the development of the ENIAC and the EDVAC and that therefore it was only fair that these ideas be placed in the public domain. Moreover, as a mathematician rather than an engineer, he believed that it was critically important to distinguish the logical design from the engineering implementation – probably a wise decision given the immaturity of computer engineering and the fact that it was undesirable to lock in too early on an untried technology.

In November 1945, the ENIAC at last sprang into life. It was a spectacular sight. In a basement room at the Moore School, measuring fifty by thirty feet, the units of the ENIAC were arranged in a U-shaped plan around the two longest walls and the shorter end wall. Altogether there were forty individual units, each two feet wide by two feet deep by eight feet tall. Twenty of the units were the “accumulators,” each of which contained five hundred tubes and stored a ten-digit decimal number. Hundreds of twinkling neon lights on the accumulators made for a science-fiction-like sight when the room was darkened. The other racks around the room included control units, circuits for multiplication and division, and equipment for controlling the IBM card readers and punches so that information could be fed into the machine and the results punched out. Two great twenty-horsepower blowers were installed to draw in cool air and expel the 150 kilowatts of heat that were produced by the machine.

By the spring of 1946, with the ENIAC completed and the war over, there was little to hold the Moore School computer group together. There was mounting tension between Eckert and Mauchly on the one hand, and von Neumann, Goldstine, and Burks on the other. Eckert’s resentment over von Neumann’s sole authorship of the *EDVAC Report* continued to simmer, squabbles about patents, and an inept new Moore School administration all contributed to the breakup of the group.

A new administrative regime at the Moore School took over in early 1946, when the dean, Harold Pender, appointed Irven Travis as research director. A former assistant professor at the Moore School who had risen to the rank of commander during the war, Travis had spent the war supervising contracts for the Naval Ordnance Laboratory. Travis was a highly effective administrator (he would later head up Burroughs’s computer research), but he had an abrasive personality and soon ran into conflict with Eckert and Mauchly over patents by requiring them to assign all future patent rights to the university. This Eckert and Mauchly absolutely refused to do and instead tendered their resignations.

Eckert could well afford to leave the Moore School because he was not short of offers. First, von Neumann had returned at the end of the war to the Institute for Advanced Study as a full-time faculty member, where he had decided to establish his own computer project; Goldstine and Burks had elected to join him in Princeton, and the offer was extended to Eckert. Second, and even more

tempting, was an offer from Thomas Watson Sr. at IBM. Watson had caught wind of the computer and, as a defensive research strategy, had decided to establish an electronic computer project. He made an offer to Eckert. However, Eckert was convinced that there was a basis for a profitable business making computers, and so in partnership with Mauchly he secured venture capital. In March 1946 in Philadelphia they set up in business as the Electronic Control Company to build computers.

The Moore School Lectures

Meanwhile, there was intense interest in the ENIAC from the outside world. On 16 February 1946, it had been inaugurated at the Moore School. The event attracted considerable media attention for what was still an abstruse subject. The ENIAC was featured in movie newsreels around the nation, and it received extensive coverage from the press as an “electronic brain,” including a front-page story in the *New York Times*. Apart from its gargantuan size, the feature the media found most newsworthy was its awesome ability to perform 5,000 operations in a single second. This was a thousand times faster than the Harvard Mark I, the only machine with which it could be compared and a machine that had captured the public imagination several years before. The inventors liked to point out that ENIAC could calculate the trajectory of a speeding shell faster than the shell could fly.

Computers were receiving scientific as well as public attention. Scientists were on the move to learn about wartime developments. This was especially true of the British, for whom travel to the United States had been impossible during the war. The first British visitor to the Moore School was the distinguished computing expert L. J. Comrie; he was followed by “two others connected with the British Post Office Research Station”¹⁹ (in fact, people from the still top-secret code-breaking operation). Then came visitors from Cambridge University, Manchester University, and the National Physical Laboratory – all soon to embark on computer projects. Of course, there were many more American visitors than British ones, and there were many requests from university, government, and industrial laboratories for their researchers to spend a period working at the Moore School. In this way they would be able to bring back the technology of computers to their employers. All this attention threatened to overwhelm the Moore School, which was already experiencing difficulties with the exodus of people to join von Neumann in Princeton, and to work at Eckert and Mauchly’s company.

Dean Pender, however, recognized the duty of the school to ensure that the knowledge of stored-program computing was effectively transmitted to the outside world. He decided to organize a summer school for thirty to forty invitation-only participants, one or at most two representatives from each of the major research organizations involved with computing. The Moore School Lectures,

as the course later became known, took place over eight weeks from 8 July to 31 August 1946. The list of lecturers on the course read like a *Who's Who* of computing of the day. It included luminaries such as Howard Aiken and von Neumann, who made guest appearances, as well as Eckert, Mauchly, Goldstine, Burks, and several others from the Moore School who gave the bread-and-butter lectures.

The students attending the Moore School Lectures were an eclectic bunch of established and promising young scientists, mathematicians, and engineers. For the first six weeks of the course, the students worked intensively for six days a week in the baking Philadelphia summer heat, without benefit of air-conditioning. There were three hours of lectures in the morning and an afternoon-long seminar after lunch. Although the ENIAC was well explained during the course, for security reasons only limited information was given on the EDVAC's design (because Project PY was still classified). However, security clearance was obtained toward the end of the course, and in a darkened room the students were shown slides of the EDVAC block diagrams. The students took away no materials other than their personal notes. But the design of the stored-program computer was so classically simple that there was no doubt in the minds of any of the participants as to the pattern of computer development in the foreseeable future.

Maurice Wilkes and EDSAC

It is possible to trace the links between the Moore School and virtually all the government, university, and industrial laboratories that established computer projects in America and Britain in the late 1940s. Britain was the only country other than the United States not so devastated by the war that it could establish a serious computer research program. For a very interesting reason, the first two computers to be completed were both in England, at Manchester and Cambridge universities. The reason was that none of the British projects had very much money. This forced them to keep things simple and thus to avoid the engineering setbacks that beset the grander American projects.

Britain had in fact already built up significant experience in computing during the war in connection with code-breaking at Bletchley Park. An electro-mechanical code-breaking machine – specified by Alan Turing – was operational in 1940, and this was followed by an electronic machine, the Colossus, in 1943. Although the code-breaking activity remained completely secret until the mid-1970s, it meant that there were people available who had both an appreciation of the potential for electronic computing and the technical know-how. Turing, for example, established the computing project at the National Physical Laboratory, while Max Newman, one of the prime movers of the Colossus computer, established the computer project at Manchester. Of comparable importance was the fact that Britain had extensive experience of pulse electronics and developing

storage systems for radar in both the Telecommunications Research Establishment (the British radar development laboratories) and the Admiralty.

The Manchester computer was the first to become operational. Immediately after the end of the war, Newman became professor of pure mathematics at Manchester University and secured funding to build an EDVAC-type computer. He persuaded a brilliant radar engineer, F. C. (later Sir Frederic) Williams, to move from the Telecommunications Research Establishment to take a chair in electrical engineering at Manchester.

Williams decided that the key problem in developing a computer was the memory technology. He had already developed some ideas at the Telecommunications Research Establishment in connection with the Moving Target Indicator problem, and he was familiar with the work at the MIT Radiation Laboratory and the Moore School. With a single assistant to help him, he developed a simple memory system based around a commercially available cathode-ray tube. With guidance from Newman, who “took us by the hand”²⁰ and explained the principles of the stored-program computer, Williams and his assistant built a tiny computer to test out the new memory system. There were no keyboards or printers attached to this primitive machine – the program had to be entered laboriously, one bit at a time, using a panel of press-buttons, and the results had to be read directly in binary from the face of the cathode-ray tube. Williams later recalled:

In early trials it was a dance of death leading to no useful result, and what was even worse, without yielding any clue as to what was wrong. But one day it stopped and there, shining brightly in the expected place, was the expected answer. It was a moment to remember . . . and nothing was ever the same again.²¹

The date was Monday, 21 June 1948, and the “Manchester Baby Machine” established incontrovertibly the feasibility of the stored-program computer.

As noted earlier, Comrie had been the first British visitor to the Moore School, in early 1946. Comrie had taken away with him a copy of the *EDVAC Report*, and on his return to England he visited Maurice Wilkes at Cambridge University. Wilkes, a thirty-two-year-old mathematical physicist, had recently returned from war service and was trying to reestablish the computing laboratory at Cambridge. The laboratory had been opened in 1937, but it was overtaken by the war before it could get into its stride. It was already equipped with a differential analyzer and desk calculating machines, but Wilkes recognized that he also needed to have an automatic computer. Many years later he recalled in his *Memoirs*:

In the middle of May 1946 I had a visit from L. J. Comrie who was just back from a trip to the United States. He put in my hands a document written by J. von Neumann on behalf of the group at the Moore School and entitled “Draft report on the EDVAC.” Comrie, who was spending the night at St John’s

College, obligingly let me keep it until next morning. Now, I would have been able to take a xerox copy, but there were then no office copiers in existence and so I sat up late into the night reading the report. In it, clearly laid out, were the principles on which the development of the modern digital computer was to be based: the stored program with the same store for numbers and instructions, the serial execution of instructions, and the use of binary switching circuits for computation and control. I recognized this at once as the real thing, and from that time on never had any doubt as to the way computer development would go.²²

A week or two later, while Wilkes was still pondering the significance of the *EDVAC Report*, he received a telegram from Dean Pender inviting him to attend the Moore School Lectures during July and August. There was no time to organize funds for the trip, but Wilkes decided to go out on a limb, pay his own way, and hope to get reimbursed later. In the immediate postwar period, transatlantic shipping was tightly controlled. Wilkes made his application for a berth but had heard nothing by the time the course had started. Just as he was “beginning to despair”²³ he got a phone call from a Whitehall bureaucrat and was offered a passage at the beginning of August.

After numerous hitches Wilkes finally disembarked in the United States and enrolled in the Moore School course on Monday, 18 August, with the course having just two weeks left to run. Wilkes, never short on confidence, decided that he was “not going to lose very much in consequence of having arrived late,”²⁴ since much of the first part of the course had been given over to material on numerical mathematics with which he was already familiar. As for the EDVAC, Wilkes found that the “basic principles of the stored program computer were easily grasped.”²⁵ Indeed, there was nothing more to grasp than the basic principles: all the details of the physical implementation were omitted, not least because the design of the EDVAC had progressed little beyond paper design studies. Even the basic memory technology had not yet been established. True, delay lines and cathode-ray tubes had been used for storing radar signals for a few milliseconds, but this was quite different from the bit-perfect storage of minutes’ or hours’ duration needed by digital computers.

After the end of the Moore School course, in the few days that were left to him before his departure for Britain, Wilkes visited Harvard University in Cambridge, Massachusetts. There he was received by Howard Aiken and saw both the Harvard Mark I grinding out tables and a new relay machine, the Mark II, under construction. To Wilkes, Aiken now appeared to be one of the older generation, hopelessly trying to extend the technological life of relays while resisting the move to electronics. Two miles away, at MIT, he saw the new electronic differential analyzer, an enormous machine of about 2,000 tubes, thousands of relays, and 150 motors. It was another dinosaur. Wilkes was left in no doubt that the way forward had to be the stored-program computer.

Sailing back on the *Queen Mary*, Wilkes began to sketch out the design of a “computer of modest dimensions very much along the lines of the EDVAC Proposal.”²⁶ Upon his return to Cambridge in October 1946, he was able to direct the laboratory’s modest research funds toward building a computer without the necessity for formal fundraising. From the outset, he decided that he was interested in *having* a computer, rather than in trying to advance computer engineering technology. He wanted the laboratory staff to become experts in using computers – in programming and mathematical applications – rather than in building them. In line with this highly focused approach, he decided to use delay-line storage rather than the much more difficult, though faster, CRT-based storage. This decision enabled Wilkes to call the machine the Electronic Delay Storage Automatic Calculator; the acronym EDSAC was a conscious echo of its design inspiration, the EDVAC.

During the war Wilkes had been heavily involved in radar development and pulse electronics, so the EDSAC presented him with few technical problems. Only the mercury delay-line memory was a real challenge, but he consulted a colleague who had successfully built such a device for the Admiralty during the war and who was able to specify with some precision a working design. Wilkes copied these directions exactly. This decision was very much in line with his general philosophy of quickly getting a machine up and running on which he could try out real programs instead of dreaming them up for an imaginary machine.

By February 1947, working with a couple of assistants, Wilkes had built a successful delay line and stored a bit pattern for long periods. Buoyed up by the confidence of a mercury delay-line memory successfully implemented, he now pressed on with the construction of the full machine. There were many logistical as well as design problems; for example, the supply of electronic components in postwar Britain was very uncertain and required careful planning and advanced ordering. But luck was occasionally on his side, too:

I was rung up one day by a genial benefactor in the Ministry of Supply who said that he had been charged with the disposal of surplus stocks, and what could he do for me. The result was that with the exception of a few special types we received a free gift of enough tubes to last the life of the machine.²⁷

Wilkes’s benefactor was amazed at the number of electronic tubes – several thousand – that Wilkes could make use of.

Gradually the EDSAC began to take shape. Eventually thirty-two memory delay lines were included, housed in two thermostatically controlled ovens to ensure temperature stability. The control units and arithmetic units were held in three long racks standing six feet tall, all the tubes being open to the air for maximum cooling effect. Input and output were by means of British Post Office

teletype equipment. Programs were punched on telegraph tape, and results were printed on a teletype page printer. Altogether the machine contained 3,000 tubes and consumed 30 kilowatts of electric power. It was a very large machine, but it had only one-sixth the tube count of the ENIAC and was correspondingly smaller.

In early 1949 the individual units of the EDSAC were commissioned and the system, equipped with just four temperamental mercury delay lines, was assembled. Wilkes and one of his students wrote a simple program to print a table of the squares of the integers. On 6 May 1949, a thin ribbon of paper containing the program was loaded into the computer; half a minute later the teleprinter sprang to life and began to print 1, 4, 9, 16, 25. . . . The world's first practical stored-program computer had come to life, and with it the dawn of the computer age.

Notes to Chapter 4

The best scholarly account of the Moore School developments is contained in Nancy Stern's *From ENIAC to UNIVAC: An Appraisal of the Eckert-Mauchly Computers* (1981).

Herman G. Goldstine's *The Computer: From Pascal to von Neumann* (1972) contains an excellent firsthand account of the author's involvement in the development of the ENIAC and EDVAC. A "view from below" is given in Herman Lukoff's *From Dits to Bits* (1979); the author was a junior engineer on the ENIAC and later became a senior engineer in the UNIVAC Division of Sperry Rand. Scott McCartney's *ENIAC* (1999) is an engaging read that contains biographical material on Eckert and Mauchly not found elsewhere. By far the best and most authoritative account of the ENIAC and its consequences is Thomas Haigh et al.'s *ENIAC in Action* (2016).

Atanasoff's contributions to computing are described in Alice and Arthur Burks's *The First Electronic Computer: The Atanasoff Story* (1988). Two biographies are Clark R. Mollenhoff's *Atanasoff: Forgotten Father of the Computer* (1988) and Jane Smiley's *The Man Who Invented the Computer* (2010).

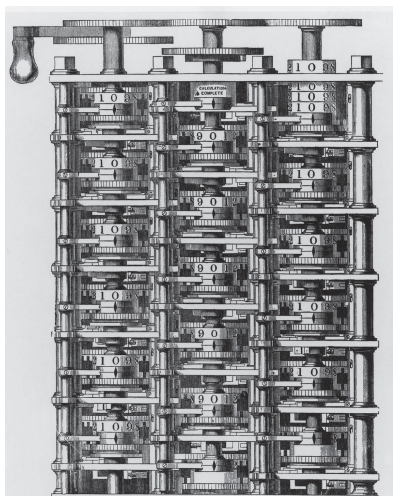
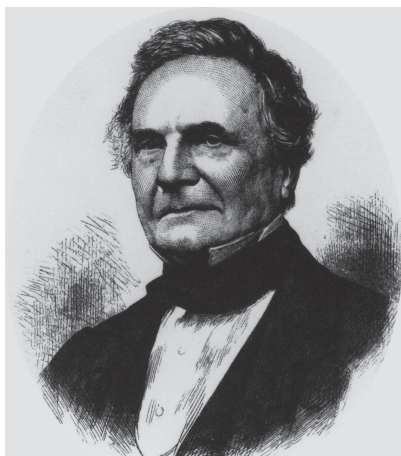
The controversy over the invention of the stored-program concept is discussed at length in both the Stern and the Goldstine books, with some bias toward Eckert-Mauchly and von Neumann, respectively. A more objective account is contained in William F. Aspray's *John von Neumann and the Origins of Modern Computing* (1990).

The background to the Moore School Lectures is given in Martin Campbell-Kelly and Michael R. Williams's *The Moore School Lectures* (1985). The best accounts of computing at Manchester University and Cambridge University are contained in, respectively, Simon H. Lavington's *Manchester University Computers* (1976) and Maurice V. Wilkes's *Memoirs of a Computer Pioneer* (1988).

Notes

1. Quoted in Baxter 1946, p. 14.
2. Bush 1970, p. 48.
3. Lukoff 1979, p. 21.
4. *Ibid.*, p. 18.
5. Mollenhoff 1988.
6. Stern 1981, p. 8.
7. Goldstine 1972, p. 149.
8. Eckert 1976, p. 8.
9. See Ridenour 1947, chap. 16.
10. Mauchly 1942.
11. Goldstine 1972, pp. 164–6.
12. Stern 1981, p. 21.
13. *Ibid.*, p. 12.
14. Goldstine 1972, p. 154.
15. *Ibid.*, p. 182.
16. *Ibid.*
17. *Ibid.*, p. 190.
18. Stern 1981, pp. 219–20.
19. Goldstine 1972, p. 217.
20. Williams 1975, p. 328.
21. *Ibid.*, p. 330.
22. Wilkes 1985, p. 108.
23. *Ibid.*, p. 116.
24. *Ibid.*, p. 119.
25. *Ibid.*, p. 120.
26. *Ibid.*, p. 127.
27. *Ibid.*, pp. 130–1.

From Babbage's Difference Engine to System/360



Impression from a woodcut of a small portion of Mr. Babbage's Difference Engine No. 1, the property of Government, at present deposited in the Museum at South Kensington.

It was commenced 1822.
This portion put together 1833.
The construction abandoned 1842.
This plate was printed June, 1862.
This portion was in the Exhibition 1862.

Figures 1, 2, and 3 In 1820 the English mathematician Charles Babbage invented the Difference Engine, the first fully automatic computing machine. Babbage's friend Ada Lovelace wrote a *Sketch of the Analytical Engine* (1843), which was the best description of the machine until recent times. In the 1970s the programming language Ada was named in Lovelace's honor. BABBAGE PORTRAIT AND DIFFERENCE ENGINE COURTESY OF CHARLES BABBAGE INSTITUTE ARCHIVES (CBIA), UNIVERSITY OF MINNESOTA; LOVELACE COURTESY OF SCIENCE MUSEUM GROUP, LONDON



Figure 4 The Central Telegraph Office, London, routed telegrams between British provincial towns. This 1874 engraving from the *Illustrated London News* shows the buzz of activity as incoming telegrams were received for onward transmission. COURTESY OF CHARLES BABBAGE INSTITUTE ARCHIVES (CBIA), UNIVERSITY OF MINNESOTA

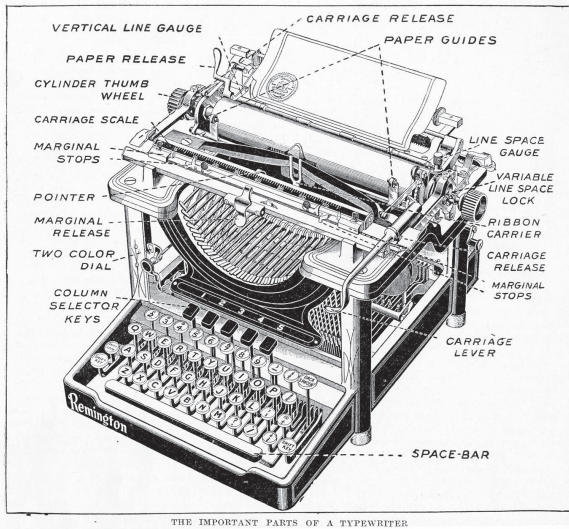


Figure 5 An early twentieth-century typewriter, showing the QWERTY keyboard and type-basket arrangement. From the *Harmsworth Self-Educator*, an educational magazine published 1905–1907. COURTESY OF CHARLES BABBAGE INSTITUTE ARCHIVES (CBIA), UNIVERSITY OF MINNESOTA

An hour's figuring—done in fifteen minutes

How the Comptometer accomplishes this remarkable saving for the Lufkin Rule Co.

Clerks are entering in the office as well as at the shop. They can be kept down to zero, depending, of course, on the amount of work. Because of its high speed and accuracy on all figures, when adding, multiplying, dividing, subtracting, the Comptometer supplies the means of increasing the productive power of each figure clerk.

In accomplishing this result, the Comptometer has an important part to play. It does not do the work, but it does the work so well that it saves time in converting errors by catching and blocking them out at the source.

"In these days of high wages, clerk's time is one of the highest-priced raw materials a manufacturer buys.

Any saving in time is well worth investigating, particularly where the saving is forty-five minutes out of every hour.

Now read what A. C. Byron, Cash Accountant of the Lufkin Rule Co., Saginaw, Mich., writes:

"Handling figures work rapidly and accurately is the lookout of most accounting departments. Our Comptometers 'cleaned the act' in our organization.

"We use two of these rapid-fire calculating machines for adding accounts, preparing factory reports and figuring costs of our 20 different departments. In comparison clerks handle the greater portion of this work and our Comptometers are constantly in the go-pumping from one desk to another, adding here, subtracting there and multiplying at the next desk, and so on.

It adds, subtracts, multiplies, divides with an accuracy that eliminates 'brain-fag' errors, as Mr. Byron puts it.

In direct action keyboard permits of machine-gun rapidity of operation. Its Controlled-key feature makes errors from fumbled key strokes impossible.

A fifteen-minute demonstration will convince you of the time-saving, error-checking efficiency of the Comptometer. Put it up to a Comptometer man to show you.

No merchant or manufacturer who has figures to deal with can afford to put off investigating the Comptometer.

These are the two Comptometers doing the work referred to above by Mr. Byron

CONTROLLED KEY
Comptometer
REG. TRADE MARK
ADDING AND CALCULATING MACHINE

If not made by Felt & Tarrant, it's not a Comptometer

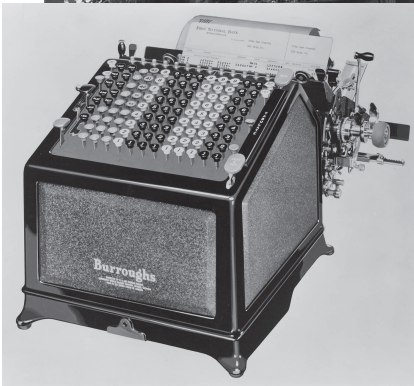
Figure 6 Comptometer advert, 1920. Note the use of all five fingers to enter numbers into the machine. Experience operators could use both hands simultaneously. COURTESY OF CHARLES BABBAGE INSTITUTE ARCHIVES (CBIA), UNIVERSITY OF MINNESOTA



Figure 7 A small insurance office, United States, late 1930s. The image shows a Smith Premier typewriter and a Comptometer in the foreground. At the rear of the picture are vertical filing cabinets of steel construction. COURTESY OF CHARLES BABBAGE INSTITUTE ARCHIVES (CBIA), UNIVERSITY OF MINNESOTA



Figure 8 The London Comptometer School in the early 1940s. Comptometer Schools were located in major cities throughout the United States and Europe. The schools had largely disappeared by the 1960s. COURTESY OF CHARLES BABBAGE INSTITUTE ARCHIVES (CBIA), UNIVERSITY OF MINNESOTA



Figures 9 and 10 The Burroughs Adding Machine Company was the leading manufacturer of adder-lister machines (*left*). These complex machines were assembled from thousands of parts and subassemblies. The factory shot (*above*) shows a mid-century assembly line in the Detroit plant. COURTESY OF CHARLES BABBAGE INSTITUTE ARCHIVES (CBIA), UNIVERSITY OF MINNESOTA



Figures 11 and 12 In 1911 Thomas J. Watson Sr. became the general manager of the Tabulating Machine Company. The photograph (*above*) shows Watson addressing a sales school. In 1924 the firm was renamed International Business Machines (IBM) and became the world's most successful office-machine company. The photograph (*below*) shows a typical punched-card office of the 1920s. PHOTO OF WATSON: REPRINT COURTESY OF IBM CORPORATION ©; PUNCHED CARD OFFICE: COURTESY OF CHARLES BABBAGE INSTITUTE ARCHIVES (CBIA), UNIVERSITY OF MINNESOTA

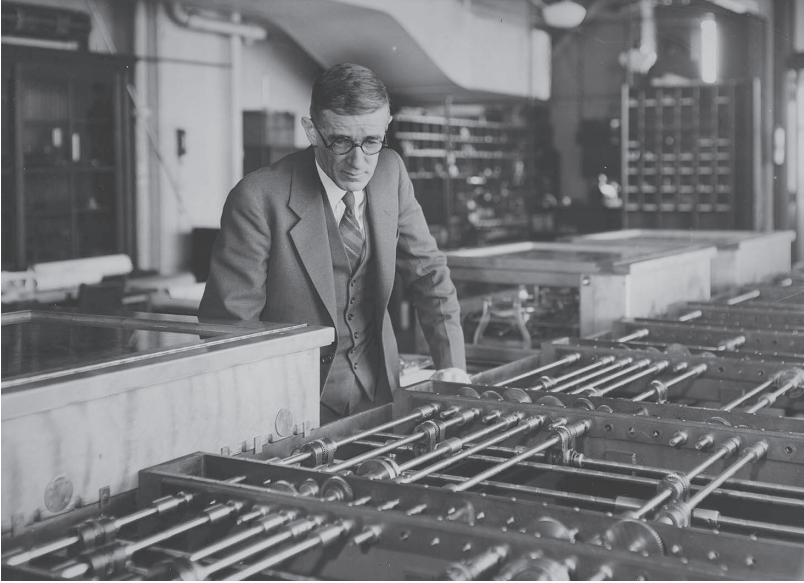
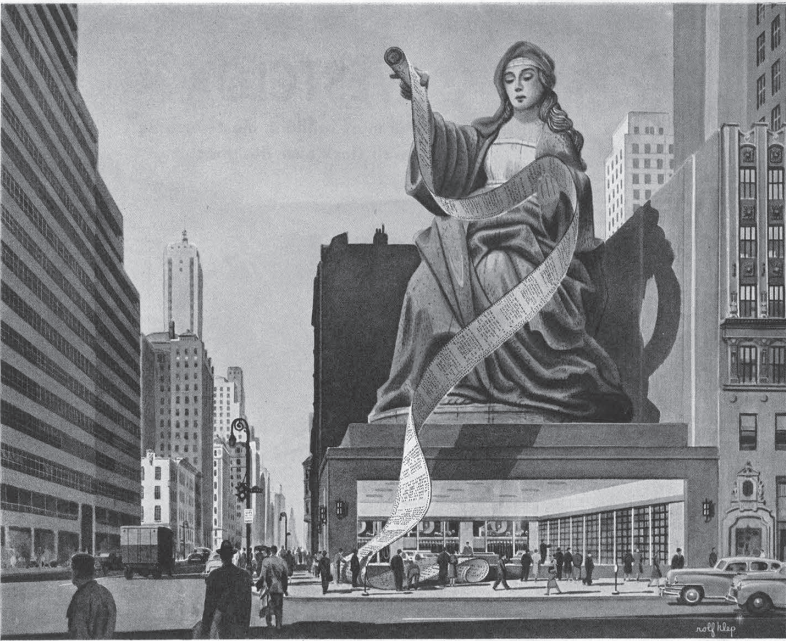


Figure 13 Shown with its inventor, Vannevar Bush, in the mid-1930s the Differential Analyzer was the most powerful analog computing machine developed between the world wars. Bush, a distinguished MIT alumnus, later became chief scientific adviser to President Roosevelt and head of civilian war research during World War II. COURTESY OF MIT MUSEUM



Figure 14 The IBM Automatic Sequence Controlled Calculator, usually known as the Harvard Mark I, was placed in operation in 1943. Weighing 5 tons and measuring 51 feet across, it could perform approximately three operations per second. REPRINT COURTESY OF IBM CORPORATION ©

THE SATURDAY EVENING POST December 16, 1950



INTERNATIONAL BUSINESS MACHINES, famous name in office equipment, builds some of the world's most complex and efficient machines. Shell Industrial Lubricants are used in many operations.

Oracle on 57th Street

Ask IBM's big computer on 57th Street in New York a question. Ask it a tough one like: "At what speed will forced vibration tear the wings from a jet plane built to fly more than 1,000 miles an hour?" You'll get the right answer *days* before scientists could compute it.

Calculating machines, the question-answering "oracles" of business, are just part of IBM's output. This firm works in a world of machinery and parts, all

carefully planned. *Planned lubrication is essential.*

Shell research technicians have collaborated with IBM engineers in helping to find the best lubricants for both general and special requirements. Shell oils and greases play an important part in assuring smooth functioning and long life for many items of equipment.

Development of better lubricants is only one way in which

Shell demonstrates leadership in the petroleum industry, and in petroleum products. Wherever you see the Shell name and trade mark, Shell Research is your guarantee of quality.



Shell Research leads to finer Products — more for your money

© 1950, SHELL OIL COMPANY

Figure 15 Completed in 1948, IBM's Selective Sequence Electronic Calculator (SSEC) was located in full view of passersby in the ground floor of its headquarters at 57th Street and Madison Avenue in Manhattan, New York City. Shell Oil, an early user, touted its scientific research credentials in this full-page advertisement in the *Saturday Evening Post* in December 1950. COURTESY OF CHARLES BABBAGE INSTITUTE ARCHIVES (CBIA), UNIVERSITY OF MINNESOTA

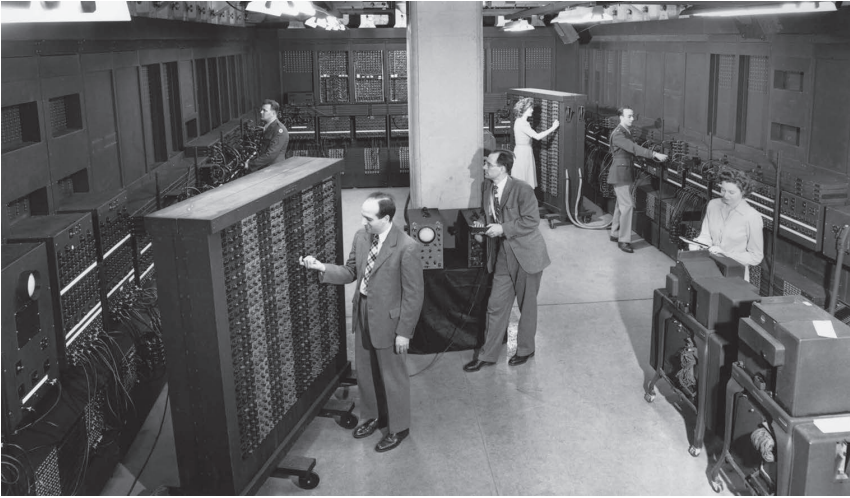


Figure 16 The ENIAC, developed during World War II, was capable of 5,000 basic arithmetic operations per second. It contained 18,000 electronic tubes, consumed 150 kilowatts of electric power, and filled a room 30 × 50 feet. The inventors of the ENIAC, John Presper Eckert and John Mauchly, are seen in the foreground (*left*) and center respectively. US ARMY PHOTO

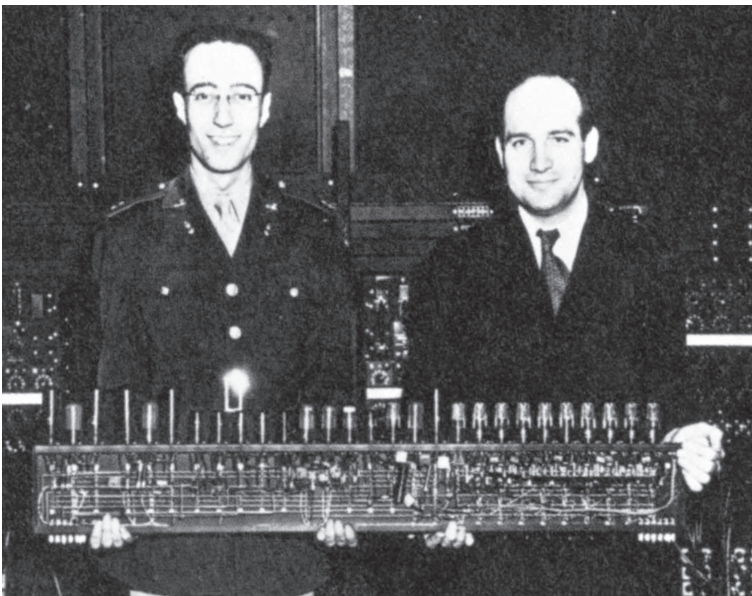


Figure 17 Herman H. Goldstine (*left*) and John Presper Eckert holding a decade counter from the ENIAC. This 22-tube unit could store just one decimal digit. US ARMY PHOTO

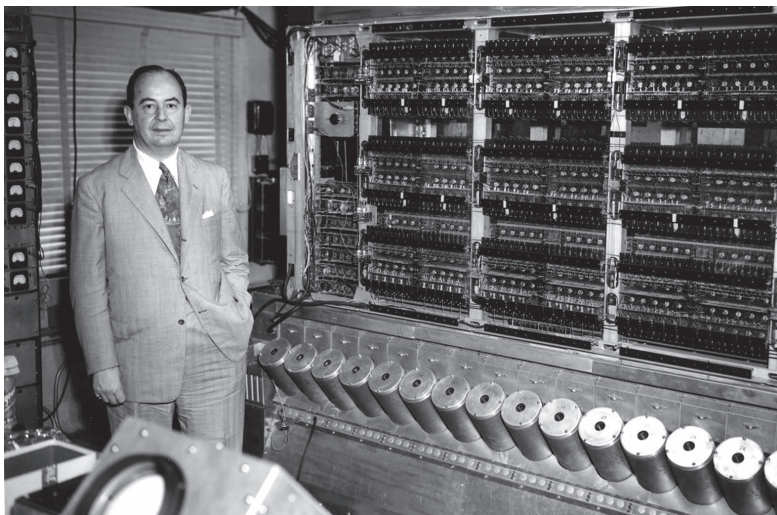


Figure 18 During 1944–1945 John von Neumann collaborated with the developers of the ENIAC to produce the design for a new computer. The stored-program computer that resulted has been the theoretical basis for almost all computers built since. The photograph shows von Neumann with the computer built in the early 1950s at the Institute for Advanced Study, Princeton. ALAN RICHARDS PHOTOGRAPHER. FROM THE SHELBY WHITE AND LEON LEVY ARCHIVES CENTER, INSTITUTE FOR ADVANCED STUDY, PRINCETON

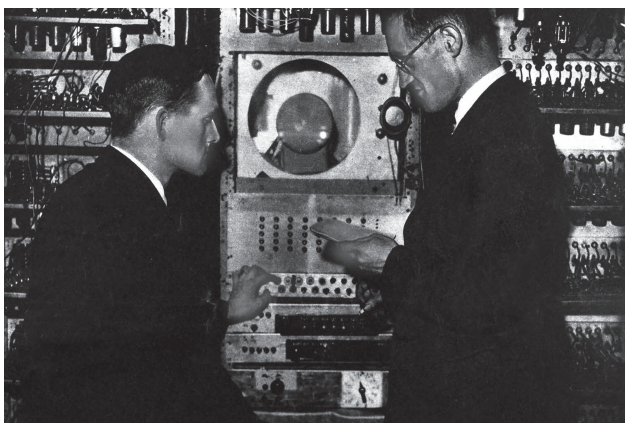


Figure 19 The first electronic stored-program computer to operate, in 1948, was the “baby machine” built at Manchester University in England. Although too small to perform realistic calculations, it established the feasibility of the stored-program concept. The photograph shows the developers Tom Kilburn (*left*) and Fredrik C. Williams at the control panel. COURTESY OF DEPARTMENT OF COMPUTER SCIENCE, MANCHESTER UNIVERSITY

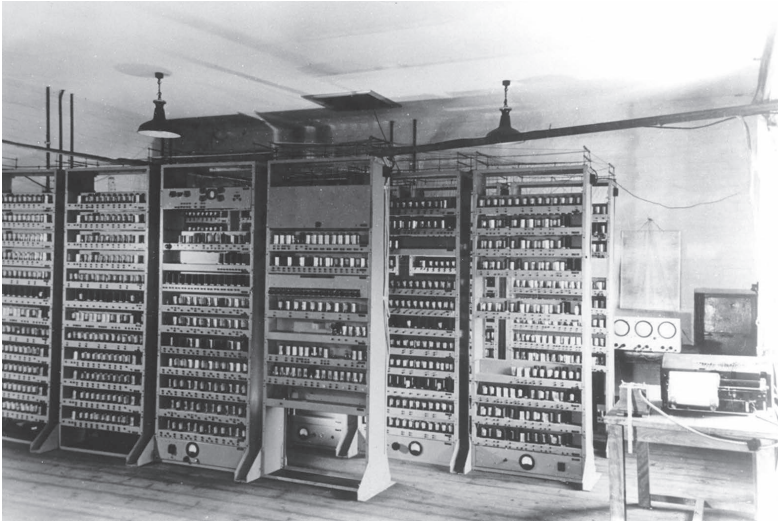


Figure 20 Completed at Cambridge University in 1949, the EDSAC was the first full-scale stored-program computer to go into regular service. COURTESY OF UNIVERSITY OF CAMBRIDGE, DEPARTMENT OF COMPUTER SCIENCE AND TECHNOLOGY

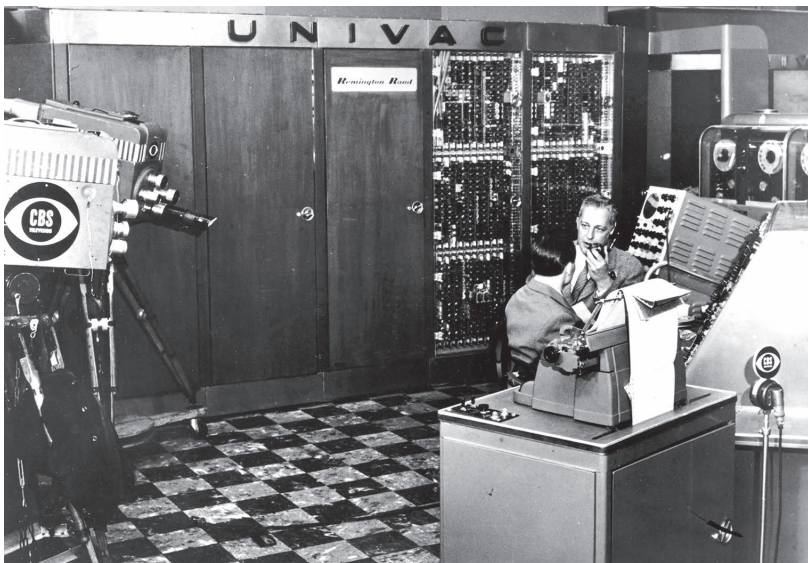


Figure 21 The UNIVAC, developed by Eckert and Mauchly, was America's first commercially manufactured electronic computer. It attracted publicity on election night 1952, when it was used to forecast the results of the presidential election. COURTESY OF HAGLEY MUSEUM AND LIBRARY

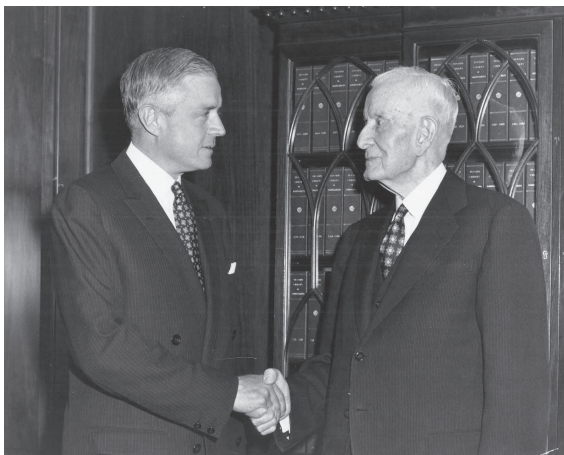


Figure 22 The old order changeth: Thomas J. Watson Sr. cedes the presidency of IBM to his son Thomas J. Watson Jr. in 1952. Under the leadership of Watson Jr., IBM became the world's leading supplier of mainframe computers and information systems. REPRINT COURTESY OF IBM CORPORATION ©



Figure 23 In 1959 IBM announced the model 1401 computer, a fast and reliable "second-generation" machine that used transistors in place of electronic tubes. REPRINT COURTESY OF IBM CORPORATION ©



Figure 24 System/360, announced in 1964, enabled IBM to dominate the computer industry well into the 1980s. REPRINT COURTESY OF IBM CORPORATION ©

5 The Computer Becomes a Business Machine

AN APOCRYPHAL TALE about Thomas J. Watson Sr., the onetime president of IBM, claims that, in the 1940s, he decided that there would not be a market for more than a handful of computers and that IBM had no place in that business. This anecdote is usually told to show that the seemingly invincible leader of IBM was just as capable of making a foolish decision as the rest of the human race. Conversely, the story portrays as heroes those few individuals, such as Eckert and Mauchly, who foresaw a big market for computers. There is perhaps some justice in this verdict of posterity regarding the conservative Watson versus the radical Eckert and Mauchly. Another way of interpreting these responses to the computer is as the rational versus the irrational, based on business acumen and past experience. Thus Watson was rational but wrong, while Eckert and Mauchly were irrational but right. It could just as easily have been the other way around – and then Watson would still have been a major business leader, but not many people would have heard of Eckert and Mauchly.

What really happened in the 1950s, as suggested by the title of this chapter, is that the computer was reconstructed – mainly by computer manufacturers and business users – to be an electronic data-processing machine rather than a mathematical instrument. Once IBM had recognized this change, in about 1951, it altered its sales forecasts, rapidly reoriented its R&D, manufacturing, and sales organizations, and used its traditional business strengths to dominate the industry within a period of five years – and dominated it for the next 40. This was a heroic achievement too, though not one of which myths are made.

In the late 1940s and early 1950s, some thirty firms entered the computer business in the United States. The only other country to develop a significant computer industry at this time was the United Kingdom, where, in an effort to capture a share of what was seen as a promising new postwar industry, about ten computer companies came into being – size for size, a comparable performance with the United States. Britain even had the world's first computer on the market. This was a machine known as the Ferranti Mark I, based on the Manchester University computer and delivered in February 1951. Unfortunately the enthusiasm for manufacturing computers in Britain was not matched with an

enthusiasm for using them among the country's old-fashioned businesses, and by the end of the decade Britain's computer industry was fighting for survival.

As for the rest of the world, all the industrially developed countries of continental Europe – Germany, France, Holland, and Italy – were so ravaged by World War II that they would not be capable of entering the computer race until the 1960s. The Eastern bloc and Japan were even slower to develop computers. Hence, throughout the 1950s, US firms had the biggest market for computers – the United States – to themselves, and from this springboard they were able to dominate the world.

There were three types of firms that entered the computer industry: electronics and control equipment manufacturers, office-machine companies, and entrepreneurial start-ups. Most of the established firms had been defense contractors during the war and in some cases afterwards. As shown in later chapters, defense spending by the US government helped nurture and sustain the electronics and computer industries. The electronics and control equipment manufacturers – including RCA, General Electric, Raytheon, Philco, Honeywell, Bendix, and several others – were the firms for which the computer was the most “natural” product. They were well accustomed to selling high-cost electronic capital goods, such as radio transmitters, radar installations, X-ray equipment, and electron microscopes. The computer, for which these firms coined the term *mainframe*, was simply another high-value item in their product line.

The second class of firms entering the computer market was the business-machine manufacturers such as IBM, Remington Rand, Burroughs, NCR, Underwood, Monroe, and Royal. They lagged the electronics firms by a couple of years because, quite rationally, they did not see the manufacture of mathematical computing instruments as being an appropriate business for them to enter. It was not until Eckert and Mauchly produced the UNIVAC computer in 1951, proving that computers had a business application, that they entered the race with conviction.

The third class of firms to enter the computer business – and in many ways the most interesting – was the entrepreneurial start-up. The period from 1948 to 1953 was a narrow window of opportunity during which it was possible to join the computer business at a comparatively low cost, certainly less than a million dollars. A decade later, because of the costs of supplying and supporting peripherals and software, one would need \$100 million to be able to compete with the established mainframe computer companies. The first entrepreneurial computer firms were Eckert and Mauchly's Electronic Control Company (set up in 1946) and Engineering Research Associates (ERA, also set up in 1946). These firms were soon taken over by Remington Rand to become its computer division: Eckert and Mauchly became famous as the pioneers of a new industry, while William Norris, one of the founders of ERA, moved on to found another company – the highly successful Control Data Corporation.

Following quickly on the heels of the Eckert-Mauchly company and ERA came a rush of computer start-ups, sometimes funded with private money, but more often the subsidiaries of established electronics and control companies. Firms emerging in the early 1950s included Electrodata, the Computer Research Corporation, Librascope, and the Laboratory for Electronics. The luckiest of these firms were taken over by office-machine companies, but most simply faded away, unable to survive in the intensely competitive environment. The story of the Eckert-Mauchly company illustrates well the precarious existence shared by virtually all the start-up companies.

“More than Optimistic”: UNIVAC and BINAC

When Eckert and Mauchly established their Electronic Control Company in March 1946, they were almost unique in seeing the potential for computers in business data processing, as opposed to science and engineering calculations. Indeed, even before the end of the war, Eckert and Mauchly had paid several visits to the Bureau of the Census in Washington, canvassing the idea of a computer to help with census data processing, though nothing came of the idea at the time. The vision of computerized data processing probably came more from Mauchly than from Eckert. Thus, at the Moore School Lectures in the summer of 1946, while Eckert had talked about the technical details of computer hardware, Mauchly was already giving a lecture on sorting and collating – topics that really belonged in the world of punched-card business machines, not mathematical computation.

When Eckert and Mauchly set out to build their business, there were no venture-capital firms of the kind we are now familiar with. They therefore decided that the most effective way to fund the company would be to obtain a firm order for their computer and use a drip feed of incremental payments to develop the machine. Those wartime visits to the Census Bureau paid off. In the spring of 1946, the Bureau of the Census agreed to buy a computer for a total purchase price of \$300,000. Eckert and Mauchly, who were hopelessly optimistic about the cost of developing their computer, thought it would cost \$400,000 to develop the machine (much the same as ENIAC), but \$300,000 was the best deal they could negotiate. They hoped to recover their losses in subsequent sales.

In fact, the computer would cost closer to a million dollars to develop. As one historian has noted, “Eckert and Mauchly were more than optimistic, they were naive.”¹ Perhaps it was a foolish way to start a business, but, like entrepreneurs before and after, Eckert and Mauchly wished away the financial problems in order to get on with their mission. If they had stopped to get financially rewarding terms, the project would likely have been strangled at birth. The contract to build an EDVAC-type machine for the Census Bureau was signed in October 1946, and the following spring the computer was officially named UNIVAC, for UNIVersal Automatic Computer.

Eckert and Mauchly started operations in downtown Philadelphia, taking the upper two stories of a men's clothing store at 1215 Walnut Street. A narrow-fronted building, though running deep, it was more than sufficient for the half dozen colleagues they had persuaded to join them from the Moore School. By the autumn of 1947, there were about twelve engineers working for the company, all engaged in the various aspects of the UNIVAC.

Eckert and Mauchly had embarked upon a project whose enormousness began to reveal itself only during this first year of the company's operation. Building the UNIVAC was much more difficult than simply putting together a number of ready-made subsystems, as would be the case when building a computer a decade later. Not even the basic memory technology had been established. Thus, one group of engineers was working on a mercury delay-line memory – and, in the process, discovering that it was one thing to develop a system for laboratory use but quite another to produce a reliable system for fuss-free use in a commercial environment.

The most ambitious feature of the UNIVAC was the use of magnetic-tape storage to replace the millions of punched cards used by the Census Bureau and other organizations and businesses. This was a revolutionary idea, which spawned several other projects. In 1947 the only commercially available tape decks were analog ones used in sound recording studios. A special digital magnetic-tape drive had to be developed so that computer data could be put onto the tape, and high-speed servomotors had to be incorporated so that the tape could be started and stopped very rapidly. Then it turned out that ordinary commercial magnetic tape, based on a plastic substrate, stretched unacceptably. The company now had to set up a small chemistry laboratory to develop tape-coating materials, and to use a metallic tape that did not stretch. Next, it was necessary to develop both a machine that would allow data to be keyed onto the tape and a printer that would list the contents of the tape. Later, machines would be needed to copy decks of cards onto magnetic tape, and vice versa. In many ways the magnetic-tape equipment for the UNIVAC was both the most ambitious and the least successful aspect of the project.

Eckert and Mauchly were now both working intensely hard for very long hours, as was every engineer in the company. The company was also beset by financial problems, and to keep the operation afloat Eckert and Mauchly needed more orders for UNIVACs and more advance payments. These orders were not easy to come by. However, a few months earlier, Mauchly had been hired by the Northrop Aircraft Corporation to consult on guidance computers for the Snark long-range guided missile that the company was developing. Northrop decided it needed a small airborne computer, and Eckert and Mauchly were invited to tender an offer for the development contract for this machine – to be called the BINAC (for BINary Automatic Computer).

The BINAC would be a very different order of machine than the UNIVAC. It would be much smaller; it would be a scientific machine, not a data processor;

and it would not use magnetic tape – perhaps the biggest task in Eckert and Mauchly's portfolio. The contract to develop the BINAC would keep the company afloat, and yet it would also detract greatly from the momentum of the UNIVAC. But there was no real choice: it was the BINAC or go out of business. In October 1947 they signed a contract to develop the BINAC for \$100,000, of which \$80,000 was paid in advance.

Despite the distraction of the BINAC, the UNIVAC remained central to the long-term ambition of the company, and the search for UNIVAC orders continued. The most promising prospect was the Prudential Insurance Company, which Mauchly had first approached in 1946 to no avail, but whose interest had now started to revive. The Prudential was a pioneer in the use of office machinery and had even developed its own punched-card system in the 1890s, long before IBM punched-card machines were commercially available; so it was no surprise that it would become an early adopter of the new computer technology.

The Prudential's computer expert was a man named Edmund C. Berkeley, who was to write the first semi-popular book on computers, *Giant Brains*, published in 1949. Early in 1947, Mauchly and Berkeley had tried to convince the Prudential board to order a UNIVAC, but the company was unwilling to risk several hundred thousand dollars to buy an unproved machine from an unproved company. However, after several months of negotiation, the Prudential agreed to pay \$20,000 for Mauchly's consulting services, with an option to buy a machine when it was developed. It wasn't much, but every little bit helped, and there was after all the distant prospect that the Prudential would eventually buy a machine.

By the end of 1947 Eckert and Mauchly's operation was spending money much faster than it was recovering it, and it needed further investment capital. In order to attract investors, they incorporated the company as the Eckert-Mauchly Computer Corporation (EMCC). Eckert and Mauchly were desperately seeking two things: investors and contracts for the UNIVAC.

In the spring of 1948, the A. C. Nielsen market research firm in Chicago agreed to buy the second UNIVAC for \$150,000, and toward the end of the year the Prudential offered to buy the third for the same price. This was a ludicrously unrealistic price tag, although, as always, Eckert and Mauchly wished away the problems.

Buoyed up by the prospect of firm orders, they recruited more engineers, technicians, and draftsmen, and the company moved to larger premises, taking the seventh and eighth floors of the Yellow Cab Building on the edge of central Philadelphia. Eckert and Mauchly were now employing forty people, including twenty engineers, and they needed an additional \$500,000 in working capital. This financial support would shortly be offered by the American Totalisator Company of Baltimore.

It so happened that Eckert and Mauchly's patent attorney was friendly with Henry Strauss, a vice president of American Totalisator. Strauss was a

remarkable inventor. In the 1920s, he had perfected and patented a totalisator machine for racetrack betting that soon dominated the market. The totalisator was, in its way, one of the most sophisticated mechanical computing systems of its era. Strauss immediately saw that the UNIVAC represented a technological breakthrough that might prove a source of future competition in the totalisator field. In August 1948 he persuaded his company to take a 40 percent shareholding in the Eckert-Mauchly Computer Corporation. Strauss, an inventor himself, had the wisdom to leave Eckert and Mauchly in full control of their company, while he would provide mature guidance by serving as chairman of the board. American Totalisator offered about \$500,000 for its share of EMCC and offered further financing through loans. It was a truly generous offer, and Eckert and Mauchly grasped it thankfully.

With financial stability at last, EMCC had turned the corner. True, virtually every deadline and schedule had been missed and the future promised nothing different, but this was to some extent to be expected in a high-tech company. The important thing was that the money from American Totalisator and the contracts with the Census Bureau, Northrop, Nielsen, and Prudential ensured that the company could expand confidently. The following spring, it moved for the third time in as many years, this time to a self-contained two-story building in North Philadelphia. A former knitting factory, when emptied of its plant, the interior was so vast that it was "difficult . . . to believe that we would ever outgrow this facility."² Situated at the foot of a hill, the area was so hot in the summer that they called it Death Valley.

As soon as they moved in, there was a burst of activity to complete the BINAC. The first American stored-program computer to operate, the BINAC was never a reliable machine. In the early summer a visiting Northrop engineer reported back that it was operating about one hour a week, "but very poorly."³ It was operating only a little more reliably when it was finally shipped off to Northrop in September 1949. It never really performed to expectation, but then few early computers did, and Northrop was happy enough to pay the outstanding \$20,000 of the \$100,000 purchase price.

With the BINAC out of the way, Eckert and Mauchly could at last focus single-mindedly on the UNIVAC, for which three more sales had been secured. EMCC was now a bustling company in its new headquarters, with 134 employees and contracts for six UNIVAC systems totaling \$1.2 million. Things had never looked brighter. A few weeks later, however, Eckert and Mauchly received the news that Henry Strauss had been killed when his privately owned twin-engine airplane crashed. Since Strauss was the prime mover behind American Totalisator's investment in EMCC, when he died so did American Totalisator's commitment to the company. It pulled out and required its loans to be repaid. EMCC was immediately thrown into financial disarray, and all the old uncertainties resurfaced. It was as though someone had turned out the light.

IBM: Evolution, Not Revolution

Meanwhile IBM, the company that was to become Eckert and Mauchly's prime competitor, was beginning to respond to the computer. As mentioned at the chapter's opening, there is a myth that IBM made a fateful decision shortly after the war not to enter the computer business, made a swift reversal of policy about 1951, and by the mid-1950s had secured a position of dominance. What is most beguiling about this myth is that it is sufficiently close to the truth that people take it at face value. Yet where it is misleading is that it portrays IBM as sleeping soundly at the start of the computer race. The truth is more complex. IBM had several electronics and computer development projects in the laboratories in the late 1940s; what delayed them being turned into products was the uncertainty of the market.

Immediately after the war, IBM (like the other accounting-machine companies such as Remington Rand, Burroughs, and NCR) was trying to respond to three critical business challenges: product obsolescence, electronics, and the computer. Product obsolescence was the most critical of these challenges. All of IBM's traditional electromechanical products were in danger of becoming obsolete because during the war most of its R&D facilities had been given over to military contracts such as the development of gun aimers and bomb sights. For the very survival of IBM's business, revamping its existing products had to be the top postwar priority.

The second challenge facing the office-machine companies was the threats and opportunities presented by electronics technology. Electronics had been greatly accelerated by the war, and IBM had itself gained some expertise by constructing code-breaking machines and radio equipment. The race was now on to incorporate electronics into the existing products. The third challenge was the stored-program computer. But in 1946, when the computer was a specialized mathematical instrument of no obvious commercial importance, it had to be the lowest priority; anything else would have been irrational. IBM institutionalized its attitude to electronics and computers through the slogan "evolution not revolution."⁴ By this, the company meant that it would incorporate electronics into existing products to make them faster, but they would not otherwise be any different. Other industry pundits likened the office-machine manufacturers' attitude toward electronics as similar to that of aircraft manufacturers toward the jet engine: the new technology would make their products faster but would not change their function.

Thomas Watson Sr., often portrayed as an electromechanical diehard, was in fact among the first to see the future potential of electronics. As early as October 1943 he had instructed his R&D chief to "find the most outstanding Professor on electronics and get him for IBM."⁵ Although cost was no object, no suitable candidate could be found because all the electronics experts were committed to the war effort at that time. Nothing more might have happened for

a few years had there not been that unpleasant incident in August 1944 when the Mark I calculator was inaugurated at Harvard University, and Howard Aiken refused to acknowledge IBM's role in inventing and building the machine.

Watson decided to slight Aiken's Mark I by building a more powerful, electronic calculator. In March 1945 IBM hired Professor Wallace Eckert (no relation to Presper) to establish the Watson Scientific Computation Laboratory at Columbia University and develop a "supercalculator" to be known as the Selective Sequence Electronic Calculator (SSEC). Watson's objective – apart from thumbing his nose at Aiken – was to ensure that IBM had in the Watson Laboratory a test bed for new ideas and devices. It was not expected that the SSEC – or any other computing machine – would develop into a salable product. The decision to build the SSEC was made a couple of months before the publication of von Neumann's *EDVAC Report*. For this reason, the SSEC was a one-of-a-kind development that never became part of the mainstream of stored-program computing.

Meanwhile, IBM's product development engineers were responding to the less glamorous challenge of incorporating electronics into the company's existing products. The first machine to receive this treatment was the Model 601 multiplying punch, first marketed in 1934. The problem with this machine was that its electromechanical multiplier could handle only about ten numbers per minute. The heart of the machine was ripped out and replaced with electronics – giving the machine a tenfold speed improvement, and it was now able to perform about one hundred multiplications per minute. Containing approximately three hundred vacuum tubes, the machine was first demonstrated at the National Business Show in New York in September 1946. It was marketed as the IBM Model 603, and about one hundred machines were produced before it was replaced two years later by the much more versatile Model 604 calculating punch. The latter electronic workhorse went on to sell a staggering 5,600 units over a ten-year period. With 1,400 tubes and a limited programming capability, the 604 provided "speed and flexibility of operation unmatched by any calculator in the market for some time."⁶ More than that, the 604 provided the heart of a powerful computing setup known as the CPC – the Card Programmed Calculator.

The CPC was developed in 1947 in a cooperative effort between IBM and one of its West Coast customers, Northrop Aircraft, the same company that had commissioned the BINAC from Eckert and Mauchly. Northrop was a heavy user of IBM equipment for calculating missile trajectories and took early delivery of a Model 603 multiplier. As a means of improving the flexibility of the 603, the machine was hooked up to a specially developed storage unit and other punched-card equipment. The result was a highly effective calculating setup capable of about one thousand operations per second. Very quickly, news of Northrop's installation began to spread, particularly among West Coast aerospace companies, and by the end of 1948 IBM had received a dozen requests for a similar setup. This was the first demonstration IBM had received that there was a significant market for scientific computing equipment.

IBM product engineers set to work to develop the CPC, in which the more powerful 604 calculating punch replaced the 603 multiplier. Customer deliveries began in late 1949. The CPC was not a true stored-program computer, but it was an effective and very reliable calculating tool in 1949 and 1950, when there were no commercially available computers on the market. Moreover, even when computers did become commercially available, the low cost and superior reliability of the CPC continued to make it the most cost-effective computing system available. Approximately seven hundred systems were delivered in the first half of the 1950s, more than the world total of stored-program computers at that date.

Commentators have often failed to realize the importance of the CPC, not least because it was called a “calculator” instead of a “computer.” Watson insisted on this terminology because he was concerned that the latter term, which had previously always referred to a human being, would raise the specter of technological unemployment. It was why he had also insisted on calling the Harvard Mark I and the SSEC “calculators.” But the fact remained that, as far as effective scientific-computing products were concerned, IBM had the lead from the very beginning.

Thus by 1949 IBM had built up an excellent R&D capability in computers. In addition to the CPC, there was the SSEC, which was completed in early 1948. Although, as noted, this was a one-of-a-kind development and not technically a stored-program computer, it was the most advanced and powerful machine available when it was completed. Watson had the machine placed on public view on the ground floor of the IBM headquarters in Manhattan. One overseas visitor wrote:

This machine was completed and put into operation by IBM at their Headquarters in New York early in 1948. It could be seen from the street by passing pedestrians who affectionately christened it “Poppa.” It was a very large machine and contained 23,000 relays and 13,000 [tubes]. . . . The machine in operation must have been the most spectacular in the world. Thousands of neon lamps flashed on and off, relays and switches buzzed away and the tape readers and punches worked continuously.⁷

IBM courted publicity about the machine. Staff writers for *The New Yorker* reported that “[p]eople who just want to look at the calculator, as we did, are always welcome,”⁸ and were themselves given a tour conducted by Robert Seiber Jr., a senior engineer. The magazine produced a wonderful vignette of the machine that reflected exactly the popular fascination with “electronic brains”:

The principal cerebral parts of the machine are tubes and wires behind glass panels, covering three walls of the room. Two hoppers, looking rather like oversized mailboxes, stand near the middle of the room. One is the “in” hopper, into which questions are inserted on punched cards or tapes; the other is

the “out” hopper, from which, if all goes well, the answer emerges. High on one wall of the room is a large sign reading “THINK,” but this admonition isn’t addressed to the calculator. . . . The calculator has tackled any number of commercial problems. When we arrived, it was just warming up for one that involved getting more oil out of oil fields. . . . As we moved along, relays began to rattle in a carefree manner and patterns of light danced across the panels. “The oil problem,” Seeber said.

The SSEC did a great deal to enhance IBM’s reputation as a leader in computer technology. There was, however, no prospect that the SSEC could become a product. It cost far too much (\$950,000) and was technically obsolete as soon as it was built. Its real importance to IBM, apart from publicity and image, was to create a cadre of experienced computer engineers.

IBM also had two further full-scale computers under development in 1949: the Magnetic Drum Calculator (MDC) and the Tape Processing Machine (TPM). The MDC was to sell for perhaps a tenth of the price of a UNIVAC, but would give true stored-program computing for the kinds of firms currently using the CPC, as well as for ordinary business users currently using punched-card machines. The key money-saving technology behind it was the use of a magnetic drum for the main memory, instead of mercury delay lines or electrostatic storage tubes. The drum memory consisted of a rapidly rotating magnetized cylinder, which made for a very economical and reliable form of memory, but also one that was quite slow.

IBM’s Tape Processing Machine was much more on a par with the UNIVAC. As early as 1947 IBM had begun to explore the phenomenon of magnetic-tape recording and its potential as a replacement for punched cards in data processing, but the project moved slowly because of the competing demands of the SSEC. However, in mid-1949 reports of activities at the Eckert-Mauchly operation began to filter through to IBM – first, the successful completion of the BINAC, and second, the news that several orders had been taken for UNIVACs. The latter news convinced IBM to accelerate the TPM so that it would have a large, tape-based data-processing computer that would be competitive with UNIVAC.

If IBM had pressed on with either or both of these computers in 1949 or 1950, it would have dominated the market for data-processing computers much earlier than it did. In fact, these machines did not reach the market for another five years. The delay of the MDC was due largely to conservatism in IBM’s marketing. In 1949 the view in IBM’s “Future Demands” department was that the MDC was too expensive to compete with ordinary punched-card machines but too slow to compete with the UNIVAC. The delay with the Tape Processing Machine, however, had quite different causes: the start of the Korean War and a strategic decision made by Watson’s son, Thomas J. Watson Jr.

By 1950 Watson senior was in his mid-seventies, and while he remained at the helm of IBM until 1956, his thirty-five-year-old elder son was being groomed

to lead the company. Thomas J. Watson Jr. joined IBM as a young man in 1937, learning the ropes as a salesman. After war service, in 1945 he returned to IBM and four years later was made an executive vice president. When the Korean War broke out in June 1950, as in previous emergencies Watson senior telegraphed the White House and put IBM at the president's disposal. While Watson senior's motive was "primarily patriotic," Watson junior "seized on the idea as an opportunity to speed the company's development of large-scale computers."⁹ Market reports indicated that there would be a potential market for at least six large-scale scientific computers with defense contractors, so Watson junior committed the company to developing a computer based on the machine that von Neumann was building at the Institute for Advanced Study.

This would be a very powerful machine indeed; initial estimates put the rental at \$8,000 per month, equivalent to a purchase price of \$500,000 – an estimate that turned out to be just about half of the eventual cost. Development of the Defense Calculator, as the machine was called, required that the resources devoted to the Tape Processing Machine be taken away. In effect, Watson's decision to build the Defense Calculator in the summer of 1950, while accelerating IBM's entry into scientific computers, dashed its chances of an early entry into the data-processing market. It was a business error, one that allowed Eckert and Mauchly to capture the early market for data-processing computers with the UNIVAC.

UNIVAC Comes to Life

It will be recalled that in late 1949, Eckert and Mauchly had just learned of the withdrawal of American Totalisator and were urgently seeking a new backer to sustain the momentum of the UNIVAC development and fund the payroll for their 134 employees in Philadelphia. Eckert and Mauchly were, of course, already well known to corporations with an interest in computers. Indeed, when the BINAC had been inaugurated in August 1949, representatives from several companies, including IBM and Remington Rand, had attended.

Early in 1950 Eckert and Mauchly decided that they would have to try to sell the company to IBM. They secured an interview – for an unexplained purpose – with the Watsons, father and son, in their New York office. Tom Watson Jr. recalled the interview:

I was curious about Mauchly, whom I'd never met. He turned out to be a lanky character who dressed sloppily and liked to flout convention. Eckert, by contrast, was very neat. When they came in, Mauchly slumped down on the couch and put his feet up on the coffee table – damned if he was going to show any respect for my father. Eckert started describing what they'd accomplished. But Dad had already guessed the reason for their visit, and our lawyers had told him that buying their company was out of the question. UNIVAC was one of the few competitors we had, and antitrust law said we

couldn't take them over. So Dad told Eckert, "I shouldn't allow you to go on too far. We cannot make any kind of arrangement with you, and it would be unfair to let you think we could. Legally we've been told we can't do it."¹⁰

Eckert took the point immediately and thanked the Watsons for their time, even though the interview had been unproductive, while "Mauchly never said a word; he slouched out the door after an erect Eckert." Perhaps Watson's recollection says as much about his own values and those of IBM as it does about those of Mauchly. But it was quite plain that Mauchly would never have fit in with the IBM culture. In any case the Watsons must surely have judged that Eckert and Mauchly would bring very little to IBM that it did not already have under development; by 1950 the company had well over a hundred R&D workers engaged in electronics and computer research, an effort that comfortably matched that of Eckert and Mauchly's company.

Remington Rand and NCR were also interested in making some kind of agreement with Eckert and Mauchly, and the entrepreneurs were "so desperate that they were ready to consider the first reasonable offer."¹¹ Remington Rand got its bid in first.

In fact, even before the end of World War II, James Rand Jr., the charismatic president of Remington Rand, had drawn up a grand postwar expansion plan. He had been deeply impressed by the application of science to wartime activities, and he intended that after the war Remington Rand would develop a whole raft of high-technology products, many of them incorporating electronics. These would include microfilm recorders, xerographic copiers, industrial television systems, and so on. In late 1947 he recruited General Leslie R. Groves, the famous administrative chief of the Manhattan Project, to coordinate the research effort and head Remington Rand's development laboratories. Groves's involvement with atomic weapons research during the war had made him aware of the ENIAC and of Eckert and Mauchly's company; he well understood that computers would come to play an important role in the office.

James Rand invited Eckert and Mauchly down to his retreat in Florida, where they were royally entertained aboard his yacht, and he made them an offer: Remington Rand would repay all of American Totalisator's investment in the company (amounting to \$438,000), would pay Eckert, Mauchly, and their employees \$100,000 for the stock they owned, and would retain Eckert and Mauchly at a salary of \$18,000 per year. It was not a wonderful offer, for they would now become mere employees of the company they had founded, but there was no real alternative. Eckert and Mauchly accepted, and the Eckert-Mauchly Computer Corporation became a wholly owned subsidiary of Remington Rand. Although nominally under the direction of Groves, the general wisely avoided disrupting the UNIVAC operation or relocating the engineering program. Rather, he let things roll along with the considerable momentum Eckert and Mauchly had already built up.

While Remington Rand took a relaxed view of the firm's engineering development, it rapidly set the chronic financial condition to rights. After one of their visits to the Eckert-Mauchly operation, Rand and Groves and their financial advisers realized that the UNIVAC was being sold at perhaps a third of its true cost. First they approached the Census Bureau to try to renegotiate the price, threatening somewhat abrasively to cancel the order otherwise. The Census Bureau threatened to countersue, so the original price of \$300,000 had to stand. The Prudential and Nielsen machines had been offered at the giveaway price of \$150,000, whereas Remington Rand now realized it could not make a profit on anything less than \$500,000. In order to extricate itself from their contractual obligations, Remington Rand threatened to involve the Prudential and Nielsen in court proceedings that would hold up delivery of the UNIVAC for several years, by which time it would be obsolete. Nielsen and Prudential canceled their contracts and had their money returned. Eventually both bought their first computers from IBM.

While these negotiations were going on in the background, Eckert and Mauchly focused on completing the first UNIVAC. By the spring of 1950 it was at last beginning to take shape: "first, one bay was set up on the floor, then another, and another."¹² As ever, Eckert was the hub around which the whole project revolved. Still a young man, in his early thirties, he worked longer hours than anyone:

Each day he worked a little later and consequently was forced to arrive a little later in the morning. As the days went by, his work period slipped later and later into the day and then the evening. Finally he had to take a day off so that he could get re-phased and come in with everyone else in the morning.¹³

Presper Eckert had always had a curious habit of having to work out his ideas by talking them through with someone who could act as a sounding board. In the ENIAC days it was Mauchly who had been the sounding board, but now it was usually one of the other engineers:

Pres's nervous energy was so great he couldn't sit in a chair or stand still while he was thinking. He usually crouched on top of a desk or else paced back and forth. In many of his discussions with [one of his engineers], the pair started out at the computer test site, walked to the back of the second floor, then down a flight of steps to the first floor. After an hour or so, they would arrive back at the test site, having made a circular tour of the building, but completely oblivious to the fact that they had done so. The intensity of their discussions locked out all distractions.¹⁴

By contrast, Mauchly had a laid-back style: "John was always a great morale booster for everyone. His sense of humor penetrated the black clouds. He spoke in a slow, halting way as he related one interesting anecdote after another."¹⁵

By the summer of 1950 the UNIVAC subsystems were close to completion and were being tested in the baking Philadelphia heat. The equipment was located on the second floor of the UNIVAC building, beneath an uninsulated flat black roof. There was no air-conditioning, and even if there had been it could not have begun to dissipate the 120 kilowatts of heat produced by the 5,000-tube machine. First neckties and jackets were discarded; a few days later further layers of clothing were dispensed with; and finally "shorts and undershirts became the uniform of the day." One engineer recalled of another: "He had a half dozen soda bottles sitting on the edge of his desk and every once in a while, he reached for a soda bottle, placed it over the top of his head, and poured. The bottles were filled with water!"¹⁶

By the beginning of 1951, UNIVAC was functioning as a computer and would soon be ready to commence its acceptance tests for the Census Bureau. Before this was possible, however, some basic software had to be developed. The programming team was initially led by Mauchly, but was later run by Grace Murray Hopper recruited from the Harvard Computation Laboratory. She would later become the driving force behind advanced programming techniques for commercial computers and the world's foremost female computer professional.

On 30 March 1951 UNIVAC ran its acceptance tests, performing altogether some seventeen hours of rigorous computing without a fault. The machine then became the property of the Census Bureau. Over the following year, two more UNIVACs were completed and delivered to the government, and orders for a further three were taken.

In late 1952, Remington Rand achieved a spectacular publicity stunt for UNIVAC. The company persuaded the CBS television network to use UNIVAC to predict the outcome of the presidential election. Some months before the election, John Mauchly, with the help of a statistician from the University of Pennsylvania, devised a program that would use the early returns from a number of key states to predict the result, based on the corresponding voting patterns in 1944 and 1948.

On election night, CBS cameras were installed in the UNIVAC building in Philadelphia with reporter Charles Collingwood as the man on the spot. At CBS headquarters, Walter Cronkite served as anchorman, while in the studio a dummy UNIVAC console was installed for dramatic effect – the drama being heightened by flickering console lights, driven by nothing more sophisticated than blinking Christmas tree lights.

The UNIVAC printed its first prediction at 8:30 P.M.:¹⁷

IT'S AWFULLY EARLY, BUT I'LL GO OUT ON A LIMB.

UNIVAC PREDICTS - WITH 3,398,745 VOTES IN -

	STEVENSON	EISENHOWER
STATES	5	43
ELECTORAL	93	438
POPULAR	18,986,436	32,915,049

THE CHANCES ARE NOW 00 TO 1 IN FAVOR OF THE ELECTION OF EISENHOWER.

UNIVAC was forecasting a landslide for Eisenhower – in complete contrast to the Gallup and Roper opinion polls taken the previous day, which had predicted a close race. A member of the UNIVAC team recalled:

Our election officials . . . looked on in disbelief. The computer called for an Eisenhower victory by an overwhelming landslide. The odds by which he would win exceeded the two digits allowed in the program; thus the printout showed 00 to 1 instead of 100 to 1. The officials put their heads together and said, “We can’t let this go out. The risk is too great.” It was beyond their comprehension that with so few votes counted the machine could predict with such a degree of certainty that the odds would be greater than 100 to 1.¹⁸

The UNIVAC operators quickly adjusted the parameters of the program, so that it would produce more believable results. At 9:15, the UNIVAC’s first broadcast prediction anticipated an Eisenhower win with the more modest odds of 8 to 7. But as the night wore on, it became clear that a landslide was indeed developing. A UNIVAC spokesman later appeared on television to come clean and admit to the suppression of UNIVAC’s original prediction. The final result was a win for Eisenhower against Stevenson of 442 to 89 electoral votes – within a whisker of UNIVAC’s original prediction of 438 to 93.

Of course, UNIVAC’s programmers and Remington Rand’s managers kicked themselves for holding back the initial prediction, but they could hardly have invented a more convincing demonstration of the apparent infallibility of a computer. Or, rather, of a UNIVAC – for UNIVAC was rapidly becoming the generic name for a computer. The appearance of the UNIVAC on election night was a pivotal moment in computer history. Before that date, while some people had heard about computers, very few had actually seen one; after it, the general public had been introduced to computers and had seen at least a mock-up of one. And that computer was called a UNIVAC, not an IBM.

IBM's Big Push

In fact, IBM had begun to see the writing on the wall several months before UNIVAC's election-night stunt. It was the launch of the Census Bureau UNIVAC eighteen months earlier that had galvanized IBM into action. According to Watson junior, he first learned of the completion of the UNIVAC from the chief of IBM's Washington office while they were having a routine meeting. It came to Watson as a bolt from the blue:

I thought, "My God, here we are trying to build Defense Calculators, while UNIVAC is smart enough to start taking all the civilian business away!" I was terrified.

I came back to New York in the late afternoon and called a meeting that stretched long into the night. There wasn't a single solitary soul in IBM who grasped even a hundredth of the potential the computer had. We couldn't visualize it. But the one thing we could understand was that we were losing business. Some of our engineers already had a fledgling effort under way to design a computer for commercial applications. We decided to turn this into a major push to counter UNIVAC.¹⁹

There was a great deal of ego at the top of IBM, and Watson junior was creating his own mythology. It was absurd to say, as Watson did, that "There wasn't a single solitary soul in IBM who grasped . . . the potential the computer had." As we have seen, in the five or six years since the war IBM had developed a superb organizational capability in electronics and computers, and even had two serious data-processing computers under arrested development (the Tape Processing Machine and the Magnetic Drum Calculator). It was a consequence of Watson's autocratic management style that he had – essentially on a hunch – promoted the Defense Calculator to the detriment of the two data-processing computers.

However, one advantage of Watson's autocratic style was that once he had decided that IBM had to move into data-processing computers in a big way – and had convinced his father – it could be made to happen by decree. As of the spring of 1951, IBM had three major computer projects running full steam ahead: the Defense Calculator, the Tape Processing Machine, and the low-cost Magnetic Drum Calculator. These would later emerge as Models 701, 702, and 650. Watson recalled: "People at IBM invented the term 'panic mode' to describe the way we operated: there were moments when we thought we were aboard the *Titanic*."²⁰ The development cost of the three concurrent projects was enormous, and large numbers of R&D workers had to be recruited. A year later, 35 percent of IBM's laboratory staff was working on electronics, 150 of them on the Defense Calculator alone.

The Defense Calculator, renamed Model 701, was the farthest along of the projects. Originally expected to rent for \$8,000 per month, the machine had quickly taken ten orders. In early 1952, however, it became clear that the

machine would have to rent for about twice that amount – equivalent to a purchase price of about \$1 million, much the same as the UNIVAC was turning out to cost. Customers who had written initial letters of intent for the machine had to be advised of the new rental, but fortunately for IBM most of the orders held up on the revised terms. This ready acceptance of the higher rentals reflected the greater maturity and knowledge of computer buyers in 1952: large, fast scientific computers were going to cost about \$1 million, regardless of who made them.

Surprisingly, many conservatives within IBM still argued against developing the Tape Processing Machine, maintaining as late as 1953 that conventional punched-card machines would be cheaper and more cost-effective. However, this failed to take account of the fact that computers were hot news, and business magazines were buzzing with stories about electronic brains for industry and commerce. Cost-effectiveness was no longer the only, or even the most important, reason for a business to buy a computer. A January 1952 article in *Fortune* magazine titled “Office Robots” caught the mood exactly:

The longest-haired computer theorists, impatient of half measures, will not be satisfied until a maximum of the human element and paper recording is eliminated. No intervening clerical operators. No bookkeepers. No punched cards. No paper files. In the utility billing problem, for instance, meter readings would come automatically by wire into the input organs of the central office’s electronic accounting and information processing machine, which, when called on, would compare these readings with customers’ accounts in its huge memory storage, make all computations, and return the new results to storage while printing out the monthly bills.²¹

Of course, this journalistic hyperbole was little short of science fiction, but Watson and IBM’s sales organization realized by this time that IBM had to be seen as an innovator and had to develop a data-processing computer, whether or not it made money.

The first Model 701 scientific computer came off the production line in December 1952, and IBM derived the maximum publicity it could. In a dramatic out-with-the-old-in-with-the-new gesture, designed in part to draw attention away from the UNIVAC, the 701 was installed in the IBM showroom in its New York headquarters – in the place that had been occupied by the SSEC, which was now unceremoniously dismantled.

IBM’s first real data-processing computer, the Model 702 – based on the TPM – was announced in September 1953. By the following June, IBM had taken fifty orders. However, deliveries did not begin until early 1955, four years after delivery of the first UNIVAC. During the eighteen months between the announcement of the 702 and the first deliveries, it was only Remington Rand’s inability to deliver more UNIVACs that enabled IBM to protect its order book.

Although the IBM 702 Electronic Data Processing Machine (EDPM) had superficially much the same specification as the UNIVAC, in the underlying engineering there were major differences that enabled IBM's product quickly to outdistance the UNIVAC in the marketplace. Instead of the mercury delay lines chosen for the UNIVAC, IBM decided to license the Williams Tube technology developed for the Manchester University computer in England. Both were difficult and temperamental technologies, but the Williams Tube memory was probably more reliable and certainly twice as fast. The magnetic-tape systems on the 700 series were far superior to those of the UNIVAC, which were never completely satisfactory and required constant maintenance. Developing a successful magnetic-tape system was much more of a mechanical-engineering problem than an electronic one, and IBM's outstanding capability in electromechanical engineering enabled it to make a system that was much faster and more reliable.

Another advantage over the UNIVAC was that IBM's computers were modular in construction – that is, they consisted of a series of “boxes” that could be linked together on site. In addition to making shipment easier, modular construction led to greater flexibility in the size and specification of a machine. By contrast, the UNIVAC was such a monolith that just moving it from the factory to a customer's site was a major operation. Unlike UNIVAC, IBM set sizes of modules to fit into standard elevators. Indeed, the first UNIVAC for the Census Bureau spent its initial several months being run in the Philadelphia plant rather than undergoing the risk of being moved. Finally, IBM's greatest advantage was its reputation as a service-oriented vendor. Recognizing the importance of training from the very beginning, the company organized programming courses for users and established field-engineering teams that provided a level of customer service superior to that offered by any other vendor.

In fact, neither the 701 nor the 702 proved to be quite the “UNIVAC-beater” that IBM had hoped. For both machines, the Williams Tube memories proved troublesome, making them unreliable by IBM's normal high standards. By 1953, however, a new technology known as core memory had been identified. Several groups worked independently on core memories, but the most important development was the work of Jay Forrester at MIT (see Chapter 7). Although it was still a laboratory prototype at this time, IBM mounted a crash research program to develop core memory into a reliable product. More reliable and faster core-memory versions of the 701 and 702 were announced in 1954 as the Model 704 scientific computer and the Model 705 EDPM.

According to Watson junior, 1955 was the turning point for IBM. That year, orders for IBM's 700 series computers exceeded those for UNIVACs. As late as August 1955, UNIVACs had outsold 700s by about 30 to 24. But a year later, there were 66 IBM 700 series installations in place compared with 46 UNIVACs; and there were, respectively, 193 to 65 machines on order.

Yet it was not the large-scale 700 series that secured IBM's leadership of the industry but the low-cost Magnetic Drum Calculator. The machine was

announced as the Model 650 in 1953 and rented for \$3,250 a month (the equivalent of a purchase price of about \$200,000). Although only a quarter of the cost of a 701, it was still a very expensive machine – costing up to twice as much as comparable machines from other makers. However, it succeeded in the market by virtue of its superior engineering, reliability, and software. Eventually 2,000 of these machines were delivered, yielding revenues far exceeding those of the entire 700 series.

As Watson junior observed: “While our giant, million-dollar 700 series got the publicity, the 650 became computing’s ‘Model T.’”²² With an astute understanding of marketing, IBM placed many 650s in universities and colleges, offering machines with up to a 60 percent discount – provided that courses were established in computing. The effect was to create a generation of programmers and computer scientists nurtured on IBM 650s as well as a trained workforce for IBM’s products. It was a good example of IBM’s mastery of marketing, which was in many ways more important than mastery of technology.

With the success of the 650, IBM rapidly began to eclipse Remington Rand’s UNIVAC division. Even so, since the launch of the first UNIVAC in 1951, Remington Rand had not been sitting idly by, but had been actively expanding its computer division. In 1952 it had acquired the small Minneapolis-based start-up, Engineering Research Associates, which developed Remington Rand’s first scientific computer, the UNIVAC 1103. In mid-1955 Remington Rand merged with Sperry Gyroscope – and was renamed Sperry Rand – to further develop its technology. But in 1954 or 1955 it had already begun to slip behind IBM. Part of the reason for this was a failure in Remington Rand’s marketing: the company had been nervous that computers might lower sales of its traditional punched-card equipment and had therefore failed to integrate its punched-card and computer sales staff, often missing sales opportunities. Two other reasons for Remington Rand’s decline were infighting between its Philadelphia and Minneapolis factories and its unwillingness to invest heavily enough in computer software and marketing. While UNIVAC was still the public’s idea of a computer, corporate decision makers were beginning to favor IBM. In the memorable phrase of the computer pundit Herb Grosch, Remington Rand – in losing its early lead to IBM – “snatched defeat from the jaws of victory.”²³

Events in 1956 served to reinforce IBM’s future course in computing. Four years earlier, in January 1952, the Department of Justice had filed an antitrust suit against IBM alleging monopolistic behavior – for example, using patents to exclude competitors and forcing customers to lease IBM equipment, thereby preventing the development of a secondhand market. Watson Sr. had resisted any change for several years, but his son judged that loosening IBM’s hold on the punch-card machine market “might be just what was needed”;²⁴ it would force the company to look forward, not back, and make its future in the new technology of computers. With his father’s reluctant agreement, in January 1956 Watson Jr. instructed IBM’s lawyers to sign a “consent decree” agreeing to the

Department of Justice's terms. The following May, Watson Sr. passed the role of chief executive to his son; he died the following month.

The Computer Race

Although the story of IBM versus UNIVAC is a fair metaphor for the progress of the computer industry in the 1950s, the picture was much more complicated. The scene was one of a massive shakeout toward the end of the 1950s.

In the early 1950s the electronics and control equipment manufacturers had been able to participate in the computer business by making machines for science and engineering – if not profitably, then with tolerable losses. By the end of the decade, however, as the computer was transformed into a business machine manufactured and sold in high volumes, these companies were all faced with the same decision: whether to exit the business or to spend the enormous sums necessary to develop peripherals and applications software, and pay for the marketing costs, to compete with IBM. Of the major firms, only three – RCA, GE, and Honeywell – chose to commit the resources to stay in the race. The remainder, including such established companies as Philco and Raytheon, fell by the wayside.

However, by far the most vulnerable firms were the small computer start-ups, of which very few survived into the 1960s. They included firms such as the Computer Research Corporation (CRC – a spin-off of Northrop Aircraft Corporation), Datamatic (a joint venture of Honeywell and Raytheon), Electrodata, the Control Data Corporation (CDC), and the Digital Equipment Corporation (DEC). Of these, only two – CDC and DEC – retained their independence and developed into major firms. And of these two, only CDC had become a major player in mainframe computers by the end of the 1950s. DEC would survive, but it was not until it found a niche in minicomputers that it became a major player (see Chapter 9). After old-fashioned bankruptcy, the next most common fate of the small computer firms was to be absorbed by one of the office-machine companies – just as EMCC and ERA had been absorbed by Remington Rand.

Apart from IBM, none of the office-machine firms entered the postwar world with much in the way of electronics experience, nor any interest in making computers. Like IBM, the office-machine companies had prudent doubts about the market for business computers and initially did nothing more than put electronics into their existing products in an evolutionary way. NCR, for example, devised a machine called the “Post-Tronic,” which was an electronic enhancement of its ordinary accounting machines supplied to banks. The Post-Tronic, however, was a brilliant success – bigger than almost any computer of its era. The machine created a sensation when it was announced in 1956 at the American Bankers Association meeting. It went on to do \$100 million worth of business and “became an important factor in making NCR’s heavy investment for computer development possible.”²⁵ However, to make the final leap into computer production, NCR acquired CRC in 1953 and launched a series of business computers in 1956.

Burroughs also tried to develop its own computer division, and in 1948 hired Irven Travis, then research director of the Moore School, to run its Paoli, Pennsylvania, computer development laboratory. Burroughs initially spent several million dollars a year on electronics research and development, a decision that “represented courage and foresight, because in the 1946–1948 period, our revenue averaged less than \$100 million a year and our net profit was as low as \$1.9 million in 1946.”²⁶ Despite this investment, Burroughs failed to develop a successful data-processing computer, and so in 1956 acquired the Electrodata Company, a start-up that had already developed a successful business computer, the Datatron. Burroughs paid \$20.5 million for Electrodata, an enormous sum at the time, but with the combination of Electrodata’s technology and Burroughs’s sales force and customer base, it was soon able to secure an acceptable position in the computer company league.

Several other office-machine companies tried to enter the computer race in the 1950s, by either internal development or acquisition. These included Monroe, Underwood, Friden, and Royal, but none of their computer operations survived into the 1960s. Thus, by the end of the 1950s the major players in the computer industry consisted of IBM and a handful of also-rans: Sperry Rand, Burroughs, NCR, RCA, Honeywell, GE, and CDC. Soon, journalists would call them IBM and the seven dwarfs.

Despite the gains of the 1950s in establishing a computer industry, as the decade drew to a close the computer race had scarcely begun. In 1959, IBM was still deriving 65 percent of its income from its punched-card machines in the United States, and in the rest of the world the figure was 90 percent. The 1950s had been a dramatic period of growth for IBM, with its workforce increasing from thirty thousand to nearly a hundred thousand, and its revenue growing more than fivefold, from \$266 million to \$1,613 million. In the next five years, IBM would come to utterly dominate the computer market. The engine that would power that growth was a new computer, the IBM 1401.

Notes to Chapter 5

The most detailed account of the Eckert-Mauchly computer business is contained in Nancy Stern’s *From ENIAC to UNIVAC: An Appraisal of the Eckert-Mauchly Computers* (1981), while Herman Lukoff’s *From Dits to Bits* (1979) contains a great deal of atmospheric detail on the building of the UNIVAC. Arthur Norberg’s *Computing and Commerce* (2005) sets the Eckert-Mauchly Computer Corporation and ERA in a broad industrial context. The story of UNIVAC and the election is told in great detail in Ira Chinoy’s PhD dissertation “Battle of the Brains: Election-Night Forecasting at the Dawn of the Computer Age” (2010).

The best overall account of IBM’s computer developments is Emerson Pugh’s *Building IBM* (1995). Technical developments are described in Charles Bashe et al.’s *IBM’s Early Computers* (1986). Thomas J. Watson’s autobiography

Father and Son & Co. (1990) is a valuable, if somewhat revisionist, view from the top of IBM. The best recent history of IBM is James W Cortada's *IBM: The Rise and Fall and Reinvention of a Global Icon* (2019).

The many players in the computer industry in the 1950s are detailed in Tom Haigh and Paul Ceruzzi's *A New History of Modern Computing* (2021), James W. Cortada's *The Computer in the United States: From Laboratory to Market, 1930–1960* (1993), Katherine Fishman's *The Computer Establishment* (1981), and Franklin M. Fisher et al.'s *IBM and the U.S. Data Processing Industry* (1983).

Notes

1. Stern 1981, p. 106.
2. Lukoff 1979, p. 81.
3. Quoted in Stern 1981, p. 124.
4. Campbell-Kelly 1989, p. 106.
5. Quoted in Pugh 1995, p. 122.
6. *Ibid.*, p. 131.
7. Bowden 1953, pp. 174–5.
8. Anonymous 1950.
9. Pugh 1995, p. 170.
10. Watson 1990, pp. 198–9.
11. Stern 1981, p. 148.
12. Lukoff 1979, p. 96.
13. *Ibid.*, p. 106.
14. *Ibid.*
15. *Ibid.*, p. 75.
16. *Ibid.*, pp. 99–100.
17. The UNIVAC forecast is reproduced in *ibid.*, p. 130.
18. *Ibid.*, pp. 130–1.
19. Watson 1990, p. 227.
20. *Ibid.*, p. 228.
21. Anonymous 1952, p. 114.
22. Watson 1990, p. 244.
23. Quoted in Rodgers 1969, p. 199.
24. Maney 2003, p. 387.
25. Allyn 1967, p. 161.
26. Quoted in Fisher et al. 1983, p. 81.

6 The Maturing of the Mainframe

The Rise of IBM

THE 1957 FILM *Desk Set*, a light romantic comedy starring Katherine Hepburn and Spencer Tracy, was also the first major motion picture to feature an electronic computer. In the film Tracy plays the “efficiency expert” Richard Sumner, who is charged with introducing computer technology into the research division of the fictional Federal Broadcasting Network. Opposed to his efforts is Bunny Watson (the Hepburn character), who resents what she believes to be an attempt to replace her and her fellow librarians with an “electronic brain machine.” As Tracy and Hepburn spar and banter, and eventually fall in love, the imposing EMERAC (Electro-Magnetic Memory and Research Arithmetic Calculator) is gradually revealed to be charmingly simple-minded and nonthreatening. When a technician mistakenly asks the computer for information on the island “Curfew” (rather than Corfu), “Emmy” goes haywire and starts smoldering – until Bunny Watson steps in to help fix “her” with the loan of a helpful bobby pin.

Although the name EMERAC is an obvious play on both the ENIAC and UNIVAC, the computer manufacturer most closely associated with *Desk Set* was the IBM Corporation. The film was a triumph of product placement for IBM: it opens with a close-up of an IBM computer, IBM provided the training and equipment used by actors and producers, and IBM was prominently acknowledged in the film’s closing credits. And while the film accurately reflects contemporary concerns about the possibility of technologically driven unemployment, its ultimate conclusion about the computer is unrelentingly positive: computers were machines for eliminating drudgery, not human employees. In many respects, the film was an extended advertisement for electronic computing in general and for the IBM Corporation in particular.

Desk Set was also, however, a remarkably insightful mirror of a key period in the history of modern computing. The script writers, Henry and Phoebe Ephron, were astute social observers. Both the combination of the electronic computer with the efficiency expert and the ambivalence that many Americans felt toward “electronic brains” were accurate reflections of contemporary attitudes. Like Bunny Watson, workers (and their managers) had to learn to embrace the computer.

But even during the time that *Desk Set* was showing in movie theaters, its inanimate star EMERAC was beginning to look curiously old-fashioned. Its gray enameled casing and rounded corners harked back to an austere office-machine past. IBM was about to transform its standing in the industry – and the image of its computers – with the model 1401 announced in fall 1959. Three years earlier, IBM had hired Eliot Noyes, one of America’s foremost industrial designers, and the 1401 bore all the hallmarks of his influence. Its sharp-cornered design and colorful paintwork shrieked modernity.

The IBM 1401

The 1401 was not so much a computer as a computer *system*. Most of IBM’s competitors were obsessed with designing central processing units, or “CPUs,” and tended to neglect the system as a whole. There were plenty of technical people in IBM similarly fixated on the architecture of computers: “Processors were the rage,”¹ one IBM technical manager recalled. “Processors were getting the resources, and everybody that came into development had a new way to design a processor.” The technocrats at IBM, however, were overridden by the marketing managers, who recognized that customers were more interested in the solution to business problems than in the technical merits of competing computer designs. As a result, IBM engineers were forced into taking a total-system view of computers – that is, a holistic approach that required them to “take into account programming, customer transition, field service, training, spare parts, logistics, etc.”² This was the philosophy that brought success to the IBM 650 in the 1950s and would soon bring to the model 1401. It is, in fact, what most differentiated IBM from its major competitors.

The origin of the 1401 was the need to create a transistorized follow-up to the tube-based model 650 Magnetic Drum Computer. By 1958, eight hundred model 650s had been delivered – more than all the computers produced by all the other mainframe manufacturers put together. Even so, the 650 had failed to make big inroads into IBM’s several thousand traditional punched-card accounting-machine installations. There were a number of sound reasons why most of IBM’s customers were still resisting the computer and continuing to cling to traditional accounting machines. Foremost among these was cost: for the \$3,250 monthly rental of a 650, a customer could rent an impressive array of punched-card machines that did just as good a job or better. Second, although the 650 was probably the most reliable of any computer on the market, it was a tube-based machine that was fundamentally far less reliable than an electro-mechanical accounting machine. Third, the would-be owner of a 650 was faced with the problem of hiring programming staff to develop applications software, since IBM provided little programming support at this time. This was a luxury that only the largest and most adventurous corporations could afford. Finally, most of the “peripherals” for the 650 – card readers and punches, printers, and

so on – were lackluster products derived from existing punched-card machines. Hence, for most business users, the 650 was in practice little more than a glorified punched-card machine. It offered few real advantages but many disadvantages. Thus the IBM 407 accounting machine remained IBM's most important product right up to the end of the 1950s.

During 1958 the specification of the 1401 began to take shape, and it was heavily influenced by IBM's experience with the 650. Above all, the new machine would have to be cheaper, faster, and more reliable than the 650. This was perhaps the easiest part of the specification to achieve since it was effectively guaranteed merely by the substitution of improved electronics technology: transistors for tubes and core memory for the magnetic drum of the 650, which would produce an order-of-magnitude improvement in speed and reliability. Next, the machine needed peripherals that would give it decisive advantages over the 650 and electric accounting machines: new card readers and punches, printers, and magnetic-tape units. By far the most important new peripheral under development was a high-speed printer, capable of 600 lines per minute.

IBM also had to find a technological solution to the programming problem. The essential challenge was how to get punched-card-oriented business analysts to be able to write programs without a huge investment in retraining them and without companies having to hire a new temperamental breed of programmer. The solution that IBM offered was a new programming system called Report Program Generator (RPG). It was specially designed so that people familiar with wiring up plugboards on accounting machines could, with a day or two of training, start to write their own business applications using familiar notations and techniques. The success of RPG exceeded all expectations, and it became one of the most used programming systems in the world. However, few RPG programmers were aware of the origins of the language or realized that some of its more arcane features arose from the requirement to mimic the logic of a punched-card machine.

However, not every customer wanted to be involved in writing programs, even with RPG; some preferred to have applications software developed for them by IBM – for applications such as payroll, invoicing, stock control, production planning, and other common business functions. Because of its punched-card machine heritage, IBM knew these business procedures intimately and could develop software that could be used with very little modification by any medium-sized company. IBM developed entire program suites for the industries that it served most extensively, such as insurance, banking, retailing, and manufacturing. These application programs were very expensive to develop, but because IBM had such a dominant position in the market it could afford to “give” the software away, recouping the development costs over tens or hundreds of customers. Because IBM's office-machine rivals – Sperry Rand, Burroughs, and NCR – did not have nearly as many customers, they tended to develop software

for the one or two industries in which they had traditional strengths rather than trying to compete with IBM across the board.

The IBM 1401 was announced in October 1959, and the first systems were delivered early in 1960 for upward of \$2,500 per-month rental (the equivalent of a purchase price of \$150,000). This was not much more than the cost of a medium-sized punched-card installation. IBM had originally projected that it would deliver around one thousand systems. This turned out to be a spectacular underestimate, for twelve thousand systems were eventually produced.

How did IBM get its forecast so wrong? The 1401 was certainly an excellent computer, but the reasons for its success had very little to do with the fact that it was a computer. Instead, the decisive factor was the new 1403 “chain” printer that IBM supplied with the system. The printer used a new technology by which slugs of type were linked in a horizontal chain that rotated rapidly and were hit on the fly by a series of hydraulically actuated hammers. The printer achieved a speed of 600 lines per minute, compared with the 150 lines per minute of the popular 407 accounting machine that was based on a prewar printing technology. Thus, for the cost of a couple of 407s, the 1401 provided the printing capacity of four standard accounting machines – and the flexibility of a stored-program computer was included, as it were, for free. The new printing technology was an unanticipated motive for IBM’s customers to enter the computer age, but no less real for that.

For a while, IBM was the victim of its own success, as customer after customer decided to turn in old-fashioned accounting machines and replace them with computers. In this decision they were aided and abetted by IBM’s industrial designers, who surpassed themselves by echoing the modernity and appeal of the new computer age: out went the round-cornered steel-gray punched-card machines, and in came the square-cornered light-blue computer cabinets. For a firm with less sophisticated financial controls and less powerful borrowing capabilities than IBM, coping with the flood of discarded rental equipment would have been a problem. But as a reporter in *Fortune* noted:

[F]ew companies in U.S. business history have shown such growth in revenues and such constantly flourishing earnings. I.B.M. stock, of course, is the standard example of sensational growth sustained over many years. As everybody has heard, anyone who bought 100 shares in 1914 (cost, \$2,750), and put up another \$3,614 for rights along the way, would own an investment worth \$2,500,000 today.³

With a reputation like that, the unpredicted success of a new product was a problem that IBM’s investors were willing to live with. As more and more 1401s found their way into the nation’s offices in their powder-blue livery, IBM earned a sinister new name: Big Blue.

IBM and the Seven Dwarfs

Meanwhile, IBM's success was creating a difficult environment for its competitors. The 1401 was more than a successful product for IBM: it heralded a sea-change in office machinery. Transistorization, improved reliability, and price declines meant that old-style accounting machines could no longer compete with computers – whether in speed, price, or flexibility. By 1960 the mainframe computer industry had already been whittled down to just IBM and seven others. Of all the mainframe suppliers, Sperry Rand had suffered the biggest reverse, consolidating a decline that had begun well before the launch of the 1401. Despite being the pioneer of the industry, it had never made a profit in computers and was gaining a reputation bordering on derision. Another *Fortune* writer noted:

The least threatening of I.B.M.'s seven major rivals . . . has been Sperry Rand's Univac Division, the successor to Remington Rand's computer activities. Few enterprises have ever turned out so excellent a product and managed it so ineptly. Univac came in too late with good models, and not at all with others; and its salesmanship and software were hardly to be mentioned in the same breath with I.B.M.'s.⁴

As a result “the upper ranks of other computer companies are studded with ex-UNIVAC people who left in disillusionment.” In 1962 Sperry Rand brought in an aggressive new manager from ITT, Louis T. Rader, who helped UNIVAC address its deficiencies. But despite making a technologically successful entry into computer systems for airline reservations, Rader was soon forced to admit: “It doesn't do much good to build a better mousetrap if the other guy selling mousetraps has five times as many salesmen.”⁵

In 1963 UNIVAC turned the corner and started to break even at last. Yet the machine that brought profits, the UNIVAC 1004, was not a computer at all but a transistorized accounting machine, targeted at its existing punched-card machine users. Even with the improving outlook, UNIVAC still had only a 12 percent market share and a dismal one-sixth as many customers as IBM. This was little better than the market share that Remington Rand had secured with punched-card machines in the 1930s. It seemed that the company would be forever second.

IBM's other business-machine rivals, Burroughs and NCR, were a long way behind IBM and even UNIVAC, both having market shares of around 3 percent – only a quarter of that of UNIVAC. In both cases, their strategy was safety first: protect their existing customer base, which was predominantly in the banking and retail sectors, respectively. They provided computers to enable their existing customers to move ahead with the new technology in an evolutionary way, as they felt the need to switch from electromechanical to electronic machines.

Doing much better than any of the old-line office-machine companies, the fastest-rising star in the computer scene of the early 1960s was Control Data Corporation, the start-up founded in 1957 by the entrepreneur William Norris. Control Data had evolved a successful strategy by manufacturing mainframes with a better price performance than those of IBM – by using superior electronics technology and by selling them into the sophisticated scientific and other markets where the IBM sales force had not so great an impact. By 1963 Control Data had risen to third place in the industry, not far behind UNIVAC.

The only other important survivors in the mainframe industry of the early 1960s were the computer divisions of the electronics and control giants – RCA, Honeywell, and General Electric. All three had decided to make the necessary investment to compete with IBM. In every case this was a realistic possibility only because they rivaled IBM in power and size. The difference, however, was that for all of them the computer was not a core business but an attempt to enter a new and unfamiliar market. For all of these corporations entry into the computer industry could not be allowed to imperil their core businesses; there was no certainty that they would all survive as computer makers. Each had plans to make major product announcements toward the end of 1963.

Honeywell was planning to launch its model 200 computer that would be compatible with the IBM 1401 and able to run the same software, and therefore IBM's existing customers would be an easy target. Honeywell's technical director, a wily ex-UNIVAC engineer named J. Chuan Chu, had reasoned that IBM was sufficiently nervous about its vulnerability to antitrust litigation to be "too smart not to let us take 10 per cent of the business."⁶ Inside RCA, General David Sarnoff, its legendary chief executive officer, had committed the company to spending huge sums to develop a range of IBM-compatible computers and was in the process of making licensing agreements with overseas companies to manufacture the computers in the rest of the world. Finally, General Electric also had plans to announce a range of three computers – one small, one medium-sized, and one large – in late 1963.

Revolution, Not Evolution: System/360

Although in marketing terms the IBM 1401 had been an outstanding success, inside IBM there was a mishmash of incompatible product lines that threatened its dominance of the industry. Many IBM insiders felt that this problem could be resolved only by producing a "compatible" range of computers, all having the same architecture and running the same software.

In 1960 IBM was producing no fewer than seven different computer models – some machines for scientific users, others for data-processing customers; some large machines, some small, and some in between. This caused problems for both IBM and its customers. In manufacturing terms IBM was getting precious little benefit from its enormous scale. By simultaneously manufacturing seven

different computer models, IBM was being run almost as a federation of small companies instead of an integrated whole. Each computer model had a dedicated marketing force trained to sell into the niche that computer occupied, but these specialized sales forces were unable to move easily from one machine or market niche to another. Each computer model required a dedicated production line and its own specialized electronic components. Indeed, IBM had an inventory of no fewer than 2,500 different circuit modules for its computers. Peripherals also presented a problem, since hundreds of peripheral controllers were required so that any peripheral could be attached to any processor.

All this has to be compared with IBM's punched-card products, where a single range of machines (the 400 series accounting machines) satisfied all its customers. The resulting rationalization of production processes and standardization of components had reduced manufacturing costs to such an extent that IBM had no effective competition in punched-card machines at all.

The biggest problem, however, was not in hardware but in software. Because the number of software packages IBM offered to its customers was constantly increasing, the proliferation of computer models created a nasty gearing effect: given m different computer models, each requiring n different software packages, a total of $m \times n$ programs had to be developed and supported. This was a combinatorial explosion that threatened to overwhelm IBM at some point in the not-too-distant future.

Just as great a problem was that of the software written by IBM's customers. Because computers were so narrowly targeted at a specific market niche, it was not possible for a company to expand its computer system in size by more than a factor of about two without changing to a different computer model. If this was done, then all the user's applications had to be reprogrammed. This often caused horrendous organizational disruption during the changeover period. Indeed, reprogramming could be more costly than the new computer itself. The "switching costs" for computer users was one of the big challenges of the era. As IBM appreciated only too well, once a company had decided to switch computer models, it could look at computers from all manufacturers – not just those made by IBM.

All these factors suggested that IBM would sooner rather than later have to embrace the compatible-family concept. But in IBM's case this would be particularly challenging technically because of the wide spectrum of its customers in terms of size and applications. A compatible series would have to satisfy all of IBM's existing customers, from the very smallest to the very largest, as well as both its scientific and commercial customers. And the new machine would have to be compatible throughout the product range, so that programs written on one machine would execute on any other – more slowly on the small machines, certainly, but they would have to run without any reprogramming at all.

The decision to produce a compatible family was not so clear-cut as it appears in hindsight, and there was a great deal of agonizing at IBM. For one thing, it

was by no means clear that compatibility to the extent proposed was technically feasible. And even if it were, it was feared that the cost of achieving compatibility might add so much to the cost of each machine that it would not be competitive in the marketplace. Another complication was that there were factions within IBM that favored consolidating its current success by building on its existing machines. The 1401 faction, for example, wanted to make more powerful versions of the machine. These people thought it was sheer madness for IBM to think of abandoning its most successful product ever. If IBM dropped the 1401, they argued, thousands of disaffected users would be left open to competitors. Another faction inside IBM favored, and had partially designed, a new range – to be known as the 8000 series – to replace IBM's large 7000 series machines.

But it was the software problem that was to determine product strategy, and by late 1960 the tide was beginning to turn toward the radical solution. Not one of IBM's computers could run the programs of another, and if IBM was to introduce more computer models, then, as a top executive stated, "we are going to wind up with chaos, even more chaos than we have today."⁷ For the next several months planners and engineers began to explore the technical and managerial problems of specifying the new range and of coordinating the fifteen to twenty computer development groups within IBM to achieve it. Progress was slow, not least because those involved in the discussions had other responsibilities or preferred other solutions – and the compatible-family concept was still no more than a possibility that might or might not see the light of day.

In order to resolve the compatible-family debate rapidly, in October 1961 T. Vincent Learson, Tom Watson Jr.'s second-in-command at IBM, established the SPREAD task group, consisting of IBM's thirteen most senior engineering, software, and marketing managers. SPREAD was a contrived acronym that stood for Systems, Programming, Review, Engineering, And Development, but which was really meant to connote the broad scope of the challenge in establishing an overall plan for IBM's future data-processing products. Progress was slow and, after a month, Learson, an archetypal IBM vice president, became impatient for a year's-end decision. In early November he sequestered the entire task group to a motel in Connecticut, where it would not be distracted by day-to-day concerns, with "orders not to come back until they had agreed."⁸

The eighty-page SPREAD Report, dated 28 December 1961, was completed on virtually the last working day of the year. It recommended the creation of a so-called New Product Line that was to consist of a range of compatible computers to replace all of IBM's existing computers. On 4 January the SPREAD Report was presented to Watson Jr., Learson, and the rest of IBM's top management. The scope of the report was breathtaking – as was the expense of making it a reality. For example, the software alone would cost an estimated \$125 million – at a time when IBM spent just \$10 million a year on all its programming

activities. Learson recalled that there was little enthusiasm for the New Product Line at the meeting:

The problem was, they thought it was too grandiose. . . . The job just looked too big to the marketing people, the financial people, and the engineers. Everyone recognized it was a gigantic task that would mean all our resources were tied up in one project – and we knew that for a long time we wouldn't be getting anything out of it.⁹

But Watson and Learson recognized that to carry on in the same old way was even more dangerous. They closed the meeting with the words, "All right, we'll do it."

Implementation of the New Product Line got under way in the spring of 1962. Considerable emphasis was placed on commercial secrecy. For example, the project was blandly known as NPL (for New Product Line), and each of the five planned processors had a misleading code number – 101, 250, 315, 400, and 501 – that gave no hint of a unified product line and in some cases was identical to the model numbers of competitive machines from other manufacturers; even if the code numbers leaked, they would merely confuse the competition.

The New Product Line was one of the largest civilian R&D projects ever undertaken. Until the early 1980s, when the company began to loosen up somewhat, the story of its development was shrouded in secrecy. Only one writer succeeded in getting past the barrier of IBM's press corps, the *Fortune* journalist Tom Wise. He coined the phrase "IBM's \$5 billion gamble" and wrote "not even the Manhattan Project which produced the atomic bomb in World War II cost so much"; this sounded like hyperbole at the time, but Wise's estimate was about right.¹⁰ Wise reported that one of the senior managers "was only half joking when he said: 'We called this project "You bet your company."'" It is said that IBM rather liked the swashbuckling image that Wise conveyed, but when his articles went on to recount in detail the chaotic and irrational decision-making processes at IBM, Watson Jr. was livid and "issued a memorandum suggesting that the appearance of the piece should serve as a lesson to everyone in the company to remain uncommunicative and present a unified front to the public, keeping internal differences behind closed doors."¹¹

The logistics of the research program were awesome. Of the five planned computer models, the three largest were to be developed at IBM's main design facility in Poughkeepsie, New York, the smallest in its Endicott facility in upstate New York, and the fifth machine in its Hursley Development Laboratories in England. Simply keeping the machine designs compatible between the geographically separate design groups was a major problem. Extensive telecommunications facilities were used to keep the development groups coordinated, including two permanently leased transatlantic lines – an unprecedented expense in civilian R&D projects at the time. In New York hundreds and eventually thousands of programmers worked on the software for the New Product Line.

All told, direct R&D came to around \$500 million. But ten times as much again was needed for implementation – to tool up the factories, retrain marketing staff, and re-equip field engineers. One of the major costs was for building up a capability in semiconductor manufacturing, in which IBM had previously been weak. Tom Watson Jr., who had to cajole his board of directors into sanctioning the expenditure, recalled:

Ordinary plants in those days cost about forty dollars per square foot. In the integrated circuit plant, which had to be kept dust free and looked more like a surgical ward than a factory floor, the cost was over one hundred and fifty dollars. I could hardly believe the bills that were coming through, and I wasn't the only one who was shocked. The board gave me a terrible time about the costs. "Are you really sure you need all this?" they'd say. "Have you gotten competitive bids? We don't want these factories to be luxurious."¹²

This investment put IBM into the position of being the world's largest manufacturer of semiconductors.

By late 1963, with development at full momentum, top management began to turn its thoughts to the product launch. The compatible range had now been dubbed System/360, a name "betokening all points of the compass"¹³ and suggesting the universal applicability of the machines. The announcement strategy was fraught with difficulty. One option was to make a big splash by announcing the entire series at once, but this carried the risk that customers would cancel their orders for existing products and IBM would be left with nothing to sell until the new range came on stream. A safer and more conventional strategy would be to announce one machine at a time over a period of a couple of years. This would enable the switch from the old machines to the new to be achieved much more gently, and it was, in effect, how IBM had managed the transition from its old punched-card machines to computers in the 1950s.

However, all internal debate about the announcement strategy effectively ceased in December 1963 when Honeywell announced its model 200 computer. The Honeywell 200 was the first machine to challenge IBM by aggressively using the concept of IBM compatibility. The Honeywell computer was compatible with IBM's model 1401, but by using more up-to-date semiconductor technology it was able to achieve a price/performance superiority of as much as a factor of four. Because the machine was compatible with the 1401, it was possible for one of IBM's customers to return an existing rented machine to IBM and acquire a more powerful model from Honeywell for the same cost – or a machine of the same power for a lesser cost. Honeywell 200s could run IBM programs without reprogramming and, using a provocatively named "liberator" program, could speed up existing 1401 programs to make full use of the Honeywell 200's power.

It was a brilliant strategy and a spectacular success: Honeywell took 400 orders in the first week following the announcement, more than it had taken

in the previous eight years of its computer operation's existence. The arrival of the Honeywell 200 produced a noticeable falloff in IBM 1401 orders, and some existing users began to return their 1401s to switch over to the new Honeywell machine. Inside IBM it was feared that as many as three-quarters of its 1401 users would be tempted by the Honeywell machine.

Despite all the planning done on System/360, IBM was still not irrevocably committed to the New Product Line, and the arrival of the Honeywell 200 caused all the marketing plans to be worked over once again. At this, the eleventh hour, there seemed to be two alternatives: to carry on with the launch of the entire System/360 range and hope that the blaze of publicity and the implied obsolescence of the 1401 would eclipse the Honeywell 200; or to abandon System/360 and launch a souped-up version of the 1401 – the 1401S – that was already under development and that would be competitive with the Honeywell machine.

On 18–19 March a final make-or-break decision was made in a protracted risk-assessment session with Watson Jr., IBM's president Al Williams, and the thirty top IBM executives:

At the end of the risk assessment meeting, Watson seemed satisfied that all the objections to the 360 had been met. Al Williams, who had been presiding, stood up before the group, asked if there were any last dissents, and then, getting no response, dramatically intoned, "Going . . . going . . . gone!"¹⁴

Activity in IBM now reached fever pitch. Three weeks later the entire System/360 product line was launched. IBM staged press conferences in sixty-three cities in the United States and fourteen foreign countries on the same day. In New York a chartered train conveyed two hundred reporters from Grand Central Station to IBM's Poughkeepsie plant, and the visitors were shown into a large display hall where "six new computers and 44 new peripherals stretched before their eyes." Tom Watson Jr. – gray-haired, but a youthful not-quite fifty, and somewhat Kennedyesque – took center stage to make "the most important product announcement in company history," IBM's third-generation computer, System/360.¹⁵

The computer industry and computer users were stunned by the scale of the announcement. While an announcement from IBM had long been expected, its tight security had been extremely effective, so that outsiders were taken aback by the decision to replace the entire product line. Tom Wise, the *Fortune* reporter, caught the mood exactly when he wrote:

The new System/360 was intended to obsolete virtually all other existing computers. . . . It was roughly as though General Motors had decided to scrap its existing makes and models and offer in their place one new line of cars, covering the entire spectrum of demand, with a radically redesigned engine and an exotic fuel.¹⁶

This “most crucial and portentous – as well as perhaps the riskiest – business judgment of recent times” paid off handsomely. Literally thousands of orders for System/360 flooded in, far exceeding IBM’s ability to deliver; in the first two years of production, it was able to satisfy less than half of the nine thousand orders on its books. To meet this burgeoning demand, IBM hired more marketing and production staff and opened new manufacturing plants. Thus in the three years following the System/360 launch, IBM’s sales and leasing revenue soared to over \$5 billion, and its employee head count increased by 50 percent, taking it to nearly a quarter of a million workers.

System/360 has been called “the computer that IBM made, that made IBM.” The company did not know it at the time, but System/360 was to be its engine of growth for the next thirty years. In this respect, the new series was to be no more and no less important to IBM than the classic 400 series accounting machines that had fueled its growth from the early 1930s until the computer revolution of the early 1960s.

The Dwarfs Fight Back

In the weeks that followed the System/360 announcement, IBM’s competitors began to formulate their responses. System/360 had dramatically reshaped the industry around the concept of a software-compatible computer range. Although by 1964 the compatible-range concept was being explored by most manufacturers, IBM’s announcement effectively forced the issue.

Despite IBM’s dominance of the mainframe computer industry, it was still possible to compete with the computer giant simply by producing better products, which was by no means impossible. In the mythology of computer history, System/360 has often been viewed as a stunning technical achievement and “one of this country’s greatest industrial innovations”¹⁷ – a view that IBM naturally did all it could to foster. But in technological terms, System/360 was no more than competent. The idea of a compatible range was not revolutionary at all, but was well understood throughout the industry; and IBM’s implementation of the concept was conservative and pedestrian. For example, the proprietary electronics technology that IBM had chosen to use, known as Solid Logic Technology (SLT), was halfway between the discrete transistors used in second-generation computers and the true integrated circuits of later machines. There were good, risk-averse decisions underlying IBM’s choice, but to call System/360 the “third generation” was no more than a marketing slogan that stretched reality.

Perhaps the most serious design flaw in System/360 was its failure to support time-sharing – the fastest-growing market for computers – which enabled a machine to be used simultaneously by many users. Another problem was that the entire software development program for System/360 was little short of a fiasco (as we shall see in Chapter 8) in which thousands of programmers consumed over \$100 million to produce software that had many shortcomings. IBM was not so cocooned from the real world that it was unaware of what poor value it

had derived from the \$500 million System/360 research costs. As Tom Watson Jr. caught wind of the new computer's failings, he personally demanded an engineering audit, which fully confirmed his suspicion that IBM's engineering was mediocre.

But as always, technology was secondary to marketing in IBM's success – and there was no question of trying to compete with IBM in marketing. Hence, to compete with IBM at all, the seven-dwarf competitors had to aim at one of IBM's weak spots.

The first and most confrontational response was to tackle IBM head-on by making a 360-compatible computer with superior price performance – just as Honeywell had done with its 1401-compatible model 200. RCA, one of the few companies with sufficient financial and technical resources to make the attempt, decided on this strategy. Indeed, two years before the System/360 announcement, RCA had committed \$50 million to produce a range of IBM-compatible computers. RCA had deliberately kept its plans flexible so that it would be able to copy whatever architecture and instruction code IBM finally adopted, which it believed “would become a standard code for America and probably for the rest of the world.”¹⁸

Right up to the launch of System/360 in April 1964, RCA product planners had no inkling either of the date of the announcement or of the eventual shape of System/360. There was no industrial espionage, and it was only on the day of the announcement that they got their first glimpse of the new range. Within a week they had decided to make the RCA range fully System/360-compatible and, working from IBM's publicly available manuals, “started to copy the machine from the skin in.”¹⁹ In order to compete with IBM, RCA needed a 10 or 15 percent advantage in price/performance; this it planned to achieve by using its world-leading capability in electronic components to produce a series that employed fully integrated circuits in place of the SLT technology used by IBM. This would make the RCA machines – dollar for dollar – smaller, cheaper, and faster than IBM's. RCA announced its range as Spectra 70 in December 1964 – some eight months after the System/360 launch.

RCA was the only mainframe company to go IBM-compatible in a big way. The RCA strategy was seen as very risky because of the potential difficulty involved in achieving a superior price/performance relative to IBM, given IBM's massive economies of scale. IBM compatibility also put the competitor in the position of having to follow slavishly every enhancement that IBM made to its computer range. Even worse, IBM might drop System/360 altogether one day.

Hence the second and less risky strategy to compete with System/360 was product differentiation – that is, to develop a range of computers that were software-compatible with one another but not compatible with System/360. This was what Honeywell did. Taking its highly successful model 200, it extended the machine up and down the range to make a smaller machine, the model 120, and four larger machines, the 1200, 2200, 4200, and 8200, which were announced

between June 1964 and February 1965. This was a difficult decision for Honeywell, since it could be seen as nailing its flag to the mast of an obsolete architecture. However, Honeywell had the modest ambition of capturing just 10 percent of IBM's existing 1401 customer base – about one thousand installations in all – which it comfortably did. The same strategy, of producing non-IBM-compatible ranges, was also adopted by Burroughs with its 500 series, announced in 1966, and by NCR with its Century Series in 1968.

The third strategy to compete with IBM was to aim for niche markets that were not well satisfied by IBM and in which the supplier had some particular competitive advantage. Control Data, for example, recognized that the System/360 range included no really big machines, and so decided to give up manufacturing regular mainframes altogether and make only giant number-crunching computers designed primarily for government and defense research agencies. Similarly, General Electric recognized IBM's weakness in time-sharing, and – in collaboration with computer scientists at Dartmouth College and MIT – quickly became the world leader in time-shared computer systems (see Chapter 9). In other cases, manufacturers capitalized on their existing software and applications experience. Thus UNIVAC continued to compete successfully with IBM in airline reservation systems, without any major machine announcements. Again, even though their computer products were lackluster, Burroughs and NCR were able to build on their special relationship with bankers and retailers, which dated from the turn of the century, when they had been selling adding machines and cash registers. It was the old business of selling solutions, not problems. With these strategies and tactics, all seven dwarfs survived into the 1970s – just.

BY THE END of the 1960s, IBM appeared invincible. It had about three-quarters of the market for mainframe computers around the world. Its seven mainframe competitors each had between 2 and 5 percent of the market; some were losing money heavily, and none of them was more than marginally profitable. IBM's extraordinarily high profits, estimated at 25 percent of its sales, enabled the other companies to shelter under its "price umbrella." It was often said that the seven dwarfs existed only by the grace of IBM, and if it were ever to fold its price umbrella, they would all get wet – or drown. This is what happened in the first computer recession of 1970–1971. As the journalists put it at the time, IBM sneezed and the industry caught a cold. Both RCA and General Electric, which had problems in their core electronics and electrical businesses associated with the global economic downturn of the early 1970s, decided to give up the struggle and terminate their in-the-red computer divisions. Their customer bases were bought up by Sperry Rand and Honeywell, respectively, which helped them gain market share at relatively little cost. From then on, the industry was no longer characterized as IBM and the seven dwarfs but, rather, as IBM and the BUNCH – standing for Burroughs, UNIVAC, NCR, Control Data, and Honeywell.

Notes to Chapter 6

The competitive environment of the computer industry between 1960 and 1975 is described in Gerald W. Brock's *The U.S. Computer Industry* (1975). Other useful sources are Kenneth Flamm's monographs *Targeting the Computer: Government Support and International Competition* (1987) and *Creating the Computer: Government, Industry and High Technology* (1988).

A history of the IBM 1401 is given in Charles J. Bashe et al.'s *IBM's Early Computers* (1986). The 1401's industrial design is described in John Harwood's *The Interface: IBM and the Transformation of Corporate Design 1945–1976* (2011). The technical development of the IBM 360 and 370 computer series is described in Emerson W. Pugh et al.'s *IBM's 360 and Early 370 Systems* (1991). Tom Wise's *Fortune* articles "I.B.M.'s \$5,000,000,000 Gamble" and "The Rocky Road to the Market Place" (1966a and 1966b) remain the best external account of the System/360 program. Good sources on the international mainframe industry include Kenneth Flamm's *Creating the Computer* (1988) and Paul Gannon's *Trojan Horses and National Champions* (1997). European anxiety over the American domination of the computer industry is captured well by Jean-Jacques Servan-Schreiber's *The American Challenge* (1968) and Simon Nora and Alain Minc's *The Computerization of Society* (1980). Good backgrounders on the Japanese mainframe industry are Robert Sobel's *IBM vs. Japan* (1986) and Keizo Kawata's *Fujitsu: The Story of a Company* (1996).

Notes

1. Bashe et al. 1986, p. 476.
2. *Ibid.*
3. Sheehan 1956, p. 114.
4. Burck 1964, p. 198.
5. Sobel 1981, p. 163.
6. Burck 1964, p. 202.
7. Quoted in Pugh et al. 1991, p. 119.
8. Watson 1990, p. 348.
9. Wise 1966a, p. 228.
10. *Ibid.*, pp. 118–9.
11. Rodgers 1969, p. 272.
12. Watson 1990, p. 350.
13. Pugh et al. 1991, p. 167.
14. Wise 1966b, p. 205.
15. *Ibid.*, p. 138.
16. Wise 1966a, p. 119.
17. Quoted in DeLamarter 1986, p. 60.
18. Quoted in Campbell-Kelly 1989, p. 232.
19. Fishman 1981, p. 182.

Part Three

Innovation and Expansion



Taylor & Francis

Taylor & Francis Group

<http://taylorandfrancis.com>

7 Real Time

Reaping the Whirlwind

MOST OF THE computer systems installed in commerce and government during the 1950s and 1960s were used in a pedestrian way. They were simply the electronic equivalents of the punched-card accounting machines they replaced. While computers were more glamorous than punched-card machines, giving an aura of modernity, they were not necessarily more efficient or cheaper. For many organizations, computers did not make any real difference; data-processing operations would have gone on much the same without them. Where computers made a critical difference was in real-time systems.

A *real-time* system is one that responds to external messages in “real time,” which usually means within seconds, although it can be much faster. No previous office technology had been able to achieve this speed, and the emergence of real-time computers had the potential for transforming business practice.

Project Whirlwind

The technology of real-time computing did not come from the computer industry. Rather, the computer industry appropriated a technology first developed for a military purpose in MIT’s Project Whirlwind – a project that, in turn, grew out of a contract to design an “aircraft trainer” during World War II.

An aircraft trainer was a system to speed up the training of pilots to fly new aircraft. Instead of learning the controls and handling characteristics of an airplane by actually flying one, a pilot would use a simulator, or trainer: a mock-up of a plane’s cockpit, with its controls and instruments, attached to a control system. When the pilot “flew” the simulator, the electromechanical control system fed the appropriate data to the aircraft’s instruments, and a series of mechanical actuators simulated the pitch and yaw of an aircraft. Sufficient realism was attained to train a pilot far more cheaply and safely than using a real airplane.

A limitation was that a different simulator had to be built for each type of aircraft. In the fall of 1943 the Special Devices Division of the US Bureau of Aeronautics established a project to develop a universal flight trainer that would be suitable for simulating any type of aircraft. This could be achieved only with

a much more sophisticated computing system. A contract for an initial feasibility study was given to the Servomechanisms Laboratory at MIT.

MIT's Servomechanisms Laboratory was the leading center for the design of military computing systems in the United States. It had been established in 1940 as part of the war effort to develop sophisticated electromechanical control and computing devices for use in fire control (that is, gun aiming), bomb aiming, automatic aircraft stabilizers, and so on. By 1944 the laboratory had a staff of about a hundred people working on various projects for the military. An assistant director of the laboratory, Jay W. Forrester, was placed in charge of the trainer project. Then only twenty-six, Forrester showed an outstanding talent for the design and development of electromechanical control equipment.

Forrester was a very resilient individual who, for the next decade, drove his research project forward in the face of a hostile military establishment that justifiably balked at its cost. Many of the early computer projects ran far over their original cost and time estimates, but none so much as the Whirlwind. It changed its objectives midway, it took eight years to develop instead of the projected two, and it cost \$8 million instead of the \$200,000 originally estimated. In a more conservative and rational research environment, the project might have been terminated. The project did manage to survive and had an impact that eventually did justify the expenditure. Apart from the SAGE and SABRE projects (to be described later), Project Whirlwind established real-time computing and helped lay the foundations of the computer industry around Boston, Massachusetts – later known as the “Route 128” computer industry. In 1944, however, all this lay in the unforeseeable future.

For the first few months of 1945, Forrester worked virtually alone on the project, drawing up the specification of the new-style trainer. He envisioned a simulator with an aircraft cockpit with conventional instruments and controls harnessed to an analog computing system that, by means of a hydraulic transmission system, would feed back the correct operational behavior of the plane being modeled. The cockpit and instruments appeared to be relatively straightforward; and although the problem of the analog computer was much more challenging, it was a proven technology and did not appear insurmountable.

On the basis of Forrester's initial study, MIT proposed in May 1945 that the trainer project be undertaken by the Servomechanisms Laboratory, at a cost of some \$875,000 over a period of eighteen months. In the context of the war with Japan – whose end was still four months in the future – \$875,000 and eighteen months seemed like a sound investment, and MIT was given the go-ahead. During the following months, Forrester began to recruit a team of mechanical and electrical engineers. To serve as his assistant he recruited a high-flying MIT graduate student, Robert R. Everett (later to become president of the MITRE Corporation, one of the United States' major defense contractors). Over the next decade, while Forrester retained overall administrative and technical leadership,

Everett assumed day-to-day responsibilities as Forrester became increasingly absorbed with financial negotiations.

As the engineering team built up, Forrester put into motion the development of the more straightforward mechanical elements of the aircraft trainer, but the problem of the computing element remained stubbornly intractable. However much he looked at it, what remained clear was that an analog computer would not be nearly fast enough to operate the trainer in real time.

Seeking a solution to the control problem, Forrester found time in his busy schedule to survey the computing field. In the summer of 1945 he learned for the first time about digital computers from a fellow MIT graduate student, Perry Crawford. Crawford was perhaps the first to appreciate real-time digital control systems, long predating the development of practical digital computers. As early as 1942 he had submitted a master's thesis on "Automatic Control by Arithmetic Operations," which discussed the use of digital techniques for automatic control. Up to that date, control systems had always been implemented using mechanical or electromechanical analog technologies – techniques used on hundreds of control devices for World War II weapons. The characteristic shared by all these devices was that they used some physical analog to model a mathematical quantity.

For example, an array of discs and gear wheels could be used to perform the mechanical integration of the differential equations of motion, or a "shaped resistor" could be used to model a mathematical function electrically. By contrast, in a digital computer, mathematical variables were represented by numbers, stored in digital form. Crawford was the first person to fully appreciate that such a general-purpose digital computer would be potentially faster and more flexible than a dedicated analog computer. All this Crawford explained to Forrester while the two were deep in conversation on the steps in front of the MIT building on 77 Massachusetts Avenue. As Forrester later put it, Crawford's remarks "turned on a light in [my] mind."¹ The course of Project Whirlwind was set.

With the end of the war in August 1945, the digital computation techniques that had been developed in secret during the war began to diffuse into the civilian arena. In October, the National Defense Research Committee organized a Conference on Advanced Computational Techniques at MIT, which was attended by both Forrester and Crawford. It was here that Forrester learned in detail of the "Pennsylvania Technique" of electronic digital calculation used in the ENIAC, and in its successor, the EDVAC. When the Moore School ran its summer school the following year, Crawford gave a lecture² on the real-time application of digital computers, and Everett was enrolled as a student.

Meanwhile, the trainer project had acquired its own momentum, even though the computing problem remained unresolved. As was typical of university projects, Forrester staffed the project with young graduate engineers and gave them considerable independence. This produced an excellent esprit de corps within the laboratory – but outside, the result was a reputation for arrogance and an

impression that the project was a “gold-plated boondoggle.”³ One of Forrester’s Young Turks, Kenneth Olsen – subsequently a founder and president of the Digital Equipment Corporation – later remarked, “Oh we were cocky! We were going to show everybody! And we did.”⁴

By early 1946 Forrester had become wholly convinced that the way forward was to follow the digital path. Even though this would result in a major cost escalation, he believed the creation of a general-purpose computer for the aircraft trainer would permit the “solution to many scientific and engineering problems other than those associated with aircraft flight.”⁵

In March, Forrester made a revised proposal to the Special Devices Division of the Bureau of Aeronautics requesting that his contract be renegotiated to add the development of a full-scale digital computer, at a projected cost of \$1.9 million, tripling the budget for the overall project. By laying meticulous technical groundwork and cultivating the support of his superiors at MIT, Forrester succeeded in selling the revised program – now formally named Project Whirlwind – to the Bureau.

Forrester’s whole laboratory now became reoriented to the design and development of a digital computer, Whirlwind. There was a total of about a hundred staff members, working in ten groups and costing \$100,000 a month to support. One group under Everett was at work on the “block diagrams” (or computer architecture, as we would now call it), another group was at work on electronic digital circuits, two groups were engaged in storage systems, another group worked on mathematical problems, and so on.

As did everyone leading an early computer project, Forrester soon discovered that the single most difficult problem was the creation of a reliable storage technology. The most promising storage technology was the mercury delay-line memory, on which many other centers were working. But this system would be far too slow for the Whirlwind. Whereas mathematical laboratories were working on computers that would run at speeds of 1,000 or 10,000 operations per second, the MIT machine, if it was successfully to fulfill its real-time function, needed to run ten or a hundred times faster – on the order of 100,000 operations per second.

Forrester therefore tried another promising technology – the electrostatic storage tube – which was a modified form of TV tube in which each bit of information was stored as an electrical charge. Unlike the mercury delay-line memory, which used sonic pulses, the electrostatic memory could read and write information at electronic speeds of a few microseconds. In 1946, however, electrostatic storage was no more than a blue-sky idea, and a long and expensive research project lay ahead. Moreover, storage was just one component – albeit the most difficult – of the whole system. As the magnitude of the digital computer project began to dawn, the original flight trainer aspect receded into the background. As the official historians of Project Whirlwind later put it, “the tail had passed through and beyond the point of wagging the dog and had become the dog.”⁶

The years 1946 and 1947 passed fruitfully in developing and testing the Whirlwind computer's basic components, but progress appeared less dramatic to the outside world. Forrester's cozy relationship with the Bureau of Aeronautics was coming to an end. In the immediate postwar period, military funding had been reorganized, and Project Whirlwind now came under the remit of the Mathematics Branch of the Office of Naval Research (ONR), which took over the Special Devices Division of the Bureau of Aeronautics. While expensive crash projects had been normal in time of war, a new value-for-money attitude had accompanied the shrinking defense budgets of the postwar environment. From late 1947 Forrester found himself under increasing scrutiny from the ONR, and several independent inquiries into the cost-effectiveness of Project Whirlwind and its scientific and engineering competence were commissioned. The Mathematics Branch was headed by Mina S. Rees, a mathematician who was then and later one of the United States' foremost scientific administrators. It was the role of her branch to reflect the "consensus of visitors to the project" that there was "too much talk and not enough machine."⁷ Forrester tended to shield his team from these political and financial problems, as he "saw no reason to allocate time for his engineers to stand wringing their hands."⁸

Fortunately Forrester was blessed with one key ally in the ONR: Perry Crawford. Crawford had joined the ONR as a technical adviser in October 1945, and his vision for real-time computing was undiminished. Indeed, if anything, it had widened even beyond that held by Forrester. Crawford foresaw a day when real-time computing would control not just military systems but whole sectors of the civilian economy – such as air traffic control.

Meanwhile, Project Whirlwind was consuming 20 percent of ONR's total research budget, and its cost was far out of line with the other computer projects it was funding. The ONR could not understand why, when all the other computer groups were building computers with teams of a dozen or so engineers and budgets of half a million dollars or less, Forrester needed a staff of over a hundred and a budget approaching \$3 million.

There were two reasons for Whirlwind's much higher costs. First, it needed to be an order of magnitude faster than any other computer to fulfill its real-time function. Second, it needed to be engineered for reliability because breakdowns that would cause no more than a minor irritation in a routine mathematical computation were unacceptable in a real-time environment. Unfortunately, the people in ONR who were responsible for computer policy were primarily mathematicians; while they appreciated the utility of computers for mathematical work, they had not yet been convinced of their potential for control applications. By the summer of 1948 the ONR's confidence in Forrester and Project Whirlwind was ebbing fast. At the same time, Forrester's main ally, Perry Crawford, with his "fearless and imaginative jumps into the future,"⁹ was under a cloud and had been moved to another assignment, leaving Forrester to handle the ONR as best he could.

A major confrontation occurred in the fall of 1948 when Forrester requested a renewal of his ONR contract at a rate of \$122,000 a month. This was met by a counteroffer of \$78,000 a month. Forrester's reaction was outrage that the project should be trimmed when he believed it was on the threshold of success. Moreover, he now saw Project Whirlwind simply as a stepping stone to a major military information system, which would take perhaps fifteen years to develop at a cost of \$2 billion. Forrester argued that Whirlwind was a national computer development of such self-evident importance that the costs, however great, were immaterial. On this occasion, bravado and brinkmanship won the day, but a further crisis was not long in coming.

Forrester may have half-convinced the ONR that Whirlwind was on the threshold of success, but by no means were all the technical issues resolved – especially the storage problem. The electrostatic storage tubes had proved expensive and their storage capacity was disappointing. Forrester, always on the lookout for an alternative memory technology, was struck by an advertisement he saw for a new magnetic ceramic known as “Deltamax.” In June 1949 he made an entry in his notebook sketching out the idea for a new kind of memory, and he quietly set to work. A former graduate student in the laboratory later recalled: “Jay took a bunch of stuff and went off in a corner of the lab by himself. Nobody knew what he was doing, and he showed no inclination to tell us. All we knew was that for about five or six months or so, he was spending a lot of time off by himself working on something.”¹⁰ The following fall Forrester handed the whole project over to a young graduate student named Bill Papian and awaited results.

By late 1949 inside ONR confidence in Project Whirlwind had reached its lowest point. Forrester's high-flown vision of national military information systems was judged to be both “fantastic” and “appalling.”¹¹ Another critic found the Whirlwind to be “unsound on the mathematical side” and “grossly over-complicated technically.” A high-level national committee found the Whirlwind to be lacking “a suitable end-use” and asserted that if the ONR could not find one, then “further expenditure on the machine should be stopped.” The ONR had all the evidence he needed, and in 1950 it reviewed Project Whirlwind's budget. Forrester was offered approximately \$250,000 rather than the requested \$1.15 million for fiscal 1951 to bring Whirlwind to a working state. Fortunately, as one door shut another had already begun to open.

The SAGE Defense System

In August 1949 US intelligence revealed that the Russians had exploded a nuclear bomb and that they possessed bomber aircraft capable of carrying such a weapon over the North Pole and into the United States. America's defenses against such an attack were feeble. The existing air-defense network, which was essentially left over from World War II, consisted of a few large radar stations where information was manually processed and relayed to a centralized

command-and-control center. The greatest weakness of the system was its limited ability to process and make operational use of the information it gathered. There were communication difficulties at all levels of the system, and there were gaps in radar coverage, so that, for example, no early warning of an attack coming from over the North Pole could be provided.

The heightened Cold War tension of the fall of 1949 provided the catalyst to get the nation's air-defense system reviewed by the Air Force's Scientific Advisory Board. A key member of this board was George E. Valley, a professor of physics at MIT. Valley made his own informal review of the air-defense system that confirmed its inadequacy. In November 1949 he proposed that the Board create a subcommittee with the remit of recommending the best solution to the problem of air defense.

This committee, the Air Defense System Engineering Committee (ADSEC), came into being the following month. Known informally as the Valley Committee after its originator and chairman, it made its first report in 1950, describing the existing system as "lame, purblind, and idiot-like."¹² It recommended upgrading the entire air-defense system to include updated interceptor aircraft, ground-to-air missiles, and antiaircraft artillery, together with improved radar coverage and computer-based command-and-control centers.

At the beginning of 1950, Valley was actively seeking computer technology to form the heart of a new-style command-and-control center when an MIT colleague pointed him in the direction of Forrester and Project Whirlwind. Valley later recalled: "I remembered having heard about a huge analog computer that had been started years before; it was an enterprise that I had carefully ignored, and I had been unaware that the Forrester project had been transformed into a digital computer."¹³ As Valley asked around, the reports he received about Project Whirlwind "differed only in their degree of negativity."

Valley decided to find out for himself and made a visit to Project Whirlwind, where Forrester and Everett showed him all he wanted to see. By a stroke of good fortune, the Whirlwind had just started to run its first test programs – stored in the available twenty-seven words of electrostatic memory. Valley was able to see the machine calculate a freshman mechanics problem and display the solution on the face of a cathode-ray tube. This demonstration, combined with Forrester and Everett's deep knowledge of military information systems, swayed him toward Whirlwind – though in truth there was no other machine to be had.

At the next Project Whirlwind review meeting with the ONR on 6 March 1950, George Valley made an appearance. He stated his belief that the Air Force would be willing to allocate some \$500,000 to the project for fiscal 1951. This independent commendation of the Whirlwind transformed the perception of Forrester's group within the ONR, and the criticism began to disappear. Over the next few months further Air Force monies were found, enabling the project to proceed on the financial terms Forrester had always insisted upon. A few months after, Whirlwind became the major component of a much larger project, Project

Lincoln – later the Lincoln Laboratory – established at MIT to conduct the entire research and development program for a computerized national air-defense system. It would take another decade for the resulting air-defense system – the Semi-Automatic Ground Environment (SAGE) – to become fully operational.

By the spring of 1951 the Whirlwind computer was an operational system. The only major failing was the electrostatic storage tubes, which continued to be unreliable and had a disappointing capacity. The solution to this problem finally came into sight at the end of 1951 when Bill Papian demonstrated a prototype core-memory system. Further development work followed, and in the summer of 1953 the entire unreliable electrostatic memory was replaced by core memory with an access time of nine microseconds – making Whirlwind by far the fastest computer in the world and also the most reliable.

Papian was not the only person working on core memory – there were at least two other groups working on the idea at the same time – but the “MIT core plane” was the first system to become operational. In five years’ time, core memory would replace every other type of computer memory and net MIT several million dollars in patent royalties. The value to the nation of the core-memory spin-off alone could be said to have justified the cost of the entire Whirlwind project.

With the installation of the core memory and the consequent improvement in speed and reliability, Whirlwind was effectively complete. The research and development phase was over, and now production in quantity for Project SAGE had to begin. Gradually the technology was transferred to IBM, which transformed the prototype Whirlwind into the IBM AN/FSQ-7 computer. In 1956 Forrester left active computer engineering to become an MIT professor of industrial and engineering organization in the Sloan School of Management; he subsequently became well known to the public for his computer-based studies of the global environment, published as *World Dynamics* in 1971.

The SAGE system that was eventually constructed consisted of a network of twenty-three Direction Centers distributed throughout the country. Each Direction Center was responsible for air surveillance and weapons deployment in a single sector – typically a land/sea area of several thousand square miles. At the heart of each Direction Center were two IBM AN/FSQ-7 computers, which were “duplexed” for reliability – one machine operating live and the other in standby mode. With 49,000 tubes and weighing 250 tons, the computer was the largest ever put into service.¹⁴ Feeding the computer were some one hundred sources of data coming into the Direction Center. These sources included ground-based and ship-based radar installations, weapons and flight bases, on-board missile and airplane radar, and other Direction Centers. Within the Direction Center over one hundred air force personnel monitored and controlled the air defense of their sector. Most of these people sat at consoles that displayed surveillance data on CRT screens – giving the positions and velocities of all aircraft in their area, while suppressing irrelevant information. By interrogating the console, the

operator could determine the identity of aircraft and weapons – whether civilian or military, friend or foe – and make tactical plans based on the use of computer-stored environmental and weather data, flight plans, and weapons characteristics.

From a strictly military point of view, however, the complete SAGE system, fully deployed by 1963 at an estimated cost of \$8 billion, was an “expensive white elephant.”¹⁵ The original purpose of defending the United States from bomber-borne nuclear weapons had been undercut by the arrival of the new technology of Intercontinental Ballistic Missiles (ICBMs) in the 1960s. The ICBMs made bombers a second-order threat, although still enough of a concern to warrant continuing support for the SAGE system until the early 1980s, when it was finally decommissioned.

The real contribution of SAGE was thus not to military defense but, rather, through technological spin-off to civilian computing.¹⁶ An entire subindustry was created as industrial contractors and manufacturers were brought in to develop the basic technologies and implement the hardware, software, and communications. These industrial contractors included IBM, Burroughs, Bell Laboratories, and scores of smaller firms. Many of the technological innovations that were developed for SAGE were quickly diffused throughout the computer industry. For example, the development of printed circuits, core memories, and mass-storage devices – some of the key enabling technologies for the commercial exploitation of computers in the 1960s – was greatly accelerated by SAGE. The use of CRT-based graphical displays created not just a novel technology but a whole culture of interactive computing that the rest of the world did not catch up with until the 1970s. American companies also gained a commanding lead in digital communications technologies and wide-area networks. Finally, the SAGE system provided a training ground for software engineering talent: some 1,800 programmer years of effort went into the SAGE software, and as a result “the chances [were] reasonably high that on a large data-processing job in the 1970s you would find at least one person who had worked with the SAGE system.”¹⁷

Of the contractors, IBM was undoubtedly the major beneficiary. As Tom Watson Jr. put it, “It was the Cold War that helped IBM make itself the king of the computer business.”¹⁸ The SAGE project accounted for half a billion dollars of IBM’s income in the 1950s, and at its peak between seven thousand and eight thousand people were working on the project – some 20 percent of IBM’s workforce. IBM obtained a lead in processor technology, mass-storage devices, and real-time systems, which it soon put to commercial use.

SABRE: A Revolution in Airline Reservations

The first major civilian real-time project to draw directly on IBM’s know-how and the technology derived from the SAGE project was the SABRE airline reservations system developed for American Airlines. SABRE, and systems like it,

transformed airline operations and helped create the modern airline as we know it. SABRE would also usher in a host of real-time applications such as credit cards, ATMs, and barcodes, discussed in the final section of this chapter.

To understand the impact of real-time reservations systems on the airlines, one first needs to appreciate how reservations were handled before computers came on the scene. If during the 1940s or 1950s you had managed to gain access to the reservations office of a major commercial airline, you would have been confronted by an awesome sight, one reminiscent of a war operations room. At the center of the room, amid the noisy chattering of teletype machines, about sixty reservations clerks would be dealing with incoming telephone calls at the rate of several thousand a day. At any one moment, each reservations clerk would be responding to one or more of three general types of inquiry from a customer or travel agent: requests for information about flight availability, requests to reserve or cancel seats on a particular flight, and requests to purchase a ticket. To deal with any of these requests, the reservations clerks would have to refer to a series of well-lit boards displaying the availability of seats on each flight scheduled to depart over the next few days. For flights further ahead in time, the agent would have to walk across the room to consult a voluminous card file.

If the inquiry resulted in a reservation, a cancellation, or a ticket sale, the details of the transaction would be recorded on a card and placed in an out-tray. Every few minutes these cards would be collected and taken to the designated availability-board operator, who would then adjust the inventory of seats available for each flight. Once a ticket was sold and the availability board had been updated, the sales information found its way to the back office, where another forty or so clerks maintained passenger information and issued tickets. The whole process was altogether more complicated if a trip involved connecting flights; these requests came and went out via teletypes to similar reservations offices in other airlines. The day or two before the departure of a flight produced a flurry of activity as unconfirmed passengers were contacted, last-minute bookings and cancellations were made, and a final passenger list was sent to the departure gate at the airport.

Although tens of thousands of reservations were processed in this manner every day, in reservations offices at several different airports, there was almost no mechanization. The reason for this largely manual enterprise – it could have been run on almost identical lines in the 1890s – was that all the existing data-processing technologies, whether punched-card machines or computers, operated in what was known as batch-processing mode. Transactions were first batched in hundreds or thousands, then sorted, then processed, and finally totaled, checked, and dispatched. The primary goal of batch processing was to reduce the cost of each transaction by completing each subtask for all transactions in the batch before the next subtask was begun. As a result the time taken for any individual transaction to get through the system was at least an hour, but more typically half a day. For most businesses, batch processing was

a cost-effective solution to the data-processing problem and had been employed since the 1920s and 1930s: it was how banks processed checks, insurance companies issued policies, and utility companies invoiced their customers. As computers replaced punched-card machines, they, too, were used in batch-processing mode. However, batch processing – whether on punched-card machines or computers – could not address the instantaneous processing needs of an airline reservations system.

The fact that airline reservations were done manually was not a problem until the 1950s. Up to then, air travel was viewed by the general public as a luxury purchase and so was not particularly price-sensitive. Airlines could still turn a profit operating with load factors – that is, the proportion of seats occupied by fare-paying passengers – that were well below capacity.

However, sweeping changes were becoming apparent in the airline market in the mid-1940s. In 1944, when American Airlines first decided to automate its reservations operation, it was approaching the limit of what was possible using purely manual methods. As the volume of business increased and more and more flights were added to the availability boards, the boards became more crowded and the airline's reservations clerks had to sit farther and farther away from them – sometimes even resorting to binoculars. In an effort to resolve this problem, the airline commissioned an equipment manufacturer, Teleregister, to develop an electromechanical system that would maintain an inventory of the number of seats available on each flight. Using this system, called the Reservisor, a reservations clerk could determine the availability of a seat on a particular flight using a specially designed terminal, eliminating the need for the availability boards.¹⁹

The Reservisor brought considerable benefits, enabling the airline to cope with two hundred extra flights in the first year of operation with twenty fewer clerks. However, a major problem was that the Reservisor was not directly integrated into the overall information system. Once a sale had been recorded by the Reservisor, the passenger and reservation records still had to be adjusted manually. Discrepancies inevitably arose between the manual system and the Reservisor, and it was estimated that about one in twelve reservations was in error. It was said that “most business travelers returning home on a Friday would have their secretaries book at least two return flights.”²⁰ The result of these errors led either to underselling a flight, which caused a financial loss, or overbooking it, with consequent damage to customer relations.

Further refinement led to the Magnetrone Reservisor installed at La Guardia Airport in New York in 1952. Capable of storing details of a thousand flights ten days ahead, the key advance of this new system was an arithmetic capability enabling a sales agent in the New York area to sell and cancel seats as well as simply determine availability. Even so, the Reservisor maintained nothing more than a simple count of the number of seats sold and available on each aircraft: passenger record keeping and ticket issuing still had to be done manually. As the manual part of the reservations operation continued to expand, American

Airlines began planning for a new reservations office occupying 30,000 square feet, with seating positions for 362 reservations clerks and a capacity for dealing with 45,000 telephone calls a day.

By 1953 American Airlines – which was the world’s largest airline – had reached a crisis. Increased traffic and scheduling complexity were making reservations costs insupportable. This situation was exacerbated by the advent of \$5 million jet airplanes. American Airlines was planning to buy thirty Boeing 707s (each with 112 passenger seats), which would reduce transcontinental flying time from ten to six hours. This was faster than the existing reservations system could process messages about a flight, making it pointless to update passenger lists for last-minute additions or no-shows. At the same time, the airline had to respond to increasing competition. The most effective way to increase profitability was by improving load factors.

American Airlines’ data-processing problems were thus high on the agenda of its president, C. R. Smith, when by chance he found himself sitting next to Blair Smith (unrelated), one of IBM’s senior salespeople, on a flight from Los Angeles to New York in the spring of 1953. Naturally, their conversation quickly turned to the airline reservations problem. Subsequently, Blair Smith was invited to visit the La Guardia reservations office, after which there was continued contact between the engineering and planning staffs of the two companies.

IBM was in fact well aware of the Reservisor system but had not shown any interest when the system was first developed in the 1940s. By 1950, however, Thomas Watson Jr. had begun to assume control of IBM and had determined that it would enter the airlines reservations market as well as similar “tough applications, such as Banks, Department Stores, Railroads, etc.”²¹ In 1952 the company had hired Perry Crawford away from the Office of Naval Research, and he was to play a key role in initial real-time developments; as early as the summer of 1946 Crawford had been publicly advocating the development of control systems for commercial airlines as well as the more obvious military applications. A small joint study group was formed, under the co-leadership of Crawford, to explore the development of a computerized real-time reservations system that would capitalize on IBM’s SAGE experience.

In June 1954 the joint study team made its initial report, noting that while a computerized reservations system was a desirable goal in the long run, it would be some years before cost-effective technology became available. The report stated that, in the short term, the existing record-keeping system should be automated with conventional punched-card equipment. Thus an impressive, though soon-to-be-obsolete, new punched-card system and an improved Reservisor were brought into operation during 1955–1959. Simultaneously an idealized, integrated, computer-based reservations system was planned, based on likely future developments in technology. This later became known as a “technology intercept” strategy. The system would be economically feasible only when reliable solid-state computers with core memories became

available. Another key requirement was a large random-access disk storage unit, which at that time was a laboratory development rather than a marketable product. Although, in light of its SAGE experience, IBM was well placed to make informed judgments about emerging technologies and products, a technology-intercept approach would have been too risky without the backup of the parallel development of a system using conventional punched-card machine technology.

At the beginning of 1957, the project was formally established. During the next three years the IBM-American Airlines team prepared detailed specifications, while IBM technical advisers selected mainframes, disk stores, and telecommunications facilities, and a special low-cost terminal based on an IBM Selectric typewriter was developed for installation at over a thousand agencies. In 1960 the system was formally named SABRE – a name inspired by a 1960 advertisement for the Buick LeSabre – an automobile with tail fins and a streamlined aerodynamic aesthetic that shrieked modernity. SABRE bore a somewhat contrived acronym – Semi-Automatic Business Research Environment – which was later dropped, although the system continued to be known as SABRE, or Saber. The complete system specification was presented to American Airlines' management in the spring of 1960. The total estimated cost was just under \$40 million. After agreeing to go ahead, American Airlines' President Smith remarked: "You'd better make those black boxes do the job, because I could buy five or six Boeing 707s for the same capital expenditure."²²

The system was fully implemented during 1960–1963. The project was easily the largest civilian computerization task ever undertaken up to that time, involving some two hundred technical personnel producing a million lines of program code. The central reservations system, housed at Briarcliff Manor, about 30 miles north of New York City, was based on two IBM 7090 mainframe computers, which were duplexed for reliable operation. These processors in turn were connected to sixteen disk storage units with a total capacity of 800 million characters – the largest online storage system ever assembled up to that time. All told, through more than 10,000 miles of leased telecommunication lines, the system networked some 1,100 agents using desktop terminals in fifty cities throughout the United States. The system was capable of handling close to 10 million passenger reservations per year, involving "85,000 daily telephone calls – 30,000 daily requests for fare quotations – 40,000 daily passenger reservations – 30,000 daily queries to and from other airlines and 20,000 daily ticket sales."²³ Most transactions were handled in three seconds.

More important than the volume of transactions, however, was the dramatic improvement in the reservations service provided by the system. In place of the simple inventory of seats sold and available as provided by the original Reservisor, there was now a constantly updated and instantly accessible passenger-name record containing information about the passenger, including telephone

contacts, special meal requirements, and hotel and automobile reservations. The system quickly took over not just reservations but the airline's total operation: flight planning, maintenance reporting, crew scheduling, fuel management, air freight, and most other aspects of its day-to-day operations.

The system was fully operational in 1964, and by the following year it was stated to be amply recouping its investment through improved load factors and customer service. The project had taken ten years to implement. The development may have seemed leisurely from an external perspective, but an orderly, evolutionary approach was imperative when replacing the information system of such a massive business. In engineering terms it was the difference between a prototype and a product. All told, the system cost a reported \$30 million, and although disparagingly known as "the Kids' SAGE"²⁴ it was a massive project in civilian terms and not without risk.

What was innovation for one airline soon became a competitive necessity for the others. During 1960 and 1961 Delta and Pan American signed up with IBM, and their systems became operational in 1965. Eastern Airlines followed in 1968. Other alliances between computer manufacturers and airlines developed, some of which lacked the conservative approach of the IBM-American Airlines system and resulted in "classic data-processing calamities"²⁵ that took years to become operational. Overseas, the major international carriers began developing reservations systems in the mid-1960s. By the early 1970s all the major carriers possessed reliable real-time systems and communications networks that had become an essential component of their operations. Perhaps a single statistic gives a hint of the extraordinary developments that were to follow: by 1987 the world's largest airline reservations system, operated by United Airlines, made use of eight powerful IBM 3090 computers that processed over one thousand internal and external messages in a single *second*.

The airline reservations problem was unusual in that it was largely unautomated in the 1950s and so there was a strong motivation for adopting the new real-time technology. Longer-established and more traditional businesses were much slower to respond. But where the airlines pioneered, traditional businesses eventually followed. Among the first of these was the financial services industry with the introduction of credit cards and Automated Teller Machines (ATMs).

Credits Cards, ATMs, and Barcodes

There were two roots to the financial revolutions of credit cards and ATMs. First, the idea of "cashless" payment cards and, second, the development of cash-dispensing machines.

In 1949 two entrepreneurs formed the first independent payment card organization, Diners Club, signing up restaurants in New York City and soon expanding

to travel and entertainment merchants nationwide. Cardholders were initially affluent business people; they could present their Diners Club card (actually a cardboard booklet) as payment for meals, travel tickets, rental cars, and so on. Diners Club users were billed monthly and full payment required monthly. Gas-station and department-store “charge cards” had been around for decades, but Diners Club and the credit and debit cards that followed kick-started a revolution in cashless and checkless payments.

Before ATMs, banks developed machines that dispensed cash and nothing more. The first such efforts – in Japan, Sweden, and Great Britain – happened independently at around the same time in the late 1960s. On 27 June 1967 in North London, Barclays Bank installed the first cash dispensing machine in either Europe or the United States. Marketed as “Barclaycash,” customers could purchase punched card vouchers for fixed sums during banking hours that they could insert into the machine, along with a six-digit authorization number, to obtain cash.

On 9 September 1969 a Chemical Bank branch in Long Island, New York, became the first to use an automated cash dispenser in the United States, using cards with the now ubiquitous magstripe – the magnetic strip on the back of the card containing the card’s data (an invention from IBM from nearly a decade earlier). In 1971 a subsequent model at Chemical Bank became a true ATM by allowing customers to make deposits, check balances, and transfer funds between savings and checking accounts.

Plastic credit and debit cards, like automobiles, have become ubiquitous artifacts many of us use on a daily basis that are part of much larger technological systems. With automobiles, the artifact itself is complex (many components) and its inner workings mostly opaque, but much of the larger technological system is readily apparent – roads, highways, gas stations, repair shops, parking lots, and motels. With credit and debit cards the artifact is quite basic, while the vast and complex underlying technological system is largely invisible. At the heart of this system, and critical to the adoption and use of these cards, are real-time computers. Typical of this hidden infrastructure was the Visa payment system; it became by far the world’s largest such system and was the creation of the financial visionary, Dee Hock.

In the late 1950s a number of banks began to experiment with charge cards. Bank of America, the largest bank in the United States, introduced the BankAmericard “credit card” in 1958 by distributing 65,000 unsolicited cards to residents of Fresno, California. In contrast to charge cards like Diners Club, credit cards offered credit if the customer chose not to fully pay off the monthly balance. With its credit card operation already profitable by the early 1960s, Bank of America launched a subsidiary BankAmericard Service Corporation to provide cards and processing services for other smaller banks. A critical problem with credit cards was enforcing a customer’s credit limit – at the time all that could be done was permit purchases below a “ceiling” value of a few tens

of dollars without authorization and obtain telephone approval for high-value transactions. Slow settlement and clearing, however, led to significant problems with fraud – criminals often could make numerous “under the limit” charges over several weeks before being detected.

For a few years Bank of America had a virtual monopoly in credit card services to other banks; eventually these other banks decided to shake free from Bank of America and establish their own cooperative issuer called National BankAmericard Incorporated (NBI) in 1970. NBI and its international arm, Ibanco, were renamed Visa USA and Visa International, respectively, in 1976. For its first 14 years NBI/Visa was led by Dee Hock, who previously had run the BankAmericard operation for a Seattle bank. Hock, who always emphasized his humble origins in depression-era rural Utah, never wanted to be viewed as “a typical banker.” He had a commanding and eccentric personality and as a historian of Visa has related, those who worked for Hock found him to be “inspirational” and “brilliant,” as well as “intimidating” and “aggravating.”²⁶ Hock believed that any value, whether a credit account, checking account, or savings account, should be available for electronic-based purchases or transfers worldwide at any time. This was an extraordinary vision that today we take wholly for granted.

Authorization of purchases was by far the greatest problem NBI/Visa faced. For purchases above the credit ceiling a phone call had to be made to an authorization center, which would have to communicate in turn by telephone or telex with the purchaser’s bank. Authorizations were a hassle that could take as long as an hour (the purchaser would make a purchase and come back later for the goods when authorization had been obtained). Hock’s vision was to make this process entirely automatic using real-time computers. NBI/Visa set out to build two computer systems BASE I and BASE II – the former for authorization and the latter for clearing and settlement.

NBI/Visa contracted with TRW and management consultants McKinsey & Company to develop BASE I to automate authorizations. The project involved installing terminals at each processing center, adding network hardware, and writing, testing, and debugging software. Reliability was critical and virtually all components of the system had backup equipment. Unlike other major real-time projects, including SAGE and SABRE, BASE I was delivered in April 1973 on schedule and within budget. It provided authorizations in a matter of seconds.

Prior to BASE II, all clearing and settlement had been manual and prone to error. Banks would exchange paper sales drafts, which were then reconciled. Hock sought an automated computer system that eliminated the mailing of paper by transforming the sales drafts into electronic records and clearing them through a BASE II centralized computer, a powerful IBM 370/145 serving 88 processing centers. NBI/Visa’s BASE II was America’s first nationwide automated clearinghouse. It reduced clearing and settlement from a week to being done overnight – resulting in far quicker fraud detection.

In the early 1980s Visa introduced magstripe technology and inexpensive point-of-sale terminals that connected directly with BASE I. With these state-of-the-art systems in place, the number of accepting merchants worldwide, member institutions, and issued Visa cards all more than doubled between 1975 and 1983, while sales volume went up more than fivefold to \$70 billion. More powerful systems were needed to accommodate higher volume, and Visa implemented IBM's Airline Control Program operating system, which the computer giant had developed for SABRE, to accommodate faster processing.

In accord with Hock's grand electronic value exchange vision, Visa also introduced an "asset card," initially named Entrée, in 1975. Drawing directly on a cardholder's bank account, it was soon renamed Visa Debit. Getting merchants and consumers on board was slow at first, but the debit card became a common form of payment after point-of-sale terminals began to be widely adopted in the mid-1980s and the subsequent linking of these cards to ATM networks.

Like credit cards and debit cards, ATMs were a transformative technology in banking. ATMs would reduce both bank-teller operations and the clearing of checks. As with credit cards, automating processes for deposits and withdrawals occurred in stages that first involved overnight processing, but evolved into real-time computer systems and the instantaneous transactions we now associate with ATMs.

By the end of the 1980s the major banks and many of the smaller ones had ATMs, and a number of these machines were online and updated accounts instantaneously. During that decade national networks were launched so customers were not limited to using only ATMs for their own bank. In time, these international networks allowed travelers to access cash at fair rates of exchange throughout much of the world and made traveler's checks increasingly obsolete.

A good portion of Dee Hock's vision for ever widening electronic value exchange was realized by the 1990s, and online banking in the World Wide Web era only extended this as networked personal computers commonly became point-of-sale terminals.

THE UNIVERSAL PRODUCT CODE (UPC) is the barcode now almost universally found on food products manufactured or packaged for retail. The UPC was built on the foundations of the modern food manufacturing and distribution industries that emerged in the 1920s and 1930s. The most important early innovation was the supermarket with its labor-saving innovation of moving customers past the goods in a store (that is, self-service) instead of having a salesperson bring the goods to the customer.

The first supermarket, the Piggly Wiggly store in Memphis, Tennessee, opened in 1916 and in the next half century the supermarket drove out the traditional main-street store. By 1941 there were 8000 supermarkets America. In parallel with the developments in supermarket retailing, food manufacturing saw national branded goods, which had begun in a small way in the 1890s, come

to dominate the food industry with their promotion in newspapers and through radio advertising in the 1930s and 1940s. Supermarket growth resumed after the war, with over 15,000 supermarkets opened by 1955, and over 30,000 by 1965.

By the end of the 1960s the US food industry had become dominated by the supermarket, which accounted for three quarters of the total grocery business. But, as is characteristic of a mature industry, the generous profit margins of the boom years were being eroded. The most obvious area for productivity improvement was the checkout, which accounted for almost a quarter of operating costs and often caused bottlenecks that exasperated customers.

The need to improve these “front end” operations coincided with the development of a new generation point-of-sale terminals that offered the possibility of automatic data capture for pricing and stock control. The new point-of-sale terminals required the application of product codes to merchandise; the information gathered at the checkout could then be used to maintain inventories and determine the cost of goods. But this was not generally practical for supermarkets. The large number of product lines offered (10,000 to 20,000) and the low average value of items made the individual application of product codes too expensive.

The structure of the food industry was such that product coding would be economic only if firms acted cooperatively, so that manufacturers, wholesalers, and retailers all used the same UPC. This implied an extraordinary degree of cooperation among all these organizations, and the implementation of the UPC was thus as much a political achievement as a technical one.

In 1969 the National Association of Food Chains appointed the management consultants McKinsey & Company to advise on the implementation of a coding system for the food industry. From the outset, McKinsey understood that product coding could only succeed on an all or nothing basis. In August 1970 an “Ad Hoc Committee of the Grocery Industry” was set up, consisting of “a blue ribbon group of ten top industry leaders, five representing food manufacturers and five representing food distributors,”²⁷ which was chaired by the president of the Heinz Company. Besides Heinz, other major food manufacturers included General Foods and General Mills, while the distributors included Kroger, SuperValu, and Associated Food Stores. The ad hoc committee in turn retained McKinsey as consultants who, in the course of the next few months, made some fifty presentations to the industry. These presentations were invariably given to the top echelon of each organization, as presenting a vision of the future.

Simultaneously, many technical decisions had to be made. One of the first was to agree upon a product code of ten digits – five digits representing the manufacturer and five representing the product. This in itself was a compromise between the manufacturers, who wanted a longer code for greater flexibility, and the retailers, who wanted a shorter one to reduce equipment costs. (In fact, this

was arguably a bad decision. Europe, which developed its own European Article Numbering system somewhat later, adopted a six-plus-six code.) It was also decided that the product code would be applied by the manufacturer, which would be cheaper than in-store application by retailers; and the article number would contain no price information because the price would differ from store to store.

By mid-1971 the principle of a UPC had been accepted throughout the industry, but there remained the major issues of agreeing on the technology to be used and pressing the technology into service. This was yet another politically sensitive area. Applying the code to products had to be inexpensive in order not to disadvantage small manufacturers; and the expense of product coding could not add significantly to the cost of goods. The checkout equipment also had to be relatively inexpensive because millions of barcode readers would eventually be needed. In the spring of 1973 the barcode system with which we are now all familiar was formally adopted. This system was largely developed by IBM, and it was selected simply for being the cheapest and the most trouble-free option. The scanning technology was inexpensive, the barcode could be printed in ordinary ink, and readers were not overly sensitive to smearing and distortion of the packaging.

By the end of 1973 over 800 manufacturers in the United States had been allocated identification numbers. These included virtually all the largest manufacturers, and between them they accounted for over 84 percent of retail food sales. The first deliveries of barcoded merchandise began in 1974, along with the availability of scanners manufactured by IBM and NCR. The barcode principle diffused very rapidly to cover not just food items but most packaged retail goods (note the ISBN barcode on the back cover of this book). The barcode has become an icon of great economic significance.

Alongside the physical movement of goods there is now a corresponding information flow that enables computers to track the movement of this merchandise. Thus the whole manufacturing-distribution complex has become increasingly integrated. In retail outlets, shelves do not have to become empty before the restocking process is started. As backroom stock levels fall, orders can be sent automatically and electronically to wholesalers and distribution centers, which in turn can automatically send orders to the manufacturers. Special protocols for the exchange of electronic information among manufacturers, distributors, and retailers in a standard form were developed to expedite this process.

In a sense, there are two systems co-existing, one physical and one virtual. The virtual information system in the computers is a representation of the status of every object in the physical manufacturing and distribution environment – right down to an individual can of peas. This would have been possible without computers, but it would not have been economically feasible because the cost of collecting, managing, and accessing so much information would have been overwhelming.

For the food industry itself, there were negative impacts of bar coding, much as the rise of the supermarkets spelled an end to the owner operated grocery store. For example, the barriers to entry into food manufacturing became greater. Not only were there the bureaucratic hurdles of UPC allocation to surmount, but penetrating the retail network of national chains now required at minimum an operation of substantial scale. Indeed, a curious development of the late twentieth century was the emergence of folksy specialty foods – apparently made by small scale food preparers, but in fact mass produced and sold by national chains. As I write, I have in front of me a box of the “finest hand baked biscuits,” a gift from a friend returning from a vacation in Devon, England. On the bottom of the carton, out of sight, is a barcode for the supermarket checkout. There could be few more potent ironies of modern living.

Notes to Chapter 7

An excellent overview that sets Whirlwind, SAGE, and other military computer developments in the Cold War context is Paul Edwards’s *The Closed World: Computers and the Politics of Discourse in Cold War America* (1996). Stuart W. Leslie’s *The Cold War and American Science* (1993) also places these developments in a broad political and administrative framework. The detailed history of Project Whirlwind is the subject of Kent C. Redmond and Thomas M. Smith’s monograph *Project Whirlwind: The History of a Pioneer Computer* (1980), while their subsequent book *From Whirlwind to MITRE: The R&D Story of the SAGE Air Defense Computer* (2000) completes the story. An excellent source on SAGE is a special issue of the *Annals of the History of Computing* (vol. 5, no. 4, 1981). John F. Jacobs’s *The SAGE Air Defense System* (1986) is an interesting personal account (the book was published by the MITRE Corporation, of which Robert E. Everett became president).

There are several good accounts of American Airlines’ SABRE. The best are James L. McKenney’s *Waves of Change* (1995) and Jon Eklund’s article “The Reservoir Automated Reservation System” (1994). The discussion of the Visa payment system draws from David L. Stearn’s excellent study *Electronic Value Exchange* (2011) as well as Dennis W. Richardson’s *Electric Money* (1970). The histories of cash-dispensing machines and ATMs are described in Bernardo Batiz-Lazo’s *Cash and Dash: How ATMs and Computers Changed Banking* (2018). There is no full-length history of the barcode, but we have made good use of Alan Q. Morton’s excellent article “Packaging History: The Emergence of the Uniform Product Code” (1994). Another useful source is Steven Brown’s personal account of the negotiations to implement barcodes in the US grocery industry, *Revolution at the Checkout Counter* (1997).

Notes

1. Redmond and Smith 1980, p. 33.
2. The lecture is reprinted in Campbell-Kelly and Williams 1985, pp. 375–92.
3. Quoted in Redmond and Smith 1980, p. 38.
4. *Ibid.*
5. *Ibid.*, p. 41.
6. *Ibid.*, p. 46.
7. *Ibid.*, p. 68.
8. *Ibid.*, p. 47.
9. *Ibid.*, p. 96.
10. *Ibid.*, p. 183.
11. *Ibid.*, pp. 145, 150.
12. *Ibid.*, p. 172.
13. Valley 1985, pp. 207–8.
14. Jacobs 1986, p. 74.
15. Flamm 1987, p. 176.
16. See Flamm 1987, 1988.
17. Bennington 1983, p. 351.
18. Watson and Petre 1990, p. 230.
19. For an account of the Reservisor, see Eklund 1994.
20. McKenney 1995, p. 98.
21. Quoted in Bashe et al. 1986, p. 5.
22. Quoted in McKenney 1995, p. 111.
23. Plugge and Perry 1961, p. 593.
24. Burck 1965, p. 34.
25. McKenney 1995, p. 119.
26. Stearns 2011, p. 40.
27. Morton 1994, p. 105.

8 Software

THE CONSTRUCTION of the first practical stored-program computer, the Cambridge University EDSAC, has been widely – and justifiably – celebrated as a triumphal moment in the history of modern computing. But it was not long after the EDSAC became operational in May 1949 that the Cambridge team realized that the completion of the machine itself was only the first step in the larger process of developing a working computer system. In order to take advantage of the theoretical power of their new general-purpose computer, the EDSAC team first had to program it to solve real-world problems. And as Maurice Wilkes, the leader of Cambridge development, would shortly realize, developing such programs turned out to be surprisingly difficult:

By June 1949 people had begun to realize that it was not so easy to get a program right as had at one time appeared. I well remember when this realization first came on me with full force. The EDSAC was on the top floor of the building and the tape-punching and editing equipment one floor below on a gallery that ran round the room in which the differential analyzer was installed. I was trying to get working my first non-trivial program, which was one for the numerical integration of Airy's differential equation. It was on one of my journeys between the EDSAC room and the punching equipment that "hesitating at the angles of stairs" the realization came over me with full force that a good part of the remainder of my life was going to be spent in finding errors in my own programs.¹

At first, such mistakes in programs were simply called mistakes, or errors. But within a few years they were called "bugs" and the process of correcting them was called "debugging." And while Wilkes might have been the first computer programmer to contemplate the seeming impossibility of eradicating every such bug, he would be by no means the last. Over the course of the next decade, the costs and challenges associated with developing computer programs, or "software," became increasingly apparent. Even as computer technology was becoming ever smaller, faster, and more reliable, computer software development

projects acquired a reputation for being over-budget, behind-schedule, and bug-ridden. By the end of the decade a full-blown “software crisis” had been declared by industry leaders, academic computer scientists, and government officials. The question of why software was so hard, and what, if anything, could be done about it, would dominate discussions about the health and future of the computer industry in the decades ahead.

First Steps in Programming

The discovery of debugging was not the first time Wilkes had thought about the problem of software development (although the word *software* itself would not be coined until the late 1950s). Already by 1948, while the EDSAC was still under construction, he had begun to turn his thoughts away from hardware and toward the programming problem. Inside every stored-program computer (including those of today) programs are stored in pure binary – a pattern of machine representations of ones and zeros. For example, in the EDSAC, the single instruction “Add the short number in memory location 25” was stored as

```
11100000000110010
```

As a programming notation, binary was unacceptable because humans cannot readily read or write long strings of binary digits. It was therefore an obvious idea that occurred to all the computer groups to use a more programmer-friendly notation when designing and writing programs. In the case of the EDSAC, the instruction “Add the short number in memory location 25” was written

```
A 25 S
```

where A stood for “add,” 25 was the decimal address of the memory location, and S indicated that a “short” number was to be used.

Although John von Neumann and Herman Goldstine had come up with a similar programming notation for the EDVAC, they had always assumed that the conversion of the human-readable program into binary would be done by the programmer – or by a more lowly technician “coder.” Wilkes, however, realized that this conversion could be done by the computer itself. After all, a computer was designed to manipulate numbers, and letters of the alphabet were represented by numbers inside a computer; and in the last analysis, a program was nothing more than a set of letters and numbers. This was an insight that is obvious only in retrospect.

In October 1948 the Cambridge group was joined by a twenty-one-year-old research student, David Wheeler, who had just been awarded a Cambridge “first” in mathematics. Wilkes turned over the programming problem to Wheeler as the major part of his PhD topic. To translate programs into binary,

Wheeler wrote a very small program, which he called the Initial Orders, that would read in a program that was punched in symbolic form on teleprinter tape, convert it to binary, and place it in the memory of the computer ready for execution. This tiny program consisted of just thirty instructions that he had honed to perfection.

But as the Initial Orders solved one problem – converting a symbolic program into binary – another problem came into view: actually getting programs to run and produce the correct results.

Quite a lot could be done to reduce the incidence of errors by checking a program very carefully before the programmer attempted to run it. It was found that half an hour spent checking a program would save lots of time later trying to fix it – not to mention the machine time, which, in the early 1950s, cost perhaps \$50 an hour. Even so, it became almost a tradition that newcomers to programming simply could not be told: almost all of them had to learn the truth about debugging in the same painfully slow and expensive way. The same remains true today.

The Cambridge group decided that the best way of reducing errors in programs would be to develop a “subroutine library” – an idea that had already been proposed by von Neumann and Goldstine. It was realized that many operations were common to different programs – for example, calculating a square root or a trigonometric function, or printing out a number in a particular format. The idea of a subroutine library was to write these operations as kind of miniprograms that programmers could then copy into their own programs. In this way a typical program would consist of perhaps two-thirds subroutines and one-third new code. (At this time a program was often called a routine – hence the term *subroutine*. The term *routine* died out in favor of *program* in the 1950s, but the term *subroutine* has stayed with us.) The idea of reusing existing code was and remains the single most important way of improving programmer productivity and program reliability.

Naturally, Wilkes gave Wheeler the job of organizing the details of the subroutine library. Wheeler’s solution was simple and elegant. Each subroutine was written so that, when it was loaded into the memory immediately following the previous subroutine in the program, it would run correctly. Thus all the components of a program were loaded into the bottom of the memory working upward without any gaps, making the best possible use of every one of the precious few hundred words of memory.

In 1951 the Cambridge group put all their programming ideas into a textbook entitled *The Preparation of Programs for an Electronic Digital Computer*, which Wilkes chose to publish in the United States in order to reach the widest possible readership. At this date the first research computers were only just coming into operation, and generally the designers had given precious little thought to programming until they suddenly had a working machine. The Cambridge book was the only programming textbook then available, and it rushed in to fill

the vacuum. The Cambridge model thus set the programming style for the early 1950s, and even today the organization of subroutines in virtually all computers still follows this model.

Flowcharts and Programming Languages

Programming a computer is, at its essence, a twofold series of translations: first, the translation of a real-world problem (or rather, its solution) into a series of steps, known as an *algorithm*, that can be performed by a machine; and, second, the translation of this algorithm into the specific instructions used by a particular computer. Both types of translations were fraught with difficulties and the potential for error and miscommunication.

The earliest tool developed to facilitate the first of these translations was the *flowchart*. A flowchart was a graphical representation of an algorithm or process in which the various steps of the process were represented visually by boxes, and the relationship between them by lines and arrows. Decision points indicated possible branches in the process flow (for example, “if condition A is true, then proceed to step 2; if not, proceed to step 3”). The first flowcharts were developed for use in industrial engineering in the 1920s, and von Neumann (who had first trained as a chemical engineer) began applying them to computer programs in the 1950s. Flowcharts were intended to serve as the blueprint for computer programmers, an intermediate step between the analysis of a system and its implementation as a software application.

The second step in the translation process involved the coding of the algorithm in machine-readable form, which had already been partially addressed by Wilkes and Wheeler. By about 1953, however, the center of programming research had moved from England to the United States, where there was heavy investing in computers with large memories, backed up with magnetic tapes and drums. Computers now had perhaps ten times the memory of the first prototypes. The programming systems designed to be shoehorned into the first computers with tiny 1,000-word memories were no longer appropriate, and there was a desire to exploit the increased potential of “large” memories (as much as 10,000 words) by designing more sophisticated programming techniques. Moreover, the patient crafting of programs, instruction by instruction, was becoming uneconomic. Programs cost too much in programmer time and took too long to debug.

A number of American research laboratories and computer manufacturers began to experiment with “automatic programming” systems that would enable the programmer to write a program in some high-level programming code, which was easy for humans to write since it looked like English or algebra, but which the computer could convert into binary machine instructions. The goal was to allow, as much as possible, the mechanical translation of a design flowchart into a completed application. Automatic programming was partly a

technological problem and partly a cultural one. In many ways it was the latter that was more difficult. There is no craft that is so tradition-bound as one that is five years old, and many of the seasoned programmers of the 1950s were as reluctant to change their working methods as had been the hand weavers to adopt looms two hundred years earlier.

Probably no one did more to change the conservative culture of 1950s programmers than Grace Murray Hopper, the first programmer for the Harvard Mark I in 1943, then with UNIVAC. For several years in the 1950s she barnstormed around the country, proselytizing the virtues of automatic programming at a time when the technology delivered a good deal less than it promised. At UNIVAC, for example, Hopper had developed an automatic programming system she called the A-0 compiler. (Hopper chose the word *compiler* because the system would automatically put together the pieces of code that made up the complete program.) The A-0 compiler worked, but the results were far too inefficient to be commercially acceptable. Even quite simple programs would take as long as an hour to translate, and the resulting programs were woefully slow. The successor to the A-0 compiler, B-0 (or Flow-Matic, as it was more commonly known), was unique in that it was designed specifically for business applications, but it, too, was slow and inefficient.

Thus, in 1953–1954 the outstanding problem in programming technology was to develop an automatic programming system that produced programs as good as those written by an experienced programmer. The most successful attack on this problem came from IBM, in a project led by a twenty-nine-year-old researcher named John Backus. The system he produced was called the Formula Translator—or FORTRAN for short. FORTRAN became the first really successful “programming language,” and for many years it was the lingua franca of scientific computing. In FORTRAN a single statement would produce many machine instructions, thus greatly magnifying the expressive power of a programmer. For example, the single statement

$$X = 2.0 * \text{FSIN}(A+B)$$

would produce all the code necessary to calculate the function $2\sin(a + b)$ and assign the value to the variable denoted by x .

Backus made the case for FORTRAN mainly on economic grounds. It was estimated in 1953 that half the cost of running a computer center was accounted for by the salaries of programmers, and that one-quarter to one-half of the average computer’s available time was spent in program testing and debugging. Backus argued that “programming and debugging accounted for as much as three-quarters of the cost of operating a computer; and obviously, as computers got cheaper, this situation would get worse.”² This was the cost-benefit analysis that Backus made to his boss at IBM in late 1953 in a successful bid to get a budget and staff to develop FORTRAN for IBM’s soon-to-be-announced model

704 computer. During the first half of 1954, Backus began to put together his programming team.

The FORTRAN project did not have a high profile at IBM – as evidenced by its unglamorous location in an annex to IBM’s headquarters on Madison Avenue, “on the 19th floor . . . next to the elevator machinery.” FORTRAN was regarded simply as a research project with no certain outcome, and it had no serious place in the product plan for the 704 computer. Indeed, it was not at all clear in 1954 that it would even be possible to make an automatic programming system that would produce code as good as that written by a human programmer. There were plenty of skeptics in IBM who thought Backus was attempting the impossible. Thus, producing code as good as a human being’s was always the overriding objective of the FORTRAN team. In 1954 the group produced a preliminary specification for the language. Very little attention was paid to the elegance of the language design because everything was secondary to efficiency. It never occurred to Backus and his colleagues that they were designing a programming language that would still be around in the twenty-first century.

Backus and his team now began to sell the idea of FORTRAN to potential users. Most of the feedback they received was highly skeptical, coming from seasoned programmers who had already become cynical about the supposed virtues of automatic programming: “[T]hey had heard too many glowing descriptions of what turned out to be clumsy systems to take us seriously.” Work began on writing the translator early in 1955, and Backus planned to have the project completed in six months. In fact it took a team of a dozen programmers two and a half years to write the eighteen thousand instructions in the FORTRAN system. The FORTRAN translator was by no means the longest program that had been written up to that date, but it was a big program and very complex. Much of the complexity was due to the attempt to get the compiler to produce programs that were as good as those written by human programmers. The schedules continued to slip, and during 1956 and 1957 the pace of debugging was intense. To get as much scarce machine time as possible, the team would make use of the slack night shift: “often we would rent rooms in the Langdon Hotel . . . on 56th Street, sleep there a little during the day and then stay up all night.”

The first programmer’s manual for FORTRAN, handsomely printed between eye-catching IBM covers, announced that FORTRAN would be available in October 1956, which turned out to be “a euphemism for April 1957”³ – the date on which the system was finally released.

One of the first users of the FORTRAN system was the Westinghouse-Bettis nuclear power facility in Maryland. Late one Friday afternoon, 20 April 1957, a large box of punched cards arrived at the computer center. There was no identification on the cards, but it was decided that they could only be for the “late 1956” FORTRAN compiler. The programmers decided to try to run a small test program. They were amazed when the FORTRAN system swallowed the test program and produced a startlingly explicit diagnostic statement on the printer:

"SOURCE PROGRAM ERROR . . . THE RIGHT PARENTHESIS IS NOT FOLLOWED BY A COMMA."⁴ The error was fixed and the program retranslated; it then proceeded to print out correct results for a total of twenty-two minutes. "Flying blind," the Westinghouse-Bettis computer center had become the first FORTRAN user.

In practice the FORTRAN system produced programs that were 90 percent as good as those written by hand, as measured by the memory they occupied or the time they took to run. FORTRAN was a phenomenal aid to the productivity of a programmer. Programs that had taken days or weeks to write and get working could now be completed in hours or days. By April 1958, half of the twenty-six IBM 704 installations surveyed by IBM were using FORTRAN for half of their problems. By that fall, some sixty installations were using the system. Meanwhile, Backus's programming team was rapidly responding to the feedback from users by preparing a new version of the language and compiler called FORTRAN II. This contained 50,000 lines of code and took fifty man-years to develop before it was released in 1959.

Although IBM's FORTRAN was the most successful scientific programming language, there were many parallel developments by other computer manufacturers and computer users around the same time. For example, Hopper's group at UNIVAC had improved the A-0 compiler and supplied it to customers as the Math-Matic programming language. Within the academic community, programming languages such as MAD at the University of Michigan and IT at Carnegie-Mellon were developed. There was also an attempt to develop an international language, called ALGOL, by the Association for Computing Machinery in the United States and its European sister organizations. Several dozen different scientific programming languages were in use by the late 1950s.

However, within a couple of years FORTRAN had come to dominate all the other languages for scientific applications. It has sometimes been supposed that this outcome was the result of a conscious plan by IBM to dominate the computer scene, but there is no real evidence for any such deliberate premeditation. What actually happened was that FORTRAN was simply the first effective high-level language to achieve widespread use. In spring 1957 Grace E. Mitchell, only the second woman to join the FORTRAN team, wrote the *FORTRAN Programmer's Primer*. Issued by IBM in late 1957, the *Primer* "had an important influence on the subsequent growth in the use of the system . . . it was the only available simplified instruction manual."⁵ In 1961 Daniel D. McCracken published the first FORTRAN textbook, and universities and colleges began to use it in their undergraduate programming courses.⁶ In industry, standardizing on FORTRAN had the advantage of making it possible to exchange programs with other organizations that used different makes of computer – FORTRAN would work on any machine for which a translator was available. Also there was the advantage of a growing supply of FORTRAN-trained programmers in the job market. Soon, virtually all scientific computing in the United States was

done in FORTRAN; although ALGOL survived for a few years in Europe, it, too, was soon ousted by FORTRAN there. In all such cases, the overwhelming attraction of FORTRAN was that it had become a “standard” language accepted by the whole user community. In 1966 FORTRAN became the first programming language to be formally standardized by the American National Standards Institute.

While FORTRAN was becoming the standard for scientific applications, another language was emerging for business applications. Whereas FORTRAN had become a standard language largely by accident, COBOL – an acronym for the COMmon Business Oriented Language – was in effect created as a standard by the action of the US government. By the time FORTRAN was accepted as a standard, there had been a serious economic loss to industry and government because, whenever they changed their computers, they had to rewrite all their programs. This was expensive and often disruptive. The government was determined not to let this happen with its commercial application programs, so in 1959 it sponsored a Committee on Data Systems and Languages (CODASYL) to design a new standard language for commercial data processing. This language emerged as COBOL 60 the following year. By writing their programs in the new language, users would be able to retain all their existing software investment when they changed computers.

Grace Murray Hopper played an active role in promoting COBOL. Although she was not one of the language’s designers, COBOL’s design was heavily influenced by the Flow-Matic data-processing language she had developed at UNIVAC. COBOL was particularly influenced by what one critic described as Hopper’s “missionary zeal for the cause of English language coding.”⁷ In COBOL a statement designed to compute an employee’s net pay by deducting tax from the gross pay might have been written:

SUBTRACT TAX FROM GROSS-PAY GIVING NET-PAY.

This provided a comforting illusion for administrators and managers that they could read and understand the programs their employees were writing, even if they could not write them themselves. While many technical people could see no point in such “syntactic sugar,” it had important cultural value in gaining the confidence of decision makers.

Manufacturers were initially reluctant to adopt the COBOL standard, preferring to differentiate their commercial languages from their competitors – thus IBM had its Commercial Translator, Honeywell had FACT, and UNIVAC had Flow-Matic. Then, in late 1960 the government declared that it would not lease or purchase any new computer without a COBOL compiler unless its manufacturer could demonstrate that its performance would not be enhanced by the availability of COBOL. No manufacturer ever attempted such a proof, and they all immediately fell in line and produced COBOL compilers.

Although for the next twenty years COBOL and FORTRAN would dominate the world of programming languages (between them, they accounted for 90 percent of applications programs), designing new programming languages for specific application niches remained the single biggest software research activity in the early 1960s. Hundreds of languages were eventually produced, most of which have died out.⁸

Software Contractors

Although programming languages and other utilities helped computer users to develop their software more effectively, programming costs remained the single largest expense of running a computer installation. Many users preferred to minimize this cost by obtaining ready-made programs wherever possible, instead of writing their own.

Recognizing this preference, in the late 1950s computer manufacturers began to develop applications programs for specific industries such as insurance, banking, retailing, and manufacturing. Programs were also produced for applications that were common to many industries, such as payroll, cost accounting, and stock control. All these programs were supplied at no additional charge by the computer manufacturer. There was no concept in the 1950s of software being a salable item: manufacturers saw applications programs simply as a way of selling hardware. Indeed, applications programming was often located within the marketing division of the computer manufacturers. Of course, although the programs were “free,” the cost of developing them was ultimately reflected in the total price of the computer system.

Another source of software for computer users was the free exchange of programs within cooperative user groups such as SHARE for users of IBM computers and USE for UNIVAC users. At these user groups, programmers would trade programming tips and programs. IBM also facilitated the exchange of software between computer users by maintaining a library of customer-developed programs. These programs could rarely be used directly as they were supplied, but a computer user’s own programmers could often adapt an existing program more quickly than writing one from scratch.

The high salaries of programmers, and the difficulty of recruiting and managing them, discouraged many computer users from developing their own software. This created the opportunity for the new industry of *software contracting*: firms that wrote programs to order for computer users. In the mid-1950s there were two distinct markets for the newly emerging software contractors. One market, which came primarily from government and large corporations, was for very large programs that the organizations did not have the capability to develop for themselves. The second market was for “custom” programs of modest size, for ordinary computer users who did not have the in-house capability or resources to develop software.

The first major firm in the large-systems sector of software contracting was the RAND Corporation, a nonprofit defense contractor closely aligned to the US government, which developed the software for the SAGE air-defense project. When the SAGE project was begun in the early 1950s, although IBM was awarded the contract to develop and manufacture the mainframe computers, writing the programs for the system – estimated at a million lines of code – was way beyond IBM's or anyone else's experience. The contract for the SAGE software was therefore given to the RAND Corporation in 1955. Although lacking any actual large-scale software writing capability, the corporation was judged to have the best potential for developing it. To undertake the mammoth programming task, the corporation created a separate organization in 1956, the System Development Corporation (SDC), which became America's first major software contractor.⁹

The SAGE software development program has become one of the great episodes in software history. At a time when the total number of programmers in the United States was estimated at about 1,200 people, SDC employed a total staff of 2,100, including 700 programmers on the SAGE project. The central operating program amounted to nearly a quarter of a million instructions, and ancillary software took the total to over a million lines of code. The SAGE software development has been described as a university for programmers. It was an effective, if unplanned, way to lay the foundations of the US software industry.

In the late 1950s and early 1960s several other major defense contractors and aerospace companies – such as TRW, the MITRE Corporation, and Hughes Dynamics – entered the large-scale software contracting business. The only other organizations with the technical capability to develop large software systems were the computer manufacturers themselves. By developing large one-of-a-kind application programs for customers, IBM and the other mainframe manufacturers became (and remain) an important sector of the software contracting industry. IBM, for example, having collaborated with American Airlines to develop the SABRE airline reservations system, proceeded to develop reservations systems for other carriers including Delta, Pan American, and Eastern in the United States and Alitalia and BOAC in Europe. Most of the other mainframe computer manufacturers also became involved in software contracting in the 1960s, usually building on the capabilities inherited from their office-machine pasts: for example, NCR developed retailing applications and Burroughs targeted the banks.

Whereas the major software contractors and mainframe computer manufacturers developed very large systems for major organizations, there remained a secondary market of developing software for medium-sized customers. The big firms did not have the small-scale economies to tap this market effectively, and it was this market opportunity that the first programming entrepreneurs exploited. Probably the first small-time software contractor was the Computer Usage Company (CUC), whose trajectory was typical of the sector.

The Computer Usage Company was founded in New York City in March 1955 by two scientific programmers from IBM. It was not expensive to set up in the software contracting business, because "all one needed was a coding pad and a pencil,"¹⁰ and it was possible to avoid the cost of a computer, either by renting time from a service bureau or by using a client's own machine. CUC was established with \$40,000 capital, partly private money and partly secured loans. The money was used to pay the salaries of a secretary and four female programmers, who worked from one of the founders' private apartments until money started coming in. The founders of CUC came from the scientific programming division of IBM, and most of their initial contracts were with the oil, nuclear power, and similar industries; in the late 1950s the company secured a number of NASA contracts. By 1959 CUC had built up to a total of fifty-nine employees. The company went public the following year, which netted \$186,000. This money was used to buy the firm's first computer.

By the early 1960s CUC had been joined by several other start-up software contractors. These included the Computer Sciences Corporation (co-founded by a participant of IBM's FORTRAN project team), the Planning Research Corporation, Informatics, and Applied Data Research. These were all entrepreneurial firms with growth patterns similar to those of the Computer Usage Company, although with varied application domains. Another major start-up was the University Computing Company in 1965.

The first half of the 1960s was a boom period for software contractors. By this time the speed and size of computers, as well as the machine population, had grown by at least an order of magnitude since the 1950s. This created a software-hungry environment in which private and government computer users were increasingly contracting out big software projects. In the public sector the defense agencies were sponsoring huge data-processing projects, while in the private sector banks were moving into real-time computing with the use of Automated Teller Machines.

By 1965 there were between forty and fifty major software contractors in the United States, of whom a handful employed more than a hundred programmers and had annual sales in the range of \$10 million to \$100 million. CUC, for example, had grown to become a major firm, diversifying into training, computer services, facilities management, packaged software, and consultancy, in addition to its core business of contract programming. By 1967 it had twelve offices, seven hundred staff members, and annual sales of \$13 million.

The major software contractors, however, were just the tip of an iceberg beneath which lay a very large number of small software contractors, typically with just a few programmers. According to one estimate, in 1967 there were 2,800 software contracting firms in the United States.

The specialist software contractors had also been joined by computer services and consulting firms such as Ross Perot's EDS, founded in 1962, and

Management Science America (MSA), formed in 1963, which were finding contract programming an increasingly lucrative business opportunity.

The Software Crisis

The emergence of a software contracting industry helped alleviate one of the most immediate challenges associated with programming computers – namely, finding enough experienced programmers to do the work. And the development of high-level programming languages and programming utilities helped eliminate much of the cost and tedium associated with finding and eradicating programming bugs. But by the late 1960s a new set of concerns had emerged about software and software development, to the point that many industry observers were talking openly about a looming “software crisis” that threatened the future of the entire computer industry. For the next several decades, the language of the “software crisis” would shape many of the key developments – technological, economic, and managerial – within electronic computing.

One obvious explanation for the burgeoning software crisis was that the power and size of computers were growing much faster than the capability of software designers to exploit them. In the five years between 1960 and the arrival of System/360 and other third-generation computers, the memory size and speed of computers both increased by a factor of ten – giving an effective performance improvement of a hundred. During the same period, software technology had advanced hardly at all. The programming technology that had been developed up to this time was capable of writing programs with 10,000 lines of code, but it had problems dealing with programs that were ten times longer. Projects involving a million lines of code frequently ended in disaster. By the end of the 1960s, such large software development projects had become “a scare item for management . . . an unprofitable morass, costly and unending.”¹¹ The business literature from this period is replete with stories about software projects gone wrong and expensive investments in computer technology that had turned out to be unprofitable.

If the 1960s was the decade of the software debacle, the biggest debacle of all was IBM’s operating system OS/360. The “operating system” was the raft of supporting software that all the mainframe manufacturers were supplying with their computers by the late 1950s. It included all the software, apart from programming languages, that the programmer needed to develop applications programs and run them on the computer. For IBM’s second-generation computers, the operating software consisted of about 30,000 lines of code. By the early 1960s a powerful operating system was a competitive necessity for all computer manufacturers.

When IBM began planning System/360 in 1962, software had become an essential complement to the hardware, and it was recognized that the software development effort would be huge – an initial estimate put the development

budget at \$125 million. No fewer than four distinct operating systems were planned – later known as BOS, TOS, DOS, and OS/360. BOS was the Batch Operating System for small computers; TOS was the operating system for medium-sized magnetic tape-based computers; and DOS was the Disk Operating System for medium and large computers. These three operating systems represented the state of the art in software technology and were all completed within their expected time scales and resources. The fourth operating system, OS/360, was much more ambitious, however, and became the most celebrated software disaster story of the 1960s.

The leader of the OS/360 project was one of Howard Aiken's most able PhD students, a software designer in his mid-thirties named Frederick P. Brooks Jr., who subsequently left IBM to form the computer science department at the University of North Carolina. An articulate and creative individual, Brooks would become not only a leading advocate in the 1970s of the push to develop an engineering discipline for software construction but also the author of the most famous book on software engineering, *The Mythical Man-Month*.

Why was OS/360 so difficult to develop? The essential problem was that OS/360 was the biggest and most complex program artifact that had ever been attempted. It would consist of hundreds of program components, totaling more than a million lines of code, all of which had to work in concert without a hiccup. Brooks and his co-designers had reasonably supposed that the way to build a very large program was to employ a large number of programmers. This was rather like an Egyptian pharaoh wanting to build a very large pyramid by using a large number of slaves and a large number of stone blocks. Unfortunately, that works only with simple structures; software does not scale up in the same way, and this fact lay at the heart of the software crisis.

An additional difficulty was that OS/360 was intended to exploit the new technology of "multiprogramming," which would enable the computer to run several programs simultaneously. The risk of incorporating this unproved technology was fully appreciated at the time and, indeed, there was much internal debate about it. But multiprogramming was a marketing necessity, and at the System/360 launch on 7 April 1964 OS/360 was "the centerpiece of the programming support,"¹² with delivery of a full multiprogramming system scheduled for mid-1966.

The development of the OS/360 control program – the heart of the operating system – was based at the IBM Program Development Laboratories in Poughkeepsie, New York. There it had to compete with other System/360 software projects that were all clamoring for the company's best programmers. The development task got under way in the spring of 1964 and was methodically organized from the start – with a team of a dozen program designers leading a team of sixty programmers implementing some forty functional segments of code. But as with almost every other large software project of the period, the schedules

soon began to slip – not for any specific reason, but for a myriad of tiny causes. As Brooks explained:

Yesterday a key man was sick, and a meeting couldn't be held. Today the machines are all down, because lightning struck the building's power transformer. Tomorrow the disk routines won't start testing, because the first disk is a week late from the factory. Snow, jury duty, family problems, emergency meetings with customers, executive audits – the list goes on and on. Each one only postpones some activity by a half-day or a day. And the schedule slips, one day at a time.¹³

More people were added to the development team. By October 1965 some 150 programmers were at work on the control program, but a “realistic” estimate now put slippage at about six months. Early test trials of OS/360 showed that the system was “painfully sluggish” and that the software needed extensive rewriting to make it usable. Moreover, by the end of 1965 fundamental design flaws emerged for which there appeared to be no easy remedy. For the first time, OS/360 “was in trouble from the standpoint of sheer technological feasibility.”¹⁴

In April 1966 IBM publicly announced the rescheduling of the multiprogramming version of OS/360 for delivery in the second quarter of 1967 – some nine months later than originally announced. IBM's OS/360 software problems were now public knowledge. At a conference of anxious IBM users, the company's chairman, Tom Watson Jr., decided the best tactic was to make light of the problems:

A few months ago IBM's software budget for 1966 was going to be forty million dollars. I asked Vin Learson [the head of the System/360 development program] last night before I left what he thought it would be, and he said, “Fifty million.” This afternoon I met Watts Humphrey, who is in charge of programming production, in the hall here and said, “Is this figure about right? Can I use it?” He said, “It's going to be sixty million.” You can see that if I keep asking questions we won't pay a dividend this year.¹⁵

But beneath Watson's levity, inside IBM there was a “growing mood of desperation.”¹⁶ The only way forward appeared to be to assign yet more programmers to the task. This was later recognized by Brooks as being precisely the wrong thing to do: “Like dousing a fire with gasoline, this makes matters worse, much worse. More fire requires more gasoline, and thus begins a regenerative cycle which ends in disaster.”¹⁷ Writing a major piece of software was a subtle creative task and it did not help to keep adding more and more programmers: “The bearing of a child takes nine months, no matter how many women are assigned.”

Throughout 1966 more and more people were added to the project. At the peak more than a thousand people at Poughkeepsie were working on

OS/360 – programmers, technical writers, analysts, secretaries, and assistants – and altogether some five thousand staff-years went into the design, construction, and documentation of OS/360 between 1963 and 1966.

OS/360 finally limped into the outside world in mid-1967, a full year late. By the time OS/360 and the rest of the software for System/360 had been delivered to customers, IBM had spent half a billion dollars on it – four times the original budget – which, Tom Watson explained, made it “the single largest cost in the System/360 program, and the single largest expenditure in company history.”¹⁸ But there was perhaps an even bigger, human cost:

The cost to IBM of the System/360 programming support is, in fact, best reckoned in terms of the toll it took of people: the managers who struggled to make and keep commitments to top management and to customers, and the programmers who worked long hours over a period of years, against obstacles of every sort, to deliver working programs of unprecedented complexity. Many in both groups left, victims of a variety of stresses ranging from technological to physical.¹⁹

Had OS/360 merely been late, it would not have become such a celebrated disaster story. But when it was released, the software contained dozens of errors that took years to eradicate. Fixing one bug would often simply replace it with another. It was like trying to fix a leaking radiator. The lesson of OS/360 seemed to be that not only were big software projects expensive, they were also unreliable. As such, OS/360 became a symbol for an endemic software problem affecting the entire computer industry that persisted for decades.

Software as an Industrial Discipline

It is one thing to identify the symptoms of a disease, quite another to accurately diagnose its underlying causes. Even more difficult is the development of an effective therapy. Although the existence of a software crisis was widely acknowledged, interpretations of its underlying nature and causes differed wildly. Corporate employers tended to focus on the problems associated with hiring and managing programmers; aspiring computer scientists, on the lack of theoretical rigor in software development methodologies; the programmers themselves, on their relative lack of professional status and autonomy. Not unsurprisingly, each group recommended very different solutions based on their own interpretation of the fundamental problem.

One theme common to most explanations of the software crisis was the idea that “computer people” represented a new and different breed of technical expert. Computer programming in particular, almost from its very origins, was regarded as more of a “black art” than a scientific or engineering discipline. Until well into the 1960s there were no formal academic programs in programming: the

first programmers came from a wide variety of backgrounds and disciplines, were generally self-taught, developed idiosyncratic techniques and solutions, and tended to work largely in isolation from a larger community of practitioners. It was also not clear what skills and abilities made for a good programmer: mathematical training was seen as being vaguely relevant, as was an aptitude for chess or music, but for the most part, talented programmers appeared to be “born, not made.” This made the training and recruitment of large numbers of programmers particularly difficult, and corporate recruiters struggled to meet the growing demand for programmers by developing aptitude tests and personality profiles aimed at identifying the special few with natural programming ability. The original assumption held by John von Neumann and others – that “coding” was routine work that could be done by relatively unskilled (and low-paid, and therefore often female) labor – was quickly proved inaccurate. By the end of the 1950s computer programming had become largely the province of self-assured, proficient, and ambitious young men.

The relative scarcity of skilled and experienced programmers, combined with a growing demand for programming labor, meant that most programmers in this period were well paid and highly valued. But this privileged treatment also created resentments on the part of corporate employers, many of whom felt that they were being held hostage by their “whiz kid” technologists. There was a growing perception among managers that computer programmers were too often flighty, difficult-to-manage “prima donnas.” These perceptions were reinforced by the personality profiles used to identify potential programmer trainees, which suggested that one of the “striking characteristics” of programmers was their “disinterest in people.” By the end of the 1960s the stereotype of the socially awkward computer nerd – longhaired, bearded, sandal wearing, and “slightly neurotic” – had been well established in both the industry literature and popular culture.²⁰

The widespread belief that computer programmers were uniquely gifted but also potentially troublesome (among other things, turnover rates among programming personnel ran as high as 25 percent annually) suggested both an explanation for and a solution to the emerging software crisis. If part of the problem with software was that its development was overly dependent on skilled, expensive programming labor, one solution was to reduce the level of expertise required of programmers. The design of COBOL was influenced, at least in part, by the desire to create computer code that could be read, understood, and perhaps even written by corporate managers. Over the course of the 1960s new languages, such as IBM’s PL/I, were even more explicit in their claims to eliminate corporate dependence on expert programmers.

As it turned out, however, technological solutions to the “programming problem” were rarely successful. It became increasingly clear that developing good software was about more than simply writing efficient and bug-free code: the more complex and ambitious software projects became, the more they involved

programmers in a broad range of activities that included analysis, design, evaluation, and communication – none of which were activities that could be easily automated. As Frederick Brooks lamented in his post-mortem analysis of the failures of the OS/360 operating system, throwing more resources at a programming problem not only didn't help but often actually made the situation worse. To borrow a phrase from the language of industrial engineering, software was a problem that did not scale well. Machines could be mass-produced in factories; computer programs could not.

Another promising approach to rationalizing the process of programming was to develop a more rigorous academic approach to software development. The first professional society for computer specialists was founded in 1946 as the Association for Computing Machinery (ACM). As the name indicates, ACM's early emphasis was on hardware development, but by the 1960s its focus had shifted to software. The ACM was instrumental in the professional accreditation of academic departments of computer science, which by the mid-1960s had emerged out of mathematics and electrical engineering as its own independent discipline. In 1968 the ACM issued its Curriculum '68 guidelines, which helped standardize education in computer science programs throughout the United States.

The emergence of computer science programs in universities helped solve some critical problems for the software industry. Computer scientists developed the theoretical foundations of the discipline, established formal methods for evaluating and validating algorithms, and created new tools and techniques for software development. In doing so, they both improved programming practices and raised the status of programming as a legitimate intellectual discipline. But computer science departments could not, in and of themselves, produce enough programmers to satisfy the demands of the larger commercial computing industry. University degree programs took too long, cost too much, and excluded too many people (including, in this period, many women and minorities). More significantly, the theoretical concerns of academic computer scientists were not always seen as being relevant to the problems faced by working corporate programmers: as one IBM recruiter declared in a 1968 conference on personnel research,

the new departments of computer science in the universities . . . are too busy teaching simon-pure courses in their struggle for academic recognition to pay serious time and attention to the applied work necessary to educate programmers and systems analysts for the real world.²¹

In the face of such concerns, alternative approaches to “professionalizing” computer programming were developed. In the early 1960s, for example, the Data Processing Management Association introduced its Certificate in Data Processing program, which was meant to establish basic standards of competence. Vocational trade schools in computer programming also emerged as an alternative to university education.

The most visible and influential response to the software crisis was the “software engineering” movement. The seminal moment for this new movement occurred in 1968 at a NATO-sponsored conference held in Garmisch, Germany. The 1968 NATO conference, which included participants from industry, government, and military, both reflected and magnified contemporary concerns about the looming software crisis. “We build systems like the Wright brothers built airplanes,” confessed one representative from MIT: “build the whole thing, push it off the cliff, let it crash, and start over again.”²² Another compared software developers unfavorably to hardware designers: “they are the industrialists and we are the crofters. Software production today appears in the scale of industrialization somewhere below the more backward construction industries.” The solution to this crisis, argued the conference organizers, was for a new kind of software manufacture “to be based on the types of theoretical foundations and practical disciplines that are traditional in the established branches of engineering.”²³

The advocates of software engineering emphasized the need to impose industrial discipline on informal and idiosyncratic craft practices of programmers. They rejected the notion that large software projects were inherently unmanageable and recommended, instead, that software developers adopt methods and techniques borrowed from traditional manufacturing. The ultimate goal would be a kind of “software factory” complete with interchangeable parts (or “software components”), mechanized production, and a largely deskilled and routinized workforce. The tools used to achieve this goal included structured design, formal methods, and development models.

The most widely adopted engineering practice was the use of a “structured design methodology.” Structured design reflected the belief that the best way to manage complexity was to limit the software writer’s field of view. Thus, the programmer would begin with an overview of the software artifact to be built. When it was satisfactory it would be put to one side, and all the programmer’s attention would be directed to a lower level of detail – and so on, until eventually actual program code would be written at the lowest level in the design process. Structured programming was a good intuitive engineering discipline and was probably the most successful programming idea of the 1970s. A whole subculture of consultants and gurus peddled the patent medicine of structured programming, and techniques for structured programming in FORTRAN and COBOL were developed. The structured-programming concept was also embodied in new computer languages, such as Pascal, invented in 1971, which for twenty years was the most popular language used in undergraduate programming education. Structured programming was also one of the key concepts embodied in the Ada programming language commissioned by the US Department of Defense for developing safety-critical software.

Another attack on the complexity problem was the use of formal methods. Here, an attempt was made to simplify and mathematize the design process by which programs were created. For technical and cultural reasons, formal

methods failed to make a major impact. Technically, they worked well on relatively small programs but failed to scale up to very large, complex programs. And the cultural problem was that software practitioners – several years out of college, with rusty math skills – were intimidated by the mainly academic proponents of formal methods.

Another widely adopted concept was the development model, which was as much a management tool as a technical one. The development model viewed the software writing process not as a once-and-for-all construction project, like the Hoover Dam, but as a more organic process, like the building of a city. Thus software would be conceived, specified, developed, and go into service – where it would be improved from time to time. After a period of use the software would die, as it became increasingly obsolete, and then the process would begin again. The different stages in the software life-cycle model made projects much easier to manage and control, and it recognized the ability of software to evolve in an organic way.

These more rigorous approaches to software development all had prominent thought-leaders. For example, the American Barry Boehm was a software engineering authority, the Dutch computer scientist Edsger Dijkstra was a driving force behind structured programming and much besides, while in Britain Tony Hoare (later Sir Anthony Hoare) made breakthroughs in formal methods. All were subsequently elected to their national science or engineering academies.

In the end, software engineering proved to be only a partial solution to the problems associated with software development. Software production continued to prove resistant to industrialization, but not because software writers were unwilling or unable to adopt more “scientific” approaches to their discipline, or because nerdy programmers could not communicate clearly with their colleagues. Unlike hardware, whose specifications and characteristics could be readily articulated and measured, software continued to be difficult to precisely define and delineate. The development of a successful corporate payroll application, for example, required the integration of general business knowledge, organization-specific rules and procedures, design skills, highly technical programming expertise, and a close understanding of particular computer installations. This was an inherently difficult and often organizationally disruptive undertaking. There were good reasons why the majority of software in this period was custom-produced for individual corporations in a process that resembled more the hiring of a management-consulting firm than the purchase of a ready-made product. But, in fact, the idea of a “software product” was beginning to take root.

Software Products

Despite the improving software development techniques that emerged toward the end of the 1960s, the gulf between the capabilities of computers to exploit software and its supply continued to widen. Except for the very largest corporations, it had become economically infeasible for most computer users to exploit

the potential of their computers by writing very large programs because it would never be possible to recover the development costs. This was true whether they developed the programs themselves or had them written by a software contractor.

This situation created an opportunity for existing software contractors to develop software products – packaged programs, whose cost could be recovered through sales to ten or even a hundred customers. The use of a package was much more cost-effective than custom software. Sometimes a firm would tailor the package to its business, but just as often it would adjust its business operations to take advantage of an existing product.

After the launch of the IBM System/360, which established the first stable industry-standard platform, a handful of software contractors began to explore the idea of converting some of their existing software artifacts into packages. In 1967 the market for software packages was still in its infancy, however, with fewer than fifty products available.

One of the first packaged software products was – significantly – designed specifically to be used by other software developers. In 1965 the software contractor Applied Data Research (ADR) released Autoflow, which could generate the flowchart that corresponded to an already existing program. That this was exactly the reverse of the intended relationship between flowcharts and computer code (the flowchart was supposed to be the design specification that guided the programming process rather than a retrospective justification of a programmer's implementation) is telling, suggesting that at least some of the complaints about the haphazard practices of programmers were correct. Indeed, many programmers felt that drawing a flowchart served no purpose other than satisfying the whims of ill-educated management. Autoflow promised to eliminate that particular chore, and it became the first computer program to be sold as a product.

The development of the software-products industry was accelerated by IBM's unbundling decision of December 1968, by which it decided to price its hardware and software separately. Prior to this time IBM, in line with its total-systems approach, had supplied hardware, software, and system support, all bundled as a complete package. The user paid for the computer hardware and got the programs and customer support for "free." This was the standard industry practice at the time and was adopted by all of IBM's mainframe competitors. IBM, however, was at this time under investigation by the antitrust division of the Department of Justice because of its alleged monopolistic practices, including the bundling of its products. Although IBM decided to voluntarily unbundle its software and services, this did not affect the Department of Justice's decision, and an antitrust suit was begun in January 1969. The suit rolled on inconclusively for more than a decade before it was eventually dismissed in 1982.

It took about three years for the software-products market to become fully established after unbundling. No doubt a thriving packaged-software

industry would have developed in the long run, but the unbundling decision accelerated the process by transforming almost overnight the common perception of software from free good to tradable commodity. For example, whereas in the 1960s the American insurance industry had largely made do with IBM's free-of-charge software, unbundling was "the major event triggering an explosion of software firms and software packages for the life insurance industry."²⁴ By 1972 there were 81 vendors offering 275 packages just for the life-insurance industry. Several computer services firms and major computer users also recognized the opportunity to recover their development costs by spinning off packaged-software products. Boeing, Lockheed, and McDonnell Douglas had all developed engineering-design and other software at great expense.

Perhaps the most spectacular beneficiary of the new environment for software products was Informatics, the developer of the top-selling Mark IV file management system – one of the first database products. Informatics was founded in 1962 as a regular software contractor. In 1964, however, the company recognized that the database offerings of the mainframe manufacturers were very weak and saw a niche for a for-sale product. It took three years and cost about \$500,000 to develop Mark IV. When the product was launched in 1967, there were few precedents for software pricing; its purchase price of \$30,000 "astounded"²⁵ computer users, long used to obtaining software without any separate charge. By late 1968 it had only a modest success, with 44 sales. After unbundling, however, growth was explosive – 170 installations by the spring of 1969, 300 by 1970, and 600 by 1973. Mark IV took on a life of its own and even had its own user group – called the IV (pronounced "ivy") League. Mark IV was the world's most successful software product for a period of fifteen years, until 1983, by which time it had cumulative sales of over \$100 million.

By the mid-1970s all the existing computer manufacturers were important members of the software-products industry. IBM was, of course, a major player from the day it unbundled. Despite its often mediocre products, it had two key advantages: inertia sales to the huge base of preexisting users of its software and the ability to lease its software for a few hundred dollars per month. The latter factor gave it a great advantage over the rest of the software industry, which needed outright sales to generate cash flow.

The most successful software-products supplier of the late 1970s and 1980s was Computer Associates. Formed in 1976, the company initially occupied a niche supplying a sorting program for IBM mainframes. Computer Associates was one of the first computer software makers to follow a strategy of growth by buying out other companies. All Computer Associates' acquisitions, however, were aimed at acquiring software assets – "legitimate products with strong sales"²⁶ – rather than the firms themselves; indeed, it usually fired half the staff of the companies it acquired. Over the next fifteen years, Computer Associates took over more than two dozen software companies, including some of

the very largest. By 1989 it had annual revenues of \$1.3 billion and was the largest independent software company in the world. This position did not last for long, however. By the 1990s, the old-line software-products industry was being eclipsed by Microsoft and the other leading personal-computer software companies.

Notes to Chapter 8

The literature of the history of software is patchy. Although good for its kind, much of it is “internalist,” technical, and focused on narrow software genres, which makes it difficult for a nonexpert to get a foothold in the topic. An excellent starting point is Steve Lohr’s engaging biographically oriented *Go To: The Story of the Programmers Who Created the Software Revolution* (2001). Grace Murray Hopper’s role in promoting programming languages is well described in Kurt Beyer’s *Grace Hopper and the Invention of the Information Age* (2009). To try to bring some order to the history of the software field, a conference was organized in 2000 by the Heinz Nixdorf Museums Forum, Paderborn, Germany, and the Charles Babbage Institute, University of Minnesota. The papers presented at the conference were published as *History of Computing: Software Issues* (Hashagen et al., eds., 2002). This book gives the best overview of the subject so far published.

Saul Rosen’s *Programming Systems and Languages* (1967) gives about the best coverage of the early period. There is a very full literature on the history of programming languages, of which the best books are Jean Sammet’s *Programming Languages: History and Fundamentals* (1969) and the two edited volumes of the History of Programming Languages conferences that took place in 1976 and 1993: *History of Programming Languages* (Richard Wexelblat, ed., 1981), and *History of Programming Languages*, vol. 2 (Thomas Bergin et al., eds., 1996). The extensive literature on the history of programming languages has distorted the view of the history of software, so that most other aspects are neglected by comparison. The OS/360 debacle is discussed at length in Fred Brooks’s classic *The Mythical Man-Month* and in Emerson W. Pugh et al.’s *IBM’s 360 and Early 370 Systems* (1991). There is no monographic history of the software crisis, but Peter Naur and Brian Randell’s edited proceedings of the Garmisch conference, *Software Engineering* (1968), is rich in sociological detail. For a history of computer programmers, see Nathan Ensmenger’s *The Computer Boys Take Over: Computers, Programmers, and the Politics of Technical Expertise*.

The two best books on the history of the software industry are Martin Campbell-Kelly’s *From Airline Reservations to Sonic the Hedgehog: A History of the Software Industry* (2003), which focuses on the US scene, and David Mowery’s edited volume, *The International Computer Software Industry* (1996).

Notes

1. Wilkes 1985, p. 145.
2. Backus 1979, pp. 22, 24, 27, 29.
3. Rosen 1967, p. 7.
4. Bright 1979, p. 73.
5. Backus 1979, p. 33.
6. McCracken 1961.
7. Rosen 1967, p. 12.
8. For a history of early programming languages see Sammet 1969.
9. For a history of SDC see Baum 1981.
10. Quoted in Campbell-Kelly 1995, p. 83.
11. Naur and Randell 1968, p. 41.
12. Pugh et al. 1991, p. 326.
13. Brooks 1975, p. 154.
14. Pugh et al. 1991, pp. 336, 340.
15. Watson 1990, p. 353.
16. Pugh et al. 1991, p. 336.
17. Brooks 1975, pp. 14, 17.
18. Watson 1990, p. 353.
19. Pugh et al. 1991, p. 344.
20. Ensmenger 2010, pp. 146, 144, 159.
21. Ibid., p. 133.
22. Naur and Randell 1968, p. 7.
23. Ibid.
24. Yates 1995, p. 129.
25. Campbell-Kelly 1995, p. 92.
26. Ibid.

From SAGE to the Internet



Figures 25 and 26 SAGE, an early 1960s air defense system, was located in some thirty “direction centers” strategically placed around the country to provide complete radar coverage for the United States against airborne attacks. The consoles (*above*) were designed for use by ordinary airmen and established the technology of human-computer interaction using screens and light pens in place of punched cards or the clunky teletypes of previous systems. COURTESY OF CHARLES BAB-BAGE INSTITUTE ARCHIVES (CBIA), UNIVERSITY OF MINNESOTA.

Program cards (left) for the SAGE operating system, which consisted of more than a million lines of code. The software for SAGE was developed by the Systems Development Corporation, which became the wellspring of the US software industry. REPRINTED WITH PERMISSION OF THE MITRE CORPORATION.

Figure 27 Jay W. Forrester, the leader of Project Whirlwind, with a prototype core memory plane. Core memory consisted of a matrix of small magnetic ceramic toroids, or “cores,” strung on a wire mesh. Each core could store a single bit. Perfected in the mid-1950s, core memory superseded all earlier memory systems until it was itself superseded by semiconductor memory in the 1970s. COURTESY OF MIT MUSEUM.



Figure 28 A psychologist by training, J.C.R. Licklider was a consultant to the SAGE project, advising on human-machine interaction. During the 1960s and 1970s he set in place many of the research programs that culminated in the user-friendly personal computer and the internet. Licklider was a consummate political operator, who motivated a generation of computer scientists and obtained government funding for them to work in the fields of human-computer interaction and networked computing. COURTESY OF MIT MUSEUM.



Figure 29 The SABRE reservation system, developed by IBM for American Airlines, became operational in 1962. It was the first real-time commercial computer network and connected over a thousand agents in fifty cities across the United States. REPRINT COURTESY OF IBM CORPORATION ©.

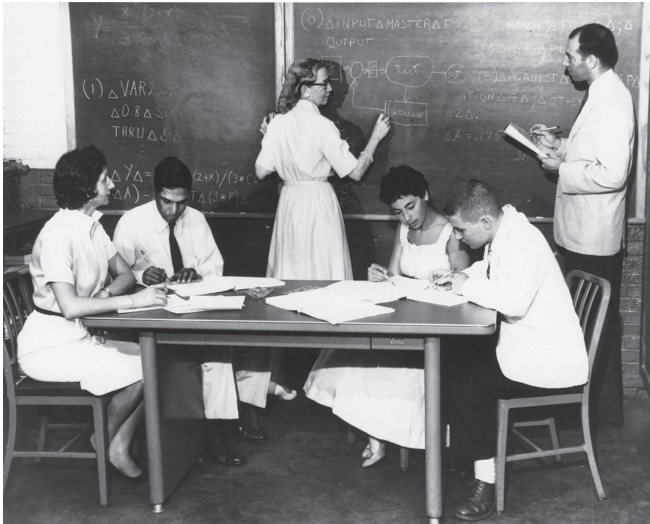


Figure 30 Grace Murray Hopper (at the chalk board) with a programming class for the UNIVAC in the mid-1950s. Hopper was the leading advocate for COBOL, the most popular programming language for business. COURTESY OF CHARLES BABBAGE INSTITUTE ARCHIVES (CBIA), UNIVERSITY OF MINNESOTA.

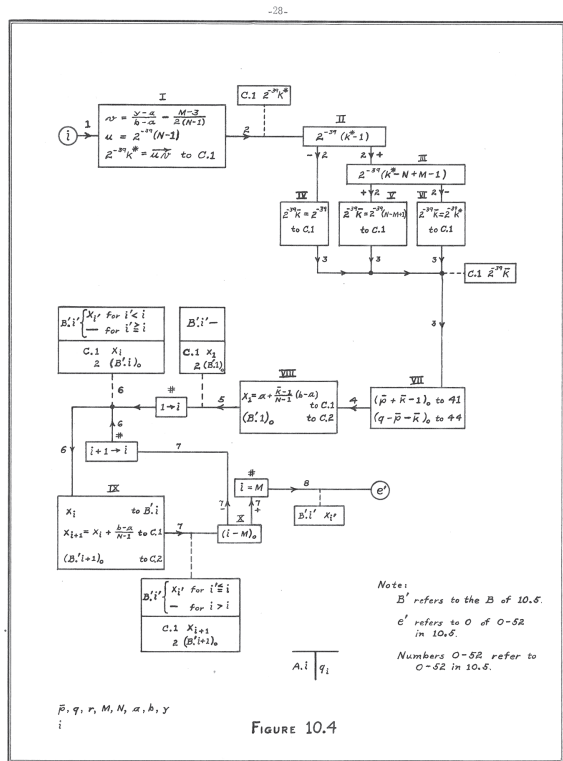


FIGURE 10.4



Figures 31 and 32 Working at the Institute for Advanced Study, Herman Goldstone and John von Neumann introduced the “flow diagram” (above) as a way of managing the complexity of programs and communicating them to others. FROM THE SHELBY WHITE AND LEON LEVY ARCHIVES CENTER OF THE INSTITUTE FOR ADVANCED STUDY, PRINCETON, NJ, USA.

Programming languages, such as FORTRAN, COBOL, and BASIC, improved the productivity of programmers and enabled non-experts to write programs. FORTRAN, designed by IBM’s John Backus (left), was released in 1957 and was for decades the most widely used scientific programming language. REPRINT COURTESY OF IBM CORPORATION ©.



Figure 33 John Kemeny (*left*) and Thomas Kurtz (*center*), inventors of the BASIC programming language, help a student with a computer program in about 1964. Originally designed for students, BASIC went mainstream and became the most popular programming language for personal computers. COURTESY OF DARTMOUTH COLLEGE LIBRARY.



Figure 34 In the mid-1960s time-sharing transformed the way people used computers, by providing inexpensive access to a powerful computer with user-friendly software. The time-sharing system developed at Dartmouth College, shown here, exposed most of the college's students to computers using the BASIC programming language. COURTESY OF DARTMOUTH COLLEGE LIBRARY.

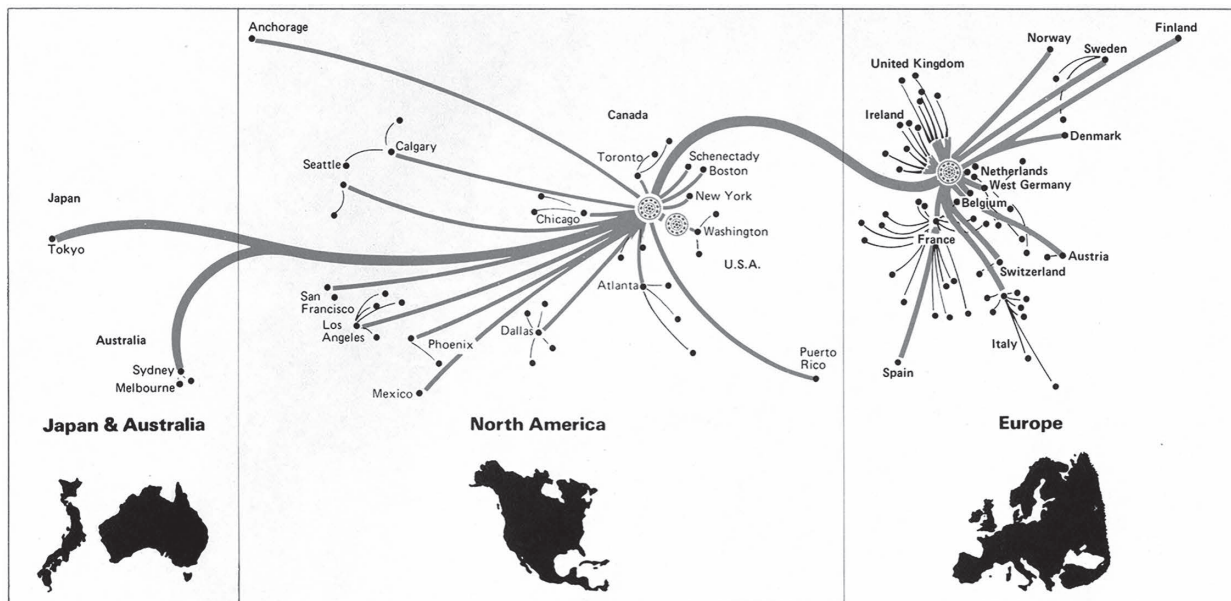
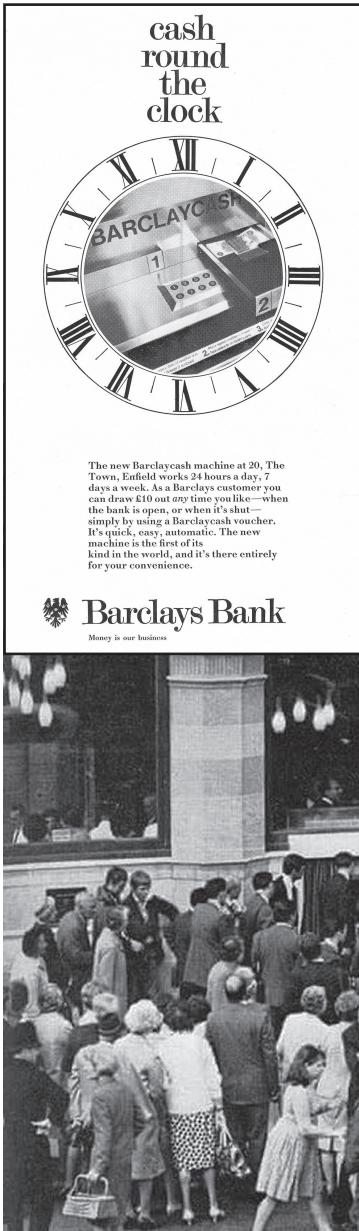


Figure 35 Before the internet, the largest time-sharing system was the Mark III service run by General Electric's GEISCO and Honeywell. The service operated globally from three "supercenters" – two in the United States and a third in Amsterdam – each equipped with multiple mainframe computers. In the late 1970s, the system supported several thousand simultaneous business users in over 500 cities around the world. COURTESY OF CHARLES BABBAGE INSTITUTE ARCHIVES (CBIA), UNIVERSITY OF MINNESOTA AND HONEYWELL UK.



Figures 36 and 37 On 27 June 1967 Barclays Bank unveiled the world's first cash machine in Enfield, London. The inaugural withdrawal was made by the British comedy actor Reg Varney, who attracted a crowd of fans and passersby. COURTESY OF BARCLAYS GROUP ARCHIVES.

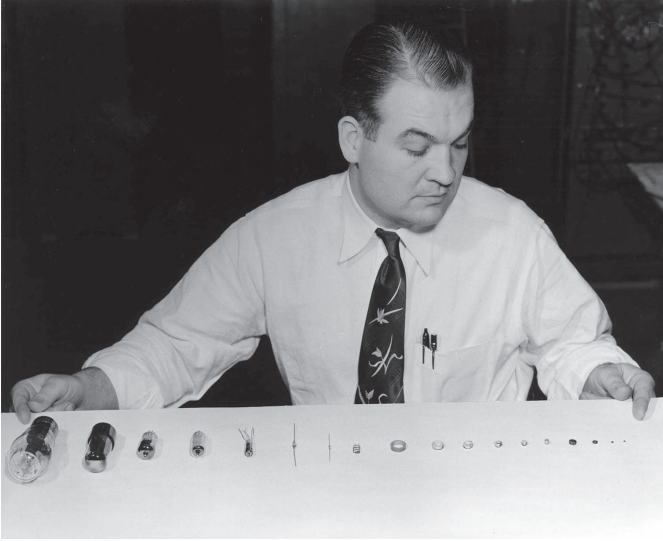


Figure 39 A display showing the ever-decreasing size in the progression from vacuum tubes of the 1940s to semiconductors of the 1960s. COURTESY OF CHARLES BABBAGE INSTITUTE ARCHIVES (CBIA), UNIVERSITY OF MINNESOTA.

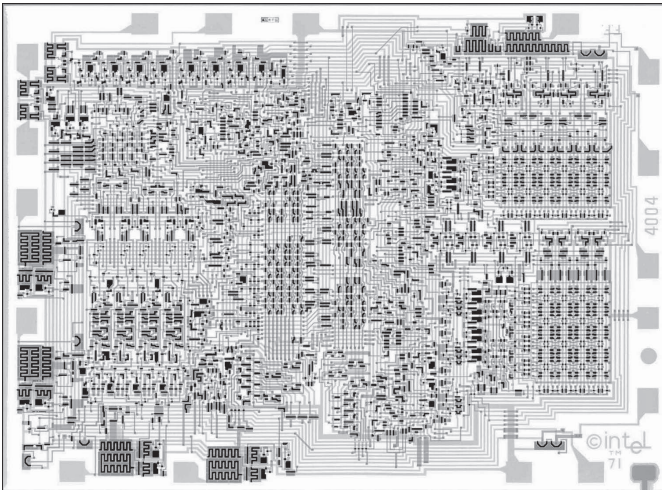


Figure 40 The Intel 4004 Microprocessor, announced in 1971, was the first “computer on a chip.” It contained 2,300 transistors. COURTESY INTEL CORPORATION.



Figure 41 Videogames were made possible by the advent of microchips. The hugely successful *Space Invaders* was introduced as an arcade game in 1978. A domestic version was produced by Atari two years later, establishing one of the most popular videogame genres. COPYRIGHT BOARD OF TRUSTEES OF STANFORD UNIVERSITY; PROVIDED COURTESY OF THE STANFORD LIBRARIES.

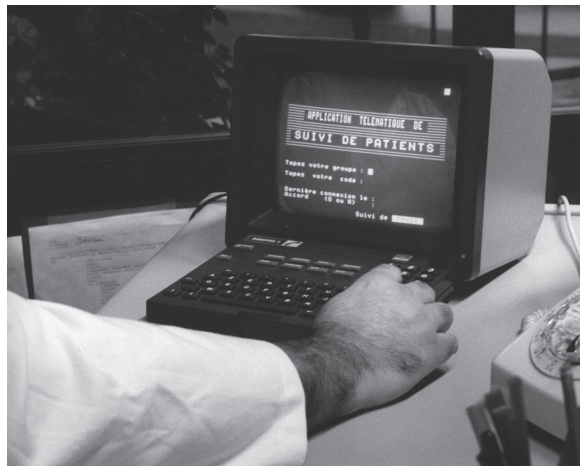


Figure 42 In the early 1980s the French government kick-started a national information network by distributing millions of Minitel terminals free to telephone users. As well as directory inquiries, Minitel offered services such as chat rooms, entertainment, and mail order. Minitel finally yielded to the global internet in 2012 when the service was decommissioned. COURTESY OF ARCHIVES DE FRANCE TÉLÉCOM.



Figure 43 The Apple II, launched in 1977, established the paradigm of the personal computer: a central processing unit equipped with a keyboard and screen, and floppy disk drives for program and data storage. It was successful both as a consumer product and as a business machine. COURTESY OF CHARLES BABBAGE INSTITUTE ARCHIVES (CBIA), UNIVERSITY OF MINNESOTA.



Figure 44 In 1981 the corporate personal computer was fully legitimated by the announcement of the IBM PC, which rapidly became the industry standard. Within five years, 50 percent of new personal computers were PC “clones,” produced by major manufacturers such as Compaq, Gateway, Dell, Olivetti, and Toshiba. COURTESY OF CHARLES BABBAGE INSTITUTE ARCHIVES (CBIA), UNIVERSITY OF MINNESOTA.

Figure 45 Rather than make a clone of the IBM PC, Apple Computer decided to produce the Macintosh, a low-cost computer with a user-friendly graphical user interface. The Macintosh was launched in 1984, and its user-friendliness was unrivaled for several years. COURTESY OF CHARLES BABBAGE INSTITUTE ARCHIVES (CBIA), UNIVERSITY OF MINNESOTA.

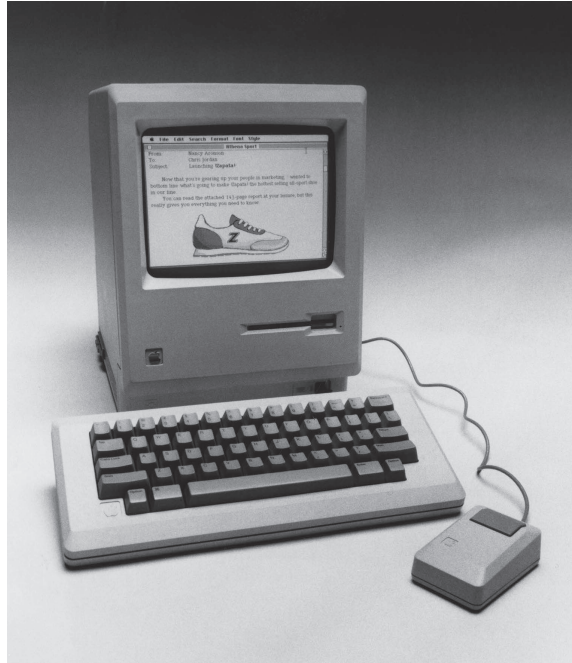


Figure 46 The Osborne 1, announced in spring 1981, was one of the first attempts at a portable personal computer. Although the machine would fit under an airplane seat, it was inconveniently heavy and sometimes described as “luggable” rather than “portable.” CREATIVE COMMONS <https://commons.wikimedia.org/wiki/File:Osborne01.jpg>

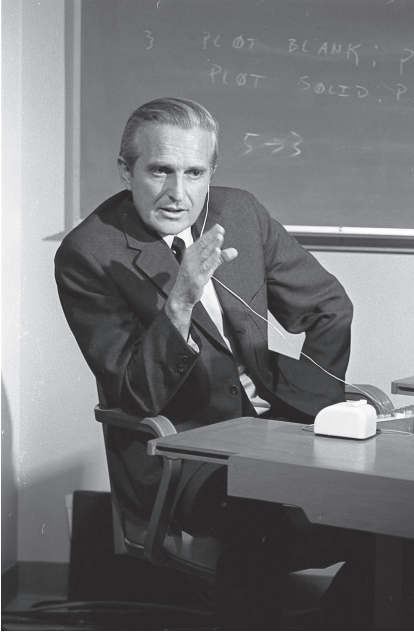
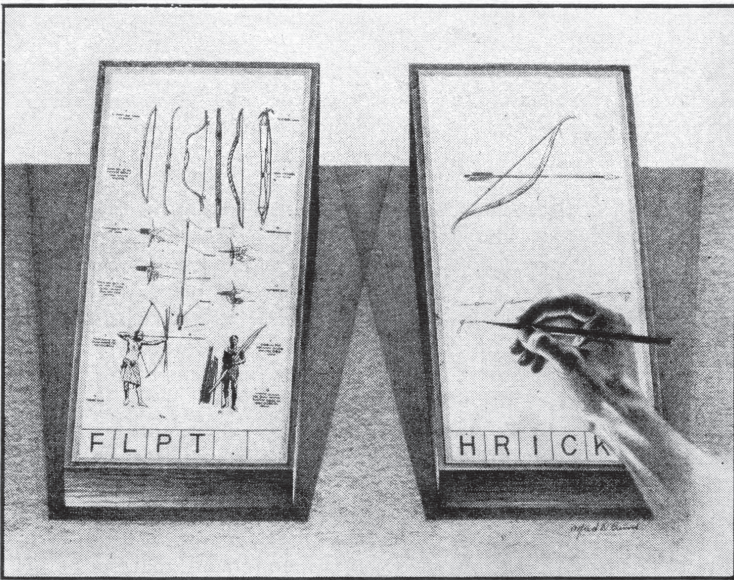
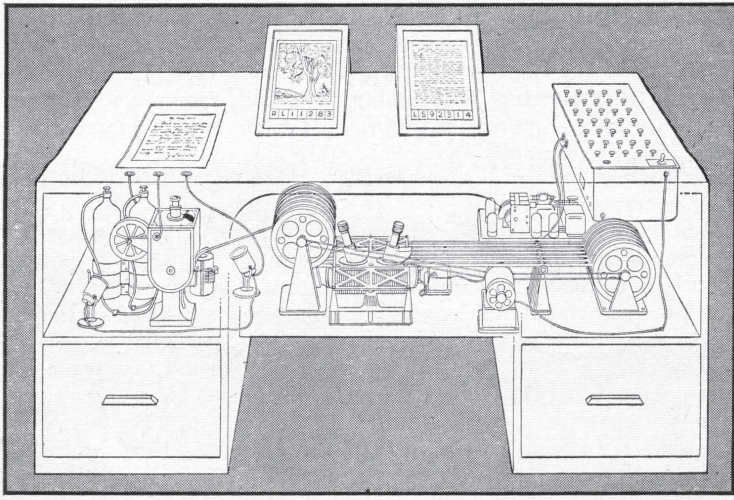


Figure 47 Douglas Engelbart established the Human Factors Research Center at the Stanford Research Institute in 1963. In the era before personal computers, he helped shape the way computers would be used for office work. This photograph, taken about 1968, shows him with his best-known invention, the point-and-click mouse, which subsequently became universal on desktop computers. COURTESY OF SRI INTERNATIONAL.

Figure 48 Bill Gates has engendered more interest and controversy than any other figure in the history of the computer. He co-founded Microsoft, which developed the MS-DOS and Windows operating systems installed on hundreds of millions of personal computers. Gates, who was simultaneously portrayed as an archetypal computer nerd and a ruthless, Rockefeller-style businessman, came to epitomize the entrepreneurs riding the rollercoaster of the information economy. USED WITH PERMISSION FROM MICROSOFT.





Figures 49 and 50 In 1945 Vannevar Bush proposed the “memex,” an information machine to help deal with the postwar information explosion. His concept was an important influence on hypertext and other innovations that eventually led to the World Wide Web. The upper illustration shows the microfilm mechanism which would store vast amounts of information; the lower illustration shows the viewing screens close-up. COURTESY OF CHARLES BABBAGE INSTITUTE ARCHIVES (CBIA), UNIVERSITY OF MINNESOTA.

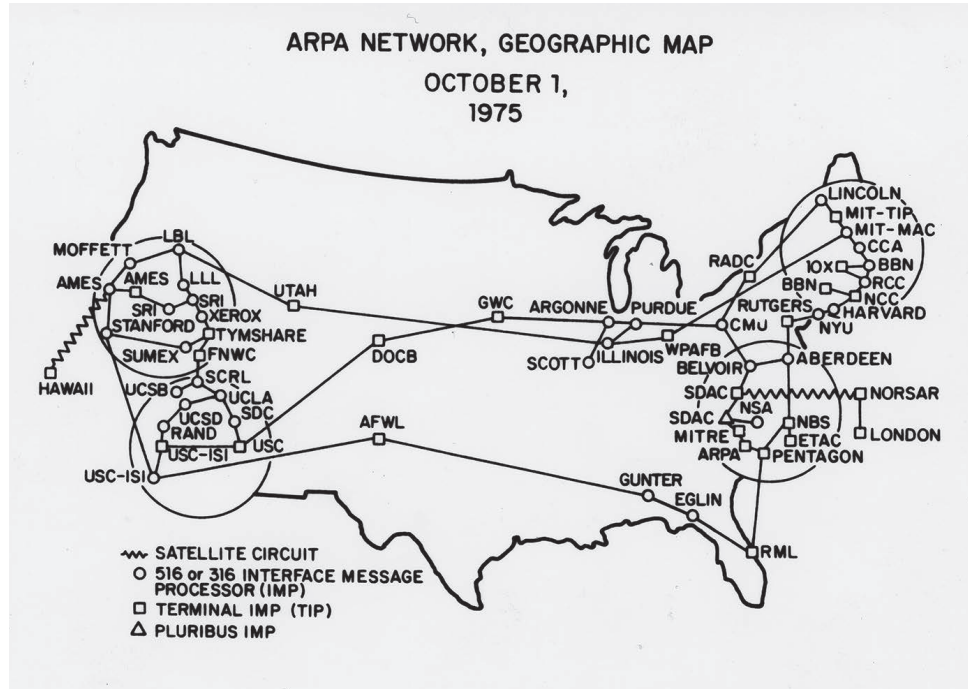
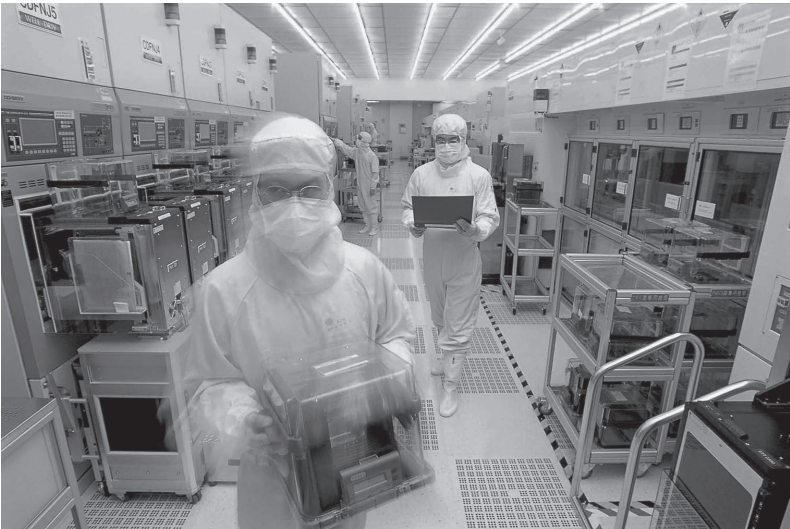


Figure 51 This diagram shows the internet in 1975, when there were just a few dozen networked computers. The availability of low-cost personal computers and the invention of the World Wide Web created an explosion of interest among both hobbyists and commercial users in the early 1990s. By 1995, when the internet was privatized, there were estimated to be about 16 million users of the internet. By 2020, there were estimated to be about 4.5 billion users, more than half of the world's population. COURTESY OF CHARLES BABBAGE INSTITUTE ARCHIVES (CBIA), UNIVERSITY OF MINNESOTA.



Figures 52 and 53 Top: No. 8 fabrication plant of the Taiwan Semiconductor Manufacturing Corporation (TSMC). This 8-inch wafer fabrication plant, set up in the 1990s, is located in Hsinchu Science Park, dubbed Taiwan's Silicon Valley. Rather than designing its own chips, TSMC's strategy has been to focus solely on the fabrication of its customers' designs. Bottom: Two TSMC employees working in a fabrication plant of 8-inch wafers. COURTESY OF TSMC.



Figure 54 “Mobile Lovers” by Banksy. This street-art image appeared on the door of a youth club in Bristol, UK, in April 2014. It was removed for safe keeping, displayed at the Bristol Museum and Art Gallery, and subsequently sold to private buyer. The proceeds were used to support the youth club. PHOTO BY SALLY ILLET.



Figure 55 Launched in 2007, the Apple iPhone revolutionized mobile computing. The phone was *Time* magazine’s “Person of the Year” for 2007. COURTESY NIGEL LINGE.

Oct. 6, 1970
Filed Dec. 24, 1968

M. A. GOETZ
AUTOMATIC SYSTEM FOR CONSTRUCTING
AND RECORDING DISPLAY CHARTS

3,533,086
38 Sheets-Sheet 1

Fig. 1.

Mar.
BY
Millman

Would you give her AUTOFLOW® for Christmas?

You Should!
Most programmers and managers agree that AUTOFLOW is "great". Christmas time, any time, or all year 'round. Your favorite programmer would appreciate AUTOFLOW because AUTOFLOW users at over 300 installations agree that they're enjoying ultimate performance in automatic flowcharting—with AUTOFLOW. ADR's AUTOFLOW, a Computer Documentation System, is a proprietary, automated flowchart system that accepts FORTRAN, COBOL, PL-1 or Assembly language and produces two dimensional flowcharts accurately, completely, and effortlessly.

AUTOFLOW is available on lease or service basis for IBM System/360, 1400 and 7090 series, RCA Spectra 70, and Honeywell 200 Series.
A new, speedy 360/DOS AUTOFLOW is now available to new and existing users.
As always, AUTOFLOW can be yours for a free 30 day trial. You should review all of the merits of AUTOFLOW beyond speed, economy and dependability. They—ask yourself, "Would you give her AUTOFLOW for Christmas?"
... Or why not at least a "ting"—at 800-921-8557

 **APPLIED DATA RESEARCH, INC.**
ROUTE 208 CENTER, PRINCETON, N.J. 08540 • PHONE: 609-921-8550
Offices in principal cities throughout the world.

Figures 56 and 57 Introduced in 1966, Autoflow was the first software product. The program was one of the first to gain a modern software patent, which prevented competitors from producing a clone and selling it for a lower price. ADVERT: COURTESY OF CHARLES BABBAGE INSTITUTE ARCHIVES (CBIA), UNIVERSITY OF MINNESOTA.

9 New Ways of Computing

BY THE MID-1960s data-processing computers for business had become well established. The commercial computer installation was characterized by a large, centralized computer manufactured by IBM or one of the other half-dozen main-frame computer companies, running a batch-processing or real-time application. The role of the user in this computing environment was to feed data into the computer system and interact with it in the very restricted fashion determined by the application – whether it was an airline reservations system or an Automated Teller Machine. In the case of the latter, users were probably unaware that they were using a computer at all. In a period of twenty years, computer hardware costs had fallen dramatically, producing ever greater computing power for the dollar, but the nature of data processing had changed hardly at all. Improved machines and software enabled more sophisticated applications and many primitive batch-processing systems became real time, but data processing still consisted of computing delivered to naive users by an elite group of systems analysts and software developers.

Today, when most people think of computing, they think of personal computers, tablets, and smartphones. It is hard to see a relationship between this kind of computing and commercial data processing because there *is* no relationship. The personal computer, which preceded tablets and smartphones, grew from new ways of computing, which are the subject of this chapter. These new forms of computing included time-sharing, the computer utility, Unix, minicomputers, and new microelectronic devices.

Time-Sharing

In the mainframe era, a time-sharing computer was one organized so that it could be used simultaneously by many users, each person having the illusion of being the sole user of the system – which, in effect, became his or her personal machine.

The first time-sharing computer system originated at MIT in 1961, initially as a way of easing the difficulties that faculty and students were beginning to

encounter when trying to develop programs. In the very early days of computing, when people were using the first research machines at MIT and elsewhere, it was usual for a user to book the computer for a period of perhaps half an hour or an hour, during which he (less often she) would run a program, make corrections, and, it was hoped, obtain some results. In effect, the computer was the user's personal machine during that interval of time.

This was a very wasteful way to use a computer, however, because users would often spend a lot of time simply scratching their heads over the results and might get only a few minutes of productive computing done during the hour they monopolized the machine. Besides being an uneconomical way to use a machine costing perhaps \$100 an hour to run, it was an immense source of frustration to the dozens of people competing to use the machine, for whom there were simply not enough hours of hands-on access in the day. Up until the early 1960s this problem had been addressed by batch-processing operating regimens. In a batch-processing system, in order to get the maximum amount of work from an expensive computer, it was installed in a "computer center"; users were required to punch their programs onto cards and submit them to the computer – not directly, but through the center's reception desk. In the computer center, a team of operators would put several programs together (a batch) and run them through the machine in quick succession. Users would then collect their results later the same day or the following day.

When the Department of Electrical Engineering at MIT got its first commercially manufactured computer, an IBM 704, in 1957, it introduced a batch-processing regimen. Although batch processing made excellent use of the machine's time, faculty and researchers found it made very poor use of their time. They had to cool their heels waiting for programs to percolate through the system, which meant they could test and run their programs only once or twice a day. A complex program might take several weeks to debug before any useful results were obtained. Much the same was happening at other universities and research organizations around the world.

A possible solution to this problem, time-sharing, was first described by a British computer scientist named Christopher Strachey in 1959 when he proposed the idea of attaching several operating consoles, each complete with a card reader and a printer, to a single mainframe computer. With some clever hardware and software, it would then be possible for all the users simultaneously to use the expensive mainframe computer, which would be sufficiently fast that users would be unaware that they were sharing it with others. At MIT John McCarthy, also a pioneer in artificial intelligence, came up with a similar, though more intimate, idea: users of the time-sharing system would communicate through typewriter-like terminals rather than via the consoles that Strachey had envisaged.

The computing center at MIT was the first to implement a time-sharing system, in a project led by Robert Fano and Fernando Corbató. A demonstration

version of MIT's CTSS – the Compatible Time-Sharing System – was shown in November 1961. This early, experimental system allowed just three users to share the computer, enabling them independently to edit and correct programs and to do other information-processing tasks. For these users it was as though, once again, they had their own personal machine. Time-sharing set the research agenda for computing at MIT and many other universities for the next decade. Within a year of MIT's successful demonstration, several other universities, research organizations, and computer manufacturers had begun to develop time-sharing systems.

Dartmouth BASIC

The best known of these systems was the Dartmouth Time-Sharing System (DTSS). While MIT's CTSS was always a system for computer scientists – whatever the intentions of its designers – the one at Dartmouth College was intended for use by a much broader spectrum of users. The design of the system began in 1962, when a mathematics professor, John Kemeny (later the college's president), and Thomas E. Kurtz of the college computer center secured funding to develop a simple time-sharing computer system. They evaluated several manufacturers' equipment and eventually chose General Electric computers because of the firm's stated intentions of developing time-sharing systems. Kemeny and Kurtz took delivery of the computers at the beginning of 1964. General Electric had made some hardware modifications to support timesharing, but there was no software provided at all – Kemeny and Kurtz would have to develop their own. They therefore decided to keep the system very simple so that it could be entirely developed by a group of student programmers. The group developed both the operating system and a simple programming language called BASIC in the spring of 1964.

BASIC – the Beginners All-purpose Symbolic Instruction Code – was a very simple programming language, designed so that undergraduates – liberal arts as well as science students – could write their own programs. Until BASIC was introduced, most undergraduates were forced to write their programs in FORTRAN, which had been designed for an entirely different context than education – for scientists and engineers to write technical computer applications. FORTRAN contained many ugly and difficult features. For example, to print out a set of three numbers, one was obliged to write something like:

```
WRITE (6, 52) A, B, C
52 FORMAT (1H , 3F10.4)
```

Another problem with FORTRAN was that it was very slow: even quite short programs would take several minutes to translate on a large IBM mainframe. For industrial users this was acceptable, as they tended to write production programs

that were translated once and then used many times. But Kemeny and Kurtz rejected FORTRAN as a language for Dartmouth College's liberal arts students. They needed a new language that was

simple enough to allow the complete novice to program and run problems after only several hours of lessons. It had to respond as expected to most common formulas and constructions. It had to provide simple yet complete error messages to allow the nonexpert to correct his program quickly without consulting a manual. It also had to permit extensions to carry on the most sophisticated tasks required by experts.¹

The language that Kemeny designed had most of the power of FORTRAN but was expressed in a much easier-to-use notation. For example, to print out three numbers a programmer would simply write:

PRINT A, B, C

Starting in April 1964, Kemeny, Kurtz, and a dozen undergraduate students began to write a BASIC translator and the operating software for the new time-sharing system. The fact that a small team of undergraduate programmers could implement a BASIC translator was itself a remarkable testament to the unfussy design and the simplicity of the language. By contrast, the big programming languages then being developed by the mainframe computer manufacturers were taking fifty or more programmer-years to develop.

Dartmouth BASIC became available in the spring of 1964, and during the following academic year incoming first-year students began learning to use the language in their foundation mathematics course. BASIC was so simple that it required just two one-hour lectures to get novices started on programming. Eventually most students, of whom only about a quarter were science majors, learned to use the system – typically for homework-type calculations, as well as for accessing a large range of library programs, including educational packages, simulations, and computer games. By 1968 the system was being used by twenty-three local schools and ten New England colleges.

BASIC was carried along on the crest of the wave of time-sharing systems that were coming into operation in the late 1960s and early 1970s. Every computer manufacturer had to provide a BASIC translator if it was to compete in the educational market. Very soon, BASIC became the introductory language for almost all nonspecialist students learning computing. In 1975 it became the first widely available programming language for the newly emerging personal computer and laid the foundations for Microsoft.

Computer scientists have often criticized BASIC, seeing it as a step backward in terms of software technology. This may have been true from a technical viewpoint, but most critics overlooked BASIC's much more important cultural

significance in establishing a user-friendly programming system that enabled ordinary people to use computers without a professional computer programmer as an intermediary. Before BASIC came on the scene there were two major constituencies of computer users: computer professionals who developed applications for other people and naive computer users, such as airline reservations clerks, who operated computer terminals in a way that was entirely prescribed by the software. BASIC created a third group: users who could develop their own programs and for whom the computer was a personal-information tool.

J.C.R. Licklider and the Advanced Research Projects Agency

Up to the mid-1960s, although the time-sharing idea caught on at a few academic institutions and research organizations, the great majority of computer users continued to use computers in the traditional way. What propelled time-sharing into the mainstream of computing was the lavish funding to develop systems that the Advanced Research Projects Agency (ARPA) began to provide in 1962. ARPA was one of the great cultural forces in shaping US computing.

ARPA was initially established as a response to the Sputnik crisis of October 1957, when the Soviet Union launched the first satellite. This event caused something close to panic in political and scientific circles by throwing into question the United States' dominance of science and technology. President Eisenhower responded with massive new support for science education and scientific research, which included ARPA to sponsor and coordinate defense-related research. The requirement for ARPA was not to deliver immediate results but, rather, to pursue goals that would have a long-term payoff.

Among ARPA's projects in 1962 was a \$7 million program to promote the use of computers in defense. The person appointed to direct this program was an MIT psychologist and computer scientist, J.C.R. Licklider. Insofar as today's interactive style of computing can be said to have a single parent, that parent was Licklider. The reason for his achievement was that he was a psychologist first and a computer scientist second.

J.C.R. "Lick" Licklider studied psychology as an undergraduate, worked at Harvard University's Acoustical Laboratory during the war, and stayed on as a lecturer. In 1950 he accepted a post at MIT and established the university's psychology program in the Department of Electrical Engineering, where he was convinced it would do the most good, by encouraging engineers to design with human beings in mind. For a period in the mid-1950s he was involved with the SAGE defense project, where he did important work on the human-factors design of radar display consoles. In 1957 he became a vice president of the Cambridge-based R&D firm Bolt, Beranek and Newman (BBN), where he formulated a manifesto for human-computer interaction, published in 1960. His classic paper, "Man-Computer Symbiosis," was to reshape computing over the next twenty years.

The single most important idea in Licklider's paper was his advocacy of the use of computers to augment the human intellect. This was a radical viewpoint at the time and, to a degree, at odds with the prevailing view of the computer establishment, particularly artificial intelligence (AI) researchers. Many computer scientists were caught up by the dream that AI would soon rival the human intellect in areas such as problem solving, pattern recognition, and chess playing. Soon, they believed, a researcher would be able to assign high-level tasks to a computer. Licklider believed that such views were utopian.

In particular, he asserted that computer scientists should develop systems that enabled people to use computers to enhance their everyday work. He believed that a researcher typically spent only 15 percent of his or her time "thinking" – the rest of the time was spent looking up facts in reference books, plotting graphs, doing calculations, and so on. It was these low-level tasks that he wanted computers to automate instead of pursuing a high-flown vision of AI. He argued that it might be twenty years before computers could do useful problem-solving tasks: "That would leave, say, five years to develop man-computer symbiosis and fifteen years to use it. The fifteen may be ten or 500, but those years should be intellectually the most creative and exciting in the history of mankind."²

Licklider was elaborating these ideas at BBN in a project he called the "Library of the Future" when he was invited in 1962 to direct a program at ARPA. Taking a wide view of his mandate, he found at his disposal the resources to pursue his vision of man-computer symbiosis.

A consummate political operator, Licklider convinced his superiors to establish within ARPA the Information Processing Techniques Office (IPTO) with a budget that for a time exceeded that of all other sources of US public research funding for computing combined. Licklider developed an effective – if somewhat elitist – way of administering the IPTO program, which was to place his trust in a small number of academic centers of excellence that shared his vision – including MIT, Stanford, Carnegie-Mellon, and the University of Utah – and give them the freedom to pursue long-term research goals with a minimum of interference. The vision of interactive computing required research advances along many fronts: computer graphics, software engineering, and the psychology of human-computer interaction. By producing a stream of new technologies of wide applicability, Licklider kept his paymasters in the government happy.

Early on, Licklider encouraged his chosen centers to develop time-sharing systems that would facilitate studies of human-computer interaction. The centerpiece of the time-sharing programs was a \$3 million grant made to MIT to establish a leading-edge system, Project MAC – an acronym that was "variously translated as standing for multiple-access computer, machine-aided cognition, or man and computer."³ Based on IBM mainframes, the system became operational in 1963 and eventually built up to a total of 160 typewriter consoles on campus and in the homes of faculty. Up to thirty users could be active at any one time. Project MAC allowed users to do simple calculations, write programs, and run

programs written by others; it could also be used to prepare and edit documents, paving the way for word-processing systems. During the summer of 1963, MIT organized a summer school to promote the time-sharing idea, particularly within the mainframe computer companies. Fernando Corbató recalled:

The intent of the summer session was to make a splash. We were absolutely frustrated by the fact that we could not get any of the vendors to see a market in this kind of machine. They were viewing it just as some special purpose gadget to amuse some academics.⁴

The summer school had a mixed success in changing attitudes. While the degree of interest in time-sharing was raised in the academic community, IBM and most of the other conservative mainframe computer manufacturers remained unmoved. It would be a couple of years before the climate for time-sharing would start to change.

By 1965 Project MAC was fully operational but was so heavily overloaded that MIT decided, with ARPA funding, to embark on an even bigger time-sharing computer system that would be able to grow in a modular way and eventually support several hundred users. This system would be known as Multics (for Multiplexed Information and Computing Service). MIT had a long-standing relationship with IBM, and it had been assumed by the senior administrators in both organizations that MIT would choose to use an IBM machine again. At the grassroots level in MIT, however, there was growing resistance to choosing another IBM machine. IBM seemed complacent, and its mainframes were technologically unsuited to time-sharing; MIT decided to choose a more forward-looking manufacturer.

Unfortunately for IBM, when it had designed its new System/360 computers in 1962, time-sharing was barely on the horizon. Already overwhelmed by the teething problems of its new range of computers, the company was unwilling to modify the design to fulfill MIT's needs. By contrast, General Electric, which had already decided to make a time-sharing computer based on the Dartmouth system, was happy not only to collaborate with MIT but to have it directly participate in the specification of its new model 645 computer, which was to be designed from the ground up as a dedicated time-sharing computer. It was an opportunity for GE to make a last effort to establish a profitable mainframe operation by attacking a market where IBM was particularly weak.

Multics was to be the most ambitious time-sharing system yet; costing up to \$7 million, it would support up to a thousand terminals, with three hundred in use at any one time. To develop the operating software for the new system, MIT and General Electric were joined by Bell Labs. Bell Labs became the software contractor because the company was rich in computing talent but was not permitted, as a government-regulated monopoly, to operate as an independent computer-services company. The Multics contract enabled Bell Labs to find an

outlet for its computing expertise and to continue building up its software-writing capabilities.

Having been pushed aside by Project MAC, IBM's top management realized that the company could no longer bury its head in the sand and would have to get into the time-sharing scene. In August 1966 it announced a new member of the System/360 family, the time-sharing model 67. By this time, most other computer manufacturers had also tentatively begun to offer their first time-sharing computer systems.

The Computer Utility

The development of the first time-sharing computer systems attracted a great deal of attention both inside and outside the computer-using community. The original idea of time-sharing was broadened into the "computer utility" concept.

The idea of a computer utility originated at MIT around 1964 as an analogy to the power utilities. Just as the ordinary electricity user drew power from a utility, rather than owning an independent generating plant, so, it was argued, computer users should be served by giant centralized mainframes from which they could draw computing power, instead of owning individual computers. Although some conservative forces regarded the computer utility as a fad, most computer users were caught up by the vision. Time-sharing and the idea of the computer utility became the "hottest new talk of the trade" in 1965.⁵ Skeptics remained, but they were a minority. One correspondent of the *Datamation* computer magazine, for example, remarked that he was "amazed" by the way the computer-using community had "overwhelmingly and almost irreversibly committed itself emotionally and professionally to time-sharing" and was being seduced by the prestige of MIT and Project MAC into "irrational, bandwagon behavior."⁶

Even if the skeptics were right (and so they were proved), the appeal of the computer utility was seemingly unstoppable. The computer and business press regularly featured articles on the subject, and books appeared with titles such as *The Challenge of the Computer Utility* and *The Future of the Computer Utility*. *Fortune* magazine reported that computer industry prophets were convinced that the computer business would "evolve into a problem-solving service . . . a kind of computer utility that will serve anybody and everybody, big and small, with thousands of remote terminals connected to the appropriate central processors."⁷

There were two key forces driving the computer utility idea, one economic and one visionary. Computer manufacturers and buyers of computer equipment were primarily interested in the economic motive, which was underpinned by "Grosch's Law."⁸ Herb Grosch was a prominent computer pundit and industry gadfly of the period whose claims to fame included a flair for self-publicity and an ability to irritate IBM; history probably would not remember him except for his one major observation. Grosch's Law quantified the economies of scale in computing by observing that the power of a computer varied as the square of its

price. Thus a \$200,000 computer would be approximately four times as powerful as one costing \$100,000; while a \$1 million computer would be twenty-five times as powerful as the \$200,000 model. Because of Grosch's Law, it was clear that it would be much cheaper to have twenty-five users sharing a single \$1 million computer than to provide each of them with their own \$200,000 computer. Without this economic motive, there would have been no financial incentive for time-sharing and no time-sharing industry.

The visionaries of the computer utility, however, were caught up by a broader aim for the democratization of computing. Martin Greenberger, a professor at the MIT Management School who was probably the first to describe the utility concept in print, argued in an *Atlantic Monthly* article that the drive for computer utilities was unstoppable and that "[b]arring unforeseen obstacles, an on-line interactive computer service, provided commercially by an information utility, may be as commonplace by A.D. 2000 as the telephone service is today."⁹

The computer utility vision was widely shared by computer pundits at the end of the 1960s. Paul Baran, a computer-communications specialist at the RAND Corporation, waxed eloquent about the computer utility in the home:

And while it may seem odd to think of piping computing power into homes, it may not be as far-fetched as it sounds. We will speak to the computers of the future in a language simple to learn and easy to use. Our home computer console . . . would be up-to-the-minute and accurate. We could pay our bills and compute our taxes via the console. We would ask questions and receive answers from "information banks" – automated versions of today's libraries. We would obtain up-to-the-minute listing of all television and radio programs. . . . We could store in birthdays. The computer could, itself, send a message to remind us of an impending anniversary and save us from the disastrous consequences of forgetfulness.¹⁰

Using computers to remind the user about impending birthdays and anniversaries was a vacuous suggestion common to many home-computing scenarios of the period. This suggests a paucity of ideas for the domestic uses of computers that was never really addressed in the stampede toward time-sharing.

By 1967 time-sharing computers were popping up all over the United States – with twenty firms competing for a market estimated at \$15 million to \$20 million a year. IBM was offering a service known as QUICKTRAN for simple calculations in five cities, with plans to double that number. General Electric had established timesharing computers in twenty US cities, and Licklider's old employer, BBN, had established a nationwide service called Telcomp. Other new entrants were Tymshare in San Francisco, Keydata in Boston, and Comshare in Ann Arbor, Michigan. One of the biggest time-sharing operations was that of the Dallas-based University Computing Company (UCC), which established major computer utilities in New York and Washington. By 1968 it

had several computer centers with terminals located in thirty states and a dozen countries. UCC was one of the glamour stocks of the 1960s; during 1967–1968 its stock price rose from \$1.50 to \$155. By the end of the 1960s, the major time-sharing firms had expanded across North America and into Europe.

Then, almost overnight, the bottom dropped out of the computer utility market. As early as 1970 the computer utility was being described by some industry insiders as an “illusion” and one of the “computer myths of the 60’s.”¹¹ By 1971 several firms were in trouble. UCC saw its share price dive from a peak of \$186 to \$17 in a few months. The demise of the time-sharing industry would set back the computer utility dream for the best part of twenty years. While simple time-sharing computer systems survived, the broad vision of the computer utility was killed off by two unforeseen and unrelated circumstances: the software crisis and the fall in hardware prices.

Big computer utilities were critically dependent on software to make them work. The more powerful the computer utility, the harder it became to write the software. IBM was the first to discover this. The company was already embroiled in the disastrous OS/360 software development for its new System/360 mainframes, and launching the time-sharing model 67 involved it in another major challenge to develop time-sharing software. With the TSS/360 time-sharing operating system originally due for delivery in the fall of 1966, the schedule slipped again and again until IBM ended up with a “team of hundreds . . . working two and a half shifts trying to get the software ready.”¹² By the time the first late and unreliable release of the software appeared in 1967, many of the original orders had been canceled. It was estimated that IBM lost \$50 million in this foray into time-sharing.

General Electric was even more badly hit by the software crisis. Having succeeded so easily with the relatively simple Dartmouth-based system, and with thirty time-sharing centers in the United States and Europe running commercially, GE expected development of the new Multics system to be straightforward. The company and its partners, MIT and Bell Labs, were wholly unprepared for the magnitude of the software development problems. After four years of costly and fruitless development, Bell Labs finally pulled out of the project in early 1969, and it was not until the end of that year that an early and rather limited version of the system limped into use at MIT. In 1970 General Electric decided to withdraw its model 645 computer from the market, fearing that the software problems were simply insuperable; a few months later it pulled out of the mainframe computer business altogether.

For the next several years the firms that remained in the time-sharing business provided systems that were modest in scope, typically with thirty to fifty users, mainly supplying niche markets for science, engineering, and business calculations. By the late 1970s the technology had matured, and the largest systems supported several hundred users. These did not survive the personal computer, however, and were in decline by the early 1980s. Time-sharing systems were

almost exclusively used in a commercial setting and never provided the domestic form of mass computing envisioned by the “computer utility.” That would have to wait until the arrival of the commercial Internet in the 1990s.

Unix

There was, however, one very beneficial spin-off of the Multics imbroglio: the Unix operating system, developed at Bell Labs during 1969–1974. The software disasters epitomized by OS/360 and Multics occurred because they were too baroque. This led a number of systems designers to explore a small-is-beautiful approach to software that tried to avoid complexity through a clear minimalist design. The Unix operating system was the outstanding example of this approach. It was the software equivalent of the Bauhaus chair.

Unix was developed by two of Bell Labs’ programmers, Ken Thompson and Dennis M. Ritchie, both of whom – bearded, long-haired, and portly – conformed to the stereotype of the 1960s computer guru. When Bell Labs pulled out of the Multics project in 1969, Thompson and Ritchie were left frustrated because working on Multics had provided them with a rather attractive programming environment:

We didn’t want to lose the pleasant niche we occupied, because no similar ones were available; even the time-sharing service that would later be offered under GE’s operating system did not exist. What we wanted to preserve was not just a good environment in which to do programming, but a system around which a fellowship could form.¹³

Thompson and Ritchie had a free rein at Bell Labs to pursue their own ideas and decided to explore the development of a small, elegant operating system that they later called Unix – “a somewhat treacherous pun on ‘Multics.’”¹⁴ Hardware was less forthcoming than freedom to pursue this research whim, however, and they “scrounged around, coming up with a discarded obsolete computer”¹⁵ – a PDP-7 manufactured by the Digital Equipment Corporation (PDP stands for “programmed data processor”). The PDP-7 was a small computer designed for dedicated laboratory applications and provided only a fraction of the power of a conventional mainframe. The design of Unix evolved over a period of a few months in 1969 based on a small set of primitive concepts. (One of the more elegant concepts was the program “pipe,” which enabled the output from one program to be fed into another – an analogy to the pipework used in an industrial production process. By means of pipes, very complex programs could be constructed out of simpler ones.)

By early 1970 Unix was working to its creators’ satisfaction, providing remarkably powerful facilities for a single user on the PDP-7. Even so, Thompson and Ritchie had a hard time convincing their colleagues of its merits. Eventually, by

offering to develop some text-processing software, they coaxed the Bell Labs patent department into using the system for preparing patent specifications. With a real prospective user, funding was obtained for a larger computer, and the newly launched Digital Equipment PDP-11/45 was selected. The system was completely rewritten in a language called “C,” which was designed especially for the purpose by Ritchie. This was a “systems implementation language,” designed for writing programming systems in much the same way that FORTRAN and COBOL were designed for writing scientific and commercial applications. The use of C made Unix “portable,” so that it could be implemented on any computer system – a unique operating-system achievement at the time. (Soon C, like Unix, was to take on a life of its own.)

Unix was well placed to take advantage of a mood swing in computer usage in the early 1970s caused by a growing exasperation with large, centralized mainframe computers and the failure of the large computer utility. A joke circulating at the time ran:

Q: What is an elephant?

*A: A mouse with an IBM operating system.*¹⁶

Independent-minded users were beginning to reject the centralized mainframe in favor of a decentralized small computer in their own department. Few manufacturers offered suitable operating systems, however, and Unix filled the void. Ritchie recalled: “Because they were starting afresh, and because manufacturers’ software was, at best, unimaginative and often horrible, some adventurous people were willing to take a chance on a new and intriguing, even though unsupported, operating system.”¹⁷ This probably would have happened irrespective of Unix’s merits, since there was little else available and Bell Labs had adopted the benign policy of licensing the system to colleges and universities at a nominal cost. But the minimalist design of Unix established an immediate rapport with the academic world and research laboratories. From about 1974, more and more colleges and universities began to use the system; and within a couple of years graduates began to import the Unix culture into industry.

A major change in the Unix world occurred in 1977, when it began to grow organically: more and more software was added to the basic system developed by Thompson and Ritchie. The clean, functional design of Unix made this organic growth possible without affecting the inherent reliability of the system. Very powerful versions of Unix were developed by the University of California at Berkeley, the computer workstation manufacturer Sun Microsystems, and other computer manufacturers. In much the same way that FORTRAN had become a standard language by default in the 1950s, Unix was on the path to becoming the standard operating system for the 1980s. In 1983 Thompson and Ritchie received the prestigious A. M. Turing Award, on which it was stated: “The

genius of the UNIX system is its framework, which enables programmers to stand on the work of others.”¹⁸ Unix provided elegance and economy combined with stability, universality, and extensibility. It is one of the design masterpieces of the twentieth century.

In the late 1990s Unix took on yet another mantle as the Linux operating system. Originally started by a Finnish computer science undergraduate Linus Torvalds in 1992, the operating system was perfected and extended by contributions of countless individual programmers, working without direct payment, often on their own time. Linux has become one of the centerpieces of the “open-source movement” (discussed further in Chapter 14).

The Microelectronics Revolution

In the 1973 film *Live and Let Die*, the stylish secret agent James Bond traded his signature wristwatch, a Rolex Submariner, for the latest in high-tech gadgetry, a Hamilton Pulsar digital watch. Unlike a traditional timepiece, the Pulsar did not represent time using the sweep of hour and minute hands; instead, it displayed time digitally, using a recent innovation in microelectronics called the light-emitting diode, or LED. In the early 1970s, the glowing red lights of an LED display represented the cutting edge of integrated circuit technology, and digital watches were such a luxury item that, at \$2,100 for the 18-karat gold edition, they cost more even than an equivalent Rolex. The original Pulsar had actually been developed a few years earlier for the Stanley Kubrick film *2001: A Space Odyssey*, and, for a time, access to this technology was limited to the domain of science fiction and international espionage.

Within just a few years, however, the cost (and sex appeal) of a digital watch had diminished to almost nothing. By 1976 Texas Instruments was offering a digital watch for just \$20 and, within a year, had reduced the price again by half. By 1979 Pulsar had lost \$6 million, had been sold twice, and had reverted to producing more profitable analog timepieces. By the end of the 1970s, the cost of the components required to construct a digital watch had fallen so low that it was almost impossible to sell the finished product for any significant profit. The formerly space-age technology had become a cheap commodity good – as well as something of a cliché.

The meteoric rise and fall of the digital watch illustrates a larger pattern in the unusual economics of microelectronics manufacturing. The so-called planar process for manufacturing integrated circuits, developed at Fairchild Semiconductor and perfected by companies such as Intel and Advanced Micro Devices (AMD), required a substantial initial investment in expertise and equipment, but after that the marginal cost of production dropped rapidly. In short, the cost of building the very first of these new integrated circuit technologies was enormous, but every unit manufactured after that became increasingly inexpensive.

The massive economies of scale inherent in semiconductor manufacture – combined with rapid improvements in the complexity and capabilities of integrated circuits, intense competition within the industry, and the widespread availability of new forms of venture capital – created the conditions in which rapid technological innovation was not only possible but essential. In order to continue to profit from their investment in chip design and fabrication, semiconductor firms had to create new and ever-increasing demand for their products.

The personal computer, video game console, digital camera, and cellphone are all direct products of the revolution in miniature that occurred in the late 1960s and early 1970s. But while this revolution in miniature would ultimately also revolutionize the computer industry, it is important to recognize that it did not begin with the computer industry. The two key developments in computing associated with this revolution – the minicomputer and the microprocessor – were parallel strands unconnected with the established centers of electronic digital computing.

From MIT to Minicomputers

Between 1965 and 1975, the introduction of integrated circuit electronics reduced the cost of computer power by a factor of a hundred, undermining the prime economic justification for time-sharing – that is, sharing the cost of a large computer by spreading it across many users. By 1970 it was possible to buy for around \$20,000 a “minicomputer” with the power of a 1965 mainframe that had cost ten times as much. Rather than subscribe to a time-sharing service, which cost upward of \$10 per hour for each user, it made more economic sense for a computer user to buy outright a small time-sharing system that supported perhaps a dozen users. During the 1970s the small in-house time-sharing system became the preferred mode of computing in universities, research organizations, and many businesses.

People often suppose that the minicomputer was simply a scaled-down mainframe, emerging out of the established computer industry. This is not the case: it was in effect a computer born of MIT and the East Coast microelectronics industry. The firm most associated with the minicomputer, the Digital Equipment Corporation (DEC), was formed by two members of the MIT Lincoln Laboratory. One of them, Kenneth Olsen, was a 1950 graduate in electrical engineering from MIT who went on to become a research associate on Project Whirlwind – where he did extensive work turning the prototype core-memory development into a reliable system. The other, Harlan Anderson, had studied programming at the University of Illinois, joined Lincoln Laboratory, and was working for Olsen on a new transistor-based computer. In 1957 the two men quit to start their own company.

Although Olsen retained strong links with MIT, he was more excited by turning the technologies into real products than in academic research. In 1957 he

secured \$70,000 of venture capital from American Research and Development (ARD). ARD was founded in 1946 by a Harvard Business School professor, General George F. Doriot – the “father of venture capital”¹⁹ – to finance the commercial exploitation of technologies developed during the war. ARD was the prototype venture-capital firm, and the development of such financial operations was a key factor responsible for the dynamism of the new high-tech industries in the United States. Venture capital was very different to the business finance that had come before. In traditional capitalism, a business promotor would have to secure loans against physical assets, often against their own home – a risk that few individuals could countenance. By contrast, in venture capitalism the financier would assume all the risk in exchange for a portion of the “equity” of the company. It was a commonplace of venture capitalism that although a majority of ventures would founder, the ones that succeeded would pay handsomely for the ones that failed. This was exactly the case with ARD and the spectacular success of DEC. Most overseas countries found it very difficult to compete with US firms until they established their own venture-funding organizations.

Olsen’s aim was to go into the computer business and compete with the mainframe manufacturers. However, in the late 1950s this was not a realistic short-term goal. The barriers to entry into the mainframe business were rising. In order to enter the mainframe business, one needed three things, in addition to a central processing unit: peripherals (such as magnetic tape and disk drives), software (both applications and program development tools), and a sales force. It would cost several hundred million dollars to establish all these capabilities. Because of these formidable barriers to entering the computer industry, Doriot convinced Olsen to first establish the firm with more attainable objectives.

DEC set up operations in part of a pre-Civil War woolen mill in Maynard, Massachusetts, close to the Route 128 electronics industry. For the first three years of its existence, the company produced digital circuit boards for the booming digital electronics industry. This proved to be a successful niche and provided the funding for DEC’s first computer development. DEC announced its first computer, the PDP-1, in 1960. By this time DEC has been joined by Gordon Bell who rose to become the company’s leading computer architect (and later co-founder of the Computer Museum in Boston). Olsen chose the term “programmed data processor” because the new computer cost \$125,000 for a basic model and existing computer users “could not believe that in 1960 computers that could do the job could be built for less than \$1 million.”²⁰

How was it possible for DEC to produce a computer for a fifth or a tenth of what it cost the mainframe computer industry? The answer is that in a mainframe computer, probably only 20 percent of the cost was accounted for by the central processor – the rest was attributable to peripherals, software, and marketing. Olsen aimed the PDP-1 not at commercial data-processing users, however, but at the science and engineering market. These customers did not need advanced peripherals, they were capable of writing their own software, and they did not

require a professional sales engineer to analyze their applications. The PDP-1 and half a dozen other models produced over the next five years were a modest success, though they did not sell well enough to reshape the industry. This would happen with the introduction of the PDP-8 minicomputer in 1965. The PDP-8 was one of the first computers to exploit the newly emerging technology of integrated circuits, and as a result it was far smaller and cheaper than any of DEC's previous computers. The PDP-8 would fit in a packing case and sold for just \$18,000.

The PDP-8 was an instant success, and several hundred systems were delivered during the following year. The success of the PDP-8 enabled DEC to make its first public offering in 1966. By that time the firm had sold a total of 800 computers (half of them PDP-8s), employed 1,100 staff members, and occupied the entirety of its Maynard woolen-mill headquarters. The offering netted \$4.8 million – a handsome return on ARD's original investment. Over the next ten years, the PDP-8 remained constantly in production, eventually selling between 30,000 and 40,000 systems. Many of these new computers were used in dedicated applications, such as factory process automation, where a traditional computer would have been too expensive. Other systems were sold directly to engineering corporations for inclusion in advanced instruments such as medical scanners.

Many PDP-8s found their way into colleges and research laboratories, where their low price enabled research students and faculty to experience hands-on computing in a way that had not been possible since the 1950s. Some of these people found themselves redirecting their careers into computing, regardless of their original disciplines or intentions. Many of the users of PDP-8s became very attached to them, regarding them as their "personal" computers. Some users developed games for the machines – one of the most popular was a simulation of a moon-landing vehicle that the user had to guide to a safe landing. The experience of hands-on computing produced a strong computer-hobbyist culture, not only among students and young technicians but also in the community of seasoned engineers.

One such engineer, Stephen Gray, editor of *Electronics* magazine, founded the Amateur Computer Society (ACS) in May 1966 in order to exchange information with other hobbyists about building computers. Drawing support from a widely scattered community of technically trained enthusiasts working in universities, defense firms, and electronics and computer companies, Gray began circulating his *ACS Newsletter* to 160 members in 1966. It presented information on circuit design and component availability and compatibility, and it encouraged subscribers by printing stories of successful home-built computers. In general, however, owning a computer was not a reasonable aspiration for the average computer hobbyist, when the cheapest machine cost \$10,000. But in the 1970s the latent desire of computer hobbyists to own computers was a powerful force in shaping the personal computer.

By 1969, when the term *minicomputer* first came into popular use, small computers were a major sector of the computer industry. DEC had been joined by several other minicomputer manufacturers, such as Data General (formed by a group of ex-DEC engineers) and Prime Computer – both of which became major international firms. Several of the established electronics and computer manufacturers had also developed minicomputer divisions, including Hewlett-Packard, Harris, and Honeywell. DEC, as the first mover in the minicomputer industry, always had by far the strongest presence, however. By 1970 it was the world's third-largest computer manufacturer, after IBM and the UNIVAC Division of Sperry Rand. When the aging General Doriot retired from ARD in 1972, its original stake in DEC was worth \$350 million.

Semiconductors and Silicon Valley

For the first several decades of its existence, the geographical center of the computer industry was the East Coast of the United States, concentrated around well-established information-technology firms such as IBM and Bell Labs and elite universities such as MIT, Princeton, and the University of Pennsylvania. Beginning in the mid-1950s, however, a remarkable shift westward occurred, and by the end of the 1960s it was the Silicon Valley region of northern California that had become the place where innovation in the computer industry occurred. This thriving technological ecosystem, a productive collaboration of scientists, industry entrepreneurs, venture capitalists, military contractors, and university administrators, would become the model that other regions and countries would thereafter emulate.

Perhaps the best method for tracking this larger shift from East Coast to West is to follow the career trajectory of a single individual. In the early 1950s, Robert Noyce was a Midwestern boy from the small Iowa town of Grinnell (named after Josiah Grinnell, the abolitionist minister whom the journalist Horace Greeley had famously advised to “Go West, young man”). After graduating from local Grinnell College, however, Noyce first looked east, to the Massachusetts Institute of Technology, in order to pursue his professional and academic ambitions. Through one of his Grinnell professors, Noyce had learned of the development of the transistor in 1946 by Bell Labs scientists John Bardeen, Walter Brattain, and William Shockley. The transistor performed many of the functions of a vacuum tube, but in a smaller, more durable, more energy-efficient package. Although, as we have seen, transistors were used in the construction of many of the early electronic computers, Bell Labs was primarily interested in using them as switching circuits in its telephone networks. Noyce studied transistors as part of his pursuit of a PhD in physics and, later, as an employee of the Philadelphia-based electronics manufacturer Philco. Like most observers of the new technology, he reasonably assumed that the focus of development in transistors would be the established East Coast electronics firms.

The migration westward began with William Shockley, the most ambitious (and later, notorious) of the original Bell Labs inventors. By the mid-1950s Shockley had left Bell to found his own company, Shockley Semiconductor Laboratories, headquartered in the then sleepy rural college town of Palo Alto, California. There are a number of reasons why Shockley selected this otherwise inauspicious location, including its proximity to Stanford University (where an entrepreneurial engineering professor named Frederick Terman was actively recruiting electronics firms to locate nearby), its relative closeness to its parent company Beckman Instruments, and, last but not least, the fact that Shockley had grown up in Palo Alto and his mother still lived there. At the time, there was no reason to suspect that anyone but Shockley's employees would join him in this largely arbitrary relocation to northern California. One of the twelve bright young semiconductor physicists who did move west to join Shockley on his "PhD production line"²¹ was Robert Noyce, who joined the company shortly after its founding in 1956.

But while Shockley was an excellent physicist (in 1956, he and Bardeen and Brattain were awarded the Nobel Prize for the invention of the transistor), he was a difficult and demanding boss, and not a particularly astute entrepreneur (the particular form of transistor technology that he chose to develop had limited commercial possibilities). By 1957 he had alienated so many of his employees that eight of them (the so-called Shockley Eight, or, as Shockley himself referred to them, the "Traitorous Eight") left to form their own start-up company aimed at competing directly with Shockley Semiconductor. The leader of this group was Robert Noyce, who by this time had developed a reputation for being not only brilliant but charismatic. Among his other responsibilities, Noyce was charged with raising capital for the new firm, which at this point required him to again look to East Coast establishments. Noyce not only succeeded in raising \$1.5 million from the Fairchild Camera and Instrument Company (acquiring at the same time the name for his new company, Fairchild Semiconductor) but, in the process, also attracted to Northern California Arthur Rock, the young banker who would become one of the founding fathers of the modern venture-capital industry. The combination of semiconductor manufacturing, venture capital, and military contracting would soon transform the apricot and prune orchards around Palo Alto into the best-known high-tech development center in the world.

More than any other company, it was Fairchild Semiconductor that spawned the phenomenon we now know as Silicon Valley. To begin with, unlike Shockley Semiconductor, Fairchild focused on products that had immediate profit potential, and quickly landed large corporate and military clients. In 1959 Fairchild co-founder Jean Hoerni developed a novel method for manufacturing transistors called the planar process, which allowed for multiple transistors to be deposited on a single wafer of silicon using photographic and chemical techniques. Noyce, then head of research at Fairchild Semiconductor, extended the planar process to allow for the connection of these transistors into complete circuits. It was these

wafers of silicon that would give Silicon Valley its name, and their transformation into “integrated circuits” or “chips” that would consolidate the region’s status as the birthplace of the revolution in miniature (as well as its central role in subsequent developments in personal computing and the Internet). Integrated circuits built using the planar process could be used to replace entire subsystems of electronics, packing them into a single durable and relatively low-cost chip that could be mass-produced in high volumes.

Almost as significant as Fairchild Semiconductor’s technological innovations were its contributions to the creation of a unique “technological community” that soon took shape in Silicon Valley. The rebellious and entrepreneurial spirit that made possible the founding of Fairchild remained part of its fundamental culture. Engineers at Fairchild, many of whom were young men with little industry experience and no families, worked long hours in a fast-paced industry that encouraged risk-taking and individual initiative, in a company that discouraged hierarchy and bureaucracy. And just as Fairchild itself was a spin-off of Shockley Semiconductor, so too it spun off many start-ups, which then proceeded to spin-off still others. Half of the eighty-five major semiconductor manufacturers in the United States originated from Fairchild, including such notable firms as Intel, AMD, and National Semiconductor. Most top managers in the Silicon Valley semiconductor firms spent some period of their careers in the company; at a conference in Sunnyvale, California, in 1969, it was said that of the four hundred engineers present, fewer than two dozen had never worked for Fairchild. “Fairchild University,” as it was called, served as the training ground for an entire generation of industry entrepreneurs, many of whom relocated nearby (largely in order to make it easier to recruit other ex-Fairchild employees). In the years between 1957 and 1970, almost 90 percent of all semiconductor firms in the United States were located in Silicon Valley. Perhaps the most famous of all the “Fairchildren” was Intel, the second of the start-up firms founded by Robert Noyce. Frustrated with the growing bureaucracy at Fairchild Semiconductor, as well as with the fact that much of the profits were being funneled back to its parent company back east, Noyce once again defected. This time, he and his partner Gordon Moore had no problem raising capital. By this point Arthur Rock had established his own venture-capital firm in nearby San Francisco and, with nothing more than a verbal presentation from Noyce, was able to raise almost \$3 million in start-up money. Noyce and Moore recruited away from Fairchild the best experts in planar process manufacturing, and within two years of its incorporation in the summer of 1968, Intel (a combination of the words *integrated electronics*) was profitably producing integrated-circuit memory chips for the computer industry. Within five years it was a \$66 million company with more than 2,500 employees.

The US government served to nurture the industry by being a major customer for defense electronics – at one time accounting for 70 percent of the semiconductor market. But just as important was the furious pace of innovation: the

planar process, once perfected, promised regular and dramatic improvements in transistor density – and, as a result, processing power. The first integrated circuits produced in the early 1960s cost about \$50 and contained an average of half a dozen active components per chip. After that, the number of components on a chip doubled each year. By 1970 it was possible to make LSI chips (for Large Scale Integration) with a thousand active components. These were widely used in computer memories, and during the early 1970s they decimated the core-memory industry. The rapidly increasing density of semiconductor chips was first commented upon by Gordon Moore (the co-founder of Intel) in 1965, when he observed that the number of components on a chip had “increased at a rate of roughly a factor of two per year” and that “this rate can be expected to continue for at least ten years.” This projection became known as “Moore’s Law,” and he calculated “that by 1975, the number of components per integrated circuit will be 65,000.”²² In fact, while the rate of doubling settled down at about eighteen months, the phenomenon persisted into the twenty-first century, by which time integrated circuits contained tens of millions of components. Every investment in the underlying chip manufacturing process translated directly into improvements in the end product – and, by extension, all of the increasing number of technological systems built using integrated circuits.

Chips in Consumer Products

The “upside-down economics” of integrated circuits – the fact that prices so quickly went down even as performance rose – meant that firms like Intel were driven to innovate, in terms of both improving existing product lines and creating new markets. An excellent example of this is their expansion into the calculator industry. The first electronic calculating machines, developed in the mid-1960s, used both off-the-shelf and custom chips – as many as a hundred in a machine. They were very expensive, and the old electromechanical technology remained competitive until the late 1960s. With the use of integrated circuits, however, the cost of building a calculator could be reduced dramatically.

Price competition in the calculator transformed it from a low-volume market of primarily business users to a high-volume one for educational and domestic users. The first handheld calculators were introduced around 1971 at a price of \$100; by 1975 this had fallen to under \$5. The drop in prices drove many of the US calculator firms out of business, leaving production to Japanese and Far East firms – although they remained dependent on US-designed chips. In a similar manner, the market for digital watches would first rise and then collapse, as the technology became a commodity and world production shifted to the “rising Tigers” of Japan, Taiwan, and Korea. At the same time, however, new industries were being created around the widespread availability of off-the-shelf electronics being churned out by Silicon Valley.

Of these new users of high-density integrated circuits, perhaps the most significant was the video game industry. Although there were some early amateur

video games developed for electronic computers (often for the entertainment of bored graduate students), the commercial video game industry developed independently of (and parallel to) the electronic computer industry. Although some of the early entrants into this industry, such as Magnavox, were outgrowths of traditional electronics firms, the most influential, Atari, was associated with the amusement industry. Atari was founded in 1971 by a thirty-eight-year-old entrepreneur, Nolan Bushnell. Its first product was an electronic table-tennis game called Pong (so called to avoid trademark infringement on the name Ping-Pong). The company initially supplied its Pong game for use in amusement arcades, much as the rest of the amusements industry supplied pinball machines and jukeboxes. Its success spawned both US and Japanese competitors, and video games became an important branch of the amusements industry.

Whereas other firms were devastated by the collapse in chip prices, Bushnell saw an opportunity to move Atari from low-volume production for arcades to high-volume production for the domestic market, much as had happened with calculators. Atari developed a domestic version of Pong, which plugged into the back of an ordinary TV set. The home version of Pong hit the stores in time for Christmas 1974 and sold for \$350. Atari followed up with a string of domestic games, and by 1976 the firm had annual sales of \$40 million, even though the price of a typical game had dropped to \$50. By this time video games had become a major industry with many firms. Because the value of a video game system was largely in the software (the games themselves) rather than in the increasingly commodified hardware, the intense price cutting in the underlying technology did not cause the same degree of disruption that it did for calculator and digital watch manufacturers. If anything, the rising cost of developing games software led to further consolidation in the industry around a few key manufacturers. In fact, in 1976 Bushnell felt the need to sell Atari to Time-Warner, which had sufficient money to develop the business.

Although the video game industry would collapse briefly in the early 1980s, ultimately video games would become one of the largest entertainment industries in the world, rivaling even television and the motion picture industries. In the late 1970s their popularity helped define the market for consumer electronics and, in many ways, shaped the course of the development of the personal computer. The two technologies were closely related, with many microcomputers (as they were then known) being purchased primarily to play games, and many video game manufacturers (including Atari) also manufacturing microcomputers. Both industries were a direct outgrowth of the semiconductor firms that took root in Silicon Valley in the previous decade rather than of established electronic-computing companies like IBM or DEC. It was only in the early 1980s that these two industries (semiconductors and electronic computers) would converge around a single product – the personal computer.

IN THE NEXT PART of this book we describe the development of the personal computer and the arrival of the Internet. Computing did not evolve from the

world of the mainframe to personal computing and the Internet in one giant leap. The purpose of this part of the book has been to describe some of the incremental steps by which computing was transformed: these steps included real-time computing, the maturing of software as a technology, time-sharing and BASIC, the computer utility idea, and the dramatic transformation of semiconductor electronics. These steps were not necessarily connected, though some were – for example, Unix was a direct outcome of the deepening understanding of software, and minicomputers were made possible only by the miniaturization and cost reduction of electronics. It was the coming together of these largely uncoordinated and disparate developments that enabled the personal computer and all that was to follow.

Notes to Chapter 9

Judy E. O'Neill's PhD dissertation "The Evolution of Interactive Computing Through Time-Sharing and Networking" (1992) gives the best overview of the development of time-sharing. Time-sharing at MIT was the subject of a double special issue of *Annals of the History of Computing*, edited by J. A. N. Lee (1992a and 1992b). An early account of the Dartmouth Time-Sharing System is John Kemeny and Thomas Kurtz's article "Dartmouth Time-Sharing," published in *Science* (1968). The history of Dartmouth BASIC is described in Kemeny's contributed chapter to Richard Wexelblat's *History of Programming Languages* (1981). More recently, Joy Lisi Rankin's *A People's History of Computing in the United States* (2018) describes the technical and social histories of the development of time-sharing and BASIC at Dartmouth. A good historical account of Unix is Peter Salus's *A Quarter Century of Unix* (1994). Glyn Moody's *Rebel Code* (2001) describes the Linux phenomenon.

A good overview of the minicomputer industry is given in Tom Haigh and Paul Ceruzzi's *A New History of Modern Computing* (2021). The most authoritative history of the Digital Equipment Corporation is *DEC Is Dead, Long Live DEC* (Edgar Schein et al. 2003); also useful is Glenn Rifkin and George Har-rar's *The Ultimate Entrepreneur: The Story of Ken Olsen and Digital Equipment Corporation* (1985).

Good histories of the rise of the microelectronics industry are Ernest Braun and Stuart MacDonald's *Revolution in Miniature* (1982), P. R. Morris's *A History of the World Semiconductor Industry* (1990), and Andrew Goldstein and William F. Aspray's edited volume *Facets: New Perspectives on the History of Semiconductors* (1997). Also worth reading is Michael Malone's *The Microprocessor: A Biography* (1995). Leslie Berlin's *The Man Behind the Microchip* (2005) is a fine biography of Robert Noyce. The Silicon Valley phenomenon is explored in AnnaLee Saxenian's *Regional Advantage: Culture and Competition in Silicon Valley and Route 128* (1994), Martin Kenney's edited volume *Understanding Silicon Valley* (2000), and Christophe Lécuyer's *Making Silicon Valley* (2006).

There is no monographic study of the calculator industry, although An Wang's *Lessons* (1986) and Edwin Darby's *It All Adds Up* (1968) give useful insights into the decline of the US industry. The most analytical account of the early US video game industry is Ralph Watkins's Department of Commerce report *A Competitive Assessment of the U.S. Video Game Industry* (1984). Of the popular books on the video game industry, Steven Poole's *Trigger Happy* (2000) and Scott Cohen's *Zap! The Rise and Fall of Atari* (1984) offer good historical insights. Leonard Herman's *Phoenix: The Fall and Rise of Home Videogames* (1997) contains a wealth of chronologically arranged data. Nick Montfort and Ian Bogost's *Racing the Beam* (2009) represents the best of the emerging scholarship on video game platform studies.

Notes

1. Kemeny and Kurtz 1968, p. 225.
2. Licklider 1960, p. 132.
3. Fano and Corbato 1966, p. 77.
4. Lee 1992a, p. 46.
5. Main 1967, p. 187.
6. Quoted in O'Neill 1992, pp. 113–4.
7. Burck 1968, pp. 142–3.
8. See Ralston, Reilly, and Hemmendinger 2000, p. 962; Grosch 1989, pp. 180–2.
9. Greenberger 1964, p. 67.
10. Baran 1970, p. 83.
11. Gruenberger 1971, p. 40.
12. Main 1967, p. 187.
13. Ritchie 1984a, p. 1578.
14. *Ibid.*, p. 1580.
15. Slater 1987, p. 278.
16. See, for example, Barron 1971, p. 1.
17. Ritchie 1984b, p. 758.
18. *Ibid.*
19. Gompers 1994, p. 5.
20. Quoted in Fisher et al. 1983, p. 273.
21. Wolfe 1983, p. 352.
22. Quotations from Moore 1965.



Taylor & Francis

Taylor & Francis Group

<http://taylorandfrancis.com>

Part Four

Getting Personal



Taylor & Francis

Taylor & Francis Group

<http://taylorandfrancis.com>

10 The Shaping of the Personal Computer

TODAY'S PERSONAL COMPUTING landscape of laptops, tablets, and smartphones was set in train by the development of the so-called "desktop" personal computer in the period from the mid-1970s to the early 1980s. Personal computing was perhaps the most life-changing consumer phenomenon of the second half of the twentieth century, and it continues to surprise in its ever-evolving applications and forms. Why did personal computing happen where it did and when it did? Why did people want to bring an expensive and inconvenient information technology into their homes? We know much of *what* happened, but less of *why* it happened.

If we consider an earlier invention, domestic radio, it took about 50 years for historians to start writing really satisfactory accounts. We should not expect to fully understand personal computing in a lesser time scale. The histories that are being written today are necessarily contingent, and this is true of what we write here. The history of the personal computer lies in a rich interplay of cultural forces and commercial interests.

Radio Days

If the idea that the personal computer was shaped by an interplay of cultural forces and commercial interests appears nebulous, it is helpful to compare the development of the personal computer with the development of radio in the opening decades of the twentieth century, whose history is well understood. There are some useful parallels in the social construction of these two technologies, and an understanding of one can deepen one's understanding of the other.

In the 1890s the phenomenon we now call radio was a scientific novelty in search of an application. Radio broadcasting as we now know it would not emerge for a generation. The first commercial application of the new technology was in telegraphy, by which Morse signals were transmitted from one point to another – a telegraph without wires. Wireless telegraphy was demonstrated in a very public and compelling way in December 1901 when Guglielmo Marconi transmitted the letter S repeatedly in Morse across the Atlantic from Poldu

in Cornwall, England, to St. Johns, Newfoundland, Canada. Banner newspaper headlines reported Marconi's achievement, and his firm began to attract the attention of telegraph companies and private investors.

Over the next few years, wireless telegraphy was steadily perfected and incorporated into the world's telegraph systems, and voice transmission and marine-based telegraphs were developed. The latter, particularly, captured many headlines. In 1910 a telegraph message from the *S.S. Montrose* resulted in the capture of the "acid-bath murderer" Dr. Hawley Crippen as he fled from England to Canada. Two years later the life-saving role of the telegraph in the *Titanic* disaster resulted in legislation mandating that all ships holding fifty or more people must carry a permanently manned wireless station. All this media attention served to reinforce the dominant mode of the technology for the point-to-point transmission of messages.

While wireless telegraphy was in the process of being institutionalized by the telegraph companies and the government, it also began to draw the attention of men and boy hobbyists. They were attracted by the glamour associated with wireless telegraphy and by the excitement of "listening in." But mostly the amateurs were attracted by the technology itself – the sheer joy of constructing wireless "sets" and communicating their enthusiasm to like-minded individuals. By 1917 there were 13,581 licensed amateur operators in the United States, and the number of unlicensed receiving stations was estimated at 150,000.

The idea of radio broadcasting arose spontaneously in several places after World War I, although David Sarnoff (later of RCA) is often credited with the most definite proposal for a "radio music box" while he was working for the American Marconi Company in New York. Broadcasting needed an audience, and radio amateurs constituted that first audience. But for the existence of amateur operators and listeners, radio broadcasting might never have developed.

Once the first few radio stations were established, broadcasters and listeners were caught in a virtuous circle: more listeners justified better programs, and better programs enticed more listeners. Between 1921 and 1922, 564 radio stations came into existence in the United States. The flood fueled a demand for domestic radio receivers, and the radio-set industry was born. The leading firm, RCA, led by David Sarnoff, sold \$80 million of radio sets in the four years beginning with 1921. Existing firms such as Westinghouse and General Electric also began to make radio sets, competing fiercely with start-up firms such as Amrad, De Forest, Stromberg-Carlson, Zenith, and many more. By the mid-1920s the structure of American radio broadcasting had been fully determined, and it was remarkably resilient to assaults in turn from cinema, television, satellite broadcasting, and cable.

Three key points emerge from this thumbnail history of American radio broadcasting. First, radio came from a new enabling technology whose long-term importance was initially unrecognized. Originally promoted as a point-to-point communications technology, radio was reconstructed into something quite

different: a broadcast entertainment medium for the mass consumer. Second, a crucial set of actors in this transformation were the radio amateurs. They built the first receivers when there was no radio-set industry, thus enabling broadcasting to take off. They are the unsung heroes of the radio story. Finally, once radio broadcasting was established, it was quickly dominated by a few giant firms – radio-set manufacturers and broadcasters. Some of these firms were the creations of individual entrepreneurs, while others came from the established electrical engineering industry. Within a decade, the firms were virtually indistinguishable.

As we shall see, the personal computer followed a similar path of development. There was an enabling technology, the microprocessor, which took several years to be used in a mass-market product for consumers. The computer amateur played an important but underappreciated role in this transformation, not least by being a consumer for the first software companies – whose role was analogous to that of the radio broadcasters. And the personal computer spawned a major industry – with entrants coming from both entrepreneurial start-ups and established computer firms such as IBM.

Microprocessors

The enabling technology for the personal computer, the microprocessor, was developed during the period 1969–1971 in the semiconductor firm Intel. (Like many of the later developments in computer history, the microprocessor was independently invented in more than one place – but Intel was undoubtedly the most important locus.) As noted in Chapter 9, Intel was founded in 1968 by Robert Noyce and Gordon Moore, both vice presidents of Fairchild Semiconductor and two of the original Shockley Eight. The microprocessor itself, however, was suggested not by them but by Intel engineer Ted Hoff, then in his early thirties.

When Intel first began operations in 1968, it specialized in the manufacture of semiconductor memory and custom-designed chips. Intel's custom-chip sets were typically used in calculators, video games, electronic test gear, and control equipment. In 1969 Intel was approached by the Japanese calculator manufacturer Busicom to develop a chip set for a new scientific calculator – a fairly up-market model that would include trigonometric and other advanced mathematical functions. The job of designing the chip set was assigned to Ted Hoff and his co-workers.

Hoff decided that instead of specially designed logic chips for the calculator, a better approach would be to design a general-purpose chip that could be programmed with the specific calculator functions. Such a chip would of course be a rudimentary computer in its own right, although it was some time before the significance of this dawned inside Intel.

The new calculator chip, known as the 4004, was delivered to Busicom in early 1971. Unfortunately, Busicom soon found itself a victim of the calculator

price wars of the early 1970s and went into receivership. Before it did so, however, it negotiated the price of the 4004 downward in exchange for Intel acquiring the rights to market the new chip on its own account. Intel did this in November 1971 by placing an advertisement in *Electronics News* that read: "Announcing a new era of integrated electronics . . . a microprogrammable computer on a chip."¹ The company's first microprocessor sold for about \$1,000.

The phrase "computer on a chip" was really copywriter's license; in any real application several other memory and controller chips would need to have been attached to the 4004. But it was a potent metaphor that helped reshape the microelectronics industry over the next two years. During this period Intel replaced the 4004, a relatively low-powered device that processed only four bits of information at a time, with an eight-bit version, the 8008. A subsequent chip, the 8080, which became the basis for several personal-computer designs, appeared in April 1974. By this time other semiconductor manufacturers were starting to produce their own microprocessors – such as the Motorola 6800, the Zilog Z80, and the MOS Technology 6502. With this competition, the price of microprocessors soon fell to around \$100.

It would not be for another three years, however, that a real personal computer emerged, in the shape of the Apple II. The long gestation of the personal computer contradicts the received wisdom of its having arrived almost overnight. It was rather like the transition from wireless telegraphy to radio broadcasting, which the newspapers in 1921 saw as a "fad" that "seemed to come from nowhere";² in fact, it took several years, and the role of the hobbyist was crucial.

Computer Hobbyists and "Computer Liberation"

The computer hobbyist was typically a young male technophile. Most hobbyists had some professional competence. If not working with computers directly, they were often employed as technicians or engineers in the electronics industry. The typical hobbyist had cut his teeth in his early teens on electronics construction kits, bought through mail-order advertisements in one of the popular electronics magazines. Many of the hobbyists were active radio amateurs. But even those who were not radio amateurs owed much to the "ham" culture, which descended in an unbroken line from the early days of radio. After World War II, radio amateurs and electronics hobbyists moved on to building television sets and hi-fi kits advertised in magazines such as *Popular Electronics* and *Radio Electronics*. In the 1970s, the hobbyists lighted on the computer as the next electronics bandwagon.

Their enthusiasm for computing had in many cases been produced by the hands-on experience of using a minicomputer at work or in college. The dedicated hobbyist hungered for a computer at home for recreational use, so that he could explore its inner complexity, experiment with computer games, and hook it up to other electronic gadgets. However, the cost of a minicomputer – typically

\$20,000 for a complete installation – was way beyond the pocket of the average hobbyist. To the nonhobbyist, why anyone would have wanted his own computer was a mystery: it was sheer techno-enthusiasm, and one can no more explain it than one can explain why people wanted to build radio sets sixty years earlier when there were no broadcasting stations.

It is important to understand that the hobbyist could conceive of hobby computing only in terms of the technology with which he was familiar. This was not the personal computer as we know it today; rather, the computing that the hobbyist had in mind in the early 1970s was a minicomputer hooked up to a teletype equipped with a paper-tape reader and punch for getting programs and data in and out of the machine. While teletypes were readily available in government surplus shops, the most expensive part of the minicomputer – the central processing unit – remained much too costly for the amateur. The allure of the microprocessor was that it would reduce the price of the central processor by vastly reducing the chip count in the conventional computer.

The amateur computer culture was widespread. While it was particularly strong in Silicon Valley and around Route 128, Boston, computer hobbyists were to be found all over the country. The computer hobbyist was primarily interested in tinkering with computer hardware; software and applications were very much secondary issues.

Fortunately, the somewhat technologically fixated vision of the computer hobbyists was leavened by a second group of actors: the advocates of “computer liberation.” It would probably be overstating the case to describe computer liberation as a movement, but there was unquestionably a widely held desire to bring computing to ordinary people. Computer liberation was particularly strong in California, and this perhaps explains why the personal computer was developed in California rather than, say, around Route 128.

Computer liberation sprang from a general malaise in the under-thirty crowd in the post-Beatles, post-Vietnam War period of the late 1960s and early 1970s. There was still a strong anti-establishment culture that expressed itself through the phenomena of college dropouts, campus riots, communal living, hippie culture, and alternative lifestyles sometimes associated with drugs. Such a movement for liberation would typically want to wrest communications technologies from vested corporate interests. In an earlier generation the liberators might have wanted to appropriate the press, but in fact the technology of printing and distribution channels were freely available, so the young, liberal-minded community was readily able to communicate through popular magazines such as *Rolling Stone* as well as through a vast underground press. On the other hand, computer technology was unquestionably not freely available; it was mostly rigidly controlled in government bureaucracies or private corporations. The much-vaunted computer utility was, at \$10 to \$20 per hour, beyond the reach of ordinary users.

Few computer-liberation advocates came from the ranks of the student-led New Left, who commonly protested against IBM punched cards and all they

symbolized. Instead, most individuals who viewed computers as tools for liberation were politically agnostic, more focused on forming alternative communities, and inclined to embrace new technology as a means to better achieve personal liberty and human happiness – what one scholar has labeled as the “New Communalists.”³³ Stewart Brand, Stanford University biology graduate turned publishing entrepreneur, became a leading voice for the New Communalists through creating *The Whole Earth Catalog*. Deeply influenced by cybernetics visionary Norbert Wiener, electronics media theorist Marshall McLuhan, and architect and designer Buckminster Fuller, Brand pressed NASA to publicly release a satellite photo of the Earth in 1966. Two years later the photo adorned the cover of the first edition of *The Whole Earth Catalog*. Publishing regularly between 1968 and 1971, Brand’s catalog identified and promoted key products or tools for communal living and, in doing so, sought to help “transform the individual into a capable, creative person.”³⁴ The only “catalog” to ever win a National Book Award, the publication was inspirational to many personal-computer pioneers including Apple Computer co-founder Steve Jobs, who later reminisced: “*The Whole Earth Catalog* . . . was one the bibles of my generation. . . . It was a sort of Google in paperback form, 35 years before Google came along: it was idealistic, and overflowing with neat tools and great notions.”³⁵

While Brand and *The Whole Earth Catalog* offered inspiration, the most articulate spokesperson for the computer-liberation idea was Ted Nelson, the financially independent son of Hollywood actress Celeste Holm. Among Nelson’s radical visions of computing was an idea called *hypertext*, which he first described in the mid-1960s. Hypertext was a system by which an untrained person could navigate through a universe of information held on computers. Before such an idea could become a reality, however, it was necessary to “liberate” computing: to make it accessible to ordinary people at a trivial cost. In the 1970s Nelson promoted computer liberation as a regular speaker at computer-hobbyist gatherings. He took the idea further in his self-published books *Computer Lib* and *Dream Machines*, which appeared in 1974. While Nelson’s uncompromising views and his unwillingness to publish his books through conventional channels perhaps added to his anti-establishment appeal, they created a barrier between himself and the academic and commercial establishments. He influenced mainly the young, predominantly male, local Californian technical community.

Personal computing in 1974, whether it was the vision of computer liberation or that of the computer hobbyist, bore little resemblance to the personal computer that emerged three years later – that is, the configuration of a self-contained machine, somewhat like a typewriter, with a keyboard and screen, an internal microprocessor-based computing engine, and a floppy disk for long-term data storage. In 1974 the computer-liberation vision of personal computing was that of a terminal attached to a large, information-rich computer utility at very low cost, while the computer hobbyist’s vision was that of a traditional

minicomputer. What brought together these two groups, with such different perspectives, was the arrival of the first hobby computer, the Altair 8800.

The Altair 8800

In January 1975 the first microprocessor-based computer, the Altair 8800, was announced on the front cover of *Popular Electronics*. The Altair 8800 is often described as the first personal computer. This was true only in the sense that its price was so low that it could be realistically bought by an individual. In every other sense the Altair 8800 was a traditional minicomputer. Indeed, the front cover of *Popular Electronics* described it as exactly that: “Exclusive! Altair 8800. The most powerful minicomputer project ever presented – can be built for under \$400.”⁶

The Altair 8800 closely followed the electronics hobbyist marketing model: it was inexpensive (\$397) and was sold by mail order as a kit that the enthusiast had to assemble himself. In the tradition of the electronics hobbyist kit, the Altair 8800 did not always work when the enthusiast had constructed it; and even if it did work, it did not do anything very useful. The computer consisted of a single box containing the central processor, with a panel of switches and lights on the front; it had no display, no keyboard, and minimal memory. Moreover, there was no way to attach a device such as a teletype to the machine to turn it into a useful computer system.

The only way the Altair 8800 could be programmed was by entering programs in pure binary code by flicking the hand switches on the front. When loaded, the program would run; but the only evidence of its execution was the change in the shifting pattern of the lights on the front. This limited the Altair 8800 to programs that only a dedicated computer hobbyist would ever be able to appreciate. Entering the program was extraordinarily tedious, taking several minutes – but as there were only 256 bytes of memory, there was a limit to the complexity of programs that could be attempted.

The Altair 8800 was produced by a tiny Albuquerque, New Mexico, electronics kit supplier, Micro Instrumentation Telemetry Systems (MITS). The firm had originally been set up by an electronics hobbyist, Ed Roberts, to produce radio kits for model airplanes. In the early 1970s Roberts began to sell kits for building electronic calculators, but that market dried up in 1974 during the calculator wars. Although he had toyed with the idea of a general-purpose computer for some time, it was only when the more obvious calculator market faded away that he decided to take the gamble.

Despite its many shortcomings, the Altair 8800 was the grit around which the pearl of the personal-computer industry grew during the next two years. The limitations of the Altair 8800 created the opportunity for small-time entrepreneurs to develop “add-on” boards so that extra memory, conventional teletypes, and audiocassette recorders (for permanent data storage) could be added to the

basic machine. Almost all of these start-up companies consisted of two or three people – mostly computer hobbyists hoping to turn their pastime to profit. A few other entrepreneurs developed software for the Altair 8800.

The most important of the early software entrepreneurs was Bill Gates, the co-founder of Microsoft. Although his ultimate financial success was extraordinary, his background was quite typical of a 1970s software nerd – a term that conjures up an image of a pale, male adolescent, lacking in social skills, programming by night and sleeping by day, oblivious to the wider world and the need to gain qualifications and build a career. This stereotype, though exaggerated, contains an essential truth; nor was it a new phenomenon – the programmer-by-night has existed since the 1950s. Indeed, programming the first personal computers had many similarities to programming a 1950s mainframe: there were no advanced software tools, and programs had to be handcrafted in the machine's own binary codes so that every byte of the tiny memory could be used to its best advantage.

Gates, born in 1955 in Seattle to upper-middle-class parents, was first exposed to computers in 1969, when he learned to program in BASIC using a commercial time-sharing system on which his high school rented time. He and his close friend, Paul Allen, two years his senior, discovered a mutual passion for programming. They also shared a strong entrepreneurial flair from the very beginning: when Gates was only sixteen, long before the personal-computer revolution, the two organized a small firm for the computer analysis of traffic data, which they named Traf-O-Data. Whereas Allen went on to study computer science at Washington State University, Gates decided – under the influence of his lawyer father – to prepare for a legal career at Harvard University, where he enrolled in the fall of 1973. However, he soon found that his studies did not engage his interest, and he continued to program by night.

The launch of the Altair 8800 in 1975 transformed Gates's and Allen's lives. Almost as soon as they heard of the machine, they recognized the software opportunity it represented and proposed to MITS's Ed Roberts that they should develop a BASIC programming system for the new machine. Besides being easy to develop, BASIC was the language favored by the commercial time-sharing systems and minicomputers that most computer hobbyists had encountered and would therefore be the ideal vehicle for the personal-computer market. Roberts was enthusiastic, not least because BASIC would need a lot more memory to run than was normally provided with the Altair 8800; he expected to be able to sell extra memory with a high margin of profit.

Gates and Allen formed a partnership they named Micro-Soft (the hyphen was later dropped), and after six weeks of intense programming effort they delivered a BASIC programming system to MITS in February 1975. Now graduated, Allen became software director at MITS – a somewhat overblown job title for what was still a tiny firm located in a retail park. Gates remained at Harvard for a few more months, more from inertia than vocation; by the end of the academic year the direction of the booming microcomputing business was clear, and Gates

abandoned his formal education. During the next two years, literally hundreds of small firms entered the microcomputer software business, and Microsoft was by no means the most prominent.

The Altair 8800, and the add-on boards and software that were soon available for it, transformed hobby electronics in a way not seen since the heyday of radio. In the spring of 1975, for example, the "Homebrew Computer Club" was established in Menlo Park, Silicon Valley. Besides acting as a swap shop for computer components and programming tips, it provided a forum for the computer-hobbyist and computer-liberation cultures to meld.

During the first quarter of 1975, MITS received over \$1 million in orders for the Altair 8800 and launched its first "worldwide" conference. Speakers at the conference included Ed Roberts, Gates and Allen as the developers of Altair BASIC, and the computer-liberation guru Ted Nelson. At the meeting Gates launched a personal diatribe against hobbyists who pirated software. This was a dramatic position: he was advocating a shift in culture from the friendly sharing of free software among hobbyists to that of an embryonic branch of the software-products industry. Gates encountered immense hostility – his speech was, after all, the very antithesis of computer liberation. But his position was eventually accepted by producers and consumers, and over the next two years it was instrumental in transforming the personal computer from a utopian ideal into an economic artifact.

The period 1975–1977 was a dramatic and fast-moving one in which the microcomputer was transformed from a hobby machine to a consumer product. The outpouring of newly launched computer magazines remains the most permanent record of this frenzy. Some of them, such as *Byte* and *Popular Computing*, followed in the tradition of the electronics hobby magazines, while others, such as the whimsically titled *Dr. Dobb's Journal of Computer Calisthenics and Orthodontia*, responded more to the computer-liberation culture. The magazines were important vehicles for selling computers by mail order, in the tradition of hobby electronics. Mail order was soon supplanted, however, by computer stores such as the Byte Shop and ComputerLand, which initially had the ambience of an electronics hobby shop: full of dusty, government-surplus hardware and electronic gadgets. Within two years, ComputerLand would be transformed into a nationwide chain, stocking shrink-wrapped software and computers in colorful boxes.

While it had taken the mainframe a decade to be transformed from laboratory instrument to business machine, the personal computer was transformed in just two years. The reason for this rapid development was that most of the subsystems required to create a personal computer already existed: keyboards, screens, disk drives, and printers. It was just a matter of putting the pieces together. Hundreds of firms sprang up over this two-year period – not just on the West Coast but all over the United States and the world beyond. They were mostly tiny start-ups, consisting of a few computer hobbyists or young computer professionals;

they supplied complete computers, add-on boards, peripherals, or software. Within months of its initial launch at the beginning of 1975, the Altair 8800 had itself been eclipsed by dozens of new models produced by firms such as Applied Computer Technology, IMSAI, North Star, Cromemco, and Vector.

The Rise of Apple Computer

Most of the new computer firms fell almost as quickly as they rose, and only a few survived beyond the mid-1980s. Apple Computer was the rare exception in that it made it into the Fortune 500 and achieved long-term global success. Its initial trajectory, however, was quite typical of the early hobbyist start-ups.

Apple was founded by two young computer hobbyists, Stephen Wozniak and Steve Jobs. Wozniak grew up in Cupertino, California, in the heart of the booming West Coast electronics industry. Like many of the children in the area, he lived and breathed electronics. Wozniak took to electronics almost as soon as he could think abstractly; he was a talented hands-on engineer, lacking any desire for a deeper, academic understanding. He obtained a radio amateur operating license while in sixth grade, graduated to digital electronics as soon as integrated circuits became available in the mid-1960s, and achieved a little local celebrity by winning an inter-schools science prize with the design of a simple adding circuit. Unmotivated by academic studies, he drifted in and out of college without gaining significant qualifications, although he gained a good working knowledge of minicomputers.

Like many electronics hobbyists, Wozniak dreamed of owning his own mini-computer, and in 1971 he and a friend went so far as to construct a rudimentary machine from parts rejected by local companies. It was around this time that he teamed up with Steve Jobs, five years his junior, and together they went into business making “blue boxes” – gadgets that mimicked dial tones, enabling telephone calls to be made for free. While blue boxes were not illegal to make and sell, using them was illegal, as it defrauded the phone companies of revenues; but many of these hobbyists regarded it as a victimless crime – and in the moral climate of the West Coast computer hobbyist, it was pretty much on a par with pirating software. This in itself is revealing of how far cultural attitudes would shift as the personal computer made the transition from hobby to industry.

Despite his lack of formal qualifications, Wozniak’s engineering talent was recognized and he found employment in the calculator division of Hewlett-Packard in 1973; were it not for what amounted to a late-twentieth-century form of patronage that prevailed in the California electronics industry, Wozniak might have found his career confined to that of a low-grade technician or repairman.

While Wozniak was a typical, if unusually gifted, hobbyist, Steve Jobs bridged the cultural divide between computer hobbyism and computer liberation. That Apple Computer ultimately became a global player in the computer industry is largely due to Jobs’s evangelizing about the personal computer, his ability to

harness Wozniak's engineering talent, and his willingness to seek out the organizational capabilities needed to build a business.

Born in 1955, Jobs was brought up by adoptive blue-collar parents. Although not a child of the professional electronic-engineering classes, Jobs took to the electronics hobbyism that he saw all around him. While a capable enough engineer, he was not in the same league as Wozniak. There are many stories of Jobs's astounding, and sometimes overbearing, self-confidence, which had a charm when he was young but was seen as autocratic and immature when he became the head of a major corporation. One of the more celebrated stories about him is that, at the age of thirteen, when he needed some electronic components for a school project, he telephoned William Hewlett, the multimillionaire co-founder of Hewlett-Packard. Hewlett, won over by Jobs's chutzpah, not only gave him the parts but offered him a part-time job with the company.

Something of a loner, and not academically motivated, Jobs drifted in and out of college in the early 1970s before finding a well-paid niche as a games designer for Atari. An admirer of the Beatles, like them Jobs spent a year pursuing transcendental meditation in India and turned vegetarian. Jobs and Wozniak made a startling contrast: Wozniak was the archetypal electronics hobbyist with social skills to match, while Jobs affected an aura of inner wisdom, wore open-toed sandals, had long, lank hair, and sported a Ho Chi Minh beard.

The turning point for both Jobs and Wozniak was attending the Homebrew Computer Club in early 1975. Although Wozniak knew about microprocessors from his familiarity with the calculator industry, up to that point he had not realized that they could be used to build general-purpose computers and had not heard of the Altair 8800. But he had actually built a computer, which was more than could be said of most Homebrew members at that date, and he found himself among an appreciative audience. He quickly took up the new microprocessor technology and, within a few weeks, had thrown together a computer based on the MOS Technology 6502 chip. He and Jobs called it the "Apple," for reasons that are now lost in time, but possibly as a nod to the Beatles' record label.

While Jobs never cared for the "nit-picking technical debates"⁷ of the Homebrew computer enthusiasts, he did recognize the latent market they represented. He therefore cajoled Wozniak into developing the Apple computer and marketing it, initially through the Byte Shop. The Apple was a very crude machine, consisting basically of a naked circuit board and lacking a case, a keyboard, a screen, or even a power supply. Eventually about two hundred were sold, each hand-assembled by Jobs and Wozniak in the garage of Jobs's parents.

In 1976 Apple was just one of dozens of computer firms competing for the dollars of the computer hobbyist. Jobs recognized before most, however, that the microcomputer had the potential to be a consumer product for a much broader market if it were appropriately packaged. To be a success as a product, the microcomputer would have to be presented as a self-contained unit in a plastic case, able to be plugged into a standard household outlet just like any other

appliance; it would need a keyboard to enter data, a screen to view the results of a computation, and some form of long-term storage to hold data and programs. Most important, the machine would need software to appeal to anyone other than an enthusiast. First this would be BASIC, but eventually a much wider range of software would be required. This, in a nutshell, was the specification for the Apple II that Jobs passed down to Wozniak to create.

For all his naïveté as an entrepreneur Jobs understood, unlike many of his contemporaries, that if Apple was to become a successful company, it would need access to capital, professional management, public relations, and distribution channels. None of these was easy to find at a time when the personal computer was unknown outside hobbyist circles. Jobs's evangelizing was called on in full measure to acquire these capabilities. During 1976, while Wozniak designed the Apple II, Jobs secured venture capital from Mike Markkula, to whom he had been introduced by his former employer at Atari, Nolan Bushnell. Markkula was a thirty-four year-old former Intel executive who had become independently wealthy from stock options. Through Markkula's contacts, Jobs located an experienced young professional manager from the semiconductor industry, Mike Scott, who agreed to serve as president of the company. Scott would take care of operational management, leaving Jobs free to evangelize and determine the strategic direction of Apple. The last piece of Jobs's plan fell into place when he persuaded the prominent public relations company Regis McKenna to take on Apple as a client.

Throughout 1976 and early 1977, while the Apple II was being perfected, Apple Computer remained a tiny company with fewer than a dozen employees occupying two thousand square feet of space in Cupertino, California.

Software: Making Personal Computers Useful

During 1977 three distinct paradigms for the personal computer emerged, represented by three leading manufacturers: Apple, Commodore Business Machines, and Tandy, each of which defined the personal computer in terms of its own existing culture and corporate outlooks.⁸

If there can be said to be a single moment when the personal computer arrived in the public consciousness, then it was at the West Coast Computer Faire in April 1977, when the first two machines for the mass consumer, the Apple II and the Commodore PET, were launched. Both machines were instant hits, and for a while they vied for market leadership. At first glance the Commodore PET looked very much like the Apple II in that it was a self-contained appliance with a keyboard, a screen, a cassette tape for program storage, and with BASIC ready-loaded so that users could write programs.

The Commodore PET, however, coming from Commodore Business Machines – a firm that had originally made electronic calculators – was not so much a computer as a calculator writ large. For example, the keyboard had the

tiny buttons of a calculator keypad rather than the keyboard of a standard computer terminal. Moreover, like a calculator, the PET was a closed system, with no potential for add-ons such as printers or floppy disks. Nevertheless, this narrow specification and the machine's low price appealed to the educational market, where it found a niche supporting elementary computer studies and BASIC programming; eventually several hundred thousand machines were sold.

By contrast, the Apple II, although more expensive than the PET (it cost \$1,298, excluding a screen), was a true computer system with the full potential for adding extra boards and peripherals. The Apple II was therefore far more appealing to the computer hobbyist because it offered the opportunity to engage with the machine by customizing it and using it for novel applications that the manufacturers could not envisage.

In August 1977 the third major computer vendor, Tandy, entered the market, when it announced its TRS-80 computer for \$399. Produced by Tandy's subsidiary, Radio Shack, the TRS-80 was aimed at the retailer's existing customers, who consisted mainly of electronics hobbyists and buyers of video games. The low price was achieved by having the customer use a television set for a screen and an audiocassette recorder for program storage. The resulting hook-up was no hardship to the typical Tandy customer, although it would have been out of place in an office.

Thus, by the fall of 1977, although the personal computer had been defined physically as an artifact, a single constituency had not yet been established. For Commodore the personal computer was seen as a natural evolution of its existing calculator line. For Tandy it was an extension of its existing electronics-hobbyist and video games business. For Apple the machine was initially aimed at the computer hobbyist.

Jobs's ambition went beyond the hobby market, and he envisioned the machine also being used as an appliance in the home – perhaps the result of his experience as a designer of domestic video games. This ambiguity was revealed by the official description of the Apple II as a “home/personal computer.” The advertisement that Regis McKenna produced to launch the Apple II showed a housewife doing kitchen chores while in the background her husband sat at the kitchen table hunched over an Apple II, seemingly managing the household's information. The copy read:

The home computer that's ready to work, play and grow with you. . . . You'll be able to organize, index and store data on household finances, income taxes, recipes, your biorhythms, balance your checking account, even control your home environment.⁹

These domestic projections for the personal computer were reminiscent of those for the computer utility in the 1960s, and were equally misguided. Moreover, the advertisement did not point out that these domestic applications were pure

fantasy – there was no software available for “biorhythms,” accounts, or anything else.

The constituency for the personal computer would be defined by the software that was eventually created for it. At that time it was very easy to set up as a personal-computer software entrepreneur: all one needed was a machine on which to develop the software and the kind of programming know-how possessed by any talented first-year computer science student, which many hobbyists had already picked up in their teenage years. The barriers to entry into personal-computer software were so low that literally *thousands* of firms were established – and their mortality rate was phenomenal.

Up to 1976 there were only a handful of personal-computer software firms, mainly producing “system” software. The most popular products included Microsoft’s BASIC programming language and Digital Research’s CP/M operating system, which were each used in many different makes of computer. This software was usually bundled with the machine, and the firm was paid a royalty included in the overall price of the computer. In 1977 personal-computer software was still quite a small business: Microsoft had just five employees and annual sales of only \$500,000.

With the arrival of consumer-oriented machines such as the Apple II, the Commodore PET, and the Tandy TRS-80, however, the market for “applications” software took off. Applications software enabled a computer to perform useful tasks without the owner having to program the machine directly. There were three main markets for applications software: games, education, and business.¹⁰

The biggest market, initially, was for games software, which reflected the existing hobbyist customer base:

When customers walked into computer stores in 1979, they saw racks of software, wall displays of software, and glass display cases of software. Most of it was games. Many of these were outer space games – *Space*, *Space II*, *Star Trek*. Many games appeared for the Apple, including Programma’s simulation of a video game called *Apple Invaders*. Companies such as Muse, Sirius, Broderbund, and On-Line Systems reaped great profits from games.¹¹

Computer games played an important role in the early development of personal computers. Programming computer games created a corps of young programmers who were very sensitive to human-computer interaction. The most successful games were ones that needed no manuals and gave instant feedback. The most successful business software had similar, user-friendly characteristics. As for the games-software companies themselves, the great majority of them faded away. While a handful of firms became major players, the market for recreational software never grew as large as that for business applications.

The second software market was for educational programs. Schools and colleges were the first organizations to buy personal computers on a large

scale: software was needed to learn mathematics; simulation programs were needed for science teaching; and programs were needed for business games, language learning, and music. Much of this early software was developed by teachers and students in their own time and was of rather poor quality. Some major programs were developed through research grants, but because of the charitable status of its institutional creators, the software was either free or sold on a nonprofit basis. As a result the market for educational software developed haphazardly.

The market of packaged software for business applications developed between 1978 and 1980, when three generic applications enabled the personal computer to become an effective business machine: the spreadsheet, the word processor, and the database.

The first application to receive wide acceptance was the VisiCalc spreadsheet. The originator of VisiCalc was a twenty-six-year-old Harvard MBA student, Daniel Bricklin, who thought of the idea of using a personal computer as a financial analysis tool, as an alternative to using a conventional mainframe computer or a timesharing terminal. Bricklin sought the advice of a number of people, including his Harvard-professor supervisor, but they were all somewhat discouraging because his idea seemed to offer no obvious advantage over a conventional computer. Bricklin was not dissuaded, however, and during 1977–1978 he went into partnership with a programmer friend, Bob Frankston. In their spare time they developed a program for the Apple II computer. To market the program, Bricklin approached a former colleague from his MBA course who was then running a company called Personal Software, which specialized in selling games software. They decided to call the program VisiCalc, for *Visible Calculator* (and Personal Software was subsequently renamed VisiCorp).

Bricklin's program used about 25,000 bytes of memory, which was about as big as a personal computer of the period could hold, but was decidedly modest by mainframe standards. The personal computer, however, offered some significant advantages that were not obvious at the outset. Because the personal computer was a stand-alone, self-contained system, changes to a financial model were displayed almost instantaneously compared with the several seconds it would have taken on a conventional computer. This fast response enabled a manager to explore a financial model with great flexibility, asking what were known as "what if?" questions. It was almost like a computer game for executives.

When it was launched in December 1979, VisiCalc was a word-of-mouth success. Not only was the program a breakthrough as a financial tool but its users experienced for the first time the psychological freedom of having a machine of one's own, on one's desk, instead of having to accept the often mediocre take-it-or-leave-it services of a computer center. Moreover, at \$3,000, including software, an Apple II and VisiCalc could be bought on a departmental, or even a personal, budget.

The success of VisiCalc has become one of the great heroic episodes of the personal-computer revolution by getting business people to take personal

computers seriously. Apple estimated that 25,000 of the 130,000 computers it sold before September 1980 were bought on the strength of VisiCalc. Important as VisiCalc was, it seems highly likely that if it had not existed, then a word-processor or database application would have brought the personal computer into corporate use by the early 1980s.

Word processing on personal computers did not develop until about 1980. One reason for this was that the first generation of personal computers displayed only forty uppercase letters across the screen, and good-quality printers were expensive. This did not matter much when a spreadsheet was being used, but it made a personal computer much less attractive for word processing than an electric typewriter or a dedicated word-processing system. By 1980, however, new computers were coming onto the market capable of displaying eighty letters across the screen, including both upper and lower cases. The new computers could display text on the screen that was identical to the layout of the printed page – known as “what you see is what you get,” or WYSIWYG. Previously, this facility had been available only in a top-of-the-line word processor costing several thousand dollars. Low-cost “dot matrix” printers – now an almost forgotten technology, coming primarily from Japanese manufacturers – also greatly helped the word-processing market.

The first successful firm to produce word-processing software was MicroPro, founded by the entrepreneur Seymour Rubinstein in 1978. Rubinstein, then in his early forties, was formerly a mainframe software developer. He had a hobbyist interest in amateur radio and electronics, however, and when the first microcomputer kits became available, he bought one. He recognized very early on the personal computer’s potential as a word processor, and he produced a program called Word-Master in 1978. This was replaced in mid-1979 with a full WYSIWYG system called WordStar, which quickly gained a two-thirds market share. WordStar sold hundreds of copies a month, at \$450 a copy. During the next five years MicroPro sold nearly a million copies of its processing software and became a \$100-million-a-year business.

During 1980, with dozens of spreadsheet and word-processing packages on the market and the launch of the first database products, the potential of the personal computer as an office machine became clearly recognizable. At this point the traditional business-machine manufacturers, such as IBM, began to take an interest.

The IBM PC and the PC Platform

IBM was not, in fact, the giant that slept soundly during the personal-computer revolution. IBM had a sophisticated market research organization that attempted to predict market trends, and once the personal computer became clearly defined as a business machine in 1980, IBM reacted with surprising speed. The proposal that IBM should enter the personal-computer business came from William C.

Lowe, a senior manager who headed the company's "entry-level systems" division in Boca Raton, Florida. In July 1980 Lowe made a presentation to IBM's senior management in Armonk, New York, with a radical plan: not only should IBM enter the personal-computer market but it should also abandon its traditional development processes in order to match the dynamism of the booming personal-computer industry.

For nearly a century IBM had operated a bureaucratic development process by which it typically took three years for a new product to reach the market. Part of the delay was due to IBM's century-old vertical integration practice, by which it maximized profits by manufacturing in-house all the components used in its products: semiconductors, switches, plastic cases, and so on. Lowe argued that IBM should instead adopt the practice of the rest of the industry by outsourcing all the components it did not already have in production, including software. Lowe proposed yet another break with tradition – that IBM should not use its direct sales force to sell the personal computer but should instead use regular retail channels.

Surprisingly, in light of its stuffy image, IBM's top management agreed to all that Lowe recommended, and within two weeks of his presentation he was authorized to go ahead and build a prototype, which had to be ready for the market within twelve months. The development of the personal computer would be known internally as Project Chess. IBM's relatively late entry into the personal-computer market gave it some significant advantages. Not least, it could make use of the second generation of microprocessors (which processed sixteen bits of data at a time instead of eight); this would make the IBM personal computer significantly faster than other machines on the market. IBM chose to use the Intel 8088 chip, thereby guaranteeing Intel's future prosperity.

Although IBM was the world's largest software developer, paradoxically it did not have the skills to develop software for personal computers. Its bureaucratic software-development procedures were slow and methodical, and geared toward large software artifacts; the company lacked the critical skills needed to develop the "quick-and-dirty" software needed for personal computers.

IBM initially approached Gary Kildall of Digital Research – the developer of the CP/M operating system – for operating software for the new computer, and herein lies one of the more poignant stories in the history of the personal computer. For reasons now muddled, Kildall blew the opportunity. One version of the story has it that he refused to sign IBM's nondisclosure agreement, while another version has him doing some recreational flying while the dark-suited IBMers cooled their heels below. In any event, the opportunity passed Digital Research by and moved on to Microsoft. Over the next decade, buoyed by the revenues from its operating system for the IBM personal computer, Microsoft became the quintessential business-success story of the late twentieth century, and Gates became a billionaire at the age of thirty-one. Hence, for all of Gates's self-confidence and remarkable business acumen, he owed almost everything to being in the right place at the right time.

The IBM entourage arrived at Bill Gates and Paul Allen's Microsoft headquarters in July 1980. It was then a tiny (thirty-eight-person) company located in rented offices in downtown Seattle. It is said that Gates and Allen were so keen to win the IBM contract that they actually wore business suits and ties. Although Gates may have appeared a somewhat nerdy twenty-five-year-old who looked fifteen, he came from an impeccable background, was palpably serious, and showed a positive eagerness to accommodate the IBM culture. For IBM, he represented as low a risk as any of the personal-computer software firms, almost all of which were noted for their studied contempt for Big Blue. It is said that when John Opel, IBM's president, heard about the Microsoft deal, he said, "Is he Mary Gates's son?"¹² He was. Opel and Gates's mother both served on the board of the United Way.

At the time that Microsoft made its agreement with IBM for an operating system, it did not have an actual product, nor did it have the resources to develop one in IBM's time scale. However, Gates obtained a suitable piece of software from a local software firm, Seattle Computer Products, for \$30,000 cash and improved it. Eventually, the operating system, known as MS-DOS, would be bundled with almost every IBM personal computer and compatible machine, earning Microsoft a royalty of between \$10 and \$50 on every copy sold.

By the fall of 1980 the prototype personal computer, known internally as the Acorn, was complete; IBM's top management gave final authorization to go into production. Lowe, his mission essentially accomplished, moved up into the higher echelons of IBM, leaving his second-in-command, Don Estridge, in overall charge. Estridge was an unassuming forty-two-year-old. Although, as the corporate spokesman for the IBM personal computer, he later became as well known as any IBMer apart from the company's president, he never attracted as much media attention as the Young Turks such as Gates and Jobs.

The development team under Estridge was now increased to more than a hundred, and factory arrangements were made for IBM to assemble computers using largely outsourced components. Contracts for the bulk supply of subsystems were finalized with Intel for the 8088 microprocessor, with Tandon for floppy disk drives, with Zenith for power supplies, and with the Japanese company Epson for printers. Contracts were also firmed up for software. Besides Microsoft for its operating system and BASIC, arrangements were made to develop a version of the VisiCalc spreadsheet, a word processor, and a suite of business programs. A games program, Adventure, was also included with the machine, suggesting that even at this late date it was not absolutely clear whether the personal computer was a domestic machine, a business machine, or both.

Not everyone at IBM was happy to see the personal computer – whether for home or business – in the company's product line. One insider was reported as saying:

Why on earth would you care about the personal computer? It has nothing at all to do with office automation. It isn't a product for big companies that use

“real” computers. Besides, nothing much may come of this and all it can do is cause embarrassment to IBM, because, in my opinion, we don’t belong in the personal computer business to begin with.¹³

Overriding these pockets of resistance inside the company, IBM began to actively consider marketing. The economics of the personal computer determined that it could not be sold by IBM’s direct sales force because the profit margins would be too slender. The company negotiated with Sears, the United States’ largest chain of department stores, to sell the machine at its Business Centers and contracted with ComputerLand to retail the machine in its stores. For its traditional business customers, IBM would also sell the machines in its regular sales offices, alongside office products such as electric typewriters and word processors.

Early in 1981, only six months after the inception of Project Chess, IBM appointed the West Coast-based Chiat Day advertising agency to develop an advertising campaign. Market research suggested that the personal computer still lay in the gray area between regular business equipment and a home machine. The advertising campaign was therefore ambiguously aimed at both the business and home user. The machine was astutely named the IBM Personal Computer, suggesting that the IBM machine and the personal computer were synonymous. For the business user, the fact that the machine bore the IBM logo was sufficient to legitimate it inside the corporation. For the home user, however, market research revealed that although the personal computer was perceived as a good thing, it was also seen as intimidating – and IBM itself was seen as “cold and aloof.”¹⁴ The Chiat Day campaign attempted to allay these fears by featuring in its advertisements a Charlie Chaplin lookalike and alluding to Chaplin’s famous movie *Modern Times*. Set in a futuristic automated factory, *Modern Times* showed the “little man” caught up in a world of hostile technology, confronting it, and eventually overcoming it. The Charlie Chaplin figure reduced the intimidation factor and gave IBM “a human face.”¹⁵

The IBM Personal Computer was given its press launch in New York on 12 August 1981. There was intense media interest, which generated many headlines in the computer and business press. In the next few weeks the IBM Personal Computer became a runaway success that exceeded almost everyone’s expectations, inside and outside the company. While many business users had hesitated over whether to buy an Apple or a Commodore or a Tandy machine, the presence of the IBM logo convinced them that the technology was for real: IBM had legitimated the personal computer. There was such a demand for the \$2,880 machine (fully equipped) that production could not keep pace, and retailers could do no more than placate their customers by placing their names on a waiting list. Within days of the launch, IBM decided to quadruple production.

During 1982–1983 the IBM Personal Computer became an industry standard. Most of the popular software packages were converted to run on the machine, and the existence of this software reinforced its popularity. This encouraged

other manufacturers to produce “clone” machines, which ran the same software and lent momentum to the increasingly dominant “PC platform.” Producing clone machines was very easy to do because the Intel 8088 microprocessor used by IBM and almost all the other subsystems were readily available on the open market. Among the most successful of the early clone manufacturers was Houston-based Compaq, which produced its first machine in 1982. In its first full year of business, it achieved sales of \$110 million. The software industry published thousands of programs for the IBM-compatible personal computer – or the IBM PC, as the machine soon became known. Based on its rapid impact, in January 1983 the editors of *Time* magazine nominated as their “Man of the Year” not a person but a machine: the PC.

Almost all of the companies that resisted the switch to the IBM standard soon went out of business or were belatedly forced into conforming. The only important exception was Apple Computer, whose founder, Steve Jobs, had seen another way to compete with the IBM standard: not by making cheaper hardware but by making better software.

Notes to Chapter 10

A good overview of the early development of the personal computer is Paul Freiberger and Michael Swaine’s *Fire in the Valley* (1984 and subsequent editions). Stan Veit’s *History of the Personal Computer* (1993) is full of anecdotes and insights on the early development of the industry to be found nowhere else; Veit participated in the rise of the personal computer, first as a retailer and then as editor of *Computer Shopper*. The later period is covered by Robert X. Cringely’s *Accidental Empires* (1992); although less scholarly than some other books, it is generally reliable and is written with a memorable panache. Fred Turner’s *Counterculture to Cyberculture* (2006) provides a scholarly examination of the “New Communalists”’ connections with and contributions to personal computing and online communities.

Interesting analyses of entrepreneurial activity in the early personal-computer industry appear in Robert Levering et al.’s *The Computer Entrepreneurs* (1984) and Robert Slater’s *Portraits in Silicon* (1987). There have been several books written about most of the major firms in the industry. For Apple Computer we found the most useful to be Jim Carlton’s *Apple: The Inside Story* (1997) and Michael Moritz’s *The Little Kingdom: The Private Story of Apple Computer* (1984). There is a seemingly inexhaustible supply of histories of Microsoft. We have found the most useful on the early years to be Stephen Manes and Paul Andrews’s *Gates: How Microsoft’s Mogul Reinvented an Industry* (1994), James Wallace and Jim Erickson’s *Hard Drive: Bill Gates and the Making of the Microsoft Empire* (1992), and Daniel Ichbiah and Susan L. Knepper’s *The Making of Microsoft* (1991). The best account of the creation of the IBM PC is James Chposky and Ted Leonsis’s *Blue Magic* (1988).

In pursuing the analogy between the development of radio broadcasting and the personal computer, we have been guided by Susan J. Douglas's *Inventing American Broadcasting* (1987), Susan Smulyan's *Selling Radio* (1994), and Erik Barnouw's *A Tower in Babel* (1966).

Notes

1. The ad is reproduced in Augarten 1984, p. 264.
2. Douglas 1987, p. 303.
3. Turner 2006, p. 4.
4. Ibid., p. 84.
5. Jobs 2005, p. 1.
6. *Popular Electronics*, January 1975, p. 33; reproduced in Langlois 1992, p. 10.
7. Moritz 1984, p. 136.
8. See Langlois 1992 for an excellent economic analysis of the personal-computer industry.
9. Quoted in Moritz 1984, p. 224.
10. For a discussion of the personal-computer software industry, see Campbell-Kelly 2003, chap. 7.
11. Freiburger and Swaine 1984, p. 135.
12. Ichbiah and Knepper 1991, p. 77.
13. Chposky and Leonsis 1988, p. 107.
14. Ibid., p. 80.
15. *Time*, 11 July 1983, quoted in Chposky and Leonsis 1988, p. 80.

11 Broadening the Appeal

MOST NEW TECHNOLOGIES start out imperfect, difficult to master, and capable of being used only by enthusiastic early adopters. In the early 1920s, radios used crude electronic circuits that howled and drifted and needed three hands to tune; they ran on accumulators that had to be taken to the local garage for recharging, while the entertainment available often consisted of nothing more than a broadcast from the local dance hall. But over a period of a few years, the super-hetrodyne circuit transformed the stability and ease of tuning of radio sets, and it became possible to plug one into any electrical outlet; the quality of programs – drama, music, news, and sports – came to rival that available in the movie theaters. Soon “listening in” was an everyday experience.

Something very similar happened to personal computers in the 1980s, so that people of ordinary skill would be able to use them and want to use them. The graphical user interface made computers much easier to use, while software and services made them worth owning. A new branch of the software industry created thousands of application programs, while the CD-ROM disc brought information in book-like quantities to the desktop. And when computer networks enabled users to reach out to other individuals, by “chatting” or exchanging e-mail, the personal computer had truly become an information machine.

Maturing of the Personal-Computer Software Industry

On 24 August 1995 Microsoft launched Windows 95, its most important software product to date. The advance publicity was unparalleled. In the weeks before the launch, technology stocks were driven sharply higher in stock markets around the world, and in the second half of August the Microsoft publicity juggernaut reached maximum speed. Press reports estimated that the Windows 95 launch cost as much as \$200 million, of which \$8 million alone was spent acquiring the rights to use the Rolling Stones’ “Start Me Up” as background music on television commercials. In major cities, theaters were hired and video screens installed so that Microsoft’s chairman Bill Gates could deliver his address to the waiting world.

When we left Microsoft in the last chapter, in 1980, it was a tiny outfit with thirty-eight employees and annual sales of just \$8 million. Ten years later, in 1990, it had 5,600 employees and sales of \$1.8 billion. As suggested by the rise of Microsoft, the important personal-computer story of the 1980s was not hardware but software.

The personal-computer software industry developed in two phases. The first phase, which can be characterized as the gold-rush era, lasted from about 1975 to 1982; during this period, barriers to entry were extremely low, and there were several thousand new entrants, almost all of which were undercapitalized two- or three-person start-ups. The second phase, which began about 1983, following the standardization of the personal-computer market around the IBM-compatible PC, was a period of consolidation in which many of the early firms were shaken out, new entrants required heavy inputs of venture capital, and a small number of (American) firms emerged as global players.

Apart from its meteoric growth, the most remarkable aspect of the industry was its almost total disconnection from the existing software-products industry, which in 1975 was a billion-dollar-a-year business with several major international suppliers. There were both technical and cultural reasons for this failure to connect. The technical reason was that the capabilities of the existing software firms – with their powerful software tools and methodologies for developing large, reliable programs – were irrelevant for developing programs for the tiny memories of the first personal computers; indeed, they were likely to be counterproductive. Entrants to the new industry needed not advanced software-engineering knowledge but the same kind of savvy as the first software contractors in the 1950s: creative flair and the technical knowledge of a bright undergraduate. The existing software companies simply could not think or act small enough: their overhead costs did not allow them to be cost-competitive with software products for personal computers.

But the cultural reasons were at least as important. Whereas the traditional packaged-software firms marketed their programs using dark-suited salespeople with IBM-type backgrounds, the personal-computer software companies sold their products through mail order and retail outlets. And if anyone attended an industry gathering wearing a tie, someone would “cut it off and throw you in the pool.”¹

Despite the large number of entrants in the industry, a small number of products quickly emerged as market leaders. These included VisiCorp’s VisiCalc spreadsheet, MicroPro’s WordStar word processor, and Ashton-Tate’s dBase database. By the end of 1983 these three products dominated their respective markets, with cumulative sales of 800,000, 700,000, and 150,000, respectively.

Personal-computer software was a new type of product that had to evolve its own styles of marketing. When industry pundits searched for an analogy, they often likened the personal-computer software business to pop music or book publishing. For example, a critical success factor was marketing. Advertising

costs typically took up 35 cents of each retail dollar. Promotions took the form of magazine advertising, free demonstration diskettes, point-of-sale materials, exhibitions, and so on. Marketing typically accounted for twice the cost of actually developing a program. As one industry expert put it, the barriers to entry into the software business were “marketing, marketing, and marketing.”² By contrast, the manufacturing cost – which consisted simply of duplicating floppy disks and manuals – was the smallest cost component of all.

The analogy with pop music or book publishing was a good one. Every software developer was seeking that elusive “hit,” so that the marketing and R&D costs would be spread over as high a number of sales as possible.

By about 1983 the gold rush was over. It was estimated that fifteen companies had two-thirds of the market, and three significant barriers had been erected to entry into the personal-computer software business. The first was technological, caused by the dramatically improving performance of personal computers. The new generation of IBM-compatible PCs that had begun to dominate the market was capable of running software comparable with that used on small mainframes and required similar technological resources for its development. Whereas in 1979 major software packages had been written by two or three people, now teams of ten and often many more people were needed. (To take one example, while the original VisiCalc had contained about 10,000 instructions, mature versions of the Lotus 1-2-3 spreadsheet contained about 400,000 lines of code.) The second barrier to entry was know-how. The sources of knowledge of how to create personal-computer software with an attractive interface had become locked into the existing firms. This knowledge was not something that could be learned from the literature or in a computer science class. The third, and probably the greatest, barrier was access to distribution channels. In 1983 it was said that there were thirty-five thousand products competing for a place among the two hundred products that a typical computer store could stock – three hundred word-processing packages just for the IBM-compatible PC. A huge advertising expenditure, and therefore a large injection of capital, was needed to overcome this barrier.

One might have expected that these barriers to entry would have protected the existing firms such as VisiCorp, MicroPro, and Ashton-Tate, but this was not the case. By 1990 these and most of the other prominent software firms of the early 1980s had become also-rans, been taken over, or gone out of business altogether.

The reasons for this transformation are complex, but the dominating cause was the importance of a single “hit” product whose arrival could transform a balance sheet for several years, but whose demise could send the firm into a spiral of decline. Perhaps the most poignant of these dramatic turns of fortune was that of VisiCorp, the publisher of VisiCalc. At its peak in 1983, VisiCorp had annual revenues of \$40 million, but by 1985 it had ceased to exist as an independent entity. VisiCorp was effectively wiped out by the arrival of a competing product, Lotus 1-2-3.

To create a software hit, one needed access either to big sources of venture capital or to a healthy revenue stream from an existing successful product. The story of the Lotus Development Corporation, which was formed by a thirty-two-year-old entrepreneur, Mitch Kapor, in 1982, illustrates the financial barriers that had to be overcome to establish a successful start-up in computer software.³ Kapor was a charismatic freelance software developer who had produced a couple of successful packages in 1979 for VisiCorp. He had originally received a royalty on sales from VisiCorp but decided to sell all the rights to his packages to the company for a single payment of \$1.7 million. He used this cash – plus further venture-capital funds, totaling \$3 million – to develop a new spreadsheet called Lotus 1-2-3, which would compete head-on with the top-selling package, VisiCalc. In order to beat VisiCalc in the marketplace, Lotus 1-2-3 needed to be a much more technically sophisticated package, and it was estimated to cost \$1 million to develop. Lotus reportedly spent a further \$2.5 million on the initial launch of its new spreadsheet. Of the retail price of \$495, it was said that about 40 percent went into advertising. With this blaze of publicity, Lotus 1-2-3 sold 850,000 copies in its first eighteen months and instantly became the market leader.

The story of Microsoft illustrates the value of an income stream from an already successful product as an alternative to raising venture capital. By 1990 Microsoft had emerged as the outstanding leader of the personal-computer software industry, and biographies and profiles of its founder, William Henry Gates III, poured forth. In 1981, when firms such as MicroPro and VisiCorp were on the threshold of becoming major corporations, Microsoft was still a tiny outfit developing programming languages and utilities for microcomputers. Microsoft's market was essentially limited to computer manufacturers and technically minded hobbyists. However, as described in the previous chapter, in August 1980 Gates signed a contract with IBM to develop the operating system MS-DOS for its new personal computer.

As the sales of IBM-compatible computers grew, a copy of Microsoft's MS-DOS operating system was supplied with almost every machine. As hundreds of thousands, and eventually millions, of machines were sold, money poured into Microsoft. By the end of 1983 half a million copies of MS-DOS had been sold, netting \$10 million. Microsoft's position in the software industry was unique: MS-DOS was the indispensable link between hardware and applications software that every single user had to buy.

This revenue stream enabled Microsoft to diversify into computer applications without having to rely on external sources of capital. But, contrary to the mythology, Microsoft has had many more unsuccessful products than successful ones, and without the MS-DOS revenue stream it would never have grown the way it did. For example, Microsoft's first application was a spreadsheet called Multiplan, which was intended to compete with VisiCalc in much the same way that Lotus 1-2-3 had. In December 1982 Multiplan even got a Software of the

Year award, but it was eclipsed by Lotus 1-2-3. This would have been a major problem for any firm that did not have Microsoft's regular income. In mid-1982 Microsoft also began to develop a word-processing package called Word. The product was released in November 1983 with a publicity splash that rivaled even that of Lotus 1-2-3. At a cost of \$350,000 some 450,000 diskettes demonstrating the program were distributed in the *PC World* magazine. Even so, Word was initially not a successful product and had a negligible impact on the market leader, WordStar. Microsoft was still no more than a medium-sized company living off the cash cow of its MS-DOS operating system.

Graphical User Interface

For the personal computer to become more widely accepted and reach a broader market, it had to become more "user-friendly." During the 1980s user-friendliness was achieved for one-tenth of computer users by using a Macintosh computer; the other nine-tenths would later achieve it through Microsoft Windows software. Underlying both systems was the concept of the *graphical user interface*.

Until the arrival of the Macintosh, personal computers communicated with the user through a disk operating system, or DOS. The most popular operating system for the IBM-compatible PC was Microsoft's MS-DOS. Like so much else in early personal computing, DOS was derived from mainframe and mini-computer technology – in this case the efficient but intimidating Unix operating system. The early DOS-style operating systems were little better than those on the mainframes and minicomputers from which they were descended, and they were often incomprehensible to people without a computer background. Ordinary people found organizing their work in MS-DOS difficult and irritating.

The user interacted with the operating system through a "command line interface," in which each instruction to the computer had to be typed explicitly by the user, letter-perfect. For example, if one wanted to transfer a document kept in a computer file named SMITH from a computer directory called LETTERS to another directory called ARCHIVE, one had to type something like:

```
COPY A:\LETTERS\SMITH.DOC B:\ARCHIVE\SMITH.DOC  
DEL A:\LETTERS\SMITH.DOC
```

If there was a single letter out of place, the user had to type the line again. The whole arcane notation was explained in a fat manual. Of course, many technical people delighted in the intricacies of MS-DOS, but for ordinary users – office workers, secretaries, and authors working at home – it was bizarre and perplexing. It was rather like having to understand a carburetor in order to be able to drive an automobile.

The graphical user interface (GUI, pronounced “goosey”) was an attempt to liberate the user from these problems by providing a natural and intuitive way of using a computer that could be learned in minutes rather than days, for which no user manual was needed, and which would be used consistently in all applications. The GUI was also sometimes known as a WIMP interface, which stood for its critical components: Windows, Icons, Mouse, and Pull-down menus. A key idea in the new-style interface was to use a “desktop metaphor,” to which ordinary people could respond, and which was far removed from technical computing. The screen showed an idealized desktop on which would be folders and documents and office tools, such as a notepad and calculator. All the objects on the desktop were represented by “icons” – for example, a document was represented by a tiny picture of a page of typing, a picture of a folder represented a group of documents, and so on. To examine a document, one simply used the mouse to move a pointer on the screen to select the appropriate icon; clicking on the icon would then cause a “window” to open up on the screen so that the document could be viewed. As more documents were selected and opened, the windows would overlap, rather like the documents on a real desk would overlap one another.

The technology of user-friendliness long predated the personal computer, although it had never been fully exploited. Almost all the ideas in the modern computer interface emanated from two of the laboratories funded by ARPA’s Information Processing Techniques Office in the 1960s: a small human-factors research group at the Stanford Research Institute (SRI) and David C. Evans’s and Ivan Sutherland’s much larger graphics research group at the University of Utah.

The Human Factors Research Center was founded at the SRI in 1963 under the leadership of Doug Engelbart, later regarded as the doyen of human-computer interaction. Since the mid-1950s, Engelbart had been struggling to get funding to develop a computer system that would act like a personal-information storage and retrieval machine – in effect replacing physical documents by electronic ones and enabling the use of advanced computer techniques for filing, searching, and communicating documents. When ARPA started its computer research program in 1962 under the leadership of J.C.R. Licklider, Engelbart’s project was exactly in tune with Licklider’s vision of “man-computer symbiosis.” ARPA funds enabled Engelbart to build up a talented group of a dozen or so computer scientists and psychologists at SRI, where they began to develop what they called the “electronic office” – a system that would integrate text and pictures in a way that was then unprecedented.

The modern GUI owes many of its details to the work of Engelbart’s group, although by far the best-known invention is the mouse. Several pointing devices were tried, but, as Engelbart later recalled, “[t]he mouse consistently beat out the other devices for fast, accurate screen selection in our working context. For

some months we left the other devices attached to the workstation so that a user could use the device of his choice, but when it became clear that everyone chose to use the mouse, we abandoned the other devices.”⁴ All this happened in 1965; “[n]one of us would have thought that the name would have stayed with it out into the world, but the thing that none of us would have believed either was how long it would take for it to find its way out there.”

In December 1968 a prototype electronic office was demonstrated by Engelbart’s group at the National Computer Conference in San Francisco. By means of a video projector, the computer screen was enlarged to twenty-foot width so that it could be clearly seen in the large auditorium. It was a stunning presentation, and although the system was too expensive to be practical, “it made a profound impression on many of the people” who later developed the first commercial graphical user interface at the Xerox Corporation.⁵

The second key actors in the early GUI movement came from the University of Utah’s Computer Science Laboratory where, under the leadership of David Evans and Ivan Sutherland, many fundamental innovations were made in computer graphics. The Utah environment supported a graduate student named Alan Kay, who pursued a blue-sky research project that in 1969 would result in a highly influential computer science PhD thesis. Kay’s research was focused on a device, which he then called the Reactive Engine but later called the Dynabook, that would fulfill the personal-information needs of an individual. The Dynabook was to be a personal-information system, the size of a notebook, that would replace ordinary printed media. By using computer technology the Dynabook would store vast amounts of information, provide access to databases, and incorporate sophisticated information-finding tools. Of course, the system that Kay came up with for his PhD program was far removed from the utopian Dynabook. Nonetheless, at the end of the 1960s, Engelbart’s electronic office and Kay’s Dynabook concept were “the two threads” that led to the modern graphical user interface.

What held back the practical development of these ideas in the 1960s was the lack of a compact and cost-effective technology. In the mid-1960s a fully equipped minicomputer occupied several square yards of floor space and cost up to \$100,000; it was simply not reasonable to devote that much machinery to a single user. But by the early 1970s prices were falling fast and it became feasible to exploit the ideas commercially. The first firm to do so was the Xerox Corporation, America’s leading manufacturer of photocopiers.

In the late 1960s strategic planners in Xerox were becoming alarmed at the competition from Japanese photocopier manufacturers and saw the need to diversify from their exclusive reliance on the copier business. To help generate the products on which its future prosperity would depend, in 1969 Xerox established the Palo Alto Research Center (PARC) in Silicon Valley to develop the technology for “the office of the future.” Approximately half of the \$100 million poured into PARC in the 1970s was spent on computer science research. To

head up this research, Xerox recruited Robert Taylor, a former head of the ARPA Information Processing Techniques Office. Taylor was an articulate disciple of Licklider's "man-computer symbiosis" vision, and his mission at PARC was to develop "an architecture of information"⁶ that would lay the foundations for the office products of the 1980s.

The concrete realization of this mission was to be a network of "Alto" computers. Work on the Alto began in 1973, by which time a powerful team of researchers had been recruited – including Alan Kay from Utah; Butler Lampson and Charles Simonyi, originally from the University of California at Berkeley; and several other well-known industry figures such as Larry Tesler. Many of PARC's academic recruits had been previous beneficiaries of support from ARPA's IPTO, but found it difficult to secure funding in the 1970s after the agency became more focused on mission-critical military research. In the course of developing the Alto, Xerox PARC evolved the graphical user interface, which became "the preferred style in the 1980s just as time-sharing was the preferred style of the 1970s."⁷

The Alto computer was designed with a specially constructed monitor that could display an 8½ × 11-inch sheet of "paper." Unlike ordinary terminals, it displayed documents to look like typeset pages incorporating graphical images – exactly as Kay had envisaged in the Dynabook. The Alto had a mouse, as envisaged by Engelbart, and the now-familiar desktop environment of icons, folders, and documents. In short, the system was all one would later expect from a Macintosh or a Windows-based IBM-compatible PC. However, all this was taking place in 1975, before the advent of personal computers, when "it was hard for people to believe that an entire computer" could be "required to meet the needs of one person."⁸

Xerox decided to produce a commercial version of the Alto, called the Xerox Star computer – or, more prosaically, the model 8010 workstation. The new computer was the most spectacular product launch of the year when it was unveiled at the National Computer Conference in Chicago, in May 1981. With its eye-catching graphical user interface and its powerful office software, it was unmistakably a glimpse of the future. Yet, in commercial terms, the Xerox Star turned out to be one of the product disappointments of the decade. While Xerox had gotten everything right technically, in marketing terms it got almost everything wrong.

Fundamentally, the Xerox Star was too expensive: it was hard to justify the cost of a year's salary for a powerful workstation when a simple personal computer could do much the same job, albeit less glamorously, for one-fifth of the cost. Despite being a failure in commercial terms, the Xerox Star presented a vision that would transform the way people worked on computers in the 1980s.

Along with the Alto, PARC also pioneered a "local area network" technology that enabled all the elements of an office system to be connected. Known as Ethernet the technology was widely adopted but, as with the GUI, Xerox failed to reap any significant rewards from its innovation.

Steve Jobs and the Macintosh

In December 1979 Steve Jobs was invited to visit Xerox PARC. When he made his visit, the network of prototype Alto computers had just begun to demonstrate Xerox's office-of-the-future concept, and he was in awe of what he saw. Larry Tesler, who demonstrated the machines, recalled Jobs demanding, "Why isn't Xerox marketing this? . . . You could blow everybody away!"⁹ This was of course just what Xerox intended to do with the Xerox Star, which was then under development.

Returning to Apple's headquarters at Cupertino, Jobs convinced his colleagues that the company's next computer would have to look like the machine he had seen at Xerox PARC. In May 1980 he recruited Tesler from Xerox to lead the technical development of the new computer, which was to be known as the Lisa.

The Lisa took three years to develop. When it was launched in May 1983, it received the same kind of ecstatic acclaim that the Xerox Star had received two years earlier. The Lisa, however, was priced at \$16,995 for a complete system, which took it far beyond the budget of personal-computer users and even beyond that of the corporate business-machine market. Two years earlier Xerox, with its major strength in direct office sales, had not been able to make the Star successful because it was too expensive. Apple had no direct sales experience at all, and so, not surprisingly, the Lisa was a resounding commercial failure.

The failure of the Lisa left Apple dangerously exposed; its most successful machine, the aging Apple II, had been eclipsed by the IBM-compatible PC; its new machine, the Lisa, was far too expensive for the personal-computer market. What saved Apple from extinction was the Macintosh.

The Macintosh project originated in mid-1979, as the idea of Jef Raskin, then manager of advanced systems at Apple.¹⁰ He had spent time at Xerox PARC in the early 1970s and conceived of the Macintosh as an "information appliance," which would be a simple, friendly computer that the user would simply plug in and use. Physically, he envisaged the machine as a stand-alone device, with a built-in screen and a small "footprint" so that it could sit unobtrusively on the user's desk, just like a telephone. The Macintosh was named for Raskin's favorite California-grown apple – and that, as it sadly turned out for Raskin, was one of the few personal contributions to the machine he was able to make before the project was appropriated by Jobs. Raskin left Apple in the summer of 1982.

A powerful mythology has grown up around the Macintosh development. Jobs cocooned the Macintosh design group of eight young engineers in a separate building over which – only half jokingly – a pirate's flag was hoisted. Jobs had a remarkable intuitive understanding of how to motivate and lead the Macintosh design team. John Sculley, later the CEO of Apple, recalled that "Steve's 'pirates' were a hand-picked band of the most brilliant mavericks inside and outside Apple. Their mission, as one would boldly describe it, was to blow people's

minds and overturn standards. United by the Zen slogan ‘The journey is the reward,’ the pirates ransacked the company for ideas, parts, and design plans.”¹¹

The Macintosh was able to derive all the key software technology from the Lisa project. The Lisa achieved its excellent performance by using specialized hardware, which was reflected in the high price tag. Much of the design effort in the Macintosh went into obtaining Lisa-like performance at a much lower cost.

In its unique and captivating case, the Macintosh computer was to become one of the most pervasive industrial icons of the late twentieth century. As the computer was brought closer to the market, the original design team of eight was expanded to a total of forty-seven people. Jobs got each of the forty-seven to sign his or her name inside the molding from which the original Macintosh case was pressed. (Long obsolete, those original Macintoshes are now much sought after by collectors.)

In early 1983, with less than a year to go before the Macintosh launch, Jobs persuaded Sculley to become CEO of Apple. This was seen by commentators as a curious choice because the forty-year-old Sculley had achieved national prominence by masterminding the relaunch of Pepsi-Cola against Coca-Cola in the late 1970s. But behind the move lay Jobs’s vision of the computer as a consumer appliance that needed consumer marketing.

In what was one of the most memorable advertising campaigns of the 1980s, Apple produced a spectacular television advertisement that was broadcast during the Super Bowl on 22 January 1984:

Apple Computer was about to introduce its Macintosh computer to the world, and the commercial was intended to stir up anticipation for the big event. It showed a roomful of gaunt, zombie-like workers with shaved heads, dressed in pajamas like those worn by concentration camp prisoners, watching a huge viewing screen as Big Brother intoned about the great accomplishments of the computer age. The scene was stark, in dull, gray tones. Suddenly, a tanned and beautiful young woman wearing bright red track clothes sprinted into the room and hurled a sledgehammer into the screen, which exploded into blackness. Then a message appeared: “On January 24, Apple Computer will introduce the Macintosh. And you’ll see why 1984 won’t be like 1984.”¹²

Apple ran the commercial just once, but over the following weeks it was replayed on dozens of news and talk shows. The message was reinforced by a blaze of publicity eventually costing \$15 million. There were full-page newspaper advertisements and twenty-page copy inserts in glossy magazines targeted at high-income readers.

Although priced at \$2,500 – only 15 percent of the cost of the Lisa – sales of the Macintosh after the first flush of enthusiasm were disappointing. Much of the hope for the Macintosh had been that it would take off as a consumer

appliance, but it never did. Sculley realized that he had been misled by Jobs and that the consumer appliance idea was ill-conceived:

People weren't about to buy \$2,000 computers to play a video game, balance a checkbook, or file gourmet recipes as some suggested. The average consumer simply couldn't do something useful with a computer. Neither could the home market appreciate important differences in computer products. Computers largely looked alike and were a mystery for the average person: they were too expensive and too intimidating. Once we saturated the market for enthusiasts, it wasn't possible for the industry to continue its incredible record of growth.¹³

Apple Computer was not the only manufacturer whose anticipation of a consumer market for personal computers was premature. In October 1983 IBM had launched a new low-cost machine, the PC Jr., hoping to attract Christmas buyers for the domestic market. The machine was so unsuccessful it had to be withdrawn the following year. If the domestic market for computers did not exist, then the only alternative was for Apple Computer to relaunch the Macintosh as a business machine.

Unfortunately the Macintosh was not well suited to the business market either. Corporate America favored an IBM-compatible personal computer for much the same reasons that an earlier generation had preferred IBM mainframes. The Macintosh fared much better in the less conservative publishing and media industries, where its powerful "desktop publishing" capabilities made it the machine of choice. The Macintosh was also popular in education, where its ease of use made it especially attractive for young children and casual student users.

Microsoft had been made privy to the Macintosh project in 1981, when it was contracted to develop some minor parts of the operating software. Although Microsoft had been successful with its MS-DOS operating system for IBM-compatible computers, it had very little success in writing applications such as spreadsheets and word processors in the face of strong competitors such as Lotus and MicroPro. The Macintosh enabled Microsoft to develop a range of technologically sophisticated applications, largely insulated from the much more competitive IBM-compatible market. Later it would be able to convert the same applications so that they would run on the IBM-compatible PC. By 1987 Microsoft was deriving half of its revenues from its Macintosh software.

More important, working on the Macintosh gave Microsoft firsthand knowledge of the technology of graphical user interfaces, on which it based its new Windows operating system for the IBM-compatible PC.

Microsoft's Windows

The launch of the Macintosh in January 1984 made every other personal computer appear old-fashioned and lackluster, and it was clear that the next big

thing in personal computing would be a graphical user interface for the IBM-compatible PC.

In fact, following the launch of the Xerox Star in 1981, several firms had already started to develop a GUI-based operating system for the PC – including Microsoft, Digital Research, and IBM itself.¹⁴ The rewards of establishing a new operating system for the PC were immense – as Microsoft had already demonstrated with MS-DOS. The firm securing the new operating system would be assured of a guaranteed stream of income that would power its future growth.

The development of a GUI-based operating system for the PC was technologically very demanding for two reasons. First, the PC was never designed with a GUI in mind, and it was hopelessly underpowered. Second, there was a strategic problem in working around the existing MS-DOS operating system; one had to either replace MS-DOS entirely with a new operating system or else place the new operating system on top of the old, providing a second layer of software between the user's applications and the hardware. In the first case, one would no longer be able to use the thousands of existing software applications; but in the second case, one would have an inherent inefficiency.

Perhaps the company with the strongest motive for developing a GUI-based operating system was Gary Kildall's Digital Research, the original developer of CP/M, the operating system used on 8-bit microcomputers. It was estimated that 200 million copies of CP/M were eventually sold. As noted in the previous chapter, Kildall's Digital Research failed to obtain the contract to develop the IBM PC operating system, in favor of Gates's Microsoft. Although Digital Research did develop a PC operating system called CP/M 86, it was delivered too late, it was too expensive, and by Kildall's own account, it "basically died on the vine."¹⁵ The development of a new operating system presented a last chance for Digital Research to reestablish itself as the leading operating systems supplier.

Digital Research's GEM operating system (for Graphics Environment Manager) was launched in the spring of 1984. Unfortunately, as users quickly discovered, the system was in reality little more than a cosmetic improvement; although visually similar to the Macintosh, it lacked the capability of a full-scale operating system. Sales of GEM could not compensate for the ever-declining sales of Digital Research's CP/M operating system for the now largely obsolete 8-bit machines. In mid-1985, amid growing financial problems, Kildall resigned as CEO of Digital Research and the company he had formed faded from sight. IBM's TopView fared no better. It was released in 1984 but was so slow it "was dubbed 'TopHeavy' by customers and became one of the biggest flops in the history of IBM's PC business."¹⁶

Microsoft Windows was the last of the new operating systems for the PC to appear. Microsoft began work on a graphical user interface project in September 1981, shortly after Gates had visited Steve Jobs at Apple and seen the prototype Macintosh computer under development. The Microsoft project was initially called the Interface Manager, but was later renamed Windows in a neat

marketing move designed “to have our name basically define the generic.”¹⁷ It was estimated that six programmer-years would be needed to develop the system. This proved to be a gross underestimate. When version 1 of Windows was released in October 1985 – some two and a half years and many traumas after it was first announced – it was estimated that the program contained 110,000 instructions and had taken eighty programmer-years to complete.

Microsoft’s Windows was heavily based on the Macintosh user interface – partly because it was a design on which it was very hard to improve, but also because it was felt the world would benefit from having a similar user interface in both Macintosh and Windows environments. On 22 November 1985, shortly after Windows was launched, Microsoft signed a licensing agreement with Apple to copy the visual characteristics of the Macintosh.

Although Windows was competitively priced at \$99, sales were sluggish at first because it was “unbearably slow.”¹⁸ This was true even on the most advanced PC then available, which used the new Intel 286 microprocessor. Although a million copies were eventually sold, most users found the system little more than a gimmick, and the vast majority of users stayed with the aging MS-DOS operating system. Thus, in 1986, the PC GUI appeared to be technologically unsupported on the existing generation of IBM-compatible computers. It would only be in the late 1980s, when the next generations of microprocessors – the Intel 386 and 486 – became available, that the GUI would become truly practical. Only two firms had the financial stamina to persist with a GUI-based operating system: Microsoft and IBM. In April 1987 IBM and Microsoft announced their intention to jointly develop a new operating system, OS/2, for the next generation of machines. This would be the long-term replacement for MS-DOS.

Microsoft, meanwhile, was still being carried along by the revenues from MS-DOS and was beginning to succeed in the applications market at last, with its Excel spreadsheet and Word. Gates decided that, notwithstanding OS/2, there would be short-term gains to be derived from relaunching Windows to take advantage of its own improving software technology and the faster PCs. In late 1987 Microsoft announced a new release of Windows, version 2.0. This was a major rewrite of the earlier version and had a user interface that was not merely like the Macintosh interface – as had been the case with Windows 1.0 – but was for all intents and purposes identical to it. It appeared that a convergence was taking place in which, by copying the look-and-feel of the Macintosh, the IBM-compatible PC would become indistinguishable from it and Apple’s unique marketing advantage would be lost.

Three months later, on 17 March 1988, Apple filed a lawsuit alleging that Microsoft’s Windows version 2 had infringed “the Company’s registered audiovisual copyrights protecting the Macintosh user interface.”¹⁹ (Microsoft’s original 1985 agreement with Apple had covered only version 1 of Windows, and Microsoft had not sought to renew the agreement for version 2.) The suit was of enormous importance for the future of the personal-computer industry. If it were

upheld, then it would have a severe negative effect on the great majority of users who did not use Macintosh computers and would be interpreted by developers as meaning that all user interfaces had to be different. If Apple were to succeed in the suit, it would be like a car manufacturer being able to copyright the layout of its instruments and every manufacturer having to ensure that its autos had a novel and different instrument layout. This would clearly be against the public interest.

The lawsuit would rattle on for three more years before it was finally dismissed. In the meantime, while the law ran its leisurely pace, Microsoft was achieving one of the most dramatic growth spurts of any business in the twentieth century, and Gates had become the world's youngest self-made billionaire. Some 2 million copies of Windows had been sold by early 1989, massively outselling IBM's OS/2 operating system, which had been launched in early 1988 (and in which Microsoft no longer expressed any interest). It was moot as to which of the systems was technically superior; the important factor was that Microsoft's marketing was much better.

Much of the software industry's rapid growth in the late 1980s, including that of Microsoft, was due to the ever-increasing sophistication of its products. Applications packages were routinely updated every eighteen months or so, generating additional revenues as users traded up for more sophisticated packages, making better use of rapidly improving hardware. Windows was no exception. Undeterred by the Apple-Microsoft lawsuit, by mid-1990 Windows was ready for yet another relaunch. It now consisted of 400,000 lines of code, nearly four times the size of the first version released in 1985. In total it was estimated to have cost \$100 million to develop.

On 22 May 1990 Windows 3.0 was launched around the world:

[S]ome 6,000 people were on hand as the City Center Theater in New York City became center stage for the multimedia extravaganza. Gala events took place in 7 other North American cities, which were linked via satellite to the New York stage for a live telecast, and in 12 major cities throughout the world, including London, Amsterdam, Stockholm, Paris, Madrid, Milan, Sydney, Singapore, and Mexico City. The production included videos, slides, laser lights, "surround sound," and a speech by Bill Gates, who proclaimed Windows 3 "a major milestone" in the history of software, saying that it "puts the 'personal' back into millions of MS-DOS-based computers."²⁰

Ten million dollars was spent on a blaze of publicity for Windows. By Gates's own account, it was "the most extravagant, extensive, and expensive software introduction ever."²¹

Microsoft's dominance of PC operating systems was reinforced by what economists termed "network effects." Software firms were motivated to create products for Windows-based PCs because it was the biggest market. In turn, users

were motivated to choose Windows-based PCs to take advantage of the growing array of software products. More users lead to more software products, and so on in a virtuous circle.

In a way that became fully apparent only in the mid-1990s following the launch of Windows 95, Microsoft had come to own the “platform” of personal computing. This would give Microsoft an industry prominence comparable with that of IBM in its heyday.

CD-ROMs and Encyclopedias

As well as software, users of personal computers wanted information – ranging from reference works such as encyclopedias and dictionaries to multimedia entertainment. By the late-1990s such information would be routinely obtained from the Internet, but even though consumer networks and online videotex services existed in the mid-1980s (discussed later in this chapter), the speed and capacity of these networks was simply too limited for supplying large volumes of information. The principal bottleneck was the *modem* – the device that connected a home computer to a network through a telephone line. To transmit an article – of, say, the length of a chapter in this book – would have taken thirty minutes; add a few images and that time would have been doubled. Until inexpensive high-speed communications arrived, the CD-ROM disc played a historic role in the way that large volumes of information were shipped to home computers.

The CD-ROM (Compact Disc-Read Only Memory) was the outcome of a joint research development by Sony and Philips in the early 1980s, and devices were first marketed in 1984. The CD-ROM was based on the technology of the audio CD and offered a storage capacity of over 500 megabytes – several hundred times that of a floppy disk. Although CD-ROM drives were expensive upon their introduction, costing \$1,000 or more for several years, their huge capacity gave them the potential to create an entirely new market for computer-enabled content that the embryonic computer networks would not be able to satisfy for another fifteen years.

As long as the price of a CD-ROM drive remained around \$1,000, its use was limited to corporations and libraries, mainly for business information and high-value publications. In the consumer arena, two of the industry leaders in personal-computer software, Gary Kildall and Bill Gates, played critical roles in establishing a market for CD-ROM media. Both Kildall and Gates hit on the idea of the CD-ROM encyclopedia as the means to establish the home market for CD-ROMs. Kildall explained: “Everyone knows encyclopedias usually cost about \$1,000. Someone can rationalize buying a computer that has the encyclopedia, if it’s in the same price range as the printed encyclopedia.”²²

While still running Digital Research, Kildall started a new CD-ROM publishing operation as a separate company. He secured the rights to the *Grolier*

Encyclopedia, a venerable junior encyclopedia established shortly after World War I. The *Grolier Encyclopedia* on CD-ROM was released in 1985, well ahead of any similar product. Priced at several hundred dollars, the Grolier CD-ROM can best be described as worthy but dull. It sold well to schools, but it did not induce consumers to invest in CD-ROM equipment.

Microsoft took much longer to get an encyclopedia onto the market. As early as 1985 Gates had tried to get the rights to the *Encyclopedia Britannica*, the leader in both content and brand. However, Encyclopedia Britannica Inc. was wary of cannibalizing its lucrative hard-copy sales, and in any case it already had a CD-ROM version of its lesser-known *Compton's Encyclopedia* under way. Next, Microsoft tried the *World Book Encyclopedia*, but it too had its own CD-ROM project in the pipeline.

Microsoft gradually worked its way down the pantheon of American encyclopedias, finally obtaining the rights to *Funk and Wagnall's New Encyclopedia* in 1989. *Funk and Wagnall's* had an unglamorous but profitable niche as a \$3-per-volume impulse purchase in supermarkets. Microsoft's decision to acquire the rights to *Funk and Wagnall's* was, in some quarters, derided as pandering to the educationally challenged working class, a criticism more than a little tinged by academic snobbery. But it was in truth very much a junior encyclopedia of negligible authority. Microsoft, however, proceeded to sculpt this unpromising material into *Encarta*, adding video and sound clips and other multimedia elements to the rather thin content. Released in early 1993 at a price of \$395, it sold only modestly.

In fact, the CD-ROM encyclopedia did not turn out to be the killer application that would start the CD-ROM revolution. Quite independently, the cost of CD-ROM drives had fallen slowly but continuously over a period of about seven years, to a price of about \$200 by 1992 – comparable to that of a printer or a hard disk drive. With the fall in price of CD-ROM drives, there was at last the possibility of a mass market for CD-ROM media. In 1992 the recreational software publisher Broderbund released *Grandma and Me* for \$20, which became an international bestseller and probably the best-known CD-ROM title of its time. Thousands of other titles from hundreds of publishers soon followed. At about the same time publishers of video games started to release CD-ROM titles with multimedia effects. The price of CD-ROM encyclopedias, so recently priced at several hundred dollars, fell into line. The price of *Encarta* was cut to \$99 for the 1993 Christmas season, and its competitors followed suit.

The availability of low-cost CD-ROM encyclopedias had a devastating effect on the market for traditional hard-bound encyclopedias. As *Forbes* magazine put it:

How long does it take a new computer technology to wreck a 200-year-old publishing company with sales of \$650 million and a brand name recognized all over the world? Not very long at all. There is no clearer, or sadder,

example of this than how the august Encyclopedia Britannica Inc. has been brought low by CD-ROM technology.²³

In 1996, under new ownership, a CD-ROM edition of the *Encyclopedia Britannica* finally hit the market for less than a hundred dollars. In 2012 the *New York Times* reported that the encyclopedia's current print edition would be its last – in “acknowledgement of the realities of the digital age.”²⁴

The Rise and Fall of Videotex

By the late 1970s networks that enabled desktop computers and workstations to access information services were beginning to proliferate. At one extreme was the hobbyist “Bulletin Board System” or simply “BBS.” A BBS, typically based on just a single PC, allowed users to post messages in the manner of a thumbtack bulletin board. According to *Boardwatch*, a magazine for BBS enthusiasts, by 1993 there were “at least 100,000 publicly accessible bulletin boards in operation worldwide.”²⁵ At the other extreme was USENET, begun in 1978, which was essentially a global bulletin board. USENET quickly established the idea of “newsgroups” so that like-minded users could post and share messages on particular topics. The network was popular in universities and research organizations as well as with individuals. By 1990, “it was one of the largest networks by any measure having about 265,000 users and 9,700 hosts on five continents.”²⁶

The great majority of networks in the 1980s arose spontaneously with little or no government involvement or subsidy. The most important exception was “videotex,” a networking technology promoted by several governments in the developed world – though not by the United States; if it had been, history might have unrolled differently.

Videotex evolved from a technology developed in the mid-1960s by RCA known as teletext, which provided a one-way transmission of text data to specially modified televisions. The British Broadcasting Corporation was an early adopter of teletext, and in the early 1970s the British Post Office moved to the forefront of research and development of an interactive videotex system – two-way communication using the telephone network. This system, initially called ViewData, was soon branded as Prestel. The Post Office had an early lead in videotex systems when pilot tests of Prestel were conducted in 1976. Yet by 1985 Prestel had secured only 62,000 subscribers, far short of the original goal of 2 million. Inflated expectations and disappointing fates similarly characterized other national videotex systems including Germany's Bildschirmtext, Japan's CAPTAIN, and Canada's Telidon. Development of international standards and a videotex global network were hampered by politics and national pride as well as by the fact that each country had developed its own display standards.

But there was one magnificent exception – France. The French system, Minitel, thrived owing to political, technological, and cultural factors coupled

with key early decisions by the French government. The 1973 Arab OPEC oil embargo, the United States' dominance in the computing industries, France's woeful telecommunications infrastructure (only 8 percent of French households possessed a telephone in 1970), and the growing recognition of the future importance of the services economy to developed countries all contributed to a sense of crisis in France in the early to mid-1970s. In 1974 the government's telecommunications agency – Direction Générale des Télécommunications – submitted a plan to add 14 million new phone lines in seven years to better position the country for growth in the increasingly services-oriented economic environment. By 1981, 74 percent of French households had telephones. And by 1989, that number had increased to 95 percent – a level roughly equal to or exceeding that in other major European nations and the United States.

British Telecom's Prestel and other videotex systems had lacked a compelling means of driving widespread public demand. To be sure, Prestel provided services such as weather information and transport timetables, but there were few consumer-oriented offerings such as online shopping, banking, entertainment, or chat rooms. Until the early 1980s Minitel grew slowly. However, France's eventual expansion of telephone service and infrastructure created a unique opportunity for the government to partially offset the rapidly rising costs of printing and distributing phone books, thereby providing the impetus for Minitel to take off. Between 1983 and 1991 the French government distributed more than 5 million Minitel units and developed an online telephone/address directory that people could use for free. The Minitel terminals, with their monochrome 9-inch screen and small keypads, cost approximately 500 francs (less than \$100) to manufacture. The massive free distribution of these basic terminals (in place of white-page directories) led to network effects: more Minitel users generated the need for more service providers, more service providers attracted more users, and so on. The provision of new services, in turn, accelerated the demand for upscale Minitel units with larger screens and color monitors, which were sold or leased by France Télécom.

A further contribution to Minitel's financial success was the permissiveness of France Télécom and the French government regarding the types of services offered. One example from early in Minitel's history was the proliferation of "messageries roses," or sex-chat services. In 1987, "calls" to such sites "totaled 2 million hours a month, or close to half of all [Minitel] consumer calls at the time."²⁷ This paralleled an Internet phenomenon of the 1990s, when *Penthouse* and *Playboy* were among the "top ten" most popular sites: late in the decade, these sites – along with "PornCity" – were receiving millions of visitors daily.²⁸ As with the Internet, when more services and content became available on Minitel – from news and sports to travel, weather, and business – the proportion of sexually oriented material offered and usage declined. By 1992 Minitel had more than 20,112 services and the number of terminals distributed had reached nearly 6.3 million. In addition to home units, Minitel terminals were available in post

offices and other public locations. Given Minitel's ease of use, the system had many unlikely IT adopters. For example, dairy farmers in northwestern France's Brittany region relied on Minitel to "call the inseminator when a cow [was] in heat or to request [that] the authorities come to haul away animal carcasses,"²⁹ to track market prices, and to relay the results of chemical tests on their dairy products.

Minitel was finally decommissioned in mid-2012. It had been an extraordinary development that, for more than twenty years, gave almost everyone in France facilities later associated with the Internet. Inevitably, the existence of Minitel inhibited France's mass adoption of the Internet until the early 2000s, but the country soon caught up.

Consumer Networks in the United States

In the United States neither the government nor videotex played a significant role in the early development of consumer networks. The leading consumer network throughout the 1980s was CompuServe. The network had originally been set up in 1969, as an old-line time-sharing service of the type described in Chapter 9. Unfortunately, CompuServe was established just as the fad for computer utilities was on the wane. Only after two years of heavy losses did it begin to turn a modest profit, selling time-sharing services to the insurance industry.

Rather like electric utilities, business-oriented time-sharing services suffered from wildly fluctuating demand. During nine-to-five office hours, the service was used to capacity, but in the evening when the business day was over, and on weekends, the system was heavily underused. In 1978, with the arrival of personal computers, CompuServe's founder and leader Jeff Wilkins saw an opportunity to sell this spare capacity to computer hobbyists. Working with the Midwest Association of Computer Clubs – which distributed the access software to its members – he created a service called MicroNet. Subscribers could access the system at off-peak times for a few dollars an hour. MicroNet proved very popular and Wilkins decided to extend the service nationally. However, he first had to get the access software into the hands of consumers nationwide. To do this he made an arrangement with the Tandy Corporation to distribute a \$39.95 "starter kit" in its eight thousand Radio Shack stores. The kit included a user manual, access software, and \$25 worth of free time.

Unlike CompuServe's corporate customers, who mainly wanted financial analysis programs and insurance services, home-computer users were much more interested in "content" and in communicating with other subscribers. Newspapers were signed up – starting with the *Columbus Dispatch* in 1980, but eventually including major newspapers such as the *New York Times*. CompuServe services included computer games, electronic mail, and a precursor to chat rooms that allowed users a forum through which they could communicate in real time. By summer 1984, CompuServe had 130,000 subscribers in three

hundred cities, served by twenty-six mainframe computers and a staff of six hundred in its Columbus headquarters. For the basic monthly fee of \$12/hour (\$6 on nights and weekends), CompuServe subscribers could book an airline reservation, read newspapers, check a stock portfolio, browse a data bank or encyclopedia, and participate with bulletin boards by posting messages. It was also possible to use CompuServe for its original purpose: timesharing on a large computer.

CompuServe was the most successful early consumer network, and for several years the number of subscribers exceeded those of all the other services put together. As PCs moved into the workplace, CompuServe also tapped the business market, offering "Executive Services" – primarily access to conventional databases such as Standard & Poor's company information service and Lockheed's Dialog information database. Users paid a premium for these services, which were shared between CompuServe and the information provider.

Although only about one in twenty computer owners used online services in the mid-1980s, such services were widely perceived as an opportunity with tremendous potential, and CompuServe began to attract some major competitors. The two most important were The Source and Prodigy. The Source was a network set up by the Reader's Digest Association and the Control Data Corporation, while Prodigy was created by Sears and IBM. In each case, one partner saw the network as an extension to its existing business while the other provided the computing know-how. Thus Reader's Digest Association saw consumer networks as a new form of publishing (which in time it became, though not until long after the Association had quit the business), and Sears saw the network as an extension of its famous mail-order catalog. Both organizations also had excellent distribution channels through which they could get access software into the hands of their subscribers. But success was elusive, and it proved very difficult to compete with CompuServe. The Source was sold to CompuServe in 1989, while Prodigy incurred huge losses before its owners abandoned it in 1996.

CompuServe's relative success benefited from network effects. At a time when users of one network could not communicate with those of another, it made sense to belong to the biggest network. By 1990, CompuServe had 600,000 subscribers and it offered literally thousands of services: from home banking to hotel reservations, from Roger Ebert's movie reviews to *Golf Magazine*. In 1987 CompuServe had extended into Japan (with a service called NiftyServe) and in the early 1990s it began a service in Europe.

CompuServe did, however, prove vulnerable to one newcomer in the mid-1980s: America On-Line (AOL). AOL's success was fairly modest until it developed a service for the Macintosh computer. Rather in the way the Macintosh carved a successful niche with its graphical user interface, the AOL service for the Macintosh benefited from developing a distinctive friendly interface. In 1991 the service was extended to the IBM-compatible PC, providing "an easy-to-use

point-and-click interface anyone's grandmother could install."³⁰ As always with consumer networks, a major barrier was getting access software into the hands of consumers and inducing them to try out the service. AOL hit on the idea of inserting a floppy disc with access software in computer magazines, together with a free trial period. AOL became famous for "carpet-bombing"³¹ America with floppy discs and, later, CD-ROMs. By making the service free to try out, it encouraged hundreds of thousands of consumers to do just that. And because the service was user-friendly, many of them stayed on as paying customers. True, an even greater number did not stay, creating a phenomenon known as "churn" – whereby as many subscribers were said to be leaving as joining. Nonetheless, by the end of 1995 AOL claimed that it had 4.5 million subscribers (including many in Europe where, surprisingly, AOL did not feel the need to make its name less imperialistic).

THE INTERNET WAS to become overwhelmingly the most important development in computing from the mid-1990s onward. But this was possible only because in the previous decade the personal computer had been transformed from an unfriendly technological system into something approaching an information appliance. Software companies provided application programs that transformed the general-purpose personal computer into a special-purpose machine that met the needs of particular users – whether they were business analysts using spreadsheets, individuals word-processing documents, or players of games. While many of these applications simplified using a computer – for example, by making a word-processor program appear like a familiar typewriter – it was the graphical user interface that opened up the computer to the wider society. Instead of typing complicated commands, users could now simply point and click their way through the most complex software, while knowing essentially nothing about the inner workings of a computer.

Although the computer of the mid-1980s had become easier to use, it was an isolated machine. It was the modern replacement for the typewriter or electronic calculator, and even a popular new item in the games room – but it was no substitute for a library or a telephone. To become truly useful, the personal computer needed access to large quantities of information comparable with that of a modest library, and it needed the ability to communicate with other computers. CD-ROM technology, consumer networks, and videotex systems played complementary roles in this transformation. CD-ROM technology provided information in high volume, but it was information that was static and rapidly became obsolete. Consumer networks and videotex systems held large volumes of information, but users could obtain it only in tiny quantities as it trickled through the bottleneck of a modem. In the early years of the twenty-first century, the rise of the Internet and high-speed broadband connections would at last provide users with the information they needed at an acceptable speed.

Notes to Chapter 11

The fullest account of the history of the personal-computer software industry appears in Martin Campbell-Kelly's *From Airline Reservations to Sonic the Hedgehog* (2003). Although the numerous histories of Microsoft (of which several were listed in the notes to Chapter 10) take a Microsoft-centric view of the software universe, they also consider some of its competitors.

The history of the graphical user interface is authoritatively covered in *A History of Personal Workstations*, edited by Adele Goldberg (1988). The inside story of Xerox PARC is told in Michael Hiltzik's *Dealers of Lightning: Xerox PARC and the Dawn of the Computer Age* (1999) and in Douglas Smith and Robert Alexander's earlier *Fumbling the Future: How Xerox Invented, Then Ignored, the First Personal Computer* (1988). Thierry Bardini's *Bootstrapping: Douglas Engelbart, Coevolution, and the Origins of Personal Computing* (2000) does justice to a once-unsung hero of the personal-computer revolution.

Books focusing on the Macintosh development include John Sculley's insider account *Odyssey: Pepsi to Apple* (1987) and Steven Levy's external view *Insanely Great* (1994). A selection of business histories of Microsoft (all of which discuss the Windows operating system) was listed in the notes to Chapter 10. We found the most thoughtful account of Microsoft's forays into CD-ROM publishing and consumer networks to be Randall Stross's *The Microsoft Way* (1996).

The history of consumer networks has largely been sidelined by the rise of the Internet. The leading book on Minitel is Valérie Schafer and Benjamin G. Thierry's *Le Minitel: L'enfance numérique de la France*, and a particularly useful article is Amy L. Fletcher's "France Enters the Information Age: A Political History of Minitel" (2002). The best account of AOL is Kara Swisher's *Aol.com* (1999), which examines the company's earlier history as well as its meteoric growth in the 1990s. There are no substantial histories of early players CompuServe, The Source, Prodigy, or Genie, although they appear fleetingly in Swisher's *Aol.com*. These and other developments in the pre-Internet era appear in William F. Aspray and James W. Cortada's *From Urban Legends to Political Fact-Checking* (2019).

Notes

1. Quoted in Campbell-Kelly 1995, p. 103.
2. Quoted in Sigel 1984, p. 126.
3. See Petre 1985.
4. Quoted in Goldberg 1988, pp. 195–6.
5. *Ibid.*, 294–5.
6. *Ibid.*
7. *Ibid.*

8. Ibid.
9. Smith and Alexander 1988, p. 241.
10. See interview with Raskin in Lammers 1986, pp. 227–45.
11. Sculley 1987, p. 157.
12. Wallace and Erickson 1992, p. 267.
13. Sculley 1987, p. 248.
14. See Markoff 1984.
15. Slater 1987, p. 260.
16. Carroll 1994, p. 86.
17. Quoted in Wallace and Erickson 1992, p. 252.
18. Ichbiah and Knepper 1991, p. 189.
19. This form of words appears in Apple Computer's annual reports.
20. Ichbiah and Knepper 1991, p. 239.
21. Ibid.
22. Lammers 1986, p. 69.
23. Quoted in Campbell-Kelly 2003, p. 294.
24. Bosman, Julie. 2012. "After 244 Years, Encyclopaedia Britannica Stops the Presses." *New York Times*, 13 March, <https://archive.nytimes.com/mediadecoder.blogs.nytimes.com/2012/03/13/after-244-years-encyclopaedia-britannica-stops-the-presses/>
25. Aboba 1993, p. 59.
26. Quarterman 1990, p. 235.
27. OECD Directorate for Science, Technology and Industry, Committee for Information, Computer and Communications Policy 1998, p. 14.
28. Coopersmith 2000, p. 31.
29. Sayare, Scott. 2012. "On the Farms of France, the Death of a Pixelated Workhorse." *New York Times*, 28 June, p. A8.
30. Swisher 1998, p. 66.
31. Ibid., p. 99.

12 The Internet

IN THE EARLY 1990s the Internet became big news. In the fall of 1990 there were just 313,000 computers attached to the Internet; five years later the number was approaching 10 million, and by the end of 2000 the number had exceeded 100 million. Although computer technology is at the heart of the Internet, its importance is economic and social: the Internet gives computer users the ability to communicate, to gain access to information sources, to express themselves, and to conduct business.

The Internet sprang from a confluence of three desires, two that emerged in the 1960s and one that originated much further back in time. First, there was the rather utilitarian desire for an efficient, fault-tolerant networking technology, suitable for military communications, that would never break down. Second, there was a wish to unite the world's computer networks into a single system. Just as the telephone would never have become the dominant person-to-person communications medium if users had been restricted to the network of their particular provider, so the world's isolated computer networks would be far more useful if they were joined together. But the most romantic ideal – perhaps dating as far back as the Library of Alexandria in the ancient world – was to make readily available the world's store of knowledge.

From the Encyclopedia to the Memex

The idea of making the world's store of knowledge available to the ordinary person is a very old dream. It was the idea, for example, that drove the French philosopher Denis Diderot to create the first great encyclopedia in the eighteenth century. The multivolume *Encyclopédie* was one of the central projects of the Age of Enlightenment, which tried to bring about radical and revolutionary reform by giving knowledge and therefore power to the people. The *Encyclopédie* was in part a political act, which it was hoped would promote rationalism and democracy; similarly, the Internet has a political dimension. The first English-language encyclopedia, the *Encyclopedia Britannica*, appeared in 1768 and was modeled directly on the *Encyclopédie*. Of course, neither the

Encyclopédie nor the *Encyclopedia Britannica* could contain *all* of the world's knowledge. But they both contained a significant fraction of it and – at least as important – they brought order to the universe of knowledge, giving people a sense of what there was to know.

The nineteenth century saw an explosion in the production of human knowledge. In the early decades of the century, it was possible for a person of learning to be comfortable with the whole spectrum of human knowledge, in both the arts and the sciences. For example, Peter Mark Roget – now famous only as the compiler of the thesaurus – earned his living as a physician, but he was also an amateur scientist and a member of the Royal Society of London, an educationalist, and a founder of the University of London. And Charles Babbage, besides being famous for his calculating engines, was an important economist; he also wrote works on mathematics and statistics, geology and natural history, and even theology and politics.

By the twentieth century, however, the enormous increase in the world's knowledge had brought about an age of specialization during which it was very unusual for a person to be equally versed in the arts and the sciences, and virtually impossible for anyone to have a deep knowledge of more than a very narrow field of learning. It was said, for example, that by 1900 no mathematician could be familiar even with all the different subdisciplines of mathematics.

In the years between the two world wars, a number of leading thinkers began to wonder whether it might be possible to arrest this trend toward specialization by organizing the world's knowledge systematically so that, at the very least, people could once again know what there was to know. The most prominent member of this movement was the British socialist, novelist, and science writer H. G. Wells, best known in the United States as the author of *The War of the Worlds* and *The Time Machine*. During his own lifetime, Wells had seen the world's store of knowledge double and double again. He was convinced that narrow specialization – such that even educated people were familiar with no more than a tiny fraction of the world's knowledge – was causing the world to descend into barbarism in which people of learning were being “pushed aside by men like Hitler.”¹ During the 1930s he wrote pamphlets and articles and gave speeches about his project for a World Encyclopedia that would do for the twentieth century what Diderot had done for the eighteenth. Wells failed to interest publishers owing to the enormous cost of such a project and, in the fall of 1937, embarked on a US lecture tour hoping to raise funds.

He covered five cities, and his last venue, in New York, was also broadcast on the radio. In his talk, titled “The Brain Organization of the Modern World,” Wells explained that his World Encyclopedia would not be an encyclopedia in the ordinary sense:

A World Encyclopedia no longer presents itself to a modern imagination as a row of volumes printed and published once for all, but as a sort of mental clearing house for the mind, a depot where knowledge and ideas are received,

sorted, summarized, digested, clarified and compared. . . . This Encyclopedic organization need not be concentrated now in one place; it might have the form of a network [that] would constitute the material beginning of a real World Brain.²

Wells never explained how his “network” for the World Brain would be achieved, beyond supposing that it would be possible to physically store all the data on microfilm. All the information in the world would do no good, however, if it were not properly organized. He thus envisaged that “[a] great number of workers would be engaged perpetually in perfecting this index of human knowledge and keeping it up to date.”³

During his tour, Wells was invited to lunch as the distinguished guest of President Roosevelt.⁴ Wells lost no time in trying to interest his host in the World Brain project, but, perhaps not surprisingly, Roosevelt had more pressing problems. Wells left the lunch a disappointed man. Time was running out for the World Brain; as the world drifted into World War II, he was forced to abandon the project and fell into a depression from which he never emerged. He lived to see the end of the war but died the following year, at the age of eighty.

Wells’s dream did not die with him. In the postwar period the idea resurfaced and was given new vigor by Vannevar Bush, the scientist and inventor who had developed analog computers and risen to become chief scientific adviser to the president and head of the Office of Scientific Research and Development, where he directed much of the United States’ scientific war effort.

In fact, Bush had first proposed an information storage machine several years before the war. This was to be a desk-like device that could hold the contents of a university library “in a couple of cubic feet.”⁵ With the start of the war Bush had to put these ideas to one side, but in its final months he turned his mind to what scientists might do in the postwar era. For him, one problem stood out above all others: coping with the information explosion. In July 1945 he set his ideas down in a popular article, “As We May Think,” for the *Atlantic Monthly*. A few weeks later, when it was reprinted in *Life* magazine, it reached a wider readership. This article articulated a dream that had been pursued by information scientists for decades.

During the war, as Wells had predicted, microfilm technology advanced to a point where the problem of storing information was essentially solved, although the problem of retrieving that information remained. Through the use of microfilm technology, Bush noted, it would be possible to contain the *Encyclopedia Britannica* in a space the size of a matchbox, and “a library of a million volumes could be compressed into one end of a desk.”⁶ Thus the problem was not so much *containing* the information explosion as being able to make use of it. Bush envisaged a personal-information machine that he called the memex:

A memex is a device in which an individual stores all his books, records, and communications, and which is mechanized so that it may be consulted with

exceeding speed and flexibility. It is an enlarged intimate supplement to his memory.

It consists of a desk, and while it can presumably be operated from a distance, it is primarily the piece of furniture at which he works. On the top are slanting translucent screens, on which material can be projected for convenient reading. There is a keyboard, and sets of buttons and levers. Otherwise it looks like an ordinary desk.

The memex would allow the user to browse through information:

If the user wishes to consult a certain book, he taps its code on the keyboard, and the title page of the book promptly appears before him, projected onto one of his viewing positions. Frequently used codes are mnemonic, so that he seldom consults his code book; but when he does, a single tap of a key projects it for his use. Moreover, he has supplemental levers. On deflecting one of these levers to the right he runs through the book before him, each page in turn being projected at a speed which just allows a recognizing glance at each. If he deflects it further to the right, he steps through the book 10 pages at a time; still further at 100 pages at a time. Deflection to the left gives him the same control backwards.

A special button transfers him immediately to the first page of the index. Any given book of his library can thus be called up and consulted with far greater facility than if it were taken from a shelf. As he has several projection positions, he can leave one item in position while he calls up another. He can add marginal notes and comments . . . just as though he had the physical page before him.

Bush did not quite know how, but he was sure that the new technology of computers would be instrumental in realizing the memex. He was one of very few people in 1945 who realized that computers would one day be used for something more than rapid arithmetic.

In the twenty years that followed Bush's article, there were a number of attempts at building a memex using microfilm readers and simple electronic controls, but the technology was too crude and expensive to make much headway. When Bush revisited the memex idea in 1967 in his book *Science Is Not Enough*, his dream of a personal-information machine was still "in the future, but not so far."⁷

At the time he wrote those words, the intellectual problems involved in constructing a memex-type information system using computer technology had, in principle, been largely solved. J.C.R. Licklider, the head of ARPA's Information Processing Techniques Office, for one, was working as early as 1962 on a project he called the Libraries of the Future, and he dedicated the book he published with that title "however unworthy it may be, to Dr. Bush."⁸ In the mid-1960s, Ted Nelson coined the term *hypertext* and Douglas Engelbart was working on

the practical realization of similar ideas at the Stanford Research Institute. Both Nelson and Engelbart claim to have been directly influenced by Bush. Engelbart later recalled that, as a lowly electronics technician in the Philippines during World War II, he “found this article in *Life* magazine about his [Bush’s] memex, and it just thrilled the hell out of me that people were thinking about something like that . . . I wish I could have met him, but by the time I caught onto the work, he was already in a nursing home and wasn’t available.”⁹

Although the intellectual problems of designing a memex-type machine were quickly solved, establishing the physical technology – a network of computers – would take a long time.

The Arpanet

The first concrete proposal for establishing a geographically distributed network of computers was made by Licklider in his 1960 “Man-Computer Symbiosis” paper:

It seems reasonable to envision, for a time 10 or 15 years hence, a “thinking center” that will incorporate the functions of present-day libraries together with anticipated advances in information storage and retrieval . . . The picture readily enlarges itself into a network of such centers, connected to one another by wide band communication lines and to individual users by leased-wire services. In such a system, the speed of the computers would be balanced, and the cost of the gigantic memories and the sophisticated programs would be divided by the number of users.¹⁰

In 1963, when the first ARPA-sponsored time-shared computer systems became operational, Licklider set into action a program he privately called his Intergalactic Computer Network, publicly known as the Arpanet.

The stated motive for Arpanet was an economic one. By networking ARPA’s computer systems together, the users of each computer would be able to use the facilities of any other computer on the network; specialized facilities would thus be available to all, and it would be possible to spread the computing load over many geographically separated sites. For example, because the working day for East Coast computer users started several hours ahead of the West Coast’s, they could make use of idle West Coast facilities in the morning; and when it was evening on the East Coast, their positions would be reversed. Arpanet would behave rather like a power grid in which lots of power plants worked in harmony to balance the load.

In July 1964, when Licklider finished his two-year tenure as head of the Information Processing Techniques Office (IPTO), he had a powerful say in the appointment of his successors, who shared his vision and carried it forward. His immediate successor was Ivan Sutherland, the MIT-trained graphics expert at the University of Utah, who held the office for two years. He was followed

by Robert Taylor, another MIT alumnus, who later became head of the Xerox PARC computer science program.

Between 1963 and 1966 ARPA funded a number of small-scale research projects to explore the emerging technology of computer networking. ARPA was not the only organization sponsoring such activity; there were several other groups interested in computer networking in the United States, England (especially at the National Physical Laboratory), and France. By 1966 the computer networking idea was ready for practical exploitation, and IPTO's head, Robert Taylor, decided to develop a simple experimental network. He invited a rising star in the networking community, Larry Roberts, to head up the project.

Larry Roberts had received his PhD from MIT in 1963. He was first turned on to the subject of networking by a discussion with Licklider at a conference in November 1964, and, as he later put it, "his enthusiasm infected me."¹¹ When he was approached by Taylor to head IPTO's networking program, Roberts was working on an IPTO-sponsored networking project at MIT's Lincoln Laboratory. Roberts initially did not want to trade his role as a Lincoln Lab scientist for this important bureaucratic task, but Taylor went over his head and reminded Roberts' boss that the lab received more than half its funding from ARPA. When he took over the Arpanet project in 1966, Roberts had to find solutions to three major technical problems before work could begin on a practical network. The first problem was how to physically connect all the time-sharing systems together. The difficulty here was that, if every computer system had to be connected to every other, the number of communications lines would grow geometrically. Networking the seventeen ARPA computers that were then in existence would require a total of 136 (that is, $17 \times 16 / 2$) communications lines. The second problem was how to make economic use of the expensive high-speed communications lines connecting computers. Experience with commercial time-sharing computers had already shown that less than 2 percent of the communications capacity of a telephone line was productively used because most of a user's time was spent thinking, during which the line was idle. This drawback did not matter very much where local phone lines were concerned, but it would be insupportable on high-speed long-distance lines. The third problem Roberts faced was how to link together all the computer systems, which came from different manufacturers and used many varieties of operating software that had taken several years to develop. Enough was known about the software crisis at this stage to want to avoid the extensive rewriting of operating systems.

Unknown to Roberts, a solution to the first two problems had already been invented. Known as "store-and-forward packet switching," the idea was first put forward by Paul Baran of the RAND Corporation in 1961 and was independently reinvented in 1965 at the National Physical Laboratory in England by Donald Davies, who coined the term *packet switching*.¹² Davies recognized the packet-switching concept to be similar to an older telegraph technology.

In telegraph networks, engineers had already solved the problem of how to avoid having every city connected to every other. Connectivity was achieved by using a number of switching centers located in major cities. Thus if a telegram was sent from, say, New York to San Francisco, the message might pass through intermediate switching centers in Chicago and Los Angeles before arriving in San Francisco. In the early days of the telegraph, during the late nineteenth century, at each switching center an incoming telegram would be received on a Morse sounder and written out by a telegraph clerk. It would then be retransmitted by a second telegraph clerk to the next switching center. This process would be repeated at each switching center, as the telegram was relayed across the country to its final destination. An incidental advantage of creating a written copy of the telegram was that it could act as a storage system, so that if there was a buildup of traffic, or if the onward switching center was too busy to receive the message, it could be held back until the lines became quieter. This was known as the store-and-forward principle. In the 1930s these manual switching centers were mechanized in “torn-tape offices,” where incoming messages were automatically recorded on perforated paper tape and then retransmitted mechanically. In the 1960s the same functions were being computerized using disk stores instead of paper tape as the storage medium.

Store-and-forward packet switching was a simple elaboration of these old telegraph ideas. Instead of having every computer connected to every other, store-and-forward technology would be used to route messages through the network; there would be a single “backbone” communications line that connected the computers together, with other connections being added as the need arose. Packet-switching technology addressed the problem of making economic use of the high-speed communications lines. So that a single user did not monopolize a line, data would be shuttled around the network in packets. A packet was rather like a short telegram, with each packet having the address of the destination. A long message would be broken up into a stream of packets, which would be sent as individual items into the network. This would enable a single communications line to carry many simultaneous human-computer interactions by transmitting packets in quick succession. The computers that acted as the switching centers – called *nodes* in the Arpanet – would simply receive packets and pass them along to the next node on the route toward the destination. The computer at the destination would be responsible for reconstituting the original message from the packets. In effect, by enabling many users to share a communications line simultaneously, packet switching did for telecommunications what time-sharing had done for computing.

All of this was unknown to Roberts until he attended an international meeting of computer network researchers in Gatlinburg, Tennessee, in October 1967. There he learned of the packet-switching concept from one of Donald Davies’s English colleagues. He later described this as a kind of revelation: “Suddenly I learned how to route packets.”¹³

The final problem that remained for Roberts was how to avoid the horrendous software problems of getting the different computers to handle the network traffic. Fortunately, just as Roberts was confronting this problem, the first minicomputers had started to enter the market and the solution came to him in a eureka moment in a taxicab ride: the interface message processor (IMP). Instead of the existing software having to be modified at each computer center, a separate inexpensive minicomputer, the IMP, would be provided at every node to handle all the data communications traffic. The software on each computer system – called a host – would need only a relatively simple modification to collect and deliver information between itself and the IMP. Thus there was only one piece of software to worry about – a single software system to be used in all the IMPs in the network.

The Arpanet project – little more than a gleam in IPTO's eye when Roberts took it over in 1966 – had become concrete. In the summer of 1968 he succeeded Taylor as director of IPTO, and the Arpanet project went full steam ahead. Work began on a \$2.5 million pilot scheme to network together four computer centers – at the University of California at Los Angeles, the University of California at Santa Barbara, the University of Utah, and the Stanford Research Institute. The software for the IMPs was contracted out to Bolt, Beranek and Newman (BBN), while on the university campuses a motley collection of graduate students and computer center programmers worked on connecting up their host computers to the IMPs. The participation of so many graduate students in developing Arpanet, typically working on a part-time basis as they completed their computer science graduate studies, created a distinctive and somewhat anarchic culture in the network community. This culture was as strong in its way as the computer-amateur culture of the personal-computer industry in the 1970s. But unlike the personal-computer world, this culture was much more persistent, and it accounted for the unstructured and anarchic state of the early Internet.

By 1970 the four-node network was fully operational and working reliably. The other ARPA-funded computer centers were soon joined up to the network, so that by the spring of 1971 there were twenty-three hosts networked together. Although impressive as a technical accomplishment, Arpanet was of little significance to the tens of thousands of ordinary mainframe computers in the world beyond. If the networking idea was ever to move beyond the ARPA community, Roberts realized that he had to become not merely a project manager but an evangelist. He proselytized Arpanet to the technical community at academic conferences, but found he was preaching to the converted. He therefore decided to organize a public demonstration of the Arpanet, one that simply no one could ignore, at the first International Conference on Computer Communications (ICCC) in Washington in the fall of 1972.

The conference was attended by more than a thousand delegates, and from the Arpanet demonstration area, equipped with forty terminals, people were able to directly use any one of more than a dozen computers – ranging from Project

MAC at MIT to the University of California computer center. A hookup was even arranged with a computer in Paris. Users were able to undertake serious computing tasks such as accessing databases, running meteorological models, and exploring interactive graphics; or they could simply amuse themselves with an air-traffic simulator or a chess game. The demonstration lasted three days and left an indelible impression on those who attended.

The ICCC demonstration was a turning point both for the Arpanet and for networking generally. The technology had suddenly become real, and many more research organizations and universities clamored to become connected to Arpanet. A year after the conference, there were 45 hosts on the network; four years later there were 111. It was an improvement, but growth was still slow.

The Popularity of E-Mail

It was not, however, the economics of resource sharing, the ability to use remote computers, or even the pleasure of playing computer games that caused the explosion of interest in networking; indeed, most users never made use of any of these facilities. Instead, it was the opportunity for communicating through electronic mail that attracted users.

Electronic mail had never been an important motivation for Arpanet. In fact, no electronic mail facilities were provided initially, even though the idea was by no means unknown. An e-mail system had been provided on MIT's Project MAC in the mid-1960s, for example; but because users could only mail other MIT colleagues, it was never much more than an alternative to the ordinary campus mail.

In July 1971 two BBN programmers developed an experimental mail system for Arpanet. According to one member of the BBN network team:

When the mail [program] was being developed, nobody thought at the beginning it was going to be the smash hit that it was. People liked it, they thought it was nice, but nobody imagined that it was going to be the explosion of excitement and interest that it became.¹⁴

Electronic mail soon exceeded all other forms of network traffic on Arpanet, and by 1975 there were over a thousand registered e-mail users. Networked computing and e-mail were becoming an integral part of the modern way of doing business, and the existing computer services industry was forced to respond. First, the time-sharing firms began to restructure themselves as network providers. BBN's Telcomp time-sharing service, for example, relaunched itself as the Telnet network in 1975 (with Larry Roberts as its CEO). Telnet initially established nodes in seven cities, and by 1978 there were nodes in 176 US cities and 14 overseas countries. In 1979 Telcomp's rival Tymshare created a network subsidiary, Tymnet. Established communications firms such as Western Union and

MCI were also providing e-mail services, and the business press was beginning to talk of “The Great Electronic Mail Shootout.”¹⁵ In the public sector, government organizations such as NSF and NASA also developed networks, while in the education sector consortia of colleges developed networks such as Merit, Edunet, and Bitnet, all of which became operational in the first half of the 1980s.

Electronic mail was the driving force behind all these networks. It became popular because it had so many advantages over conventional long-distance communications, and these advantages were consolidated as more and more networks were established: the more people there were on the networks, the more useful e-mail became. Taking just a few minutes to cross the continent, e-mail was much faster than the postal service, soon derisively called “snail mail.” Besides being cheaper than a long-distance phone call, e-mail eliminated the need for both parties having to synchronize their activities, freeing them from their desks. E-Mail also eliminated some of the problems associated with different time zones. It was not just e-mail users who liked the new way of doing business; managers, too, had become enthusiastic about its economic benefits, especially for coordinating group activity – it could reduce the number of meetings.

Electronic mail was a completely new communications medium, however, and, as such, it brought with it a range of social issues that fascinated organizational psychologists and social scientists. For example, the speed of communications encouraged kneejerk rather than considered responses, thus increasing rather than decreasing the number of exchanges. Another problem was that the terse “telegraphese” of e-mail exchanges could easily cause offense to the uninitiated, whereas the same message conveyed over the phone would have been softened by the tone of voice and a more leisurely delivery. Gradually an unwritten “netiquette” emerged, enabling more civilized interactions to take place without losing too many of the benefits of e-mail. While some of the new computer networks of the 1970s were based on the technology developed in Arpanet, this was not true of all of them. Computer manufacturers, in particular, developed their own systems such as IBM’s Systems Network Architecture (SNA) and Digital Equipment Corporation’s DECNET. By reworking a very old marketing strategy, the manufacturers were hoping to keep their customers locked into proprietary networks for which they would supply all the hardware and software. But this was a shortsighted and mistaken strategy, because the real benefits of networking came through *internetworking* – in which the separate networks were connected and literally every computer user could talk to every other.

Fortunately, IPTO had been aware of this problem as early as 1973 – not least so that electronic mail would be able to cross network boundaries. Very quickly, what was simply an idea, internetworking, was made concrete as the Internet. The job of IPTO at this stage was to establish “protocols” by which the networks could communicate. (A network protocol is simply the ritual electronic

exchange that enables one network to talk to another – a kind of electronic Esperanto.) The system that ARPA devised was known as the Transmission Control Protocol/Internet Protocol, or TCP/IP – a mysterious acronym familiar to most experienced users of the Internet. Although the international communications committees were trying to evolve internetworking standards at the same time, TCP/IP quickly established itself as the de facto standard (which it remains).

Yet it would be a full decade before significant numbers of networks were connected. In 1980 there were fewer than two hundred hosts on the Internet, and as late as 1984 there were still only a thousand. For the most part these networks served research organizations and science and engineering departments in universities – a predominantly technical community, using conventional time-sharing computers. The Internet would become an important economic and social phenomenon only when it also reached the broad community of ordinary personal-computer users.

The World Wide Web

This broad community of users began to exert its influence in the late 1980s. In parallel with the development of the Internet, the personal computer was spreading across the educational and business communities and finding its way into American homes. By the late 1980s most professional computer users (though not home users) in the United States had access to the Internet, and the number of computers on the Internet began an explosive growth.

Once this much broader community of users started to use the Internet, people began to exchange not just e-mail but whole documents. In effect, a new electronic publishing medium had been created. Like e-mail and news groups, this activity was largely unforeseen and unplanned. As a result, there was no way to stop anyone from publishing anything and placing it on the net; soon there were millions of documents but no catalog and no way of finding what was useful. Sifting the grains of gold amid the tons of trash became such a frustrating activity that only the most dedicated computer users had the patience to attempt it. Bringing order to this universe of information became a major research issue.

Numerous parallel developments were under way to create finding aids for information on the Internet. One of the first systems, “archie,” was developed at McGill University in 1990. By combing through the Internet, archie created a directory of all the files available for downloading so that a user wanting a file did not need to know on what machine it was actually located. A more impressive system was the Wide Area Information Service (WAIS), developed the following year by the Thinking Machines Corporation of Waltham, Massachusetts. WAIS enabled users to specify documents by using keywords (say, *smallpox* and *vaccine*), at which point it would display all the available documents on the Internet that matched those criteria. The most popular early finding aid was “gopher,” developed at the University of Minnesota. (A highly appropriate name, gopher is

slang for one who runs errands but also a reference to the university's mascot.) The system, which became operational in 1991, was effectively a catalog of catalogs that enabled a user to drill down and examine the contents of gopher databases maintained by hundreds of different institutions.

All of these systems treated documents as individual entities, rather like books in a library. For example, when a system located a document about smallpox vaccine, say, it might tell the user that its inventor was Edward Jenner; in order to discover more about Jenner, the user would need to search again. The inventors of hypertext – Vannevar Bush in the 1940s and Engelbart and Nelson in the 1960s – had envisaged a system that would enable one to informally skip from document to document. At the press of button, as it were, one could leap from smallpox to Jenner to The Chantry in Gloucestershire, England (the house where Jenner lived and now a museum to his memory). Hypertext was, in fact, a lively computer research topic throughout the 1980s, but what made it so potent for the Internet – ultimately giving rise to the World Wide Web – was that it would make it unnecessary to locate documents in centralized directories. Instead, links would be stored in the documents themselves, and they would instantly whisk the reader to related documents. It was all very much as Vannevar Bush had envisioned the memex.

The World Wide Web was invented by Tim Berners-Lee. Its origins dated back to Berners-Lee's early interest in hypertext in 1980, long before the Internet was widely known. Berners-Lee was born in London in 1955, the son of two mathematicians (who were themselves pioneers of early British computer programming). After graduating in physics from Oxford University in 1976, he worked as a software engineer in the UK before obtaining a six-month consulting post at CERN, the international nuclear physics research laboratory in Geneva. While he was at CERN, Berners-Lee was assigned to develop software for a new particle accelerator, but in his spare time he developed a hobby program for a hypertext system he called Enquire. (The program was named after a famous Victorian book of household advice called *Enquire Within Upon Everything* that had long before gone out of print, but that his parents happened to own and he liked to browse as a child.) Despite the similarity to Bush's vision, Berners-Lee had no firsthand knowledge of his work; rather, Bush's ideas had simply become absorbed into the hypertext ideas that were in the air during the 1980s. There was nothing very special about Enquire: it was simply another experimental hypertext system like dozens of others. When Berners-Lee left CERN, it was effectively orphaned.

The personal-computer boom was in full swing when Berners-Lee returned to England, and he found gainful employment in the necessary, if rather mundane, business of developing software for dot-matrix printers. In September 1984, he returned to CERN as a permanent employee. In the years he had been away, computer networking had blossomed. CERN was in the process of linking all its computers together, and he was assigned to help in this activity. But it was

not long before he dusted off his Enquire program and revived his interest in hypertext. Berners-Lee has described the World Wide Web as “the marriage of hypertext and the Internet.”¹⁶ But it was not a whirlwind affair; rather, it was five years of peering through the fog as the technologies of hypertext and the Internet diffused and intertwined. It was not until 1989 that Berners-Lee and his Belgian collaborator Robert Cailliau got so far as to make a formal project proposal to CERN for the resources necessary for creating what they had grandiosely named the World Wide Web.

There were two sides to the World Wide Web concept: the server side and the client side. The server would deliver hypertext documents (later known as web pages) to a client computer, typically a personal computer or a workstation, which in turn would display them on the user’s screen. By the late 1980s hypertext was a well-established technology that had moved far beyond the academy into consumer products such as CD-ROM encyclopedias, although it had not yet crossed over to the Internet. By 1991 Berners-Lee and Cailliau were sufficiently confident of their vision to offer a paper on the World Wide Web to the Hypertext’91 conference in December in San Antonio, Texas. The paper was turned down, but they managed to put on a half-hearted demonstration. Theirs was the only project that related to the Internet. Berners-Lee later reflected that when he returned to the conference in 1993, “every project on display would have something to do with the Web.”¹⁷

During that two-year period, the World Wide Web had taken off. It was a classic chicken-and-egg situation. Individuals needed web “browsers” to read web pages on their personal computers and workstations, while organizations needed to set up web servers filled with interesting and relevant information to make the process worthwhile. Several innovative web browsers came from universities, often developed by enthusiastic students. The web browser developed at CERN had been a pedestrian affair that was geared to text-only hypertext documents, not hypermedia documents that were enriched with pictures, sounds, and video clips. The new generation of browsers (such as the University of Helsinki’s Erwise and Viola at UC-Berkeley) provided not only the user-friendly point-and-click interface that people were starting to demand but also full-blown hypermedia features. Web servers needed software considerably more complex than a web browser, although this software was largely invisible to the average user. Here again, volunteer efforts, mainly from universities, produced serviceable solutions. The programs evolved rapidly as many individuals, communicating through the Internet, supplied bug fixes, program “patches,” and other improvements. The best-known server program to evolve by this process was known as “apache,” a pun on “a patchy” server. The Internet enabled collaborative software development (also referred to as the open-source movement) to flourish to an extent previously unknown. And the fact that open-source software was free offered an entirely new model for software development to the conventional for-profit software company – of which more in Chapter 14.

While all this was happening, Berners-Lee had a useful index of how the web was taking off – namely, the number of “hits” on his original web server at CERN. The number of hits grew from a hundred a day in 1991 to a thousand in 1992 and to ten thousand in 1993. By 1994, there were several hundred publicly available web servers and the World Wide Web was rapidly overtaking the gopher system in popularity – partly owing to the fact that, in the spring of 1993, the University of Minnesota had decided to assert ownership of the intellectual property in gopher software and no doubt would eventually commercialize it. It was perhaps the first time that the commercialization of Internet software had surfaced, and forever after there would be an uneasy tension between the open-source community that promoted free software and the entrepreneurs who saw a business opportunity.

In any event, Berners-Lee was on the side of the angels when it came to commercial exploitation. He persuaded CERN to place web technology in the public domain, so that all could use it free of charge for all time. In the summer of 1994 he moved from CERN to the Laboratory for Computer Science at MIT, where he would head the World Wide Web Consortium (W3C), a nonprofit organization to encourage the creation of web standards through consensus.

Browser Wars

What started the hockey-stick growth of the World Wide Web was the Mosaic browser. The first web browsers mostly came from universities; they were hastily written by students, and it showed. The programs were difficult to install, buggy, and had an unfinished feel. The Mosaic browser from the National Center for Supercomputer Applications at the University of Illinois at Urbana-Champaign was the exception.

Developed by a twenty-two-year-old computer science undergraduate, Marc Andreessen, Mosaic was almost like shrink-wrapped software you could buy in a store. There was a version for the PC, another for the Macintosh, and another for the Unix workstations beloved by computer science departments. Mosaic was made available in November 1993, and immediately thousands – soon hundreds of thousands – of copies were downloaded. Mosaic made it easy for owners of personal computers to get started surfing the web. Of course they had to be enthusiasts, but no deep knowledge was needed.

In the spring of 1994 Andreessen received an invitation to meet with the Californian entrepreneur Jim Clark. In the 1970s Clark had co-founded Silicon Graphics, a highly successful maker of Unix workstations. He had recently sold his share of the company and was on the lookout for a new start-up opportunity. The upshot of the meeting was that on 4 April 1994 the Mosaic Communications Corporation was incorporated to develop browsers and server software for the World Wide Web. The name of the corporation was changed to Netscape Communications a few months later, because the University of Illinois had licensed the Mosaic name and software to another firm, Spyglass Inc.

Andreessen immediately hired some of his programming colleagues from the University of Illinois and got started cranking out code, doing for the second time what they had done once before. The results were very polished. In order to quickly establish the market for their browser, Clark and Andreessen decided to give it away free to noncommercial users. When they had gotten the lion's share of the market, they could perhaps begin to charge for it. Server software and services were sold to corporations from the beginning, however. This established a common pattern for selling software for the Internet.

In December 1994 Netscape shipped version 1.0 of its browser and complementary server software. In this case "shipping" simply meant making the browser available on the Internet for users – who downloaded it by the millions. From that point on, the rise of the web was unstoppable: by mid-1995 it accounted for a quarter of all Internet traffic, more than any other activity.

In the meantime Microsoft, far and away the dominant force in personal computer software, was seemingly oblivious to the rise of the Internet. Its online service MSN was due to be launched at the same time as the Windows 95 operating system in August 1995. A proprietary network from the pre-Internet world, MSN had passed the point of no return, but it would prove an embarrassment of mistiming. Microsoft covered its bets by licensing the Mosaic software from Spyglass and including a browser dubbed Internet Explorer with Windows 95, but it was a lackluster effort.

Microsoft was not the only organization frozen in the headlights of the Internet juggernaut. A paradigm shift was taking place – a rapid change from one dominant technology to another. It was a transition from closed proprietary networks to the open world of the Internet. Consumers also had a difficult choice to make. They could either subscribe to one of the existing consumer networks – AOL, CompuServe, Prodigy, or MSN – or go with a new type of supplier called an Internet service provider (ISP). The choice was between the mature, user-friendly, safe, and content-rich world of the consumer network or the wild frontier of the World Wide Web. ISPs did not provide much in the way of content – there was plenty of that on the World Wide Web and it was rapidly growing. The ISP was more like a telephone service – it gave you a connection, but then you were on your own. The ISP customer needed to be a little more computer savvy to master the relatively complex software and had to be wary to avoid the less savory aspects of the Internet (or was free to enjoy them).

For the existing consumer networks, responding to the World Wide Web was a huge technical and cultural challenge. AOL licensed Microsoft's Internet Explorer for use in its access software. This gave its subscribers the best of both worlds – the advantages of the existing service plus a window into the World Wide Web. CompuServe did much the same, though it was eventually acquired by AOL in 1998. The other consumer networks did not manage to emulate AOL's masterly segue. Prodigy's owners, Sears and IBM, decided to sell the network in 1996, and it thereafter faded from sight. Microsoft, on the

other hand, decided to cut its losses on MSN. In December 1995 Bill Gates announced that Microsoft would “embrace and extend” the Internet; it “was willing to sacrifice one child (MSN) to promote a more important one (Internet Explorer).”¹⁸ Thereafter, MSN was not much more than an up-market Internet service provider – it was no threat to AOL.

Whereas in the 1980s the operating system had been the most keenly fought-for territory in personal computing, in the 1990s it was the web browser. In 1995 Netscape had seemed unstoppable. In the summer – when the firm was just eighteen months old – its initial public offering netted \$2.2 billion, making Marc Andreessen extraordinarily wealthy. By year’s end its browser had been downloaded 15 million times, and it enjoyed over 70 percent of the market. But Microsoft did not intend to concede the browser market to Netscape.

During the next two years Microsoft and Netscape battled for browser supremacy, each producing browser upgrades every few months. By January 1998, with a reported investment of \$100 million a year, version 4.0 of Microsoft’s Internet Explorer achieved technical parity with Netscape. Because Internet Explorer was bundled at no extra cost with the new Windows 98 operating system, it became the most commonly used browser on PCs – perhaps not through choice, but by the path of least resistance. A version of Internet Explorer was also made available free of charge for the Macintosh computer.

Distributing Internet Explorer to consumers at no cost completely undermined the business plans of Netscape. How could it sell a browser when Microsoft was giving it away for nothing? Microsoft had fought hard, perhaps too hard, for its browser monopoly. Its alleged practices of tying, bundling, and coercion provoked the US Department of Justice into filing an antitrust suit in May 1998. Like all antitrust suits, this one proceeded slowly – especially for an industry that spoke of “Internet time” in the way that canine enthusiasts spoke of dog years. Microsoft’s self-serving and arrogant attitude in court did nothing to endear it to consumers or the press, though this stance did little to arrest its progress. In the five years between 1995, when Microsoft was first wrong-footed by the rise of the Internet, and 2000, when Judge Thomas Penfield Jackson produced his verdict, Microsoft’s revenues nearly quadrupled from \$6 billion to \$23 billion, and its staff numbers doubled from 18,000 to 39,000. Judge Jackson directed that Microsoft be broken up – a classic antitrust remedy for the alleged wrongdoings of a monopolist – although this was set aside on appeal and less drastic remedies were imposed.

Privatization

During the second half of the 1990s, the Internet became not so much an information revolution as a commercial revolution; users were able to purchase the full range of goods and services that society could offer. In 1990, however, the Internet was largely owned by agencies of the US government, and the political

establishment did not permit the use of publicly owned assets for private profit. Hence privatization of the Internet was an essential precursor for electronic commerce to flourish.

Even before privatization had become an issue, it was necessary to separate the civilian and military functions of the Internet. Since its inception in 1969, Arpanet had been funded by the Advanced Research Projects Agency. In 1983 the military network was hived off as Milnet, and Arpanet became the exclusive domain of ARPA's research community. Once the military constraints were removed, the network flourished – by 1985 some two thousand computers had access to the Internet. To broaden access to the US academic community as a whole, beyond the exclusive circle of ARPA's research community, the National Science Foundation had created another network, NSFnet. In a way that proved characteristic of the development of the Internet, NSFnet rapidly overtook Arpanet in size and importance, eventually becoming the “backbone” of the entire Internet. By 1987 some thirty thousand computers – mostly from US academic and research communities – had access to the Internet. At the same time, other public and private networks attached themselves, such as Usenet, FidoNet (created by amateur “bulletin board” operators), and IBM's Bitnet. Who paid for their access was moot, because NSF did not have any formal charging mechanisms. It was a wonderful example of the occasional importance of turning a blind eye. If any bureaucrat had looked closely at the finances, the whole project for integrating the world's computer networks would likely have been paralyzed. As more and more commercial networks came on board, additional infrastructure was added and they evolved their own labyrinthine mechanisms for allocating the costs. By 1995, the commercially owned parts of the Internet far exceeded those owned by the government. On 30 April 1995, the old NSFnet backbone was shut down, ending altogether US government ownership of the Internet's infrastructure.

The explosive growth of the Internet was in large part due to its informal, decentralized structure; anyone was free to join in. However, the Internet could not function as a commercial entity in a wholly unregulated way – or chaos and lawlessness would ensue. The minimal, light-touch regulation that the Internet pioneers evolved was one of its most impressive features. A good example of this is the domain-name system. Domain names – such as *amazon.com*, *whitehouse.gov*, and *princeton.edu* – soon became almost as familiar as telephone numbers.

In the mid-1980s the Internet community adopted the domain-name system devised by Paul Mockapetris of the Information Sciences Institute at the University of Southern California. The system would decentralize the allocation of thousands (and eventually millions) of domain names. The process started with the creation of six top-level domains, each denoted by a three-letter suffix: *com* for commercial organizations, *edu* for educational institutions, *net* for network operators, *mil* for military, *gov* for government, and *org* for all other

organizations. Six registration authorities were created to allocate names within each of the top-level domains. Once an organization had been given a unique domain name, it was free to subdivide it within the organization by adding prefixes as needed (for example, *cs* for computer science in *cs.princeton.edu*), without the necessity of permission from any external authority.

Outside the United States, countries were allocated a two-letter suffix – *uk* for the United Kingdom, *ch* for Switzerland, and so on. Each country was then free to create its own second-level domain names. In Britain, for example, *ac* was used for educational (that is, academic) institutions, *co* for commercial organizations, and so on. Thus the Computer Laboratory at Cambridge had the domain name *cl.cam.ac.uk*, while *bbc.co.uk* needs no explanation. The United States is the only country that does not use a country suffix; in this regard it is exercising a privilege rather like that of Great Britain, which invented the postage stamp in 1841 and is the only country whose stamps do not bear the name of the issuing nation.

Early predictions of how the Internet would evolve were seemingly extrapolations of H. G. Wells's World Brain or Vannevar Bush's memex. In 1937 Wells wrote of his World Brain: "The time is close at hand when any student, in any part of the world, will be able to sit with his projector in his own study at his or her own convenience to examine *any* book, *any* document, in an exact replica."¹⁹ In 1996 (when the first edition of the present book went to press) this seemed a reasonable prediction, but one that might have been twenty or thirty years in the making. Although we see no reason to revise the time scale, the progress made has nonetheless been astonishing. Moreover – and this Wells did not predict – the information resources on the Internet went far beyond books and documents, encompassing audio, video, and multimedia as well. By the year 2000 a large fraction of the world's printed output was available online. Indeed, for university and industrial researchers, visits to libraries were made with ever-decreasing frequency.

Extraordinary as the progress of the Internet as an information resource has been, the rise of electronic commerce – a phenomenon largely unpredicted as late as the mid-1990s – was to prove even more so. And as the Internet became increasingly familiar, newspapers and manuals of English style dropped the capitalization – the *Internet* had become the *internet*.

Notes to Chapter 12

Since the first edition of this book appeared, there has been a flurry of general histories of the Internet, several of which are excellent. We recommend Janet Abbate's *Inventing the Internet* (1999), John Naughton's *A Brief History of the Future* (2001), Michael and Ronda Hauben's *Netizens: On the History and Impact of Usenet and the Internet* (1997), and Christos Moschovitis's *History of the Internet* (1999). For Internet statistics we have used data published by the Internet Systems Consortium (www.isc.org).

The historical context of the World Brain is given in a new edition of Wells's 1938 classic, edited by Alan Mayne: *World Brain: H. G. Wells on the Future of World Education* (1995). Historical accounts of Bush's memex are given in James M. Nyce and Paul Kahn's edited volume *From Memex to Hypertext: Vannevar Bush and the Mind's Machine* (1991) and Colin Burke's *Information and Secrecy: Vannevar Bush, Ultra, and the Other Memex* (1994).

The history of the DARPA Information Processing Techniques Office (IPTO), which effectively created the Internet, is very fully described in Arthur L. Norberg and Judy E. O'Neill's *Transforming Computer Technology: Information Processing for the Pentagon, 1962–1986* (2000) and Alex Roland and Philip Shiman's *Strategic Computing: DARPA and the Quest for Machine Intelligence, 1983–1993* (2002). Peter A. Freeman, W. Richards Adrion, and William F. Aspray's *Computing and the National Science Foundation, 1950–2016* (2019), tells the story of NSF's role in the transition from ARPANET to the Internet. Mitchell Waldrop has written a fine biography of J.C.R. Licklider, IPTO's founder and the seminal figure of personal computing: *The Dream Machine: J.C.R. Licklider and the Revolution That Made Computing Personal* (2001).

The context and evolution of the World Wide Web has been described by its inventor Tim Berners-Lee in *Weaving the Web* (1999), and by his colleagues at CERN James Gillies and Robert Cailliau in *How the Web Was Born* (2000). The "browser wars" are recounted in detail in Michael Cusumano and David Yoffie's *Competing on Internet Time* (1998). The privatization of the Internet is described in depth in Shane Greenstein's *How the Internet Became Commercial* (2015).

Notes

1. Wells 1938, p. 46.
2. Ibid., pp. 48–9.
3. Ibid., p. 60.
4. See Smith 1986.
5. Bush 1945.
6. Ibid.
7. Bush 1967, p. 99.
8. Licklider 1965, p. xiii.
9. Quoted in Goldberg 1988, pp. 235–6.
10. Licklider 1960, p. 135.
11. Quoted in Goldberg 1988, p. 144.
12. See Campbell-Kelly 1988.
13. Quoted in Abbate 1994, p. 41.
14. Quoted in *ibid.*, p. 82.
15. A headline in *Fortune*, 20 August 1984.
16. Berners-Lee 1999, p. 28.
17. Ibid., p. 56.
18. Cusumano and Yoffie 1998, pp. 10, 112.
19. Wells 1938, p. 54.



Taylor & Francis

Taylor & Francis Group

<http://taylorandfrancis.com>

Part Five

The Ubiquitous Computer



Taylor & Francis

Taylor & Francis Group

<http://taylorandfrancis.com>

13 Globalization

COMPUTER PRODUCTION AND CONSUMPTION has long ceased to be only a Western phenomenon. Although globalization of the computer industry had begun well before the dawn of the twenty-first century, it was greatly accelerated by the rise of mobile computing in the form of laptops, tablets, cell-phones, and smartphones. In the present century, American firms such as Microsoft, Apple, and IBM remained highly innovative and profitable. The continued success of American manufactures, however, was sustained in large part by *offshoring* and *outsourcing*. Offshoring companies – for which India became the most prominent location – produce software and provide computer services at low cost for tech giants abroad. Outsourcing enterprises – of which Taiwanese-owned Foxconn is perhaps the best-known example – undertake the low-cost manufacture of products designed and largely sold elsewhere.

Asian entrepreneurs, engineers, and workforces have made remarkable contributions to developing IT products and services. Emulating the global success of American and European firms, Asian manufacturers like Samsung, Lenovo, Huawei, and the Taiwan Semiconductor Manufacturing Company (TSMC) have aimed to reach customers around the globe. In this chapter we focus on two of these places: Latin America and Africa. With its huge population and largely untapped market, multinational companies were already attracted to Latin America in the twentieth century. Latin American engineers and entrepreneurs, however, have also developed their own products and services – with software development emerging as a promising industry in the twenty-first century. Africa is perhaps the most well-known example of a region with a population whose economic and social lives have been transformed by mobile phones. But the globalization of digital technology, at the levels of both production and consumption, has also widened inequalities within and among various countries.

Apple, Microsoft, and IBM: Taking Stock

The iPhone epitomizes Apple's worldwide success and the cachet of its products. Since the iPhone's release in 2007, every time a newer model has been

launched, Apple loyalists queued in long lines overnight to get their hands on the latest model. This happened world over, outside Apple stores in New York City, Vancouver, Beijing, Singapore, Sydney, Dubai, Paris, and London. Police were sometimes called to control the lines because of the overwhelming number of buyers or because of conflicts with iPhone “flippers” who intended to make an easy profit by reselling their iPhones.

The worldwide iPhone frenzy arrived after Steve Jobs returned to lead the company in 1997. Apple Computer was in trouble, with its already small market share of personal computers shrinking. The company was making a financial loss at a time when most other computer makers were doing well, given the steady growth in computer ownership. Within a few years, Jobs refocused the company on a smaller number of products made attractive through outstanding aesthetic design. Jobs spurred the company to create globally acclaimed and profitable consumer electronics such as iPods, iPads, and iPhones. The latter, especially, enabled Apple to outpace Microsoft and IBM in revenues by 2010. Jobs’s success came from two personal characteristics: an obsessive belief in designing consumer electronics that were both playful and easy-to-use; and his strong, sometimes brittle, management style.

In 2011, after resurrecting Apple and turning it into a highly profitable company in little more than a decade, Jobs, only in his fifties, died of pancreatic cancer. Earlier that year, he had stepped down from the CEO position due to his deteriorating health, and chief operating officer Tim Cook took over the position with Jobs’s full support. Cook had joined Apple in 1998 after a successful career at IBM and Compaq, recruited by Jobs to help steer the company back to financial stability. Cook aimed to slim down Apple’s poorly managed inventory, control manufacturing costs, and enhance product quality and delivery times. His solution was to outsource almost all the remaining in-house manufacturing operations to Apple’s long-term subcontractors, primarily in Korea, Taiwan, and China. This strategy worked effectively and swiftly, as it had for many other American computer makers. While Jobs focused on product design and consumer experience, Cook built upon his knowledge of manufacturing – in particular, global supply chain logistics.

Cook, the polar opposite of Jobs, was born and grew up in Alabama. Described as “soft-spoken, genuinely humble, and quietly intense,”¹ Cook remained in the shadow of his charismatic predecessor during the first few years after Job’s death. Industry observers expressed doubts about the future of Apple in the hands of its new CEO. But under Cook’s direction, Apple has remained beloved by consumers and admired by its investors. Successful new products continued to appear, particularly the Apple Watch. Apple’s innovative services, such as its App Store, Apple Pay, and television and music streaming, became big revenue earners. By 2020 Apple had become the most valuable publicly traded company in the world, overtaking both oil giant Saudi Aramco and Microsoft. The media praised Cook’s success. Cook is the first among the CEOs of tech giants to come

out publicly as “proud to be gay,”²² and he has spoken out against discrimination of lesbian, gay, bisexual, and transgender people. It is an open question whether Cook’s declaration has helped advance equality in the tech industry.

In summer 2011, when Cook became CEO and Jobs’s medical leave became public, Bill Gates drove to Jobs’s house for a courtesy visit. The two personal computer industry pioneers had each built tech empires that had been rivals for more than three decades, with just occasional strategic collaboration. The afternoon of the visit, they chatted about computers, education, and business. Gates and Jobs had fundamentally divergent philosophies on the relationship between hardware and software. Microsoft Windows was designed to run on as many devices as possible, whereas Apple products were closed systems, shutting the door to compatibility with software or hardware of other companies. Jobs’s biographer Walter Isaacson summarized their rapprochement: “I used to believe that the open, horizontal model would prevail,” Gates shared with Jobs, “But you proved that the integrated, vertical model could also be great.”²³

By the time of Gates’s visit, he was no longer the CEO of Microsoft. In 2000, after the Windows operating system’s worldwide success, he decided to step down, primarily to focus on philanthropic activities in education and later global health, medicine, and climate change. He continued as chairman of the board of Microsoft until 2014 and a board member until 2020, but he was no longer engaged in the day-to-day running of the business.

Steve Ballmer, Gates’s initial successor as CEO, did not at first instill confidence in industry observers. Like the contrast between Jobs and Cook, Ballmer – a college friend of Gates – was not a software visionary, but a veteran of Microsoft’s sales and management since 1980. As CEO between 2000 and 2014, Ballmer confounded his critics and Microsoft’s headcount and revenues grew by nearly a factor of four. Successes included the Xbox gaming system launched in 2001, which established Microsoft’s presence in the hardware field. But Microsoft failed spectacularly in launching a mobile phone operating system in 2010. Gates and Ballmer had high ambitions for Windows-based smartphones. But even after allying with Nokia, then a major player in cellphones, the Windows smartphone operating system lost out to Apple’s and Google’s phones announced three years earlier. The Windows phone was eventually discontinued in 2016.

Microsoft’s partnership with Nokia was a rational move. Nokia phones were once a cultural icon of mobile digital technology, appearing in popular films such as *The Matrix* in 1999 and *Twilight* in 2008. Headquartered in Finland, Nokia had had a major global presence in the mobile phone market since the mid-1990s. The company, however, was unprepared for consumers’ swift adoption of smartphones. It was focused on price-sensitive regions, where the sales of its traditional cellphones did not slow immediately. Elsewhere, Nokia’s Windows-based smartphones did not perform well enough to attract consumers.

Ballmer was credited for Microsoft's early investment in cloud computing services for business customers. His successor as CEO, Satya Nadella, has benefitted from the maturing of this technology, which boosted the company's revenues, surpassing those of IBM in 2015. Nadella obtained his undergraduate degree in electrical engineering from India's Manipal Institute of Technology and master's degrees in computer science and business administration from the University of Wisconsin, Milwaukee, and the University of Chicago. He joined Microsoft in 1992 as an engineer and oversaw Microsoft's cloud computing business from 2011 until becoming CEO in 2014. Nadella embodies several values of diversity and inclusion. Alongside Google CEO Sundar Pichai, appointed in 2015, and IBM CEO Arvind Krishna, appointed in 2020, more and more technologists of Indian heritage have become top executives. Known for his affable, collaborative style of leadership, he has made Microsoft fewer enemies and more friends. For example, he declared Microsoft's affection for Linux in 2014. While this was a strategy to expand Microsoft's market share in the cloud computing business by welcoming Linux users, it painted a more welcoming image of Microsoft, in contrast to its long-standing reputation as "an evil empire, scheming for domination."⁴

Microsoft's empire is global, like Apple's. By the turn of the century, nine out of ten personal computers worldwide used Microsoft's Windows operating systems. Although Microsoft is known for its sprawling Redmond campus, thirty minutes outside Seattle, Washington, the company's development activities have been spreading across the globe, by offshoring and outsourcing, especially in India. Microsoft's offshoring expanded dramatically in India and China from the mid-2000s, hiring thousands of software engineers for research and development. For its outsourcing, Microsoft has established close business partnerships with Indian software giants Wipro and Infosys – for example, in 2004 Wipro employed more than 5,000 workers to provide Microsoft services, including call centers, IT services, and software development.

While Apple illustrates American tech giants' moving their manufacturing to Asia, IBM's strategy at the turn of the twenty-first century was even more radical, especially for a century-old enterprise. IBM had been the iconic computer company throughout the second half of the twentieth century, but it gradually lost its mantle. In 1992 IBM sustained a \$5 billion loss, said to be the largest in corporate history. The company downsized and appointed a new CEO Louis V. Gerstner Jr. – formerly with American Express – to redirect the global giant and explore new and profitable lines of business. He was the first CEO not drawn from within the company, and this gave him a new perspective to evaluate the business opportunities for the company.

Gerstner decided to focus on services. To take advantage of the company's existing large corporate customer base, he led IBM to strengthen services such as installing and maintaining computer systems – a major concern for these customers because of surging costs. IBM's notable clients included the retailer

Sears and the State of California. The software business also gained importance as these clients increasingly relied on IBM to supply their software needs.

To focus on these new directions, IBM divested itself of small device manufacturing and sales in the 1990s. It sold its printer business in 1991 and its hard disk drive unit in 2002. IBM's personal computer sales had been declining since the 1990s, and it arranged in 2005 for the Chinese computer maker Lenovo to buy the IBM PC business. The biggest gain for IBM in the Lenovo deal was the possibility of opening up the Chinese market to IBM's software, services, and consulting businesses. Although IBM recovered profitability, it was no longer in the same league as Apple or Microsoft – in the year 2020, Apple's revenues were five times those of IBM and Microsoft's nearly three times.

The Rise of the Indian Software and Services Industry

Microsoft's Bill Gates has been visiting India on a regular basis since 1997. His first several trips were characterized by charitable activities, and he was greeted by enthusiastic crowds. These trips were not only for public relations but also to meet with politicians and to solidify business relations. His early twentieth-century trips to India likely addressed his concern about the world's second-most populous country's growing enthusiasm for Linux systems, which threatened the dominance of Microsoft's Windows. Overall, Gates saw India as on its way to becoming a "software superpower." During his 1997 trip, he praised India's "excellent university system," noting that its "computer scientists are among the leaders of companies worldwide. Its technology centers in Bangalore, Pune, and other places are well respected."⁵

Gates was not the first to notice these technological strengths. By the turn of the twenty-first century, India was known as "the world's back office," supporting overseas companies in services ranging from customer service to accounting. India's software exports had grown spectacularly, increasing by a factor of 400, from US\$7 million in 1985 to US\$3 billion in 1999. A major contributing factor was the arrival of the Y2K problem, or the millennium bug – potential computer errors caused by using two digits to represent the four-digit calendar year (a common practice to save computer memory in the early days of computing, when memory was expensive). To fix the problem, many American and European enterprises contracted with the major Indian software companies Tata Consultancy Services (TCS), Wipro, and Infosys. At least 185 *Fortune 500* companies outsourced some part of their software-related work, including software development, to Indian companies in 1999.

Saving labor costs was a driving force in the growth of the software services industry in India. The bursting of the "dot-com bubble" in 2000 prompted 35,000 skilled professionals to return from the United States and elsewhere to India, to join the already outstanding home-grown talent of the Indian software industry. A notable example showing the importance of Indian software talent to

American companies was the release of Microsoft's Xbox 360 in 2005. Several industry insiders in India pointed out that Xbox 360 was launched ahead of competitor Sony's PlayStation 3 because of the rapid quality testing of Xbox 360, outsourced to a company in India.

The historical transformation that made India a sought-after location for software and services had begun much earlier and was initiated by its domestic enterprises. In the 1970s, TCS had worked on programming tasks for Burroughs, one of the seven dwarfs in the mainframe computer industry, described in Chapter 6. Tata Group, India's largest conglomerate, was established in 1868. Three MIT graduates in their mid-twenties persuaded the company to set up the Tata Computer Centre in 1966. These MIT alumni had observed the various applications of computers in the United States and sought to computerize Tata's business practices, but they had limited scope to transform the staid, century-old enterprise. They repositioned their office as Tata Consultancy Services (TCS), offering management consulting and computing services to Indian enterprises beyond the Tata Group in 1968. In 1969, F. C. Kohli, who held a top-level position in Tata Electric, was assigned to oversee TCS. He was in his mid-forties and also an MIT graduate. Having been appointed to the board of directors of the Institute of Electrical and Electronics Engineers (IEEE) in 1972, Kohli used his attendance at IEEE meetings in the United States to visit Burroughs in Detroit to explore business opportunities for TCS. Burroughs hired and relocated a small number of TCS personnel to the United States. The work of these TCS engineers often involved converting an existing computer system of an organization, say an IBM system for a bank, to run on new Burroughs machines. The scale of these "body shopping" tasks was limited in the 1970s but expanded after TCS established an office in New York in 1979.

The 1980s witnessed the rise of a software export industry in India. The establishment of Software Technology Parks with satellite-based communications systems offered channels for Indian programmers to work for their global clients remotely from India. Some operations were set up by multinational corporations – for example, Citicorp Overseas Software Limited was created in 1985 to provide banking software for Citibank's global offices and other banks. Other exporters were domestic companies. Infosys, founded by N. R. Narayana Murthy and six software engineers in 1981, followed a similar business model to that of TCS. Infosys targeted the overseas software market by sending skilled programmers to work in customers' offices in the United States. Murthy and the other founders had all been employed and inspired by an MIT-trained, Indian-American entrepreneur Narendra Patni. In 1972, Patni, once an assistant to Jay Forrester, based his company Data Conversion Inc. in Cambridge, Massachusetts, but employed a hundred data-entry operators in India to convert text into machine-readable paper tapes. The tapes were then shipped to Massachusetts. This offshore data conversion task was used to develop the famous LexisNexis database for Arthur D. Little, a prominent consulting firm. A few years later,

Patni set up Patni Computer Systems in India to sell minicomputers and develop software for Indian customers.

TCS, Wipro, and Infosys are the three largest software services corporations in India at the time of writing. TCS's focus on software migration tasks in the 1980s gradually shifted to writing applications programs in the 1990s, and later to real-time database management, cloud services, and analytics. Tata Group was already a global enterprise at the dawn of the twentieth century. Through business growth and the acquisition of other companies in the 2000s, TCS attracted a global group of clients for its IT services. By 2015, it was one of the top ten IT services companies in the world by revenue.

Kohli and Patni exemplified a group of US-educated Indians and Indian-Americans who built India's software and services industry. They received US university degrees in science and engineering and worked in prominent US tech companies beginning in the mid-1960s. They typically extended their business opportunities to Indian companies and created international connections that fostered the exchange of talent and knowledge between the countries. There are similar histories of Taiwan's and Israel's high-tech industries.

Outsourcing of Manufacturing – Foxconn and Samsung

Just as Microsoft's profitability depended on its strong alliance with Indian software enterprises, Apple's success depended on its East Asian subcontractors and suppliers, including Foxconn, Samsung, and TSMC. East Asian contracting manufacturers rose during the 1980s era of microcomputers, as much more affordable microprocessors available. The outstanding quality and lower cost of their products, coupled with their flexible capacity to meet customer demands, provided incentives for major technology manufacturers to outsource their manufacturing to them. For example, by the early 1990s, the Taiwanese company Acer was manufacturing PCs and notebook computers for multinational companies including Apple, Texas Instruments, ICL, and Siemens. In 2011 alone, Taiwan's laptop manufacturers built 200 million laptops – nine-tenths of the world market.

Foxconn (also known as Hon Hai) has been a major contractor in making Apple's products, with factories in China taking advantage of a low-wage workforce and minimal legal regulation of factory working conditions. Self-made billionaire Terry Guo founded Foxconn in Taiwan in 1974. At first, Foxconn manufactured just one product – tuner knobs for televisions. By the 1980s it was making connectors for Atari's game consoles. Guo opened his first factory in China in 1988, and its manufacturing facilities became the powerhouse to supply desktop computer chassis for Dell Computer and Compaq from the mid-1990s. Foxconn also won contracts to manufacture the popular Sony PlayStation and Nintendo Wii. In 2002, Apple awarded Foxconn a contract to manufacture parts for and to assemble the iMac desktop computer. *Forbes* business magazine

announced in 2005 that Guo was the richest person in Taiwan. By then, he and Jobs had met to plan the manufacture of the first-generation iPhone. Foxconn's manufacturing capabilities proved to be extremely adaptable: not long before the launch of the iPhone, Jobs realized that the quality of the display on the prototype was inadequate, getting quickly scratched after he kept it in his pocket. He needed to replace the original plastic screen with one of scratch-resistant glass. Sourcing this type of glass from Corning and creating the logistics to cut such glass were complicated, leaving an extremely short time frame for Foxconn to install the new screens. Nevertheless, by mobilizing its large workforce Foxconn was able to deliver the iPhones in time for the launch.

Foxconn entered the public eye after international media uncovered cases of suicide of its factory workers between 2010 and 2012 at its Shenzhen facility in China, where employees assembled outsourced iPhones and other devices. Foxconn's Shenzhen campus had grown from 45,000 workers in the 1990s to at least 250,000 employees in the early 2010s. Most factory workers were young adults from rural areas in China and lived in dorms. Some roommates did not even know one another because of nonoverlapping working shifts. Managers considered these workers to be docile and willing to accept whatever tasks their supervisors assigned them. Obligatory overtime, alienation, and other unreasonable working conditions contributed to the series of suicides in the early 2010s. Nevertheless, thousands of prospective workers would show up outside the Foxconn facility when the company posted hiring notices. The facility had the capacity to hire 3,000 people overnight. The flexibility offered by such a large workforce was one of the major reasons that Foxconn continued as a competitive and indispensable subcontractor for Apple for years.

As the largest private enterprise in China, Foxconn had hired over 1 million employees by the mid-2010s. By comparison, Amazon's employees in the United States reached the same number only in 2020. As well as assembly line workers, Foxconn's engineers were also important to Foxconn's enterprise customers. In February 2011, Jobs had dinner with then-US President Barack Obama and other tech giant CEOs. Sitting next to Obama, Jobs explained to him that Apple counted on Foxconn's manufacturing engineers in China to oversee production by its 700,000 factory workers. He added, "These factory engineers did not have to be PhDs or geniuses; they simply needed to have basic engineering skills for manufacturing."⁶ Jobs's comments echoed Gates's observation a decade earlier about India: that educated engineers trained by domestic universities in China and India were one of the key driving forces of their thriving technology industries.

Like Foxconn, Samsung Electronics has manufactured chips and displays for Apple's iPods and, later, iPhones. However, Samsung was also making its own smartphones, computers, and home appliances, becoming a familiar brand name to consumers around the world. To Samsung, Apple is both a formidable competitor and an indispensable customer.

Samsung Electronics is part of the Samsung Group, a *chaebol* or family-run conglomerate, established in 1938. The Samsung Group dramatically expanded in the 1950s through the support of the Korean government for the modernization of the country's economy. Samsung Electronics was founded in 1969 as a home appliance maker. Manufacturing television sets locally was especially important for the company. After the 1970s oil crisis made imports of semiconductor parts challenging, the Samsung Group's founder, Lee Byung-chul, and his third son, Lee Kun-hee, recognized the benefits of fabricating transistors locally. In 1983, they decided to focus on the manufacture of dynamic random-access memory (DRAM). Relying on licensed technology from Japanese and American companies in the early years, the company's success in the development of advanced products ensured its dominance of the DRAM market by the mid-1990s. Samsung became the second-largest semiconductor vendor in the world in the mid-2000s, surpassed only by Intel. Samsung actively spends on research and development as well as advertising to stay competitive in the global market. Disaffected Koreans, however, unhappy with Samsung executives' questionable ties to the country's politicians, ironically call their country "the Republic of Samsung."

Outsourcing of Chip Manufacturing – TSMC

Samsung has been competing not only with Apple for the smartphone market, but also with TSMC and Intel in chipmaking. TSMC and Samsung have also been vying for Apple's profitable manufacturing contracts for smartphones since the early 2010s.

TSMC was established in Taiwan in 1987 by an American tech industry veteran. Unlike Samsung, which is a manufacturer in its own right and a household name around the world, TSMC is a contracting manufacturer that fabricates custom chips only for its enterprise customers. That is why the company is little known by electronics consumers, despite the fact that they likely have owned gaming consoles, desktop computers, digital cameras, routers, cellphones, or smartphones that use chips made by TSMC. TSMC trails behind Samsung and Intel in some respects but boasts a world-class manufacturing capacity for made-to-order, high-end chips. It became the largest of the made-to-order chip makers at the turn of the twenty-first century, and it has held over 80 percent of the market for high-end chips since 2014. The rise of TSMC illustrates how the outsourcing of manufacturing in the computer industry has evolved from the labor-intensive assembly of transistors beginning in the mid-1960s to the capital-intensive fabrication of sophisticated chips in the 2000s.

TSMC's founder, Morris C. M. Chang, was born in China in 1931, moved to Hong Kong in 1948, and graduated with bachelor's and master's degrees in mechanical engineering from MIT in 1953. He planned to pursue a doctorate at MIT but failed his qualifying exam twice. In 1955, he left MIT and began

to work for Sylvania, where he learned about early transistor technology. After three years, he decided to leave Sylvania, disaffected because two of the engineers he had recruited were laid off and because the company's semiconductor department was stagnating. Aged 27, Chang joined Texas Instruments in 1958. He was assigned to manage a production line for the mass manufacture of a transistor for a particular customer (IBM). After months of trial and error, the yield of his production line turned out to be outstanding. He rapidly climbed the corporate ladder, first promoted to vice president for the Components Group and then to senior vice president for Corporate Quality, Training, and Productivity.

As an established, high-level manager at Texas Instruments, Chang was disappointed by the lack of opportunities for further advancement. He left the company in 1984 to become President and Chief Operating Officer of General Instrument, an electronics manufacturer making TV components and transistors in the 1960s, and integrated circuits from the 1970s. However, General Instrument's business strategy of acquisitions and mergers held little interest for Chang. He subsequently received an offer from the Taiwanese government to lead its Industrial Technology Research Institute. In 1985, he arrived in Taiwan. He was immediately asked to propose a business plan for a new company that could improve the semiconductor industry's performance in Taiwan and bring more economic prosperity to the country. Within a week, he proposed the idea of a dedicated "foundry," which would fabricate chips solely for integrated circuit (IC) design houses that did not intend to invest heavily in fabrication plants. Later, TSMC extended its services to other IC manufacturers, including Intel, when they needed extra capacity.

Chang's concept of a dedicated foundry was inspired by a 1979 book, *Introduction to VLSI Systems*, written by Caltech professor Carver Mead and Xerox PARC computer scientist Lynn Conway. This famous book was a guide to the design of VLSI (Very Large-scale Integration) chips. Because VLSI chips are highly sophisticated integrated circuits, both their design and their manufacture are challenging. Mead and Conway separated the concerns of IC design from IC manufacture, and the book was widely used to teach students how to develop VLSI chips, independent of the manufacturing standards of the established semiconductor firms. While the book gave rise to the IC design industry, it also gave Chang a business idea for IC manufacture.

The dedicated foundry idea was not well received by the semiconductor industry in the United States in the late 1980s. There were few "fabless" IC design companies. However, Chang was aware that many IC designers he had met were keen to leave Texas Instruments and General Instrument to start their own companies but knew that they could not raise sufficient funds to manufacture their own ICs in-house. Chang's hope for the rise of fabless IC design companies became a reality in just a few years. Fabless companies boomed in Silicon Valley in the early 1990s – one example was NVIDIA, best known for its graphics processing chips used in gaming consoles. In the UK, the processors that power

most smartphones are designed by Arm, another fabless chip designer. Benefiting from the rise of the fabless semiconductor industry, TSMC has been profitable since 1991.

In 2017, Chang announced his plan to retire the following year at the age of 86. To a journalist's question about his achievements, he replied that "Without TSMC, the arrival of smartphones would be later than it was."⁷ He explained that the technology had an enormous impact on billions of people around the world, and his company TSMC helped that happen. His assertion may appear bold, because we usually attribute the ascendancy of smartphones to companies such as Apple. Yet Chang spoke not only for TSMC but also for its competitors, including Samsung. The dedicated foundry concept and the expansion of TSMC's production capacity was one of the driving forces behind Moore's Law – supplying ever more sophisticated chips for the consumer electronics market.

The Rise of Chinese Firms: Lenovo and Huawei

Lenovo exemplifies the rise of Chinese manufacturers of digital technologies after leader Deng Xiaoping opened up the Chinese economy in 1978. Legend Computer, founded in 1984, was renamed Lenovo in 2003, when the company was ready to globalize its business but found the name Legend already taken in the global marketplace. Chuanzhi Liu was the co-founder and chairman of Legend Computer. Born in 1944, he graduated from Xian Military Communication Engineering College with a degree in radar communications. During the Cultural Revolution of 1966–1976, Liu was assigned to work on state-owned farms. In 1970, an opportunity arose for him to become a researcher at the Institute of Computing Technology of the Chinese Academy of Science, a state-funded research institute. In 1984, when the research institute sought to commercialize its research, it offered seed money to Liu and his ten colleagues to set up a company. In its early years, Legend Computer's most successful product was the Legend Chinese card that enabled users to type Chinese characters on a personal computer.

While the Chinese government continued to restrict private business practices during the 1980s, Liu had to cautiously navigate a path for his company. For example, in 1987, when Legend Computer failed to obtain a permit to manufacture personal computers, Liu set up a company in Hong Kong to supply computer parts to China. Legend was fined by the Chinese government for the unauthorized importing of computer components in 1990. Legend was also subject to massive fines for violations of regulations on product prices and staff bonuses. In the 1990s, the Chinese state at last recognized Legend's technical capability and invited it, along with other companies, to build computer networks for the Chinese government's operations in finance, taxation, and customs. Having now become part of the government-led "golden projects," Legend grew rapidly.

Shortly before IBM approached Lenovo about selling its personal computer division in 2005, Liu and his successor, Yuanqing Yang, had already begun planning to grow its 30 percent share of the Chinese personal computer market. During the negotiations between IBM and Lenovo, skeptics in China referred to the purchase as “a snake swallowing an elephant.”⁸ But Lenovo quickly rose to become the third-largest PC maker in the world. In 2014 it acquired Motorola to expand its smartphone business. While its smartphone business is not as large as Huawei’s and other Chinese smartphone makers, Lenovo is nevertheless a substantial player.

Huawei is another example of an entrepreneur seizing a business opportunity after the Chinese economic reform of 1978. Former army engineer Zhengfei Ren founded Huawei in 1987 in Shenzhen, China, a city bordering Hong Kong. Shenzhen had flourished after Deng Xiaoping’s designation of the area as a Special Economic Zone for preferential trading with foreign corporations. At first, Huawei focused on selling telephone switches imported from Hong Kong, later manufacturing the switches domestically. In 1989, China’s landline telephone users numbered just over 5.5 million among the country’s 1.1 billion population. Huawei grew rapidly as telephones were installed throughout China in the early 1990s. In the domestic market for telecommunications equipment, Huawei was able to defeat international competitors, notably the American firm Cisco. Since the early 2000s, Huawei has become a global presence, winning contracts to build mobile phone networks in the Middle East, Europe, and Africa. It began to produce cellphones for UK-based multinational corporation Vodafone in 2006 and has been manufacturing high-end smartphones since the 2010s.

Computing in the Global South: Brazil and Beyond

Latin America’s populous countries have proved lucrative markets for multinational computer manufacturers, although domestic firms have also competed. From the early twentieth century, IBM regarded Latin America as an important market for its punched-card machines. By the mid-1960s, universities and large enterprises in Brazil, Chile, Mexico, and other Latin American countries had installed many IBM and Burroughs mainframe computers. During the 1970s and 1980s, however, Brazil introduced policies to restrict foreign imports to protect domestic computer manufacturing. The goal was to ensure that local companies could grow technically and financially, protected from formidable multinational corporations. These policies had both positive and negative effects. In the 1980s, Brazil was one of only three countries to produce domestically half of the computer products sold – joining just the United States and Japan. But the lack of collaborative R&D between Brazilian and foreign companies limited advances in technology. Moreover, locally made computers carried hefty price tags, resulting in an underground market of computers and computer parts. It was estimated that Brazilians illegally imported half of their personal computers from overseas in the 1980s.

Brazilian protectionist policies were relaxed in the early 1990s. Multinational computer manufacturers from around the world began to compete in Brazil's personal computer market – Brazil is the sixth-largest country in the world by population. However, tariffs and other restrictive laws still applied to hardware importation, resulting in computers costing about twice as much as in the United States. Despite the cost, Brazilian consumers were strongly committed to digital technologies. By 2009, Brazil's mobile phone and internet use ranked fifth largest among countries in the world. In 2019, Facebook had 130 million users in Brazil, its third-largest community, while for Instagram, Brazil was its second-largest community.

Although Brazilian computer manufacturers lost market share after the end of the protectionist policies, Brazilian software enterprises prospered by providing services for local corporate clients. TOTVS is a noteworthy example. The company was founded in 1983 by Laércio Cosentino, a mainframe software engineer, to develop software for small- and medium-sized businesses. TOTVS (meaning “totality” in Latin) grew to become the largest software company in Brazil in the 1990s. By 2013, TOTVS owned half of the Brazilian market for computer services and software. It has also established R&D centers, subsidiaries, and service operations throughout Latin America and the United States, and has become a major competitor of Germany-based SAP, one of the largest enterprise software companies in the world. Brazil also has a thriving outsourcing software industry, used by multinational companies.

More than half the companies in the *Fortune* 500, including Google, Amazon, and Netflix, chose São Paulo, the world's fourth-largest city, to be their Latin American headquarters. It is also home to many Brazilian start-up companies. A prominent start-up, founded in 2012, is “99 Taxis” (later just “99”), a taxi-hailing app which first operated in São Paulo and then expanded into other Brazilian cities. It was acquired by China's largest app-based vehicle-for-hire company, DiDi, in 2018. Beyond Brazil, Argentina's Globant is another successful Latin American start-up focusing on software development. In per capita terms, Chile has the highest number of start-ups on the continent and has aimed at making Santiago into “Chilecon Valley.”

To tackle income and wealth disparities, several Latin American states were especially keen to embrace the MIT Media Lab's One Laptop Per Child (OLPC) project, launched by Nicholas Negroponte in 2005. The goal of the project was to bring low-priced (\$100) laptops to school children worldwide, especially those who could not afford them. Negroponte believed the XO laptop, developed by the OLPC project, would revolutionize classrooms that had no access to digital educational technologies. His belief was based on his colleague Seymour Papert's “constructionist” learning theory, which asserts that children learn better if they actively look for the knowledge, instead of being instructed by teachers. The XO laptop would be a tool by which children could learn by finding knowledge themselves. By 2012, 85 percent of the XO laptops ended up in ten

Latin American countries. Peru and Uruguay were the most enthusiastic about the initiative. By 2012, Peru had spent \$225 million to purchase 850,000 XO laptops and distributed them to schools in disadvantaged areas.

Negroponte's plan to supply affordable laptops started something of a price war among competing manufacturers. Even so, the low-price commitment proved impossible, and the XO laptop was eventually priced closer to \$200 than \$100. Moreover, critics also argued that the XO laptop could have better incorporated the needs and concerns from the frontline users in the Global South, especially in the strengthening of its technical support system. As *The Economist* commented in 2008, "Mr Negroponte's vision for a \$100 laptop was not the right computer, only the right price. Like many pioneers, he laid a path for others to follow." The OLPC project folded in 2014.

Africa: Transformation Through Mobile Technology

By the 1990s, the African continent's low degree of adoption of personal computers had drawn the attention of international development agencies. In 1998, about 300 personal computers could be found for every thousand people in developed countries, whereas in Sub-Saharan Africa, there was less than one personal computer for every thousand citizens. In 2003, another survey indicated that only between 3 and 10 of every thousand Africans had access to a personal computer. The percentage of the population with internet access was even lower. To increase the availability of digital technologies on the continent, development agencies, nonprofit organizations, and national governments all contributed to bringing more computers and internet access to Africa. Beginning in the 1990s, the International Development Research Centre initiated and sponsored several "SchoolNet" projects in Africa – the SchoolNet movement had begun in North America and Western Europe in the 1980s, with the goal to assist schools with access to computer networks. In February 2000, for example, the Namibian government began implementing SchoolNet and provided computers or internet access to 120 schools in rural and disadvantaged areas.

While the high price of computing equipment was the main reason for the low number of computers purchased by African organizations and individuals, the existing telecommunications structure also slowed down the installation of computer networks. In 1998, for example, it took two to three months for a new telephone line to be installed in Côte d'Ivoire, which had one of the highest annual per-capita incomes on the continent. Of the four thousand computers sold there in that year, 80 percent were purchased by the government and private companies. Africa Online, eventually the largest internet service provider in Africa, was able to attract only two thousand Côte d'Ivoire customers by 1998. With the limited digital infrastructure, Africans found cyber cafés to be a solution to accessing computers and the internet. In 2003, in Kenya's capital, Nairobi, one could find around 300 cyber cafés; and in Ghana's capital, Accra, the number

was around 130. The two cities were at the higher end of the spectrum in terms of the density of cyber cafés in Africa, but they showed that cyber cafés were realistic alternatives to personal computer ownership.

Many of the computers used in SchoolNet projects and cyber cafés were secondhand. These required a high degree of maintenance and had a short life span, similar to the large numbers of used cars in Africa. Ghanaian entrepreneurs and expats, particularly, were well known for their extraordinary capabilities in trading, fixing, and utilizing older, used computers. The top four countries that Ghanaians imported their computers from were the United Kingdom, the United States, Germany, and the Netherlands. Unfixable secondhand computer parts, however, became part of the e-waste dumped in the district of Agbogbloshie in Accra.

Unlike PCs, Africa's digital technology users enthusiastically welcomed cell-phones. The ownership of mobile phones increased from 1 percent of the population in 2000 to 54 percent in 2012. A Pew Research Center report showed that the percentage of mobile phones (including basic mobile phones with limited or no internet connectivity) in South Africa and Nigeria was close to that of the United States by 2014. In that year, 80 percent of Africans used a mobile phone to send text messages, 53 percent to take pictures or videos, and 30 percent to make or receive payments. Among mobile phone owners, men outnumbered women. For example, in Uganda, 77 percent of men had a mobile phone, but only 54 percent of women did. Those who did not have their own mobile phones used creative strategies, such as sharing them. Another Pew Research report revealed that, among non-owners in Kenya in 2013, 58 percent shared a mobile phone with someone else. Users of shared phones could distinguish for whom an incoming phone call was intended by the number of rings or other prior arrangements. To minimize mobile phone bills, callers might use an agreed pattern of beeping or ringing to request that receivers call back.

Mobile money is one of the recent innovative technological services developed in Africa. In 2007, Vodafone and Kenyan mobile network operator Safaricom launched M-Pesa. The service allows a user with no bank account to deposit, withdraw, and transfer money using a mobile phone that can send text messages, or by visiting a mobile money agent whose function is similar to an ATM. Kenyan policymakers accepted the creation of this mobile money system alongside the existing banking system, contributing to M-Pesa's success. The service extends economic opportunities and flexibility to many Africans with no bank accounts, who outnumbered those having bank accounts. Mobile money services help mitigate poverty. For example, if one loses one's job, a mobile money service increases one's opportunity to receive immediate help from family and friends. Mobile money service is now a fiercely competitive market in Sub-Saharan Africa with its population of over a billion. In 2020, there were 157 mobile money services operating in the region, with 159 million active accounts. That same year, mobile technologies and services accounted for 8 percent of the region's GDP. Software developers and entrepreneurs in Africa have been

creating mobile phone apps to meet local digital needs, such as apps to search for nearby gas stations with the least expensive prices, to hail motorcycle rides, and to sell farm produce directly to buyers without a middleman.

Social innovations are critical in facilitating the take-up of digital technologies. For example, since 2001, nongovernment organizations and the International Development Research Centre have carried out translation projects in Africa, the Middle East, and South Asia to make computer software accessible through local languages, including Zulu, Swahili, Pashto, and Nepali. These organizations translated the free open-source Firefox web browser and OpenOffice into local languages to enable a more inclusive group of users. They also developed optical character recognition systems, text-to-speech systems, and keyboards for speakers of these languages to better utilize the new digital tools.

Disempowerment and Opportunities

The business opportunities that arose with the globalization of computing went in tandem with the disempowerment of the worldwide labor force in the computing industries. While job opportunities in electronics manufacturing factories enabled the higher labor participation of women since the mid-1960s, they also gave rise to the stereotypical treatment of women. For example, women of color around the world have been portrayed as having nimble fingers well-suited to assembling electronics. Such job opportunities in the factory can be taken away overnight. Manufacturers seeking cheap labor have opened new factories while closing factories elsewhere to save on labor costs. Foxconn workers' suicide cases, noted earlier in this chapter, generated widespread concerns over electronics-assembly workers' rights in the early 2010s. Around the same time, public media revealed that mobile phones and many other computing devices have sourced minerals from mines in Congo and Indonesia, which have been poorly regulated, unsafe, and exploited child labor. These cases prompted consumers, producers, and activists to consider the ethics of manufacturing digital devices. However, the long-term effect of this activism is uncertain. As Lizhi Xu, a Foxconn worker who took his own life in 2014, at the age of 24, wrote in one of his poems – no one would pay attention to “someone plunged to the ground,” as no one would notice “a screw fell to the ground . . . in this dark night of overtime.”¹⁰

The wide availability of electronics and microprocessors left remarkable changes in cities, towns, and villages worldwide. For example, Japan's Akihabara, which emerged in Tokyo after World War II as a market for radios and vacuum tubes, became renowned for its subcultures of manga, anime, games, and electronics retail shops. After the 1980s, shopping malls and whole streets were taken over by stores where one could purchase or even customize Apple and IBM clones – including Taiwan's Chunghua and Kuanghua arcade buildings, South Korea's Cheonggyecheon electronics market, Hong Kong's Sham Shui Po, Singapore's Sim Lim Square, Colombia's Unilago Mall and Monterrey

Mall, and others. These sites were critical venues for societies beyond Europe and North America to explore and obtain hands-on experience using computers and to tinker with them. In no time, these malls evolved from the territory of pirated goods into legitimate domestic computer makers' retail shops. Societies across the globe will keep weaving digital technologies into their histories.

Notes to Chapter 13

Discussion of Apple's globalization since 1998 draws on Apple's CEO Tim Cook's biography—Leander Kahney's *Tim Cook: The Genius Who Took Apple to the Next Level* (2019). IBM's strategic moves to software and services after the 1990s are chronicled in James W. Cortada's *IBM: The Rise and Fall and Reinvention of a Global Icon* (2019). Microsoft, IBM, and other American companies' outsourcing of software development to India is thoroughly covered in *Globalization and Offshoring of Software* (2006) edited by William F. Aspray, Frank Mayadas, and Moshe Vardi. A comprehensive history of various Indian companies' outsourcing business is delineated in Dinesh C. Sharma's *The Outsourcer* (2015). The contribution of MIT-educated Indian elites to India's IT industry is elaborated in Ross Bassett's *The Technological Indian* (2016), especially chapter 9.

Discussion of the founding of TSMC draws in part from Morris Chang's oral history interview by the Computer History Museum. A business strategy-focused analysis of Samsung is given in Sea-Jin Chang's *Sony vs Samsung: The Inside Story of the Electronics Giants' Battle for Global Supremacy* (2011). An authoritative account of the rise of Chinese enterprises and their critical business strategies is given in Michael Useem et al.'s *Fortune Makers: The Leaders Creating China's Great Global Companies* (2017).

In the discussion of Latin America, we have been guided by *Beyond Imported Magic: Essays on Science, Technology, and Society in Latin America* (2014), edited by Eden Medina et al. Brazilian protectionist policies for the computer industry are thoroughly reviewed in Jeff Seward's *The Politics of Capitalist Transformation: Brazilian Informatics Policy, Regime Change, and State Autonomy* (2016). A critical analysis of the racial and technological politics of OLPC is given in Rayvon Fouché's "From Black Inventors to One Laptop Per Child" (2012).

For Ghana's computers and users, an excellent ethnography is given in Jenna Burrell's *Invisible Users: Youth in the Internet Cafés of Urban Ghana* (2012). African immigrants' creative use of mobile phones is beautifully illustrated in Henrietta Mambo Nyamnjoh's dissertation, *Bridging Mobilities: ICTs Appropriation by Cameroonians in South Africa and The Netherlands* (2013), especially chapter 4. For mobile money, we have been guided by Tavneet Suri and William Jack's *Science* article, "The long-run poverty and gender impacts of mobile money" (2016).

Notes

1. Kahney 2019, p. 113.
2. Cook, Tim. 2014. "Tim Cook Speaks Up." *Bloomberg Businessweek*, 30 October. www.bloomberg.com/news/articles/2014-10-30/tim-cook-speaks-up#xj4y7vzkg.
3. Quoted in Isaacson 2011, p. 554.
4. Anon. 2019. "What Microsoft's Revival Can Teach Other Tech Companies." *The Economist*, 27 July, p. 10.
5. Microsoft. 1997. "Bill Gates: Technology Investments Will Make India an Economic and Software Superpower." 3 March. <https://news.microsoft.com/1997/03/03/bill-gates-technology-investments-will-make-india-an-economic-and-software-superpower-2/>.
6. Quoted in Isaacson 2011, p. 546.
7. Quoted in Business Weekly Magazine 2018.
8. Liu 2007, p. 574.
9. Anon. 2008. "One Clunky Laptop Per Child." *The Economist*, 4 January, <https://www.economist.com/science-and-technology/2008/01/04/one-clunky-laptop-per-child>.
10. Quoted in Chan et al. 2020, p. 191.

14 The Interactive Web

Clouds, Devices, and Culture

IN THE LATE 1990s, the distinction between producers of content on the World Wide Web – primarily companies and other organizations – and the many more consumers of content was clear. To be sure, some early online businesses such as eBay provided platforms for users to add text and images (so their goods could be effectively auctioned) and some computer-savvy individuals set up web logs or “blogs” to opine or to publicly journal, but those creating content on the web were few compared to the millions who browsed on it daily. In the early 2000s the relatively static web, characterized by a limited number of active producers and many passive consumers, began to rapidly change as platforms for facilitating and encouraging user-generated content and interaction proliferated. Search engines proved especially important for navigating the ever-growing scale and scope of websites. Improving navigation, in turn, incentivized content creation on the web. A prescient industry consultant in 1999 coined the term *Web 2.0* for this turn toward greater interaction, and a Web 2.0 industry conference in 2004 solidified this nomenclature. Several years later, the launch and rapid adoption of smartphones, and the growth of “the cloud” – a vast network of remote computer servers that host platforms, software, and data – greatly accelerated the interactive web.

Mobile Computing and Smartphones

From shortly after the advent of personal computing, computers have become increasingly mobile. In 1968 Alan Kay conceived of a notebook-style portable computer, formalized as the “Dynabook” concept at Xerox PARC in 1972 (described in Chapter 11). That year, Kay and PARC colleagues incorporated many user-friendly elements of the Dynabook into the Xerox Alto workstation, but easy mobility was not one of them. Portability had to await the maturing of microelectronics. The first “portable” computers were available by the early 1980s, but they were a far cry from the modern laptop. For example, the 1981 \$1,800 Osborne 1 lacked a battery, and so had to be plugged into an electrical outlet, weighed more than 23 pounds, had a tiny 5-inch CRT screen, and, when

folded up for carrying, resembled a mid-sized hard-shell suitcase. Various models of commercial portable computers hit the market as the decade progressed, including the Tandy TRS-80 100, the first popular portable to use an LCD screen – a semiconductor technology that came to replace clunky CRT screens. These portables, however, gave up much in screen size, processing power, and memory to achieve a degree of mobility. By the early 1990s advances in microprocessors and memory-chip capacity, coupled with improvements in LCD screens, were sufficient for the affordable clamshell-design modern laptop – with models from IBM, Compaq, and others becoming thinner, lighter, and more powerful with each passing year. The price-to-performance of laptops approached that of desktop computers, and the advantage of mobility led laptop production to surpass that of desktops by 2008.

While carrying laptops on the go, especially for work, became increasingly common, their size and weight prevented these machines from becoming personal accessories. Computers became common every day and everywhere devices for the masses only with the advent and growing popularity of smartphones – ushering in a transformative new era of mobile computing by altering how people communicate, work, and socialize.

The smartphone evolved from so-called feature phones developed in the 1990s. Feature phones incorporated facilities such as a camera, a music player, and rudimentary internet access in a cellular phone. By contrast a smartphone was a handheld computer that included a wireless phone. Smartphones were characterized by their operating system, and two new platforms emerged in 2007–2008 that rapidly gained dominant positions in the smartphone marketplace: Apple's iOS and Google's Android. Within five years these two had captured over 90 percent of smartphone sales.

Apple's successful entry into smartphones came with the return of its charismatic co-founder Steve Jobs to lead the company. While Apple's Macintosh had been a technical success at its launch in 1984, it was not a financial success. Apple was struggling as a company in the mid-1980s, and Jobs lost a boardroom battle, was isolated, and elected to resign from the firm. In 1985 Jobs formed NeXT, a computer development company focused on the educational and business markets. NeXT acquired the small computer graphics division of Lucasfilm Ltd., which it later spun off as Pixar. Walt Disney acquired Pixar in 1996, and Jobs became Disney's largest individual stockholder and a board member. This broad exposure beyond computing was influential in making Jobs aware of new media and consumer electronics opportunities when he was invited back to lead Apple Computer in 1997.

By the time Jobs returned, Apple Computer already had a long history in portable computing. There had been a continuous stream of new laptop models from the 1989 Mac Portable forward. The firm also launched the Newton in 1993, a notebook-style PDA computer – standing for “personal digital assistant” – which was a technical success but a market failure and was withdrawn in 1998.

Jobs envisioned the firm's expansion into consumer electronics. Small digital music players, based on the compressed digital music standard MP3, had been around for several years before Apple launched the iPod in 2001. Apple's player, however, rapidly captivated consumers and gained market leadership. In addition to the iPod's sleek design, Jobs's charisma with product introductions, and Apple's inspired television advertising using bold animation and the music of pop mega-group U2, the key to the iPod's immense success was the simultaneous launch of Apple's iTunes store. Overcoming the conservatism of the Recording Industry Association of America, which had resisted digital music and online delivery, Apple secured attractive terms to sell music at a profit and, in time, to extend the iTunes store into other media. With the iPod line and iTunes store, Apple gained a leading reputation in marketing popular handheld devices and in supplying content – both critical to the success of the iPhone.

Apple's iPhone, launched in 2007, transformed the smartphone market with its easy-to-use touchscreen and its successful cultivation of third-party applications developers. Apps for the iPhone quickly included an abundance of games, as well as thousands of other entertainment, work, and educational offerings. The iPhone App Store, launched in 2008, resembled the familiar iTunes store. It dwarfed the offerings of competitors and drove demand for Apple phones that for years sold at a substantial price premium to other smartphones. The company charged a lucrative 30 percent commission to app developers for their sales on the App Store. This income grew steadily to become substantial, complementing what quickly became over half of Apple's revenues, iPhone sales.

Meanwhile, in 2005, Google had acquired Android, Inc., a producer of an open-source, Linux-based mobile operating system. Google licensed the platform to smartphone manufacturers on an open-source, royalty-free basis. Google retained the Android trademark, ensuring adherence to standards. The company derived income because its advertising-supported apps (such as Gmail) were central to the platform. The first Android-based phones were announced in 2008 and soon an extensive third-party Android app ecosystem emerged. With iOS and Android's ascendance, other platforms – once among the biggest in the mobile phone industry – were eclipsed and exited the market, among them Blackberry, Nokia, Palm, and Microsoft's Windows Phone. The outcome of this emergent duopoly, and the Apple and Android commercial battle, however, is less important than the momentum that mobile computing gave to "The Cloud," platforms, social networking, the gig economy, and other infrastructures of work and leisure.

The Cloud: Time-Sharing for the Twenty-First Century

The mid-1960s advent and multi-decade expansion of time-sharing (explored in Chapter 9) allowed scientists, engineers, students, and others to interact with computers directly for the first time. As MIT and Dartmouth were pioneering

time-sharing on their campuses in the 1960s, a nascent time-sharing industry was gaining traction offering services to organizational users. At first, time-sharing mainframes operated from local hubs because of high long-distance telecommunication rates. By the late 1960s, however, national communications networks had been established, which soon grew to serve international markets. The major time-sharing companies, such as General Electric's GEISCO, Tymshare, and Control Data, offered small but growing libraries of applications software for accounting, payroll, tax preparation, and the like. These forerunners of "apps" complemented their primary business of selling computer time. Accelerating personal computer adoption by the mid-1980s led to the time-sharing industry's steady decline, and by the early 1990s, its demise. Time-sharing, at once, had been highly influential and limited. At its 1982 peak, only a small fraction of 1 percent of the United States population, and even fewer globally, were time-sharing users; and nearly all of this was confined to work or education at universities and schools. Time-sharing fell far short of the 1960s ideal of a domestic computer utility for the masses. At the same time, it was important in what it offered and achieved in education, research, and industry, and as a vision of the future.

With the growth of the web by the late 1990s, networking and remote applications and services revived interest in what time-sharing had pioneered. In 1999, Marc Benioff, a vice president of the Oracle Corporation, left his employer to establish Salesforce.com in a rented San Francisco apartment. His vision was a web platform for data and applications, focusing on his area of expertise – customer relationship management (CRM). CRM integrated data on customers from different sources within an organization, such as sales, marketing, accounting, and customer service. Salesforce.com became the fastest-growing enterprise software company in history – jumping from \$5 million in revenue in 2001 to \$1.3 billion in 2010. Key to this rapid growth was the aid that the CRM platform provided to salespeople in the field. It allowed them to readily record, search, and analyze data on customers and potential customers. By 2010, a salesperson using the new Salesforce.com iPhone app could access data and analytics on its customers on the go as never before.

What Benioff and Salesforce helped pioneer – borrowing from and reinventing time-sharing of the past – was Software as a Service. Software as a Service, or SaaS, allowed software and data to reside on the remote servers of cloud service providers but to be accessed on personal computers and mobile devices. Benioff became a leading and outspoken evangelist for the SaaS idea – disrupting the software industry, including giant incumbents such as his former employer Oracle, Microsoft, SAP, and others. They subsequently followed Salesforce into the SaaS space. For example, Microsoft, which had long sold its applications as boxed software, offered Microsoft Office in a SaaS form in 2011 – branded as Office 365. Instead of an outright sale, Office 365 was leased month-by-month. Leasing rather than selling smoothed out fluctuating sales, greatly boosted

overall revenue, and ensured that customers had all the latest updates. The preceding year, Microsoft had launched Azure, a cloud platform to support its many applications and services and those of other software enterprises. With Azure, and a shift to SaaS, Microsoft catapulted to become the second-largest cloud service. The largest was Amazon Web Services.

The term “The Cloud” was coined by two engineers at Compaq in 1996 and gained currency after its use in 2006 talks by Google’s CEO Eric Schmidt and Amazon’s founder Jeff Bezos. For Bezos, who had launched Amazon in 1994 as an online retailer, the occasion was his 2006 address at MIT’s Emerging Technologies Conference, where he unveiled Amazon Web Services (AWS). By that time, Amazon had built up a vast infrastructure of servers and software for its e-commerce operations and would now make them available for corporate, government, and other organizational clients. Companies, which had in the past maintained their own computing infrastructure, increasingly began contracting with specialists such as AWS, Microsoft, and other leaders in the rapidly growing field of cloud computing.

Smartphones and cloud computing greatly accelerated in the 2010s. Among the factors behind cloud computing’s adoption by many companies and other organizations were economies of scale and reduced costs, continuously updated software, the integration of global operations, as well as improved security and disaster recovery. Lower up-front software costs (in renting versus owning) were also a financial benefit. Although the cloud may come to be seen as a major inflection point in computing, adoption has been far from universal. Long-established user organizations, with legacy computer systems and existing personnel, may take a technological generation to adapt. But for newer enterprises – such as Facebook and Netflix, both clients of AWS – cloud computing was the path of least resistance.

In the May 1964 issue of the *Atlantic Monthly* MIT industrial engineering professor Martin Greenberger wrote on the “Computers of Tomorrow,” speculating that the “information utility may be as commonplace by 2000 AD as telephone service is today.”¹ Observing the rise of time-sharing, he predicted that computers would become ubiquitous in homes, offices, laboratories, and business. Technically, his prediction of the year 2000 was too optimistic – it took another two decades for household internet adoption in the United States to match the home phone service levels of 1964, but much in today’s digital world far exceeds his grand vision.

Google: From “Do No Evil” to Surveillance Capitalism

Some of the early online business successes were for listing services to locate information on the unstructured web. For example, Yahoo (sometimes written as Yahoo!) began as a simple listing service (“Jerry’s Guide to the World Wide Web”) created by two computer science graduate students at Stanford University,

David Filo and Jerry Yang. By late 1993, the site listed a modest two hundred websites, though at the time that constituted a significant fraction of the world's websites. During 1994, however, the web experienced rapid growth. As new sites came online daily, Filo and Yang sifted, sorted, and indexed them. Toward the end of 1994 Yahoo experienced its first million-hit day – representing perhaps 100,000 users. The following spring Filo and Yang secured venture capital, moved into offices in Mountain View, California, and began to hire staff to surf the web to maintain and expand the index. It was very much as H. G. Wells had envisaged for the World Brain when he wrote of “a great number of workers engaged in perfecting this index of human knowledge and keeping it up to date.”² One key question remained: How to pay for the service? The choices included subscriptions, sponsorship, commissions (a model where third-party sellers pay a percentage of revenue on a sale), or advertising. As with early broadcasting, advertising was the obvious choice.

By the mid-1990s Yahoo was well established, with several competitors including Lycos, Excite, Infoseek, and Altavista, when two other Stanford University doctoral students, Larry Page and Sergey Brin, began work on the Stanford Digital Library Project. Funded in part by the National Science Foundation, their research changed forever the process of finding content on the internet. Page became interested in a dissertation project on the mathematical properties of the web and found strong support from his adviser Terry Winograd, a pioneer of artificial intelligence research and human-computer interaction. Using a program to gather information on page titles, images, text, and “back-links” (web pages that linked to a particular site), Page, now teamed up with Brin, created their “PageRank” algorithm. This algorithm was based on back-links ranked by importance – the more prominent the linking site, the more influence it would have on the linked site's page rank. They reasoned that this would provide the basis for more useful web searches than any existing tools and that there would be no need to hire a corps of indexing staff. Thus was born their “search engine,” Backrub, renamed Google shortly before they launched the URL *google.stanford.edu* in September 1997. The name was a modification of a friend's suggestion of *googol* – a term referring to the number 1 followed by 100 zeros. Brin misspelled the term as *google*, but the internet address for *googol* was already taken so the catchy misspelling stuck. While just a silly made-up word to most users, the original term was indicative of the complex mathematics behind Page and Brin's creation, as well as of the large numbers (in terms of web indexing and searches) that their search tool would later attain.

In 1998 Page and Brin launched Google Inc. in a friend's garage in nearby Menlo Park. Early the following year, they moved the small company to offices in Palo Alto. By the early 2000s Google had gained a loyal following, and thereafter it rapidly rose to become the leading web search engine. Taking \$25 million in loans from leading Silicon Valley venture-capital firms to refine the technology, hire more staff, and greatly extend the infrastructure

(the ever-expanding number of servers), the two founders were forced to hire a professional CEO, Eric Schmidt, early in 2001. Now with “adult supervision,” Google developed three primary revenue streams: licensing its search technology to Yahoo and others, custom search services for clients, and sponsored search results tied to search words, or what it labeled AdWords. AdWords ensured that a few sponsored ads would appear above other results. For instance, if a search was “new automobiles,” Ford, GM, BMW, and Toyota might pay Google to appear at the top of the search results. AdWords grew rapidly and became Google’s primary revenue source, far eclipsing licensing or custom search.

As the new century opened there was a loss of investor confidence in the over-hyped prospects for the internet that resulted in the “dot.com bust” in which many internet ventures went bankrupt, and IT stocks lost three-quarters of their value.³ Google’s successful 2004 Initial Public Offering (IPO) symbolized the revival of both IT and Silicon Valley. It raised \$1.9 billion, at the time, an unprecedented sum for an IPO in the IT industry. That year, Google achieved revenues of \$3.19 billion, also unequalled at the time for an IT company just over a half decade from its founding.⁴ It became and has remained dominant ever since, handling the great majority of billions of searches every day. Search-based advertising has remained its primary source of income, but in the 2000s Google expanded into complementary offerings. These included the Gmail e-mail service in 2004, video sharing with YouTube in 2006, the Chrome browser in 2008, followed by the Android smartphone operating system.

Like many internet businesses, Google has refined its advertising model by capturing users’ data to deliver targeted ads. In Google’s case, by extracting data from search results, browsing history, e-mail exchanges, and YouTube viewing patterns. A prominent psychologist has aptly referred to our times as *The Age of Surveillance Capitalism*. Monitoring and profiling have become central to advertising, marketing, and commerce in the digital world. The depth and type of usage of user data extracted by Google and the social networking giants are unprecedented. The application of artificial intelligence and machine learning offer further windows into our behaviors and predictive patterns, which is very valuable to advertisers. Americans and others claim to value privacy, yet paradoxically, will quickly trade it for a discount; grocer rewards cards are but one example.

In an October 2015 corporate restructuring Google was renamed Alphabet. At the same time, it changed its long-used unofficial motto and code of conduct from “Don’t be evil” to “Do the Right Thing.” There is an irony to using the same phrase as Spike Lee’s influential 1989 film on race. Indeed, Google’s “algorithms of oppression” have been criticized for racial stereotyping, sexism, and reinforcing social inequality.⁵ The social media giant Facebook likely has had an even more profound role in encouraging and amplifying these inequalities.

Social Media: Facebook and Twitter

Web-based social networking platforms enable users to create a profile on the web and interact with others. For some users, especially teens and young adults, web-based social networking has become an everyday activity and a fundamental part of their social lives.

In the early 2000s several social networking firms emerged that achieved visibility. Among these were Friendster (formed in 2002) and MySpace (formed in 2003). These two companies, both founded in California and initially focused on the United States, allowed users to create individual public- or semi-public profile web pages and to connect with others. Friendster and MySpace grew rapidly in their first half-decade and gained millions of users, and while seen as invincible at the time, they were soon greatly overshadowed by Facebook.

Enrolling at Harvard University in 2002, freshman Mark Zuckerberg founded Facebook – then called Thefacebook – in his dormitory. Frequently occupied with designing and programming computer applications during his first semester, he created two hit programs. The first, Course Match, enabled students to match up classes with others; the second, Facemash, allowed students to compare and choose (based on attractiveness) between two competing Harvard freshmen portrait photos. Drawing from these two programs in early January 2004, and from Friendster (which he belonged to), Zuckerberg registered the domain name *theface-book.com* and created a platform that enabled Harvard University students, faculty, staff, and alumni to create profiles, post a photo of themselves, and invite other members to connect with them as “friends.” On 4 February the site went live; within four days, hundreds of Harvard students had registered and created profiles. In three weeks, Thefacebook had more than 6,000 members.

From the start, Zuckerberg wanted to expand the site beyond Harvard to other colleges and universities. Zuckerberg and a few friends he collaborated with expanded Thefacebook to other Ivy League schools and then to other universities. By midway in the 2004 academic year the site had more than 100,000 users. In the summer of 2004, he incorporated and moved his operation to a house in Palo Alto, California.

Thefacebook followed a successful strategy that controlled growth and focused on an initial user population of students. In this way, Zuckerberg created a pent-up demand and targeted a computer-literate group particularly focused on their social life. This strategy lessened the need for finance and allowed the site to grow in a measured way so that greater attention could be paid to reliability. By starting with universities and only accepting users with university-issued “.edu” e-mail addresses, the company minimized fraudulent accounts. Friendster’s reputation, in contrast, was severely hurt by service problems and fake accounts – and sometimes derisively called “fakester.”

Thefacebook was renamed Facebook in August 2005. After opening to most of American higher education, Facebook next became available to high schools;

by spring 2006 more than a million high school students had signed up. A few months later, Facebook was made available to all users aged thirteen or older. The company then began to concentrate on international expansion – eventually completing a translation project that enabled it to be used in thirty-five languages by the end of 2008. By that time “70 percent of Facebook’s then 145 million active users were already located outside the United States.”⁶ Although the company’s expansion beyond educational users opened the door to increased fraudulent accounts, it was at a smaller percentage than earlier social networking sites.

In 2006 Facebook had its first major misstep with the launch of News Feed, which automatically updated a stream of news based on changes and updates to friends’ profiles. While this did not provide any information that was not already available, the act of automatically pushing information through a user’s friend network led many to find it creepy and “stalker-esque.” A “Students Against Facebook News Feed” group quickly formed after the application went live, and within days 700,000 people had joined in the online protest. Zuckerberg, who has long stated an ideological preference for information sharing and openness, expressed surprise at this reaction. He took weeks to respond with an apology and enhanced privacy settings that enabled users to disable News Feed.

This News Feed episode highlighted one of Facebook’s greatest vulnerabilities – users’ privacy concerns. With social networking sites such as Facebook, however, much of the value to users came from their substantial time investment in building a network and profile and being on the same platform as their family and friends. This “stickiness” reduced the risk of Facebook losing consumers even when they disagreed with some of its policies and practices.

Facebook went public in May 2012 with a valuation of more than \$100 billion, by far the largest in the IT sector at the time and the third largest in US history. Late in that year it acquired Instagram for approximately \$1 billion, and less than two years later, it took over WhatsApp for \$19 billion. These acquisitions proved strategic. Both firms excelled in mobile communications using smartphones in which Facebook had been lackluster.

Twitter became Facebook’s closest rival in scale, yet it has always had less than one-fifth as many users as Facebook. A San Francisco-based company founded by Jack Dorsey in 2006, Twitter provided a platform for “microblogs” – short text-based messages, known as tweets, of 140 characters or less, a maximum doubled to 280 in 2017. Some Twitter users, generally those of a younger age, developed the habit of microblogging their entire day – from the mundane (such as a trip to the grocery store) to the more meaningful (perhaps participation in a political rally). Smartphones facilitate wide-ranging access, allowing tweets to be a surrogate for person-to-person communication – letting others know where they are, what they are doing, or when they will be back. Other tweeters focus more on writing brief perspectives on politics or entertainment. Many celebrities use Twitter to communicate with their large bases of followers. Facebook and Twitter also have figured prominently in politics. Although at

times they have been heralded for their democratic possibilities, more often they have either reinforced centralized power, or, especially with regard to Facebook, threatened democracies and fair elections.

Journalists quickly dubbed the 2009 protests in Iran – alleging fraud in the re-election of President Mahmoud Ahmadinejad – the “Twitter Revolution.” User tweets reportedly contributed to spreading the word about the protests and to voicing the opinions of dissenting citizens. Closer analysis, however, showed that only a small percentage of Iranians used Twitter. Indeed, many tweets originated from westerners; early coverage of the events “revealed intense Western longing for a world where information technology is the liberator rather than the oppressor.”⁷ At least as important to this new form of protest was the older technology of cellphone texting, which also enabled information to rapidly spread. Moreover, some opposition organizers and protesters were hesitant to go on social networking sites, given the likelihood of retribution by authoritarian regimes.

Twitter and especially Facebook have faced strong criticism from governments, the media, and the public for their lack of proper controls. They have been accused of hindering democracy and fair elections in the United States and globally. For example, fake news on Facebook sponsored by Russia supported Donald Trump’s election campaign in 2016. In 2018 *The Observer* and the *New York Times* broke the news that a few years previously Facebook had made data available on over 80 million of its users to the British consultancy Cambridge Analytica to aid the campaigns of 2016 US presidential candidates Ted Cruz and Donald Trump. This resulted in the US Federal Trade Commission fining Facebook \$5 billion.

Amazon: E-Commerce and the Changing Face of Retail

Another key early web success was mail-order selling. Jeff Bezos’s Amazon.com established many of web commerce’s early practices (including the incorporation of *.com* in the firm’s name⁸). Bezos, a Princeton University graduate in electrical engineering and computer science, had enjoyed a brief career in financial services before deciding to set up a retail operation on the web, headquartered in Seattle, Washington. After some deliberation, he lighted on bookselling as being the most promising opportunity: there were relatively few major players in bookselling, and a virtual bookstore could offer an inventory far larger than any brick-and-mortar establishment. Launched in July 1995, Amazon would quickly become the leading online bookseller and, in time, the world’s top online retailer.

While profits at Amazon remained elusive for a half decade, Bezos and his team made this less a priority than sales volume. The larger goal was always greater scale and scope in e-commerce, and new capabilities and infrastructure for related businesses. Most significantly in terms of new businesses, its

infrastructure and expertise helped set up the cloud-based AWS Division, the world's leading cloud enterprise.

Amazon's scale contributed to the decline and failure of brick-and-mortar stores. One of the first casualties was Borders, a bookstore chain that started in Ann Arbor, Michigan, in 1971 and that grew steadily for three decades throughout the United States, and to a lesser degree in the United Kingdom, Australia, and other countries. During 2008–2009, it closed down overseas outlets and finally went bankrupt in 2011. Borders customer lists were sold to Barnes & Noble, a surviving bookstore chain that was also struggling with competition from Amazon. Roughly 21,000 Borders employees lost their jobs. Likewise, many independent new and used booksellers worldwide have shuttered due to competition from Amazon. The pattern was repeated with recorded music and videogame stores after Amazon began to offer CDs and videogames. In 2007 Amazon ventured into e-books by introducing the Kindle electronic book reader, a proprietary e-book platform. E-books became a significant part of Amazon's ecosystem, built e-book loyalty, and further hurt booksellers.

In fulfilling Bezos' expansive vision, in the early 2000s Amazon's categories of product offerings continuously expanded, and before long it seemed to sell nearly everything. It appeared that no retail enterprise, however large, was safe from Amazon or its mail-order competitors. Department stores were decimated (especially Sears, JCPenney, and Macy's in the United States). Other enterprises, however, adapted more successfully to the new retail environment. For example, "Big Box" retailers successfully transitioned to offer online ordering with same-day pickup in addition to conventional in-store sales. In the United States, these included Target, Costco, and Walmart, as well as furniture giant Ikea in Europe and beyond.

Amazon's tremendous financial success has come at a high price to the company's reputation and for those who work for it, both directly and indirectly. Amazon has developed its own brands and is now among the largest garment producers worldwide, relying on workers in the Global South often working under poor conditions for low pay. There have been strikes and protests against Amazon labor practices in Spain, Germany, Brazil, India, and other countries. Global labor, environment, and health rights organizations, including UNI Global Union and Oxfam, have highlighted Amazon's poor labor practices, which include neglecting worker safety and efforts to combat labor organization. Amazon has fiercely battled unionization and has been criticized by media and labor organizers for surveillance, privacy invasions, and intimidation of its workers.

The Gig Economy in Action: Uber and Airbnb

Platforms and mobile computing devices have also greatly impacted labor by making possible the gig economy. "Gig," a word long used by musicians,

comedians, and others to refer to an agreement for giving a performance, has come to be used broadly as a term for independent contractor work in the interactive web era. Gig workers, given they are not employees, generally have no benefits such as paid vacation, and in the United States, that means no healthcare insurance.

Fast-growth gig companies have been created on cloud platforms to connect gig workers and service providers to those seeking services. The most prominent have been the ride-sharing service Uber and the lodging service Airbnb. Much as e-commerce has challenged brick-and-mortar businesses, most gig work has challenged long-existing industries and practices.

Taxicabs have a long history of regulation. In London the 1635 Hackney Carriage Act legalized horse-drawn carriage ride service. Major revisions in 1654 and 1879 regulated carriages and drivers in terms of service, carriage condition, and safety. Hackney service was a precursor to the automobile taxi services that followed fast upon the invention of the motor vehicle in the 1890s. In 1897 Walter Bersey launched the London Electro Cab with an initial fleet of a dozen vehicles, called “Berseys.” In 1907, the London Motor Taxi was the first company to use the term “taxicab.” Taxis have been a mainstay in urban areas ever since. From the 1635, 1654, and 1879 Hackney Acts to the motor age and the UK Transportation Act of 1985 and Transport for London, regulation of ride service has been the norm for centuries, in the UK and globally. Uber has been the most direct competitive threat to taxis, by innovating around and skirting regulations that were centuries in the making.

Uber Technologies, Inc. (initially Ubercabs), was founded by Garrett Camp and Travis Kalanick in 2009, in San Francisco. They started with the Uber platform for a luxury car ride service, connecting drivers using their own vehicles with customers. The service was priced higher than taxis but lower than existing limousine services. Uber expanded to other cities in the United States and to Paris in late 2011. The following year Uber introduced UberX, priced far below taxicab providers. This became its flagship service. UberX had only modest requirements for drivers and vehicles: a car less than ten years old and one to three years of driving experience. Hiring data scientists in 2012, Uber refined its predictive capabilities and algorithms for the platform by setting prices to optimize supply and demand in real time. Over time, Uber captured about two-thirds of the global ride-sharing market, and well over 10 percent of the limousine ride service industry.

While gig work as an Uber driver offered flexibility, compensation was low when fuel, vehicle depreciation, maintenance, and lack of job benefits were taken into account. Critics have argued that Uber has exploited both gig and non-gig workers. Conventional taxi drivers have sometimes invested heavily in meeting regulatory requirements and licensing fees. In many cities around the world, a transferable right to operate a taxi is very expensive. In New York City, for example, between 2002 and 2014 the price of a taxi medallion rose

from \$200,000 to over \$1 million. Uber set prices at a loss, having great access to capital through venture funding, stock issuance, and debt. Market share was the foremost goal as its leaders eyed a potentially lucrative autonomous vehicle future.

In the UK, the Supreme Court ruled in October 2016 that Uber drivers were in fact employees and entitled to the legal minimum wage and paid vacations. While this represented a substantial blow to Uber in the UK, in many countries and cities, including in the United States, Uber has largely operated independently of existing regulations for taxis and taken advantage of weak oversight and laws protecting contractors.

Airbnb has been another successful giant in gig platforms. It was founded in 2008 by Brian Chesky, its CEO, and two others. Airbnb provided a platform to match those wanting to rent out properties short term with those seeking accommodation – which could be an apartment, a house, a room in a house, a cabin, or other property. For renters, the space offered and perhaps amenities, such as a full kitchen, were often a more attractive and economical option than a hotel. For owners there was income from otherwise idle bedrooms or properties.

Airbnb went public in 2020 with a \$47 billion valuation, comparable with that of Marriott International. Although Airbnb has been largely benign for owners and renters, it has had damaging effects on the wider economy – particularly outside the United States. In some cities it has hollowed out the hotel trade, elsewhere it has priced out long-term renters, and by creating a demand for rental property investments it has inflated property prices beyond the reach of local residents. Airbnb has also led to a largely unreported gig economy of cleaning and maintenance services.

Overall, Airbnb and Uber have been trailblazers for the fast growth, transformative firms made possible by mobile devices and platforms. Regulation has been modest to date. These enterprises have created fresh alternatives for consumers but have proved to be exploitative of labor and damaging to the wider economy.

The Wisdom of the Crowd: Britannica and Wikipedia

Web 2.0 added further momentum to well-established models of shared volunteerism in the development of both code and text content. There was already a long history of cooperative code development. This started with cooperation in the mainframe-era user groups of the 1950s, which set the stage for collaborations with Unix in the 1970s, the launch of the Free Software Movement in the 1980s, and Linux in the 1990s (which we discussed in Chapter 9). Collaboration and volunteerism with text content, at least in terms of major projects, came much later and are core elements of the transition to Web 2.0. The term “crowdsourcing” originated from two editors of *Wired*, as they saw how businesses and organizations were relying more and more on knowledge from online

communities. As a collaborative content development effort, nothing has had such a major international impact as the online, free encyclopedia Wikipedia – and with it the decline of traditional encyclopedias.

In Chapter 11, we learned how the *Encyclopedia Britannica*, and its 200-year tradition of printed volumes, was brought to its knees by Encarta and other low-cost encyclopedias on CD-ROM in the early 1990s. This forced *Encyclopedia Britannica*, under new ownership, to follow suit with an inexpensive CD-ROM version in 1996. In March 2012 the publisher announced that it would cease to print encyclopedias in order to focus entirely on its online version, *Britannica*.

The phrase “The Wisdom of the Crowd” came into popular use in the 1990s. It describes a process of information gathering by which differences of opinion tend to cancel one another out and converge to a consensus. This is, for example, the underlying principle of public opinion polls: instead of relying on the opinions of an “expert,” pollsters establish the collective view of ordinary individuals.

In 2008 *Britannica* had announced that it would begin to accept unsolicited user content, which, upon acceptance by *Britannica* editors, would be published on a dedicated portion of the *Britannica* website. Shifting to exist only online and accepting some content from users was indicative of *Britannica*’s acceptance of the wisdom of the crowd, its readers valuing interactivity, and the tacit acceptance of the Web 2.0 *Wikipedia* model, which had already become the world’s most popular encyclopedia.

Wikipedia (combining the Hawaiian term *wiki*, meaning “quick,” with *encyclopedia*) was launched by Jimmy Wales and Larry Sanger in 2001. It was based almost exclusively on volunteer labor and was at heart a platform for users to write, read, and edit. This model facilitated the very low-cost creation of a comprehensive encyclopedia in short order, sharply contrasting with the expensive, multi-decade efforts involved in producing print encyclopedias. In 2005 the British science journal *Nature* carried out a peer review of selected scientific articles from *Wikipedia* and *Britannica* and found that “the difference in accuracy was not particularly great,”⁹ lending credence to the value of user-generated content. As of this writing, *Wikipedia* has editions in more than three hundred languages and many millions of articles. It is one of the web’s most visited sites.

Streaming: Disrupting Media Delivery

Prior to the rapid growth of the web, modes of media delivery involved a combination of broadcast television and radio, cable, and video stores. The latter included mom-and-pop shops as well as the giant national chain Blockbuster, which rented VHS movies.

In 1997, Netflix, which was to become the leading streaming company, established a new form of media delivery: creating a website to turbocharge a direct mail DVD subscription service. If Hewlett Packard and Apple Computer were

two men in a garage, then Netflix was built by two men in a carpool. Reed Hastings and Marc Randolph were software professionals located in Silicon Valley. In 1997, their employer was in the throes of a corporate merger and their future roles were uncertain and they discussed possible start-up ideas in e-commerce. Given Amazon's early focus on books, they reasoned they could focus on supplying videos and movies. DVDs were just hitting the market at that time so that DVD rental and sales was an unexploited business opportunity. In 1997, they co-founded Netflix, and in 1998 they met with Jeff Bezos in Seattle, who offered \$12 million for Amazon to buy Netflix. Believing this too low, Hastings and Randolph instead gave Netflix's sales business to Amazon in exchange for Amazon's promotion of the Netflix rental business.

Because Netflix had a finite supply of DVDs for a particular movie, rental orders had to be held in a queue. In 2000 Netflix hired a small team of mathematicians to create a content recommendation algorithm, known as the Cinematch System, based on rental data, DVD availability, and user ratings to make suggestions to customers. These recommendations were important for creating demand and customer loyalty. To take things further in 2006 they held a \$1 million contest to attract top AI talent to design the best matching algorithm. Netflix's algorithmic-based suggestions heavily influenced customer choices – 70 percent of films added to customer queues were selected after appearing as suggestions.

In 2003 Netflix reached one million customers for its DVD subscription service. Its inventory held “ten times the DVDs”¹⁰ as the largest Blockbuster store. As early as 2001 Hastings had envisioned that a future based on streaming would be more convenient and a catalyst for subscriber growth. Moreover, the constraints of a finite stock of DVDs and delivery logistics would be bypassed, enabling a global expansion. The launch was strategically delayed until 2007, by which time content, broadband capabilities, and access to sufficient top titles ensured success. Streaming soon became Netflix's primary delivery mode.

Netflix's move to streaming coincided with a 2007 start-up streaming service, Hulu. This was a collaboration of several American TV networks and studios (including NBC, Universal, and Disney), and the initial focus was on TV series. Several years later, with a large catalog of TV series and movies, Hulu offered a subscription service called Hulu+. Netflix and Hulu+'s success inspired others possessing or acquiring content to offer streaming services, including Amazon Prime in 2011, Disney+ in 2019, and HBO+ in 2020. Recognizing the importance of original hit series to lure and retain subscribers – a model HBO had demonstrated earlier with cable subscribers – Netflix created the original series *Lilyhammer* in 2012, and the hit *House of Cards* the following year, with many other series and films to follow. At the time of writing, in terms of paid subscribers for streaming, Netflix is the global leader with 209 million global subscribers, nearly twice that of second place Disney+.

Blockchain: An Emergent Technology

In 1991 Stuart Haber and W. Scott Stornetta, two cryptographers at Bellcore – a joint lab of the seven “Baby Bells” (from AT&T’s 1982 divestiture) – published a seminal paper in the *Journal of Cryptography*, “How to Time Stamp a Digital Document.” This was not a reference to a stamp on the media (like a watermark), but instead, for using cryptography to time stamp the underlying data in an immutable way, assuring its integrity. The article anticipated both cryptocurrencies and Non-Fungible Tokens (NFTs). Cryptocurrencies are a form of digital cash. NFTs can be files of various types – digital art, videos, photos, music, and games.

Both cryptocurrencies and NFTs consist of units of cryptographically time-stamped digital data on decentralized, public computer networks. These networks have many nodes (for major cryptocurrencies over 10,000), and all nodes carry an up-to-date copy of all data. By grouping data in blocks and chronologically linking every block into a chain, called a blockchain, cryptocurrencies and NFTs guarantee data integrity. Tampering with data causes a major, easily recognizable change in all subsequent blocks visible at every node of the network.

In 2008 an anonymous person or group with the pseudonym Satoshi Nakamoto published an online article drawing on Haber and Stornetta’s paper, and other work, to invent Bitcoin, the first cryptocurrency. Bitcoin is an electronic cash system. Its key characteristics include its data integrity achieved through applied cryptography, its decentralization and community governance, its existence on a public blockchain ledger, its anonymity, and its “work” model. The work model requires intensive computation to “mine” a Bitcoin. Each additional Bitcoin requires ever greater computation to mine. There is ultimately a finite supply of coins, which ensures the stability of the currency. Bitcoin’s anonymity, an intended attribute, has been criticized for facilitating criminal activities. For example, it has been adopted by hackers in “ransomware” attacks – in which a computing facility is disabled until a ransom in cryptocurrency has been paid. The anonymity of cryptocurrency ensures the hackers cannot be traced.

Bitcoin and other cryptocurrencies have been in use for more than a decade and captured the public imagination. They have been used in real financial transactions and there are many exchanges where they can be traded. NFTs, on the other hand, have only much recently attracted media attention in the context of guaranteeing the uniqueness of digital artworks. The reason for the media attention has primarily been some spectacular auction sales prices realized. Whether or not cryptocurrency exchanges and NFT artworks turn out to be solid investments or speculative bubbles, only time will tell. One emerging application of blockchain technology is for “smart contracts,” which automate the execution and fulfilment of commercial agreements. At the time of writing blockchain is a technology whose applications have only just begun to unfold. Rather like the internet in the 1990s, blockchain’s future trajectory is unknowable.

Notes to Chapter 14

An impressive journalistic account of Google's first decade is Steven Levy's *Into the Plex: How Google Works and Shapes Our Lives* (2011). On the early years of Amazon and Yahoo, we found useful Robert Spector's *Amazon.com: Get Big Fast* (2000) and Karen Angel's *Inside Yahoo!* (2001). While somewhat celebratory, Andrew Lih's *The Wikipedia Revolution: How a Bunch of Nobodies Created the World's Greatest Encyclopedia* (2009) is informative. The early chronology on social networking draws in part from David Kirkpatrick's *The Facebook Effect: The Inside Story of the Company That Is Connecting the World* (2010).

An excellent treatment of the important topic of surveillance capitalism is Shoshana Zuboff's *The Age of Surveillance Capitalism: The Fight for a Human Future at the Frontier of Power* (2019). For a deeper historical take on surveillance and capitalism, the best work is Josh Lauer and Kenneth Lipartito's edited volume *Surveillance Capitalism in America* (2021). There are many books offering analysis of the political limitations and the social and psychological downsides of the internet and social networking. Among the best are Danielle Allen and Jennifer S. Light's edited volume *From Voice to Influence: Understanding Citizenship in the Digital Age* (2015), Evgeny Morozov's *The Net Delusion: The Dark Side of Internet Freedom* (2011), and Sherry Turkle's *Alone Together: Why We Expect More from Technology and Less from Each Other* (2011). Useful studies of race and inequity include Ruha Benjamin's *Race After Technology: Abolitionist Tools for the Jim Code* (2019) and Safiya Umoja Noble's *Algorithms of Oppression: How Search Engines Reinforce Racism* (2018).

Jeffrey R. Yost's *Making IT Work: A History of the Computer Services Industry* (2017) offers perspectives on the changing dynamics of the industry and its users, and the history of the cloud. For important context on fake news we recommend James W. Cortada and William F. Aspray's *Fake News Nation: The Long History of Lies and Misinterpretations in America* (2019).

Notes

1. Greenberger 1964, pp. 63–7.
2. Wells 1938, p. 60.
3. For a rich account of the dot.com bust, see Cassidy 2002.
4. Google Inc. *Annual Report* 2007, p. 36.
5. Noble 2018.
6. Kirkpatrick 2010, p. 275.
7. Morozov 2011, p. 5.
8. "Amazon" is the more commonly used name and known as such outside the United States where there is no country identifier in the URL.
9. Giles 2005, p. 900.
10. Keating 2012, p. 70.

15 Computing and Governance

OUR ABILITY TO SURF THE WEB, scroll through a newsfeed, and even buy a computer depends on several overlapping systems of law and policy. Governments at all levels are involved in creating and enforcing rules that shape some of the most important themes in the history of computing: how inventors transform raw materials into new components and devices, how firms can compete to sell or lease a product, and how end-users can acquire and use technologies new and old. Conversely, the spread of new technologies can trigger major changes in law and policy, sometimes generating unprecedented legal crises that take decades to resolve. Indeed, systems of governance – from legal and regulatory institutions to more informal mechanisms like industrial policies – play crucial, and often invisible, roles in shaping histories of technology.

Understanding the global history of the computer requires us to examine the deep ways in which computing technologies and systems of governance have affected each other. The relationships between the two cannot be encapsulated succinctly in a single explanation because they can vary from one country to the next, or even across different regions of a single country. However, there is one recurring pattern around the world: a government's sustained engagement with computing technologies can worsen and reconfigure the power imbalances embedded in its own institutions and procedures. This can occur at all levels, from small towns struggling to survive in a toxic environment to the international negotiations involved in delineating and enforcing property rights.

This chapter surveys the historical entanglements between governance and computing by illustrating six key lessons: (1) Energy consumption and material extraction and disposal have had permanent and devastating environmental impacts that are, at best, mitigated by local regulations. (2) The design and manufacturing of any electronic component is subject to labor laws that vary greatly from one country to the next. (3) The highly competitive nature of the industry pushes firms to seek powerful legal means to preclude their competitors from gaining an advantage. (4) National governments sometimes step in to recalibrate the power dynamics of the industry. (5) User-generated content doesn't flow freely into users' feeds because it is subject to censorship and moderation

enforced by governments and online platforms. (6) Individual privacy in the online world has been eroding at an alarming rate since 2001.

Computing and the Environment

In 2012, for the first time, Google allowed a reporter to visit one of its data centers. The lucky journalist was Steven Levy, best known among historians of computing for his 1984 book, *Hackers: Heroes of the Computer Revolution*. Levy, now a senior reporter at *Wired*, described what he called “the beating heart of the digital age – a physical location where the scope, grandeur, and geekiness of the kingdom of bits become manifest.”¹ This data center is based in Lenoir, North Carolina, a rural city that was once best known for its numerous furniture factories. Google had several others around the world, from Council Bluffs, Iowa, to St. Ghislain, Belgium, and the company was planning to open additional servers in Hong Kong and Singapore. The servers were, in Levy’s view, “what makes Google Google.” They enable it to index billions of pages per day, manage nearly half a billion “Gmail” e-mail accounts, and stream millions of YouTube videos at the same time.

Data centers can leave a massive environmental footprint. Many of the ones at Silicon Valley have appeared on California’s Toxic Air Contaminant Inventory (a list of diesel pollutants). By one estimate, it would take about 30 nuclear power plants to meet the worldwide electricity demand of data centers, and up to one-third of that usage takes place in the United States alone. At the *New York Times*’s request, the management consultants McKinsey & Company sampled 20,000 servers across a wide range of industries (from military contracting to banks and government agencies) and found that at most 12 percent of this energy is used to perform computations. The rest of it, alongside the energy stored in thousands of high-capacity batteries, simply keeps servers in an idle state in case they’re needed to prevent an operation slowdown.² This energy usage, in turn, generates a lot of heat, so data centers require large amounts of water to cool off their machinery. One National Security Agency data center in Utah, for example, uses 1.7 million gallons of water per day.

The history of computing encompasses a rich and troubling environmental history, and not just because the digital world unfolds at physical spaces like these server farms. For decades, electronic circuit board manufacturing in Endicott, New York (home of IBM’s first manufacturing plant), has transformed the area into a toxic landscape. An area known as “the plume” comprises 300 acres contaminated with TCE, a carcinogen that IBM uses in its cleaning products. Local advocacy groups have tried, and failed, to restore the town’s environmental health; they have been far more successful in advocating for the health of former plant employees, who were exposed to the highest concentration of TCE. With a town in decline, Endicott residents continue to be exposed to poisonous vapors, and their property values have plummeted in no small part because of the stigma associated with living in a toxic town.

The computing industry's impact on the environment is intensifying and permanent. The internet itself depends on a vast infrastructure of servers and exchange hubs, and on a global network of undersea cables that connect them to each other. Manufacturing all the devices in these servers, and those connected to them, requires the extraction and refinement of very rare metals such as lanthanum, cerium, and dysprosium. Mining for these metals can be environmentally disastrous, as is the globally unregulated disposal of obsolete devices at massive e-waste dumpsites such as the ones in Agbogbloshie, Ghana, and Guiyu, China. In fact, every stage of our engagement with the digital world has an environmental consequence: from the acquisition and processing of raw materials for new devices, to the heat and waste produced by servers and the toxic fumes produced when people at e-waste dumpsites burn devices to extract and recycle rare metals.

High energy consumption and its associated heat production also occur outside massive data centers. This is most evident in the case of the Bitcoin digital currency, which one scholar has called an "ill-conceived, entirely unnecessary, environmental disaster."³ Recent studies suggest that the Bitcoin network uses much more processing power than the world's top 500 supercomputers combined – more than many nation-states. Bitcoin, which can be acquired and processed by having computers solve increasingly challenging computational puzzles, is most often "mined" in places where electricity is cheap thanks to the nearby presence of a hydroelectric or nuclear plant. In addition, Bitcoin mining equipment has a very short lifespan, averaging 1.29 years according to a recent study. This quick obsolescence of hardware is generating more than 30 metric kilotons of equipment waste per year.

Processing power has increased at a rate much higher than environmental pollution, but there is a stunning lack of environmental law and regulation applicable to data centers. The European Commission has called for higher energy efficiency in data centers and telecommunications and expects them to become climate neutral by 2030. In the United States, regulations are emerging at the state and local levels. For instance, in 2018, the city of Plattsburgh, in New York, banned Bitcoin mining. Still, major policy and legal work is required to apply environmental standards to data centers and regulate energy consumption, heat production, and waste disposal and recycling at the global level.

Labor and the IT Industry

For decades, nonprofit organizations and specialized industry groups have advocated for the professional and personal needs of women and members of underrepresented groups in the computing industry. They include Black Data Processing Associates, the American Indian Science and Engineering Society, and the National Center for Women and Information Technology. Groups like these are essential to counter the gender and racial biases that have characterized

the history of computing labor. The British government's case is particularly striking: the thousands of women who had been essential information technology workers during World War II were treated as sub-clerical workers after the war and subjected to subpar work conditions. They had no access to pensions and could be fired soon after getting married – all while sexist advertising and a new high-profile image for programming labor coded computing as masculine endeavors. In other words, the problem was not that there were no women in the talent pipeline for computing work, but that the women who were ready to perform such work were systematically excluded in favor of their male counterparts.

The demographics of computing labor in more recent years show that exclusionary hiring and management practices have taken a toll on the diversity of the computing industry. The history of this problem at today's digital giants is difficult to investigate because large firms have historically refused to publish demographic data, and some of them claim this data to be trade secrets. However, recent changes to these policies are starting to yield interesting data. For example, in 2014, the race and gender breakdown of Google's nearly 50,000 employees was the following: 83 percent male, 60 percent white, and 30 percent Asian. Only 1.9 percent were Black, and 2.9 percent were Latino.⁴

The percentages of computer science and engineer graduates from underrepresented groups were much higher than this, which suggests that there continues to be a barrier for these professionals to become employees at the company. Several anecdotes by people of color in the industry – brought to the public eye in part by lawsuits filed in the past decade and a half – report daily microaggressions, unfair targeting by colleagues and supervisors, unfair amounts and types of work assignments, and even barriers to enter the companies' buildings.⁵ It is important to note, however, that this situation is not unique to the computing industry. In a 2017 survey of professionals with so-called STEM (science, technology, engineering, and mathematics) jobs, Pew Research found that 50 percent of women reported having experienced discrimination at work (this percentage rose to 74 percent in computing). The percentages for Black, Hispanic, and Asian respondents who experienced discrimination in the general survey were, respectively, 62 percent, 42 percent, and 44 percent.⁶

Working as a computer engineer or programmer in a wealthy country can be personally satisfying and highly compensated, but the computing industry also has a long history of reducing labor and production costs by displacing labor into regions with lower wages. This could take place within a company's own country. In 1965, for example, Fairchild Semiconductor collaborated with Navajo leaders and the Bureau of Indian Affairs to open a semiconductor assembly plant on a Navajo reservation in New Mexico. The company celebrated the reservation's women as having distinctly nimble fingers and the ability to manufacture products quickly and efficiently. Over the next decade, the factory grew into a 33,000 square foot manufacturing facility employing hundreds of Navajo women. Reservations provided firms with legal exceptions to minimum wage

laws. The company shut down the factory in 1975 citing an unstable labor environment, after members of the American Indian Movement took it over.

By the end of the twentieth century, as Chapter 13 has shown, the dominant mode of displacing labor and reducing production costs was outsourcing. The remarkable story of India's information technology industry offers an example of how companies built on preexisting scientific and educational cultures began to outsource their labor. Starting in the mid-twentieth century, electronics firms established production and services facilities that aligned well with India's long-term efforts to achieve national self-sufficiency. A combination of state incentives such as tax law exceptions, educational systems that valued technical training, and telecommunications infrastructures helped to establish a vibrant IT industry. Indeed, the value of India's services and software exports grew from \$100 million in 1990 to almost \$100 billion by the mid-2010s.

The growth of outsourcing to East and South Asia has created deep socioeconomic contradictions. On the one hand, the promise of higher wages and a modern lifestyle has motivated millions of people to migrate from rural areas into industrial districts in a range of industries. In China, for example, the "Great Migration" involved the movement of more than 260 million people over the course of thirty years starting in the early 1980s. On the other hand, the working conditions that many of these migrants meet can be appalling. In 2015, workers at Foxconn Technology, a Taiwanese firm with factories in China, could earn as little as \$1 an hour for nearly 300 hours of work per month (including more than 100 hours of overtime).⁷ This ran afoul of Chinese labor laws, which technically allowed for no more than 36 hours of overtime. Workers reported living in overcrowded dormitories that afforded them little privacy, and at least a dozen of them committed suicide in 2010. One anthropologist even noted that workers who migrated into industrial districts quickly discovered that the internet was the only place where they could experience the modern lifestyle that had inspired them to leave their homes.⁸

Computing and Intellectual Property

The term "intellectual property" (IP – pronounced eye-pee) refers to a collection of exclusive rights that creators have over their works. For instance, patents grant inventors the right to exclude others from making, selling, or using their inventions. Copyrights offer protections for creative works, enabling the makers of things like photographs, novels, and music the right to exclude others from duplicating, displaying, or (if applicable) performing their creations. Trademarks cover commercial marks such as corporate logos, and trade secrets allow individuals and companies to protect commercially valuable secret knowledge from theft. There are additional forms of IP protection in addition to these four major ones, including what are called *sui generis* protections – that is, special protections crafted for a specific kind of work (including databases and microchips).

Since the eighteenth century, IP rights have been essential to the development of innovation and the creative industries around the world. Economic historians have demonstrated that patent systems have been more effective at incentivizing innovation and economic growth than systems of prizes and privileges. Legal historians have shown how modern patents – which require full public disclosure of an invention in exchange for exclusivity – have helped to forge the relationships among governments, markets, and inventors central to industrial growth since the Second Industrial Revolution.

This interrelationship among ownership, governance, and innovation is also central to the long history of information and telecommunications. To name a few early examples: Alexander Graham Bell's telephone was the subject of intense patent wars with the potential to shift market power in the early telephone industry, and Samuel Morse considered patents to be such a valuable means to control early telegraphy that he fought all the way to the Supreme Court to establish control of the industry. In the nineteenth century, automatic piano makers considered patents over piano rolls (which encoded songs into punched paper), and IBM was able to survive the Great Depression of the 1930s in large part because of the revenue generated by its patented punched cards.

The development of modern computing has challenged some of the foundational distinctions that characterize IP law. For instance, in the United States, there is a long-standing ban on patenting what are known as mental steps – steps, usually computations, that a person with proper training can perform in their minds. Courts could interpret this prohibition as implying that software was ineligible for patent protection. In the decade after World War II, however, industrial pressure to secure patents over a computer's programming prompted lawyers to patent a computer's programming as a collection of electrical circuits. This maneuver, born at Bell Labs in the late 1940s, quickly spread across industrial research laboratories and the hardware industry. By the late 1960s, young software firms looking for ways of competing against IBM employed these ideas to secure their own patents.

This maneuver – disclosing a program as a machine, also known in the 1960s as “embodying software” – has also been especially useful in the United Kingdom and Europe. British, French, and German companies adopted some version of software embodiment at some point in the second half of the twentieth century. The resulting patents were not confined to the computer industry. On the contrary, weapons manufacturing, industrial research laboratories, oil firms, telecommunications companies, banks, and automobile makers all obtained some kind of patent protection for the programs that controlled their factories and products. This made software patenting into a valuable legal and commercial tool in any industry where computer programs could offer firms a competitive advantage.

Despite these relatively uncontroversial origins, the patent protection of software came under very high scrutiny in the 1970s. Hardware manufacturers, led

by IBM, campaigned aggressively against software patents, in part to avoid having small firms use patents to fence off important new areas of computer development. In the United States, this opposition became especially prominent at the courts. In 1972 the Supreme Court opinion of a case known as *Gottschalk v. Benson* (1972) ruled that a numerical algorithm cannot be patented because “the patent would wholly pre-empt the mathematical formula and in practical effect would be a patent on the algorithm itself.”⁹ The next year, the European Patent Convention explicitly listed “programs for computers” as subject matter ineligible for patent protection, as did the United Kingdom’s Patents Act of 1977.

During this time, software embodiment provided some relief to firms interested in securing patents for computer programs, but for the most part developers thought of copyrights as more stable protections for software. Copyrights were valuable, especially during the 1980s and 1990s, in worldwide efforts to curb the unauthorized distribution of software and equipment. In the United States, for example, the Court of Appeals for the Third Circuit sided with Apple by ruling that a manufacturer of Apple-compatible devices (Franklin Computer) had violated its copyrights by distributing operating systems in microchip form. The same year, 1983, Apple won a similar battle in New South Wales, where a court ruled that a company called Computer Edge was importing copyright-infringing computers from Taiwan. Both cases were part of broader efforts to curb the global spread of cloned computers, which at some points could even involve collaboration between companies and national investigative bodies like the FBI. These efforts were grounded partially in laws that established the copyright eligibility of software. These include the Computer Software Copyright Act of 1980 in the US and the UK’s Copyrights, Design, and Patents Act of 1988.

Software developers’ relationships with IP took a sharp turn in the late 1990s, when it became clear that computers had become central components in virtually every manufacturing and service industry worldwide. In this increasingly digitalized industrial landscape, computer programs became valuable as components of a broader computerized system that controlled the production of a good or delivery of a service. This led to a readoption of software embodiment techniques for patent protection. For example, in 1999, the US Court of Appeals of the Federal Circuit ruled that an invention is patent eligible if it produces “a useful, concrete, and tangible result,” regardless of whether it involves computer programs.¹⁰ A few months earlier, the European Patent Office Board of Appeals issued an opinion stating that programs “must be considered patentable inventions when they have a technical character”¹¹ – an extraordinarily broad endorsement of software patenting. The United Kingdom adopted a much stricter approach, requiring not just a technical character, but a constellation of issues such as whether the program has an effect beyond the computer on which it runs, or whether the program improves the performance of said computer.

By the 2010s, major players in the computing industry were united in their support of software patenting and its limitations but deeply divided over the

reasons why certain software inventions should fall outside the scope of patent protection. This became clear in the 2014 US Supreme Court case of *Alice v. CLS*, which involved patents over a computerized financial transaction system. Google and Amazon started to advocate for patent rights that did not extend beyond the specific working computing system that an inventor had created. Adobe, Microsoft, and Hewlett Packard insisted that such an idea does not become patentable just because a computer is involved. At the same time advocates such as the Electronic Frontier Foundation, the Free Software Foundation, and several scholarly organizations ranged in their views from complete opposition to software patenting in all its forms to unbridled support to the strongest patent rights. The Court ultimately invalidated the patent involved, moving the United States to a stricter patenting regime.

The unstable landscape of software patenting has made copyrights more valuable than ever. Starting in the late 1990s, countries affiliated with the World Intellectual Property Organization (WIPO) started passing copyright laws designed to strengthen this form of IP's reach into the online world in alignment with international treaties. Their laws generally implemented two treaties. First, the WIPO Copyright Treaty of 1996 explicitly establishes that "computer programs, whatever the mode or form of their expression," are eligible for copyrights, as are "compilations of data or other material ('databases'), in any form."¹² It also prohibited the use of technological means to circumvent technical protections against piracy and duplication. Second, the WIPO Performances and Phonograms Treaty (which affirmed the economic rights of performers and producers) was created to curb the massive online distribution of digital media.

Unfair Competition

Antitrust law – the field of law meant to preclude firms from engaging in anti-competitive behavior – has been central to the history of computing. The United States was once the only nation with robust antitrust legislation, but by the 1980s Japan and several nations in Europe had developed their own laws resembling the American ones. This section focuses on the American story because it illustrates four major ways in which antitrust law has impacted the computing industry: it enabled the birth and growth of small firms eager to take on industry giants; it weakened the patent rights of IBM and AT&T; it facilitated the creation of new markets for services and used products; and it enabled the emergence of software as a distinct commercial entity.

The US Department of Justice's Antitrust Division has repeatedly scrutinized competition in the computing industry, weakening the competitive advantages that firms can have due to their large size, business practices, and the timing and popularity of their products. For example, the extraordinary resilience of IBM during the Great Depression in the 1930s, which Chapter 2 documented, triggered antitrust investigations aimed at the firm's use of punched cards. Sold

in packs of 1,000 for \$1.05, these cards generated about \$3.8 million in yearly revenue by the middle of the decade, not in the least because IBM required clients to use only the firm's own patented cards. In court, IBM insisted that this requirement was essential for quality control, but the United States Supreme Court ruled that the firm was engaging in an illegal "tie-in" – that is, a sale made conditional on the purchase of additional products from the same supplier. IBM was forced to allow customers to use any cards they wished.

This early encounter illustrates one of the major themes in the computing industry's relationship with antitrust law: limiting a firm's ability to quench the creation and growth of an industrial subsector. The end of IBM's punched card and equipment tie-in enabled punched-card manufacturers to compete by producing IBM-compatible cards. As briefly described in Chapter 5, another major encounter with the Antitrust Division, in the 1950s, forced the company to offer its machines for sale, ending a decades-long practice of offering them exclusively for lease. It is important to note, however, that the computing industry was not the only one in the Antitrust Division's sights. On the contrary, the Division had embarked on a program of antitrust enforcement so strong across all industries that commentators deemed the 1950s the New Sherman Era, in reference to the landmark Sherman Antitrust Act of 1890.

A crucial component of antitrust enforcement was weakening patent rights for large firms, including IBM and AT&T. At IBM, this took the form of an agreement by which, in exchange for a royalty payment, it would allow other firms to use its patents for punched cards, tabulators, and computers. The agreement also allowed an applicant for a license to reject IBM's proposed royalties and established an appeals system by which the applicant could turn to a federal court in order to demand proof that the royalties were reasonable. These developments all positioned IBM as the most vocal opponent of software patenting in the world, as discussed earlier in this chapter. At the same time, however, it created the legal environment necessary for IBM specifications to become industry standards and for firms of all sizes to develop and sell IBM-compatible systems.

By the 1960s, it was clear that the Antitrust Division was taking aim at IBM's historical production and marketing of "bundled" systems instead of individual components. This reached its height toward the end of the decade, when the Antitrust Division inquired into IBM's bundles of hardware, software, and services. IBM had been considering whether its bundles should be dissolved since the start of the decade and consumers were starting to see bundles as paying for services and programs they would never need. As described in Chapter 8, in the context of software products, software makers led by Applied Data Research repeatedly complained that it was very difficult for them to compete with a giant that could offer programs free of charge. IBM's unbundling can be seen as a turning point in the history of computing, brought about by the firm's effort to cope with long-standing legal tensions, a shift in consumer opinion, and a cohort of eager entrepreneurs looking for a way to compete with industry giants.

The United States dropped several antitrust suits (including IBM's) during the first years of the Reagan Administration, and antitrust enforcement was generally weaker until the Clinton Administration. In the 1990s, the personal computing and home networking industries grew alongside a newly invigorated program of antitrust enforcement that, according to legal commentators, tended to focus more on controlling long-term competitive dynamics than on short-term issues such as pricing. The best-known lawsuit from this time, against Microsoft, targeted two aspects of its business practices. First, requiring Windows operating system licensees to pay a fee for each processor sold regardless of whether or not that processor used the operating system. And, second, Microsoft's incorporation of the Internet Explorer browser into its Windows operating system at no additional cost. In 2001, the firm ultimately agreed to abandon these practices. (As described in Chapter 12, this came all too late for Netscape browser, the main casualty of Microsoft's unfair competition.) After the antitrust ruling, licensees would pay only for copies of Windows actually delivered, and Microsoft would share its "application programming interfaces" so that competitors could develop compatible software with Windows and Internet Explorer.

Content Moderation and Online Speech

Social media companies' turbulent relationships with regulators and lawmakers extend far beyond questions regarding their size and business strategies. Platforms have become essential tools in political speech and organizing. During the Egyptian Revolution in 2011, for example, activists used Twitter to organize the delivery of medical services around Cairo. A decade later, in the aftermath of the Capitol Riots in Washington DC that took place on 6 January 2021, Twitter banned former President Donald Trump on the grounds that he had used the platform to incite violence. In Zimbabwe, where homosexuality is punished severely, LGBTQIA people use Facebook to connect with one another and create a sense of community to counter the hostile political environment around them.

These examples illustrate a broader problem in internet governance: determining who can censor online speech and under what conditions they can do so. On the one hand, content censorship and moderation can unfold at the corporate level. Social media companies develop and implement their own rules based on several factors, including managers' own values, well-established media priorities, cost-benefit analyses of enforcement procedures, and broader national and international concerns. Bans on content such as terrorism threats, pornography of various kinds, animal cruelty, and human exploitation are common across social media. This moderation is often human in nature; by 2019 there were thousands of content moderators evaluating social media posts. They actively screened platforms for unacceptable material (often at great detriment to their own mental health) and helped to train the AI systems that would eventually

replace them. These moderators are part of a sociotechnical system that directs what users can and can't discuss online.

On the other hand, censorship can also occur in the hands of the state. For example, the Chinese state has mobilized law, media, and internet infrastructures to limit the online influx of foreign information. Its Great Firewall blocks access to major platforms and perpetuates the development of an online public sphere that the government continually moderates. Since 2019, the Russian government has imposed its own censorship programs by routing web traffic through the Roskomnadzor, a state communications agency with the capacity to slow down or even block the flow of information. Political commentators have feared that this is the first in Vladimir Putin's systematic erosion of digital life, designed to isolate citizens, create obstacles to political organization, and force popular platforms to restrict content on the state's behalf.

In countries where state agencies do not enforce this extreme level of censorship and moderation, private efforts to moderate online content run parallel to rich and often mutually incompatible legal frameworks regarding freedom of expression. In the United States, freedom of speech is guaranteed by the First Amendment to the US Constitution. This Amendment establishes, among other things, that "Congress shall make no law . . . abridging the freedom of speech, or of the press." Courts' broad interpretation of this amendment has created a robust body of law that, at least in theory, upholds the free exchange of ideas as a fundamental American value.

The European approach is different in that individual countries have a history of explicitly outlawing several kinds of speech. This different approach is tied to the fact that the free circulation of Nazi rhetoric caused countless deaths and immense suffering. Germany and France both have laws banning speech intended to deny the holocaust. In the UK, the Race Relations Act of 1965 banned racially hateful threats, insults, and abusive statements. More generally, Article 10 in the European Convention of Human Rights (1950) establishes a right to the freedom of expression, including "freedom to hold opinions and to receive and impart information and ideas without interference by public authority and regardless of frontiers." However, it also notes that this right can be restricted for reasons such as protecting national security interests, maintaining territorial integrity or public safety, preventing crime or disorder, protecting the reputation or rights of others, and even the "protection of health or morals."

Until the arrival of the internet, these two contrasting frameworks for free expression did not frequently come into conflict with one another. Content could travel across the Atlantic, but a system of media intermediaries (print publishers, satellite service providers, television broadcasters) often had the capacity to filter content. Even satellite television did not cause major issues in free expression: although legally objectionable content could travel across the EU in the form of satellite signals, state regulators could simply block it by prohibiting users' decoders from processing the signal. In the UK, this is enabled by the

Broadcasting Act of 1990, which makes it illegal to sell decoders or to publish information about channels that commit repeat violations and receive a proscription order.

Censoring US online content in the EU and UK can be much more difficult, in part because it raises difficult jurisdictional questions. Consider the 2000 French case of *LICRA & UEJF v. Yahoo! Inc. and Yahoo! France*. The League Against Racism and Anti-Semitism (LICRA) and the Union of French Jewish Students (UEJF) had noticed that *Yahoo! US* was hosting an auction of Nazi memorabilia including copies of Adolf Hitler's *Mein Kampf*. French users could access this auction through *Yahoo! France*, which provided a link to it. However, Article R645-1 of the French Criminal Code prohibits exhibiting or wearing any insignias or emblems similar to those declared illegal under the Nuremberg Charter or by people found guilty of crimes against humanity.

LICRA and UEJF filed a lawsuit in France against *Yahoo! Inc.* and *Yahoo! France*, and the Tribunal de Grande Instance de Paris agreed that the auctions ran afoul of the French Criminal Code. This was the beginning of a complex legal battle that included six judgments spread over six years, in large part over whether a French Tribunal had the capacity to regulate content hosted by US servers. This jurisdictional battle could therefore have a long-lasting impact on international internet regulation. Was freedom of expression to be governed under the laws at the location of the servers hosting the content, the location of the content's recipients, or both? The opinions slowly developed a view of online content grounded on the notion that the content was being simultaneously broadcast in the United States and France: French law could not reach US servers, but it could certainly regulate how the content was displayed in France. This meant, among other things, that *Yahoo! France* would need to display warnings on the auction sites stating that the content was illegal under French law.

Privacy, Security, and Surveillance

For twenty years, data protection in the EU was grounded on the Data Protection Act of 1998. The Act defines "data" very broadly, encompassing recorded information that forms part of indexed collections of any kind: from digital collections for use in a computerized system to manually recorded information used in a filing system. The Act applies to all data subjects, by which it means people within the EU whose data is being processed. It establishes that a data subject's personal data may only be processed with the subject's consent, unless there is a compelling reason (including compliance with legal obligations, the administration of justice, or the exercise of public functions). The Act also offers special protections to what it identifies as sensitive personal data, which includes information about a person's race, ethnicity, political opinions, health, sexuality, and trade union membership. Individuals must grant explicit consent to have this data processed, even if they have already agreed to the processing of their

personal data. Compelling reasons to bypass this consent include medical purposes and evaluations of equality of opportunity.

This approach is much stricter than the one employed by the United States, where no laws ensure this breadth and strength of personal data protection. This is a problem because the 1998 Act specifies that personal data cannot be transferred to a country outside of the EU unless that country can ensure a similar level of protection for personal data. To prevent slowing down or blocking the flow of data between the United States and Europe, the US Department of Commerce and the EU Commission created a safe harbor agreement: US companies may self-certify that they meet EU standards. Without this safe harbor, it would have been impossible for today's digital giants to transfer data between the United States and the EU.

In the United States, the terrorist attacks of 11 September 2001 triggered a very powerful legislative and political program of national security that eroded privacy rights in the name of fostering national security. Within six weeks of the attacks, Congress passed a new law called the USA PATRIOT Act, a contrived acronym for "Uniting and Strengthening America by Providing Appropriate Tools Required to Intercept and Obstruct Terrorism." The Patriot Act allowed the US government, essentially, to spy on anybody within its borders, even if that meant bypassing rights to privacy, due process, and equal protection guaranteed by the US constitution. Under the Patriot Act, the FBI could obtain a wealth of personal information, including business records, medical documentation, and educational records. President George W. Bush also launched efforts such as the President's Surveillance Program, which strengthened the National Security Agency's surveillance powers.

This is all to say that while the EU was strengthening its data subjects' privacy rights, the United States was developing a powerful surveillance program that challenged the assumption that safe harbor provisions between the two regions could be maintained without exposing EU data subjects to the kinds of violations that the 1998 Act was designed to prevent. The full breadth and depth of this surveillance state continues to be a secret. However, in 2013, Edward Snowden (a former consultant for the US National Security Agency) released a cache of top-secret documents he had obtained while working for a US-based defense and intelligence contractor. These documents contained details of several programs and surveillance technologies that the general public did not know about. Key among them was a 2007 US program called Prism, which involved collecting data from major companies, including Google, Microsoft, Facebook, Skype, YouTube (before it became a part of Google), and Apple. This data collection would occur when information traveled through US servers, regardless of its point of origin or destination.

Snowden's cache revealed that state-sponsored secret surveillance programs on both sides of the Atlantic were far stronger than anyone could have imagined. The US National Security Agency was intercepting and collecting telephone and

internet data through a program called Upstream. In the UK, the Government Communications Headquarters was extracting internet communications from fiber optic cables through a program called Tempora. Both governments also had their own programs to crack encrypted online communications (Bullrun in the US and Edgahill in the UK). Snowden's disclosures triggered an international media frenzy, and Snowden – who now resides permanently in Russia – has become one of the best-known free speech and surveillance advocates in the world.

These revelations (and especially the details about the US Prism program) sparked a turning point in EU-US data relationships. European privacy activists quickly mobilized against the safe harbor agreements, which they viewed as an affront to European privacy law and the values encoded into it. Led by Maximilian Schrems, an Austrian privacy activist who had spent some time studying in Silicon Valley, this activist effort involved several lawsuits launched through Schrems's *Europe v. Facebook* project. The Court of Justice of the European Union ruled in 2015 that national supervisory authorities have the power to examine EU-US data transfers and, more importantly, that the safe harbor agreement is invalid. This decision sent shockwaves around the privacy policy world, as law and policymakers scrambled to develop an alternative that would allow the uninterrupted flow of data between the United States and the EU.

Over the next few years, the United States and the EU tried – and failed – to develop a sustainable framework for data privacy. Chief among them was Privacy Shield, a program administered by the US Department of Commerce and the International Trade Administration. This program allowed companies to receive a certification stating that they had sufficient privacy protection to handle personal data from the EU. Interested companies needed to verify their alignment with seven principles: notice; choice; accountability for onward transfer; security; data integrity and purpose limitation; access; and recourse, enforcement, and liability. Thanks in part to activists like Schrems, the Court of Justice of the European Union struck down the proposed framework in 2020 for failing to protect EU citizens from government surveillance.

Starting in 2018, data protection in the European Economic Area was governed by the General Data Protection Regulation (GDPR). (The European Economic Area includes trading partners in the European free trade area, in addition to members of the EU.) This binding regulation creates a unified framework for data collection across the EU, ensuring that non-EU companies must comply with EU data protection standards when they provide services to EU citizens and requiring EU-based companies to be supervised at the local level to ensure compliance. Like the earlier laws, the Regulation calls individuals “data subjects” and grants them several rights. These include the right to access their personal data, to have their data erased, and to object to personal data processing for purposes other than the provision of services. The Regulations also apply to

controllers and processors outside of the Economic European Area if they offer goods or services within that area.

The relationship between information technology and privacy law is continually in flux. In the United States, the Patriot Act expired in 2020, after the House of Representatives failed to extend the few remaining provisions of the Act that the Obama administration had extended until 2019. At the time of writing, across the EU, privacy legislation is being revised, most likely to add requirements in addition to the GDPR. However, if Snowden's cache showed anything, it's that state-sponsored surveillance programs developed in the name of national security can unfold alongside, and deteriorate, privacy protections.

THE HISTORY OF COMPUTING is inseparable from the history of governance. Courts, government agencies, and industry groups periodically reconfigure how computing firms make and sell their products. This in turn determines what users can do with new technologies and the social and environmental dimensions of technological change. At the same time, new technological developments and business strategies in the computing industry can prompt changes in law, policy, and standards. However, computing and governance do not necessarily shape each other directly. Instead, they can impact one another by triggering a chain reaction of transformations in the broader industrial, commercial, political, environmental, and social contexts that connect them. For these reasons, the history of the ubiquitous information machine is essential to understand the history of how we govern the world.

Notes to Chapter 15

Excellent sources on the environmental history of electronics manufacturing and use are Nathan Ensmenger's "The Environmental History of Computing" (2019) and "The Cloud is a Factory" (2021). Peter Little's *Toxic Town* (2012) describes the shocking impact of IBM's manufacturing operations. Useful introductions to the topic from the perspective of science and technology studies are David Pellow and Lisa Sun-Hee Park's *The Silicon Valley of Dreams* (2002) and Jennifer Gabrys' *Digital Rubbish* (2011).

Mar Hicks's *Programmed Inequality* (2018) investigates the British labor history of the computing industry. Lisa Nakamura's "Indigenous Circuits" (2014) analyzes Fairchild's exploitation of Navajo women. William F. Aspray's *Women and Underrepresented Minorities in Computing* (2016) and *Participation in Computing* (2016) investigate the institutional and participatory politics of educational pipelines into the computing industry. Silvia Lindtner's *Prototype Nation* (2020) and Dinesh Sharma's *The Outsourcer* (2015) investigate East and South Asian electronics manufacturing, while Xinyuan Wang's *Social Media in Industrial China* (2016) examines everyday life in a Chinese industrial district.

Economists and legal scholars have written an enormous body of work on intellectual property and antitrust law in the computing industry. Gerardo Con Díaz's *Software Rights* (2019) investigates the relationships among technology, business, and law (especially intellectual property) in the US computing industry. Steve Usselman's essays, "Public Policies, Private Platforms" (2004) and "Unbundling IBM" (2009) are essential reading on computing and antitrust.

Zeynep Tufekci's *Twitter and Tear Gas* (2018), Ya-Wen Lei's *The Contentious Public Sphere* (2017), and Bruce Mutsvairo's *Digital Activism in the Social Media Era* (2016) address the uses and politics of online platforms. Sarah Roberts' *Behind the Screen* (2019) and Tarleton Gillespie's *Custodians of the Internet* (2021) address content moderation and censorship. Andrew Murray's *Information Technology Law* (2019) and Jeff Kosseff's *Cybersecurity Law* (2022) offer foundational legal perspectives on privacy, security, and online speech.

Notes

1. Levy, Steven. 2012. "Google Throws Open Doors to Its Top-Secret Data Center." *Wired*, 17 October. <https://www.wired.com/2012/10/ff-inside-google-data-center/>
2. Glanz, James. 2012. "Power, Pollution, and the Internet." *New York Times*, 22 September. <https://www.nytimes.com/2012/09/23/technology/data-centers-waste-vast-amounts-of-energy-belying-industry-image.html>
3. Ensmenger 2018, p. S26.
4. Dean, Sam and Johana Bhuiyan. 2020. "Why Are Black and Latino People Still Kept Out of the Tech Industry?" *Los Angeles Times*, 24 June. <https://www.latimes.com/business/technology/story/2020-06-24/tech-started-publicly-taking-lack-of-diversity-seriously-in-2014-why-has-so-little-changed-for-black-workers>
5. Bhuiyan, Sam Dean, and Suhauna Hussain Black. 2020. "Brown Tech Workers Share Their Experiences of Racism on the Job." *Los Angeles Times*, 24 June. <https://www.latimes.com/business/technology/story/2020-06-24/diversity-in-tech-tech-workers-tell-their-story>
6. Funk, Carry, and Kim Parker. 2018. "Women and Men in STEM Often at Odds Over Workplace Equity." *Pew Research*. www.pewresearch.org/social-trends/2018/01/09/women-and-men-in-stem-often-at-odds-over-workplace-equity/.
7. Barboza, David. 2020. "After Suicides, Scrutiny of China's Grim Factories." *New York Times*, 6 June. <https://www.nytimes.com/2010/06/07/business/global/07suicide.html>
8. Wang 2016, *passim*.
9. *Gottschalk v. Benson*, 409 U.S. 63, 72.
10. *In re Alappat*, 33 F.3d 1526 (Fed. Cir. 1994).
11. Board of Appeals of the European Patent Office, Case T 1173/97–3.5.1, July 1, 1998.
12. WIPO Copyright Treaty, adopted Dec. 20, 1996.

Bibliography

- Abbate, Janet. 1994. "From Arpanet to Internet: A History of ARPA-Sponsored Computer Networks, 1966–1988." PhD dissertation, University of Pennsylvania.
- Abbate, Janet. 1999. *Inventing the Internet*. Cambridge, MA: MIT Press.
- Aboba, Bernard. 1993. *The Online User's Encyclopedia: Bulletin Boards and Beyond*. Reading, MA: Addison-Wesley.
- Agar, Jon. 2003. *The Government Machine: A Revolutionary History of the Computer*. Cambridge, MA: MIT Press.
- Aiken, Howard H. 1937. "Proposed Automatic Calculating Machine." In Randell, Brian (Ed.), *Origins of Digital Computers: Selected Papers*. New York: Springer-Verlag, pp. 195–201.
- Aiken, Howard H. et al. 1946. *A Manual of Operation of the Automatic Sequence Controlled Calculator*. Cambridge, MA: Harvard University Press (Reprint, with a foreword by I. Bernard Cohen and an introduction by Paul Ceruzzi, volume 8, Charles Babbage Institute Reprint Series for the History of Computing. Cambridge, MA and Los Angeles: MIT Press and Tomash Publishers, 1985).
- Akera, Atsushi. 2007. *Calculating a Natural World: Scientists, Engineers, and Computers During the Rise of U.S. Cold War Research*. Cambridge, MA: MIT Press.
- Alborn, Tim. 1994. "Public Science, Private Finance: The Uneasy Advancement of J. W. Lubbock." In *Proceedings of a Conference on Science and British Culture in the 1830s*, 6–8 July. Cambridge: Trinity College, pp. 5–14.
- Allen, Danielle, and Jennifer S. Light. Eds. 2015. *From Voice to Influence: Understanding Citizenship in a Digital Age*. Chicago, IL: University of Chicago Press.
- Allyn, Stanley C. 1967. *My Half Century with NCR*. New York: McGraw-Hill.
- Anderson, Margo J. 1988. *The American Census: A Social History*. New Haven, CT: Yale University Press.
- Angel, Karen. 2001. *Inside Yahoo! Reinvention and the Road Ahead*. New York: John Wiley.
- Anonymous. 1874. "The Central Telegraph Office." *Illustrated London News*, 28 November, p. 506, and 10 December, p. 530.
- Anonymous. 1932. "International Business Machines." *Fortune*, January, pp. 34–50.
- Anonymous. 1940. "International Business Machines." *Fortune*, January, p. 36.
- Anonymous. 1950. "Never Stumped: International Business Machines' Selective Sequence Electronic Calculator." *The New Yorker*, 4 March, pp. 20–1.
- Anonymous. 1952. "Office Robots." *Fortune*, January, p. 82.

- Ashford, Oliver M. 1985. *Prophet—or Professor? The Life and Work of Lewis Fry Richardson*. Boston, MA: Adam Hilger.
- Aspray, William. Ed. 1990a. *Computing Before Computers*. Ames, IA: Iowa State University Press.
- Aspray, William. 1990b. *John von Neumann and the Origins of Modern Computing*. Cambridge, MA: MIT Press.
- Aspray, William. 2016a. *Participation in Computing*. Cham: Springer International Publishing.
- Aspray, William. 2016b. *Women and Underrepresented Minorities in Computing*. Cham: Springer International Publishing.
- Aspray, William, and James W. Cortada. 2019. *From Urban Legends to Political Fact-Checking*. Springer International Publishing.
- Aspray, William, Frank Mayadas, and Moshe Y. Vardi. Eds. 2006. *Globalization and Offshoring of Software: A Report of the ACM Job Migration Task Force*. New York: Association for Computing Machinery. www.acm.org/binaries/content/assets/public-policy/usacm/intellectual-property/reports-and-white-papers/full_final1.pdf.
- Augarten, Stan. 1984. *Bit by Bit: An Illustrated History of Computers*. New York: Ticknor & Fields.
- Austrian, Geoffrey D. 1982. *Herman Hollerith: Forgotten Giant of Information Processing*. New York: Columbia University Press.
- Babbage, Charles. 1822a. “A Letter to Sir Humphrey Davy.” In Babbage, Charles (Ed.), *Works of Babbage*. vol. 2. New York: American University Press, pp. 6–14.
- Babbage, Charles. 1822b. “The Science of Number Reduced to Mechanism.” In Babbage, Charles (Ed.), *Works of Babbage*. vol. 2. New York: American University Press, pp. 15–32.
- Babbage, Charles. 1834. “Statement Addressed to the Duke of Wellington.” In Babbage, Charles (Ed.), *Works of Babbage*. vol. 3. New York: American University Press, pp. 2–8.
- Babbage, Charles. 1835. “Economy of Machinery and Manufactures.” In Babbage, Charles (Ed.), *Works of Babbage*. vol. 5. New York: American University Press.
- Babbage, Charles. 1852. “Thoughts on the Principles of Taxation.” In Babbage, Charles (Ed.), *Works of Babbage*. vol. 5. New York: American University Press, pp. 31–56.
- Babbage, Charles. 1989. *Works of Babbage*. Ed. M. Campbell-Kelly. 11 vols. New York: American University Press.
- Babbage, Charles. 1994. *Passages from the Life of a Philosopher*. Edited with a New Introduction by Martin Campbell-Kelly. 1864. New Brunswick, NJ: IEEE Press and Rutgers University Press. Also in Babbage 1989, vol. 11.
- Backus, John. 1979. “The History of Fortran I, II, and III.” *Annals of the History of Computing* 1, no. 1: 21–37.
- Baran, Paul. 1970. “The Future Computer Utility.” In Taviss (Ed.), pp. 81–92.
- Bardini, Thierry. 2000. *Bootstrapping: Douglas Engelbart, Coevolution, and the Origins of Personal Computing*. Stanford, CA: Stanford University Press.
- Barnett, C. C., Jr., and Associates 1967. *The Future of the Computer Utility*. New York: American Management Association.
- Barnouw, Erik. 1967. *A Tower in Babel (History of Broadcasting in the United States)*. New York: Oxford University Press.
- Barron, David W. 1971. *Computer Operating Systems*. London: Chapman and Hall.

- Bashe, Charles J., Lyle R. Johnson, John H. Palmer, and Emerson W. Pugh. 1986. *IBM's Early Computers*. Cambridge, MA: MIT Press.
- Bassett, Ross. 2016. *The Technological Indian*. Cambridge, MA: Harvard University Press.
- Bátiz-Lazo, Bernardo. 2018. *Cash and Dash: How ATMs and Computers Changed Banking*. Oxford: Oxford University Press.
- Bátiz-Lazo, Bernardo, and R. J. K. Reid. 2011. "The Development of Cash-Dispensing Technology in the UK." *IEEE Annals of the History of Computing* 33, no. 3: 32–45.
- Baum, Claude. 1981. *The System Builders: The Story of SDC*. Santa Monica, CA: System Development Corporation.
- Baxter, James Phinney. 1946. *Scientists Against Time*. Boston, MA: Little, Brown.
- Beeching, Wilfred A. 1974. *A Century of the Typewriter*. London: Heinemann.
- Belden, Thomas G., and Marva R. Belden. 1962. *The Lengthening Shadow: The Life of Thomas J. Watson*. Boston, MA: Little, Brown.
- Benjamin, Ruha. 2019. *Race After Technology: Abolitionist Tools for the Jim Code*. Cambridge: Polity Press.
- Bennington, Herbert D. 1983. "Production of Large Computer Programs." *Annals of the History of Computing* 5, no. 4: 350–61. Reprint of 1956 article paper with a new introduction.
- Bergin, Thomas J., Richard G. Gibson, and Richard G. Gibson Jr. Eds. 1996. *History of Programming Languages*. vol. 2. Reading, MA: Addison-Wesley.
- Berkeley, Edmund Callis. 1949. *Giant Brains or Machines That Think*. New York: John Wiley.
- Berlin, Leslie. 2005. *The Man Behind the Microchip: Robert Noyce and the Invention of Silicon Valley*. New York: Oxford University Press.
- Berners-Lee, Tim. 1999. *Weaving the Web: The Past, Present, and Future of the World Wide Web by Its Inventor*. London: Orion Business Books.
- Bernstein, Jeremy. 1981. *The Analytical Engine*. New York: Random House.
- Beyer, Kurt W. 2009. *Grace Hopper and the Invention of the Information Age*. Cambridge, MA: MIT Press.
- Bliven, Bruce, Jr. 1954. *The Wonderful Writing Machine*. New York: Random House.
- Bowden, B. V. Ed. 1953. *Faster Than Thought*. London: Pitman.
- Braun, Ernest, and Stuart Macdonald. 1978. *Revolution in Miniature: The History and Impact of Semiconductor Electronics*. Cambridge: Cambridge University Press.
- Brennan, Jean F. 1971. *The IBM Watson Laboratory at Columbia University: A History*. New York: IBM Corp.
- Bright, Herb. 1979. "Fortran Comes to Westinghouse-Bettis, 1957." *Annals of the History of Computing* 7, no. 1: 72–4.
- Brock, Gerald W. 1975. *The U.S. Computer Industry: A Study of Market Power*. Cambridge, MA: Ballinger.
- Bromley, Alan G. 1982. "Charles Babbage's Analytical Engine, 1838." *Annals of the History of Computing* 4, no. 3: 196–217.
- Bromley, Alan G. 1990a. "Difference and Analytical Engines." In Aspray, William (Ed.), *Computing Before Computers*. Ames, IA: Iowa State University Press, pp. 59–98.

- Bromley, Alan G. 1990b. "Analog Computing Devices." In Aspray, William (Ed.), *Computing Before Computers*. Ames, IA: Iowa State University Press, pp. 156–99.
- Brooks, Frederick P., Jr. 1975. *The Mythical Man-Month: Essays in Software Engineering*. Reading, MA: Addison-Wesley.
- Brown, Steven A. 1997. *Revolution at the Checkout Counter: The Explosion of the Bar Code*. Cambridge, MA: Harvard University Press.
- Bud-Frierman, Lisa. Ed. 1994. *Information Acumen: The Understanding and Use of Knowledge in Modern Business*. London and New York: Routledge.
- Burck, Gilbert. 1964. "The Assault on Fortress I.B.M." *Fortune*, June, p. 112.
- Burck, Gilbert. 1965. *The Computer Age and Its Potential for Management*. New York: Harper Torchbooks.
- Burck, Gilbert. 1968. "The Computer Industry's Great Expectations." *Fortune*, August, p. 92.
- Burke, Colin B. 1994. *Information and Secrecy: Vannevar Bush, Ultra, and the Other Memex*. Metuchen, NJ: Scarecrow Press.
- Burks, Alice R., and Arthur W. Burks. 1988. *The First Electronic Computer: The Atanasoff Story*. Ann Arbor, MI: University of Michigan Press.
- Burrell, Jenna. 2012. *Invisible Users: Youth in the Internet Cafés of Urban Ghana*. Cambridge, MA: MIT Press.
- Bush, Vannevar. 1945. "As We May Think." *Atlantic Monthly*, July, pp. 101–8. Reprinted in Goldberg 1988, pp. 237–47.
- Bush, Vannevar. 1967. *Science Is Not Enough*. New York: Morrow.
- Bush, Vannevar. 1970. *Pieces of the Action*. New York: Morrow.
- Business Weekly Magazine. 2018. *Qishi: Zhang Zhongmou dazao Taijidian pandeng shijieji qiye de jingying zhidao [The Way of TSMC: The Philosophy and Strategy of Morris C.M. Chang to Make TSMC Ahead of the Market]*. Taipei: Business Weekly Magazine.
- Campbell-Kelly, Martin. 1982. "Foundations of Computer Programming in Britain 1945–1955." *Annals of the History of Computing* 4: 133–62.
- Campbell-Kelly, Martin. 1988. "Data Communications at the National Physical Laboratory (1965–1975)." *Annals of the History of Computing* 9, no. 3–4: 221–47.
- Campbell-Kelly, Martin. 1989. *ICL: A Business and Technical History*. Oxford: Oxford University Press.
- Campbell-Kelly, Martin. 1992. "Large-Scale Data Processing in the Prudential, 1850–1930." *Accounting, Business and Financial History* 2, no. 2: 117–39.
- Campbell-Kelly, Martin. 1994. "The Railway Clearing House and Victorian Data Processing." In Bud-Frierman, Lisa (Ed.), *Information Acumen: The Understanding and Use of Knowledge in Modern Business*. London and New York: Routledge, pp. 51–74.
- Campbell-Kelly, Martin. 1995. "The Development and Structure of the International Software Industry, 1950–1990." *Business and Economic History* 24, no. 2: 73–110.
- Campbell-Kelly, Martin. 1996. "Information Technology and Organizational Change in the British Census, 1801–1911." *Information Systems Research* 7, no. 1: 22–36. Reprinted in Yates and Van Maanen 2001, pp. 35–58.
- Campbell-Kelly, Martin. 1998. "Data Processing and Technological Change: The Post Office Savings Bank, 1861–1930." *Technology and Culture* 39: 1–32.

- Campbell-Kelly, Martin. 2003. *From Airline Reservations to Sonic the Hedgehog: A History of the Software Industry*. Cambridge, MA: MIT Press.
- Campbell-Kelly, Martin, Mary Croarken, Eleanor Robson, and Raymond Flood. Eds. 2003. *Sumer to Spreadsheets: The History of Mathematical Tables*. Oxford: Oxford University Press.
- Campbell-Kelly, Martin, and Michael R. Williams. Eds. 1985. *The Moore School Lectures*. Cambridge, MA and Los Angeles: MIT Press and Tomash Publishers.
- Cannon, James G. 1910. *Clearing Houses*. Washington, DC: National Monetary Commission.
- Care, Charles. 2010. *Technology for Modelling: Electrical Analogies, Engineering Practice, and the Development of Analogue Computing*. London: Springer.
- Carlton, Jim. 1997. *Apple: The Inside Story of Intrigue, Egomania, and Business Blunders*. New York: Random House.
- Carroll, Paul. 1994. *Big Blues: The Unmaking of IBM*. London: Weidenfeld & Nicolson.
- Cassidy, John. 2002. *Dot-Con: The Greatest Story Ever Sold*. New York: Harper Collins.
- Ceruzzi, Paul E. 1983. *Reckoners: The Prehistory of the Digital Computer, from Relays to the Stored Program Concept, 1935–1945*. Westport, CT: Greenwood Press.
- Ceruzzi, Paul E. 1991. "When Computers Were Human." *Annals of the History of Computing* 13, no. 3: 237–44.
- Chan, Jenny, Mark Selden, and Ngai Pun. 2020. *Dying for an iPhone: Apple, Foxconn and the Lives of China's Workers*. Chicago, IL: Haymarket Books.
- Chang, Morris. 2007. "Oral History of Morris Chang," Interviewed by Alan Patterson August 24, 2007, CHM Reference Number: X4151.2008. Computer History Museum.
- Chang, Sea-Jin. 2011. *Sony Vs Samsung: The Inside Story of the Electronics Giants' Battle for Global Supremacy*. Singapore: Wiley.
- Chase, G. C. 1980. "History of Mechanical Computing Machinery." *Annals of the History of Computing* 2, no. 3: 198–226. Reprint of 1952 conference paper, with an introduction by I. B. Cohen.
- Chinoy, Ira. 2010. "Battle of the Brains: Election-Night Forecasting at the Dawn of the Computer Age." PhD dissertation, Philip Merrill College of Journalism, University of Maryland.
- Chposky, James, and Ted Leonsis. 1988. *Blue Magic: The People, Power and Politics Behind the IBM Personal Computer*. New York: Facts on File.
- Cohen, I. Bernard. 1988. "Babbage and Aiken." *Annals of the History of Computing* 10, no. 3: 171–93.
- Cohen, I. Bernard. 1999. *Howard Aiken: Portrait of a Computer Pioneer*. Cambridge, MA: MIT Press.
- Cohen, Patricia Cline. 1982. *A Calculating People: The Spread of Numeracy in Early America*. Chicago, IL: University of Chicago Press.
- Cohen, Scott. 1984. *Zap! The Rise and Fall of Atari*. New York: McGraw-Hill.
- Collier, Bruce, and James MacLachlan. 1998. *Charles Babbage and the Engines of Perfection*. New York: Oxford University Press.
- Comrie, L. J. 1933. "Computing the 'Nautical Almanac'." *Nautical Magazine*, July, pp. 1–16.
- Comrie, L. J. 1946. "Babbage's Dream Comes True." *Nature* 158: 567–8.
- Con Diaz, Gerardo. 2019. *Software Rights: How Patent Law Transformed Software Development in America*. New Haven, CT: Yale University Press.

- Coopersmith, Jonathan. 2000. "Pornography, Videotape, and the Internet." *IEEE Technology and Society Magazine* 19, no. 1: 27–34.
- Cortada, James W. 1993a. *Before the Computer: IBM, NCR, Burroughs, and Remington Rand and the Industry They Created, 1865–1956*. Princeton, NJ: Princeton University Press.
- Cortada, James W. 1993b. *The Computer in the United States: From Laboratory to Market, 1930–1960*. Armonk, NY: M. E. Sharpe.
- Cortada, James W. 2004. *The Digital Hand, Vol. 1: How Computers Changed the Work of American Manufacturing, Transportation, and Retail Industries*. New York: Oxford University Press.
- Cortada, James W. 2006. *The Digital Hand, Vol. 2: How Computers Changed the Work of American Financial, Telecommunications, Media, and Entertainment Industries*. New York: Oxford University Press.
- Cortada, James W. 2007. *The Digital Hand, Vol. 3. How Computers Changed the Work of American Public Sector Industries*. New York: Oxford University Press.
- Cortada, James W. 2019. *IBM: The Rise and Fall and Reinvention of a Global Icon*. Cambridge, MA: MIT Press.
- Cortada, James W., and William F. Aspray. 2019. *Fake News Nation: The Long History of Lies and Misinterpretations in America*. Latham, MD: Rowman and Littlefield.
- Cringley, Robert X. 1992. *Accidental Empires: How the Boys of Silicon Valley Make Their Millions, Battle Foreign Competition, and Still Can't Get a Date*. Reading, MA: Addison-Wesley.
- Croarken, M. 1989. *Early Scientific Computing in Britain*. Oxford: Oxford University Press.
- Croarken, M. 1991. "Case 5656: L. J. Comrie and the Origins of the Scientific Computing Service Ltd." *IEEE Annals of the History of Computing* 21, no. 4: 70–1.
- Croarken, M. 2000. "L. J. Comrie: A Forgotten Figure in the History of Numerical Computation." *Mathematics Today* 36, no. 4: 114–18.
- Crowther, Samuel. 1923. *John H. Patterson: Pioneer in Industrial Welfare*. New York: Doubleday.
- Cusumano, Michael A., and Richard W. Selby. 1995. *Microsoft Secrets: How the World's Most Powerful Software Company Creates Technology, Shapes Markets, and Manages People*. New York: Free Press.
- Cusumano, Michael A., and David B. Yoffie. 1998. *Competing on Internet Time: Lessons from Netscape and Its Battle with Microsoft*. New York: Free Press.
- Darby, Edwin. 1968. *It All Adds Up: The Growth of Victor Comptometer Corporation*. Chicago, IL: Victor Comptometer Corp.
- David, Paul A. 1986. "Understanding the Economics of Qwerty: The Necessity of History." In Parker 1986, pp. 30–49.
- Davies, Margery W. 1982. *Woman's Place Is at the Typewriter: Office Work and Office Workers, 1870–1930*. Philadelphia, PA: Temple University Press.
- DeLamarter, Richard Thomas. 1986. *Big Blue: IBM's Use and Abuse of Power*. New York: Dodd, Mead.
- Douglas, Susan J. 1987. *Inventing American Broadcasting, 1899–1922*. Baltimore, MD: Johns Hopkins University Press.
- Eames, Charles, and Ray Eames, 1973. *A Computer Perspective*. Cambridge, MA: Harvard University Press.

- Eckert, J. Presper. 1976. "Thoughts on the History of Computing." *Computer*, pp. 58–65.
- Edwards, Paul N. 1996. *The Closed World: Computers and the Politics of Discourse in Cold War America*. Cambridge, MA: MIT Press.
- Eklund, Jon. 1994. "The Reservisor Automated Airline Reservation System: Combining Communications and Computing." *Annals of the History of Computing* 16, no. 1: 6–69.
- Engelbart, Doug. 1988. "The Augmented Knowledge Workshop." In Goldberg, Adele (Ed.), *A History of Personal Workstations*. New York: ACM Press, pp. 187–232.
- Engelbart, Douglas C., and William K. English. 1968. "A Research Center for Augmenting Human Intellect." In *Proceedings of the AFIPS 1968 Fall Joint Computer Conference*. Washington, DC: Spartan Books, pp. 395–410.
- Engelbourg, Saul. 1976. *International Business Machines: A Business History*. New York: Arno Press.
- Engler, George Nichols. 1969. *The Typewriter Industry: The Impact of a Significant Technological Revolution*. Los Angeles, CA: University of California, Los Angeles. Available from University Microfilms International, Ann Arbor, MI.
- Ensmenger, Nathan. 2010. *The Computer Boys Take Over: Computers, Programmers, and the Politics of Technical Expertise*. Cambridge, MA: MIT Press.
- Ensmenger, Nathan. 2018. "The Environmental History of Computing." *Technology and Culture* 59, no. 4: S7–S33.
- Ensmenger, Nathan. 2021. "The Cloud Is a Factory." In Mullaney, Thomas, Benjamin Peters, Mar Hicks, and Kavita Philip (Eds.), *Your Computer Is on Fire*. Cambridge, MA: MIT Press.
- Fano, R. M., and P. J. Corbato. 1966. "Time-Sharing on Computers." In Scientific American (Ed.), *Information*. San Francisco, CA: W. H. Freeman, pp. 76–95.
- Fisher, Franklin M., James N. V. McKie, and Richard B. Mancke. 1983. *IBM and the US Data Processing Industry: An Economic History*. New York: Praeger.
- Fishman, Katherine Davis. 1981. *The Computer Establishment*. New York: Harper & Row.
- Flamm, Kenneth. 1987. *Targeting the Computer: Government Support and International Competition*. Washington, DC: Brookings Institution.
- Flamm, Kenneth. 1988. *Creating the Computer: Government, Industry, and High Technology*. Washington, DC: Brookings Institution.
- Fletcher, Amy L. 2002. "France Enters the Information Age: A Political History of Minitel." *History and Technology* 18, no. 2: 103–19.
- Foreman, R. 1985. *Fulfilling the Computer's Promise: The History of Informatics, 1962–1968*. Woodland Hills, CA: Informatics General Corp.
- Forrester, Jay. 1971. *World Dynamics*. Cambridge, MA: Wright-Allen Press.
- Fouché, Rayvon. 2012. "From Black Inventors to One Laptop Per Child: Exporting a Racial Politics of Technology." In Nakamura, Lisa and Peter Chow-White (Eds.), *Race After the Internet*. New York: Routledge, pp. 61–83.
- Freiberger, Paul, and Michael Swaine. 1984. *Fire in the Valley: The Making of the Personal Computer*. Berkeley, CA: Osborne/McGraw-Hill.
- Freeman, Peter A. W., Richards Adrion, and William F. Aspray. 2019. *Computing and the National Science Foundation, 1950–2016*. New York: ACM Books.
- Gabrys, Jennifer. 2011. *Digital Rubbish: A Natural History of Electronics*. Ann Arbor, MI: Michigan University Press.

- Gannon, Paul. 1997. *Trojan Horses and National Champions: The Crisis in the European Computing and Telecommunications Industry*. London: Apt-Amatic Books.
- Giles, Jim. 2005. "Internet Encyclopaedias Go Head to Head." *Nature* 438: 900–1.
- Gillespie, Tarleton. 2021. *Custodians of the Internet: Platforms, Content Moderation, and the Hidden Decisions That Shape Social Media*. New Haven, CT: Yale University Press.
- Gillies, James, and Robert Cailliau. 2000. *How the Web Was Born: The Story of the World Wide Web*. Oxford: Oxford University Press.
- Goldberg, Adele. Ed. 1988. *A History of Personal Workstations*. New York: ACM Press.
- Goldsmith, Jack L., and Tim Wu. 2006. *Who Controls the Internet? Illusions of a Borderless World*. New York: Oxford University Press.
- Goldstein, Andrew, and William F. Aspray. Eds. 1997. *Facets: New Perspectives on the History of Semiconductors*. New York: IEEE Press.
- Goldstine, Herman H. 1972. *The Computer: From Pascal to von Neumann*. Princeton, NJ: Princeton University Press.
- Gompers, Paul. 1994. "The Rise and Fall of Venture Capital." *Business and Economic History* 23, no. 2: 1–26.
- Grattan-Guinness, Ivor. 1990. "Work for the Hairdressers: The Production of de Prony's Logarithmic and Trigonometric Tables." *Annals of the History of Computing* 12, no. 3: 177–85.
- Greenberger, Martin. 1964. "The Computers of Tomorrow." *Atlantic Monthly*, July, pp. 63–7.
- Greenstein, Shane. 2015. *How the Internet Became Commercial: Innovation, Privatization, and the Birth of a New Network*. Princeton, NJ: Princeton University Press.
- Grier, David A. 2005. *When Computers Were Human*. Cambridge, MA: MIT Press.
- Grosch, Herbert R. J. 1989. *Computer: Bit Slices from a Life*. Lancaster, PA: Third Millennium Books.
- Gruenberger, Fred. Ed. 1971. *Expanding Use of Computers in the 70's: Markets-Needs-Technology*. Englewood Cliffs, NJ: Prentice-Hall.
- Haber, Stuart, and W. Scott Stornetta. 1991. "How to Time Stamp a Digital Document." *Journal of Cryptology* 3, no. 2: 99–111.
- Haigh, Thomas, and Paul E. Ceruzzi. 2021. *A New History of Modern Computing*. Cambridge, MA: MIT Press.
- Haigh, Thomas, Mark Priestley, and Crispin Rope. 2016. *ENIAC in Action: Making and Remaking the Modern Computer*. Cambridge, MA: MIT Press.
- Harwood, John. 2011. *The Interface: IBM and the Transformation of Corporate Design 1945–1976*. Minneapolis, MN: University of Minnesota Press.
- Hashagen, Ulf, Reinhard Keil-Slawik, and Arthur Norberg. Eds. 2002. *History of Computing—Software Issues*. Berlin: Springer-Verlag.
- Hauben, Michael, and Ronda Hauben. 1997. *Netizens: On the History and Impact of Usenet and the Internet*. New York: Wiley-IEEE Computer Society Press.
- Herman, Leonard. 1994. *Phoenix: The Fall and Rise of Home Videogames*. Union, NJ: Rolenta Press.
- Hessen, Robert. 1973. "Rand, James Henry, 1859–1944." *Dictionary of American Biography* (suppl 3): 618–19.
- Hicks, Mar. 2018. *Programmed Inequality: How Britain Discarded Women Technologists and Lost Its Edge in Computing*. Cambridge, MA: MIT Press.

- Hiltzik, Michael. 1999. *Dealers of Lightning: Xerox PARC and the Dawn of the Computer Age*. New York: Harper Business.
- Hock, Dee. 1999. *Birth of the Chaordic Age*. San Francisco, CA: Berrett-Koehler.
- Hodges, Andrew. 1983. *Alan Turing: The Enigma*. London: Burnett Books.
- Hodges, Andrew. 2004. "Turing, Alkan Mathison (1912–1954)." In *Dictionary of National Biography*. Oxford: University Press.
- Hoke, Donald R. 1990. *Ingenious Yankees: The Rise of the American System of Manufactures in the Private Sector*. New York: Columbia University Press.
- Hopper, Grace Murray. 1981. "The First Bug." *Annals of the History of Computing* 3, no. 3: 285–6.
- Housel, T. J., and W. H. Davidson. 1991. "The Development of Information Services in France: The Case of Minitel." *International Journal of Information Management* 11, no. 1: 35–54.
- Hyman, Anthony. 1982. *Charles Babbage: Pioneer of the Computer*. Princeton, NJ: Princeton University Press.
- Ichbiah, Daniel, and Susan L. Knepper. 1991. *The Making of Microsoft*. Rocklin, CA: Prima Publishing.
- Isaacson, Walter. 2011. *Steve Jobs*. New York: Simon & Schuster.
- Jacobs, John F. 1986. *The SAGE Air Defense System: A Personal History*. Bedford, MA: Mitre Corp.
- Jiang, Shangrong, Yuze Li, Quanying Lu, Yongmiao Hong, Dabo Guan, Yu Xiong, and Shouyang Wang. 2021. "Policy Assessments for the Carbon Emission Flows and Sustainability of Bitcoin Blockchain Operation in China." *Nature Communications*, 6 April. <https://www.nature.com/articles/s41467-021-22256-3>
- Jobs, Steve. 2005. "You've Got to Find What You Love." *Stanford Report*. <http://news.stanford.edu/news/2005/june15/jobs-061505.html>.
- Kahney, Leander. 2019. *Tim Cook: The Genius Who Took Apple to the Next Level*. New York: Penguin Publishing Group.
- Kawata, Keizo. 1996. *Fujitsu: The Story of a Company*. Washington, DC: Minerva Press.
- Keating, Gina. 2012. *Netflixed: The Epic Battle for American Eyeballs*. London: Penguin.
- Kemeny, John G., and Thomas E. Kurtz. 1968. "Dartmouth Time-Sharing." *Science* 162, no. 3850: 223–8.
- Kenney, Martin. Ed. 2000. *Understanding Silicon Valley: The Anatomy of an Entrepreneurial Region*. Palo Alto, CA: Stanford University Press.
- Kirkpatrick, David. 2010. *The Facebook Effect: The Inside Story of the Company That Is Connecting the World*. New York: Simon & Schuster.
- Kosseff, Jeff. 2022. *Cybersecurity Law*. Hoboken, NJ: Wiley.
- Lammers, Susan. 1986. *Programmers at Work: Interviews with Nineteen Programmers Who Shaped the Computer Industry*. Washington, DC: Tempus, Redmond.
- Langlois, Richard N. 1992. "External Economies and Economic Progress: The Case of the Microcomputer Industry." *Business History Review* 66, no. 1: 1–50.
- Lardner, Dionysius. 1834. "Babbage's Calculating Engine." In Babbage, Charles (Ed.), *Works of Babbage*. vol. 2. New York: American University Press, pp. 118–86.
- Lavington, Simon H. 1975. *A History of Manchester Computers*. Manchester: NCC.

- Lécuyer, Christophe. 2006. *Making Silicon Valley: Innovation and the Growth of High Tech, 1930–1970*. Cambridge, MA: MIT Press.
- Lee, J. A. N. Ed. 1992a and 1992b. “Special Issue: Time-Sharing and Interactive Computing at MIT.” *Annals of the History of Computing* 14, nos. 1 and 2.
- Lei, Ya-Wen. 2017. *The Contentious Public Sphere: Law, Media, and Authoritarian Rule in China*. Princeton, NJ: Princeton University Press.
- Leslie, Stuart W. 1983. *Boss Kettering*. New York: Columbia University Press.
- Leslie, Stuart W. 1993. *The Cold War and American Science: The Military-Industrial-Academic Complex at MIT and Stanford*. New York: Columbia University Press.
- Levering, Robert, Michael Katz, and Milton Moskowitz. 1984. *The Computer Entrepreneurs*. New York: New American Library.
- Levy, Steven. 1994. *Insanely Great: The Life and Times of Macintosh, the Computer That Changed Everything*. New York: Viking.
- Levy, Steven. 2011. *Into the Plex: How Google Thinks, Works, and Shapes Our Lives*. New York: Simon & Schuster.
- Licklider, J. C. R. 1960. “Man-Computer Symbiosis.” *IRE Transactions on Human Factors in Electronics* (March): 4–11. Also in Goldberg 1988, pp. 131–40.
- Licklider, J. C. R. 1965. *Libraries of the Future*. Cambridge, MA: MIT Press.
- Lih, Andrew. 2009. *The Wikipedia Revolution: How a Bunch of Nobodies Created the World’s Greatest Encyclopedia*. New York: Hyperion.
- Lindgren, M. 1990. *Glory and Failure: The Difference Engines of Johann Muller, Charles Babbage, and Georg and Edvard Scheutz*. Cambridge, MA: MIT Press.
- Lindtner, Silvia. 2020. *Prototype Nation: China and the Contested Promise of Innovation*. Princeton, NJ: Princeton University Press.
- Little, Peter. 2012. *Toxic Town: IBM, Pollution, and Industrial Risks*. New York: NYU Press.
- Liu, Chuanzhi. 2007. “Lenovo: An Example of Globalization of Chinese Enterprises.” *Journal of International Business Studies* 38: 573–7.
- Lohr, Steve. 2001. *Go To: The Story of the Programmers Who Created the Software Revolution*. New York: Basic Books.
- Lovelace, Ada A. 1843. “Sketch of the Analytical Engine.” In Babbage, Charles (Ed.), *Works of Babbage*. vol. 3. New York: American University Press, pp. 89–170.
- Lukoff, Herman. 1979. *From Dits to Bits*. Portland, OR: Robotics Press.
- Main, Jeremy. 1967. “Computer Time-Sharing—Everyman at the Console.” *Fortune*, August, p. 88.
- Malone, Michael S. 1995. *The Microprocessor: A Biography*. New York: Springer-Verlag.
- Manes, Stephen, and Paul Andrews. 1994. *Gates: How Microsoft’s Mogul Reinvented an Industry—and Made Himself the Richest Man in America*. New York: Simon & Schuster.
- Maney, Kevin. 2003. *The Maverick and His Machine: Thomas Watson, Sr. and the Making of IBM*. Hoboken, NJ: Wiley.
- Marcosson, Isaac F. 1945. *Wherever Men Trade: The Romance of the Cash Register*. New York: Dodd, Mead.
- Markoff, John. 1984. “Five Window Managers for the IBM PC.” *ByteGuide to The IBM PC* (Fall): 65–87.

- Martin, Thomas C. 1891. "Counting a Nation by Electricity." *Electrical Engineer* 12: 521–30.
- Mauchly, John. 1942. "The Use of High Speed Vacuum Tube Devices for Calculating." In Randell, Brian (Ed.), *Origins of Digital Computers: Selected Papers*. New York: Springer-Verlag, pp. 355–8.
- May, Earl C., and Will Oursler. 1950. *The Prudential: A Story of Human Security*. New York: Doubleday.
- McCartney, Scott. 1999. *ENIAC: The Triumphs and Tragedies of the World's First Computer*. New York: Walker.
- McCracken, Daniel D. 1961. *A Guide to Fortran Programming*. New York: John Wiley.
- McKenney, James L., Duncan G. Copeland, and Richard Mason. 1995. *Waves of Change: Business Evolution Through Information Technology*. Boston, MA: Harvard Business School Press.
- Medina, Eden, Ivan da Costa Marques, and Christina Holmes. Eds. 2014. *Beyond Imported Magic: Essays on Science, Technology, and Society in Latin America*. Cambridge, MA: MIT Press.
- Mollenhoff, Clark R. 1988. *Atanasoff: Forgotten Father of the Computer*. Ames, IA: Iowa State University Press.
- Montfort, Nick, and Ian Bogost. 2009. *Racing the Beam: The Atari Video Computer System*. Cambridge, MA: MIT Press.
- Moody, Glyn. 2001. *Rebel Code: Linux and the Open Source Revolution*. New York: Perseus.
- Moore, Gordon E. 1965. "Cramming More Components into Integrated Circuits." *Electronics* 38: 114–17.
- Moritz, Michael. 1984. *The Little Kingdom: The Private Story of Apple Computer*. New York: Morrow.
- Morozov, Evgeny. 2011. *The Net Delusion: The Dark Side of Internet Freedom*. New York: Public Affairs.
- Morris, P. R. 1990. *A History of the World Semiconductor Industry*. London: Peter Perigrinus/IEE.
- Morton, Alan Q. 1994. "Packaging History: The Emergence of the Uniform Product Code (UPC) in the United States, 1970–75." *History and Technology* 11: 101–11.
- Moschovitis, Christos J. P., Hilary Poole, Tami Schuyler, and Theresa M. Senft. 1999. *History of the Internet: A Chronology 1843 to Present*. Santa Barbara, CA: ABC-CLIO.
- Mowery, David C. Ed. 1996. *The International Computer Software Industry*. New York: Oxford University Press.
- Murray, Andrew. 2019. *Information Technology Law: The Law and Society*. Oxford: Oxford University Press.
- Mutsvairo, Bruce. Ed. 2016. *Digital Activism in the Social Media Era: Critical Reflections on Emerging Trends in Sub-Saharan Africa*. Switzerland: Palgrave Macmillan.
- Nakamura, Lisa. 2014. "Indigenous Circuits: Navajo Women and the Racialization of Early Electronic Manufacture." *American Quarterly* 66, no. 4: 919–41.
- Narayanan, Arvind et al. 2016. *Bitcoin and Cryptocurrency Technologies: A Comprehensive Introduction*. Princeton, NJ: Princeton University Press.
- Naughton, John. 2001. *A Brief History of the Future: From Radio Days to Internet Years in a Lifetime*. Woodstock and New York: Overlook.

- Naur, Peter, and Brian Randell. Eds. 1969. "Software Engineering." Report on a conference sponsored by the NATO Science Committee, Garmisch-Partenkirchen, 7–11 October 1968. Brussels: NATO Scientific Affairs Division.
- NCR. 1984a. *NCR: 1884–1922: The Cash Register Era*. Dayton, OH: NCR.
- NCR. 1984b. *NCR: 1923–1951: The Accounting Machine Era*. Dayton, OH: NCR.
- NCR. 1984c. *NCR: 1952–1984: The Computer Age*. Dayton, OH: NCR.
- NCR. 1984d. *NCR: 1985 and Beyond: The Information Society*. Dayton, OH: NCR.
- Nebeker, Frederik. 1995. *Calculating the Weather: Meteorology in the Twentieth Century*. New York: Academic Press.
- Nelson, Theodor H. 1974a. *Computer Lib: You Can and Must Understand Computers Now*. Chicago, IL: Theodor H. Nelson.
- Nelson, Theodor H. 1974b. *Dream Machines: New Freedoms Through Computer Screens—A Minority Report*. Chicago, IL: Theodor H. Nelson.
- Noble, Safiya Umoja. 2018. *Algorithms of Oppression: How Search Engines Reinforce Racism*. New York: New York University Press.
- Nora, Simon, and Alain Minc. 1980. *The Computerization of Society*. Cambridge, MA: MIT Press.
- Norberg, Arthur L. 1990. "High-Technology Calculation in the Early 20th Century: Punched Card Machinery in Business and Government." *Technology and Culture* 31, no. 4: 753–79.
- Norberg, Arthur L. 2005. *Computers and Commerce: A Study of Technology and Management at Eckert-Mauchly Computer Company, Engineering Research Associates, and Remington Rand, 1946–1957*. Cambridge, MA: MIT Press.
- Norberg, Arthur L., and Judy E. O'Neill. 2000. *Transforming Computer Technology: Information Processing for the Pentagon, 1962–1986*. Baltimore, MD: Johns Hopkins University Press.
- November, Joseph. 2012. *Biomedical Computing: Digitizing Life in the United States*. Baltimore, MD: Johns Hopkins University Press.
- Nyamnjoh, Henrietta Mambo. 2013. "Bridging Mobilities: ICT's Appropriation by Cameroonians in South Africa and The Netherlands." PhD dissertation, Leiden University.
- Nyce, James M., and Paul Kahn. Eds. 1991. *From Memex to Hypertext: Vannevar Bush and the Mind's Machine*. Boston, MA: Academic Press.
- OECD Directorate for Science, Technology and Industry, Committee for Information, Computer and Communications Policy. 1998. *France's Experience with the Minitel: Lessons for Electronic Commerce Over the Internet*. Paris: OECD.
- O'Neill, Judy. 1992. "The Evolution of Interactive Computing Through Time Sharing and Networking." PhD dissertation, University of Minnesota. Available from University Microfilms International, Ann Arbor, MI.
- Owens, Larry. 1986. "Vannevar Bush and the Differential Analyzer: The Text and Context of an Early Computer." *Technology and Culture* 27, no. 1: 63–95.
- Padua, Sidney. 2015. *The Thrilling Adventures of Lovelace and Babbage*. London: Particular Books.
- Parker, W. N. Ed. 1986. *Economic History and the Modern Economist*. New York: Basil Blackwell.
- Parkhill, D. F. 1966. *The Challenge of the Computer Utility*. Reading, MA: Addison-Wesley.

- Pellow, David, and Lisa Sun-Hee Park. 2002. *The Silicon Valley of Dreams: Environmental Injustice, Immigrant Workers, and the High-Tech Global Economy*. New York: NYU Press.
- Petre, Peter. 1985. "The Man Who Keeps the Bloom on Lotus" (Profile of Mitch Kapor). *Fortune*, 10 June, pp. 92–100.
- Plugge, W. R., and M. N. Perry. 1961. "American Airlines' 'SABRE' Electronic Reservations System." In *Proceedings of the AFIPS 1961 Western Joint Computer Conference*. Washington, DC: Spartan Books, pp. 592–601.
- Poole, Steven. 2000. *Trigger Happy: The Inner Life of Videogames*. London: Fourth Estate.
- Pugh, Emerson W. 1984. *Memories That Shaped an Industry: Decisions Leading to IBM System/360*. Cambridge, MA: MIT Press.
- Pugh, Emerson W. 1995. *Building IBM: Shaping an Industry and Its Technology*. Cambridge, MA: MIT Press.
- Pugh, Emerson W., Lyle R. Johnson, and John H. Palmer. 1991. *IBM's 360 and Early 370 Systems*. Cambridge, MA: MIT Press.
- Quartermann, John S. 1990. *The Matrix: Computer Networks and Conferencing Systems Worldwide*. Boston, MA: Digital Press.
- Ralston, Anthony, Edwin D. Reilly, and David Hemmendinger. Eds. 2000. *Encyclopedia of Computer Science*, 4th ed. London: Nature Publishing.
- Randell, Brian. 1982. *Origins of Digital Computers: Selected Papers*. New York: Springer-Verlag.
- Rankin, Joy Lisi. 2018. *A Peoples History of Computing in the United States*. Cambridge, MA: Harvard University Press.
- Redmond, Kent C., and Thomas M. Smith. 1980. *Project Whirlwind: The History of a Pioneer Computer*. Bedford, MA: Digital.
- Redmond, Kent C., and Thomas M. Smith. 2000. *From Whirlwind to MITRE: The R&D Story of the SAGE Air Defense Computer*. Cambridge, MA: MIT Press.
- Reid, Robert H. 1997. *Architects of the Web: 1,000 Days That Built the Future of Business*. New York: John Wiley.
- Richardson, Dennis W. 1970. *Electronic Money: Evolution of an Electronic Funds-transfer System*. Cambridge, MA: MIT Press.
- Richardson, L. F. 1922. *Weather Prediction by Numerical Process*. Cambridge: Cambridge University Press.
- Ridenour, Louis N. 1947. *Radar System Engineering*. New York: McGraw-Hill.
- Rifkin, Glenn, and George Harrar. 1985. *The Ultimate Entrepreneur: The Story of Ken Olsen and Digital Equipment Corporation*. Chicago, IL: Contemporary Books.
- Ritchie, Dennis M. 1984a. "The Evolution of the UNIX Time-Sharing System." *AT&T Bell Laboratories Technical Journal* 63, no. 8: 1577–93.
- Ritchie, Dennis M. 1984b. "Turing Award Lecture: Reflections on Software Research." *Communications of the ACM* 27, no. 8: 758–760.
- Roberts, Craig. 2021. *The Filing Cabinet: A Vertical History of Information*. Minneapolis, MN: University of Minnesota Press.
- Rodgers, William. 1969. *Think: A Biography of the Watsons and IBM*. New York: Stein & Day.

- Rojas, Raul, and Ulf Hashagen. Eds. 2000. *The First Computers: History and Architectures*. Cambridge, MA: MIT Press.
- Roland, Alex, and Philip Shiman. 2002. *Strategic Computing: DARPA and the Quest for Machine Intelligence, 1983–1993*. Cambridge, MA: MIT Press.
- Rosen, Saul. Ed. 1967. *Programming Systems and Languages*. New York: McGraw-Hill.
- Sackman, Hal. 1968. "Conference on Personnel Research." *Datamation* 14, no. 7: 74–6, 81.
- Salus, Peter H. 1994. *A Quarter Century of Unix*. Reading, MA: Addison-Wesley.
- Sammet, Jean E. 1969. *Programming Languages: History and Fundamentals*. Englewood Cliffs, NJ: Prentice-Hall.
- Saxenian, AnnaLee. 1994. *Regional Advantage: Culture and Competition in Silicon Valley and Route 128*. Cambridge, MA: Harvard University Press.
- Schafer, Valérie, and Benjamin G. Thierry. 2012. *Le Minitel: L'enfance numérique de la Franc*. Paris: Nuvis, Cigref.
- Schein, Edgar H., Peter S. DeLisi, Paul J. Kampas, and Michael M. Sonduck. 2003. *DEC Is Dead, Long Live DEC: The Lasting Legacy of Digital Equipment Corporation*. San Francisco, CA: Berrett-Koehler.
- Scientific American. 1966. *Information*. San Francisco, CA: W. H. Freeman.
- Scientific American. 1984. *Computer Software*. Special issue. September.
- Scientific American. 1991. *Communications, Computers and Networks*. Special issue. September.
- Sculley, John, and J. A. Byrne. 1987. *Odyssey: Pepsi to Apple . . . A Journey of Adventure, Ideas, and the Future*. New York: Harper & Row.
- Servan-Schreiber, Jean-Jaques. 1968. *The American Challenge*. London: Hamish Hamilton.
- Seward, Jeff. 2016. *The Politics of Capitalist Transformation: Brazilian Informatics Policy, Regime Change, and State Autonomy*. New York: Taylor & Francis.
- Sharma, Dinesh C. 2015. *The Outsourcer: The Story of India's IT Revolution*. Cambridge, MA: MIT Press.
- Sheehan, R. 1956. "Tom Jr.'s I.B.M." *Fortune*, September, p. 112.
- Sigel, Efrem. 1984. "The Selling of Software." *Datamation*, 15 April, pp. 125–8.
- Slater, Robert. 1987. *Portraits in Silicon*. Cambridge, MA: MIT Press.
- Small, James. 2001. *The Analogue Alternative: The Electronic Analogue Computer in Britain and the USA, 1930–1975*. London: Routledge.
- Smiley, Jane. 2010. *The Man Who Invented the Computer: The Biography of John Atanasoff, Digital Pioneer*. New York: Doubleday.
- Smith, Crosbie, and M. Norton Wise. 1989. *Energy and Empire: A Biographical Study of Lord Kelvin*. Cambridge: Cambridge University Press.
- Smith, David C. 1986. *H. G. Wells: Desperately Mortal: A Biography*. New Haven, CT: Yale University Press.
- Smith, Douglas K., and Robert C. Alexander. 1988. *Fumbling the Future: How Xerox Invented, Then Ignored, the First Personal Computer*. New York: Morrow.
- Smulyan, Susan. 1994. *Selling Radio: The Commercialization of American Broadcasting, 1920–1934*. Washington, DC: Smithsonian Institution Press.
- Sobel, Robert. 1981. *IBM: Colossus in Transition*. New York: Times Books.

- Sobel, Robert. 1986. *IBM vs. Japan: The Struggle for the Future*. New York: Stein and Day.
- Spector, Robert. 2000. *Amazon.com: Get Big Fast*. New York: Random House.
- Stearns, David L. 2011. *Electronic Value Exchange: Origins of the Visa Electronic Payment System*. London: Springer-Verlag.
- Stern, Nancy. 1981. *From ENIAC to UNIVAC: An Appraisal of the Eckert-Mauchly Computers*. Bedford, MA: Digital Press.
- Stross, Randall E. 1996. *The Microsoft Way: The Real Story of How the Company Out-smarts Its Competition*. Reading, MA: Addison-Wesley.
- Suri, Tavneet, and William Jack. 2016. "The Long-run Poverty and Gender Impacts of Mobile Money." *Science* 354, no. 6317: 1288–92.
- Swade, Doron. 1991. *Charles Babbage and His Calculating Engines*. London: Science Museum.
- Swade, Doron. 2001. *The Difference Engine: Charles Babbage and the Quest to Build the First Computer*. New York: Viking.
- Swisher, Kara. 1998. *AOL.COM: How Steve Case Beat Bill Gates, Nailed the Netheads, and Made Millions in the War for the Web*. New York: Times Books.
- Taviss, I. Ed., 1970. *The Computer Impact*. Englewood Cliffs, N.J.: Prentice-Hall.
- Tone, Andrea. 1997. *The Business of Benevolence: Industrial Paternalism in Progressive America*. Ithaca, NJ and London: Cornell University Press.
- Toole, B. A. 1992. *Ada, the Enchantress of Numbers*. Mill Valley, CA: Strawberry Press.
- Truesdell, Leon E. 1965. *The Development of Punch Card Tabulation in the Bureau of the Census, 1890–1940*. Washington, DC: U.S. Department of Commerce.
- Tufecki, Zeynep. 2018. *Twitter and Tear Gas: The Power and Fragility of Networked Protest*. New Haven, CT: Yale University Press.
- Turck, J. A. V. 1921. *Origin of Modern Calculating Machines*. Chicago, IL: Western Society of Engineers.
- Turing, A. M. 1954. "Solvable and Unsolvable Problems." *Science News*, no. 31: 7–23.
- Turkle, Sherry. 2011. *Alone Together: Why We Expect More from Technology and Less from Each Other*. New York: Basic Books.
- Turner, Fred. 2006. *From Counterculture to Cyberculture: Stewart Brand, the Whole Earth Network, and the Rise of Digital Utopianism*. Chicago, IL: University of Chicago Press.
- Useem, Michael, Harbir Singh, Neng Liang, and Peter Cappelli. 2017. *Fortune Makers: The Leaders Creating China's Great Global Companies*. New York: Public Affairs.
- Usselman, Steven. 2004. "Public Policies, Private Platforms: Antitrust and American Computing." In Coopey, Richard (Ed.), *Information Technology Policy: An International History*. Oxford: Oxford University Press.
- Usselman, Steven. 2009. "Unbundling IBM: Antitrust and the Incentives to Innovation in American Computing." In Clarke, Sally, Naomi Lamoreaux, and Steven Usselman (Eds.), *The Challenge of Remaining Innovative: Insights from Twentieth-Century American Business*. Palo Alto, CA: Stanford Business Books.
- Valley, George E., Jr. 1985. "How the SAGE Development Began." *Annals of the History of Computing* 7, no. 3: 196–226.
- van den Ende, Jan. 1992. "Tidal Calculations in the Netherlands." *Annals of the History of Computing* 14, no. 3: 23–33.

- van den Ende, Jan. 1994. *The Turn of the Tide: Computerization in Dutch Society, 1900–1965*. Delft: Delft University Press.
- Veit, Stan. 1993. *Stan Veit's History of the Personal Computer*. Asheville, NC: World-Comm.
- Waldrop, M. Mitchell. 2001. *The Dream Machine: J. C. R. Licklider and the Revolution That Made Computing Personal*. New York: Viking.
- Wallace, James, and Jim Erickson. 1992. *Hard Drive: Bill Gates and the Making of the Microsoft Empire*. New York: John Wiley.
- Wang, An, and Eugene Linden. 1986. *Lessons: An Autobiography*. Reading, MA: Addison-Wesley.
- Wang, Xinyuan. 2016. *Social Media in Industrial China*. London: UCL Press.
- Watkins, Ralph. 1984. *A Competitive Assessment of the U.S. Video Game Industry*. Washington, DC: U.S. Department of Commerce.
- Watson, Thomas, Jr., and Peter Petre. 1990. *Father and Son & Co: My Life at IBM and Beyond*. London: Bantam Press.
- Webster, Bruce. 1996. "The Real Software Crisis." *Byte* 21, no. 1: 218.
- Wells, H. G. 1938. *World Brain*. London: Methuen.
- Wells, H. G. 1995. *World Brain: H. G. Wells on the Future of World Education*. Ed. A. J. Mayne 1928. London: Adamantine Press.
- Wexelblat, Richard L. Ed. 1981. *History of Programming Languages*. New York: Academic Press.
- Wildes, K. L., and N. A. Lindgren. 1986. *A Century of Electrical Engineering and Computer Science at MIT, 1882–1982*. Cambridge, MA: MIT Press.
- Wilkes, Maurice V. 1985. *Memoirs of a Computer Pioneer*. Cambridge, MA: MIT Press.
- Wilkes, Maurice V., David J. Wheeler, and Stanley Gill. 1951. *The Preparation of Programs for an Electronic Digital Computer*. Reading, MA: Addison-Wesley. Reprint, with an introduction by Martin Campbell-Kelly, Los Angeles: Tomash Publishers, 1982. See also volume 1, Charles Babbage Institute Reprint Series for the History of Computing, introduction by Martin Campbell-Kelly.
- Williams, Frederik C. 1975. "Early Computers at Manchester University." *The Radio and Electronic Engineer* 45, no. 7: 327–31.
- Williams, Michael R. 1997. *A History of Computing Technology*. Englewood Cliffs, NJ: Prentice-Hall.
- Wise, Thomas A. 1966a. "I.B.M.'s \$5,000,000,000 Gamble." *Fortune*, September, p. 118.
- Wise, Thomas A. 1966b. "The Rocky Road to the Market Place." *Fortune*, October, p. 138.
- Wolfe, Tom. 1983. "The Tinkerings of Robert Noyce: How the Sun Rose on Silicon Valley." *Esquire*, December, pp. 346–74.
- Yates, JoAnne. 1982. "From Press Book and Pigeonhole to Vertical Filing: Revolution in Storage and Access Systems for Correspondence." *Journal of Business Communication* 19 (Summer): 5–26.
- Yates, JoAnne. 1989. *Control Through Communication: The Rise of System in American Management*. Baltimore, MD: Johns Hopkins University Press.
- Yates, JoAnne. 1993. "Co-evolution of Information-Processing Technology and Use: Interaction Between the Life Insurance and Tabulating Industries." *Business History Review* 67, no. 1: 1–51.

- Yates, JoAnne. 2005. *Structuring the Information Age: Life Insurance and Information Technology in the 20th Century*. Baltimore, MD: Johns Hopkins University Press.
- Yates, JoAnne, and John Van Maanen. 2001. *Information Technology and Organizational Transformation: History Rhetoric, and Preface*. Thousand Oaks, CA: SAGE.
- Yost, Jeffrey R. 2005. *The Computer Industry*. Westport, CT: Greenwood Press.
- Yost, Jeffrey R. 2017. *Making IT Work: A History of the Computer Services Industry*. Cambridge, MA: MIT Press.
- Zachary, G. Pascal. 1997. *Endless Frontier: Vannevar Bush, Engineer of the American Century*. New York: Free Press.
- Zuboff, Shoshana. 2019. *The Age of Surveillance Capitalism: The Fight for a Human Future at the Frontier of Power*. London: Profile Books.

Index

Page numbers in italics refer to figures.

- Aberdeen Proving Ground, in Maryland (US Army) 56, 74
- abstract-logical viewpoint 86
- accounting machines and systems 27, 35, 37, 44, 59, 127; *see also* punched-card systems
- Acer 307
- actuarial tables 9
- Ada programming language 51, 95, 185
- adder-lister machines 35, 99
- adding machines 24, 25, 27, 32, 59
- Ad Hoc Committee of the Grocery Industry 164
- Advanced Micro Devices (AMD) 221, 227
- Advanced Research Project Agency (ARPA) 213–15, 261, 283–7, 295
- Africa, transformation through mobile technology in 314–16
- Ahmadinejad, Mahmoud 328
- Aiken, Howard Hathaway 61–5, 89, 91, 115
- Airbnb 329–31
- air-defense networks 152, 213
- Air Defense System Engineering Committee (ADSEC) 153
- airline control program 163
- airline reservations systems 134, 143, 155–60, 177, 213
- ALGOL programming language 174, 175
- algorithms 171, 324
- Allen, Paul 242–3, 252
- Allyn, Stanley 37
- Altair 8800 241–4
- Alto computer 263–4
- Amateur Computer Society (ACS) 224
- amateurs 236, 237, 238–46
- Amazon Web Services (AWS) 323, 328–9
- American Airlines 155, 157–60, 177
- American Indian Movement 340
- American Marconi Company 236
- American National Standards Institute 175
- American Research and Development (ARD) 223
- American Totalisator Company 112–13, 118, 119
- analog: computers 2; computing systems 53–6, 61, 73, 77, 148; models 55, 56; use of term 53
- Analytical Engine 14, 48–53, 64
- Anderson, Harlan 222
- Andreessen, Marc 292–3
- Android, Inc. 321
- Android operating system 320, 321, 325
- anticipatory carriage 49
- antitrust: law 343; suits 187, 294
- AOL (America On-Line) 275, 293
- apache server program 291
- Apple Computer 245, 301–5, 332;
 - Apple II computer 201, 246, 247, 248, 249; Apple Watch 302; early success of 244–6;
 - iOS platform 320, 321; lawsuit against Microsoft 268; Macintosh computers 260–6, 275, 320;
 - media launches for products 247, 266; *see also specific devices/products*

- applications, computer 132, 136, 143, 176–9, 248–50, 266
- Applied Data Research (ADR) 187
- apps (third-party applications) 321
- archie search system 289
- architecture, computer 85–6, 131, 135, 142
- Arithmometer 33–5
- Arpanet 283–7, 295
- artificial intelligence (AI) 84, 214
- Ashton-Tate 257
- asset card 163
- Association for Computing Machinery (ACM) 67, 184
- astronomical tables 9, 56, 59
- Atanasoff-Berry Computer (ABC) 76–8
- Atanasoff, John Vincent 77–8
- Atari 229
- atomic weapons 73, 81, 86, 119, 138, 155
- authorization: check and credit card 161–2; of purchases 162
- Autoflow 187, 208
- Automated Teller Machines (ATMs) 160–6
- “Automatic Control by Arithmetic Operations” (Crawford) 149
- automatic programming systems 171–3
- Automatic Sequence Controlled Calculator (IBM) *see* Harvard Mark I
- Babbage, Charles: Analytical Engine 14, 48–53; anticipatory carriage 49; bank clearing houses and 14–16; calculating engines of 61–2, 95; character of 20; conditional branch concept and 64; Difference Engine 12–14; range of knowledge of 280; table-making projects and 10–14
- Babbage, Henry 62
- Backus, John 172–3
- ballistics calculations 60, 74–5, 83, 155
- Ballistics Research Laboratory (BRL) 74–6
- Ballmer, Steve 303, 304
- BankAmericard Service Corporation 161
- Bankers’ Clearing House (London) 14–15
- banking industry 14–15, 34, 176
- Bank of America 162
- Bank of England 16
- Baran, Paul 217
- Barclays Bank 161, 197
- barcodes 160–6
- BASE I system 162
- BASE II system 162
- BASIC 213, 230, 246–8, 252
- batch processing 157, 209, 210
- Beginners All-purpose Symbolic Instruction Code *see* BASIC
- Bell, Alexander Graham 341
- Bell Telephone Laboratories 82, 155
- Benioff, Marc 322
- Berkeley, Edmund C. 112
- Berners-Lee, Tim 292
- Berry, Clifford 77
- Bezos, Jeff 323, 328, 329, 333
- BINAC 110–13, 115, 117, 118
- Bitcoin 334, 338
- Bitnet 295
- Blackberry OS 321
- Bletchley Park code-breaking operation 67, 89
- blockchain technology 334
- blogs 319
- “body shopping” tasks 306
- Bolt, Beranek and Newman (BBN) firm 213, 286–7
- bookkeepers and bookkeeping systems 33
- BOS (Batch Operating System) 180
- Brand, Stewart 240
- Brazil 312–14
- Bricklin, Daniel 249
- Brin, Sergey 324
- Britannica* 331–2
- British Telecom 273
- Brooks, Frederick P. 180–1, 184
- Brown, E. W. 60, 61
- Brown’s Tables of the Moon* 60
- browser and server software 291, 292–4
- Bryce, James 62–3
- bugs and debugging 168–73, 179, 182, 183, 210
- Bureau of Ships, U.S. Navy 64
- Burks, Arthur W. 75, 84, 87, 89
- Burroughs Corporation: adder-lister machine of 99; business machine industry and 24, 40; business software developed by 132; business strategies of 134, 143; computer industry and 27, 33, 109, 128, 306; as defense contractor 155; as IBM competitor 143; postwar business challenges

- to 114; scientific data processing and 59; as software contractors 177; technological innovations of 34–5
- Burroughs, William S. 24, 33, 34
- Bushnell, Nolan 229, 246
- Bush, Vannevar 55–6, 73–4, 281–3, 290, 101, 204
- Bush's differential analyzer 56, 61, 74–5, 90, 101
- Busicom 237
- business computers 132–3, 135
- business depressions 42, 43, 45
- business machine industry: adding machine companies 24, 25, 27, 33, 59; America's affinity for technology 24; manufacturers 108; personal computers and 27, 249–50; postwar business challenges to 114; technical innovation in 42, 44; typewriter companies 24, 25, 27–31, 34, 159; U.S. tax laws and 35; *see also* Burroughs Corporation; IBM; NCR; Remington Rand; sales and marketing strategies; training and training organizations
- Byron, Ada *see* Lovelace, Ada
- Byte magazine 243
- Byte Shop 243, 245
- Byung-chul, Lee 309
- Cailliau, Robert 291
- calculating engines *see* Analytical Engine; Difference Engine
- calculating machines 56, 61, 73, 75, 115, 116; *see also* accounting machines and systems; adding machines; punched-card systems
- calculators, electronic 237–8, 247
- California, Toxic Air Contaminant Inventory 337
- Cambridge University 89, 90, 105, 168–71
- Camp, Garrett 330
- card index systems 32
- Card Programmed Calculator (CPC) 115–16
- card readers and punches 132
- Carnot, Lazare 11
- cash-dispensing machines 160–6
- cash registers 35–8
- cathode-ray tube (CRT) storage *see* CRT (cathode-ray tube) display; vacuum tubes
- CBS television network 121
- CD-ROM disks and drives 270–2, 276
- cellphones 222, 301, 303, 309, 312
- census data processing 19–20, 31, 40, 113, 120, 121, 123, 125
- Central Telegraph Office (UK) 18–19, 96
- Challenge of the Computer Utility, The* (Parkhill) 216
- Chang, Morris C. M. 309
- changing face of retail 328–9
- charge cards 161
- Chase, George 61
- chat rooms 274
- checks, bank 14–15
- Chemical Bank 161
- Chesky, Brian 331
- Chilecon Valley 313
- Chinese firms, rise of 311–12
- chip manufacturing, outsourcing of 309–11
- chips, silicon 227, 228–30; *see also* integrated circuits
- Chu, Jeffrey Chuan 135
- Church, Alonzo 66, 67
- circuit boards, digital 223
- Citicorp Overseas Software Limited 306
- civil engineering tables 9
- Clark, Jim 292
- clearing houses 14–19, 24, 35
- clerks: in bank clearing houses 14–15; as human computers 10–16, 57–8, 73; record-keeping duties of 32; as typewriter operators 28, 31; *see also* female clerical labor
- cloud computing 304, 323
- COBOL 175–6, 183, 185
- code breaking 67, 77, 88, 89, 114
- coding *see* programming
- Cold War 153, 155
- Colmar, Thomas de 33
- Colossus computer 89
- Columbia University 61, 115
- Committee on Data Systems and Languages (CODASYL) 175
- Commodore Business Machines 246
- Commodore PET 246
- Compact Disc-Read Only Memory (CD-ROM) 270–1, 276
- Compaq 254

- compatible-range concept 141
- compilers 172, 173, 174, 175
- comptometers 24, 33–4, 59, 97, 98
- CompuServe 274–5, 293
- computer: games 200, 222, 228–9, 247–8, 271; home-built 217; human 9–16, 57–8, 73; industry 25, 27, 31, 37, 76–8; liberation movement 239–40, 243, 244; magazines 243; use of term 9; utilities 216–19, 220, 230, 240, 274; *see also* mainframe computers; minicomputers; personal computers; programmers; real-time computing; stored-program computers; time-sharing computer systems; UNIVAC
- Computer Associates 188
- ComputerLand 243, 253
- Computer Research Corporation (CRC) 110, 127, 128
- computer science 66, 67, 127, 178, 180, 184
- Computer Usage Company (CUC) 177–8
- computing: content moderation and online speech 345–7; and environment 337–8; and intellectual property 340–3; labor and IT industry 338–40; privacy, security, and surveillance 347–50; unfair competition 343–5
- Computing-Tabulating-Recording Company (C-T-R) 39–40, 42
- Comrie, Leslie John 48, 53, 59–61, 65, 88, 90
- conditional branches 64
- Conference on Advanced Computational Techniques 149
- “constructionist” learning theory 313
- consumer electronics industry 229
- consumer networks 274–6
- content moderation and online speech 345–7
- Control Data Corporation (CDC) 109, 127, 128, 135, 143
- control systems 109, 147, 149, 158
- Conway, Lynn 310
- Cook, Tim 302
- core memory technology 125, 132, 154
- corporate personal computer 201
- Cosentino, Laércio 313
- CP/M operating system 248, 251, 267
- Crawford, Perry 149–51, 158
- credit cards 160–6
- CRT (cathode-ray tube) display 78, 153; *see also* vacuum tubes
- cryptocurrency 334
- customer relationship management (CRM) 322
- custom program developers 176
- Dartmouth College 212
- Dartmouth Time-Sharing System (DTSS) 211
- data, defined 347
- database software 188
- data-processing computers 123–7, 175; *see also* information processing; UNIVAC
- Data Processing Management Association 184
- Datron computer (Electrodata) 128
- Davy, Humphrey 12
- dBase database 257
- debit cards 161, 163
- debugging 168–73, 179, 182, 183, 210
- defense projects: Defense Calculator 118, 123–4; Project SAGE 154, 213; Project Whirlwind 147–52, 222; science education/research and 213; semiconductor manufacturers and 222; *see also* scientific war effort
- delay-line storage systems 78, 83, 92, 111, 150
- De Morgan, Augustus 49
- Deng Xiaoping 311, 312
- Densmore, James 28–9
- depressions, business 42, 43, 45
- desk calculator industry 1, 2
- desktop: personal computer development and 235; publishing 266
- Diderot, Denis 279–80
- Difference Engine: Babbage’s 12–14, 48–50, 51, 95; Scheutz’s 52
- differential analyzers 56, 59, 61, 73–6, 90, 101
- differential equations 56, 61, 83
- digital computations, human computers for 58–9, 60–1
- digital electronics 223
- Digital Equipment Corporation (DEC) 127, 219, 222, 288

- Digital Research company 248, 251, 267, 270–1
- digital watches 221, 228
- Diners Club 160–1
- Direction Centers, SAGE system 154, 191
- Disk Operating System (DOS) 180, 260; *see also* MS-DOS operating system
- Disney *see* Walt Disney (company)
- disrupting media delivery 332–3
- document preparation 27, 28, 31
- domain-name system 295
- Doriot, George 223, 225
- Dorsey, Jack 327
- DOS operating system 180, 260; *see also* MS-DOS operating system
- dot-com bubble 305
- Dynabook personal information system 262, 263, 319
- dynamic random-access memory (DRAM) 309
- eBay 319
- Eckert, John Presper: data-processing computers and 110–14; engineering insight of 78–9, 86–8; ENIAC and 79–81, 103, 105; at Moore School 89; as pioneer in computer industry 109–10; UNIVAC and 118–22
- Eckert, Wallace 61, 115
- Eckert-Mauchly Computer Corporation (EMCC) 112–13, 119, 127
- e-commerce 328–9
- Economy of Machinery and Manufactures* (Babbage) 13, 15, 49
- EDSAC 92–3, 105, 168–9
- educational software 249, 266
- EDVA
- C 83–5, 89, 90, 149
- EDVAC Report* (von Neumann) 85–6, 87
- election forecasting 105, 121–3
- electrical power networks 55
- Electrodata 110, 127, 128
- electronic brains 88, 116, 124, 130; *see also* ENIAC
- electronic commerce 294–6
- Electronic Control Company 88, 109, 110
- Electronic Delay Storage Automatic Calculator *see* EDSAC
- Electronic Discrete Variable Automatic Computer *see* EDVAC
- Electronic Frontier Foundation 343
- electronic mail 274, 287, 288, 289
- Electronic Numerical Integrator and Computer *see* ENIAC
- electronic publishing 289
- electronics: industry 229; manufacturers 109; technology 114–18, 132
- Electronics News* 238
- electronic tubes 80, 83, 103, 106; *see also* vacuum tubes
- electrostatic memory 150, 153, 154
- embodying software 341
- Encarta* 271, 332
- Encyclopedia Britannica* 271, 279, 332
- Encyclopedias 270–2, 279–83
- Encyclopédie* 279
- Engelbart, Douglas 203, 261, 263, 282, 290
- Engineering Research Associates (ERA) 109, 110, 126
- engineers, viewpoint of 85, 86–8
- ENIAC 3, 79–86, 103, 104, 110, 149
- Enquire hypertext system 290–1
- entrepreneurial start-up companies: acquisition by corporations 126, 127–8; mainframe computer manufacturing 109–10; personal computer development 237, 248, 256, 259; semiconductor manufacturing 226, 227
- Entscheidungsproblem* 66
- environment issues and computing 337–8
- Epson 252
- Equitable Society of London 14
- E. Remington and Sons 29
- Estridge, Don 252
- European Convention of Human Rights (1950) 346
- Evans, David 261, 262
- Everett, Robert R. 148–50, 153
- Excite search engine 324
- Facebook 326–8
- Fairchild Camera and Instrument Company 226
- Fairchild Semiconductor 226, 227, 237, 339
- Felt, Dorr E. 33
- Felt & Tarrant Manufacturing Company 34, 35

- female clerical labor 17, 24, 41, 60, 64–5
75
- Ferranti Mark I computer 108
- filing systems 27, 31–3
- Filo, David 324
- financial services industry 13, 33
- firing tables 74–5, 79, 148
- First Draft of a Report on the EDVAC, A*
(von Neumann) 85
- flight simulators 147–58
- Flint, Charles R. 41–2
- floppy disks 252, 270
- flowcharts 171–6, 187
- food industry 164–6
- Forbes* business magazine 307–8
- Formula Translator *see* FORTRAN
- Forrester, Jay W. 125, 148–54, 192, 306
- FORTRAN 172–6, 178, 185, 211–12, 220
- Fortune 500 244, 305, 313
- Fortune* magazine 124, 138, 140, 216
- Fourier, Joseph 10
- Foxconn 307–9
- Frankston, Bob 249
- Free Software Foundation 343
- Free Software Movement 331
- French Criminal Code 347
- Friendster site 326
- “front end” operations 164
- Fuller, Buckminster 240
- fully-integrated circuits 142
- Funk & Wagnall’s New Encyclopedia* 271
- Future of the Computer Utility, The*
(Barnett et al.) 216
- games: computer 212, 248, 274; video
200, 222, 228–9, 247–8, 271
- Gates, Bill 203; CD-ROM media and
270–1; as co-founder of Microsoft
259, 303; Internet Explorer and
294; MS-DOS and 251–2; as
software entrepreneur 242–3; at
Windows 95 launch 256; Windows
operating systems and 267, 305
- GEM operating system 267
- gender: discrimination 303, 338–339;
see also women in information
technology
- General Data Protection Regulation
(GDPR) 349
- General Electric: differential analyzer
built by 56; GEISCO 322;
- mainframe computers
manufactured by 109, 127,
128, 135; power networks and
55; radio-set industry and 236;
software crisis and 211; time-
sharing systems of 143, 211,
215–16, 217
- Gerstner, Louis V., Jr. 304
- Giant Brains* (Berkeley) 112
- gig economy 329–31
- globalization: Africa 314–16; Apple
301–5; Brazil 312–14; Chinese
firms, rise of 311–12; chip
manufacturing, outsourcing
of 309–11; computing
312–14; disempowerment and
opportunities 316–17; Foxconn
307–9; Huawei 311–12;
IBM 301–5; Indian software
and services industry 305–7;
Lenovo 311–12; manufacturing,
outsourcing of 307–9; Microsoft
301–5; Samsung 307–9;
transformation through mobile
technology 314–16; TSMC
309–11
- Goldstine, Herman H.: abstract-logical
views of 85, 86, 87, 89; EDVAC
programming and 169; ENIAC
and 79–86, 103; Institute for
Advanced Study 170, 194;
scientific war work of 75
- Google 321, 323–5
- Gopher search system 289–90
- governance, computing *see* computing
- graphical user interface (GUI) 256,
260–3, 266, 275
- Graphics Environment Manager (GEM)
267
- Gray, Stephen 224
- Great Depression 42, 45
- Greenberger, Martin 323
- Grolier Encyclopedia* 271
- Grosch, Herb 126, 216
- Grosch’s Law 216
- Groves, Leslie R. 119–20
- Guo, Terry 307–8
- Haber, Stuart 334
- Harmsworth Self-Educator* magazine 97
- Harvard Mark I 61–4, 82, 88, 91, 101, 116

- Harvard Mark II 91
 Harvard University 91, 115
 Hastings, Reed 333
 Heinz Company 164
 Hewlett-Packard (HP) 225, 244, 332
 Hewlett, William 245
 high-speed broadband 276
 Hilbert, David 66
 hobbyists 236, 238–46, 248
 Hock, Dee 162, 163
 Hoerni, Jean 226
 Hoff, Ted 237
 Hollerith, Herman 19–23, 33, 40
 Hollerith Electric Tabulating System 21, 33, 39–40, 44
 Homebrew Computer Club 243, 245
 home-built computers 224
 home computing 229
 Honeywell: competitive strategies of 143;
 FACT programming language
 175; IBM-compatible computers
 139–40, 142; mainframe
 computers 109, 127, 128, 135;
 minicomputers 225
 Hopper, Grace Murray 65, 121, 172, 174,
 175, 193
 Huawei 311–12
 human-computer interaction 214, 248,
 261
 human computers 10–17, 57–8, 73
 human-factors design 213
 Human Factors Research Center 203, 261
 human knowledge 280–1
 hypertext 240, 282, 290–1
 IBM 239; 1401 mainframe computer
 106, 128, 131–3, 137, 139–40;
 accounting machines 44, 132,
 136, 141; Airline Control
 Program 163; airline reservation
 systems 155–60; antitrust suits
 against 126–7, 187; Automatic
 Computing Plant 63; barcode
 system and 165; Bitnet 295;
 business computers 134, 141;
 business practices 27, 36, 45–6,
 108; Commercial Translator
 175; compatible product lines
 and 135; computer development
 and 88; computer industry and
 27, 109, 128, 225; core memory
 technology 125, 154; data
 processing computers 123–7;
 Defense Calculator 118, 123–4;
 as defense contractor 118, 148;
 Eckert and Mauchly's attempts to
 sell EMCC to 118–19; electronic
 technology 114–18; Endicott
 Laboratory 63, 337; FORTRAN
 developed by 172–6, 178, 185,
 211–12, 215, 220; globalization
 301–5; Harvard Mark I and
 61–4, 81, 88, 91, 101, 115;
 Hollerith and 20; IBM PC as
 industry standard 250–4; as
 leader in computer technology
 117; Lenovo 312; Magnetic
 Drum Calculator 117, 123, 125,
 131; mainframe computers 137,
 141–2; MS-DOS development
 agreement and 203, 259; naming
 of company 42; New Product
 Line 137–9, 140; OS/2 operating
 system 268; PC Jr. computer
 266; personal computers 250–4;
 PL/I programming language
 183; Program Development
 Laboratories 180; punched-card
 systems of 27, 44, 60, 100; rent-
 and-refill nature of business 42,
 43; sales strategies of 27, 42–3,
 45–6, 251; sales training 45–6;
 Selective Sequence Electronic
 Calculator (SSEC) 102, 115–17,
 124; survival of Great Depression
 by 42; System/360 computers
 107, 135–41, 215, 216; Systems
 Network Architecture 288; Tape
 Processing Machine 117–18,
 123–4; technical innovation 27,
 37, 42, 44; T-H-I-N-K motto 39,
 43, 117; time-sharing computers
 216–17; TopView operating
 system 267; total-system design
 approach 131–3; TSS/360
 operating system 218; unbundling
 187–8; unfair competition 344;
 war-related computing projects of
 82, 87
 IBM-compatible personal computers:
 developed by competitors 137,
 142; market dominance of 257,

- 263, 266; software development for 257, 259, 266
- icons, desktop 261, 263
- Illustrated London News* 18, 96
- Indian software industry 305–7
- Informatics company 178, 188
- information processing: calculating engines for 12–14; clearing houses for 14–19; human computers for 10–17; mechanical systems for 20; telegraphy and 17–18
- Information Processing Techniques Office (IPTO) 214, 261–3, 283–4
- information storage, office 27, 33
- information technology 25, 31; industry, computing 338–40; programming language 174
- Infosys 305, 306, 307
- Initial Public Offering (IPO) 325
- Institute for Advanced Study (IAS), Princeton 81, 87, 104, 118
- Institute of Electrical and Electronics Engineers (IEEE) 306
- instruction tables (Turing Machine) 66
- insurance industry 9, 98, 112, 176, 188
- integrated circuits 139, 141, 221–2, 224, 227–8
- Intel: 8088 chip 251, 252, 254; Fairchild origins of 221, 227; integrated circuit technology and 228; microprocessors and 237–8, 268
- intellectual property (IP) 340–3
- interactive computing 214, 217, 248
- interactive web: Airbnb 329–31; Amazon 328–9; blockchain technology 334; *Britannica* 331–2; changing face of retail 328–9; disrupting media delivery 332–3; e-commerce 328–9; Facebook 326–8; gig economy in action 329–31; Google 323–5; mobile computing 319–21; smartphones 319–21; social media 326–8; time-sharing for twenty-first century 321–3; Twitter 326–8; Uber 329–31; Wikipedia 331–2; wisdom of crowd 331–2
- interface message processor (IMP) 286
- International Business Machines *see* IBM
- International Computers Limited (ICL) 21
- International Conference on Computer Communications (ICCC) 286
- International Development Research Centre 314, 316
- Internet: Arpanet project 283–7, 295; economic and social role of 279; electronic mail 274, 287–9, 288; Explorer 294; memex as predecessor to 281–3, 296; Minitel and 272–3; mobile computing 316–21; privatization of 294–6; service provider (ISP) 293; social networking 325; *see also* World Wide Web
- Introduction to VLSI Systems* (Mead) 310
- iOS platform 320, 321
- Iowa State University 76–8
- iPads 302
- iPhone 207, 301–5, 321
- iPod devices 302
- Isaacson, Walter 303
- iTunes stores 321
- Jacquard loom 49, 51, 63
- Jenner, Edward 290
- Jobs, Steve: Apple Computer co-founded by 244–6, 254, 302–3, 308; computer liberation movement and 240; Macintosh development and 264–6, 321; *see also* Apple Computer
- Kalanick, Travis 330
- Kay, Alan 262, 263, 319
- Kemeny, John 195, 211
- keyboards 28
- Kilburn, Tom 104
- Kildall, Gary 251, 267, 270–1
- Krishna, Arvind 304
- Kun-hee, Lee 309
- Kurtz, Thomas E. 195, 211–12
- labor issues in the IT industry 338–40
- Lake, Claire D. 63
- Laplace, Pierre-Simon 10
- laptop computers 319, 320
- Lardner, Dionysius 52
- LCD screens 320
- League Against Racism and Anti-Semitism (LICRA) 347
- Learson, T. Vincent 137–8, 181

- Lee, Spike 325
 Legendre, Adrien-Marie 11
 Leibniz, Gottfried 1
 Lenovo 311–12
 Levy, Steven 337
 Licklider, J. C. R. 192, 213–16, 263, 282–4
 life insurance industry 9, 176
 light-emitting diode (LED) 221
 links: hypertext 291; web 325
 Linux operating system 221
 Lisa computer 264–5
 Little, Arthur D. 306
 Liu, Chuanzhi 311
 Lizhi Xu 316
 logarithmic tables 9–11, 56, 59
 logical design 85
 logic chips 237
 logicians, viewpoint of 85, 86–8
 London Comptometer School 98
 Lotus 1–2–3 spreadsheet 258, 259, 260
 Lovelace, Ada 51, 64, 95
 Lowe, William C. 250–1, 252
 LSI (Large Scale Integration) chips 228
 Lycos search engine 324

 Macintosh computers 202, 260–6, 267, 275
 MAD programming language 174
 Magnetic Drum Calculator (MDC) 117, 123, 125, 131–2
 magnetic drum memory systems 117
 magnetic-tape storage systems 111, 117, 118, 125, 132
 mainframe computers: compatible-family
 concept 136–7; competition
 among manufacturers 109,
 136–8, 141–2; costs of entering
 mainframe business 223; IBM
 1401 131–3; IBM's industry
 domination 106, 141–2;
 supporting software for 179; time-
 sharing systems 222, 230
 Manchester Baby Machine 90, 104
 Manchester University 56, 90, 104, 108, 125
 “Man-Computer Symbiosis” (Licklider)
 213–4, 263, 283
 Manhattan Project 73, 81–2, 119, 138
*Manual of Operation of the Automatic
 Sequence Controlled Calculator*
 (Aiken) 65

 Marconi, Guglielmo 235
 Mark I digital computing machine *see*
 Harvard Mark I
 Mark II digital computing machine
 (Harvard University) 91
 Mark IV file management system 188
 Markkula, Mike 246
 Maskelyne, Nevil 9
 Massachusetts Institute of Technology
 see MIT
 mass-storage devices 155
 mathematical analysis 57–8
 mathematical computations 48–9, 73
 mathematical instruments 108, 109
 mathematical logic 66, 83, 85
 mathematical tables 9–14, 48–9, 64; *see*
 also specific tables
 Mauchly, John W.: as computer industry
 pioneer 109–10; data-processing
 computers and 110–14; EDVAC
 and 87–9; electronic computer
 proposal of 76–8; ENIAC and
 79–81, 84–5, 103; as Moore
 School instructor 75, 89; UNIVAC
 and 105, 118–22
 Mauchly, Mary 75, 76
 McCarthy, John 210
 McCracken, Daniel D. 174
 McKinley, William 40
 McKinsey & Company 162, 164, 337
 McLuhan, Marshall 240
 Mead, Carver 310
 mechanical models 55, 56
 memex 204, 281–3, 296
 memory technology: delay-line storage
 systems 78–9, 83, 92, 93,
 111, 150; for EDVAC 92;
 programming technology and
 171; subroutines and 170; *see also*
 storage technology
 Menabrea, Luigi 51
 meteorology, numerical 57–8
 method of differences 11–12
 microblogs 327
 microchips 200, 340
 microcomputers 229, 245
 microelectronics 198, 221–2
 microfilm 281
 Micro Instrumentation Telemetry Systems
 (MITS) 241, 242, 243
 MicroPro 250, 259, 266

- microprocessors 199, 237–8, 251, 320
- Microsoft 203; Apple's lawsuit against 268; Gates as co-founder of 259–60; globalization 301–5; Internet Explorer 293; MS-DOS operating system 252, 259, 267, 268; MSN online service 293; OS/2 operating system 268; software development for PCs 242–3; Windows operating systems 256–7, 266, 294, 303; Xbox 360 251–2, 266, 306
- military information systems 152–3, 226
- minicomputers 222–5, 230, 238–44
- Minitel videotex system 200, 272–4
- MIT 321–2; computer utility concept 230; Conference on Advanced Computational Techniques 149; core memory technology 91, 154; differential analyzer 61; Lincoln Laboratory 154, 220, 284; power network analyzer 55, 56; Project MAC 216, 287; Project Whirlwind 147–52, 222; Radiation Laboratory 73, 78, 90; Servomechanisms Laboratory 148; software crisis 218; time-sharing computer systems 217–19; wartime research projects 75
- MITS (Micro Instrumentation Telemetry Systems) 241, 242, 243
- mobile computing 301, 312–14, 319–21
- “Mobile Lovers” (Banksy) 207
- mobile money 315
- mobile technology, transformation through 314–16
- Mockapetris, Paul 295
- Monroe Calculating Machine Company 61, 62, 109, 128
- Moon tables 60
- Moore, Gordon 227, 237
- Moore School of Electrical Engineering 74–6, 88–9, 110; *see also* EDVAC; ENIAC
- Moore's Law 228
- Morse, Samuel 341
- Mosaic browser 292, 293
- mouse, computer 262, 263
- Moving Target Indicator (MTI) devices 78
- MS-DOS operating system 252, 259, 267, 268
- MSN online service 293
- Multiple Information and Computing Service (Multics) 215–16, 218, 219
- multiprogramming systems 180
- Murthy, N. R. Narayana 306
- MySpace 326
- Nadella, Satya 304
- National Association of Food Chains 164
- National BankAmericard Incorporated (NBI) 162
- National Cash Register Company (NCR) *see* NCR
- National Defense Research Committee (NDRC) 74, 80, 82, 149
- National Science Foundation (NSF) 295, 324
- National Semiconductor 227
- Nature* 48, 65, 332
- Nautical Almanac* 9–10, 12, 54, 59
- navigation tables 10, 54, 59
- NBI/Visa 162
- NCR: accounting machines 37, 44, 59, 128; acquisition of CRC 128; barcode system and 165; business software 132–3; business strategies of 36, 40, 134, 143; cash registers 37; computer industry and 109; digital computing machines 61; non-IBM-compatible computers of 143; postwar business challenges 114; punched-card systems 27, 44; role in computer industry 27, 128; sales strategies of 36, 37, 42; as software contractors 177; technical innovation by 37; user training 36
- Negroponte, Nicholas 313, 314
- Nelson, Ted 240, 243, 283, 290
- Netflix 332–3
- netiquette 288
- Netscape Communications Corporation 292
- network(s): air-defense 152, 213; CBS television 121; computer 279; consumer 274–6; electrical power 55; protocols 288; telecommunication 138, 285; wide-area 155

- New Communalists 240
 New Deal projects 45
 Newman, Max 90
 New Product Line (IBM) 137–40
 NeXT 320
 Nokia 303, 321
 Non-Fungible Tokens (NFTs) 334
 non-IBM-compatible computers 143
 Norris, William 109, 135
 Northrop Aircraft Corporation 111, 113, 115, 127
 notebook computers 307, 320
 Noyce, Robert 225–7, 237
 Noyes, Eliot 131
 nuclear weapons 73, 81, 138, 155
 numerical meteorology 57–8

 office machine industry *see* business machine industry
 Office of Naval Research (ONR) 151–3, 158
 Office of Scientific Research and Development (OSRD) 73, 82
 office systematizers 25
 offshoring 301, 304
 Olsen, Kenneth 222–3
 “On Computable Numbers with an Application to the *Entscheidungsproblem*” (Turing) 66
 One Laptop Per Child (OLPC) 313
 Opel, John 252
 open-source software 221, 291, 316, 321
 operating systems: for mainframe computers 179–84, 211, 218, 219–21; for mobile devices 322; for personal computers 248, 252, 256–7, 259–60, 266; *see also specific aspects/types*
 Oracle Corporation 322
 OS/2 operating system 268, 269
 OS/360 operating system 179–82, 184, 218
 Osborne 1 computer 202, 319
 outsourcing of components and software 251, 252, 301
Oxford English Dictionary 9

 packaged software programs 187–8, 257
 packet-switching technology 284–5
 Page, Larry 324

 Palo Alto Research Center (PARC) 262, 264, 284, 319
 Papert, Seymour 313
 Papian, Bill 152, 154
 Pascal, Blaise 1
 Pascal programming language 185
Passages from the Life of a Philosopher (Babbage) 62
 Patni, Narendra 306–7
 Patriot Act 348, 350
 Patterson, John H. 36–8, 39, 42, 44
PC World magazine 260
 Pender, Harold 87, 91
 peripherals 132, 136, 140, 223
 Perot, Ross 178
 personal computers: Altair 8800 241–4; applications software for 248–50; BASIC as programming language for 213; CD-ROM disks and drives 270–2, 276; consumer networks 274–6; development of 216, 235, 237; GUI concept and 260–3, 266–70, 275; hobbyists and 238–44; IBM clones 253–4; IBM PC as industry standard 250–4; integrated circuit technology and 221; laptops 319, 320; Macintosh computers 260–6, 267, 268, 275; microprocessor as enabling technology 237–8; Osborne 1 computer 202, 319; software for 246–50, 256–60; use of term 224; video games 222, 228–9, 271; videotex systems 272–3; Windows operating systems 203, 256, 266–70, 294; *see also* Apple Computer; Internet
 personal digital assistants (PDAs) 320
 Pew Research 339
 Philco 109, 127, 225
 Philips 270
 Pichai, Sundar 304
 Piggly Wiggly 163
 PL/1 programming language 183
 planar process 221, 226–8
 plastic credit 161
 point-and-click interface 275–6, 291
 Pong video game 229
Popular Electronics 238, 241
Popular Science Monthly 65
 Porter, Robert P. 21, 40

- power networks 55
- Powers, James 41, 44
- Preparation of Programs for an Electronic Digital Computer, The* (Wilkes, Wheeler, and Gill) 170
- Prestel videotex system 272, 273
- printed circuitry 155
- printers and printing 34, 44, 132, 133, 252
- Prism program 348
- privacy, social networking and 347–50
- Prodigy 275, 293
- product integraph 56
- profile tracer 56
- programmed data processor (PDP) 219–20, 223–4
- programmers 51, 126, 182–6, 242
- programming: automatic systems 171; Babbage's Analytical Engine and 49; BASIC as language for 211–13; bugs and debugging 168–73, 179, 182, 183, 210; for EDSAC 92; for ENIAC 83; IBM application software for 132; for minicomputers 241; for stored-program computers 84, 92; Turing Machine and 66
- programming languages 84, 171–6; *see also specific languages*
- programs 170–1, 172, 176–9, 187, 219; *see also applications, computer; software*
- Project Chess 253
- Project MAC 216, 287
- Project PX *see* ENIAC
- Project PY *see* EDVAC
- Project SAGE 154, 192
- Project Whirlwind 147–52, 222
- Prony, Gaspard de 10–12, 14, 16, 50
- protocols, network 288
- Prudential Assurance Company 2, 24, 112, 113, 120
- punched cards 41–3, 45
- punched-card systems: accounting machines as 131–2; airline reservation systems and 158; census data processing and 20–1, 40; commercial market for 41–2, 100; IBM as market leader in 27, 42–6; Jacquard looms and 49, 51, 63; manufacturers of 33, 128; for scientific calculations 59; scientific war effort and 73, 87; stored-program computers and 84; tabulating 2
- Putin, Vladimir 346
- QUICKTRAN 217
- QWERTY keyboard layout 28, 29, 97
- Race Relations Act (1965) 346
- radar 73, 78, 92, 154
- Rader, Louis T. 134
- radio broadcasting 235–7
- Radio Shack 247, 274
- Railway Clearing House (England) 16
- Rand, James, Jr. 31, 32–3
- Rand, James, Sr. 31, 32–3, 119–20
- RAND Corporation 177
- Rand Kardex Company 31–2
- Rand Ledger Company 32
- Randolph, Marc 333
- Raskin, Jef 264
- Raytheon Company 109, 127
- RCA 127, 128, 135, 142–3
- Reagan Administration 345
- real-time computing: airline reservations systems and 155–60; financial services industry and 179; Project SAGE 154; Project Whirlwind 147–52, 222; role of user in 217
- record-keeping systems 24, 25, 27, 31–3
- Regis McKenna 246
- Remington, Philo 29
- Remington Rand: acquisition of computer firms 109, 118–20, 127; business development of 44, 45; computer industry and 27, 109; digital computing machines 61; marketing strategies of 127; merger with Sperry Gyroscope 126; postwar business challenges to 114; UNIVAC and 120–6
- Remington Typewriter Company 28, 30–3, 40
- rent-and-refill businesses 42, 43, 45
- Report Program Generator (RPG) 132
- Reservisor 157, 158, 159
- retail industry 176, 177
- Richardson, Lewis Fry 57–9
- Ritchie, Dennis M. 219
- Roberts, Ed 241, 242, 243
- Roberts, Larry 284, 286, 287

- Rock, Arthur 226, 227
 Roskomnadzor 346
 "Route 128" computer industry 148, 223, 239
 routines, use of term 170–1
 Royal typewriter company 30, 109
 Rubinstein, Seymour 250
- SABRE airline reservations system 155–60, 177, 193
 SAGE defense system 152–5, 177, 223
 sales and marketing strategies: IBM strategies 38–9, 45–6, 126; NCR strategies 38; office machine industry and 31, 35–6; for personal computer software 259; product launches 247, 266
 Salesforce.com 322
 Samsung 307–9
 Sarnoff, David 135, 236
 scale models 55, 56
 Scheutz Difference Engine 52
 SchoolNet project 314, 315
Scientific American magazine 1, 28
 Scientific Computing Service Limited 59–60
 scientific office management 13, 25
 scientific programming languages 178; *see also* FORTRAN
 scientific research, civilian 149, 151, 155
 scientific war effort: Atanasoff-Berry Computer and 76–8; Bush's role in 281; Manhattan Project 73, 81, 119, 138; research and technical development programs 73; *see also* EDVAC; ENIAC; Moore School of Electrical Engineering
 Scott, Mike 246
 Sculley, John 265, 266
 search engines and systems 290
 Seeber, Robert, Jr. 117
 Semi-Automatic Business Research Environment (SABRE) 155–60, 177
 Semi-Automatic Ground Environment defense system (SAGE) *see* Project SAGE
 semiconductors 199
 semiconductor technology 139, 221–2, 225–8, 237–8
 servers, web 291, 292–4
 services industry, rise of 305–7
 servomotors 111
 settlement, check and credit card 16
 Shockley Semiconductor Laboratories 226, 227
 Shockley, William 225, 226
 Sholes, Christopher Latham 28, 34
 silicon chips 227, 228–30, 320; *see also* integrated circuits
 Silicon Graphics 292
 Silicon Valley 225–8, 239, 243
 Singer Sewing Machine Company 30, 37
Sketch of the Analytical Engine (Lovelace) 51, 95
 smartphones 319–21
 Smith, Adam 11, 12, 13
 Smith, Blair 158
 Smith, C. R. 158
 Snowden, Edward 348–9
 social media 326–8, 345
 social networking 325
 software: bugs and debugging 168–73, 179, 182, 183, 210; for computer games 248; contracting industry 176–8, 186–9; contractors 176–8, 186–9; crisis 179–82, 218, 284; difficulties of creating 171; educational programs 249; embodying 341; engineering 155, 185; flowcharts and 171–6; Indian 305–7; as industrial discipline 182–6; for minicomputers 222–5, 242–3; for networking 286; packaged programs and software products 186–9; for personal computers 246–50, 256–60, 268; start-up companies for 248; superpower 305; use of term 169; user groups for 176; for utilities 220; for web browsers and servers 291, 292–4; *see also* programming; programming languages
 Software as a Service 322
 software-compatible computers 141
 Software Technology Parks 306
 Solid Logic Technology (SLT) 141
 Sony 270, 306
 Source, The 275
 space invaders 200
 Sperry Gyroscope 126

- Sperry Rand 126–8, 132–4, 143, 225
 spreadsheet programs 249
 Stanford Digital Library Project 324
 Stanford Research Institute (SRI) 261, 286
 Stanford University 226, 324
 storage technology: EDSAC and 92;
 electrostatic memory 150, 152–4;
 magnetic-tape storage systems
 111, 117, 125, 132; stored-
 program computers and 83; *see*
 also memory technology
 store-and-forward packet switching 284–5
 stored-program computers: BINAC
 112–13, 117; concept of 83, 85;
 EDSAC 92–3, 105, 168–71;
 engineering vs. abstract-logical
 views on 86–8; Magnetic
 Drum Calculator 117, 123, 125;
 Manchester Baby Machine 90,
 104; Moore School Lectures
 88–9, 91, 110; as postwar
 business challenge 114–15;
 program development for 168–71;
 structure of 84
 Stornetta, W. Scott 334
 Strauss, Henry 113
 subroutines 170–1
 Sutherland, Ivan 261, 262, 283
 System Development Corporation (SDC)
 177
 systems implementation languages 220

 table making 10–14, 49, 56, 61, 66; *see*
 also specific types
 tablet computers 209
 Tabulating Machine Company (TMC) 20,
 21, 40, 42, 100
 tabulating machines 20, 22–3, 41–2,
 44, 100; *see also* punched-card
 systems
 Taiwan Semiconductor Manufacturing
 Company (TSMC) 206, 301,
 309–11
 Tandy Corporation 247–8, 274
 Tandy TRS-80 113, 320
 Tape Processing Machine (TPM) 117–18,
 123–4
 Tarrant, Robert 34
 Tata Consultancy Services (TCS) 305,
 306–7
 Tata Group 306
 taxicabs 330
 Taylor, Frederick W. 25
 Taylor, Robert 263, 284, 286
 technology intercept 158
 telecommunication networks 138, 285
 telegraphs 2, 17–18, 30, 96, 236, 285
 Telnet network 287
 Terman, Frederick 226
 terminals 80, 218
 Tesler, Larry 264
 Thefacebook 326
 Thompson, Ken 219–20
 tide-predicting machines 54
 time-sharing computer systems 195, 196;
 Arpanet and 283–7; BASIC and
 211–13, 230; computer utility
 concept and 216–19; concept of
 219; GE as industry leader 143;
 human-computer interaction and
 213; operating systems for 141,
 221; for twenty-first century
 321–3
 TopView operating system 267
 Torvalds, Linus 221
 Totalisator machines 113
 total-systems marketing 131–3, 187
 TOTVS 313
 touchscreen technology 321
 training and training organizations: for
 business machine operators
 34, 35–6, 125–6; for business
 machine sales 35–6, 37, 39, 45;
 for computer programmers 125,
 126, 183; for typists 30, 34
 transistors 132, 222, 225
 Transmission Control/Internet Protocol
 (TCP/IP) 289
 Travis, Irven 87
 trigonometric tables 9
 TRW 177
 Turing, Alan Mathison 66–8, 89
 Turing Machine 66
 Twitter 326–8
 typewriters 24, 25, 27–31, 34, 97, 98, 159
 typists and typing 30, 31, 34

 Uber Technologies, Inc. 329–31
 UberX 330
 unbundling, IBM's 187–8, 344
 “under the limit” charges (credit cards) 162

- Underwood 30, 109, 128
- unfair competition 343–5
- Union of French Jewish Students (UEJF) 347
- UNIVAC: automatic programming systems for 171–3; business applications for 109; data-processing languages for 175; development of 118–22; election forecasting and 105, 122; limitations of 125–6; magnetic tape storage system for 111–14
- UNIVAC 1004 134
- UNIVAC 1103 126
- UNIVAC Division, Sperry Rand 134–5, 225
- UNIVersal Automatic Computer *see* UNIVAC
- University Computing Company (UCC) 178, 217
- University of California 220, 286
- University of Minnesota 289, 292
- University of Pennsylvania 56; *see also* Moore School of Electrical Engineering
- University of Utah 214, 261, 262, 286
- Unix operating system 219–21
- UPC (universal product code) 163–6
- USA PATRIOT Act 348
- U.S. Bureau of Aeronautics 147, 150–1
- Usenet system 295
- user-generated web content 332
- user groups 176
- vacuum tubes 79, 80, 199, 225
- Valley, George E. 153
- venture capital 222–3, 225, 226–7, 246, 257, 259, 324
- vertical filing systems 31–2
- Very Large-scale Integration (VLSI) chips 310
- video games 200, 222, 228–9, 247–8, 271
- videotex systems 272–3, 276
- Visa system 162–3
- VisiCalc spreadsheet 249–50, 252, 257, 259
- VisiCorp 259
- Von Neumann, John 3; EDVAC and 81–8, 171; *EDVAC Report* 85–6, 90, 115; flowcharts and 171; influenced by Turing 67–8; at Institute for Advanced Study 81, 87, 104, 118, 169, 194; on nature of computer programmers 182–3
- W3C (World Wide Web Consortium) 292
- WAIS (Wide Area Information Service) 289
- Wales Adding Machine Company 35
- Wales, Jimmy 332
- Walt Disney (company) 320
- wartime scientific programs *see* scientific war effort
- watches, digital 228, 229
- Watson, Thomas J., Jr.: compatible-family concept and 137; on data-processing computers 126; on effect of Cold War on IBM 155; as IBM president 106; New Product Line and 140–1; OS/360 development and 179–82, 184
- Watson, Thomas J., Sr.: business practices of 108, 116, 123–6; commitment to defense work 118; computer development at IBM 88; Harvard Mark I and 64, 65; as IBM president 106; interest in digital computing 61–3; interview with Eckert and Mauchly 118–19; at NCR 38; Patterson as role model for 36; potential of electronics seen by 115; role in success of IBM 42–5; Tabulating Machine Company and 100
- Wealth of Nations* (Smith) 11, 12, 13
- weather forecasting 56–8, 77
- Web 2.0 332
- web browsers and servers 291, 292–4
- web logs 319
- Wells, H. G. 280, 281, 296
- Westinghouse-Bettis nuclear power facility 55, 173–4, 236
- Wheeler, David 169–71
- Whirlwind 147–52, 222
- Wikipedia 331–2
- Wilkes, Maurice 90–1, 169–71
- Wilkins, Jeff 274
- Williams, Frederic C. 90, 104
- Williams Tube technology 125
- Windows operating systems 203, 256, 266–70, 294
- Winograd, Terry 324

- Wipro 305, 307
- wireless telegraphy 235, 236
- wisdom of crowd 331–2
- Wise, Tom 138, 140
- women in information technology 95,
121, 172, 174–175, 193, 338–339,
350; *see also* female clerical
labor
- Word, Microsoft 260
- Word-Master 250
- word processing software 249–50
- WordStar 250, 257, 260
- World Brain project 281, 296
- World Dynamics* (Forrester) 154
- World Encyclopedia project 280–1
- World Intellectual Property Organization
(WIPO) 343
- world knowledge 280–1
- World War I 35, 74
- World War II 67, 101, 147, 149, 152; *see*
also scientific war effort
- World Wide Web: Consortium 292;
development of 289–92; privacy
and 347–50; social networking
325; user-generated content for
332; web browsers and servers
291, 292–4
- Wozniak, Stephen 244–6
- WYSIWYG 250
- Xbox gaming system 303
- Xerox Corporation 262
- Xerox PARC 263, 264, 284
- XO laptop 313
- Yahoo 323, 324
- Yang, Jerry 312, 324
- Yuanqing Yang 312
- Zenith 236, 252
- Zuckerberg, Mark 326
- Zuse, Konrad 76